



上海朔集半导体科技有限公司

SLM32M030

电机控制系列

基于 Arm® Cortex®-M0+ 微控制器

集成 8 个电压比较器 VC，4 个可编程运算放大器 PGA 和 1 个数模转换器 DAC

适用于电机应用

用户手册

User Manual

Version: 0.9

2025 年 11 月 7 日

<http://www.soulsemi.com>

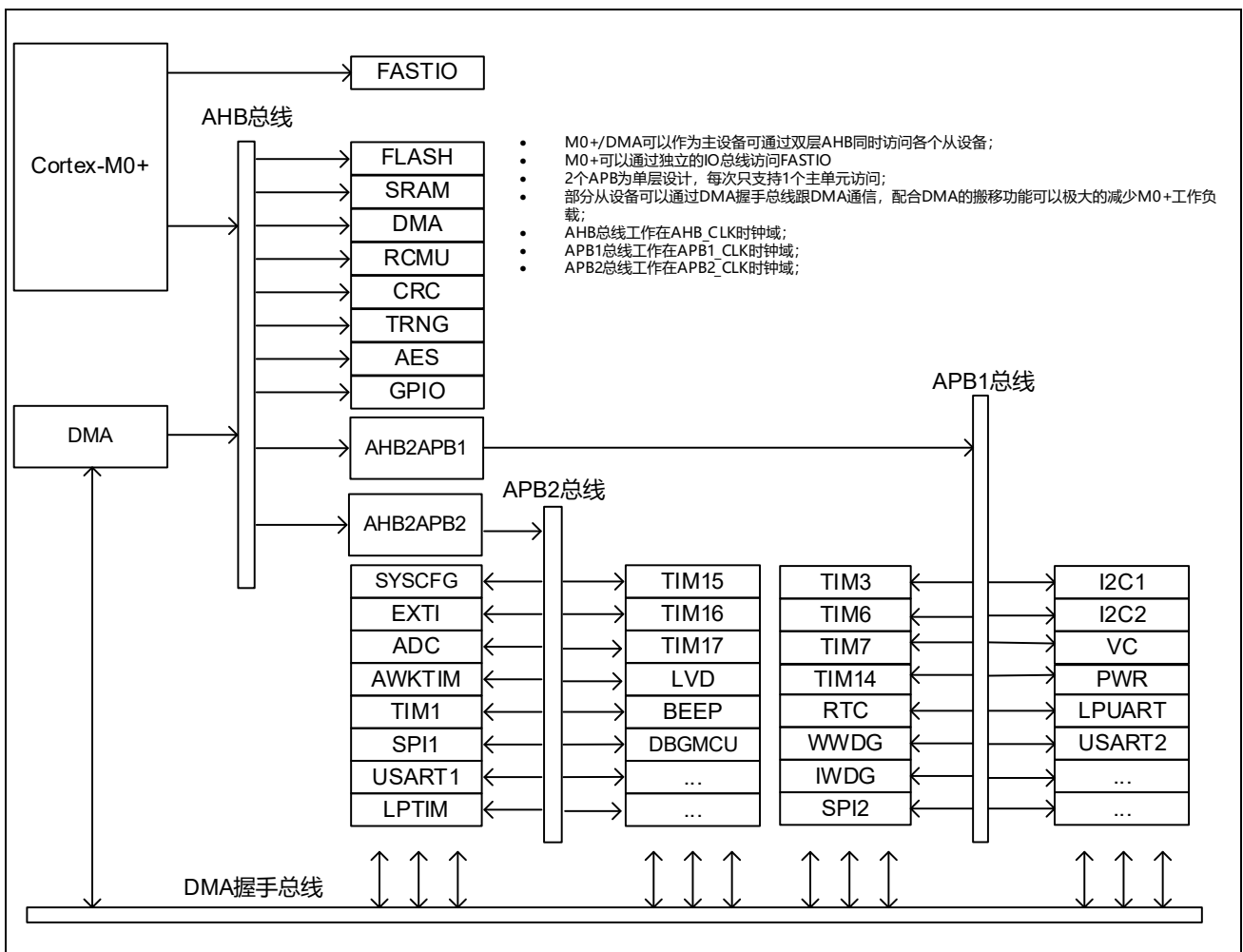
第一部分 内核与架构

1 存储器和总线架构

1.1 系统架构

整个系统主要由以下几个部分组成：2 个 AHB 主设备、8 个 AHB 从设备、14 个 APB1 从设备、14 个 APB2 从设备；

图1.1 系统顶层架构



1.2 存储架构

程序存储器，数据存储器，寄存器和 IO 端口等都统一编址在 4G 字节线性地址空间内。

字节被以小端格式存放在存储器中。一个字（word）的最低地址字节被认为是该字的最低有效字节。

1.3 地址分配

表1.1 地址分配

103 地址范围	地址范围	内容	总线
N/A	0x4801 0C00 - 0x4801 0FFF	FASTIOD	AHB



103 地址范围	地址范围	内容	总线
N/A	0x4801 0800 - 0x4801 0BFF	FASTIOC	AHB
N/A	0x4801 0400 - 0x4801 07FF	FASTIOB	AHB
N/A	0x4801 0000 - 0x4801 03FF	FASTIOA	AHB
0x4001 1400 - 0x4001 17FF	0x4800 0C00 - 0x4800 0FFF	GPIOD	AHB
0x4001 1000 - 0x4001 13FF	0x4800 0800 - 0x4800 0BFF	GPIOC	AHB
0x4001 0C00 - 0x4001 0FFF	0x4800 0400 - 0x4800 07FF	GPIOB	AHB
0x4001 0800 - 0x4001 0BFF	0x4800 0000 - 0x4800 03FF	GPIOA	AHB
0x4002 6000 - 0x4002 63FF	0x4002 3800 - 0x4002 3BFF	AES	AHB
0x4002 5000 - 0x4002 53FF	0x4002 3400 - 0x4002 37FF	TRNG	AHB
0x4002 3000 - 0x4002 33FF	0x4002 3000 - 0x4002 33FF	CRC	AHB
N/A	0x4002 2800 - 0x4002 2BFF	SRAMCTRL	AHB
N/A	0x4002 2000 - 0x4002 23FF	FLASHCTRL	AHB
0x4002 1000 - 0x4002 13FF	0x4002 1000 - 0x4002 13FF	RCMU	AHB
0x4002 0000 - 0x4002 03FF	0x4002 0000 - 0x4002 03FF	DMA	AHB
N/A	0x4001 5800 - 0x4001 5BFF	DBGMCU	APB2
N/A	0x4001 5400 - 0x4001 57FF	BEEP	APB2
N/A	0x4001 5000 - 0x4001 53FF	LVD	APB2
N/A	0x4001 4800 - 0x4001 4BFF	TIM17	APB2
N/A	0x4001 4400 - 0x4001 47FF	TIM16	APB2
N/A	0x4001 4000 - 0x4001 43FF	TIM15	APB2
N/A	0x4001 3C00 - 0x4001 3FFF	LPTIM	APB2
0x4001 3800 - 0x4001 3BFF	0x4001 3800 - 0x4001 3BFF	USART1	APB2
0x4001 3000 - 0x4001 33FF	0x4001 3000 - 0x4001 33FF	SPI1	APB2
0x4001 2C00 - 0x4001 2FFF	0x4001 2C00 - 0x4001 2FFF	TIM1	APB2
N/A	0x4001 2800 - 0x4001 2BFF	AWKTIM	APB2
0x4001 2400 - 0x4001 27FF	0x4001 2400 - 0x4001 27FF	ADC	APB2
0x4001 0400 - 0x4001 07FF	0x4001 0400 - 0x4001 07FF	EXTI	APB2
N/A	0x4001 0000 - 0x4001 03FF	SYSCFG	APB2
0x4000 7000 - 0x4000 73FF	0x4000 7000 - 0x4000 73FF	PWR	APB1
N/A	0x4000 5C00 - 0x4000 63FF	VC	AHB
0x4000 5800 - 0x4000 5BFF	0x4000 5800 - 0x4000 5BFF	I2C2	APB1
0x4000 5400 - 0x4000 57FF	0x4000 5400 - 0x4000 57FF	I2C1	APB1
N/A	0x4000 5000 - 0x4000 53FF	LPUART	APB1
0x4000 4400 - 0x4000 47FF	0x4000 4400 - 0x4000 47FF	USART2	APB1
0x4000 3800 - 0x4000 3BFF	0x4000 3800 - 0x4000 3BFF	SPI2/I2S	APB1
0x4000 3000 - 0x4000 33FF	0x4000 3000 - 0x4000 33FF	IWDG	APB1
0x4000 2C00 - 0x4000 2FFF	0x4000 2C00 - 0x4000 2FFF	WWDG	APB1
0x4000 2800 - 0x4000 2BFF	0x4000 2800 - 0x4000 2BFF	RTC	APB1
N/A	0x4000 2000 - 0x4000 23FF	TIM14	APB1
0x4000 1400 - 0x4000 17FF	0x4000 1400 - 0x4000 17FF	TIM7	APB1
0x4000 1000 - 0x4000 13FF	0x4000 1000 - 0x4000 13FF	TIM6	APB1
0x4000 0400 - 0x4000 07FF	0x4000 0400 - 0x4000 07FF	TIM3	APB1
0x2000 0000 - 0x2000 FFFF	0x2000 0000 - 0x2000 7FFF	SRAM	AHB
0x1FFF F800 - 0x1FFF FFFF	0x1FFF F800 - 0x1FFF FBFF	FLASH info	AHB
0x1FFF E800 - 0x1FFF F7FF	0x1FFF E800 - 0x1FFF F7FF	FLASH system	AHB
0x0800 0000 - 0x0803 FFFF	0x0800 0000 - 0x0803 FFFF	FLASH main	AHB

1.3.1 内嵌 SRAM

产品内嵌 32KB SRAM，可以以字节，半字（16 位）或者全字（32 位）访问。SRAM 的地址起始于 0x20000000。

1.3.2 嵌入式闪存

闪存接口的功能:

- 读取接口
- 选项字节加载器
- 闪存编程/擦除操作

表1.2 闪存地址

块	名称	基地址	大小 (字节)
主存储	Sector 0	0x0800 0000 - 0x080003FF	1K
	Sector 1	0x0800 0400 - 0x080007FF	1K
	Sector 2	0x0800 0800 - 0x08000BFF	1K
	Sector 3	0x0800 0C00 - 0x08000FFF	1K
			
	Sector 63	0x0800 FC00 - 0x0800FFFF	1K
系统存储	NVR Sector 2	0x1FFF E800 - 0x1FFFE8FF	1K
	NVR Sector 3	0x1FFF EC00 - 0x1FFFEFFF	1K
	NVR Sector 4	0x1FFF F000 - 0x1FFFF3FF	1K
	NVR Sector 5	0x1FFF F400 - 0x1FFFF7FF	1K
用户存储	NVR Sector 0	0x1FFF F800 - 0x1FFFFBFF	1k
数据配置	NVR Sector 1	0x1FFF FC00 - 0x1FFFFFFF	1k

1.3.3 启动配置

可以通过 BOOT0 引脚和 BOOT1 位 (闪存用户区) 来配置三种不同的启动方式:

表1.3 启动方式

启动方式选择		启动方式	说明
BOOT1	BOOT0		
x	0	闪存主存储区	运行程序主体
0	1	闪存系统存储区	启动自举程序, 一般用来通过 USART1 来烧录主体
1	1	内嵌 SRAM 区	一般作为调试使用, 可减少烧录次数

复位后, BOOT 引脚上的值在 SYSCLK 的上升沿被锁存。由用户决定 BOOT1 和 BOOT0 引脚设置以选择所需的启动模式, 注意从待机模式退出时, BOOT 引脚也会重新采样。

Cortex M0+的启动地址是固定的 0x00000000, 3 种启动方式的正常地址都不是 0x00000000, 所以硬件会根据启动模式用不同的映射方式将对 0x00000000 的访问转移到对应的区域, 同时 3 种启动代码的正常地址也可以照常使用。

- 从闪存主存储区启动: 0x00000000-0x0003FFFF 映射至 0x08000000-0x8003FFFF
- 从闪存系统区启动: 0x00000000-0x00000FFF 映射至 0x1FFFE800-0x1FFFF7FF
- 从内嵌 SRAM 区启动: 0x00000000-0x00007FFF 映射至 0x00020000-0x00027FFF

2 SRAM_ECC 控制器 (SRAM_ECC)

2.1 SRAM 控制

2.1.1 简介

SRAM 本身自带奇偶校验功能,在数据写入时会自动添加校验结果,读出时则进行校验,出现错误时会返回异常,并根据配置产生中断。

2.2 SRAM 寄存器

2.2.1 SRAM 控制寄存器 (SRAM_CR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															INTEN
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw

Bit	Field	Description	Reset	RW	Remark
31:1	Reserved		0x0	r	Reserved field
0	INTEN	写 1: 当读取 SRAM 数据时发生奇偶校验错误产生中断;	0x0	rw	

2.2.2 SRAM 状态寄存器 (SRAM_SR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	ERRADDR														
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															ERR
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw

Bit	Field	Description	Reset	RW	Remark
31	Reserved		0x0	r	Reserved field
30:16	ERRADDR	发生奇偶校验错误的地址	0x0	r	
15:1	Reserved		0x0	r	Reserved field
0	ERR	读 1:发生过奇偶校验错误; 写 0:清除该位;	0x0	rw	

3 嵌入式闪存 (FLASH)

3.1 嵌入式闪存

3.1.1 闪存介绍

Flash 包含 256KB (64K x 32 位) 主阵列, 外加六个 1KB (256 x 32 位) NVR 扇区。

Flash 控制模块包含的功能:

编程和擦除操作

读写保护

低功耗模式

选项字节加载

3.1.2 闪存结构

FLASH 根据功能分为主闪存区、选项字节区、系统区。包含编程擦除控制器, 用于完成 Flash 的编程和擦除操作。

表3.1 Flash 组织结构

存储模块	名称	地址	大小 (字节)
主闪存区	页 0	0x0800 0000~0x800 03FF	1Kbytes
	页 1	0x0800 0400~0x800 07FF	1Kbytes
	页 2	0x0800 0800~0x800 0BFF	1Kbytes
	页 3	0x0800 0C00~0x800 0FFF	1Kbytes
	页 4	0x0800 1000~0x800 13FF	1Kbytes
	...	...	...
NVR	页 255	0x0803 FC00~0x803 FFFF	1Kbytes
	选项字节区	0x1FFF F800~0x1FFF FBFF	1Kbytes
	系统区	0x1FFF E800~0x1FFF FFFF	1Kbytes
FLASH 控制寄存器	FLASH_ACR	0x4002 2000~0x4002 2003	4Bytes
	FLASH_KEYR	0x4002 2004~0x4002 2007	4Bytes
	FLASH_OPTKEYR	0x4002 2008~0x4002 200B	4Bytes
	FLASH_SR	0x4002 200C~0x4002 200F	4Bytes
	FLASH_CR	0x4002 2010~0x4002 2013	4Bytes
	FLASH_AR	0x4002 2014~0x4002 2017	4Bytes
	Reversed	0x4002 2018~0x4002 201B	4Bytes
	FLASH_OBR	0x4002 201C~0x4002 201F	4Bytes
	FLASH_WRPR	0x4002 2020~0x4002 2023	4Bytes

存储模块	名称	地址	大小（字节）
	FLASH_OP_MODE	0x4002 2024~0x4002 2027	4Bytes

主闪存区用于存储客户代码和数据。

系统区用于在系统区启动模式下启动芯片。该区域只给芯片制造商使用，包含引导加载程序，可用 USART1 串行接口重新编程主闪存区程序。它是由芯片生产时写入，并防止虚假写入和擦除操作。

选项字节区可以使主闪存区避免不必要的访问，包括读写擦除。有两种类型的保护：

- 页写保护
- 读保护

主闪存区和选项字节区可由芯片内部的编程擦除控制器进行编程和擦除。执行编程和擦除时，内部高频振荡器 HSI 必须开启。

3.1.3 读操作

闪存指令和数据访问通过 AHB 总线执行。

读访问可以使用以下配置选项：

- 等待周期：读取操作的等待周期数。

必须根据闪存访问频率使用这些选项。等待周期代表 SYSCLK（系统时钟）周期与 Flash 存储器访问的比率时间：

- 0 等待周期，如果 $0 < \text{SYSCLK} \leq 32\text{MHz}$
- 1 个等待周期，如果 $32\text{MHz} < \text{SYSCLK} \leq 64\text{MHz}$

3.1.4 编程擦除控制器

编程擦除控制器负责整个 Flash 的编程和擦除操作。编程擦除控制器包含 8 个 32 位寄存器。

- FLASH 控制器密钥寄存器 (FLASH_KEYR)
- FLASH 选项字节密钥寄存器 (FLASH_OPTKEYR)
- FLASH 状态寄存器 (FLASH_SR)
- FLASH 控制寄存器 (FLASH_CR)
- FLASH 地址寄存器 (FLASH_AR)
- FLASH 选项字节寄存器 (FLASH_OBR)
- FLASH 写保护寄存器 (FLASH_WRP)
- FLASH 操作模式寄存器 (FLASH_OP_MODE)

3.1.4.1 密钥值

密钥值如下：

- RDPRT 密钥=0xAA（零级保护，没有保护）
- RDPRT 密钥=0xFF（一级保护）
- RDPRT 密钥=0xCC（二级保护）
- KEY1=0x45670123
- KEY2=0xCDEF89AB

3.1.4.2 解锁 Flash

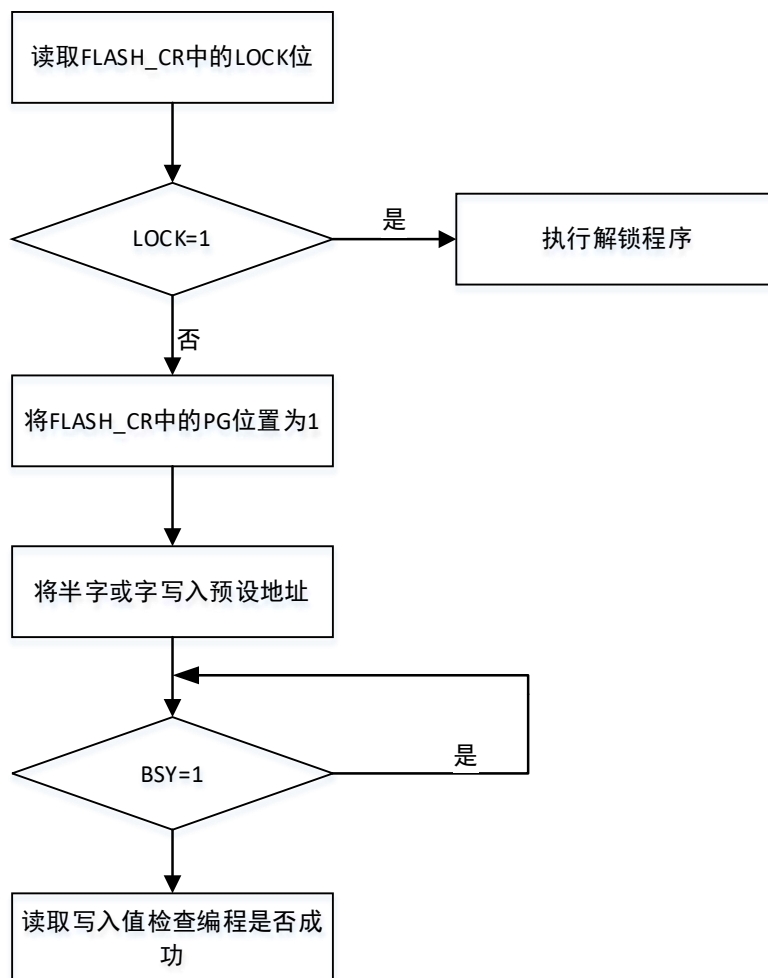
- 复位后，编程擦除控制器受到保护，FLASH_CR 寄存器写操作被禁止。应将解锁序列写入 FLASH_KEYR 寄存器以打开编程擦除控制器。该序列由两个写周期组成，两个密钥值 KEY1 和 KEY2（KEY1=0x45670123 和 KEY2=0xCDEF89AB）写入 FLASH_KEYR 寄存器。任何错误的顺序都会锁定编程擦除控制器和 FLASH_CR 寄存器，直到下一次复

位。错误的密钥序列也会返回总线错误。如果写入的 KEY1 和 KEY2 不正确，那么在写周期完成后的第二 HCLK 返回总线错误。通过将 FLASH_CR 寄存器的 LOCK 位写入 1，用户就可以用软件锁定编程擦除控制器和 FLASH_CR 寄存器。此时，FLASH_KEYR 写入正确的密钥值可以解锁编程擦除控制器。

3.1.4.3 主闪存区编程

- 主存储区一次编程为 16 位或 32 位。当将 FLASH_CR 寄存器的 PG 位置 1 时可以进行编程操作，CPU 可以将一个半字或一个字写入主闪存区。尝试写不是半字或一个字长的数据控制器会返回总线错误响应。如果在编程过程 (BSY 为 1) 中启动了读/写操作 (BSY 位置 1)，CPU 停止直到正在进行的主闪存编程结束。

图3.1 主闪存区编程流程



标准的主闪存区编程过程：

- 在此模式下，CPU 通过执行标准的半字或字操作对主闪存区编程。首先配置 FLASH_CR 寄存器中的 PG 位为 1。编程操作控制器首先读取被编程地址的数据，如果是擦除状态那么继续进行编程操作。如果不是擦除状态那么跳过编程操作，同时 FLASH_SR 寄存器的 PGERR 位发出警告信息。编程数据是 0x0 时，不会发出 PGERR 警告。如果编程地址受到 FLASH_WRPR 寄存器的写保护，那么跳过编程操作，同时 FLASH_SR 寄存器的 WRPRTERR 位发出警告信息。编程操作结束，FLASH_SR 的 EOP 位会变为 1。

主闪存区的标准编程流程如下面所示：

- 通过检查 FLASH_SR 寄存器中的 BSY 位来检查是否有正在进行的闪存操作，BSY 等于 0 进行下一步操作
- 将 FLASH_CR 寄存器中的 PG 位置 1
- 写半字或字到闪存
- 等待 BSY 位归零
- 读取写入值，检查编程是否成功

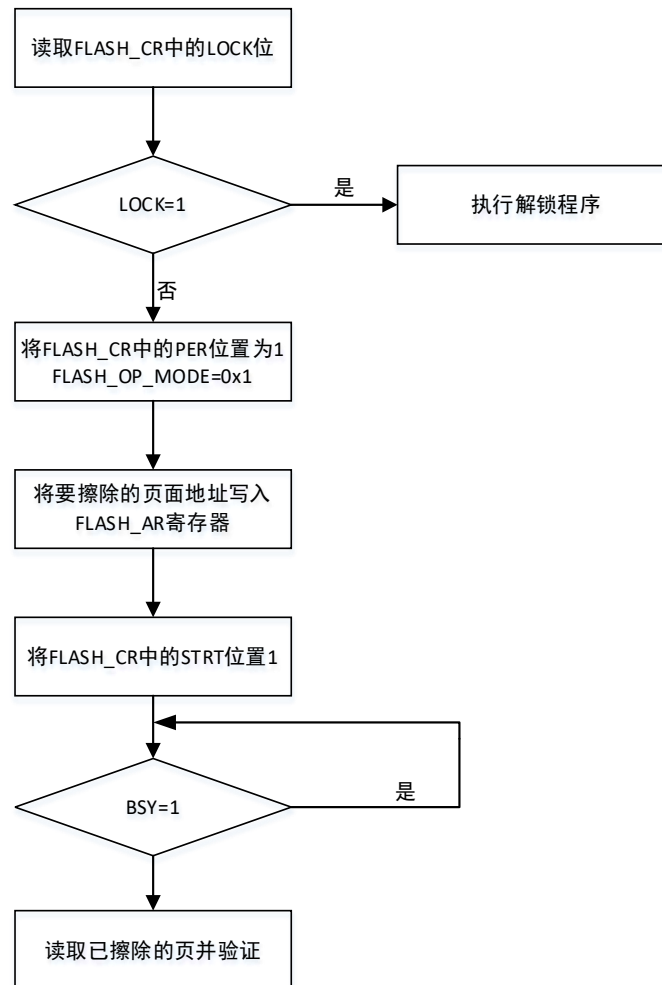
3.1.4.4 主闪存区页擦除

Flash 存储器以页为单位擦除，也可以整片擦除。

页擦除的步骤如下：

- 通过检查 FLASH_SR 寄存器中的 BSY 位来检查是否有正在进行的闪存操作，BSY 等于 0 进行下一步操作
- 配置 FLASH_OP_MODE 等于 0x1
- 将 FLASH_CR 寄存器中的 PER 位置 1
- 编程 FLASH_AR 寄存器选择一个要擦除的页面
- 将 FLASH_CR 寄存器中的 STRT 位置 1
- 等待 BSY 位归零
- 读取已擦除的页并验证

图3.2 主闪存页擦除

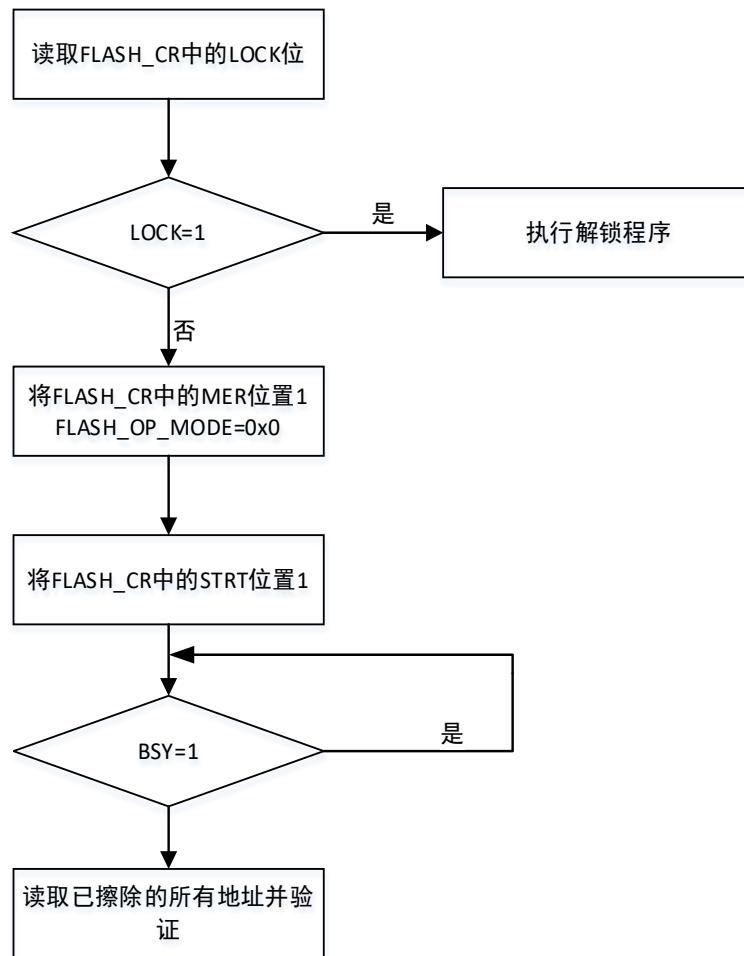


3.1.4.5 主闪存区整片擦除

用整片擦除命令可以一次性擦除整个主闪存区，但信息块不会受这个命令影响，具体步骤如下：

- 通过检查 FLASH_SR 寄存器中的 BSY 位来检查是否有正在进行的闪存操作，BSY 等于 0 进行下一步操作
- 将 FLASH_OP_MODE 配置位 0x0
- 将 FLASH_CR 寄存器中的 MER 位置 1
- 将 FLASH_CR 寄存器中的 STRT 位置 1
- 等待 BSY 位归零
- 读取所有擦除页并验证

图3.3 主存储区整片擦除



3.1.4.6 选项字节区编程

选项字节的编程与常规用户地址不同，包括 4 个写保护，1 个读保护，1 个硬件配置和 2 个用户数据区。解锁编程擦除控制器后，还需要解锁选项字节区编程，相同的密钥 KEY1 和 KEY2 写入 FLASH_OPTKEYR 寄存器，FLASH_CR 寄存器中的 OPTWRE 位会被置 1，然后就可以把 FLASH_CR 中的 OPTPG 置 1，再按半字单位或字单位写目标地址。

编程擦除控制器首先读取被编程地址的数据，并检查是否是已擦除状态。如果不是已擦除状态，相关操作会被取消并且在 FLASH_SR 中的 WRPRTERR 位提示错误。编程操作结束后，会由 FLASH_SR 寄存器的 EOP 位给出提示。

选项字节为 32 位数据，分为两个半字，高 16 位和低 16 位，在编程过程中，控制器接受低 8 位数据，会自动把高 8 位设置为低 8 位的反码，保证选项字节的写入值总是对的。步骤如下：

- 通过检查 FLASH_SR 寄存器中的 BSY 位来检查是否有正在进行的闪存操作，BSY 等于 0 进行下一步操作。
- 检查 FLASH_CR 的 OPTWRE，如果是 1，执行字节选项区解锁序列，FLASH_OPTKEYR 写入 KEY1=0x45670123 和 KEY2=0xCDEF89AB。如果是 0，继续下一步操作。
- 将 FLASH_CR 寄存器中的 OPTPG 位置 1。
- 向需要的地址中写半字或字。
- 等待 BSY 位被重置。
- 读写入的数据并验证。

当闪存读取保护选项 RDPRT 从受保护更改为不受保护时，在对闪存保护选项 RDPRT 重新编程之前将执行主闪存的整片擦除。如果用户想更改读取保护选项以外的其他选项，则不执行整体擦除。读保护选项字节的已擦除状态可保护闪存数据。

3.1.4.7 选项字节区擦除

- 选项字节擦除步骤如下：
- 通过检查 FLASH_SR 寄存器中的 BSY 位来检查是否有正在进行的闪存操作，BSY 等于 0 进行下一步操作。

检查 FLASH_CR 的 OPTWRE，如果是 1，执行字节选项区解锁序列，FLASH_OPTKEYR 写入 KEY1=0x45670123 和 KEY2=0xCDEF89AB。如果是 0，继续下一步操作。

- 配置 FLASH_OP_MODE 等于 0x1
- 将 FLASH_CR 寄存器中的 OPTER 位置 1
- 将 FLASH_CR 寄存器中的 STAR 位置 1
- 等待 BSY 位被重置
- 读擦除数据并验证

3.1.5 存储保护

可以防止闪存的用户区域被不信任的代码读取，有以下两种类型的保护：

- 读保护
- 页写保护

3.1.5.1 读保护

通过设置 RDP 选项字节，然后通过应用系统复位来重新加载新的 RDP 选项字节。

一旦启动读取保护：

- 除了用户代码外，不允许对主闪存区读取访问。（从主闪存区启动且没有启动调试模式才可以访问）
- 将代码加载到 SRAM 或从 SRAM 执行代码相关的所有功能都是有效的（JTAG/SWD 和 SRAM 启动），这可用于禁用读取保护。当读取保护选项字节更改为存储器不保护状态时，将执行整片擦除。
- 从 SRAM 启动时，不允许通过代码以及使用 DMA1 和 DMA2 读取的数据访问闪存。
- 不允许通过使用 SWD（串行线调试）进行数据访问闪存。

当 RDP 选项字节及其反码包含下表中所示的一对值时，闪存受到保护。

表3.2 闪存保护状态

RDP 值	RDP 反码	读保护状态
0xAA	0x55	零级保护
0xFF	0x00	一级保护
0xCC	0x33	二级保护
任意值	非 RDPRT 反码	一级保护

注意：擦除选项字节块不会触发整片擦除，因为擦除值（0xFF）对应于受保护的值得。

- 由读保护到禁用读保护的步骤：
- 擦除整个选项字节区域，读保护码（RDP）将为 0xFF。在此阶段，读取保护仍有效。
- 为取消内存保护，将 RDP 代码编程为 0xAA，这个操作首先强制对主闪存进行大规模擦除。

- 复位产品 (POR 复位) 以重新加载选项字节 (和新的 RDP 代码), 并禁用读保护。

3.1.5.2 主闪存区写保护

WRP3、WRP2、WRP1、WRP0 选项字节用于控制主闪存的写保护, 每一个 bit 控制 8 个 sector。

如果在受保护的页面上执行编程或擦除操作, 则闪存将在闪存状态寄存器 FLASH_SR 的 WRPRTERR 位返回保护错误标志。通过配置 WRP3、WRP2、WRP1、WRP0 选项字节, 然后应用系统复位以重新加载新的 WRP3、WRP2、WRP1、WRP0 选项字节, 可以激活写保护。

非保护

为了解除写保护, 提供了一个应用案例:

案例 1: 取消写保护后解除读保护

- 擦除整个选项字节区域。
- 取消解除读保护, 写入正确的 RDP 代码 0xAA 以取消内存保护, 此操作首先强制对主闪存进行大规模擦除。
- 复位产品 (系统复位) 以重新加载选项字节。

3.1.5.3 选项字节区写保护

默认状况下, 字节选项始终是可访问的, 并且具有写保护。为了获得对字节选项的写访问 (编程和擦除) 权限, 需将 KEY1 和 KEY2 写入 FLASH_OPTKEYR 寄存器。正确的密钥序列可以启动对选项字节的写访问, FLASH_CR 寄存器的 OPTWRE 位拉高表示写权限开启。通过软件写 0 复位 OPTWRE 位可以关闭写访问权限。

3.1.6 选项字节区描述

有 8 个选项字节, 由终端客户根据应用需求配置。例如, 独立看门狗可以选择硬件模式, 也可以选择软件模式。一个 32 位的选项字节被分割如下表所示。

表3.3 选项字节格式

31~24	23~16	15~8	7~0
选项字节 1 的反码	选项字节 1	选项字节 0 的反码	选项字节 0

选项字节结构如下表所示, 选项字节可以通过 FLASH_OBR 寄存器读取。选项字节区内容更改以后, 需要系统复位重新装载。

表3.4 选项字节结构

Flash 地址	[31:24]	[23:16]	[15:8]	[7:0]
0x1FFF F800	nUSER	USER	nRDP	RDP
0x1FFF F804	nDATA1	DATA1	nDATA0	DATA0
0x1FFF F808	nWRP1	WRP1	nWRP0	WRP0
0x1FFF F80C	nWRP3	WRP3	nWRP2	WRP2

表3.5 选项字节描述

地址	位置	名称	功能描述
0x1FFF_F800	[31:24]	nUSER	
	[23:22]	2'b11	USER[7:6], 保留选项, 没有使用。FLASH_OBR 寄存器 RES_OPT 读出
	[21]	ram_parity_check	0: RAM parity check enabled 1: RAM parity check disabled
	[20]	reversed	
	[19]	nBOOT1	与 BOOT0 协同作用, 选择启动区域
	[18]	nRST_STDBY	USER[2], 进入待机模式时的复位事件。 0: 进入待机模式时产生复位。 1: 不产生复位 FLASH_OBR 寄存器 nRST_STDBY 读出
	[17]	nRST_STOP	USER[1], 进入停止模式时的复位事件。 0: 进入停止模式时产生复位 1: 不产生复位 FLASH_OBR 寄存器 nRST_STOP 读出
	[16]	WDG_SW	USER[0], 0-硬件开启 WDG; 1-软件启动 WDG FLASH_OBR 寄存器 WDG_SW 读出
	[15:8]	nRDP	
	[7:0]	RDP	读保护选项。 读保护选项用于保护客户存储在 Flash 中的软件代码。设置 RDP 选项字节激活。RDP 配置为 0xAA, FLASH_OBR 寄存器的 RDPRT 拉低, 开启 FLASH 的读权限。
0x1FFF_F804	[31:24]	nDATA1	
	[23:16]	DATA1[7:0]	Data1, 用于客户存储数据; FLASH_OBR 寄存器 Data1 读出
	[15:8]	nDATA0	
	[7:0]	DATA0[7:0]	Data0, 用于客户存储数据; FLASH_OBR 寄存器 Data0 读出
0x1FFF_F808	[31:24]	nWRP1	
	[23:16]	WRP1	主闪存区写保护, 1-没有保护; 0-有保护。页 64 到页 127: [0]: 64~71; [1]: 72~79; [2]: 80~87; [3]: 88~95; [4]: 96~103; [5]: 104~111; [6]: 112~119; [7]: 120~127; FLASH_OBR 寄存器 WRP1 读出
	[15:8]	nWRP0	

地址	位置	名称	功能描述
	[7:0]	WRP0	主闪存区写保护，1-没有保护；0-有保护。页 0 到页 63： [0]: 0~7; [1]: 8~15; [2]: 16~23; [3]: 24~31; [4]: 32~39; [5]: 40~47; [6]: 48~55; [7]: 56~63; FLASH_OBR 寄存器 WRP0 读出
0x1FFF_F80C	[31:24]	nWRP3	
	[23:16]	WRP3	主闪存区写保护，1-没有保护；0-有保护。页 48 到页 255： [0]: 192~199; [1]: 200~207; [2]: 208~215; [3]: 216~223; [4]: 224~231; [5]: 232~239; [6]: 240~247; [7]: 248~255; FLASH_OBR 寄存器 WRP3 读出
	[15:8]	nWRP2	
	[7:0]	WRP2[7:0]	主闪存区写保护，1-没有保护；0-有保护。页 32 到页 47： [0]: 128~135; [1]: 136~143; [2]: 144~151; [3]: 152~159 [4]: 160~167; [5]: 168~175; [6]: 176~183; [7]: 184~191; FLASH_OBR 寄存器 WRP2 读出

3.2 闪存寄存器描述

3.2.1 闪存访问控制寄存器 (FLASH_ACR)

偏移地址：0x00，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										Reserved	Reserved	Reserved	LATENCY		
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW
31:3	Reserved	保留，必须保持清除。	0x000 0000	ro
2:0	LATENCY	这些位代表 SYSCLK（系统时钟）周期与闪存访问时间的比率。 0 等待状态:如果 0<HCLK≤32MHz 1 等待状态:如果 32MHz<HCLK≤64MHz 其他:禁止	0x0	rw

3.2.2 闪存控制器密钥寄存器 (FLASH_KEYR)

偏移地址：0x04，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEYR[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEYR[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bit	Field	Description	Reset value	RW
31:0	KEYR	Flash 写和擦除密钥: LOCK 的密钥, KEY1=0x45670123 KEY2=0xCDEF89AB	0x00000000	wo

3.2.3 闪存选项字节密钥寄存器 (FLASH_OPTKEYR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OPTKEYR[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPTKEYR[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bit	Field	Description	Reset value	RW
31:0	OPTKEYR	选项字节区写和擦除密钥: OPTWRE 的钥匙。 KEY1=0x45670123 KEY2=0xCDEF89AB	0x00000000	wo

3.2.4 闪存状态寄存器 (FLASH_SR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										EOP	WRPRTERR	Reserved	PGERR	Reserved	BSY
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	ro	rw	ro	ro

Bit	Field	Description	Reset value	RW
31:6	Reserved	保留, 必须保持清除。	0x000 0000	ro
5	EOP	操作结束: Flash 操作 (编程/擦除) 完成时由硬件置位。通过写入 1 复位。 注意: EOP 在每次成功的编程或擦除操作结束时被置位。	0x0	rw
4	WRPRTERR	写保护错误: 在对闪存的写保护地址进行编程时硬件自动置为 1。通过写入 1 重置为 0。	0x0	rw
3	Reserved	保留, 必须保持清除。	0x0	ro
2	PGERR	编程错误: 当要编程的地址在编程前的数据与“0xFFFF”不同时硬件自动置 1。通过写入 1 重置为 0。 注意: FLASH_CR 寄存器中的 STRT 位应在开始编程操作之前复位。	0x0	rw
1	Reserved	保留, 必须保持清除。	0x0	ro
0	BSY	Busy 标志: 1 表示正在进行 Flash 操作。这是在 Flash 操作开始时设置的, 并在操作完成或发生错误时重置。	0x0	ro

3.2.5 闪存控制寄存器 (FLASH_CR)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			EOPIE	Reserved.	ERRIE	OPTWRE	Reserved.	LOCK	STRT	OPTER	OPTPG	Reserved.	MER	PER	PG
ro	ro	ro	rw	ro	rw	rw	ro	rw	rw	rw	rw	ro	rw	rw	rw

Bit	Field	Description	Reset value	RW
31:14	Reserved	保留，必须保持清除。	0x0 0000	ro
13	OBL_LAUNCH	装载选项字节区 0: 无效 1: 装载选项字节区，产生系统复位	0x0	rw
12	EOPIE	操作结束中断使能：当 FLASH_SR 寄存器中的 EOP 位变为 1 时，该位使能中断产生。 0: 中断产生禁止 1: 中断产生使能	0x0	rw
11	Reserved	保留，必须保持清除。	0x0	ro
10	ERRIE	允许在写/擦除操作错误时产生中断，当 FLASH_SR 寄存器中 PGERR/WRPRTE RR 为 1 时会产生中断。 0: 中断产生禁止 1: 中断产生使能	0x0	rw
9	OPTWRE	1 表示可以对选项字节区进行编程。将正确的序列写入 FLASH_OPTKEYR 寄存器时硬件自动置为 1。该位可由软件写 0 复位。	0x0	rw
8	Reserved	Reserved	0x1	ro
7	LOCK	1 表示写/擦除控制和 FLASH_CR 被锁定。该位在检测到解锁序列后由硬件置 0。在解锁操作不成功的情况下，该位保持设置直到下一次复位。该位软件仅能写 1。	0x1	rw
6	STRT	设置为 1 触发 ERASE 操作。该位仅由软件设置并在 BSY 位复位时自动复位。	0x0	rw
5	OPTER	选项字节区擦除。	0x0	rw
4	OPTPG	选项字节区编程。	0x0	rw
3	Reserved	保留，必须保持清除。	0x0	ro
2	MER	擦除整个主闪存区	0x0	rw
1	PER	已选择页面擦除。	0x0	rw
0	PG	已选择闪存编程。	0x0	rw

3.2.6 闪存地址寄存器 (FLASH_AR)

偏移地址：0x14，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FAR[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FAR[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bit	Field	Description	Reset value	RW attribute
31:0	FAR	页擦除的地址。 注意：当 FLASH_SR 寄存器中的 BSY 位被设置时，对该寄存器的写访问被阻止。	0x0	wo

3.2.7 字节选项字节寄存器 (FLASH_OBR)

偏移地址：0x1C，复位值：0x03FF FFFC。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Data1								Data0							
ro								ro							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		RAM_PARITY_CHECK		Reserved		nBOOT1		nRST_STDBY		nRST_STOP		WDG_SW		Reversed	
ro		ro		ro		ro		ro		ro		ro		ro	

Bit	Field	Description	Reset value	RW attribute
31:24	Data1	Data1 存储在 Flash 选项字节区域	0xFF	ro
23:16	Data0	Data0 存储在 Flash 选项字节区域	0xFF	ro
15:14	保留	配置为 0	0x0	ro
13	RAM_PARITY_CHECK	0: RAM parity check enabled 1: RAM parity check disabled	0x1	ro
12	reversed	-	0x1	ro
11	nBOOT1	与 BOOT0 协同作用，选择启动区域	0x1	ro
10	nRST_STDBY	进入待机模式时的复位事件。 0: 进入待机模式时产生复位。 1: 不产生复位。	0x1	ro
9	nRST_STOP	进入停止模式时的复位事件。 0: 进入停止模式时产生复位 1: 不产生复位	0x1	ro
8	WDG_SW	选择看门狗事件：硬件或软件。 0: 硬件看门狗 1: 软件看门狗	0x1	ro
7:3	reversed	-	0x0	ro
2:1	RDPRT	读保护级别 00: 没有保护 01: 一级保护 11: 二级保护	0x1	ro
0	OPTERR	1 表示选项字节错误，这表示加载的选项字节不符合编码规则。注意：该位是只读的。	0x0	ro

3.2.8 闪存写保护寄存器 (FLASH_WRP3)

偏移地址：0x20，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WRP3								WRP2							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WRP1								WRP0							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset value	RW attribute	Remark
31:24	WRP3	写保护： 该寄存器包含由 OBL 加载的写保护选项字节。 0: 写保护激活 1: 写保护未激活 注意：这些位是只读的。	0xFF	ro	
23:16	WRP2	写保护： 该寄存器包含由 OBL 加载的写保护选项字节。 0: 写保护激活 1: 写保护未激活 注意：这些位是只读的。	0xFF	ro	

Bit	Field	Description	Reset value	RW attribute	Remark
15:8	WRP1	写保护： 该寄存器包含由 OBL 加载的写保护选项字节。 0: 写保护激活 1: 写保护未激活 注意：这些位是只读的。	0xFF	ro	
7:0	WRP0	写保护： 该寄存器包含由 OBL 加载的写保护选项字节。 0: 写保护激活 1: 写保护未激活 注意：这些位是只读的。	0xFF	ro	

3.2.9 闪存操作模式寄存器 (FLASH_OP_MODE)

偏移地址：0x24，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														PREPG	ERASE_MODE
ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	ro	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
31:2	Reserved	保留，必须保持清除。	0x0000 0000	ro	
1	PREPG	预编程操作选择指示： 0: 无预编程操作 1: 增加预编程操作	0x0	rw	
0	ERASE_MODE	Flash 擦除模式选择 0: 单模 1: 重试模式	0x0	rw	

4 复位和时钟控制单元 (RCMU)

4.1 复位

RCMU 模块处理芯片所需的所有复位和时钟。系统时钟在启动时选择，但是在复位时会自动将 HSI (内部 RC8MHz 振荡器) 选择为默认的 CPU 时钟。

启动后如果选择 HSE (外部 4-32MHz 时钟)，可以对其进行监测，当检测到故障，系统将自动切换回 HSI，并可以产生一个 NMI (不可屏蔽中断)。如果系统选择 PLL 为时钟，而且 PLL 用 HSE 为时钟源，同样也可以对 HSE 进行监测并在出现故障时将系统时钟切换到 HSI 并产生 NMI。

产品包含多个预分频器来配置 AHB、APB1、APB2 频率，最大频率都为 64MHz。

4.1.1 系统复位

系统复位会将所有寄存器设置为其复位值，但时钟控制器 CSR 寄存器中的复位标志和备份域中的寄存器除外。发生以下事件之一时，将生成系统复位：

- NRST 引脚上的低电平 (外部复位)
- 窗口看门狗计数结束条件 (WWDG 复位)
- 独立的看门狗结束计数条件 (IWDG 复位)
- 软件复位 (SW 复位)
- 低功耗模式复位
- CPU 锁死复位
- 电压比较复位 (LVD 复位)

可以通过检查控制/状态寄存器 RCMU_CSR 中的复位标志来识别复位源。

4.1.2 WWDG 复位

WWDG 模块可以通过设置计数器来定时产生复位，计数器使用的是系统时钟，精度高，但是待机模式下会关闭。

4.1.3 IWDG 复位

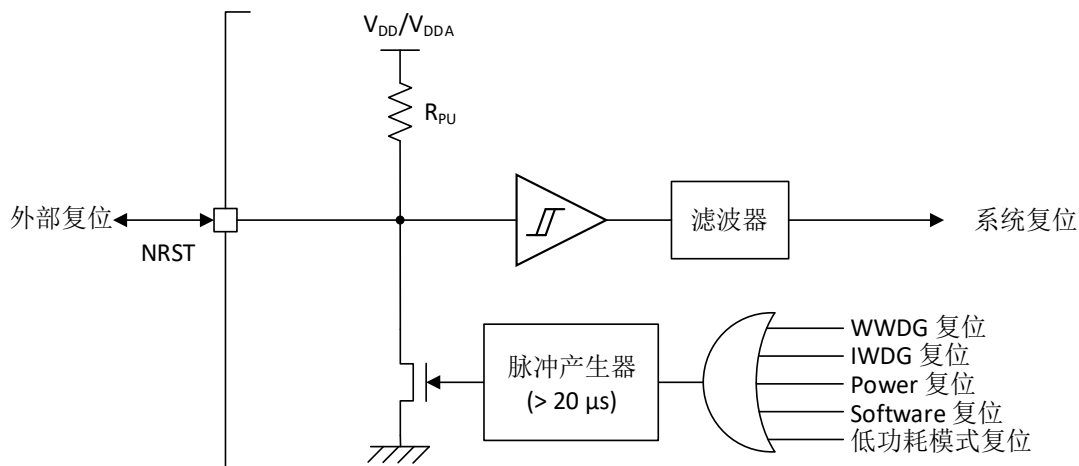
IWDG 模块可以通过设置计数器来定时产生复位，计数器使用的是 LSI 作为时钟，待机模式下不会关闭。

4.1.4 NRST 复位

外部复位可以通过 NRST (低电平有效) 进行复位，同时芯片内部复位信号也会在 NRST 引脚上输出。

脉冲产生器可以保证每个内部复位源的最小复位脉冲持续时间为 20μs。

图4.1 复位电路框图



4.1.5 软件复位

Cortex-M0+可以通过给内部寄存器的 SYSRESETREQ 位置 1 进行软件复位。

4.1.6 低功耗模式复位

有两种方法可以产生低功耗管理复位：

- 进入待机模式时的复位：通过设置闪存用户字节选项中的 nRST_STDBY 位，可以在成功执行待机模式序列后进行复位，复位时会退出待机模式。
- 进入停止模式时的复位：通过设置闪存复位用户字节选项中的 nRST_STOP 位，可以在成功执行停止模式序列后进行复位，复位时会退出停止模式。

4.1.7 CPU 锁死复位

Cortex-M0+自带的 lockup 输出信号可以直接进行系统复位。

4.1.8 电压比较复位

可以通过使用 LVD 模块进行电压比较，根据结果进行系统复位。

4.1.9 电源复位

发生以下事件之一时，将产生电源重置：

- 上电/掉电复位 (POR / PDR 复位)
- 退出待机模式时

4.2 时钟

可以使用三种不同的时钟源来驱动系统时钟 (SYSCLK)：

- HSI 内部高速时钟，8MHz
- HSE 外部高速时钟，4-32MHz
- PLL 时钟，max: 64MHz，使用 HSI 或 HSE 做源

- 闪存编程接口时钟（flash_clk）始终是 HSI 时钟。

如果在时钟中断寄存器 (RCMU_CIR) 中设置对对应使能, 则会产生一个中断。

可以使用时钟控制寄存器 (RCMU_CR) 中的 HSEON 位来打开和关闭 HSE 晶振。

外部时钟源 (HSE 旁路)

这种模式下, 必须提供一个外部时钟源, 它的频率最高可达 25 MHz。

可以通过将时钟控制寄存器 (RCMU_CR) 中的 HSEBYP 和 HSEON 位置 1 来选择此模式。

此时需要用占空比约为 50% 的外部时钟信号 (正方形, 正弦或三角形) 驱动 OSC_IN 引脚, 而 OSC_OUT 引脚应保持高电平。

4.2.2 HSI 时钟

HSI 时钟信号由内部 8MHz RC 振荡器产生, 可以直接用作系统时钟, 也可以用作 PLL 输入。HSI RC 振荡器的优势在于可以以低成本提供时钟源 (无需外部组件)。它的启动时间也比 HSE 晶体振荡器快, 但是即使进行校准, 其频率也比外部晶体振荡器或陶瓷谐振器的精度低。

时钟控制寄存器 (RCMU_CR) 中的 HSIRDY 标志指示 HSI RC 是否稳定。每次启动后硬件会一直等到 HSIRDY 拉高后才会使用该时钟。

可以使用时钟控制寄存器 (RCMU_CR) 中的 HSION 位来打开和关闭 HSI RC。

如果 HSE 晶体振荡器发生故障, 则 HSI 信号也可用作备用源 (辅助时钟)。

4.2.3 锁相环 (PLL)

内部 PLL 可用于倍增 HSI RC 输出或 HSE 晶体输出时钟频率。参见时钟控制寄存器 (RCMU_CR)。在使能 PLL 之前, 必须先完成 PLL 配置 (选择 HSI 振荡器或将 HSE 振荡器用作 PLL 输入时钟, 以及倍增系数)。一旦启用了 PLL, 就无法更改这些参数。如果在时钟中断寄存器 (RCMU_CIR) 中使能了 PLL 中断, 则在 PLL 准备就绪时会产生一个中断。

4.2.4 LSE 时钟

LSE 晶体是一个 32.768 kHz 低速外部晶体或陶瓷振荡器。它具有为实时时钟外设 (RTC) 提供低功耗, 高精度时钟源的优势, 以实现时钟/日历或其他计时功能。使用备份域控制寄存器 (RCMU_BDCR) 中的 LSEON 位可以打开和关闭 LSE 晶体。备用域控制寄存器 (RCMU_BDCR) 中的 LSERDY 标志指示 LSE 晶体是否稳定。每次启动后硬件会一直等到 LSERDY 拉高后才会使用该时钟。如果在时钟中断寄存器 (RCMU_CIR) 中使能了中断, 则会产生一个中断。

外部时钟源 (旁路 LSE)

在这种模式下, 必须提供一个外部时钟源。它的频率可以高达 1 MHz。通过将备用域控制寄存器 (RCMU_BDCR) 中的 LSEBYP 和 LSEON 位置 1, 可以选择此模式。此时需要用占空比约为 50% 的外部时钟信号 (正方形, 正弦波或三角形) 驱动 OSC32_IN 引脚, 而 OSC32_OUT 引脚应保持为高阻态。

4.2.5 LSI 时钟

LSI RC 充当低功耗时钟源, 可为独立看门狗 (IWDG) 和 RTC 在停止和待机模式下运行。时钟频率约为 40 kHz (介于 30 kHz 和 60 kHz 之间)。可以使用控制/状态寄存器 (RCMU_CSR) 中的 LSION 位打开和关闭 LSI RC。控制/状态寄存器 (RCMU_CSR) 中的 LSIRDY 标志指示低速内部振荡器是否稳定。在 LSIRDY 拉高后系统才会使用该时钟。如果在时钟中断寄存器 (RCMU_CIR) 中使能了中断, 则会产生一个中断。

4.2.6 系统时钟选择 (SYSCLK)

系统复位后, 选择 HSI 振荡器作为系统时钟, 之后可以通过修改 RCMU_CFGR 的 SW 来切换。

每次时钟的切换，硬件都会等待并确认该时钟源准备好后才会执行，此时可以通过读取 RCMU_CFGR 的 SWS 来判断是否已经切换完成，如果长时间未完成（SWS 不等于 SW），则表示新的时钟源配置可能有问题。

4.2.7 时钟安全系统 (CSS)

时钟安全系统在 HSE 启动并且稳定后即自动打开，一旦检测到 HSE 故障，CSS 会将系统时钟切换回 HSI 并产生 NMI（不可屏蔽中断），另外还会作为时钟故障事件同步发送到高级控制定时器（TIM1）的刹车输入端。

通过设置时钟中断寄存器（RCMU_CIR）中的 CSSC 位可以清除 NMI。

如果将 HSE 振荡器送给 PLL 做源，然后 PLL 作为系统时钟，此时检测到 HSE 故障的操作也是一样：启动 HSI、系统时钟切回 HSI、产生 NMI 中断、发送时钟故障事件给 TIMER。

4.2.8 RTC 时钟

RTCCLK 时钟源可以是 HSE/128，LSE 或 LSI 时钟。通过对备用域控制寄存器（RCMU_BDCR）中的 RTCSEL [1: 0]位进行编程来选择该位。

4.2.9 看门狗时钟

如果通过硬件选项或软件访问来启动独立看门狗（IWDG），则 LSI 振荡器被强制打开且无法禁用。在 LSI 振荡器稳定后，将时钟提供给 IWDG。

4.2.10 时钟输出 (MCO)

产品时钟输出（MCO）功能允许将时钟输出到外部 MCO 引脚，相应 GPIO 端口的配置寄存器必须先选成复用输出模式，之后可以选择 7 个时钟信号之一作为 MCO 时钟。由时钟配置寄存器（RCMU_CFGR）的 MCO [2: 0]位来配置。

输出前可以先进行分频，另外注意 PLL 的时钟必然会先进行一次二分频。

4.3 寄存器

4.3.1 时钟控制寄存器 (RCMU_CR)

偏移地址：0x00，复位值：0x7020 0021。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	HSE_STARTUP			Reserved	PLLRDY	PLLON	HSE_FORCE_OK	HSE_DL			Reserved	HSEBYP	HSERDY	HSEON	
r	rw	rw	rw	r	r	r	rw	rw	rw	rw	rw	rw	rw	r	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSE_EXT	HSE_STAB_RST	Reserved							HSI_FREQ_SEL			Reserved		HSIRDY	HSION
rw	rw	r	r	r	r	r	r	r	rw	rw	rw	r	r	r	rw

Bit	Field	Description	Reset	RW	Remark
31	Reserved		0x0	r	
30:28	HSE_STARTUP	HSE 时钟产生后生效的稳定时间配置。 0: 1 个 clk; 1: 256 个 clk; 2: 512 个 clk; ... 7: 16384 个 clk;	0x7	rw	
25	PLLRDY	PLL 锁定状态。 0: 未锁定 1: 已锁定	0x0	r	

Bit	Field	Description	Reset	RW	Remark
24	PLLON	PLL 启动使能; 0: 关闭 PLL 1: 启动 PLL 可软件写 1, 系统时钟不为 PLL 时可软件写 0; PLL 源为 HSE 时, HSE 发生故障后硬件自动清零; 进入停止或待机模式时硬件自动清零;	0x0	rw	
23	HSE_FORCE_OK	1: 强制拉高 HSE_OK	0x0	rw	
22:20	HSE_DL	HSE 增益控制。	0x2	rw	
19	Reserved		0x0	rw	
18	HSEBYP	HSE 直通使能; 0: HSE 正常振荡使用; 1: 仅通过 IN 直接灌入时钟; HSEON 为 0 时可软件写 1 或写 0;	0x0	rw	
17	HSERDY	HSE 稳定使能; 0: HSE 未稳定 ; 1: HSE 已稳定;	0x0	r	
16	HSEON	HSE 启动使能; 0: 关闭 HSE; 1: 启动 HSE; 可软件写 1, 系统时钟不为 HSE 时可软件写 0; HSE 发生故障后硬件自动清零; 进入停止或待机模式时硬件自动清零;	0x0	rw	
15	HSE_EXT	1: 绕过 HSE 模拟电路直接使用 PAD 灌入时钟	0x0	rw	
14	HSE_STAB_RST	1: 对 HSE_STAB 电路进行复位	0x0	rw	
13:7	Reserved		0x0	r	
6:4	HSI_FREQ_SEL	HSI 频率范围选择: 0:保留; 1:保留; 2:8MHz; 3:保留;	0x2	rw	
3:2	Reserved		0x0	r	
1	HSIRDY	HSI 稳定使能; 0: HSI 未稳定 ; 1: HSI 已稳定;	0x0	r	
0	HSION	HSI 启动使能; 0: 关闭 HSI; 1: 启动 HSI; 可软件写 1, 系统时钟不为 HSI 时可软件写 0; HSE 发生故障后硬件自动拉高; 退出停止或待机模式时硬件自动拉高;	0x1	rw	

4.3.2 时钟配置寄存器 (RCMU_CFGR)

偏移地址: 0x04, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	MCOPRE			Reserved	MCO			PPRE2				PPRE1			
r	rw	rw	rw	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	SLOW_SEL	Reserved	HPRE									SWS		SW	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r	rw	rw

Bit	Field	Description	Reset	RW	Remark
31	Reserved		0x0	r	

Bit	Field	Description	Reset	RW	Remark
30:28	MCOPRE	输出时钟分频选择。 0:1 分频; 1:2 分频; 2:4 分频; ... 7:128 分频;	0x0	rw	
27	Reserved		0x0	r	
26:24	MCO	输出时钟源选择。 0:无时钟; 1:RTC; 2:LSI; 3:LSE; 4:SYS_CLK; 5:HSI; 6:HSE; 7:PLL 二分频	0x0	rw	
23:20	PPRE2	PCLK2 频率配置, PCLK2 从 HCLK 分频而来; 0:1 分频; 1:2 分频; 2:3 分频; ... 15:16 分频;	0x0	rw	
19:16	PPRE1	PCLK1 频率配置, PCLK1 从 HCLK 分频而来; 0:1 分频; 1:2 分频; 2:3 分频; ... 15:16 分频;	0x0	rw	
15	Reserved		0x0	r	
14	SLOW_SEL	SYS_CLK 的慢时钟源选择信号, 仅在 SW=3 时有 效; 0: LSI; 1: LSE;	0x0	rw	
13	Reserved		0x0	r	
12:4	HPRE	HCLK 频率配置, HCLK 从 SYSCLK 分频而来; 0:1 分频; 1:2 分频; 2:3 分频; ... 511:512 分频;	0x0	rw	
3:2	SWS	实际系统时钟选择源, 由硬件更新; 当 SW 配置完后 SWS 没有更新, 表示新切换的时 钟未锁定或者未准备好。 00: HSI 用作系统时钟 01: HSE 用作系统时钟 10: PLL 用作系统时钟 11: LSI 或 LSE 用作系统时钟;	0x0	r	
1:0	SW	系统时钟源选择; 从停机/待机模式返回时硬件自动清零; 直接使用 HSE 或者间接使用 HSE (从 PLL) 后发 生 HSE 故障, 硬件自动清零; 00: HSI 被选为系统时钟 01: HSE 被选为系统时钟 10: PLL 被选为系统时钟 11: LSI 或 LSE 被选为系统时钟	0x0	rw	

4.3.3 时钟中断寄存器 (RCMU_CIR)

偏移地址: 0x08, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CS SC	Reserved		PLL RDY C	HSE RDY C	HSIR DY C	LSER DY C	LSIRD YC
r	r	r	r	r	r	r	r	w	rw	rw	w	w	w	w	w
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			PLL RDY E	HSE RDY E	HSIR DY E	LSER DY E	LSIRD Y E	CS SF	Reserved		PLL RDY F	HSE RDY F	HSIR DY F	LSER DY F	LSIRD YF
r	r	r	rw	rw	rw	rw	rw	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW	Remark
31:24	Reserved		0x0	r	
23	CSSC	清除时钟安全系统中断 该位由软件设置以清除 CSSF 标志。 0: 无影响 1: 清除 CSSF 安全系统中断标志位	0x0	w	
22:21	Reserved		0x0	r	
20	PLL RDY C	清除 PLL 就绪中断位 该位由软件设置来清除 PLL RDY F 标志。 0: 无影响 1: PLL RDY F 清零	0x0	w	
19	HSE RDY C	清除 HSE 就绪中断位 该位由软件设置来清除 HSE RDY F 标志。 0: 无影响 1: HSE RDY F 清零	0x0	w	
18	HSIR DY C	清除 HSI 就绪中断位 该位由软件设置来清除 HSIR DY F 标志。 0: 无影响 1: HSIR DY F 清零	0x0	w	
17	LSER DY C	清除 LSE 就绪中断位 该位由软件设置来清除 LSER DY F 标志。 0: 无影响 1: LSER DY F 清零	0x0	w	
16	LSIRD YC	清除 LSI 就绪中断位 该位由软件设置来清除 LSIRD YF 标志。 0: 无影响 1: LSIRD YF 清零	0x0	w	
15:13	Reserved		0x0	r	
12	PLL RDY E	启用/禁用由 PLL 锁定引起的中断, 由软件置位和清零。 0: PLL 锁定中断禁用 1: PLL 锁定中断使能	0x0	rw	
11	HSE RDY E	启用/禁用由 HSE 稳定引起的中断, 由软件置位和清零。 0: HSE 就绪中断禁止 1: HSE 就绪中断使能	0x0	rw	
10	HSIR DY E	启用/禁用由 HSI 稳定引起的中断, 由软件置位和清零。 0: HSI 就绪中断禁止 1: HSI 就绪中断使能	0x0	rw	
9	LSER DY E	启用/禁用由 LSE 稳定引起的中断, 由软件置位和清零。 0: LSE 就绪中断禁止 1: LSE 就绪中断使能	0x0	rw	



Bit	Field	Description	Reset	RW	Remark
8	LSIRDYIE	启用/禁任由 LSI 稳定引起的中断，由软件置位和清零。 0: LSI 就绪中断禁用 1: LSI 就绪中断使能	0x0	rw	
7	CSSF	时钟安全系统中断标志 当在 HSE 中检测到故障时由硬件置“1”。 通过软件设置 CSSC 位清除零。 0: 无 HSE 时钟故障引起的时钟安全中断 1: HSE 时钟故障引起的时钟安全中断	0x0	r	
6:5	Reserved		0x0	r	
4	PLLRDYF	PLL 就绪中断标志 当 PLL 锁定并设置 PLLRDYDIE 有效 时由硬件置“1”。通过软件写 PLLRDYC 位等于 1 清零。 0: 没有由 PLL 锁定引起的时钟就绪中断 1: PLL 锁定引起的时钟就绪中断	0x0	r	
3	HSERDYF	HSE 就绪中断标志 当 HSE 时钟稳定且 HSERDYDIE 有效时由硬件置 1。 通过软件设置 HSERDYC 位清零。 0: 无 HSE 时钟就绪中断 1: 有 HSE 时钟就绪中断	0x0	r	
2	HSIRDYF	HSI 就绪中断标志位 当 HSI 时钟稳定且 HSIRDYDIE 有效时由硬件置 1。 通过软件设置 HSIRDYC 位清零。 0: 无 HSI 时钟就绪中断 1: 有 HSI 时钟就绪中断	0x0	r	
1	LSERDYF	LSE 就绪中断标志 当 LSE 时钟稳定且 LSERDYDIE 有效时由硬件置位。通过软件设置 LSERDYC 位清零。 0: 无 LSE 时钟就绪中断 1: 有 LSE 时钟就绪中断	0x0	r	
0	LSIRDYF	LSI 就绪中断标志 当 LSI 时钟稳定且 LSIRDYDIE 有效时由硬件置位。 通过软件设置 LSIRDYC 等于 1 来清零。 0: 无 LSI 时钟就绪中断 1: 有 LSI 时钟就绪中断	0x0	r	

4.3.4 APB2 外设复位寄存器 (RCMU_APB2RSTR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DBGMCU RST	BEEP RST	LVD RST	Reserved	Tim17 RST	Tim16 Rst	Tim15 Rst
r	r	r	r	r	r	r	r	r	rw	rw	rw	r	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPTIM RST	USART1 RST	Reserved	SPI1 RST	TIM1 RST	AWKTIM RST	ADC RST	Reserved							IEMU RST	SYSCFG RST
rw	rw	r	rw	rw	rw	rw	r	r	r	r	r	r	r	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:23	Reserved		0x0	r	
22	DBGMCU_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
21	BEEP_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
20	LVD_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
19	Reserved		0x0	r	
18	TIM17_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
17	TIM16_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
16	TIM15_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
15	LPTIM_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
14	USART1_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
13	Reserved		0x0	rw	
12	SPI1RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
11	TIM1RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
10	AWKTIM_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
9	ADC_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	
8:2	Reserved	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	r	
1	IEMU_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	r	
0	SYSCFG_RST	由软件置位和清零。 0: 无影响 1: 复位接口	0x0	rw	

4.3.5 APB1 外设复位寄存器 (RCMU_APB1RSTR)

偏移地址: 0x10, 复位值: 0x0000 0000。



Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PMU RST	Reserved				VC RST	I2C2 RST	I2C1 RST	LPUART RST	Reserved		USART2 RST	Reserved
r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw	r
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	SPI2 RST	Reserved	IWDG R ST	WWDG R ST	RTC RST	Reserved	TIM14 RST	Reserved	Reserved	TIM7 RST	TIM6 RST	Reserved		TIM3 RST	Reserved
r	rw	r	rw	rw	rw	r	rw	r	r	rw	rw	r	r	rw	r

Bit	Field	Description	Reset	RW	Remark
31:29	Reserved		0x0	r	
28	PMU_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
27:24	Reserved		0x0	r	
23	VC_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
22	I2C2_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
21	I2C1_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
20	LPUART_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
19:18	Reserved		0x0	r	
17	USART2_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
16:15	Reserved		0x0	r	
14	SPI2RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
13	Reserved		0x0	r	
12	IWDGRST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
11	WWDGRST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
10	RTC_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
9	Reserved		0x0	r	
8	TIM14_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
5	TIM7_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
4	TIM6_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
3:2	Reserved		0x0	r	
1	TIM3_RST	由软件置位和清零。 0: 无影响 1: 复位	0x0	rw	
0	Reserved		0x0	r	

4.3.6 AHB 外设时钟使能寄存器 (RCMU_AHBENR)

偏移地址: 0x14, 复位值: 0x0000 0014。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												IODEN	IOCEN	IOBEN	IOAEN
r	r	r	r	r	r	r	r	r	r	r	r	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								AESEN	CRCEN	TRNGEN	FLITFEN	Reserved	SRAMEN	Reserved	DMAEN
r	r	r	r	r	r	r	r	rw	rw	rw	rw	r	rw	r	rw

Bit	Field	Description	Reset	RW	Remark
31:20	Reserved		0x0	r	
19	IODEN	由软件置位和清零。 0: IOD 时钟禁用 1: IOD 时钟使能	0x0	rw	
18	IOCEN	由软件置位和清零。 0: IOC 时钟禁用 1: IOC 时钟使能	0x0	rw	
17	IOBEN	由软件置位和清零。 0: IOB 时钟禁用 1: IOB 时钟使能	0x0	rw	
16	IOAEN	由软件置位和清零。 0: IOA 时钟禁用 1: IOA 时钟使能	0x0	rw	
15:8	Reserved		0x0	r	
7	AESEN	由软件置位和清零。 0: AES 时钟禁用 1: AES 时钟使能	0x0	rw	
6	CRCEN	由软件置位和清零。 0: CRC 时钟禁用 1: CRC 时钟使能	0x0	rw	
5	TRNGEN	由软件置位和清零。 0: TRNG 时钟禁用 1: TRNG 时钟使能	0x0	rw	
4	FLITFEN	由软件置位和清零以在休眠模式期间禁用/启用 FLITF 时钟。 0: 休眠模式期间禁用 FLITF 时钟 1: 休眠模式期间使能 FLITF 时钟	0x1	rw	
3	Reserved		0x0	r	
2	SRAMEN	由软件置位和清零以在休眠模式期间禁用/启用 SRAM 接口时钟。 0: 在休眠模式下禁用 SRAM 接口时钟 1: 在休眠模式下使能 SRAM 接口时钟	0x1	rw	
1	Reserved		0x0	r	
0	DMAEN	由软件置位和清零。 0: DMA 时钟禁用 1: DMA 时钟使能	0x0	rw	

4.3.7 APB2 外设时钟使能寄存器 (RCMU_APB2ENR)

偏移地址: 0x18, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DBGMCUEN	BEEPEN	LVDEN	Reserved	TIM17EN	TIM16EN	TIM15EN
r	r	r	r	r	r	r	r	r	rw	rw	rw	r	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPTIMEN	USART1EN	Reserved	SPI1EN	TIM1EN	AWKTIMEN	ADCEN	Reserved							IEMUEN	SYSCFGEN
rw	rw	r	rw	rw	rw	rw	r	r	r	r	r	r	r	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:23	Reserved		0x0	r	
22	DBGMCUEN	由软件置位和清零。 0: DBGMCU 接口时钟禁用 1: DBGMCU 接口时钟使能	0x0	rw	
21	BEEPEN	由软件置位和清零。 0: BEEP 时钟禁用 1: BEEP 时钟使能	0x0	rw	
20	LVDEN	由软件置位和清零。 0: LVD 时钟禁用 1: LVD 时钟使能	0x0	rw	
19	Reserved		0x0	r	
18	TIM17EN	由软件置位和清零。 0: TIM17 时钟禁用 1: TIM17 时钟使能	0x0	rw	
17	TIM16EN	由软件置位和清零。 0: TIM16 时钟禁用 1: TIM16 时钟使能	0x0	rw	
16	TIM15EN	由软件置位和清零。 0: TIM15 时钟禁用 1: TIM15 时钟使能	0x0	rw	
15	LPTIMEN	由软件置位和清零。 0: LPTIM 时钟禁用 1: LPTIM 时钟使能	0x0	rw	
14	USART1EN	由软件置位和清零。 0: USART1 时钟禁用 1: USART1 时钟使能	0x0	rw	
13	Reserved		0x0	r	
12	SPI1EN	由软件置位和清零。 0: SPI1 时钟禁用 1: SPI1 时钟使能	0x0	rw	
11	TIM1EN	由软件置位和清零。 0: TIM1 定时器时钟禁用 1: TIM1 定时器时钟使能	0x0	rw	
10	AWKTIMEN	由软件置位和清零。 0: AWKTIM 时钟禁用 1: AWKTIM 时钟使能	0x0	rw	
9	ADCEN	由软件置位和清零。 0: ADC 接口禁用 1: ADC 接口时钟使能	0x0	rw	
8:2	Reserved		0x0	r	
1	IEMUEN	由软件置位和清零。 0: IEMU 时钟禁用 1: IEMU 时钟使能	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
0	SYSCFGEN	由软件置位和清零。 0: SYSCFG 时钟禁用 1: SYSCFG 时钟使能	0x0	rw	

4.3.8 APB1 外设时钟使能寄存器 (RCMU_APB1ENR)

偏移地址: 0x1C, 复位值: 0x0000 1000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PMUEN	Reserved				VCEN	I2C2EN	I2C1EN	LPUARTEN	Reserved		USART2EN	Reserved
r	r	r	rw	r	r	r	r	rw	rw	rw	rw	r	r	rw	r
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	SPI2EN	Reserved	IWDGEN	WWDGEN	RTCEN	Reserved	TIM14EN	Reserved		TIM7EN	TIM6EN	Reserved		TIM3EN	Reserved
r	rw	r	rw	rw	rw	r	rw	r	r	rw	rw	r	r	rw	r

Bit	Field	Description	Reset	RW	Remark
31:29	Reserved		0x0	r	
28	PMUEN	由软件置位和清零。 0: PMU 时钟禁用 1: PMU 时钟使能	0x0	rw	
27:24	Reserved		0x0	r	
23	VCEN	由软件置位和清零。 0: VC 时钟禁用 1: VC 时钟使能	0x0	rw	
22	I2C2EN	由软件置位和清零。 0: I2C2 时钟禁用 1: I2C2 时钟使能	0x0	rw	
21	I2C1EN	由软件置位和清零。 0: I2C1 时钟禁用 1: I2C1 时钟使能	0x0	rw	
20	LPUARTEN	由软件置位和清零。 0: LPUART 时钟禁用 1: LPUART 时钟使能	0x0	rw	
19:18	Reserved		0x0	r	
17	USART2EN	由软件置位和清零。 0: 禁用 USART2 时钟 1: 使能 USART2 时钟	0x0	rw	
16:15	Reserved		0x0	r	
14	SPI2EN	由软件置位和清零。 0: SPI2 时钟禁用 1: SPI2 时钟使能	0x0	rw	
13	Reserved		0x0	r	
12	IWDGEN	由软件置位和清零。 0: 独立看门狗时钟禁用 1: 独立看门狗时钟使能	0x0	rw	
11	WWDGEN	由软件置位和清零。 0: 窗口看门狗时钟禁用 1: 窗口看门狗时钟使能	0x0	rw	
10	RTCEN	由软件置位和清零。 0: RTC 时钟禁用 1: RTC 时钟使能	0x0	rw	
9	Reserved		0x0	r	



Bit	Field	Description	Reset	RW	Remark
8	TIM14EN	由软件置位和清零。 0: TIM14 时钟禁用 1: TIM14 时钟使能	0x0	rw	
7:6	Reserved		0x0	r	
5	TIM7EN	由软件置位和清零。 0: TIM7 时钟禁用 1: TIM7 时钟使能	0x0	rw	
4	TIM6EN	由软件置位和清零。 0: TIM6 时钟禁用 1: TIM6 时钟使能	0x0	rw	
3:2	Reserved		0x0	r	
1	TIM3EN	由软件置位和清零。 0: TIM3 时钟禁用 1: TIM3 时钟使能	0x0	rw	
0	Reserved		0x0	r	

4.3.9 备份控制寄存器 (RCMU_BDCR)

偏移地址: 0x20, 复位值: 0x0000 1c78。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															BDRST
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTCCEN	LSE_FORCEOK	LSE_EXT	LSE_STARTUP		RTCSEL		Reserved		LSE_DL		LSEBYP		LSEBYP	LSEBYP	LSEON
rw	rw	rw	rw	rw	rw	rw	rw	r	rw	rw	rw	rw	rw	r	rw

Bit	Field	Description	Reset	RW	Remark
31:17	Reserved		0x0	r	
16	BDRST	由软件置位和清零。 0: 复位未激活 1: 复位整个备份域	0x0	rw	
15	RTCCEN	由软件置位和清零。 0: RTC 时钟禁用 1: RTC 时钟使能	0x0	rw	
14	LSE_FORCE_OK	1: 强制让 LSE_OK 为高	0x0	rw	
13	LSE_EXT	0: 使用 LSE 模拟电路产生 LSE 时钟 1: 绕过 LSE 模拟电路直接使用 PAD 灌入时钟	0x0	rw	
12:10	Reserved		0x0	rw	
9:8	RTCSEL	由软件设置以选择 RTC 的时钟源。一旦选择了 RTC 时钟源, 除非重置备份域, 否则无法再更改它。BDRST 位可用于复位它们。 00: 无时钟 01: LSE 振荡器时钟用作 RTC 时钟 10: LSI 振荡器时钟用作 RTC 时钟 11: HSE 振荡器时钟 128 分频用作 RTC 时钟	0x0	rw	
7	Reserved		0x0	r	
6:3	LSE_DL	设置 LSE 振荡器的增益控制	0x0011	rw	
2	LSEBYP	在调试模式下由软件置位和清零以绕过振荡器。只有在禁用外部 32 kHz 振荡器时才能写入该位。 0: 未旁路 LSE 振荡器 1: LSE 振荡器被旁路	0x0	rw	



Bit	Field	Description	Reset	RW	Remark
1	LSERDY	由硬件置位和清零以指示外部 32 kHz 振荡器何时稳定。LSEON 位清零后, LSERDY 在 6 个外部低速振荡器时钟周期后变低。 0: 外部 32 kHz 振荡器未准备好 1: 外部 32 kHz 振荡器准备就绪	0x0	r	
0	LSEON	由软件置位和清零。 0: 外部 32 kHz 振荡器关闭 1: 外部 32 kHz 振荡器开启	0x0	rw	

4.3.10 控制/状态寄存器 (RCMU_CSR)

偏移地址: 0x24, 复位值: 0x0c00 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPWR RSTF	WWDG RSTF	IWDG RSTF	SFT RSTF	PWR RSTF	PIN RSTF	LOCKUP RSTF	RMVF	LVD RSTF	Reserved						
rw	rw	rw	rw	rw	rw	rw	Rw	rw	r	r	r	r	r	r	r
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													LSI_FREQ_SEL	LSIRDY	LSION
r	r	r	r	r	r	r	r	r	r	r	r	r	rw	r	rw

Bit	Field	Description	Reset	RW	Remark
31	LPWRRSTF	发生低功耗管理复位时由硬件置位。通过写入 RMVF 位清零。 0: 未发生低功耗管理复位 1: 发生低功耗管理复位	0x0	r	
30	WWDGRSTF	当窗口看门狗复位发生时由硬件设置。通过写入 RMVF 位清零。 0: 没有发生窗口看门狗复位 1: 发生窗口看门狗复位	0x0	r	
29	IWDGRSTF	当发生来自 VDD 域的独立看门狗复位时由硬件置位。通过写入 RMVF 位清零。 0: 没有发生看门狗复位 1: 发生看门狗复位	0x0	r	
28	SFTRSTF	当发生软件复位时由硬件置位。通过写入 RMVF 位清零。 0: 未发生软件复位 1: 发生软件复位	0x0	r	
27	PWRRSTF	当 POR/PDR 复位发生时由硬件置位。通过写入 RMVF 位清零。 0: 未发生 POR/PDR 复位 1: POR/PDR 复位发生	0x1	r	
26	PINRSTF	当 NRST 引脚发生复位时由硬件置位。通过写入 RMVF 位清零。 0: 未发生来自 NRST 引脚的复位 1: 从 NRST 引脚发生复位	0x1	r	
25	LOCKUPSTF	当 M0+发生 LOCKUP 复位发生时由硬件置位。通过写入 RMVF 位清零。 0: 未发生 LOCKUP 复位 1: LOCKUP 复位发生	0x0	r	
24	RMVF	由软件置位以清除复位标志。 0: 无影响 1: 清除复位标志	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
23	LVDSTF	当 LVD 复位发生时由硬件置位。通过写入 RMVF 位清零。 0: 未发生 LVD 复位 1: LVD 复位发生	0x0	r	
22:3	Reserved		0x0	r	
2	LSE_FREQ_SEL	LSE 频率选择 0: 低频 1: 高频	0x0	r	
1	LSIRDY	由硬件置位和清零以指示 LSI 何时稳定。 0: LSI 未准备好 1: LSI 准备就绪	0x0	r	
0	LSION	由软件置位和清零。 0: LSI 关闭 1: LSI 开启	0x0	rw	

4.3.11 AHB 外设复位寄存器 (RCMU_AHBRSTR)

偏移地址: 0x28, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												IODRST	IOCRST	IOBRST	IOARST
r	r	r	r	r	r	r	r	r	r	r	r	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								AESRST	CRCRST	TRNGRST	FLITFRST	Reserved	SRAMRST	Reserved	DMARST
r	r	r	r	r	r	r	r	rw	rw	rw	rw	r	rw	r	rw

Bit	Field	Description	Reset value	RW	Remark
31:20	Reserved		0x0		
19	IODRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
18	IOCRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
17	IOBRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
16	IOARST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
15:8	Reserved		0x0		
7	AESRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
6	CRCRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
5	TRNGRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
4	FLITFRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
3	Reserved		0x0	r	

Bit	Field	Description	Reset value	RW	Remark
2	SRAMRST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	
1	Reserved		0x0	r	
0	DMARST	对模块进行复位; 0: 不变; 1: 复位;	0x0	rw	

4.3.12 时钟配置寄存器 2 (RCMU_CFGR2)

偏移地址: 0x2C, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIMPRE2				TIMPRE1				Reserved							
rw	rw	rw	rw	rw	rw	rw	rw	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset value	RW	Remark
31:16	Reserved		0x0	r	
15:12	TIMPRE2	TIM 时钟 2 的预分频选择: 0:1 分频; 1:2 分频; ... 15:16 分频	0x0	r	
11:8	TIMPRE1	TIM 时钟 1 的预分频选择: 0:1 分频; 1:2 分频; ... 15:16 分频	0x0	r	
7:0	Reserved		0x0	r	

4.3.13 PLL 配置寄存器 (RCMU_PLLCR)

偏移地址: 0x40, 复位值: 0x0070 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								PLL_STARTUP			Reserved		PLL_BYP		PLL_SRC
r	r	r	r	r	r	r	r	rw	rw	rw	r	r	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CFG0_OD		Reserved		CFG0_N		Reserved			CFG0_M		
r	r	r	rw	rw	r	rw	rw	r	r	r	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:23	Reserved		0x0	r	
22:20	PLL_STARTUP	PLL 时钟产生后生效时间选择 0:1 个时钟; 1:128 个时钟; 2:256 个时钟; ... 7:16384 个时钟	0x0	rw	
19:18	Reserved		0x0	r	
17	PLL_BYP	PLL 内部 BYPASS 选择; 0: 选择锁相环输出; 1: 选择输入源直通输出;	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
16	PLL_SRC	PLL 输入源选择: 0: 选择 HSI; 1: 选择 HSE;	0x0	rw	
15:13	Reserved		0x0	r	
12:11	CFG0_OD	根据 PLL 输出频率配置: 小于 16MHz: 0 16 到 32MHz: 1 32 到 48MHz: 2 48 到 64MHz: 3	0x0	Rw	
10	Reserved		0x0	r	
9:8	CFG0_N	根据 PLL 输入源频率配置: 4 MHz: 0 8 MHz: 1	0x0	Rw	
7:5	Reserved		0x0	r	
4:0	CFG0_M	PLL 倍频倍数: 1:1 倍频; 2:2 倍频; ... 32:32 倍频	0x0	Rw	

4.3.14 PLL 配置寄存器 2 (RCMU_PLLCR2)

偏移地址: 0x44, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			STAB_RST	SEL_OD	SEL_N	SEL_M	RST_SEL	RST_TH							
r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CFG1_OD			CFG1_N			Reserved	CFG1_M					
r	r	rw	rw	rw	rw	rw	rw	r	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:29	Reserved		0x0	r	
28	STAB_RST	1: 对 PLL_STAB 电路进行 RST	0x0	rw	
27	SEL_OD	0: 使用 CFG0; 1: 使用 CFG1;	0x0	rw	
26	SEL_N	0: 使用 CFG0; 1: 使用 CFG1;	0x0	rw	
25	SEL_M	0: 使用 CFG0; 1: 使用 CFG1;	0x0	rw	
24	SEL_RST	0: PLL_EN 拉高后立刻释放 PLL_RST; 1: 等待一段时间再释放;	0x0	rw	
23:16	RST_TH	PLL_EN 拉高后保持 RST 的时钟数, SEL_RST=1 时有效	0x0	rw	
15:14	Reserved		0x0	r	
13:11	CFG1_OD	直接配置 PLL_OD, SEL_OD=1 时有效	0x0	rw	
10:8	CFG1_N	直接配置 PLL_N, SEL_N=1 时有效	0x0	rw	
7	Reserved		0x0	r	
6:0	CFG1_M	直接配置 PLL_M, SEL_M=1 时有效	0x0	rw	

4.3.15 ST 寄存器 (RCMU_STCR)

偏移地址: 0x5C, 复位值: 0x0001 d4bf。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				STCLK_ON	STCLK_SEL	STCALIB[25:16]									
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
STCALIB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:28	Reserved		0x0	r	
27	STCLK_ON	外部 STCLK 使能	0x0	rw	
26	STCLK_SEL	外部 STCLK 源选择; 0: LSI; 1: LSE;	0x0	rw	
25:0	STCALIB	STCLK 相关控制, 具体参考 M0+手册	0x1d4bf	rw	

4.3.16 中断延迟配置寄存器 (RCMU_IRQLATENCY)

偏移地址: 0x60, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								Irqlatency							
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:8	Reserved		0x0	r	
7:0	irqlatency	中断响应延迟, 具体参考 M0+手册	0x0	rw	

5 电源管理单元 (PMU)

5.1 电源模块 (PMU)

5.1.1 功能描述

PMU 用于整个芯片的电源管理。芯片的主电源 VDD 供电电压范围是 1.8~5.5V，通过内置的电压调节器提供内部 2.0V 和 1.2V 数字电源。针对不同的应用场景有四种电源模式：运行模式，睡眠模式、停机模式和待机模式。

5.1.2 电源

5.1.2.1 供电方案

VDD=1.8~5.5V，外部电源通过 VDD 引脚给所有 IO 供电。

VDDA=VDD，外部模拟电源为 ADC，复位单元，内部调压器，振荡器和锁相环供电，VDDA 必须连接到 VDD。

5.1.2.2 电压调节器

两个电压调节器：2.0V 电压调节器和 1.2V 电压调节器。

2.0V 电压调节器有 2 种工作模式：主模式 (MR) 和低功耗模式 (LPR)。

- 主模式用在常规的调节模式。调节器为 2.0V 的域提供电源 (RTC、IWDG、备份寄存器)。
- 低功耗模式用在待机模式下。调节器为 2.0V 的域提供低功耗电源。

1.2V 电压调节器有 3 种模式：主模式 (MR) 低功耗模式 (LPR) 和关闭模式。

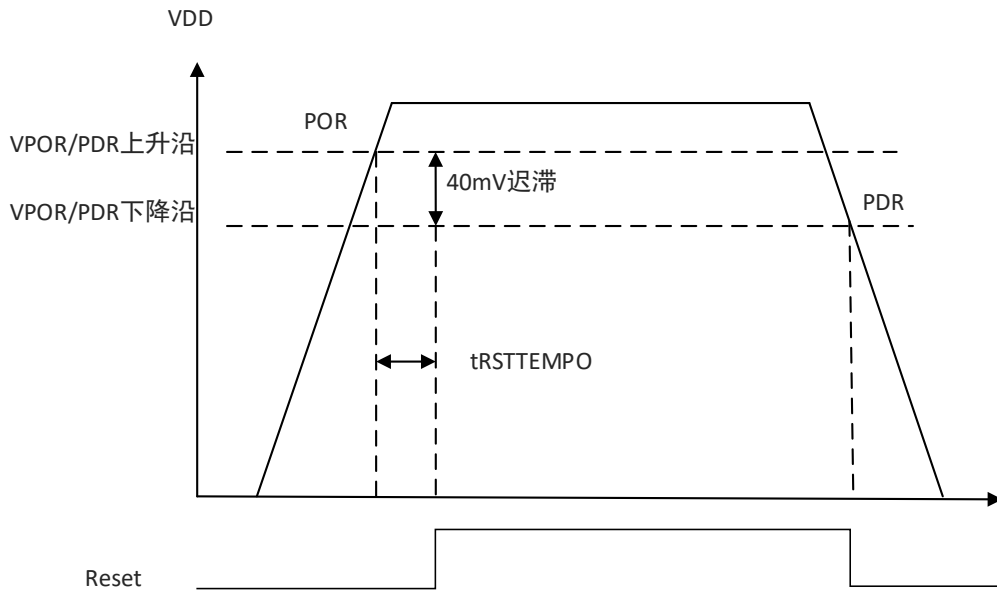
- 主模式用在常规的调节模式。调节器为 1.2V 的域提供电源 (内核，存储和数字周边)。
- 低功耗模式用在停止模式下。调节器为 1.2V 的域提供低功耗电源，保留了寄存器和 SRAM 的数据。
- 关闭模式用在待机模式下。调节器输出在高阻状态，核心电路被关闭 (寄存器和 SRAM 的内容丢失)。

5.1.3 电源管理器

5.1.3.1 上电复位 (POR) 和 掉电复位 (PDR)

产品集成 POR/PDR 电路，当供电电压达到 1.8V 时系统正常工作。当 VDD/VDDA 低于指定阈值 VPOR/PDR 时，产品保持在复位模式，而无需外部复位电路。

图5.1 上电复位和掉电复位波形



5.1.4 低功耗模式

系统或电源复位后，微控制器处于运行模式。当不需要保持 CPU 运行时，可以利用多种低功耗模式来降低功耗，例如等待某个外部事件。用户可以根据最低电源功耗、最短启动时间和可用唤醒源等条件，选择合适的低功耗模式。

有三种低功耗模式：

- 睡眠模式（CPU 时钟关闭，所有外设包括 Cortex M0+ 外设（如 NVIC，SysTick 等）均保持运行）
- 停止模式（所有时钟均停止）
- 待机模式（1.2 V 域断电）

此外，可以通过以下方式降低运行模式下的功耗：

- 降低系统时钟
- 关闭不使用的 APB 和 AHB 外设时钟

表5.1 低功耗模式

模式	进入	唤醒	1.2V 域时钟	VDD 域时钟	电压调节器
睡眠 (Sleep now or Sleep-on -exit)	WFI	任何中断	CPU 时钟关，模拟和数字时钟无影响	正常运行	开启
	WFE	唤醒事件			
停止 (Stop)	PDDS and LPDS bits + SLEEPDEEP bit + WFI 或 WFE	任何 EXTI 线 (在 EXTI 寄存器中配置)	1.2V 域时钟关	HSI 和 HSE 振荡器关	开启或处于低功耗模式 (PWR_CR 的 LPDS 控制))
待机 (Standby)	PDDS bit + SLEEPDEEP bit + WFI 或 WFE	WKUP 引脚上升沿或者下降沿 (PWR_CR 的 STBY_REDGE 控制)、RTC 闹钟、NRST 引脚复位、IWDG 复位			关闭

5.1.4.1 降低系统时钟

在运行模式下，可以通过对预分频器寄存器进行编程来降低系统时钟 (SYSCLK, HCLK, PCLK1, PCLK2) 的速度。这些预分频器还可用于在进入休眠模式之前降低外围设备的速度。有关更多详细信息，详见时钟配置寄存器 (RCMU_CFGR)

5.1.4.2 控制外部时钟

在运行模式下，可以随时停止单个外设和存储器的 HCLK 和 PCLKx，以降低功耗。为了进一步降低休眠模式下的功耗，可以在执行 WFI 或 WFE 指令之前禁用外设时钟。外设时钟由 AHB 外设时钟使能寄存器 (RCMU_AHBENR)，APB1 外设时钟使能寄存器 (RCMU_APB1ENR) 和 APB2 外设时钟使能寄存器 (RCMU_APB2ENR) 控制。

5.1.4.3 睡眠模式

进入睡眠模式

通过执行 WFI (等待中断) 或 WFE (等待事件) 指令进入睡眠模式。根据 Cortex M0+ 系统控制寄存器的 SLEEPONEXIT 位的数值，有两个选项可用于选择睡眠模式进入机制。

- 立即休眠 (Sleep-now): 如果 SLEEPONEXIT 位清零，则在执行 WFI 或 WFE 指令后，MCU 便进入休眠模式。
- 退出时休眠 (Sleep-on-exit): 如果 SLEEPONEXIT 位被置为 1，MCU 一旦退出最低优先级 ISR，便立即进入休眠模式。

在休眠模式下，所有 I/O 引脚保持与运行模式相同的状态。

退出睡眠模式

如果执行 WFI 指令进入休眠模式，任意一个嵌套向量中断控制器 (NVIC) 响应的外设中断都能将系统从休眠模式唤醒。

如果执行 WFE 指令进入休眠模式，一旦唤醒事件发生，系统退出休眠模式。唤醒事件可以通过下面方式产生：

- 在外设控制寄存器中使能一个中断（不是 NVIC 中断），并使能 Cortex M0+ 系统控制寄存器中的 SEVONPEND 位。当系统从 WFE 中唤醒后，必须清除外设的中断挂起位和外设的 NVIC IRQ 通道挂起位（在 NVIC 中断清除挂起寄存器中）。
- 配置一个外部或内部的 EXTI 线为事件模式。当系统从 WFE 恢复时，因为与事件线对应的挂起位未被设置，所以无需清除外设中断挂起位或 NVIC IRQ 通道挂起位。此模式的唤醒时间最短，因为在中断进入和退出过程中没有损失任何时间。

表5.2 Sleep-now 模式

Sleep-now 模式	描述
模式进入	在以下条件下执行 WFI（等待中断）或 WFE（等待事件）： -SLEEPDEEP = 0 和 -SLEEPONEXIT = 0 请参阅 Cortex M0+ 系统控制寄存器。
模式退出	(1) 如果执行 WFI 进入睡眠模式： 中断：参考中断和异常向量表 (2) 如果执行 WFE 进入唤醒事件： 唤醒事件：参考唤醒事件管理
唤醒延时	无

表5.3 Sleep-on-exit 模式

Sleep-on-exit 模式	描述
模式进入	在以下条件下执行 WFI 指令： -SLEEPDEEP = 0 和 -SLEEPONEXIT = 1 请参阅 Cortex M0+ 系统控制寄存器。
模式退出	中断：参考：中断和异常向量表
唤醒延时	无

5.1.4.4 停止模式

停止模式是在 Cortex M0+ 深度睡眠模式基础上结合外设的时钟控制机制，在停止模式下电压调节器可运行在正常模式或低功耗模式。在停止模式下，所有 1.2V 域中的时钟均被停止，PLL，HSI 和 HSE 被禁用，SRAM 和寄存器内容被保留。在停止模式下，所有 I/O 引脚保持与运行模式下相同的状态。

进入停止模式

为了进一步降低停止模式下的功耗，可以将电压调节器置于低功耗模式，这由电源控制寄存器（PWR_CR）的 LPDS 位配置。

如果正在进行闪存编程，直到对内存访问完成，系统才进入停止模式。

如果正在进行对 APB 的访问，直到对 APB 访问完成，系统才进入停止模式。

在停止模式下，可以通过编程各个控制位来选择以下功能：

- 独立看门狗 (IWDG)：通过写入密钥寄存器或硬件选项来启动 IWDG。IWDG 一旦启动，除了系统复位，否则无法停止。（详见 IWDG）
- 实时时钟 (RTC)：由备份控制寄存器 (RCMU_BDCR) 中的 RTCEN 位配置。
- 内部 RC 振荡器 (LSI RC)：控制/状态寄存器 (RCMU_CSR) 中的 LSION 位；
- 外部 32.768 kHz 振荡器 (LSE OSC)：由备份域控制寄存器 (RCMU_BDCR) 中的 LSEON 位配置。

在进入停止模式前，如果没有关闭 ADC，那么这些外设仍然消耗电量。将寄存器 ADC_CR 的 ADCEN 位写入 0 可关闭这两个外设。

退出停止模式

当中断或唤醒事件发生时系统会退出停止模式，这时 HSI RC 振荡器被选为系统时钟。

表5.4 停止模式

停止模式	描述
模式进入	在以下条件下执行 WFI（等待中断）或 WFE（等待事件）指令： - 设置 Cortex M0+ 系统控制寄存器中的 SLEEPDEEP 位 - 配置寄存器 PWR_CR 的 PDDS 位为 0 - 清除寄存器 PWR_CSR 的 WUF 位 - 配置寄存器 PWR_CR 的 LPDS 位选择电压调节器模式 注意：要进入停止模式，所有 EXTI 线路挂起位（寄存器 EXTI_PR）、外设中断挂起位和 RTC 警报标志必须被清除。否则停止模式的进入流程将会被跳过，程序继续运行
模式退出	(1) 如果执行 WFI 进入停止模式： 任何配置为中断模式的 EXTI 线（NVIC 中必须使能相应的外部中断向量）。请参阅中断和异常向量。 (2) 如果执行 WFE 进入停止模式： 任何配置为事件模式的任何 EXTI 线。请参阅唤醒事件管理。
唤醒延迟	HSI 唤醒时间 + 电压调节器从低功耗模式唤醒的时间

5.1.4.5 待机模式

待机模式可实现系统最低功耗。待机模式是在 Cortex M0+ 深度睡眠模式时关闭电压调节器。整个 1.2V 域电源被关闭，PLL、HSI 振荡器、HSE 振荡器被关闭，SRAM 和寄存器内容丢失，只有备份区域和待机电路维持供电。

进入待机模式

在待机模式下，可以选择以下功能：

- 独立看门狗 (IWDG)：通过写入密钥寄存器或硬件选项来启动 IWDG。IWDG 一旦启动，除了系统复位，否则无法停止。（详见 IWDG）
- 实时时钟 (RTC)：由备份控制寄存器 (RCMU_BDCR) 中的 RTCEN 位配置。
- 内部 RC 振荡器 (LSI RC)：由寄存器 RCC_CSR 中的 LSION 位配置。。
- 外部 32.768 kHz 振荡器 (LSE OSC)：由备份域控制寄存器 (RCMU_BDCR) 中的 LSEON 位配置。

退出待机模式

当发生外部复位 (NRST 引脚)、IWDG 复位、WKUP 引脚上的上升沿、RTC 警报上升沿时，系统退出待机模式。从待机模式唤醒后的程序执行等同于复位后的执行 (启动引脚采样，获取向量复位等)。电源控制/状态寄存器 (PWR_CSR) 中的 SBF 位表示系统从待机模式下被唤醒。

表5.5 待机模式

待机模式	描述
模式进入	在以下条件下 WFI (等待中断) 或 WFE (等待事件) 指令： -配置 Cortex M0+ 系统控制寄存器 SLEEPDEEP 为 1 -设置电源控制寄存器 (PWR_CR) 中的 PDDS 位为 1 -清除电源控制/状态寄存器(PWR_CSR)中的 WUF 位 -没有中断 (对于 WFI) 或事件 (对于 WFE) 待处理
模式退出	WKUP 引脚上的上升沿或下降沿 (由寄存器 PWR_STBY 的 STBY_WKUP_EDGE 选择)、RTC 报警事件的上升沿、NRST 引脚外部复位、IWDG 复位。
唤醒延迟	HSI 8M 启动时间+电压调节器启动时间+RESET 时间

待机模式下 I/O 状态

待机模式下，所有的 I/O 引脚都被锁定在进入前的状态(但是内部寄存器被复位)，除了

- Reset 引脚 (始终有效)
- 被配置为防侵入的 TAMPER 引脚
- 被使能的 WKUP 引脚

调试模式

默认情况下，如果在进行调试微处理器时微处理器进入停止或待机模式，将失去调试连接。这是因为 Cortex™-M0+ 的内核失去了时钟。但是，通过设置 DBGMCU_CR 寄存器中的某些配置位，可以在低功耗模式下调试软件。

5.1.4.6 低功耗模式的自动唤醒

RTC 可用于将系统从低功耗模式下唤醒，而无需依赖外部中断 (自动唤醒模式)。RTC 提供可编程的时间基数，用于周期性从停止或待机模式下唤醒。通过对控制寄存器 (RCMU_BDCR) 中的 RTCSEL [1: 0] 编程，三个 RTC 时钟源中的二个可以实现此功能：

- 低功耗 32.768 kHz 外部晶体振荡器 (LSE OSC)。该时钟源提供了一个精确的时基，具有非常低的功耗 (典型情况下的附加功耗小于 0.5μA)。
- 低功耗内部 RC 振荡器 (LSI RC)，该时钟源具有节省一个 32.768 kHz 晶振成本的优势。但是该内部 RC 振荡器将增加少许电源消耗。

RTC 警报中断唤醒停止模式，必须：

- 将 EXTI Line 17 配置为对上升沿敏感
- 配置 RTC 生成 RTC 警报

要从待机模式唤醒，无需配置 EXTI 17 线。

5.1.5 电源控制寄存器

5.1.5.1 电源控制寄存器 (PMU_CR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							DBP	Reserved				CSBF	CWUF	PDDS	LPDS
r	r	r	r	r	r	r	rw	r				rc_w1	rc_w1	rw	rw

Bit	Field	Description	Reset	RW
31:9	Reserved		0x0	r
8	DBP	备份域写保护。 复位后, RTC 和备份寄存器处于保护状态以防止意外写入。 设置该位以启用对这些寄存器的写保护。 0: 禁止访问 RTC 和备份寄存器 1: 启用对 RTC 和备份寄存器的访问	0x0	rw
7:4	Reserved	Reversed	0x0	-
3	CSBF	清除待机标志。 该位始终读为 0。 0: 无影响 1: 清除 SBF 待机标志 (写)。	0x0	rc_w1
2	CWUF	清除唤醒标志。 该位始终读为 0。 0: 无影响 1: 清除 WUF 唤醒标志。(写)	0x0	rc_w1
1	PDDS	掉电深度睡眠 该位由软件置位和清零。与 LPDS 协同工作。 0: CPU 进入深度睡眠时进入停止模式。 稳压器状态取决于 LPDS 位。 1: CPU 进入深度睡眠时进入待机模式。	0x0	rw
0	LPDS	低功耗深度睡眠 该位由软件置位和清零。与 PDDS 协同工作。PDDS=0 时 LPDS 配置才有效。 0: 停止模式下电压调节器是正常模式 1: 停止模式下稳压器是低功耗模式	0x0	rw

5.1.5.2 电源控制/状态寄存器 (PMU_CSR)

偏移地址: 0x04, 复位值: 0x0000 0000 (从待机模式唤醒时不被清除)。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									WUF_PIN7	Reserved		WUF_PIN4	Reserved	WUF_PIN2	WUF_PIN1
-									R	-		R	-	R	R
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	EWUP7	Reserved		EWUP4	Reserved	EWUP2	EWUP1	Reserved		WUF_RSTP	WUF_EXTI	WUF_IWDG	WUF_RTC	EWUP_PIN	SBF
-	R/W	-		R/W		R/W	R/W	-		R	R	R	R	R	R

Bit	Field	Description	Reset value	RW
31:23	Reserved	-	0x0	r
22	WUF_PIN7	PB15 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 在 WKUP 引脚上发生唤醒事件。	0x0	rw
19	WUF_PIN4	PA2 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 在 WKUP 引脚上发生唤醒事件。	0x0	rw
17	WUF_PIN2	PC13 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 在 WKUP 引脚上发生唤醒事件。	0x0	rw
16	WUF_PIN1	PA0 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 在 WKUP 引脚上发生唤醒事件。	0x0	rw
15	Reserved	-	0x0	r
14	EWUP7	启用 WKUP 引脚 PB15 该位由软件置位和清零。 0: WKUP 引脚用于通用 I/O。WKUP 引脚上的事件不会将设备从待机模式唤醒。 1: WKUP 引脚用于从待机模式唤醒并强制进入输入下拉配置(WKUP 引脚的上升沿将系统从待机模式唤醒)。 注意: 该位由系统复位复位。	0x0	rw
13:12	Reserved	-	0x0	r
11	EWUP4	启用 WKUP 引脚 PA2 该位由软件置位和清零。 0: WKUP 引脚用于通用 I/O。WKUP 引脚上的事件不会将设备从待机模式唤醒。 1: WKUP 引脚用于从待机模式唤醒并强制进入输入下拉配置(WKUP 引脚的上升沿将系统从待机模式唤醒)。 注意: 该位由系统复位复位。	0x0	rw
10	Reserved	-	0x0	r

Bit	Field	Description	Reset value	RW
9	EWUP2	启用 WKUP 引脚 PC13 该位由软件置位和清零。 0: WKUP 引脚用于通用 I/O。WKUP 引脚上的事件不会将设备从待机模式唤醒。 1: WKUP 引脚用于从待机模式唤醒并强制进入输入下拉配置 (WKUP 引脚的上升沿将系统从待机模式唤醒)。 注意: 该位由系统复位复位。	0x0	rw
8	EWUP1	启用 WKUP 引脚 PA0 该位由软件置位和清零。 0: WKUP 引脚用于通用 I/O。WKUP 引脚上的事件不会将设备从待机模式唤醒。 1: WKUP 引脚用于从待机模式唤醒并强制进入输入下拉配置 (WKUP 引脚的上升沿将系统从待机模式唤醒)。 注意: 该位由系统复位复位。	0x0	rw
7:6	Reserved	-	0x0	r
5	WUF_RSTP	RST PAD 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 出现 RST PAD 唤醒事件。	0x0	r
4	WUF_EXTI	EXTI 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 出现 EXTI 唤醒事件。	0x0	r
3	WUF_IWDG	IWDG 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 出现 IWDG 唤醒事件。	0x0	r
2	WUF_RTC	RTC 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 出现 RTC 闹钟事件。	0x0	r
1	WUF_PIN	PIN 唤醒标志 该位由硬件设置, 并只能由 POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的 CWUF 位清除。 0: 没有发生唤醒事件 1: 在 WKUP 引脚上发生唤醒事件。注: WKUP1、WKUP2、WKUP4、WKUP7 都会产生这个唤醒标志。	0x0	r
0	SBF	待机标志 该位由硬件设置, 只能通过 POR/PDR (上电复位/掉电复位)或通过设置电源控制寄存器 (PWR_CR) 中的 CSBF 位清除 0: 设备尚未处于待机模式 1: 设备一直处于待机模式	0x0	r

6 中断和事件 (IEMU)

6.1 中断和事件 (IEMU)

6.1.1 嵌套向量中断控制器

特性:

- 16 个 M0+ 内部中断外还包含 64 个可屏蔽中断通道
- 16 个可编程的优先等级
- 低延迟的异常和中断处理
- 电源管理控制
- 系统控制寄存器的实现

嵌套向量中断控制器 (NVIC) 和处理器核的接口直接相连, 可以实现低延迟的中断处理和有效处理多个中断同时到来, 更多信息请参考 ARM 的手册。

6.1.2 系统节拍 (systick) 校准值寄存器

系统节拍需要通过 RCMU 的 STCR 校准, 当系统时钟为 12MHz, 需要配置成 119999, 这样可以在 M0+ 内部产生固定 1ms 周期的 systick。

6.1.3 中断和异常向量列表

编号	优先级	优先级类型	名称	说明	地址
				保留	0x0000_0000
	-3	固定	RESET	复位	0x0000_0004
	-2	固定	NMI	RCMU 时钟安全系统(CSS)	0x0000_0008
	-1	固定	硬件失效	所有类型的失效	0x0000_000C
	0	可设置	存储管理	存储管理	0x0000_0010
	1	可设置	总线错误	预取指失败, 存储器访问失败	0x0000_0014
	2	可设置	错误应用	未定义的指令或非法状态	0x0000_0018
	-	-	-	保留	0x0000_001C
	-	-	-	保留	0x0000_0020
	-	-	-	保留	0x0000_0024
	-	-	-	保留	0x0000_0028
	3	可设置	SVCall	系统服务调用	0x0000_002C
	4	可设置	调试监控	调试监控器	0x0000_0030
	-	-	-	保留	0x0000_0034
	5	可设置	PendSV	可挂起的系统服务	0x0000_0038
	6	可设置	SysTick	系统节拍定时器	0x0000_003C
0	7	可设置	WWDG	窗口定时器中断	0x0000_0040
1	8	可设置	PVD	经过 EXTI 的电压检测中断	0x0000_0044
2	9	可设置	RTC	实时时钟全局中断	0x0000_0048
3	10	可设置	FLASH	闪存全局中断	0x0000_004C
4	11	可设置	RCMU	复位和时钟控制中断	0x0000_0050
5	12	可设置	EXTI0_1	外部事件线 0 和 1 混合中断	0x0000_0054
6	13	可设置	EXTI2_3	外部事件线 2 和 3 混合中断	0x0000_0058
7	14	可设置	EXTI4_15	外部事件线 4 到 15 混合中断	0x0000_005C
8	15	可设置	AES	AES 全局中断	0x0000_0060

编号	优先级	优先级类型	名称	说明	地址
9	16	可设置	DMA	DMA 全局中断 (6 个 CH)	0x0000_0064
10	17	可设置	AWKTIM	AWKTIM 全局中断	0x0000_0068
11	18	可设置	TAMP	入侵检测中断	0x0000_006C
12	19	可设置	ADC	ADC 全局中断	0x0000_0070
13	20	可设置	TIM1_BRK_UP_TRG_COM	TIM1 混合中断	0x0000_0074
14	21	可设置	TIM1_CC	TIM1 捕获比较中断	0x0000_0078
15	22	可设置	SRAM	SRAM 全局中断	0x0000_007C
16	23	可设置	TIM3	TIM3 局中断	0x0000_0080
17	24	可设置	TIM6	TIM6 全局中断	0x0000_0084
18	25	可设置	TIM7	TIM7 全局中断	0x0000_0088
19	26	可设置	TIM14	TIM14 全局中断	0x0000_008C
20	27	可设置	TIM15	TIM15 全局中断	0x0000_0090
21	28	可设置	TIM16	TIM16 全局中断	0x0000_0094
22	29	可设置	TIM17	TIM17 全局中断	0x0000_0098
23	30	可设置	I2C1	I2C1 全局中断	0x0000_009C
24	31	可设置	I2C2	I2C2 全局中断	0x0000_00A0
25	32	可设置	SPI1	SPI1 全局中断	0x0000_00A4
26	33	可设置	SPI2	SPI2 全局中断	0x0000_00A8
27	34	可设置	USART1	USART1 全局中断	0x0000_00AC
28	35	可设置	USART2	USART2 全局中断	0x0000_00B0
29	36	可设置	LPUART	LPUART 全局中断	0x0000_00B4
30	37	可设置	TRNG	TRNG 全局中断	0x0000_00B8
31	38	可设置	VC	VC 全局中断	0x0000_00BC

6.1.4 外部中断/事件控制器 (EXTI)

外部中断/事件控制器包括多达 24 个边缘检测器，用于生成中断/事件请求。每条输入线可以独立配置来选择类型（事件或中断）和相应的触发事件（上升或下降或两者），每条线也可以独立屏蔽。寄存器可以维持中断请求的状态。

6.1.5 中断事件管理

本产品能够处理外部或内部事件，以唤醒内核 (WFE)。可以通过以下任一方式生成唤醒事件：

- 使能外设控制寄存器中的中断而不是 NVIC 中的中断，并使能 Cortex®-M0+ 系统控制寄存器中的 SEVONPEND 位。当 MCU 从 WFE 恢复时，必须清除外围中断挂起位和外围 NVIC IRQ 通道挂起位（在 NVIC 中断清除挂起寄存器中）。
- 在事件模式下配置外部或内部 EXTI 线路，当 CPU 从 WFE 恢复时，无需清除外设中断挂起位或 NVIC IRQ 通道挂起位，因为未设置与事件线相对应的挂起位。

6.1.6 中断产生

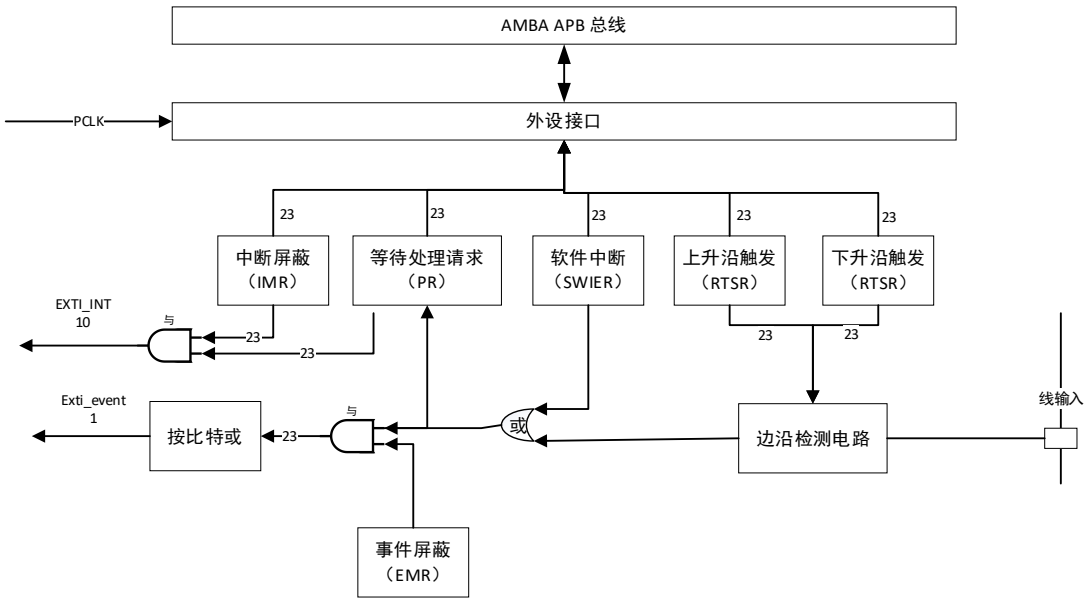
要生成中断，应配置并启用中断线。通过用所需的边沿检测对两个触发寄存器进行编程，并通过向中断寄存器的相应位写入“1”来启用中断请求，可以完成此操作。当所选边沿出现在外部中断线上时，将产生一个中断请求。与中断线相对应的未决位也被置位。通过在待处理寄存器中写入“1”来重置此请求。

硬件中断选择

要将 24 条线路配置为中断源，请使用以下过程：

- 配置 24 条中断线的屏蔽位 (EXTI_IMR)。
- 配置中断线的触发选择位 (EXTI_RTISR 和 EXTI_FTSR)。
- 配置使能和屏蔽位，以控制映射到外部中断控制器 (EXTI) 的 NVIC IRQ 通道，以便可以正确确认来自 24 条线之一的中断。

图6.1 中断与事件结构框图



6.1.7 事件产生

要生成事件，应配置并启用事件线。通过用所需的边沿检测对两个触发寄存器进行编程，以及通过向事件掩码寄存器中的相应位写入“1”来启用事件请求，即可完成此操作。当所选边沿出现在事件线上时，将生成事件脉冲。与事件行相对应的挂起位未设置。

硬件事件选择

要将 24 行配置为事件源，请使用以下过程：

- 配置 24 条事件线的屏蔽位 (EXTI_EMR)。
- 配置事件线的触发器选择位 (EXTI_RTSR 和 EXTI_FTSR)。

6.1.8 软件中断/事件产生

中断/事件请求也可以由软件生成，方法是在软件中断/事件寄存器中写入“1”。

软件中断/事件选择

24 条线可以配置为软件中断/事件线，以下是生成软件中断的过程：

- 配置 24 条中断/事件线的屏蔽位 (EXTI_IMR, EXTI_EMR)。
- 设置软件中断寄存器 (EXTI_SWIER) 的所需位。

6.1.9 外部中断/事件线映射

EXTI0-15 是通过 SYSCFG_EXTICR 寄存器配置从所有 GPIO 中选择产生，其余的都为内部模块产生。

表6.1 外部中断/事件连接

EXTI lines	Line source
exti line 0~15	GPIOs
exti line 16	LVD 输出
exti line 17	RTC 闹钟事件
exti line 18	RTC 改变事件
exti line 19	入侵检测事件

EXTI lines	Line source
exti line 20	USART1 唤醒事件
exti line 21	VC0 输出
exti line 22	VC1 输出
exti line 23	UART2 唤醒事件

6.2 寄存器

6.2.1 中断屏蔽寄存器 (EXTI_IMR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								MR23	MR22	MR21	MR20	MR19	MR18	MR17	MR16
ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:24	Reserved	保留, 必须保持清除。	0x00000000	ro	
23:0	MRx	线 x 上的中断屏蔽 0: 来自第 x 行的中断请求被屏蔽 1: 来自第 x 行的中断请求未被屏蔽	0x0	rw	

6.2.2 事件屏蔽寄存器 (EXTI_EMR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								MR23	MR22	MR21	MR20	MR19	MR18	MR17	MR16
ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
31:24	Reserved	保留, 必须保持清除。	0x00000000	ro	
23:0	EMRx	线 x 上的事件屏蔽 0: 来自第 x 行的事件请求被屏蔽 1: 来自第 x 行的事件请求未被屏蔽	0x0	rw	

6.2.3 上升沿触发选择寄存器 (EXTI_RTSTR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								TR23	TR22	TR21	TR20	TR19	TR18	TR17	TR16
ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
31:24	Reserved	保留, 必须保持清除。	0x00000000	ro	
23:0	TRx	x 线上升触发事件配置位 0: 输入线的上升触发禁用 (用于事件和中断) 1: 输入线的上升触发使能 (用于事件和中断)	0x0	rw	

外部唤醒线是边沿触发的, 这些线上不会产生毛刺。如果在写入 EXTI_RTISR 寄存器期间外部中断线上出现上升沿, 则未设置待处理位。可以为同一中断线设置上升沿和下降沿触发器, 在此设置中, 两者均会生成触发条件。

6.2.4 下降沿触发选择寄存器 (EXTI_FTSR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								TR23	TR22	TR21	TR20	TR19	TR18	TR17	TR16
ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:24	Reserved	保留, 必须保持清除。	0x00000000	ro	
23:0	TRx	x 线下降触发事件配置位 0: 输入线的下降触发禁用 (对于事件和中断) 1: 输入线的下降触发使能 (用于事件和中断)。	0x0	rw	

外部唤醒线是边沿触发的, 这些线上不会产生毛刺。

如果在写入 EXTI_FTSR 寄存器期间外部中断线上出现下降沿, 挂起位不能被正确置位。可以为同一中断线设置上升沿和下降沿触发器, 两者均会生成触发条件。

6.2.5 软件中断事件寄存器 (EXTI_SWIER)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								SWIER23	SWIER22	SWIER21	SWIER20	SWIER19	SWIER18	SWIER17	SWIER16
ro	ro	ro	ro	ro	ro	ro	ro	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER15	SWIER14	SWIER13	SWIER12	SWIER11	SWIER10	SWIER9	SWIER8	SWIER7	SWIER6	SWIER5	SWIER4	SWIER3	SWIER2	SWIER1	SWIER0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:24	Reserved	保留, 必须保持清除。	0x00000000	ro	
23:0	SWIERx	在线 x 上的软件中断 如果在 EXTI_IMR 中的这条线上启用了中断, 则在该位为“0”时向该位写入“1”会置位 EXTI_PR 中相应的挂起位, 从而产生中断请求。 通过清除 EXTI_PR 的相应位 (通过向该位写入 1) 来清除该位。	0x0	rw	

6.2.6 挂起寄存器 (EXTI_PR)

偏移地址: 0x14, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								PR23	PR22	PR21	PR20	PR19	PR18	PR17	PR16
ro	ro	ro	ro	ro	ro	ro	ro	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bit	Field	Description	Reset	RW	Remark
31:24	Reserved	保留，必须保持清除。	0x0000000	ro	
23:0	PRx	待处理位（挂起位） 0：没有触发请求发生 1：发生了选择的触发请求 当在外部中断线上发生了选择的边沿事件，该位被置位。 通过向该位写入“1”来清除该位。	0x0	rc_w1	

7 直接存储访问 (DMA)

7.1 直接存储访问 (DMA)

7.1.1 介绍

DMA 模块可以代替 CPU 自动进行数据的搬移，从而释放 CPU 的负载。

在使用中，可以通过对某个 CH 的配置，让其根据外设的工作状态，自动的请求 DMA 将数据写入或者读出 SRAM，也可以直接定义 2 块 SRAM 地址，让 DMA 来完成 MEMCOPY 功能。

7.1.2 主要特征

6 个 CH 独立工作；

支持 SRAM 和外设间的 4 种搬移模式；

传输完成中断和错误发生中断；支持连续模式；

7.1.3 外设列表

PERIPH*_SEL[5:0]	外设	PERIPH*_SEL[5:0]	外设
0x00	AES_RX	0x19	Timer1_up
0x01	AES_TX	0x1A	Timer1_Com
0x02	ADC	0x1D	Timer3_CH1
0x03	I2C1_RX	0x1E	Timer3_CH2
0x04	I2C1_TX	0x1F	Timer3_CH3
0x05	I2C2_RX	0x20	Timer3_CH4
0x06	I2C2_TX	0x21	Timer3_Trig
0x07	USART1_RX	0x22	Timer3_Up
0x08	USART1_TX	0x24	Timer6_Up
0x09	USART2_RX	0x26	Timer7_Up
0x0A	USART2_TX	0x28	Timer14_CH1
0x0B	SPI1_RX	0x29	Timer14_Up
0x0C	SPI1_TX	0x2A	Timer15_CH1
0x0D	SPI2_RX	0x2B	Timer15_Trig
0x0E	SPI2_TX	0x2C	Timer15_Up
0x14	Timer1_Ch1	0x2D	Timer15_Com
0x15	Timer1_CH2	0x2E	Timer16_CH1
0x16	Timer1_CH3	0x2F	Timer16_Up
0x17	Timer1_Ch4	0x30	Timer17_CH1
0x18	Timer1_Trig	0x31	Timer17_Up

7.1.4 使用示例

第一步，配置 DMA_CONFIG，打开 EN；

第二步，配置 DMA_PERIPH*_SEL，选择外设；

第三步，配置中断开关；

第四步，选择对应 CH，配置源端地址和目标端地址；

第五步，配置对应 CH 的 CONTROL；

第六步，配置对应 CH 的 CONFIG；

最后，根据中断响应进行更新地址等操作；

7.2 DMA 寄存器列表

7.2.1 DMA 可屏蔽通道的总中断寄存器 (DMA_INTSTA)

偏移地址：0x00，复位值：0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_STA					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_STA	可被屏蔽的各 CH 总中断； Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.2 DMA 传输中断完成寄存器 (DMA_INTTCSTA)

偏移地址：0x04，复位值：0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_TC_STA					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_TC_STA	可被屏蔽的各 CH 传输完成中断； Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.3 DMA 中断清除寄存器 (DMA_INCTTCCLR)

偏移地址：0x08，复位值：0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_TC_CLR					
r	r	r	r	r	r	r	r	r	r	w	w	w	w	w	w

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_TC_CLR	写 1 可将对应的 INT_TC_STA 和 INT_TC_RAW_STA 清零; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.4 DMA 可屏蔽错误中断寄存器 (DMA_INTERRSTA)

偏移地址: 0x0C, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_ERR_STA					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_ERR_STA	可被屏蔽的各 CH 发生错误中断; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.5 DMA 错误中断清除寄存器 (DMA_INTERRCLR)

偏移地址: 0x10, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_ERR_CLR					
r	r	r	r	r	r	r	r	r	r	w	w	w	w	w	w

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_ERR_CLR	写 1 可将对应的 INT_ERR_STA 和 INT_ERR_RAW_STA 清零; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.6 DMA 通道不可屏蔽中断寄存器 (DMA_RAWINTTCSTA)

偏移地址: 0x14, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_TC_RAW_STA					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_TC_RAW_STA	不可被屏蔽的各 CH 传输完成中断; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.7 DMA 不可屏蔽错误中断寄存器 (DMA_RAWINTERRSTA)

偏移地址: 0x18, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										INT_ERR_RAW_STA					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	INT_ERR_RAW_STA	不可被屏蔽的各 CH 发生错误中断; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.8 DMA 通道工作状态寄存器 (DMA_ENBLDCHNSTA)

偏移地址: 0x1C, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										CH_WORK					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	CH_WORK	各个 CH 的工作状态, 在对应 CONFIG, bit0 写 1 后拉高, 传输结束后自动拉低; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.9 DMA 单次数据搬移寄存器 (DMA_SOFTSREQ)

偏移地址: 0x24, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										SOFT_SREQ					
r	r	r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	SOFT_SREQ	通过软件写 1 可以启动一次单次数据搬移; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.10 DMA 过载寄存器 (DMA_OVSTA)

偏移地址: 0x28, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										OVER_RUN_STA					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	OVER_RUN_STA	该 CH 发生了一次过载 (搬移未完成时收到新的 REQ) 后拉高; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.11 DMA 溢出清除寄存器 (DMA_OVCLR)

偏移地址: 0x2C, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										OVER_RUN_CLR					
r	r	r	r	r	r	r	r	r	r	w	w	w	w	w	w

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	OVER_RUN_CLR	写 1 将对应 OVER_RUN_STA 清零; Bit5:CH5; Bit4:CH4; ... Bit0:CH0;	0x0	rw

7.2.12 DMA 配置寄存器 (DMA_CONFIG)

偏移地址: 0x30, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														RELEASE_PMU	DMA_EN
r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw	rw

Bit	Field	Description	Reset	RW
31:2	Reserved		0x0	rw
1	RELEASE_PMU	PMU 在进入低功耗模式前会先确认 DMA 数据搬移结束；该位写 1 后 PMU 不进行 DMA 状态检查；	0x0	rw
0	DMA_EN	DMA 总使能	0x0	rw

7.2.13 DMA 外设选择寄存器 (DMA_PERIPHx_SEL) (x=0/1../5)

偏移地址: 0x34-0x48, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										PERIPH_SELx					
r	r	r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:6	Reserved		0x0	rw
5:0	PERIPH_SELx	6 个外设握手接口的选择配置; X=0/1../5	0x0	rw

7.2.14 通道源地址寄存器 (DMA_CHx_SRCADDR) (x=0/1../5)

偏移地址: 0x100/0x120/0x140/0x160/0x180/0x1A0, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRC_ADDR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SRC_ADDR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:0	SRC_ADDR	该 CH 搬移数据的源地址	0x0	rw

7.2.15 通道目标地址寄存器 (DMA_CHx_DESTADDR) (x=0/1../5)

偏移地址: 0x104/0x124/0x144/0x164/0x184/0x1A4, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DEST_ADDR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DEST_ADDR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:0	DEST_ADDR	该 CH 搬移数据的目标地址	0x0	rw

7.2.16 通道控制寄存器 (DMA_CHx_CONTROL) (x=0/1/..5)

偏移地址: 0x10C/0x12C/0x14C/0x16C/0x18C/0x1AC, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				Dinc	sinc	brken	conti	Reserved							
r	rw	rw	rw	rw	rw	rw	rw	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				Transfer size											
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:28	Reserved		0x0	rw
27	Dinc	0:目标地址固定不变; 1:目标地址递增	0x0	rw
26	Sinc	0:源地址固定不变; 1:源地址递增	0x0	rw
25	Brken	1: 在连续模式下, 发生过载时立刻停止工作	0x0	rw
24	Conti	连续模式开关, 0: 所有数据传输完后结束; 1: 一轮数据传输完后重新开始;	0x0	rw
23:12	Reserved		0x0	rw
11:0	TS	写: 需要传输的总字数, 写 0x1 表示要传输 1 个字 (4 字节); 读: 实际完成传输总字数, 读 0x1 表示已完成 1 个字 (4 字节);	0x0	rw

7.2.17 通道配置寄存器 (DMA_CHx_CONFIG) (x=0/1/..5)

偏移地址: 0x110/0x130/0x150/0x170/0x190/0x1B0, 复位值: 0x0000 C000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Conti_num								Conti_clr	Reserved	destconti	srcconti	Reserved	halt	active	Reserved
r	r	r	r	r	r	r	r	rw	r	rw	rw	r	rw	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Tcint_mask	Errint_mask	mode			Reserved	Dest_periph				Reserved	Src_periph				Ch_En
rw	rw	rw	rw	rw	r	rw	rw	rw	rw	r	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:24	Conti_cnt	连续模式循环次数	0x0	rw
23	Conti_clr	写 1 可以对 conti_cnt 清零	0x0	rw
22	Reserved		0x0	rw
21	destconti	0: 连续模式下, 每次重新开始时目标端地址也恢复原始值; 1: 目标端地址继续累加;	0x0	rw
20	srcconti	0: 连续模式下, 每次重新开始时源端地址也恢复原始值; 1: 源端地址继续累加;	0x0	rw
19	Reserved		0x0	rw
18	Halt	当前一次数据搬移完成后暂停接收数据搬移请求	0x0	rw
17	Active	=1 表示正在进行一次数据搬移	0x0	rw
16	Reserved		0x0	rw
15	Tc_int_mask	1:屏蔽 TC_INT	0x0	rw
14	Err_int_mask	1: 屏蔽 ERR_INT	0x0	rw

Bit	Field	Description	Reset	RW
13:11	Mode	模式选择; 0: 从 SRAM 往 SRAM 搬数据; 1: 从 SRAM 往外设搬数据; 2: 从外设往 SRAM 搬数据; 3: 从外设往外设搬数据;	0x0	rw
10	Reserved		0x0	rw
9:6	Dest_periph	目标端外设选择; 0: 根据 periph0_sel 选择; 1: 根据 periph1_sel 选择; .. 5: 根据 periph5_sel 选择;	0x0	rw
5	Reserved		0x0	rw
4:1	src_periph	源端外设选择; 0: 根据 periph0_sel 选择; 1: 根据 periph1_sel 选择; .. 5: 根据 periph5_sel 选择;	0x0	rw
0	Ch_En	0: Disable; 1: Enable;	0x0	rw

8 调试支持 (DBGMCU)

8.1 调试支持 (DBGMCU)

8.1.1 简介

芯片使用 Cortex™-M0+内核，该内核内含硬件调试模块，支持复杂的调试操作。硬件调试模块允许内核在取指(指令断点)或访问数据(数据断点)时停止。内核停止时，内核的内部状态和系统的外部状态都是可以查询的。完成查询后，内核和外设可以被复原，程序将继续执行。

当芯片连接到调试器并开始调试时，调试器将使用内核的硬件调试模块进行调试操作。

该芯片支持串行调试接口 (SW, Serial Wire)。

8.1.2 SW 调试端口

芯片只支持 SW 调试，不支持 JTAG 调试。

SW-DP (Serial Wire Debug Port) 引脚如下表所示：

表8.1 SW-DP 引脚分配

SW 调试接口			引脚分配
名称	类型	调试功能	
SWDIO	I/O	串行数据输入/输出	PA13
SWCLK	I	串行时钟	PA14

系统复位以后，SWDIO 和 SWCLK 被初始化为可被调试器使用的专用引脚(注意，并没有初始化跟踪输出脚，除非调试器对此脚进行定义)。可以用复用重映射和调试 I/O 配置寄存器 (GPRA_AFRH[27:20])寄存器来修改 PA13/14 去其他功能。

8.1.3 ID 代码和锁定机制

芯片内含一个 MCU ID 编码，它是 DBGMCU 的一个组成部分，并且映射在 PPB 总线上，用 SW 调试口或通过用户代码都可以访问此编码。MCU 处于系统复位状态下这个编码也可以被访问。

只支持 32bit 访问，只读数值为 0x1000095A，不可写。

8.2 寄存器描述

8.2.1 调试 ID 寄存器 (DBGMCU_ID)

偏移地址：0x00，复位值：0x1000 095A。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ID[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:0	ID	产品 ID	0x1000 095A	r

8.2.2 调试控制寄存器 (DBGMCU_CTL)

偏移地址: 0x40, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												DBGCTL[19:16]			
r	r	r	r	r	r	r	r	r	r	r	r	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DBGCTL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:20	Reserved		0x0	r
19	Stop_lptim	1: 进入调试模式时 lptim 停止工作;	0x0	rw
18	Reserved		0x0	rw
17	Stop_tim17	1: 进入调试模式时 tim17 停止工作;	0x0	rw
16	Stop_tim16	1: 进入调试模式时 tim16 停止工作;	0x0	rw
15	Stop_tim15	1: 进入调试模式时 tim15 停止工作;	0x0	rw
14	Stop_tim14	1: 进入调试模式时 tim14 停止工作;	0x0	rw
13	Stop_tim7	1: 进入调试模式时 tim7 停止工作;	0x0	rw
12	Stop_tim6	1: 进入调试模式时 tim6 停止工作;	0x0	rw
11	Stop_tim3	1: 进入调试模式时 tim3 停止工作;	0x0	rw
10	Stop_tim1	1: 进入调试模式时 tim1 停止工作;	0x0	rw
9	Stop_awktim	1: 进入调试模式时 awktim 停止工作;	0x0	rw
8	Stop_beep	1: 进入调试模式时 beep 停止工作;	0x0	rw
7	Stop_i2c2	1: 进入调试模式时 i2c2 停止工作;	0x0	rw
6	Stop_i2c1	1: 进入调试模式时 i2c1 停止工作;	0x0	rw
5	Stop_iwdg	1: 进入调试模式时 iwdg 停止工作;	0x0	rw
4	Stop_wwdg	1: 进入调试模式时 wwdg 停止工作;	0x0	rw
3	Stop_rtc	1: 进入调试模式时 rtc 停止工作;	0x0	rw
2	EN_STANDBY	1:进入 standby 模式后依然可以调试;	0x0	rw
1	EN_STOP	1:进入 stop 模式后依然可以调试;	0x0	rw
0	EN_SLEEP	1:进入 sleep 模式后依然可以调试;	0x0	rw

8.2.3 调试预留寄存器 (DBGMCU_RESERVED)

偏移地址: 0x80, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															SCLKAO
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw

Bit	Field	Description	Reset	RW
31:1	Reserved		0x0	r
0	SCLK_AO	1:CPU SCLK 常开	0x0	rw

8.2.4 调试开关寄存器 (DBGMCU_DBGEN)

偏移地址: 0xC0, 复位值: 0x0000 0001。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															DBGEN
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw

Bit	Field	Description	Reset	RW
31:1	Reserved		0x0	r
0	DBGEN	0:关闭调试开关; 修改该位时 bit31-16 需要写 0x5A69;	0x1	rw

第二部分 IO 配置

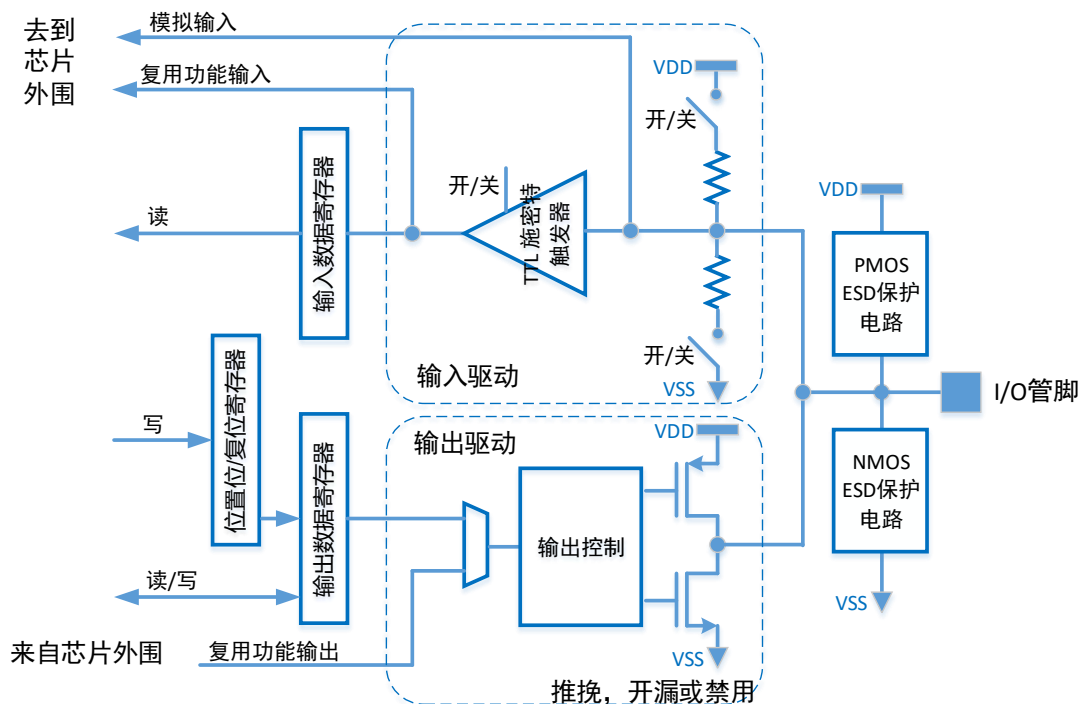
9 通用输入输出 (GPIO)

9.1 IO 功能 (IO)

9.1.1 功能描述

下图给出了一个 I/O 端口位的基本结构。

图9.1 IO 端口位的基本结构



每个通用 I/O (GPIO) 端口都有多个独立的寄存器进行功能配置：

- MODER，主模式选择：输入、输出、复用、模拟；
- OTYPE，开漏模式开关；
- OSPEEDR，输出速度选择；
- SR，输出驱动选择；
- PUPDR，上拉、下拉电阻选择；
- IDR，输入信号读取；
- ODR，输出信号配置；
- BSRR，单个端口输出值写 0 或写 1 修改；
- BRR，单个端口值写 0 修改；
- LCKR，锁定配置；
- AFRL、AFRH，复用功能配置；

在复位期间以及复位之后，所有 IO 端口寄存器全部复位为默认值 0 (PA13/14 除外)。

复位后，根据需求配置各个 IO 的主模式、开漏开关、输出能力、上下拉开关等。

配置完成后可以通过 LCKR 进行寄存器锁定。

9.1.1.1 输入/输出模式

如果主模式为输出模式，可以通过 ODR、BSRR、BRR 来修改输出值，他们的区别为：

- ODR 是多个 IO 的输出值一起修改，所以修改单个 IO 输出值时不推荐使用。
- BSRR 可以将指定的 1 个或多个 IO 输出值同时修改为 0 或 1。
- BRR 可以将指定的 1 个或多个 IO 输出值同时修改为 0。
- 每组 IO 寄存器有 2 个地址映射空间，区别为响应速度，FASTIO 地址段的延迟较小，所以对于高速的输入读取或输出修改推荐使用 FASTIO 地址段。

9.1.1.2 复用模式

主模式配置为复用模式后，可以通过每个 IO 对应的 AFRL、AFRH 寄存器选择对应功能：

	复用模式							
	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
PC13					TIM1_BKIN			
PC14						BEEP		
PC15					TIM15_BKIN	BEEP		
PD00	I2C2_SCL	USART2_TX D	LPUART_TX D	I2C1_SCL	TIM14_CH1	LPTIM_TOG		
PD01	I2C2_SDA	USART2_RX D	LPUART_RX D	I2C1_SDA	TIM15_CH1 N	LPTIM_TOG N		
PA00	SPI2_NSS	USART1_CT S	USART2_CT S					
PA01	SPI2_SCK	USART1_RT S	USART2_RT S		TIM15_CH1 N			EVENTOU T
PA02	SPI2_MIS O	USART1_TX D	USART2_TX D		TIM15_CH1			
PA03	SPI2_MO SI	USART1_RX D	USART2_RX D		TIM15_CH2			
PA04	SPI1_NSS	USART1_CK	USART2_CK		TIM14_CH1	LPTIM_EXT	VC1_OUT	
PA05	SPI1_SCK					TIM3_ETR	LVD_OUT	
PA06	SPI1_MIS O			TIM16_CH 1	TIM1_BKIN	TIM3_CH1	VC1_OUT	EVENTOU T
PA07	SPI1_MO SI			TIM17_CH 1	TIM1_CH1N	TIM3_CH2	TIM14_CH1	EVENTOU T
PB00		LPUART_TX D			TIM1_CH2N	TIM3_CH3		EVENTOU T
PB01		LPUART_RX D		TIM14_CH 1	TIM1_CH3N	TIM3_CH4	LPTIM_GAT E	
PB02				TIM15_CH 1	TIM1_BKIN	BEEP	LVD_OUT	
PB10	SPI2_SCK		I2C2_SCL	I2C1_SCL		LPTIM_TOG		
PB11	I2S_MCLK		I2C2_SDA	I2C1_SDA		LPTIM_TOG N		EVENTOU T
PB12	SPI2_NSS	SPI1_NSS			TIM1_BKIN	TIM15_CH1		EVENTOU T
PB13	SPI2_SCK	SPI1_SCK		I2C2_SCL	TIM1_CH1N			
PB14	SPI2_MIS O	SPI1_MISO		I2C2_SDA	TIM1_CH2N	TIM15_CH1		

	复用模式							
	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
PB15	SPI2_MOSI	SPI1_MOSI			TIM1_CH3N	TIM15_CH2	TIM15_CH1N	
PA08	VC0_OUT		USART1_CK		TIM1_CH1		MCO	EVENTOUT
PA09		LPUART_TXD	USART1_TXD	I2C1_SCL	TIM1_CH2	TIM15_BKIN	MCO	
PA10		LPUART_RXD	USART1_RXD	I2C1_SDA	TIM1_CH3	TIM17_BKIN		
PA11		I2C1_SCL	USART1_CTS	I2C2_SCL	TIM1_CH4	TIM15_CH1		EVENTOUT
PA12		I2C1_SDA	USART1_RTS	I2C2_SDA	TIM1_ETR	TIM15_CH2		EVENTOUT
PA13	SWDIO	IR_OUT		VC1_OUT	TIM3_ETR	BEEP	VC0_OUT	
PD06			I2C2_SCL	I2C1_SCL				
PD07			I2C2_SDA	I2C1_SDA				
PA14	SWCLK	USART1_TXD	USART2_TXD				LVD_OUT	
PA15	SPI1_NSS	USART1_RXD	USART2_RXD			LPTIM_TOG	BEEP	EVENTOUT
PB03	SPI1_SCK	USART2_TXD	LPUART_TXD			LPTIM_TON	VC0_OUT	EVENTOUT
PB04	SPI1_MISO	USART2_RXD	LPUART_RXD		TIM17_BKIN	TIM3_CH1	VC1_OUT	EVENTOUT
PB05	SPI2_MOSI		I2C1_SMB	I2C1_SDA	TIM16_BKIN	TIM3_CH2	IR_OUT	
PB06		USART1_TXD	LPUART_TXD	I2C1_SCL	TIM16_CH1N		LVD_OUT	
PB07		USART1_RXD	LPUART_RXD	I2C1_SDA	TIM17_CH1N		BEEP	
BOOT0								
PB08		USART1_TXD	USART2_TXD	I2C1_SCL	TIM16_CH1		VC1_OUT	
PB09	SPI2_NSS	USART1_RXD	USART2_RXD	I2C1_SDA	TIM17_CH1		IR_OUT	EVENTOUT

9.1.1.3 模拟模式

将对应 IO 配置成模拟模式后可以实现以下功能：

模拟模式						
PC13	WKUP2 RTC_TAMP1 RTC_OUT					
PC14		OSC32_IN				
PC15		OSC32_OUT				
PD00		OSC_IN				
PD01		OSC_OUT				
PA00	WKUP1	AIN0	VC0_P0	VC0_N0	VC1_P0	

	模拟模式					
PA01		AIN1	VC0_P1	VC0_N1	VC1_P1	
PA02	WKUP4	AIN2	VC0_P2	VC0_N2	VC1_P2	
PA03		AIN3	VC0_P3	VC0_N3	VC1_P3	
PA04		AIN4	VC0_P4	VC0_N4	VC1_P4	
PA05		AIN5	VC0_P5	VC0_N5	VC1_P5	VC1_N0
PA06		AIN6	VC0_P6	VC0_N6	VC1_P6	VC1_N1
PA07		AIN7	VC0_P7	VC0_N7	VC1_P7	VC1_N2
PB00		AIN8/Ref	VC0_P8	VC0_N8	VC1_P8	VC1_N3
PB01		AIN9	VC0_P9	VC0_N9	VC1_P9	VC1_N4
PB02						
PB10						
PB11						
PB12						
PB13						
PB14						
PB15	WKUP7					
PA08		AIN10	VC0_P10	VC0_N10	VC1_P10	VC1_N5
PA09		AIN11	VC0_P11		VC1_P11	VC1_N6
PA10		AIN12	VC0_P12		VC1_P12	VC1_N7
PA11		AIN13	VC0_P13		VC1_P13	VC1_N8
PA12		AIN14	VC0_P14		VC1_P14	VC1_N9
PA13		AIN15	VC0_P15		VC1_P15	VC1_N10
PD06						
PD07						
PA14						
PA15						
PB03						
PB04						
PB05						
PB06						
PB07						
BOOT0						
PB08						
PB09						

9.2 寄存器描述

9.2.1 主模式配置寄存器 (GPRx_MODER) (x=A/B/C/D)

偏移地址: 0x00, 复位值: 0x0000 0000(GPRA 是 0x2800 0000)。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15		MODE14		MODE13		MODE12		MODE11		MODE10		MODE9		MODE8	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7		MODE6		MODE5		MODE4		MODE3		MODE2		MODE1		MODE0	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:30	MODE15	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
29:28	MODE14	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
27:26	MODE13	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
25:24	MODE12	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
23:22	MODE11	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
21:20	MODE10	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
19:18	MODE9	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
17:16	MODE8	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw

Bit	Field	Description	Reset	RW
15:14	MODE7	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
13:12	MODE6	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
11:10	MODE5	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
9:8	MODE4	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
7:6	MODE3	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
5:4	MODE2	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
3:2	MODE1	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw
1:0	MODE0	IOy 的主模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 输入 01: 输出 10: 复用 11: 模拟	0x0	rw

9.2.2 推挽/开漏模式配置寄存器 (GPRx_OTYPER) (x=A/B/C/D)

偏移地址: 0x04, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	rw

Bit	Field	Description	Reset	RW
15	OT15	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
14	OT14	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
13	OT13	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
12	OT12	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
11	OT11	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
10	OT10	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
9	OT9	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
8	OT8	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
7	OT7	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
6	OT6	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
5	OT5	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
4	OT4	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
3	OT3	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
2	OT2	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
1	OT1	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw
0	OT0	IOy 的推挽/开漏模式配置, 可被 LCK[y] 锁定, y=0/1/.../15: 0: 推挽模式; 1: 开漏模式;	0x0	rw

9.2.3 输出速度配置寄存器 (GPRx_OSPEEDER) (x=A/B/C/D)

偏移地址: 0x08, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OS15	OS14	OS13	OS12	OS11	OS10	OS9	OS8	OS7	OS6	OS5	OS4	OS3	OS2	OS1	OS0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	rw
15	OS15	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
14	OS14	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
13	OS13	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
12	OS12	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
11	OS11	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
10	OS10	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
9	OS9	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
8	OS8	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
7	OS7	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
6	OS6	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
5	OS5	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
4	OS4	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
3	OS3	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
2	OS2	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw
1	OS1	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw

Bit	Field	Description	Reset	RW
0	OS0	IOy 的输出速度模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低速模式; 1: 高速模式;	0x0	rw

9.2.4 上拉下拉配置寄存器 (GPRx_PUPDER) (x=A/B/C/D)

偏移地址: 0x0C, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUD15		PUD14		PUD13		PUD12		PUD11		PUD10		PUD9		PUD8	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUD7		PUD6		PUD5		PUD4		PUD3		PUD2		PUD1		PUD0	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:30	PUPD15	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
29:28	PUPD14	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
27:26	PUPD13	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
25:24	PUPD12	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
23:22	PUPD11	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
21:20	PUPD10	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
19:18	PUPD9	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
17:16	PUPD8	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw

Bit	Field	Description	Reset	RW
15:14	PUPD7	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
13:12	PUPD6	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
11:10	PUPD5	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
9:8	PUPD4	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
7:6	PUPD3	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
5:4	PUPD2	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
3:2	PUPD1	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw
1:0	PUPD0	IOy 的上拉下拉模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 00: 没有上拉下拉 01: 上拉 10: 下拉 11: 没有上拉下拉	0x0	rw

9.2.5 输入观测寄存器 (GPRx_IDR) (x=A/B/C/D)

偏移地址: 0x10, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	rw
15	IDR15	IOy 的输入观测值	0x0	rw
14	IDR14	IOy 的输入观测值	0x0	rw
13	IDR13	IOy 的输入观测值	0x0	rw
12	IDR12	IOy 的输入观测值	0x0	rw
11	IDR11	IOy 的输入观测值	0x0	rw
10	IDR10	IOy 的输入观测值	0x0	rw
9	IDR9	IOy 的输入观测值	0x0	rw
8	IDR8	IOy 的输入观测值	0x0	rw
7	IDR7	IOy 的输入观测值	0x0	rw
6	IDR6	IOy 的输入观测值	0x0	rw
5	IDR5	IOy 的输入观测值	0x0	rw
4	IDR4	IOy 的输入观测值	0x0	rw
3	IDR3	IOy 的输入观测值	0x0	rw
2	IDR2	IOy 的输入观测值	0x0	rw
1	IDR1	IOy 的输入观测值	0x0	rw
0	IDR0	IOy 的输入观测值	0x0	rw

9.2.6 输出配置寄存器 (GPRx_ODR) (x=A/B/C/D)

偏移地址: 0x14, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	rw
15	ODR15	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
14	ODR14	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
13	ODR13	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
12	ODR12	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw

Bit	Field	Description	Reset	RW
11	ODR11	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
10	ODR10	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
9	ODR9	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
8	ODR8	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
7	ODR7	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
6	ODR6	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
5	ODR5	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
4	ODR4	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
3	ODR3	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
2	ODR2	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
1	ODR1	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw
0	ODR0	IOy 的输出配置值, 不可被 LCK[y]锁定, y=0/1/.../15: 0: 输出 0; 1: 输出 1;	0x0	rw

9.2.7 输出逻辑位配置 0/1 寄存器 (GPRx_BSRR) (x=A/B/C/D)

偏移地址: 0x18, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IO15C	IO14C	IO13C	IO12C	IO11C	IO10C	IO9C	IO8C	IO7C	IO6C	IO5C	IO4C	IO3C	IO2C	IO1C	IO0C
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IO15S	IO14S	IO13S	IO12S	IO11S	IO10S	IO9S	IO8S	IO7S	IO6S	IO5S	IO4S	IO3S	IO2S	IO1S	IO0S
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bit	Field	Description	Reset	RW
31	IOC15	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15; IOSy 优先级高, 当 IOCy 和 IOSy 同时写 1 时配置值改为 1;	0x0	rw
30	IOC14	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15; IOSy 优先级高, 当 IOCy 和 IOSy 同时写 1 时配置值改为 1;	0x0	rw

Bit	Field	Description	Reset	RW
29	IOC13	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
28	IOC12	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
27	IOC11	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
26	IOC10	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
25	IOC9	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
24	IOC8	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
23	IOC7	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
22	IOC6	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
21	IOC5	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
20	IOC4	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
19	IOC3	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
18	IOC2	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
17	IOC1	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
16	IOC0	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15； IOSy 优先级高，当 IOCy 和 IOSy 同时写 1 时配置值改为 1；	0x0	rw
15	IOS15	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
14	IOS14	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
13	IOS13	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
12	IOS12	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
11	IOS11	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改； 不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw

Bit	Field	Description	Reset	RW
10	IOS10	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
9	IOS9	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
8	IOS8	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
7	IOS7	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
6	IOS6	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
5	IOS5	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
4	IOS4	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
3	IOS3	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
2	IOS2	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
1	IOS1	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
0	IOS0	写 1 则将 IOy 的输出配置值改为 1，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw

9.2.8 锁定配置寄存器 (GPRx_LCKR) (x=A/B/C/D)

偏移地址：0x1C，复位值：0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LOCK
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:17	Reserved	保留	0x0	rw
16	LOCK	LCK0-LCK15 本身的锁定控制；该位依次写入 1->0->1 后 LCK0-LCK15 的值将被完全锁定，直到复位后才能重新写入；	0x0	rw
15	LCK15	IOy 的锁定配置，y=0/1/.../15； 0: IOy 的相关配置寄存器都未被锁定； 1: IOy 的相关配置寄存器都被锁定；	0x0	rw
14	LCK14	IOy 的锁定配置，y=0/1/.../15； 0: IOy 的相关配置寄存器都未被锁定； 1: IOy 的相关配置寄存器都被锁定；	0x0	rw
13	LCK13	IOy 的锁定配置，y=0/1/.../15； 0: IOy 的相关配置寄存器都未被锁定； 1: IOy 的相关配置寄存器都被锁定；	0x0	rw
12	LCK12	IOy 的锁定配置，y=0/1/.../15； 0: IOy 的相关配置寄存器都未被锁定； 1: IOy 的相关配置寄存器都被锁定；	0x0	rw
11	LCK11	IOy 的锁定配置，y=0/1/.../15； 0: IOy 的相关配置寄存器都未被锁定； 1: IOy 的相关配置寄存器都被锁定；	0x0	rw

Bit	Field	Description	Reset	RW
10	LCK10	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
9	LCK9	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
8	LCK8	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
7	LCK7	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
6	LCK6	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
5	LCK5	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
4	LCK4	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
3	LCK3	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
2	LCK2	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
1	LCK1	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw
0	LCK0	IOy 的锁定配置, $y=0/1/.../15$: 0: IOy 的相关配置寄存器都被未锁定; 1: IOy 的相关配置寄存器都被锁定;	0x0	rw

9.2.9 复用功能配置寄存器低 (GPRx_AFRl) ($x=A/B/C/D$)

偏移地址: 0x20, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7				AFR6				AFR5				AFR4			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	Rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3				AFR2				AFR1				AFR0			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:28	AFR7	IOy 的复用功能配置, 会被 LCK[y]锁定, $y=0/1/.../7$; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
27:24	AFR6	IOy 的复用功能配置, 会被 LCK[y]锁定, $y=0/1/.../7$; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw

Bit	Field	Description	Reset	RW
23:20	AFR5	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
19:16	AFR4	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
15:12	AFR3	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
11:8	AFR2	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
7:4	AFR1	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
3:0	AFR0	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw

9.2.10 复用功能配置寄存器高 (GPRx_AFRH) (x=A/B/C/D)

偏移地址: 0x24, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR15				AFR14				AFR13				AFR12			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	Rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR11				AFR10				AFR9				AFR8			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:28	AFR15	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
27:24	AFR14	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
23:20	AFR13	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw

Bit	Field	Description	Reset	RW
19:16	AFR12	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
15:12	AFR11	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
11:8	AFR10	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
7:4	AFR9	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw
3:0	AFR8	IOy 的复用功能配置, 会被 LCK[y]锁定, y=0/1/.../7; 0: 选择 AF0 功能; 1: 选择 AF1 功能; ... 7: 选择 AF7 功能;	0x0	rw

9.2.11 输出逻辑位配置 0 寄存器 (GPRx_BRR) (x=A/B/C/D)

偏移地址: 0x28, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IO15C	IO14C	IO13C	IO12C	IO11C	IO10C	IO9C	IO8C	IO7C	IO6C	IO5C	IO4C	IO3C	IO2C	IO1C	IO0C
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	rw
15	IOC15	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
14	IOC14	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
13	IOC13	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
12	IOC12	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
11	IOC11	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
10	IOC10	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
9	IOC9	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw
8	IOC8	写 1 则将 IOy 的输出配置值改为 0, 写 0 则不修改; 不被 LCK[y]锁定, y=0/1/.../15;	0x0	rw

Bit	Field	Description	Reset	RW
7	IOC7	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
6	IOC6	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
5	IOC5	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
4	IOC4	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
3	IOC3	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
2	IOC2	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
1	IOC1	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw
0	IOC0	写 1 则将 IOy 的输出配置值改为 0，写 0 则不修改；不被 LCK[y]锁定，y=0/1/.../15；	0x0	rw

9.2.12 输出驱动配置寄存器 (GPRx_SR) (x=A/B/C/D)

偏移地址: 0x2C, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SR15	SR14	SR13	SR12	SR11	SR10	SR9	SR8	SR7	SR6	SR5	SR4	SR3	SR2	SR1	SR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	rw
15	SR15	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw
14	SR14	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw
13	SR13	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw
12	SR12	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw
11	SR11	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw
10	SR10	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw
9	SR9	IOy 的输出驱动模式配置，可被 LCK[y]锁定，y=0/1/.../15： 0: 低驱模式； 1: 高驱模式；	0x0	rw

Bit	Field	Description	Reset	RW
8	SR8	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
7	SR7	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
6	SR6	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
5	SR5	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
4	SR4	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
3	SR3	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
2	SR2	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
1	SR1	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw
0	SR0	IOy 的输出驱动模式配置, 可被 LCK[y]锁定, y=0/1/.../15: 0: 低驱模式; 1: 高驱模式;	0x0	rw

第三部分 控制系统

10 系统配置 (SYSCFG)

10.1 系统配置 (SYSCFG)

10.1.1 功能描述

该模块主要包含了外部中断控制配置、红外输出调制配置以及部分系统相关配置。

10.1.1.1 外部中断复用控制逻辑

外部中断电路可以从所有的 IO 中选择最多 16 个进行检测并根据结果触发中断或者事件。

例如：EXTI0 从 PA0、PB0、PC0、PD0 中选择一个，EXTI1 从 PA1、PB1、PC1、PD1 中选择一个等。

10.1.1.2 红外输出调制

当用两个 UART 的 TX 信号或者 TIM3 的 CH1/2 输出红外信号时，可以内部将其与 TIM3 或 TIM14/15 的输出进行逻辑与/或/异或操作从而实现调制功能。

10.1.1.3 内部刹车信号生成

各个 TIMER 除了外部刹车信号外，还可以使用内部刹车信号，包括 LVD 比较异常，SRAM 奇偶校验错误和 CPU 锁定。

10.2 复用寄存器

10.2.1 系统配置寄存器 1 (SYSCFG_CFGR1)

偏移地址：0x00，复位值：0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															Lockup_rsten
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													remap		
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:17	Reserved		0x0	r
16	Lockup_rsten	1:当 M0+发生锁定时进行系统复位;	0x0	rw
15:3	Reserved		0x0	r
2:0	remap	启动模式，上电后硬件自动根据 boot0/boot1 检测生成; 1: 闪存主存储区; 2: 闪存系统存储区; 4: 内嵌 SRAM 区;	0x0	r

10.2.2 系统配置寄存器 2 (SYSCFG_CFGR2)

偏移地址：0x04，复位值：0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								Master_chk_en				Reserved		Breaken	
r	r	r	r	r	r	r	r	rw	rw	rw	rw	r	rw	rw	rw

Bit	Field	Description	Reset	RW
31:8	Reserved		0x0	r
7:4	Master_chk_en	DMA 访问拦截配置, 各个 bit 写 1 后不可写 0; Bit2=1, DMA 不可访问 FLASH; Bit1=1, DMA 不可访问 SRAM 控制寄存器; Bit0=1, DMA 不可访问 SRAM;	0x0	rw
3	Reserved		0x0	r
2:0	breaken	内置刹车信号生成选择, 各个 bit 写 1 后不可写 0; Bit2=1, 当 LVD 输出高时进行刹车; Bit1=1, 当 SRAM 奇偶校验错误时刹车; Bit0=1, 当 M0+锁定时刹车;	0x0	rw

10.2.3 外部中断配置寄存器 1 (SYSCFG_EXTICR1)

偏移地址: 0x08, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3_SEL				EXTI2_SEL				EXTI1_SEL				EXTI0_SEL			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved		0x0	r
15:12	EXTI3_SEL	0: 选择 PA3; 1: 选择 PB3; 2: 选择 PC3; 3: 选择 PD3; 其他: 固定 0;	0x0	rw
11:8	EXTI2_SEL	0: 选择 PA2; 1: 选择 PB2; 2: 选择 PC2; 3: 选择 PD2; 其他: 固定 0;	0x0	rw
7:4	EXTI1_SEL	0: 选择 PA1; 1: 选择 PB1; 2: 选择 PC1; 3: 选择 PD1; 其他: 固定 0;	0x0	rw
3:0	EXTI0_SEL	0: 选择 PA0; 1: 选择 PB0; 2: 选择 PC0; 3: 选择 PD0; 其他: 固定 0;	0x0	rw

10.2.4 外部中断配置寄存器 2 (SYSCFG_EXTICR2)

偏移地址: 0x0C, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI7_SEL				EXTI6_SEL				EXTI5_SEL				EXTI4_SEL			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved		0x0	r
15:12	EXTI7_SEL	0: 选择 PA7; 1: 选择 PB7; 2: 选择 PC7; 3: 选择 PD7; 其他: 固定 0;	0x0	rw
11:8	EXTI6_SEL	0: 选择 PA6; 1: 选择 PB6; 2: 选择 PC6; 3: 选择 PD6; 其他: 固定 0;	0x0	rw
7:4	EXTI5_SEL	0: 选择 PA5; 1: 选择 PB5; 2: 选择 PC5; 3: 选择 PD5; 其他: 固定 0;	0x0	rw
3:0	EXTI4_SEL	0: 选择 PA4; 1: 选择 PB4; 2: 选择 PC4; 3: 选择 PD4; 其他: 固定 0;	0x0	rw

10.2.5 外部中断配置寄存器 3 (SYSCFG_EXTICR3)

偏移地址: 0x10, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI11_SEL				EXTI10_SEL				EXTI9_SEL				EXTI8_SEL			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved		0x0	r
15:12	EXTI11_SEL	0: 选择 PA11; 1: 选择 PB11; 2: 选择 PC11; 3: 选择 PD11; 其他: 固定 0;	0x0	rw
11:8	EXTI10_SEL	0: 选择 PA10; 1: 选择 PB10; 2: 选择 PC10; 3: 选择 PD10; 其他: 固定 0;	0x0	rw

Bit	Field	Description	Reset	RW
7:4	EXTI9_SEL	0: 选择 PA9; 1: 选择 PB9; 2: 选择 PC9; 3: 选择 PD9; 其他: 固定 0;	0x0	rw
3:0	EXTI8_SEL	0: 选择 PA8; 1: 选择 PB8; 2: 选择 PC8; 3: 选择 PD8; 其他: 固定 0;	0x0	rw

10.2.6 外部中断配置寄存器 4 (SYSCFG_EXTICR4)

偏移地址: 0x14, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15_SEL				EXTI14_SEL				EXTI13_SEL				EXTI12_SEL			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:16	Reserved		0x0	r
15:12	EXTI15_SEL	0: 选择 PA15; 1: 选择 PB15; 2: 选择 PC15; 3: 选择 PD15; 其他: 固定 0;	0x0	rw
11:8	EXTI14_SEL	0: 选择 PA14; 1: 选择 PB14; 2: 选择 PC14; 3: 选择 PD14; 其他: 固定 0;	0x0	rw
7:4	EXTI13_SEL	0: 选择 PA13; 1: 选择 PB13; 2: 选择 PC13; 3: 选择 PD13; 其他: 固定 0;	0x0	rw
3:0	EXTI12_SEL	0: 选择 PA12; 1: 选择 PB12; 2: 选择 PC12; 3: 选择 PD12; 其他: 固定 0;	0x0	rw

10.2.7 红外输出调制配置寄存器 (SYSCFG_IRCR)

偏移地址: 0x18, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						OUT_SEL		Reserved		CAR_INV	SIG_INV	CAR_SEL		SIG_SEL	
r	r	r	r	r	r	rw	rw	r	r	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:10	Reserved		0x0	r
9:8	OUT_SEL	IR 调制模式选择; 0: 与操作; 1: 或操作; 2: 与操作后取反; 3: 或操作后取反;	0x0	rw
7:6	Reserved		0x0	r
5	CAR_INV	1: 载波取反;	0x0	rw
4	SIG_INV	1: 红外信号取反;	0x0	rw
3:2	CAR_SEL	载波选择; 0: TIM3_CH1 1: TIM3_CH2 2: TIM14_CH1 3: TIM15_CH1	0x0	rw
1:0	SIG_SEL	红外信号选择; 0: USART1 1: USART2 2: TIM3_CH1 3: TIM3_CH2	0x0	rw

11 独立看门狗 (IWDG)

11.1 独立看门狗 (IWDG)

11.1.1 简介

系统内置两个看门狗，包括独立看门狗 (IWDG) 和窗口看门狗 (WWDG)，可用于检测 and 解决由软件错误引起的故障；当计数器达到给定的超时值时，触发一个中断(仅适用于窗口型看门狗)或产生系统复位。本章节介绍独立看门狗 (IWDG)。

独立看门狗 (IWDG) 由内部低速时钟 (LSI) 驱动，即使主时钟发生故障它也仍然有效。窗口看门狗 (WWDG) 时钟由 APB1 时钟预分频得到，并有一个可配置的时间窗口，通过可配置的时间窗口来检测应用程序非正常的过迟或过早的操作。

IWDG 最适合应用于那些需要看门狗作为一个在主程序之外、能够完全独立工作、对时间精度要求较低的场合。WWDG 最适合那些要求看门狗在精确计时窗口起作用的应用程序。

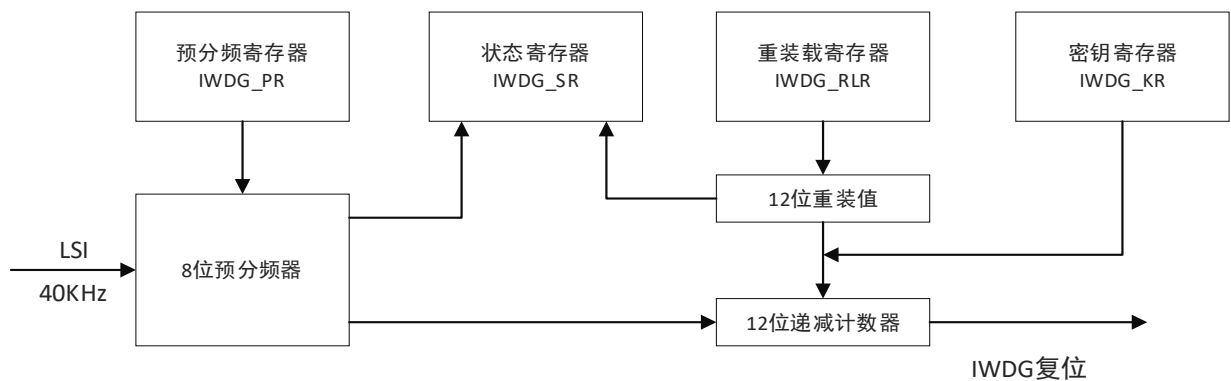
11.1.2 主要性能

- 自由运行的递减计数器
- 由独立的内部低速 RC 振荡器 (LSI) 提供时钟 (可在待机和停止模式下运行)
- 看门狗被激活后。递减计数器值达到 0x000 时复位

11.1.3 功能描述

独立看门狗的功能框图如下图所示。

图11.1 独立看门狗框图



在密钥寄存器 IWDG_KR 中写入 0xCCCC，开始启用独立看门狗，一旦启动无法关闭；此时计数器开始从其复位值 0xFFFF 递减计数。当计数器计数到末尾 0x000 时，会产生一个复位信号 (IWDG_RESET)。

无论何时，只要在密钥寄存器 IWDG_KR 中写入 0xAAAA，IWDG_PR 和 IWDG_RLR 中的值就会被重新加载到计数器，从而避免产生看门狗复位

11.1.3.1 硬件看门狗

如果用户在选项字节中启用了“硬件看门狗”功能，在系统上电复位后，看门狗会自动开始运行；如果在计数器计数结束前，软件没有向密钥寄存器写入相应的值，则系统会产生复位。

11.1.3.2 寄存器访问保护

IWDG_PR 和 IWDG_RLR 寄存器具有写保护功能。要修改这两个寄存器的值，必须先向 IWDG_KR 寄存器中写入 0x5555。不同的值写入这个寄存器将会打乱操作顺序，寄存器将重新被

保护。重装载操作(即写入 0xAAAA)也会启动写保护功能。状态寄存器 IWDG_SR 指示预分频值和递减计数器是否正在被更新。

11.1.3.3 常用操作

- 设置看门狗预分频系数和重装载值
 - 首先, IWDG_KR 寄存器中写入 0x5555, 使能 IWDG_PR 和 IWDG_RLR 写操作。
 - 然后, 配置 IWDG_PR 和 IWDG_RLR 的值。
- 实现定时喂狗以及重新加载 IWDG_RLR 到看门狗计数器里面, IWDG_KR 写入 0xAAAA,。
- 启动看门狗, IWDG_KR 写入 0xCCCC
 - IWDG 在一旦启用, 就不能再被关闭, 除非重启。

根据 IWDG_PR 和 IWDG_RLR 计算看门狗的喂狗时间 (也就是看门狗溢出时间), 计算方式为:

$$T_{out} = ((4 \times 2^{prer}) \times rlr) / 40$$

其中 T_{out} 为看门狗溢出时间 (单位为 ms); prer 为看门狗时钟预分频值 (IWDG_PR 值), 范围为 0~7; (注意 7 的预分频系数与 6 的预分频系数相同, 都是 256)

表11.1 独立看门狗定时时间 (LSI 40kHz)

预分频系数	PR[2:0]	最短时间 (ms) RL[11:0]=0x000	最长时间 (ms) RL[11:0]= 0xFFFF
4	0	0.1	409.6
8	1	0.2	819.2
16	2	0.4	1638.4
32	3	0.8	3276.8
64	4	1.6	6553.6
128	5	3.2	13107.2
256	6 或 7	6.4	26214.4

11.1.3.4 调试模式

当微控制器进入调试模式 (Cortex®-M0+内核停止) 时, 取决于 DBG 模块中的 DBG_IWDG_STOP 配置位, IWDG 计数器将继续正常工作或停止。

11.2 寄存器

11.2.1 密钥寄存器 (IWDG_KR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY															
wo															

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved		0x0	w	

Bit	Field	Description	Reset value	RW attribute	Remark
15:0	KEY	密钥值（只写，读 0000h） 软件必须以一定的间隔写入 0xAAAA，否则，当计数器为 0 时，看门狗会产生复位。 写入密钥值 0x5555h 以启用对 IWDG_PR 和 IWDG_RLR 寄存器的访问。 写入键值 0xCCCCh 启动看门狗（选择了硬件看门狗不受此命令限制）。	0x0	w	

11.2.2 预分寄存器 (IWDG_PR)

偏移地址：0x04，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													PR		
r													rw		

Bit	Field	Description	Reset value	RW attribute	Remark
31:3	Reserved		0x0	rw	
2:0	PR	预分频器 这些位受写访问保护。通过设置这些位来选择计数器时钟的预分频因子。要改变预分频因子，IWDG_SR 寄存器的 PVU 位必须为 0。 000: 4 分频 001: 8 分频 010: 16 分频 011: 32 分频 100: 64 分频 101: 128 分频 110: 256 分频 111: 256 分频	0x0	rw	

11.2.3 重装载寄存器 (IWDG_RLR)

偏移地址：0x08，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RL															
rw															

Bit	Field	Description	Reset value	RW attribute	Remark
31:12	Reserved		0x0	rw	
11:0	RL	看门狗计数器重装载值 这些位受写访问保护。用于定义看门狗计数器的重装载值，每当向 IWDG_KR 寄存器写入 0xAAAA 时，重装载值会被传送到计数器中。看门狗计数器从此值递减计数。超时周期是该值和时钟预分频器计算出来的。IWDG_SR 寄存器中的 RVU 为 0 才能更改重装载值。	0x0	rw	

11.2.4 状态寄存器 (IWDG_SR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														RVU	PVU
r														ro	ro

Bit	Field	Description	Reset	RW	Remark
31:2	Reserved		0x0	rw	
1	RVU	看门狗计数器重载值更新 该位由硬件置 1 用来指示重新加载值的更新正在进行中。当重载值更新操作完成时, 此位由硬件清 0 (最多需要 7 个 RC 40 kHz 周期)。只有当 RVU 位为 0 时才能更新重载值。	0x0	ro	
0	PVU	看门狗预分频器值更新 该位由硬件置 1 用来指示预分频器值的更新正在进行中。当预分频器更新操作完成时, 此位由硬件清 0 (最多需要 7 个 RC 40 kHz 周期)。只有当 PVU 位为 0 时, 预分频器值才能更新。	0x0	ro	

12 窗口看门狗 (WWDG)

12.1 窗口看门狗 (WWDG)

12.1.1 WWDG 简介

窗口看门狗用于检测软件故障的发生，通常由外部干扰或无法预料的逻辑条件引起，导致应用程序背离其正常的运行顺序而产生的软件故障。看门狗电路在达到预置的时间周期时会生成 MCU 复位，除非递减计数器在 T6 位清零之前被刷新。如果在递减计数器达到窗口寄存器值之前刷新（控制寄存器中的）7 位递减计数器值，也会产生 MCU 复位，这意味着必须在有限的时间窗口中刷新计数器。

12.1.2 WWDG 主要特性

- 可编程的自由运行递减计数器
- 条件复位（看门狗被启动）：
 - 当递减计数器的值小于 0x40，则产生复位
 - 当递减计数器在窗口外被重新装载，则产生复位
- 如果启动了看门狗并且被允许中断，当递减计数器等于 0x40 时产生早期唤醒中断 (EWI)，它可以被用于重装载计数器以避免 WWDG 复位。

12.1.3 WWDG 功能描述

如果看门狗被启动（WWDG_CR 寄存器中的 WDGA 位被置 1），并且当 7 位（T[6:0]）递减计数器从 0x40 翻转到 0x3f（T6 位清零）时，则产生一个复位。如果软件在计数器值大于窗口寄存器中的数值时重新装载计数器，将产生一个复位。

应用程序在运行过程中必须定期地写入 WWDG_CR 寄存器以防止 MCU 发生复位。只有当计数器值小于窗口寄存器的值时，才能进行写操作。储存在 WWDG_CR 寄存器中的数值必须在 0xFF 和 0xC0 之间。

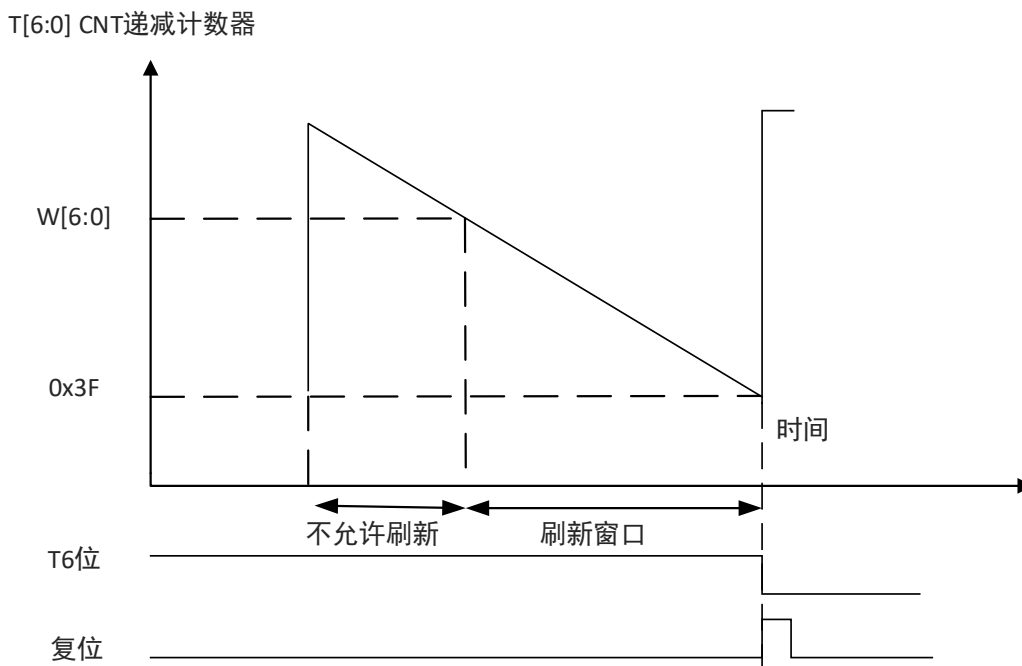
- 启动看门狗
 - 在系统复位后，看门狗总是处于关闭状态，设置 WWDG_CR 寄存器的 WDGA 位能够开启看门狗，随后它不能再被关闭，除非发生复位
- 控制递减计数器

递减计数器处于自由运行状态，即使看门狗被禁止，递减计数器仍继续递减计数。当看门狗被启用时，T6 位必须被设置为 1，以防止立即产生一个复位。T[5:0]位包含了看门狗产生复位之前的计时数目；复位前的延时时间在一个最小值和一个最大值之间变化，这是因为写入 WWDG_CR 寄存器时，预设频值是未知的。配置寄存器(WWDG_CFR) 中包含窗口的上限值：要避免产生复位，递减计数器必须在其值小于窗口寄存器的数值并且大于 0x3F 时被重新装载。另一个重装载计数器的方法是利用早期唤醒中断(EWI)。设置 WWDG_CFR 寄存器中的 WEI 位开启该中断。当递减计数器到达 0x40 时，则产生此中断，相应的中断服务程序(ISR)可以用来加载计数器以防止 WWDG 复位。在 WWDG_SR 寄存器中写 0 可以清除该中断。

- 早期唤醒中断 (EWI)
 - 首先开启中断使能信号，WWDG_CFG 的位 9 置为 1，当计数器值递减为 0x40，产生中断。
 - 如果开启了看门狗且允许中断，当递减计数器等于 0x40 时产生早期唤醒中断 (EWI)，它可以被用于重装载计数器以避免 WWDG 复位。

12.1.4 编写超时程序

图12.1 窗口看门狗时序图



下面是用中断的方式来喂狗的。采取以下四个步骤如下：

- 使能 WWDG 时钟
 - WWDG 不同于 IWDG，IWDG 使用 LIRC，开始 IWDG 之后自动开启 LIRC。WWDG 使用的是 PCLK1，需要在 APB1ENR 中使能 WWDG 时钟。
- 设置 WWDG_CFR 和 WWDG_CR 两个寄存器
 - 包括使能窗口看门狗、开启中断、设置计数器的初始值、设置窗口值以及设置分频数 WDGTB 等。
- 开启 WWDG 中断并分组
 - 在设置完了 WWDG 后，需要配置该中断的分组及使能。这点通过之前所编写的 NVIC 函数实现可以实现。
- 编写中断服务函数
 - 最后要编写窗口看门狗的中断服务函数，通过该函数来喂狗，喂狗要快，否则当窗口看门狗计数器值减到 0X3F 的时，就会引起软复位。在中断服务函数里面也要将状态寄存器的 EWIF 位清空。

上窗口值（W[6:0]）是由用户设定，根据实际要求来设计窗口值，但是一定要确保窗口值大于 0x40，否则窗口就不存在了。窗口看门狗的超时公式如下：

$$T_{\text{wwdg}} = T_{\text{wwdg}} = T_{\text{pclk1}}/4096 \times 2^{\text{WDGTB}} \times (T[5:0] + 1); \quad (\text{ms})$$

其中：

T_{wwdg} : WWDG 超时时间（单位为 ms）

T_{pclk1} : APB1 的时钟周期（单位为 ms）

WDGTB: WWDG 的预分频系数

T[5:0]: 窗口看门狗的计数器低 6 位

根据上面的公式，假设 $F_{\text{pclk1}}=36\text{Mhz}$ ，那么可以得到最小-最大超时时间表如下表所示：

表12.1 36MHz 时钟下窗口看门狗的最小最大超时表

WDGTB	最小超时值	最大超时值
0	113us	7.28ms

WDGTB	最小超时值	最大超时值
1	227us	14.56ms
2	455us	29.12ms
3	910us	58.25ms

12.1.5 工作模式

12.1.5.1 复位操作

- 配置预分频器: WWDG_CFR: WDG TB [1: 0]
- 配置 7 位窗口值: WWDG_CFR: W [6: 0]
- 配置 7 位初始计数器: WWDG_CR: T [6: 0]
- 等待 RST_GEN 触发并报告结果
-

12.1.5.2 中断操作

- 配置预分频器: WWDG_CFR: WDG TB [1: 0]
- 配置 7 位窗口值: WWDG_CFR: W [6: 0]
- 配置 7 位初始计数器: WWDG_CR: T [6: 0]
- 等待中断[EWI]触发并报告结果

12.1.5.3 正常操作

- 配置分频器: WWDG_CFR: WDG TB [1: 0]
- 配置 7 位窗口值: WWDG_CFR: W [6: 0]
- 配置 7 位初始计数器: WWDG_CR: T [6: 0]
- 定期喂狗
- 不会触发 RST_GEN 和 EW I 并报告结果

12.2 寄存器

12.2.1 寄存器 (WWDG_CR)

偏移地址: 0x00, 复位值: 0x0000 007F。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								WDGA	T						
r								rs	rw						

Bit	Field	Description	Reset value	RW attribute	Remark
31:8	Reserved	必须保持在复位值。			
7	WDGA	看门狗激活位。 该位由软件设置以启动看门狗, 仅在复位后由硬件清零。 当 WDGA = 1 时, 看门狗可以产生复位。 0: 看门狗禁用 1: 看门狗使能	0x0	rs	

Bit	Field	Description	Reset value	RW attribute	Remark
6:0	T	7 位计数器。 这些位包含看门狗计数器的值。它每 (4096×2^{WDGTB}) 个 PCLK1 周期递减。当它从 0x40 翻转到 0x3F (T6 被清零) 时会产生一个复位。	0x7F	rw	

12.2.2 寄存器 (WWDG_CFR)

偏移地址: 0x04, 复位值: 0x0000 007F。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						EWI		WDGTB		W					
r						rw		rw		rw					

Bit	Field	Description	Reset value	RW attribute	Remark
31:10	Reserved	必须保持在复位值。			
9	EWI	提前唤醒中断。 此位置 1 后, 只要计数器达到值 0x40, 就会发生中断。 该中断仅在复位后由硬件清除。	0x0	rw	
8:7	WDGTB	计时器基准。预分频器的时基可以根据如下修改: 00: CK 计数器时钟(PCLK1 div 4096) div 1 01: CK 计数器时钟(PCLK1 div 4096) div 2 10: CK 计数器时钟(PCLK1 div 4096) div 4 11: CK 计数器时钟(PCLK1 div 4096) div 8	0x0	rw	
6:0	W	7 位窗口值 这些位包含要与向下计数器进行比较用的窗口值。	0x7F	rw	

注: EWI 是提前唤醒中断, 在产生复位前一段时间即 T[6:0]=0x40 时产生中断, 提醒系统喂狗, 否则将复位。在中断里面向 WWDG_CR 重新写入计数器的值, 来达到喂狗的目的。注意这里在进入中断后, 必须在不大于 1 个窗口看门狗计数周期的时间 (在 PCLK1 频率为 36M 且 WDDTB 为 0 的条件下, 该时间为 113us) 内重新写 WWDG_CR, 否则, 看门狗将产生复位。

12.2.3 寄存器 (WWDG_SR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															EWIF
r															rcw0

Bit	Field	Description	Reset value	RW attribute	Remark
31:10	Reserved	必须保持在复位值。			
0	EWIF	提前唤醒中断标志 当计数器达到值 0x40 时, 该位由硬件设置。它必须由软件通过写入 0 来清零。写入 1 无效。如果未启用中断, 也会设置该位。	0x0	Rcw0	

该寄存器用来记录当前是否有提前唤醒的标志, 当计数器值达到 0x40 时, 此位由硬件置 1。它必须通过软件写 0 来清除。对此位写 1 无效。即使中断未被使能, 在计数器的值达到 0x40 的时候, 此位也会被置 1。

13 循环冗余校验计算单元 (CRC)

13.1 循环冗余校验计算单元 (CRC)

13.1.1 CRC 功能描述

CRC 单元主要包括一个 32 位的数据寄存器：

- 用作输入寄存器以在 CRC 计算器中输入新数据（写入寄存器时）
- 保留上一次 CRC 计算的结果（读取寄存器时）
- 有四种计算模式可选

对数据寄存器的每次写操作都会创建先前 CRC 值和新 CRC 值的组合（CRC 计算是在整个 32 位数据字上完成的，而不是每个字节一个字节）。

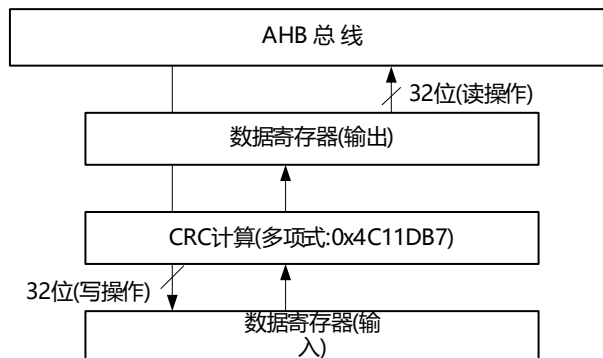
在 CRC 计算期间会暂停 CPU 的写操作，因此可以对寄存器 CRC_DR 进行背靠背写入或者连续地写-读操作。

可以通过设置寄存器 CRC_CR 的 RST 位来重置寄存器 CRC_DR 为 CRC_INIT。该操作不影响寄存器 CRC_IDR 内的数据。

13.1.2 CRC 主要特性

- 使用 CRC-32（以太网）多项式：0x4C11DB7
 - $X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$
- 一个 32 位数据寄存器用于输入/输出。
- CRC 计算时间：4 个 AHB 时钟周期（HCLK）

下图为 CRC32 计算单元框图，其他 3 种 CRC 计算模式原理类似，只是公式不同。



13.2 CRC 寄存器

13.2.1 CRC 数据寄存器 (CRC_DR)

偏移地址：0x00，复位值：0xFFFF FFFF。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DR[31:16]															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	DR	数据寄存器位： 将新数据写入 CRC 计算器时用作输入寄存器。读取时保存上一次 CRC 计算结果。	0x0	rw	

13.2.2 CRC 独立数据寄存器 (CRC_IDR)

偏移地址：0x04，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDR[31:16]															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR[15:0]															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	IDR	通用 32 位数据寄存器位。可用作 4 个字节的临时存储位置。该寄存器不受 CRC_CR 寄存器中 RST 位产生的 CRC 复位的影响，也不受其他 INV、REV 等控制。	0x0	rw	

13.2.3 CRC 控制寄存器 (CRC_CR)

偏移地址：0x08，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reserve															
ro															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								Rev_out		Rev_in	mode		Inv_out	Inv_in	RST
ro								rw		rw	rw		rw	rw	wo

Bit	Field	Description	Reset	RW	Remark
7	Rev_out	写 1：将 CRC 读出数据的每个字节内位数倒置，例如 bit7-0 换成 bit0-7；	0x0	RW	
5	Rev_in	写 1：将 CRC 写入数据的每个字节内位数倒置，例如 bit7-0 换成 bit0-7；	0x0	RW	
4:3	MODE	0: CRC32，公式：0x04C11DB7； 1: CRC16，取低 16 位，公式：0x8005； 2: CRC_CCITT，取低 16 位，公式：0x1021； 3: CRC_8，取低 8 位，公式：0x07；	0x0	RW	
2	Inv_out	写 1：将 CRC 读出数据每位取反；	0x0	RW	
1	Inv_in	写 1：将 CRC 写入数据每位取反；	0x0	RW	
0	RST	RESET 位。复位 CRC 计算单元并将数据寄存器设置为 CRC_INIT。该位只能置位，由硬件自动清零。	0x0	WO	

13.2.4 CRC 初始寄存器 (CRC_INIT)

偏移地址：0x0C，复位值：0xFFFF FFFF。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CRC_INIT[31:16]															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_INIT[15:0]															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	CRC_INIT	CRC 计算初始值	0xFFFFFFFF	RW	

14 随机数发生器 (TRNG)

14.1 随机数发生器 (TRNG)

14.1.1 功能描述

TRNG 是一个随机数生成器，它连续提供 32 位熵样本。

TRNG 提供 32 位真随机数，该真随机数是由经过线性反馈移位寄存器 (LFSR) 后处理的模拟熵源产生的。

每 42 个 TRNG 时钟周期 (专用时钟) 产生 1 个 32 位随机样本。

它允许连续的状态监测以及相关的错误管理。包括过低的采样时钟检测和重复计数测试。

可以禁用以降低功耗。

它具有 AMBA AHB 接口，只能通过 32 位字单次访问进行访问 (否则会生成 AHB 总线错误)。任何不等于 32 位的写操作都可能损坏寄存器内容。

14.2 寄存器

14.2.1 TRNG 控制寄存器 (TRNG_CR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										CED	Reserved	IE	RNGEN	Reserved	
-										RW	-	RW	RW	-	

Bit	Field	Description	Reset	RW	Bit
31:6	Reserved				
5	CED	时钟错误检测 0: 时钟错误检测使能 1: 时钟错误检测禁用 当启用 RNG 时，不能即时启用或禁用时钟错误检测，即要启用或禁用 CED，必须禁用 RNG。	0	RW	
4	Reserved				
3	IE	中断使能 0: 禁止 RNG 中断 1: RNG 中断使能。 只要 RNG_SR 寄存器中的 DRDY = '1'、SEIS = '1' 或 CEIS = '1'，中断就会挂起。	0	RW	
2	RNGEN	真随机数发生器启用 0: 真随机数发生器关闭，模拟噪声源关闭，RNG 时钟的逻辑门控。 1: 启用真随机数发生器。	0	RW	
1:0	Reserved				

14.2.2 TRNG 状态寄存器 (TRNG_SR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								SEIS	CEIS	Reserved			SECS	CECS	DRDY
-								W	W	-			R	R	R

Bit	Field	Description	Reset	RW	Bit
31:7	Reserved				
6	SEIS	种子错误中断状态 该位与 SECS 同时置位。通过将其写入 0 将其清除。 0: 未检测到错误序列 1: 至少检测到一个错误序列, 详见 SECS 位说明。 如果 RNG_CR 寄存器中的 IE = '1', 则中断正在挂起。	0	W	
5	CEIS	时钟错误中断状态 该位与 CECS 同时置位。通过将其写入 0 将其清除。 0: RNG 时钟正确 ($f_{RNGCLK} > f_{HCLK} / 16$) 1: 检测到 RNG 太慢 ($f_{RNGCLK} < f_{HCLK} / 16$) 如果 RNG_CR 寄存器中的 IE = '1', 则中断正在挂起。	0	W	
4:3	Reserved				
2	SECS	种子错误当前状态 0: 当前没有检测到故障序列 如果设置了 SEIS 位, 这意味着检测到故障序列并且情况已经恢复。 1: 噪声源之一以恒定值 ("0"或"1") 提供超过 64 个连续位, 或超过 32 个连续出现的两个位模式 ("01"或"10")	0	R	
1	CECS	时钟错误当前状态 0: RNG 时钟正确($f_{RNGCLK} > f_{HCLK} / 16$).如果 CEIS 位被设置, 这意味着检测到慢时钟并且情况已经恢复。 1: RNG 时钟太慢 ($f_{RNGCLK} < f_{HCLK} / 16$)。 注: CECS 位仅在 RNG_CR 寄存器中的 CED 位设置为"0"时有效。	0	R	
0	DRDY	数据就绪 0: RNG_DR 寄存器尚未生效, 无随机数据可用。 1: RNG_DR 寄存器包含有效的随机数据。 一旦 RNG_DR 寄存器被读取, 该位返回 0, 直到产生新的随机值。如果 RNG_CR 寄存器中的 IE = 1, 当 DRDY = 1 时产生中断。	0	R	

14.2.3 TRNG 数据寄存器 (TRNG_DR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RNDATA															
R															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RNDATA															
R															

Bit	Field	Description	Reset	RW	Bit
31:0	TRNDATA	随机数据：32 位随机数据，当 DRDY='1' 时有效，当 DRDY='0' 时 RNDATA 值为 0。	0	R	

15 AES 硬件加速器 (AES)

15.1 AES 硬件加速器

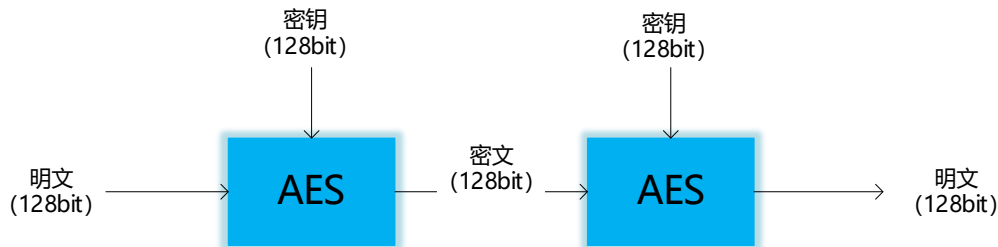
15.1.1 AES 算法

AES 的全称是高级加密标准(Advanced Encryption Standard), 又称 Rijndael 加密法, 是美国联邦政府采用的一种区块加密标准。

简单来讲, AES 的使用是发送端先用 128bit (也可以试 192bit、256bit) 的密钥对 128bit 的明文加密后得到一个同样 128bit 的密文, 接收端在收到密文后, 用同一个密钥进行解密就可以得到明文。

AES 有很多种工作模式, 主要是对于数据的分组和链接上有些区别, 这些模式有 ECB、CBC、CTR、CFB、OFB 等。

图15.1 AES 算法



15.1.2 主要特点

- 完全符合 NIST 《FIPS PUB 197》标准;
- 只支持 128bit 密钥;
- 支持密钥派生;
- 支持 ECB/CBC/CTR 链接模式;
- 支持 DMA 接口 (收发两个独立通道);
- 完成一次 AES 操作需要 220 个时钟;
- 支持错误检测
- 支持中断: 处理结束, 错误发生

15.1.3 功能描述

AES 硬件加速器主要有 4 种工作模式, 根据 MODE 寄存器选择:

- Mode1: 加密, 使用密钥寄存器的值对输入缓存中的数据进行加密, 结果保存在输出缓存中;
- Mode2: 密钥派生, 对密钥寄存器的值进行派生计算, 结果继续保存在密钥寄存器中;
- Mode3: 解密, 使用密钥寄存器的值对输入缓存中的数据进行解密, 结果保存在输出缓存中;
- Mode4: 密钥派生+解密, 对密钥寄存器的值进行派生后再使用其对输入进行解密 (CTR 模式无法使用);

AES 硬件加速器支持链接处理, 即 EN 拉高后持续在填充阶段 (此时 CCF 为低) 输入数据, 注意 EN 拉高后密钥、IV 和控制寄存器都不可更改, 有 3 种链接模式, 根据 CHMOD 寄存器选择:

- ECB: 不需要 IV, 每个块都使用固定密钥;

- CBC: 需要 IV (填充初始向量), 第一个块使用 IV, 之后每个块的计算需要使用上一个块的输出;
- CTR: 需要 IV (填充计数器初始值), 每个块的计算需要使用计数器的值, 计数器每次使用后加一;

CCF 只在计算结束后拉高, 此时数据输出缓存 (模式 1/3/4) 或者密钥寄存器 (模式 2) 中的值才有效, 设置 CCFIE=1 可以再 CCF 拉高时产生中断;

以下情况下会产生写错误:

- EN 拉高时配置 CHMOD、MODE、DATATYPE、IV、KEY;
- 计算过程中配置 CHMOD、MODE、DATATYPE;
- 输入缓存中当前块的数据未被取走计算时进行填充操作;

以下情况下会产生读错误:

- 当前块未被填充入输出缓存时进行读取操作;

密钥、输入、输出、IV 都是小端格式, 可以通过 DATATYPE 统一进行格式调整;

15.1.4 密钥派生

密钥派生时需要使用 4 个密钥寄存器, 并且派生结束后会将结果写回这 4 个寄存器;

4 个密钥寄存器中 KEYR0 是整个 128 位密钥的最低 32 位, KEYR3 是最高 32 位;

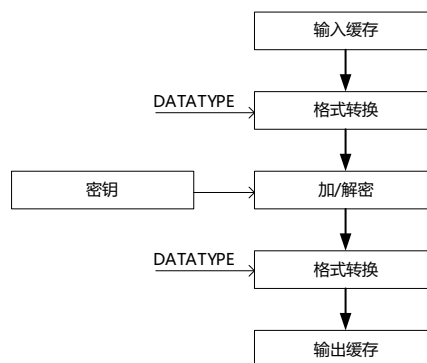
因为 EN 为低时才能配置 KEY, 所以当 MODE=2 或者 4 时将 EN 拉高后硬件就自动开始了派生计算;

可以用 MODE=2 派生完后将 EN 拉低再修改 MODE=3 开始解密, 此时不用再配置密钥, 也可以直接用 MODE=4 直接来完成这 2 个步骤。

15.1.5 AES 链接模式

15.1.5.1 ECB 模式

此模式不需要 IV, 所以实际上各个块的处理没有前后联系, 都是独立处理。



15.1.5.2 CBC 模式

此模式每个块都会用到上一个块的计算结果, 有很强的继承性, 第一个块会使用寄存器配置 IV (初始向量)。

图15.2 加密流程:

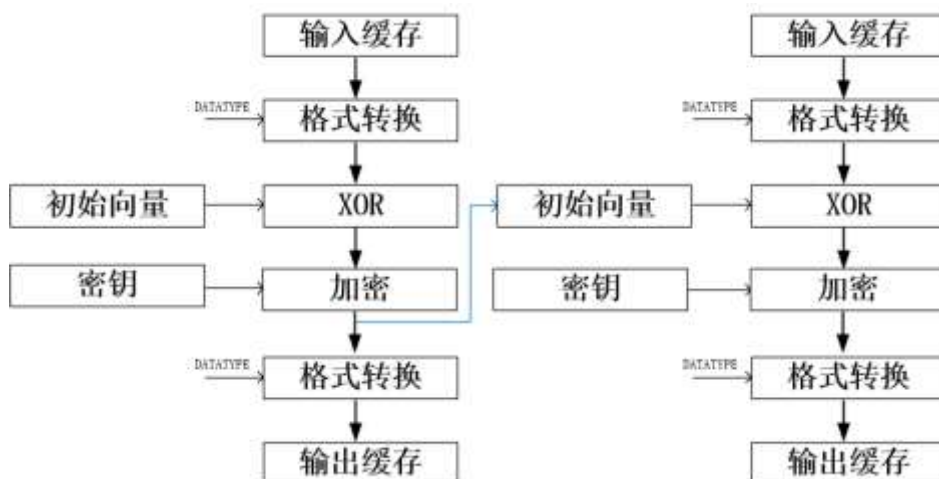
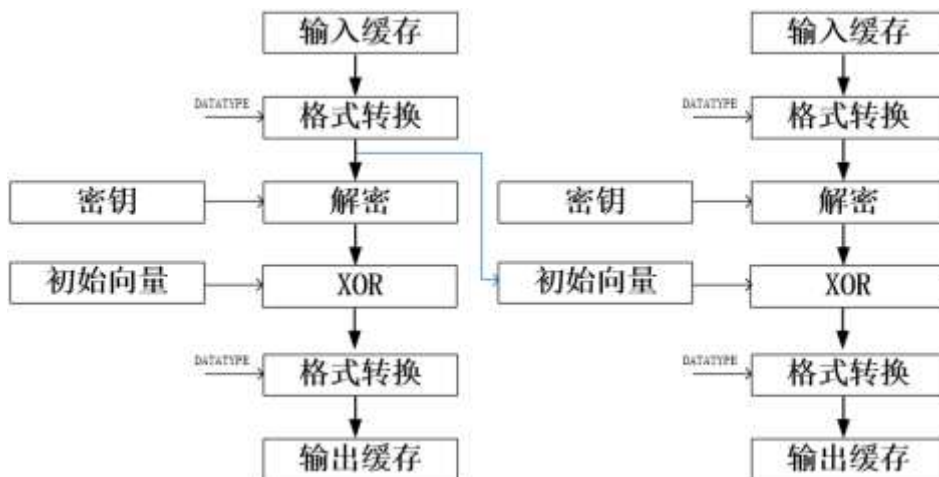
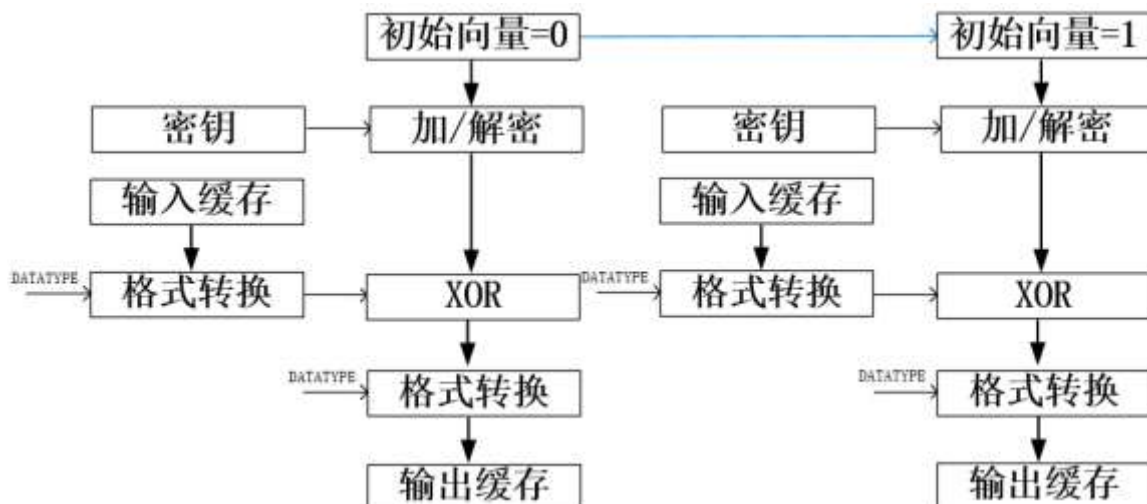


图15.3 解密流程:



15.1.5.3 CTR 模式

此模式会使用一个计数器来维持每个块的连续性。



15.1.5.4 暂停功能

在实际使用过程中，可能会有更重要的报文 B 需要用到 AES 加解密，此时可以将当前进行的报文 A 加解密流程暂停，具体流程是：

- 报文 A 第 N-1 个块结束
- 报文 A 第 N 个块结束
- 中断到来，需要处理报文 B
- 读取当前 IVR 的值 xxx，存入 SRAM
- 开始报文 B 的处理
- 报文 B 结束
- 重新开始报文 A 第 N+1 个块，IVR 从 SRAM 取回

15.1.6 数据格式转换

数据在进出缓存时都要通过 32 位的寄存器，在寄存器填充输入缓存和从输出缓存取出写入寄存器时都会有一次格式转换，通过寄存器 DATATYPE 选择：

转换前

31	30	..	24	23	22	..	16	15	14	..	8	7	6	..	0
----	----	----	----	----	----	----	----	----	----	----	---	---	---	----	---

转换后，DATATYPE=0

31	30	..	24	23	22	..	16	15	14	..	8	7	6	..	0
----	----	----	----	----	----	----	----	----	----	----	---	---	---	----	---

转换后，DATATYPE=1

15	14	..	8	7	6	..	0	31	30	..	24	23	22	..	16
----	----	----	---	---	---	----	---	----	----	----	----	----	----	----	----

转换后，DATATYPE=2

7	6	..	0	15	14	..	8	23	22	..	16	31	30	..	24
---	---	----	---	----	----	----	---	----	----	----	----	----	----	----	----

转换后，DATATYPE=3

0	1	..	7	8	9	..	15	16	17	..	23	24	25	..	31
---	---	----	---	---	---	----	----	----	----	----	----	----	----	----	----

15.1.7 使用流程

15.1.7.1 Mode1(加密)使用流程：

- 步骤 1，EN 拉低
- 步骤 2，配置 MODE=0，选择 CHMOD
- 步骤 3，配置密钥
- 步骤 4，配置 IV（CBC 和 CTR 模式）
- 步骤 5，EN 拉高
- 步骤 6，填充最高 32 位数据进入输入寄存器
- 步骤 7，继续填充 32 位数据进入输入寄存器
- 步骤 8，继续填充 32 位数据进入输入寄存器
- 步骤 9，填充最低 32 位数据进入输入寄存器
- 步骤 10，等待 CCF 拉高（中断或者轮询）

- 步骤 11, 从输出寄存器读取, 此次对应的是最高 32 位
 - 步骤 12, 继续从输出寄存器读取
 - 步骤 13, 继续从输出寄存器读取
 - 步骤 14, 从输出寄存器读取, 此次对应的是最低 32 位
- 重复步骤 6-14

15.1.7.2 Mode2(密钥派生)使用流程:

- 步骤 1, EN 拉低
- 步骤 2, 配置 MODE=1
- 步骤 3, 配置密钥
- 步骤 4, EN 拉高
- 步骤 5, 等待 CCF 拉高 (中断或者轮询)
- 步骤 6, 读取密钥

15.1.7.3 Mode3(解密)使用流程:

- 步骤 1, EN 拉低
 - 步骤 2, 配置 MODE=2, 选择 CHMOD
 - 步骤 3, 配置密钥
 - 步骤 4, 配置 IV (CBC 和 CTR 模式)
 - 步骤 5, EN 拉高
 - 步骤 6, 填充最高 32 位数据进入输入寄存器
 - 步骤 7, 继续填充 32 位数据进入输入寄存器
 - 步骤 8, 继续填充 32 位数据进入输入寄存器
 - 步骤 9, 填充最低 32 位数据进入输入寄存器
 - 步骤 10, 等待 CCF 拉高 (中断或者轮询)
 - 步骤 11, 从输出寄存器读取, 此次对应的是最高 32 位
 - 步骤 12, 继续从输出寄存器读取
 - 步骤 13, 继续从输出寄存器读取
 - 步骤 14, 从输出寄存器读取, 此次对应的是最低 32 位
- 重复步骤 6-14

15.1.7.4 Mode4(派生+解密)使用流程:

- 步骤 1, EN 拉低
- 步骤 2, 配置 MODE=3, 选择 CHMOD
- 步骤 3, 配置密钥
- 步骤 4, 配置 IV (CBC 和 CTR 模式)
- 步骤 5, EN 拉高
- 步骤 6, 填充最高 32 位数据进入输入寄存器
- 步骤 7, 继续填充 32 位数据进入输入寄存器
- 步骤 8, 继续填充 32 位数据进入输入寄存器
- 步骤 9, 填充最低 32 位数据进入输入寄存器
- 步骤 10, 等待 CCF 拉高 (中断或者轮询)
- 步骤 11, 从输出寄存器读取, 此次对应的是最高 32 位
- 步骤 12, 继续从输出寄存器读取

- 步骤 13, 继续从输出寄存器读取
 - 步骤 14, 从输出寄存器读取, 此次对应的是最低 32 位
- 重复步骤 6-14

15.1.8 DMA 接口

AES 硬件加速器包含 2 个 DMA 接口, 分别对应数据输入和输出。

输入接口工作流程:

- DMAINEN 设置为 1;
- 配置其他参数;
- EN 拉高;
- 发送请求;
- 收到数据写入输入缓存 (最高 32 位);
- 发送请求;
- 继续收到数据写入缓存;
- 发送请求;
- 继续收到数据写入缓存;
- 发送请求;
- 收到数据写入缓存 (最低 32 位);
- 停止请求, 直到此次 AES 块处理完成 (4 个 32 位输出被取走);
- 重新开始发送请求

输出接口工作流程:

- DMAOUTEN 设置为 1;
- 配置其他参数;
- EN 拉高;
- 等待 AES 计算完成, 输出缓存更新;
- 发送请求;
- 输出缓存数据 (最高 32 位) 被取走;
- 发送请求;
- 输出缓存数据继续被取走;
- 发送请求;
- 输出缓存数据继续被取走;
- 发送请求;
- 输出缓存数据 (最低 32 位) 被取走;
- 停止请求, 等待下一次输出缓存更新;
- 重新开始发送请求;

15.2 AES 寄存器

15.2.1 AES 控制寄存器 (AES_CR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			DMAOUT EN	DMAIN EN	ERR IE	CCF IE	ER RC	CC FC	CHMOD[1:0]		MODE [1:0]		DATATYPE [1:0]		EN
r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:13	Reserved		0x0	r	Reserved field
12	DMAOUTEN	启用数据输出缓存的 DMA 接口： 0: DMA (在数据输出阶段) 禁用 1: DMA (在数据输出阶段) 使能 如果设置了 DMAOUTEN 位，则在模式 1、3 或 4 中为输出数据阶段生成 DMA 请求。 该位在模式 2 (密钥派生) 中无效。	0x0	rw	
11	DMAINEN	启用数据输入缓存的 DMA 接口： 0: DMA (在数据输入阶段) 禁用 1: DMA (在数据输入阶段) 使能 如果设置了 DMAINEN 位，则在模式 1、3 或 4 中为数据输入阶段生成 DMA 请求。该位在模式 2 (密钥派生) 中不起作用。	0x0	rw	
10	ERRIE	错误中断使能 如果 RDERR 或 WRERR 两个标志中的至少一个被设置，则产生中断。 0: 错误中断禁用 1: 错误中断使能	0x0	rw	
9	CCFIE	CCF 标志中断使能 如果设置了 CCF 标志，则会产生中断。 0: CCF 中断禁止 1: CCF 中断使能	0x0	rw	
8	ERRC	错误清除 向该位写入 1 会清除 RDERR 和 WRERR 标志。 该位始终读为低电平。	0x0	rw	
7	CCFC	计算完成标志清除 向该位写入 1 会清除 CCF 标志。 该位始终读为低电平。	0x0	rw	
6:5	CHMOD	AES 链接模式 00: 电子码本 (EBC) 01: 密码块链 (CBC) 10: 计数器模式 (CTR) 11: 保留。 只能在 EN=0 时更改。	0x0	rw	
4:3	MODE	AES 工作模式 00: 模式 1: 加密 01: 模式 2: 密钥推导 10: 模式 3: 解密 11: 模式 4: 密钥推导+解密 只能在 EN=0 时更改。	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
2:1	DATATYPE	数据类型选择 (用于加密块的数据输入和数据输出) 00: 无交换。 01: 16bit 交换, 0x764356AB 转后为 0x56AB7643 10: 8bit 交换, 0x764356AB 转后为 0xAB564376。 11: 1bit 交换, 0x764356AB 转后为 0xD56AC26E 只能在 EN=0 时更改。	0x0	rw	
0	EN	AES 使能 0: AES 禁用 1: AES 使能 AES 可以随时通过重置此位重新初始化: 然后当 EN 置位时, AES 准备开始处理新块。当 AES 计算在模式 2 (密钥推导) 中完成时, 此位由硬件清零。	0x0	rw	

15.2.2 AES 状态寄存器 (AES_SR)

偏移地址: 0x04, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved[15:3]													WRERR	RDERR	CCF
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW	Remark
31:3	Reserved		0x0	r	Reserved field
2	WRERR	写入错误标志 当检测到对 AES_DINR 寄存器的意外写操作时 (在计算或数据输出阶段), 此位由硬件设置。当 AES 处于运行状态时, 尝试更改 CHMOD、MODE、DATATYPE、AES_KEY* 或 AES_IVR* 时也会设置此位即使设置了 WERR, 该标志对继续运行的 AES 没有影响。即使设置了 WERR, 该标志对继续运行的 AES 也没有影响。 它由软件通过设置 AES_CR 寄存器中的 ERRC 位来清除。 0: 未检测到写入错误 1: 检测到写错误	0x0	rw	
1	RDERR	读取错误标志 当检测到来自 AES_DOUTR 寄存器的意外读取操作时 (在计算或数据输入阶段), 此位由硬件设置。如果先前已在 AES_CR 寄存器中设置了 ERRIE 位, 则会产生中断。此标志对即使设置了 RDERR 也会继续运行的 AES。 它由软件通过设置 AES_CR 寄存器中的 ERRC 位来清除。 0: 未检测到读取错误 1: 检测到读取错误	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
0	CCF	计算完成标志 该位在计算完成时由硬件置位，如果先前在 AES_CR 寄存器中设置了 CCFIE 位，则会产生一个中断。它由软件通过设置 AES_CR 寄存器中的 CCFC 位来清除。 0: 计算完成 1: 计算不完整 注：该位仅在 DMAOUTEN = 0 时有效。当 DMAOUTEN = 1 时，它可能保持高电平。	0x0	rw	

15.2.3 AES 数据输入寄存器 (AES_DINR)

偏移地址：0x08，复位值：0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DINR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DINR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	DINR	数据输入寄存器。 该寄存器必须在输入阶段写入 4 次： 在模式 1（加密）中，必须写入 4 个字，代表从 MSB 到 LSB 的纯文本。 在模式 2（密钥派生）中，不使用该寄存器，因为该模式仅涉及从 AES_KEYRx 寄存器开始的派生密钥计算。 在模式 3（解密）和模式 4（密钥推导+解密）中，必须写 4 个字，代表密文 MSB 到 LSB。	0x0	rw	

15.2.4 AES 数据输出寄存器 (AES_OUTR)

偏移地址：0x0C，复位值：0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DOUR[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DOUR[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW	Remark
31:0	DOUR	数据输出寄存器 该寄存器是只读的。一旦设置了 CCF 标志（计算完成标志），读取该数据寄存器 4 次就可以访问 128 位输出结果： 在模式 1（加密）中，读取的 4 个字代表从 MSB 到 LSB 的密文。 在模式 2（密钥派生）中，不需要读取该寄存器，因为派生密钥位于 AES_KEYRx 寄存器中。 在模式 3（解密）和模式 4（密钥派生+解密）中，读取的 4 个字代表从 MSB 到 LSB 的纯文本。	0x0	rw	

15.2.5 AES 关键寄存器 0 (AES_KEYR0) (key[31:0])

偏移地址：0x10，复位值：0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEYR0[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEYR0[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	KEYR0	密钥寄存器 (LSB 键 [31:0])	0x0	rw	

15.2.6 AES 密钥寄存器 1 (AES_KEYR1) (LSB: key[63:32])

偏移地址: 0x14, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEYR1[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEYR1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	KEYR1	AES 密钥寄存器 (密钥 [63:32])	0x0	rw	

15.2.7 AES 密钥寄存器 2 (AES_KEYR2) (key[95:64])

偏移地址: 0x18, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEYR2[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEYR2[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	KEYR2	AES 密钥寄存器 (密钥 [95:64])	0x0	rw	

15.2.8 AES 密钥寄存器 3 (AES_KEYR3) (key[127:96])

偏移地址: 0x1C, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEYR3[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEYR3[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	KEYR3	AES 密钥寄存器 (密钥 [127:96])	0x0	rw	

15.2.9 AES 初始化向量寄存器 0 (AES_IVR0) (IVR[31:0])

偏移地址: 0x20, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IVR0[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IVR0[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	IVR0	初始化向量寄存器 (LSB IVR [31:0]) 该寄存器必须在 AES_CR 寄存器中的 EN 位被设置之前写入。	0x0	rw	

15.2.10 AES 初始化向量寄存器 1 (AES_IVR1) (IVR[63:32])

偏移地址: 0x24, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IVR0[31:16]															
rw	rw	rw	rw	rw	rw	rw	Rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IVR0[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	IVR0	初始化向量寄存器 (IVR [63:32]) 该寄存器必须在 AES_CR 寄存器中的 EN 位被设置之前写入。	0x0	rw	

15.2.11 AES 初始化向量寄存器 2 (AES_IVR2) (IVR[95:64])

偏移地址: 0x28, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IVR2[31:16]															
rw	rw	rw	rw	rw	rw	rw	Rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IVR2[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	IVR2	初始化向量寄存器 (IVR [95:64]) 该寄存器必须在 AES_CR 寄存器中的 EN 位被设置之前写入。	0x0	rw	

15.2.12 AES 初始化向量寄存器 3 (AES_IVR3) (IVR[127:96])

偏移地址: 0x2C, 复位值: 0x0000 0000。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IVR3[31:16]															
rw	rw	rw	rw	rw	rw	rw	Rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IVR3[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:0	IVR3	初始化向量寄存器 (MSB IVR [127: 96]) 该寄存器必须在 AES_CR 寄存器中的 EN 位被设置之前写入。	0x0	rw	

16 定时器 (TIMX)

16.1 定时器 (TIMX)

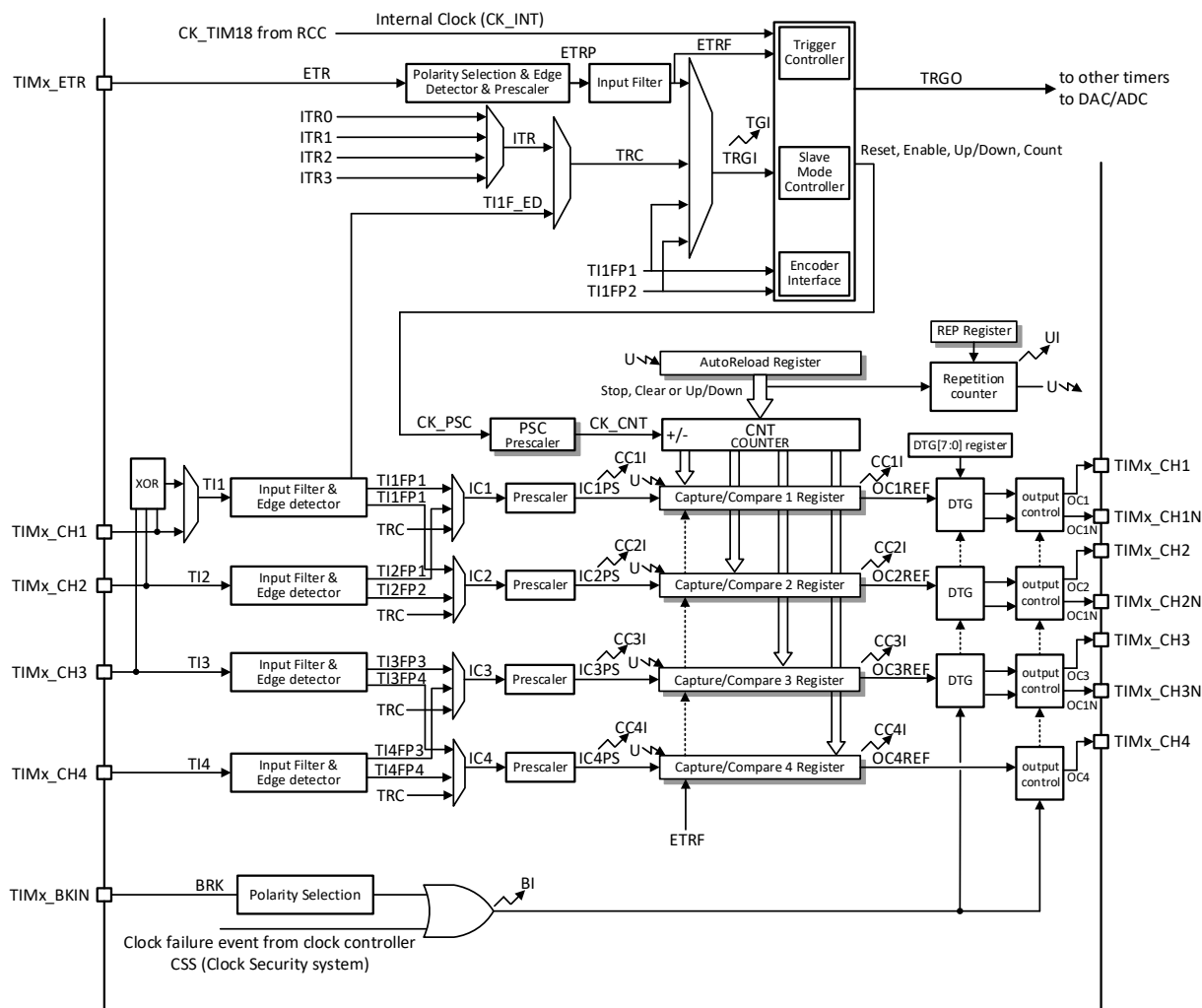
16.1.1 功能描述

计时器模块功能包括高级控制计时器 (TIM1) /通用计时器 (TIM3&TIM14 至 TIM17) /基本计时器 (TIM6&TIM7)。TIMX 包括一个由可编程预分频器驱动的 16 位自动重载计数器。它可以用于多种目的, 包括测量输入信号的脉冲长度 (输入捕获) 或生成输出波形 (输出比较, PWM, 具有死区时间插入的互补 PWM)。使用定时器预分频器和 RCMU 时钟控制器预分频器, 可以将脉冲长度和波形周期从几微秒调节到几毫秒。高级控制 (TIM1) 和通用 (TIM3&TIM14 至 TIM17) 定时器是完全独立的, 并且不共享任何资源。

TIMX 功能包括:

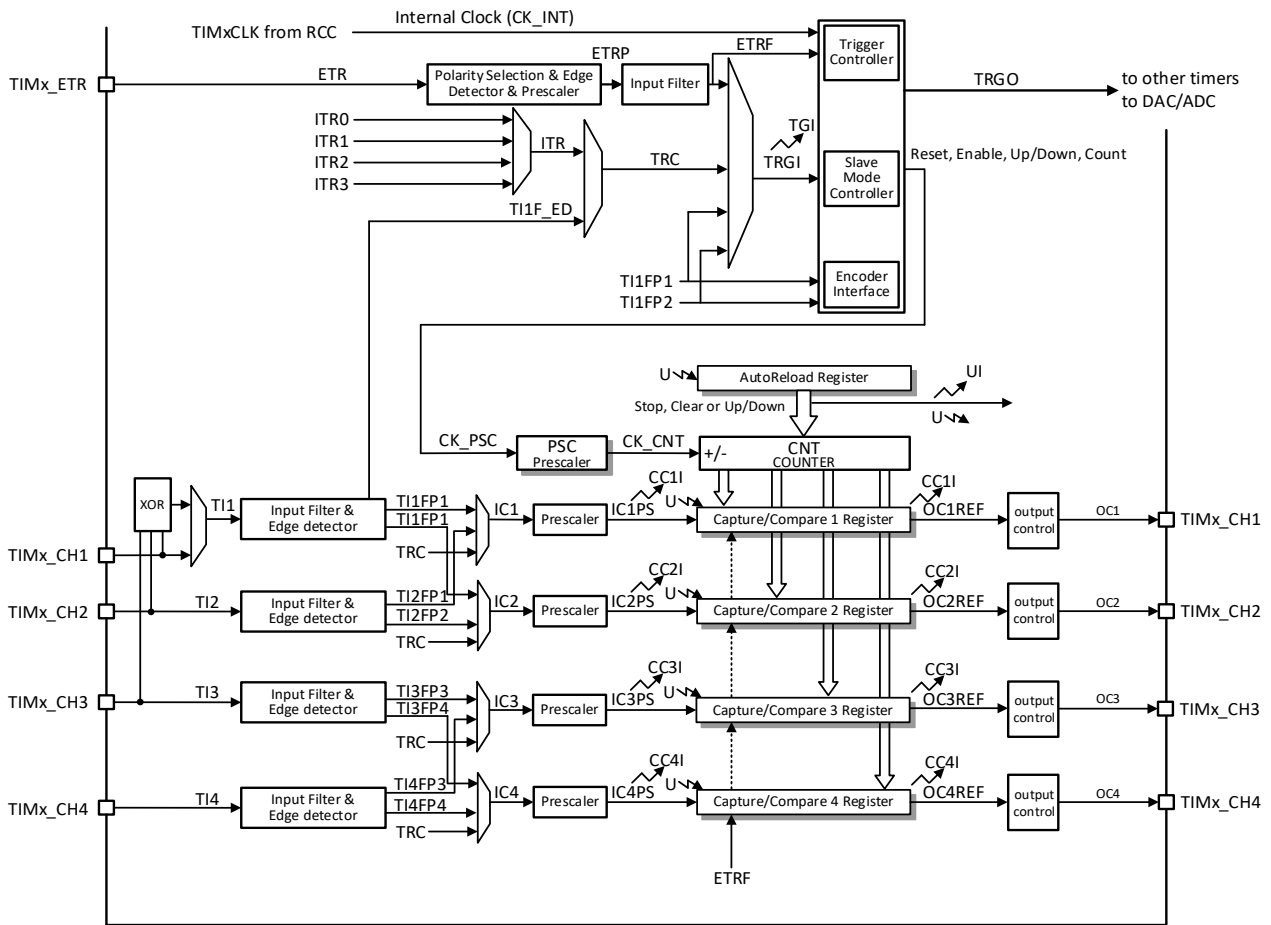
- 计时器 1:
- 32 位递增, 递减, 递增/递减自动重载计数器。
- 16 位可编程预分频器, 可将计数器时钟频率除以 1 (在运行中) 或 65535 之间的任意倍数。
- 多达 4 个独立通道, 用于:
 - 输入捕获
 - 输出比较
 - PWM 生成 (边沿和中心对齐模式)
 - 单脉冲模式输出
- 具有可编程死区时间的互补输出
- 同步电路, 用于通过外部信号控制计时器, 并将多个计时器互连在一起。
- 重复计数器仅在给定的循环次数后才更新定时器寄存器。
- 中断输入可将计时器的输出信号置于复位状态或已知状态。
- 发生以下事件时产生中断/DMA:
 - 更新: 计数器上溢/下溢, 计数器初始化 (通过软件或内部/外部触发)
 - 触发事件 (计数器的开始, 停止, 初始化或计数内部/外部触发)
 - 输入捕捉
 - 输出比较
 - 中断输入
- 支持增量 (正交) 编码器和霍尔传感器电路以进行定位
- 触发输入用于外部时钟或逐个周期电流管理

图16.1 计数器 1



- 计数器 3:
- 32 位递增, 递减, 递增/递减自动重载计数器。
- 16 位可编程预分频器, 可以将计数器时钟频率除以 1 (在运行中) 或 65535 之间的任意倍数。
- 多达 4 个独立通道, 用于:
 - 输入捕获
 - 输出比较
 - PWM 生成 (边沿和中心对齐模式)
 - 单脉冲模式输出
- 同步电路, 用于通过外部信号控制定时器, 并将多个定时器互连在一起。
- 发生以下事件时产生中断/ DMA:
 - 更新: 计数器上溢/下溢, 计数器初始化 (通过软件或内部/外部触发)
 - 触发事件 (计数器开始, 停止, 初始化或通过内部/外部触发进行计数)
 - 输入捕捉
 - 输出比较
- 触发输入用于外部时钟或逐周期电流管理

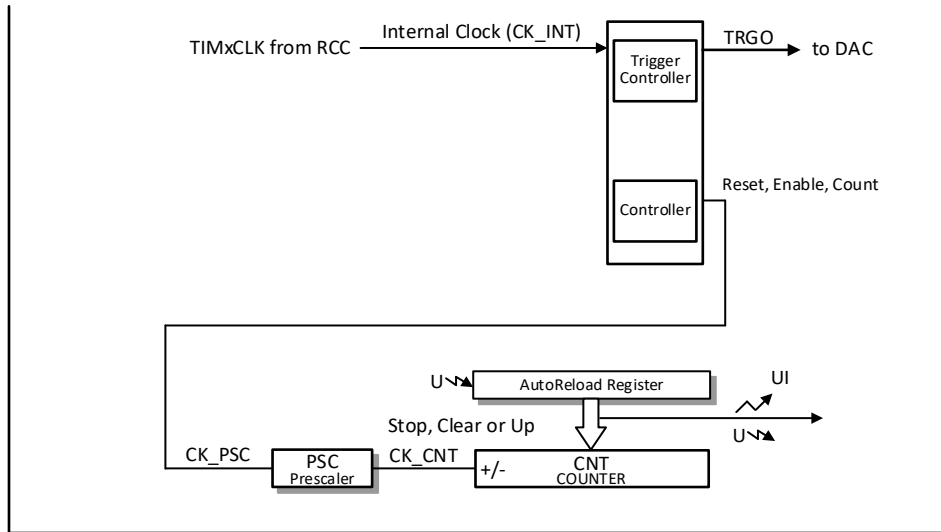
图16.2 计数器 3



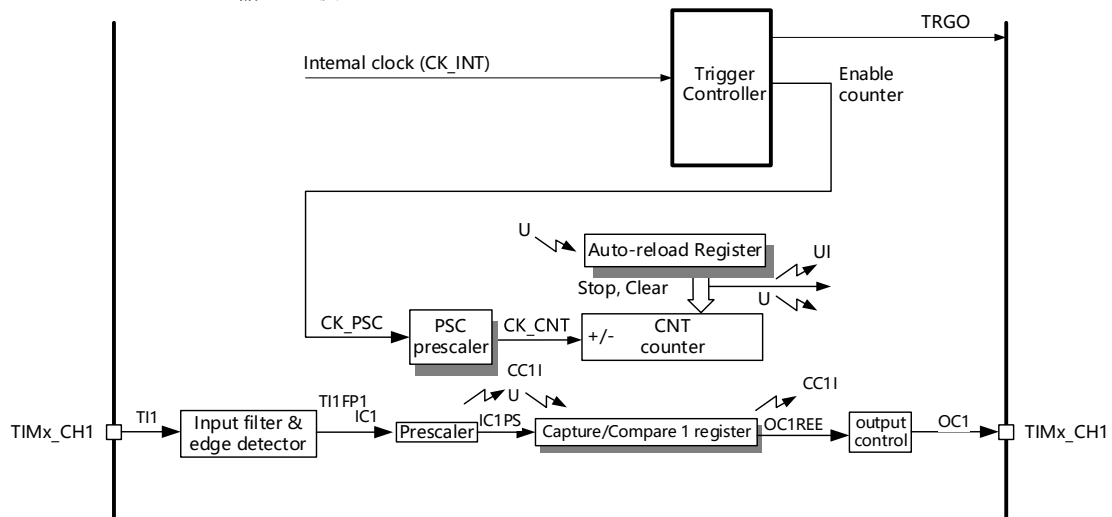
- 定时器 6/7:
- 32 位递增, 递减, 递增/递减自动重载计数器
- 16 位可编程预分频器, 可以将计数器时钟频率除以 1 (在运行中) 或 65535 之间的任意倍数
- 同步电路触发 DAC

更新事件的中断/DMA 生成: 计数器溢出

图16.3 计数器 6/7



- 计时器 14:
- 32 位递增自动重载计数器。
- 16 位可编程预分频器，可将计数器时钟频率除以 1（在运行中）或 65535 之间的任意倍数。
- 独立通道，用于：
 - 输入捕获
 - 输出比较
 - PWM 生成（边沿和中心对齐模式）
- 发生以下事件时产生中断/DMA:
 - 更新：计数器溢出，计数器初始化（通过软件）
 - 输入捕捉
 - 输出比较



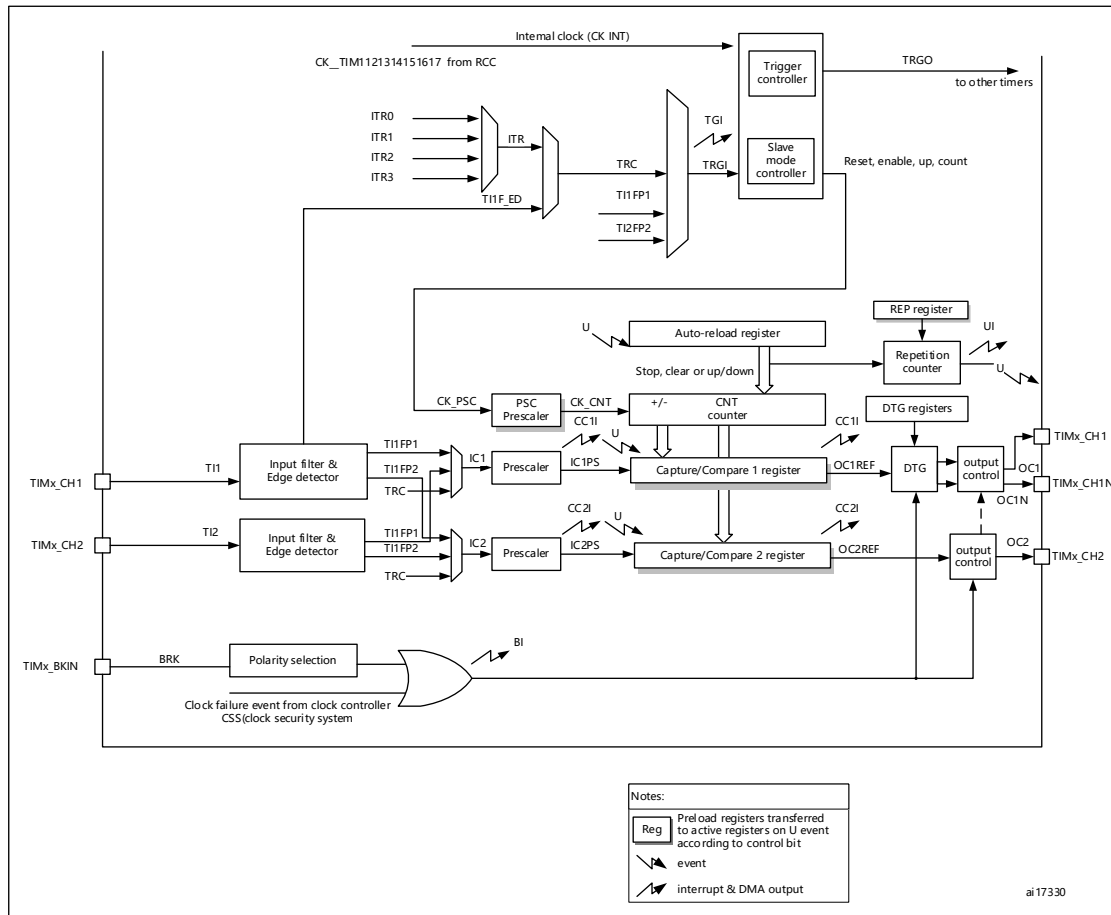
Notes:

Reg	Preload registers transferred to active registers on U event according to control bit
↘	event
↗	interrupt & DMA output

ai17725b

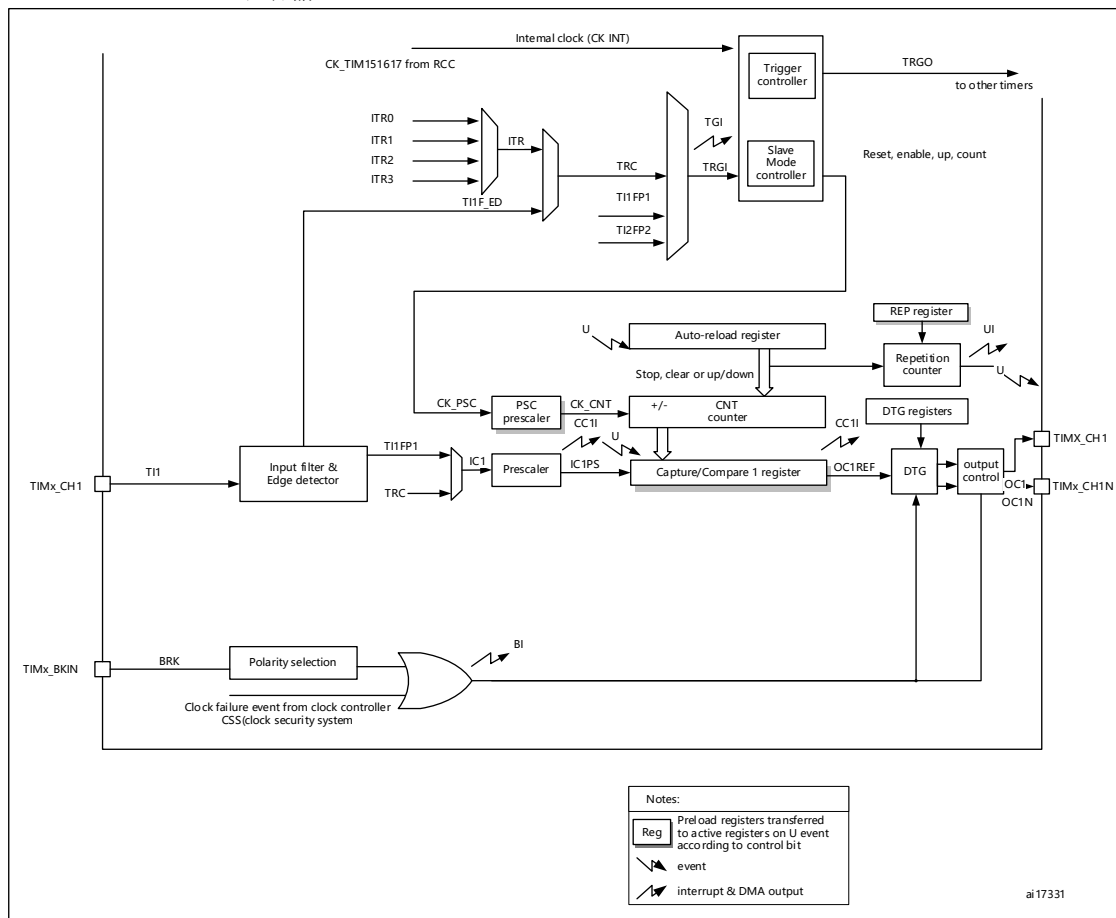
- 计时器 15:

- 32 位递增自动重载计数器。
- 16 位可编程预分频器，可将计数器时钟频率除以 1（在运行中）或 65535 之间的任意倍数。
- 2 个独立通道，用于：
 - 输入捕获
 - 输出比较
 - PWM 生成（边沿和中心对齐模式）
 - 单脉冲模式输出
- 具有可编程死区时间的互补输出（仅通道 1）
- 同步电路，用于通过外部信号控制计时器，并将多个计时器互连在一起。
- 重复计数器仅在给定的循环次数后才更新定时器寄存器。
- 中断输入可将计时器的输出信号置于复位状态或已知状态。
- 发生以下事件时产生中断/DMA：
 - 更新：计数器溢出，计数器初始化（通过软件或内部/外部触发）
 - 触发事件（计数器的开始，停止，初始化或计数内部/外部触发）
 - 输入捕捉
 - 输出比较
 - 中断输入



- 计时器 16&17:
- 32 位递增自动重载计数器。
- 16 位可编程预分频器，可将计数器时钟频率除以 1（在运行中）或 65535 之间的任意倍数。
- 2 个独立通道，用于：
 - 输入捕获

- 输出比较
- PWM 生成 (边沿和中心对齐模式)
- 单脉冲模式输出
- 具有可编程死区时间的互补输出
- 重复计数器仅在给定的循环次数后才更新定时器寄存器。
- 中断输入可将计时器的输出信号置于复位状态或已知状态。
- 发生以下事件时产生中断/DMA:
 - 更新: 计数器溢出
 - 触发事件 (计数器的开始, 停止, 初始化或计数内部/外部触发)
 - 输入捕捉
 - 输出比较
 - 中断输入



16.1.2 时基单元

可编程高级控制定时器的主要模块是一个 16 位计数器及其相关的自动重载寄存器。计数器可以向上，向下或向上和向下计数。计数器时钟可以由预分频器分频。计数器，自动重载寄存器和预分频器寄存器可以通过软件写入或读取，即使计数器正在运行也是如此。时基单元包括：

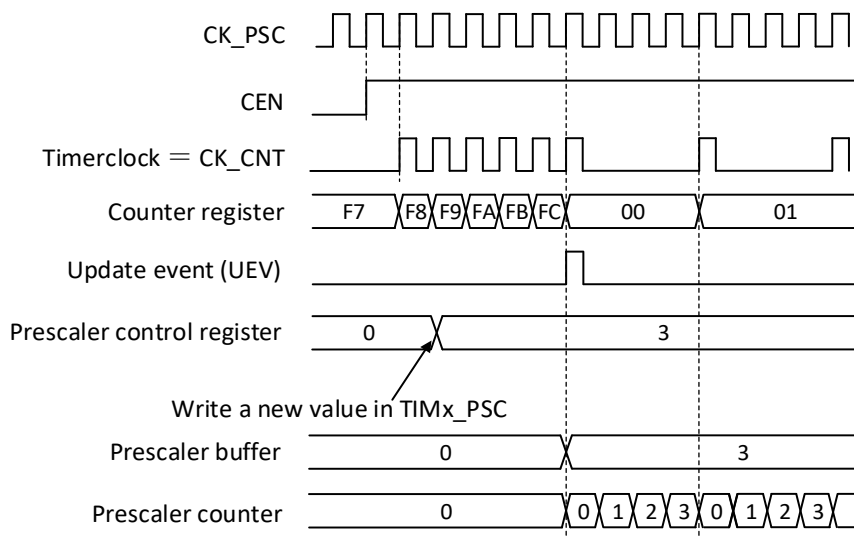
- 计数器寄存器 (TIMx_CNT)
- 预分频器寄存器 (TIMx_PSC)
- 自动重载寄存器 (TIMx_ARR)
- 重复计数器寄存器 (TIMx_RCR)

自动重载寄存器已预载，写或读自动重载寄存器会访问预载寄存器。根据 TIMx_CR1 寄存器中的自动重载预载使能位 (ARPE)，将预载寄存器的内容永久或在每个更新事件 (UEV) 传输到影子寄存器中。当计数器达到上溢 (或递减计数时的下溢) 并且 TIMx_CR1 寄存器中的 UDIS 位等于 0 时，将发送更新事件，它也可以由软件生成。将针对每种配置详细描述更新事件的生成。

计数器由预分频器输出 CK_CNT 提供时钟，只有在 TIMx_CR1 寄存器中的计数器使能位 (CEN) 置 1 时才启用。

预分频器可以将计数器时钟频率除以 1 和 65535 之间的任意因子。它基于通过 16 位寄存器 (在 TIMx_PSC 寄存器中) 控制的 16 位计数器。由于该控制寄存器已被缓冲，因此可以即时更改。新的预分频比为在下次更新事件时考虑在内。

图16.4 预分频比从 1 变为 4 的计数器时序图

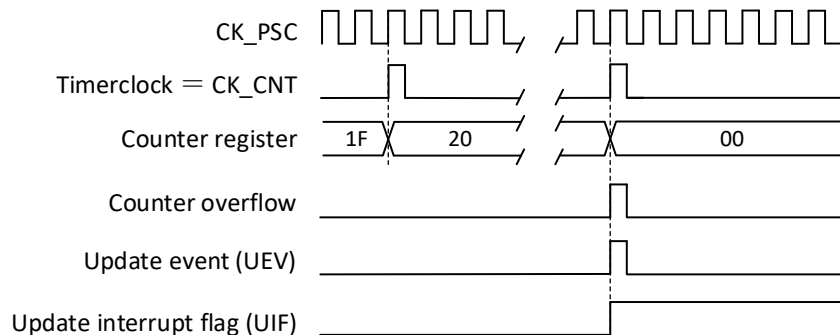


16.1.3 递增模式

在递增计数模式下，计数器从 0 计数到自动重载值 (TIMx_ARR 寄存器的内容)，然后从 0 重新启动并产生计数器溢出事件。如果使用重复计数器，则在递增计数之后生成更新事件 (UEV) 重复执行重复计数器寄存器中编程的次数再加上 1 (TIMx_RCR + 1)。否则，每次计数器溢出都会产生更新事件。通过软件或使用从模式控制器将 TIMx_EGR 寄存器中的 UG 位置 1 也会产生一个更新事件。通过将 TIMx_CR1 寄存器中的 UDIS 位置 1，可以通过软件禁用 UEV 事件。这是为了避免在将新值写入预加载寄存器时更新影子寄存器。然后，直到 UDIS 位被写入 0 为止，都没有更新事件发生。但是，计数器以及预分频器的计数器从 0 重新开始 (但预分频率不变)。另外，如果 TIMx_CR1 寄存器中的 URS 位置 1 (更新请求选择)，则将 UG 位置 1 会生成更新事件 UEV，但不设置 UIF 标志 (因此不会发送中断或 DMA 请求)。这是为了避免在捕获事件上清除计数器时生成更新和捕获中断。当发生更新事件时，所有寄存器都将更新，并且更新标志 (UIF 位位于

TIMx_SR 寄存器) 已设置 (取决于 URS 位): 重复计数器重载 TIMx_RCR 寄存器的内容, 自动重载影子寄存器用预载值 (TIMx_ARR) 更新, 预分频器的缓冲区会重新加载预加载值 (TIMx_PSC 寄存器的内容)。

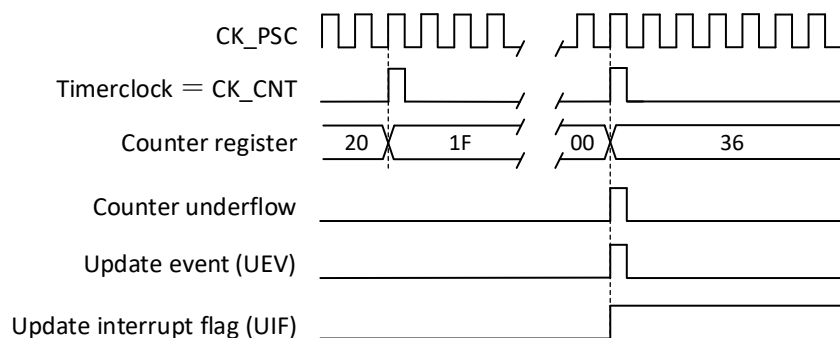
图16.5 递增模式内部时钟除以 N



16.1.4 递减模式

在递减计数模式下, 计数器从自动重载值 (TIMx_ARR 寄存器的内容) 开始递减到 0, 然后从自动重载值重新开始并生成计数器下溢事件。如果使用重复计数器, 则在递减计数重复重复计数器寄存器中编程的次数加 1 (TIMx_RCR + 1) 之后, 将生成更新事件 (UEV)。否则, 在每个计数器下溢时都会生成更新事件。通过软件或使用从模式控制器将 TIMx_EGR 寄存器中的 UG 位置 1 也会生成更新事件。通过将 TIMx_CR1 寄存器中的 UDIS 位置 1, 可以通过软件禁用 UEV 更新事件。这是为了避免在将新值写入预加载寄存器时更新影子寄存器。然后, 直到 UDIS 位写入 0 才会发生更新事件, 但是计数器将从当前状态重新启动自动重载值, 而预分频器的计数器从 0 重新启动 (但预分频率不变)。此外, 如果 TIMx_CR1 寄存器中的 URS 位置 1 (更新请求选择), 则将 UG 位置 1 事件 UEV, 但未设置 UIF 标志 (因此不发送中断或 DMA 请求)。这是为了避免在捕获事件上清除计数器时生成更新和捕获中断。当发生更新事件时, 所有寄存器都将更新, 并且更新标志 (UIF 位位于 TIMx_SR 寄存器) 已设置 (取决于 URS 位): 在重复计数器中重新加载 TIMx_RCR 寄存器的内容。在预分频器的缓冲区中重新加载预加载值 (TIMx_PSC 寄存器的内容) 自动重载有效寄存器用预载值 (TIMx_ARR 寄存器的内容) 更新。请注意, 自动重载在计数器重载之前已更新, 因此下一个周期是预期的。

图16.6 递减模式内部时钟除以 N



16.1.5 中央对齐模式

在居中对齐模式下, 计数器从 0 计数到自动重载值 (TIMx_ARR 寄存器的内容) - 1, 产生计数器溢出事件, 然后从自动重载值开始计数到 1, 并产生计数器下溢事件。然后从 0 重新开始计数。当 TIMx_CR1 寄存器中的 CMS 位不等于 00 时, 中心对齐模式有效。在以下情况下设置在输出中配置的通道的输出比较中断标志: 计数器递减计数 (中心对齐模式 1, CMS = "01"), 计数器递增计数 (中心对齐模式 2, CMS = "10"), 计数器计数上下 (居中对齐模式 3, CMS = "11")。在这种模

式下, 无法写入 TIMx_CR1 寄存器中的 DIR 方向位。它由硬件更新, 并给出计数器的当前方向。更新事件可以在每次计数器溢出和每次计数器下溢时生成, 或者通过将 TIMx_EGR 寄存器中的 UG 位置 1 (通过软件或使用从模式控制器) 来生成, 也会生成更新事件。在这种情况下, 计数器以及预分频器的计数器将从 0 开始重新计数。通过将 TIMx_CR1 寄存器中的 UDIS 位置 1, 可以通过软件禁用 UEV 更新事件。这是为了避免在将新值写入预加载寄存器时更新影子寄存器。然后, 直到 UDIS 位写入 0 为止, 都不会发生更新事件。但是, 计数器会根据当前的自动重载值继续向上和向下计数。此外, 如果 TIMx_CR1 寄存器中的 URS 位置 1 (更新请求选择), 则将 UG 位置 1 会产生 UEV 更新事件, 但不会设置 UIF 标志 (因此不会发送中断或 DMA 请求)。这是为了避免在捕获事件上清除计数器时生成更新和捕获中断。发生更新事件时, 将更新所有寄存器并设置更新标志 (TIMx_SR 寄存器中的 UIF 位) (取决于 URS 位): 在重复计数器中重新加载 TIMx_RCR 寄存器的内容。在预分频器的缓冲区中重新加载预加载值 (TIMx_PSC 寄存器的内容) 自动重载有效寄存器用预载值 (TIMx_ARR 寄存器的内容) 更新。请注意, 如果更新源是计数器溢出, 则在重新加载计数器之前更新自动重载, 以便下一个周期为预期的周期 (将新值加载到计数器中)。

图16.7 内部时钟除以 4, TIMx_ARR=0x36

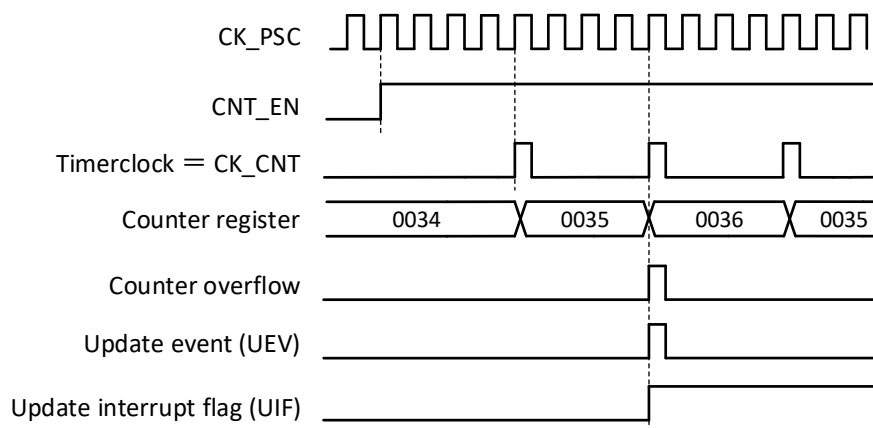
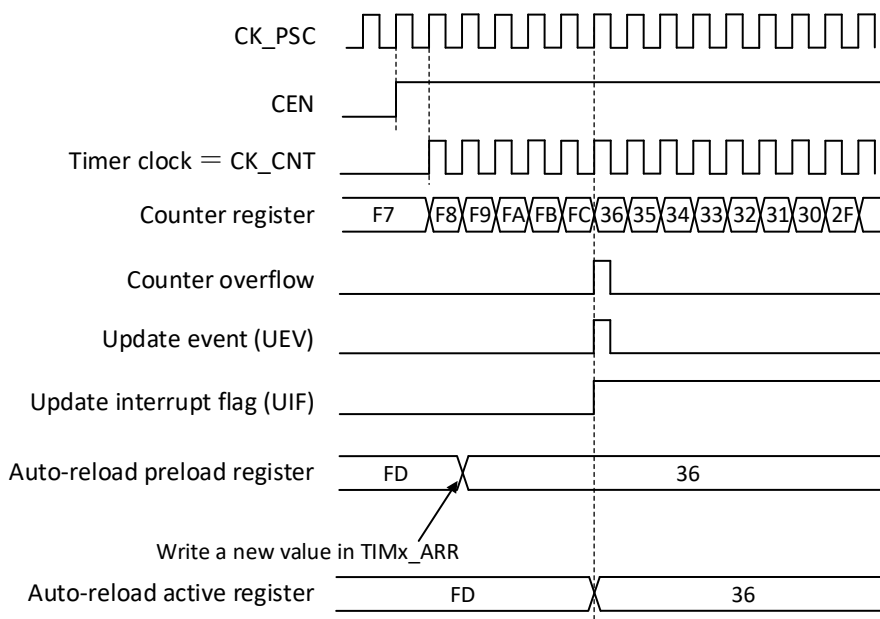


图16.8 使用 APE=1 更新时间 (计数器溢出)



16.1.6 重复计数器

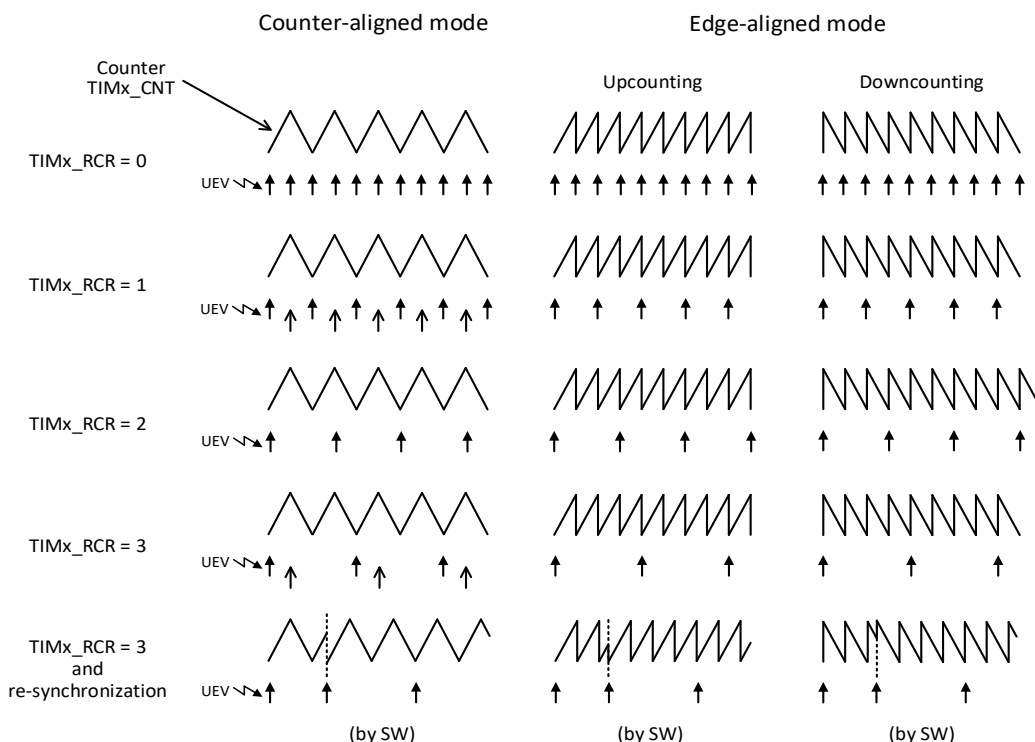
针对计数器上溢/下溢生成更新事件 (UEV)。实际上，仅当重复计数器达到零时才生成它。这在生成 PWM 信号时很有用。这意味着每 $N + 1$ 个计数器上溢或下溢，数据就从预加载寄存器传输到影子寄存器 (TIMx_ARR 自动重载寄存器, TIMx_PSC 预分频器寄存器, 以及 TIMx_CCRx 捕获/比较寄存器)，每 $N + 1$ 个计数器上溢或下溢，其中 N 为值在 TIMx_RCR 重复计数器寄存器中。重复计数器递减：

- 在递增模式下每个计数器溢出时，
- 在递减计数模式下每个计数器下溢时，
- 居中对齐模式下，每个计数器上溢和每个计数器下溢。

尽管这将最大重复次数限制为 128 个 PWM 周期，但它使每个 PWM 周期两次更新占空比成为可能。当在中心对齐模式下每个 PWM 周期仅刷新比较寄存器一次时，由于模式的对称性，最大分辨率为 $2 \times T_{ck}$ 。重复计数器是自动重载类型；重复率如 TIMx_RCR 寄存器值所定义的那样保持（参见图 72）。当更新事件由软件（通过将 TIMx_EGR 寄存器中的 UG 位置 1）或硬件通过从属模式控制器生成时，无论重复计数器的值如何，更新事件都会立即发生，并且将 TIMx_RCR 的内容重新加载到重复计数器中寄存器。

在中心对齐模式下，对于 RCR 的奇数值，更新事件将在上溢或下溢发生，具体取决于何时写入 RCR 寄存器和何时启动计数器。如果在启动计数器之前已写入 RCR，则 UEV 会发生溢出。如果在启动计数器后写入 RCR，则 UEV 发生在下溢中。例如，对于 RCR = 3，将根据写入 RCR 的时间在每个第 4 次上溢或下溢事件中生成 UEV。

图16.9 更新速率示例取决于模式和 TIMx_RCR 寄存器设置



16.1.7 时钟选择

计数器时钟可由以下时钟源提供：

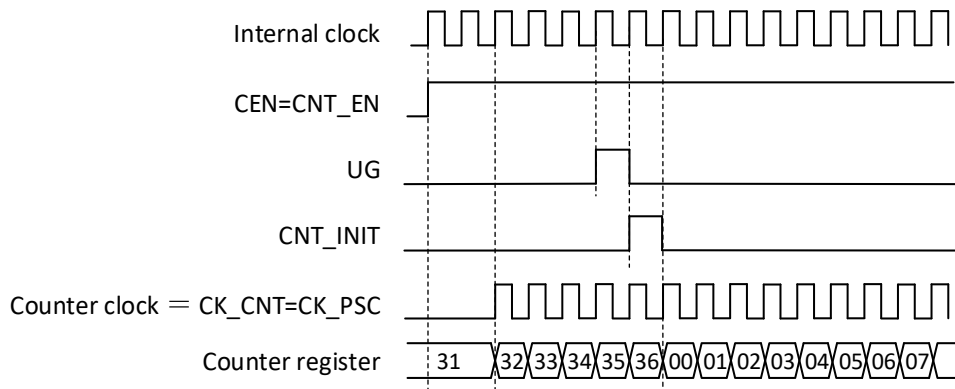
- 内部时钟 (TIMX_CLK)
- 外部时钟模式 1：外部输入引脚
- 外部时钟模式 2：外部触发输入 ETR

- 内部触发输入 (ITRx): 使用一个定时器作为另一个定时器的预分频器。例如, 用户可以将定时器 1 配置为充当定时器 3 的预分频器。

16.1.7.1 内部时钟源

如果禁用了从模式控制器 ($SMS = 000$), 则 CEN , DIR (在 $TIMx_CR1$ 中) 和 UG 位 (在 $TIMx_EGR$ 寄存器中) 是实际控制位, 仅可以通过软件更改 (UG 除外, UG 会自动清除)。一旦 CEN 位写入 1 时, 预分频器由内部时钟 $TIMX_CLK$ 提供时钟。

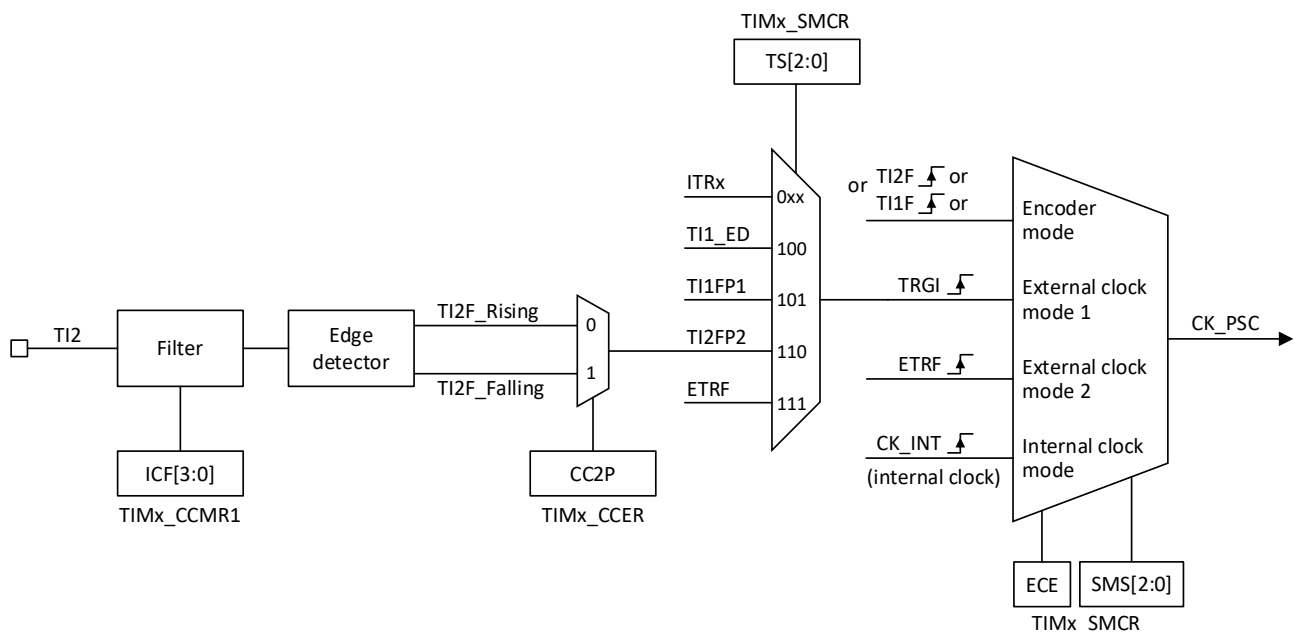
图16.10 正常模式内部时钟源除以 1



16.1.7.2 外部时钟源模式 1

当 $TIMx_SMCR$ 寄存器中的 $SMS = 111$ 时, 选择此模式。计数器可以在所选输入的每个上升沿或下降沿计数。

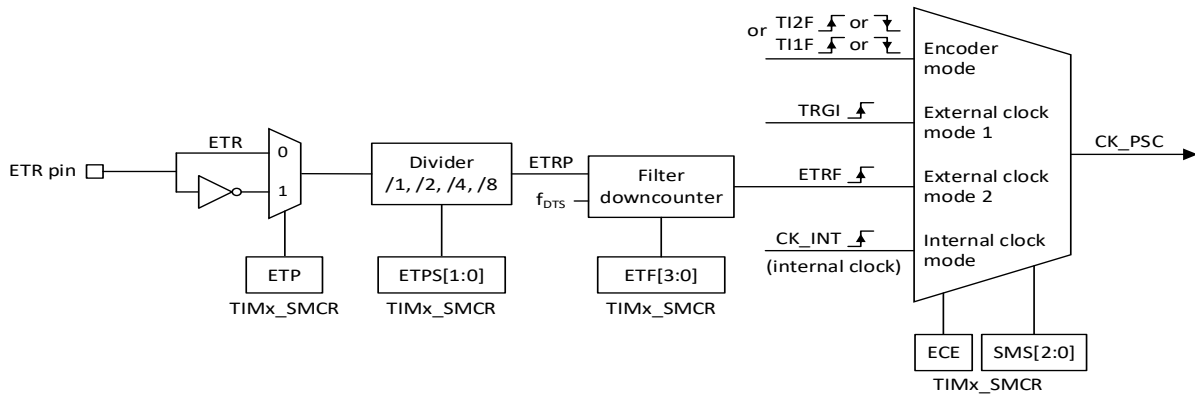
图16.11 外部时钟连



16.1.7.3 外部时钟源模式 2

通过在 $TIMx_SMCR$ 寄存器中写入 $ECE = 1$ 来选择该模式, 计数器可以在外部触发输入 ETR 的每个上升沿或下降沿计数。

图16.12 ETR 输入模块



16.1.7.4 捕获/比较通道

每个捕获/比较通道均围绕捕获/比较寄存器（包括影子寄存器），捕获的输入级（具有数字滤波器，复用和预分频器）和输出级（具有比较器和输出控制）构建。

输入级对相应的 **Ti_x** 输入进行采样以生成滤波信号 **Ti_xF**，然后，具有极性选择的边缘检测器生成信号（**Ti_xFP_x**）可用作从机模式控制器的触发输入或用作捕获命令。它在捕捉寄存器（**IC_xPS**）之前被预先缩放。输出级生成一个中间波形，该波形随后用作参考：**OC_xRef**（高电平有效），极性作用于链的末端。

捕获/比较模块由一个预加载寄存器和一个影子寄存器组成，写和读总是访问预加载寄存器。在捕获模式下，捕获实际上是在影子寄存器中完成的，然后将其复制到预加载寄存器中。在比较模式下，预加载寄存器的内容被复制到影子寄存器中，并与计数器进行比较。

图16.13 捕获通道（通道 1 输入级）

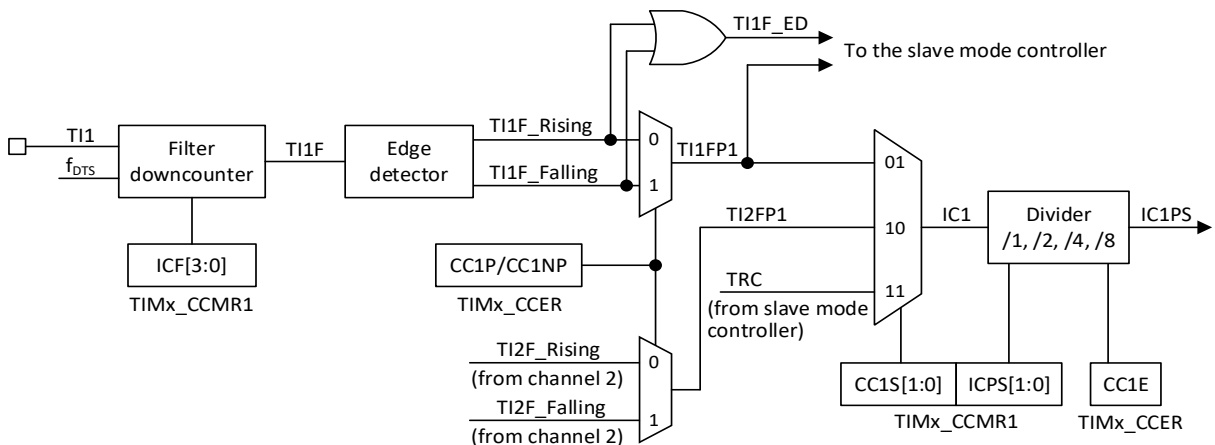


图16.14 捕获通道 (通道 1~3 输出级)

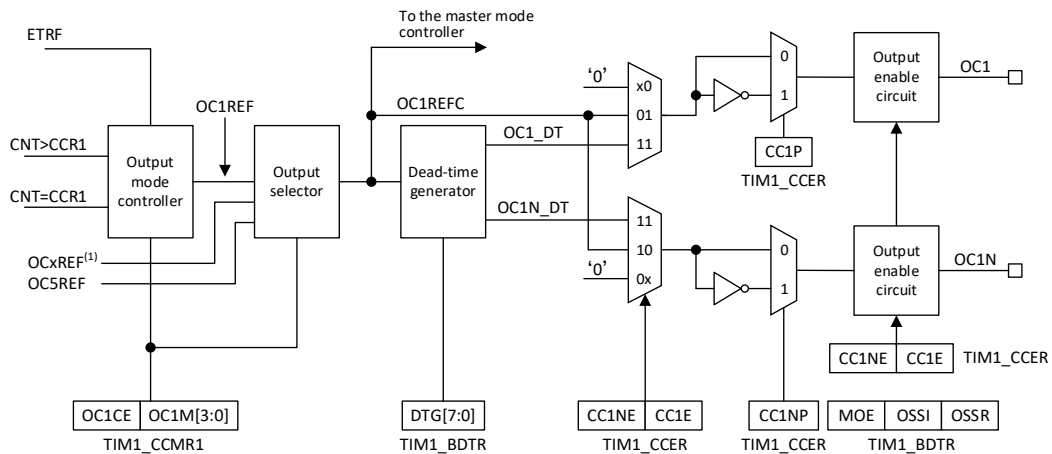
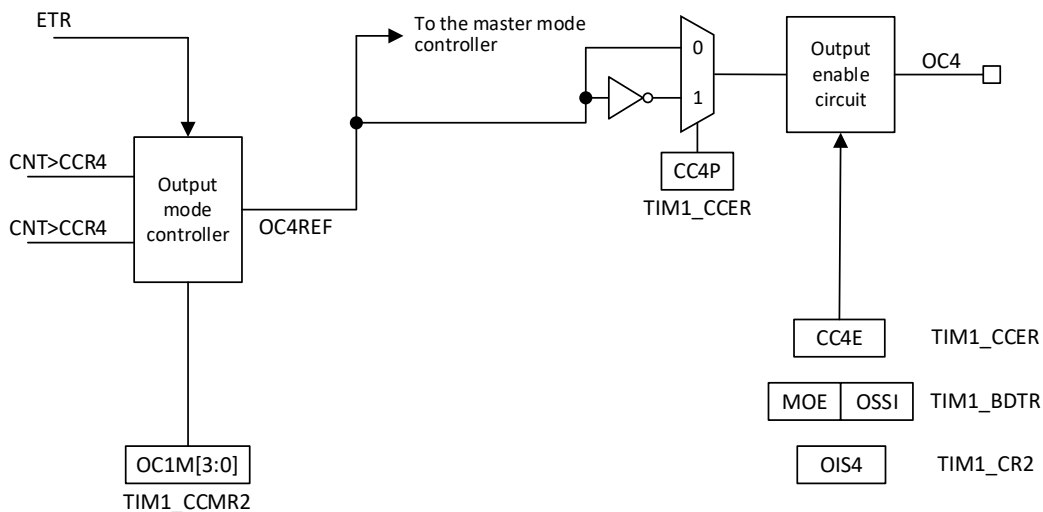


图16.15 捕获通道 (通道 4 输出级)



16.1.8 输入捕获模式

16.1.8.1 正常模式

在输入捕获模式下，捕捉/比较寄存器 (TIMx_CCRx) 用于在相应 ICx 信号检测到转换后锁存计数器的值。发生捕捉时，将设置相应的 CCXIF 标志 (TIMx_SR 寄存器)，并且如果使能了中断或 DMA 请求，则可以发送它们。如果在 CCxIF 标志已经为高电平时发生捕捉，则将设置捕捉标志 CCxOF (TIMx_SR 寄存器)。可以通过软件将 CCxIF 写入“0”或读取存储在 TIMx_CCRx 寄存器中的捕捉数据来清除 CCxIF。当写入“0”时，CCxOF 被清除。

发生输入捕获时：

TIMx_CCR1 寄存器在活动转换时获取计数器的值，CC1IF 标志置位 (中断标志)。如果至少两次连续捕捉发生且未清除标志，则 CC1OF 也被置位。

根据 CC1IE 位产生一个中断。

根据 CC1DE 位产生 DMA 请求。

为了处理超量捕捉，建议在超量捕捉标志之前读取数据。这是为了避免丢失可能在读取标志之后和读取数据之前发生的过度捕捉。

以下示例显示了 TI1 输入上升时如何在 TIMx_CCR1 中捕获计数器值。为此，请使用以下过程：选择活动输入：TIMx_CCR1 必须链接到 TI1 输入，因此将 CC1S 位写入 TIMx_CCMR1 寄存器中的 01。CC1S 不同于 00 时，将在输入中配置通道，并且 TIMx_CCR1 寄存器变为只读。

相对于连接到定时器的信号来编程所需的输入滤波器持续时间（如果输入是 TIx 输入，则通过编程 TIMx_CCMRx 寄存器中的 ICxF 位）。切换时，输入信号在至少五个内部时钟周期内不稳定。我们必须将过滤器持续时间编程为长于这五个时钟周期。当检测到连续 8 个具有新电平 of 的采样（以 fDTS 频率采样）时，我们可以验证 TI1 上的转换。然后将 IC1F 位写入 TIMx_CCMR1 寄存器中的 0011。

通过将 TIMx_CCER 寄存器中的 CC1P 位写入 0，选择 TI1 通道上有效转换的边沿（在这种情况下为上升沿）。

编程输入预分频器。在我们的示例中，我们希望在每次有效跳变时执行捕获，因此禁用了预分频器（将 TIMx_CCMR1 寄存器中的 IC1PS 位写入“00”）。

通过将 TIMx_CCER 寄存器中的 CC1E 位置 1，使能从计数器到捕获寄存器的捕获。

如果需要，可以通过将 TIMx_DIER 寄存器中的 CC1IE 位置 1 来启用相关的中断请求，和/或通过 TIMx_DIER 寄存器中的 CC1DE 位置 1 来使 DMA 请求。

16.1.8.2 PWM 输入模式

此模式是输入捕获模式的特殊情况。除以下步骤外，过程相同：

两个 ICx 信号映射到同一 TIx 输入上。

这两个 ICx 信号在极性相反的边沿有效。

选择两个 TIxFP 信号之一作为触发输入，并将从模式控制器配置为复位模式。

例如，用户可以测量周期（在 TIMx_CCR1 寄存器中）和占空比（在使用以下步骤将 TI1 上的 PWM 的 TIMx_CCR2 寄存器）在 TIMX_CLK 频率和预分频值上）：

选择 TIMx_CCR1 的有效输入：将 CC1S 位写入 01 中的 TIMx_CCMR1 寄存器（选择了 TI1）。

选择 TI1FP1 的有效极性（用于 TIMx_CCR1 中的捕获和计数器清零）：将 CC1P 位写为 0（在上升沿有效）。选择 TIMx_CCR2 的有效输入：将 CC2S 位写入 TIMx_CCMR1 寄存器中的 10（选择 TI1）。

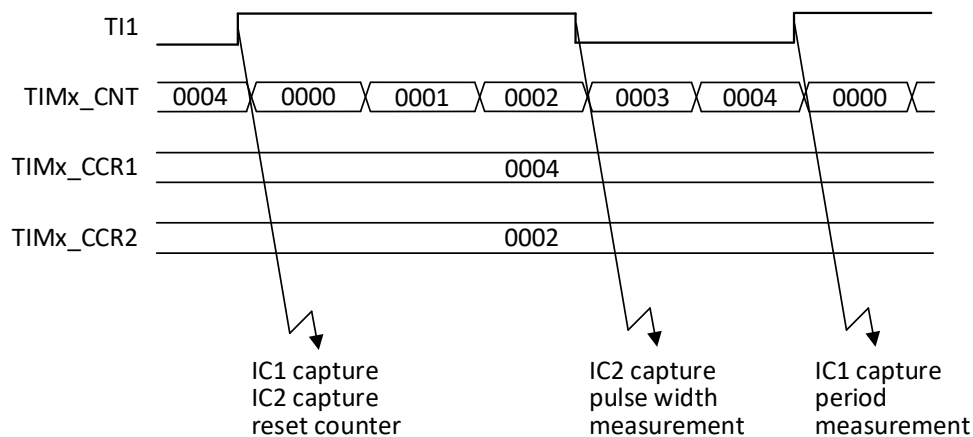
选择 TI1FP2 的有效极性（用于 TIMx_CCR2 中的捕获）：将 CC2P 位写为 1（在下降沿有效）。

选择有效的触发输入：将 TS 位写入 TIMx_SMCR 寄存器中的 101（已选择 TI1FP1）。

在复位模式下配置从模式控制器：将 TIMx_SMCR 寄存器中的 SMS 位写入 100。

启用捕获：将 TIMx_CCER 寄存器中的 CC1E 和 CC2E 位写为 1。

图16.16 -PWM 输入模式



16.1.8.3 强制输出模式

在输出模式下 (TIMx_CCMRx 寄存器中的 CCxS 位= 00), 可以直接通过软件直接将每个输出比较信号 (OCxREF 然后是 OCx / OCxN) 强制为有效或无效电平, 而与输出比较寄存器和计数器之间的任何比较无关。

为了将输出比较信号 (OCxREF / OCx) 强制为有效电平, 用户只需要在相应 TIMx_CCMRx 寄存器的 OCxM 位中写入 101。因此, OCxREF 被强制为高电平 (OCxREF 始终为高电平有效), 并且 OCx 获得与 CCxP 极性位相反的值。

16.1.9 输出捕获模式

16.1.9.1 正常模式

此功能用于控制输出波形或一段时间后过去提供指示。

当在捕获/比较寄存器和计数器之间找到匹配项时, 输出比较功能:

将相应的输出引脚分配给由输出比较模式 (TIMx_CCMRx 寄存器中的 OCxM 位) 和输出极性 (TIMx_CCER 寄存器中的 CCxP 位) 定义的可编程值。输出引脚可以保持其电平 (OCxM = 000), 设置为有效 (OCxM = 001), 设置为无效 (OCxM = 010) 或在匹配时切换 (OCxM = 011)。

设置中断状态寄存器中的标志 (TIMx_SR 寄存器中的 CCxIF 位)。如果设置了相应的中断屏蔽 (TIMx_DIER 寄存器中的 CCxIE 位), 则产生一个中断。

如果设置了相应的使能位 (TIMx_DIER 寄存器中的 CCxDE 位, TIMx_CR2 寄存器中的 CCDS 位用于 DMA 请求选择), 则发送 DMA 请求。

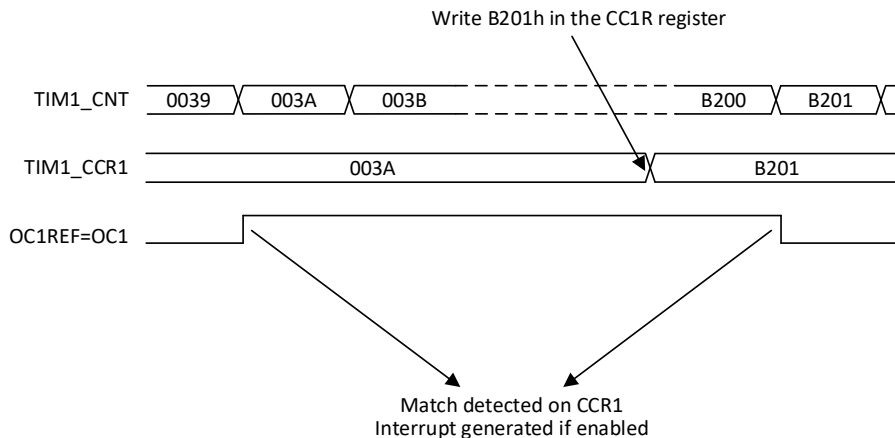
可以使用 TIMx_CCMRx 寄存器中的 OCxPE 位对 TIMx_CCRx 寄存器进行预编程或不预编程。在输出比较模式下, 更新事件 UEV 对 OCxREF 和 OCx 输出没有影响。

流程:

- 选择计数器时钟 (内部, 外部, 预分频器)。
- 将所需的数据写入 TIMx_ARR 和 TIMx_CCRx 寄存器。
- 如果要产生中断请求, 则将 CCxIE 位置 1。
- 选择输出模式。例如:
 - 当 CNT 与 CCRx 匹配时, 写入 OCxM = 011 以切换 OCx 输出引脚

- 写 $OCxPE = 0$ 以禁用预加载寄存器
- 写入 $CCxP = 0$ 以选择有效的高极性
- 写入 $CCxE = 1$ 以启用输出
- 通过将 $TIMx_CR1$ 寄存器中的 CEN 位置 1 来使能计数器。 $TIMx_CCRx$ 寄存器可由软件随时更新以控制输出波形，前提是未启用预加载寄存器 ($OCxPE = 0$)，否则为 $TIMx_CCRx$ 影子寄存器仅在下一个更新事件 UEV 时更新。

图16.17 输出比较模式



16.1.9.2 PWM 模式

脉冲宽度调制模式允许生成具有由 $TIMx_ARR$ 寄存器的值确定的频率和由 $TIMx_CCRx$ 寄存器的值确定的占空比的信号。

通过在 $TIMx_CCMRx$ 寄存器的 $OCxM$ 位中写入“110” (PWM 模式 1) 或“111” (PWM 模式 2)，可以在每个通道上独立选择 PWM 模式 (每个 OCx 输出一个 PWM)。必须通过将 $TIMx_CCMRx$ 寄存器中的 $OCxPE$ 位置 1 来使能相应的预加载寄存器，并最终通过将 $TIMx_CR1$ 寄存器中的 $ARPE$ 位置 1 来自动重载预加载寄存器 (在递增计数或中心对齐模式下)。由于仅在发生更新事件时才将预加载寄存器转移到影子寄存器，因此在启动计数器之前，用户必须通过将 $TIMx_EGR$ 寄存器中的 UG 位置 1 来初始化所有寄存器。 OCx 极性可通过 $TIMx_CCER$ 寄存器中的 $CCxP$ 位进行软件编程。可以将其编程为高电平有效或低电平有效。 OCx 输出为通过 $CCxE$, $CCxNE$, MOE , $OSSI$ 和 $OSSR$ 位 ($TIMx_CCER$ 和 $TIMx_BDTR$ 寄存器) 的组合来使能。有关更多详细信息，请参见 $TIMx_CCER$ 寄存器说明。

在 PWM 模式 (1 或 2) 下，始终将 $TIMx_CNT$ 和 $TIMx_CCRx$ 进行比较以确定 $TIMx_CCRx \leq TIMx_CNT$ 还是 $TIMx_CNT \leq TIMx_CCRx$ (取决于计数器的方向)。对齐模式取决于 $TIMx_CR1$ 寄存器中的 CMS 位。

16.1.9.3 边缘对齐模式

递增配置

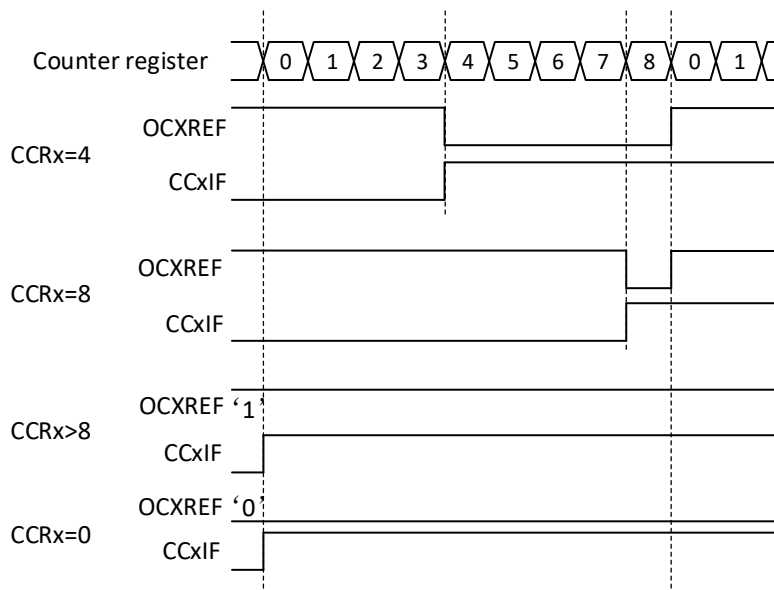
当 $TIMx_CR1$ 寄存器中的 DIR 位为低时，向上计数有效。在以下示例中，我们考虑 PWM 模式 1。只要 $TIMx_CNT < TIMx_CCRx$ ，基准 PWM 信号 $OCxREF$ 就会为高，否则它将变为低电平。如果 $TIMx_CCRx$ 中的比较值大于自动重载值 ($TIMx_ARR$ 中)，则 $OCxREF$ 保持为“1”。如果比较值为 0，则 $OCxRef$ 保持为“0”。

递减配置

当 $TIMx_CR1$ 寄存器中的 DIR 位为高时，递减计数有效。在 PWM 模式 1 中，只要 $TIMx_CNT > TIMx_CCRx$ ，基准信号 $OCxRef$ 就会为低电平，否则它将变为高电平。如果 $TIMx_CCRx$ 中的比

较值大于 TIMx_ARR 中的自动重载值，则 OCxREF 保持为“1”。在此模式下，无法使用 0% PWM。

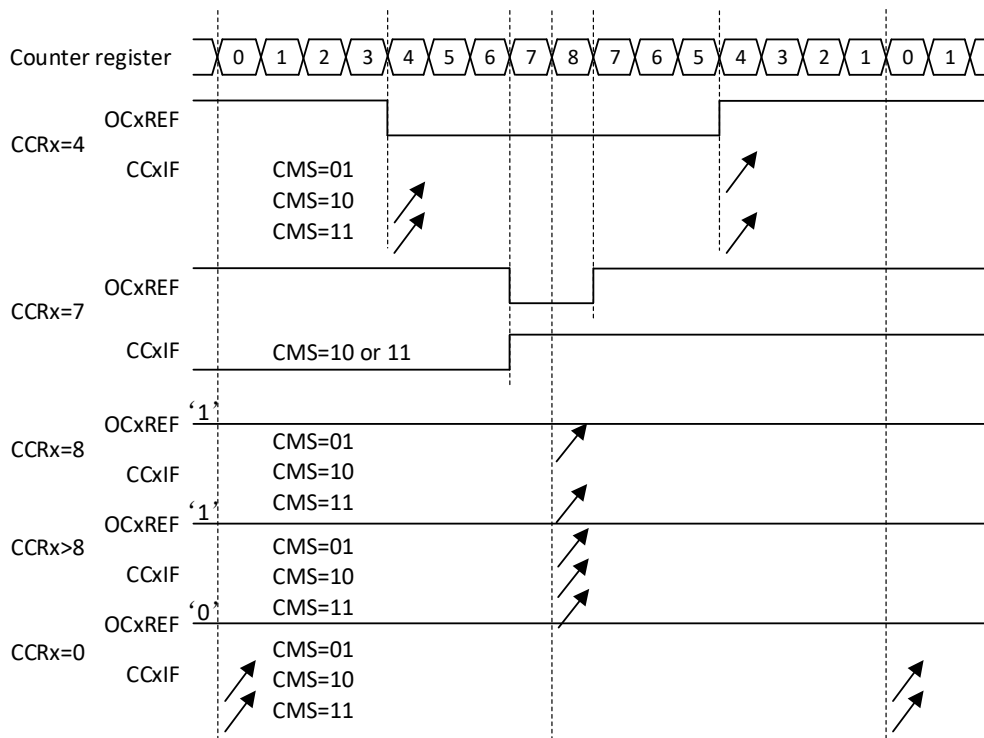
图16.18 PWM 边缘对齐模式



16.1.9.4 中央对齐模式

当 TIMx_CR1 寄存器中的 CMS 位不同于'00'（其余所有配置对 OCxRef / OCx 信号具有相同的影响）时，中心对齐模式有效。当计数器递增计数时，比较标志将置 1 向下计数，或者当它向上计数和向下计数时（取决于 CMS 位配置）同时计数。TIMx_CR1 寄存器中的方向位（DIR）由硬件更新，并且不能由软件更改。

图16.19 PWM 中央对齐模式



16.1.9.5 互补输出和死区插入 (TIM1&TIM15 至 TIM17)

高级控制定时器 (TIM1) 和通用定时器 (TIM15 至 TIM17) 可以输出互补信号, 并管理输出的关闭和打开瞬间。该时间通常称为停滞时间, 必须根据连接到输出的设备及其特性 (电平转换器的固有延迟, 电源开关引起的延迟...) 对其进行调整。用户可以为每个输出独立选择输出的极性 (主输出 OCx 或互补 OCxN)。这是通过写入 TIMx_CCER 寄存器中的 CCxP 和 CCxNP 位来完成的。

互补信号 OCx 和 OCxN 由多个控制位组合激活: TIMx_CCER 寄存器中的 CCxE 和 CCxNE 位以及 TIMx_BDTR 和 TIMx_CR2 寄存器中的 MOE, OISx, OISxN, OSSI 和 OSSR 位。特别是当切换到 IDLE 状态 (MOE 下降到 0) 时, 死区时间被激活。

通过将 CCxE 和 CCxNE 位以及 MOE 位 (如果存在断路电路) 置 1, 可以启用死区时间插入。TIMx_BDTR 寄存器的 DTG [7: 0] 位用于控制所有通道的死区时间。

它从参考波形 OCxREF 产生 2 个输出 OCx 和 OCxN。如果 OCx 和 OCxN 为高电平有效:

- OCx 输出信号与参考信号相同, 只是上升沿相对于参考上升沿有所延迟。
- OCxN 输出信号与参考信号相反, 但上升沿相对于参考下降沿有所延迟。

如果延迟大于有效输出 (OCx 或 OCxN) 的宽度, 则不会生成相应的脉冲。每个通道的死区延迟都相同, 并且可通过 TIMx_BDTR 寄存器中的 DTG 位进行编程。

图16.20 带死区插入的互补输出

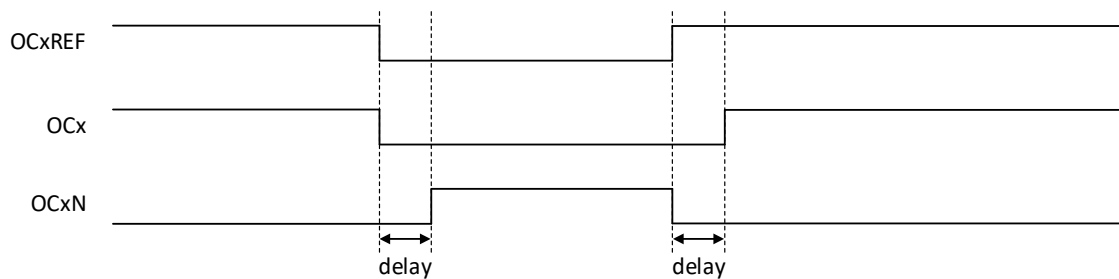


图16.21 延迟大于负脉冲的死区时间波形

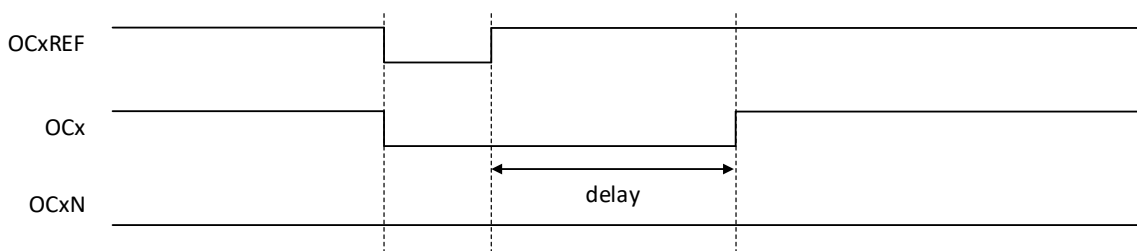


表16.1 具有中断功能的互补 OCx 和 OCxN 通道的输出控制

Control bits					Output states ⁽¹⁾	
MOE bit	OSSI bit	OSSR bit	CCxE bit	CCxNE bit	OCx output state	OCxN output state
1	X	0	0	0	Output Disabled (not driven by the timer) OCx=0, OCx_EN=0	Output Disabled (not driven by the timer) OCxN=0, OCxN_EN=0
		0	0	1	Output Disabled (not driven by the timer) OCx=0, OCx_EN=0	OCxREF + Polarity OCxN=OCxREF xor CCxNP, OCxN_EN=1
		0	1	0	OCxREF + Polarity OCx=OCxREF xor CCxP, OCx_EN=1	Output Disabled (not driven by the timer) OCxN=0, OCxN_EN=0
		0	1	1	OCREF + Polarity + dead-time OCx_EN=1	Complementary to OCREF (not OCREF) + Polarity + dead-time OCxN_EN=1
		1	0	0	Output Disabled (not driven by the timer) OCx=CCxP, OCx_EN=0	Output Disabled (not driven by the timer) OCxN=CCxNP, OCxN_EN=0
		1	0	1	Off-State (output enabled with inactive state) OCx=CCxP, OCx_EN=1	OCxREF + Polarity OCxN=OCxREF xor CCxNP, OCxN_EN=1
		1	1	0	OCxREF + Polarity OCx=OCxREF xor CCxP, OCx_EN=1	Off-State (output enabled with inactive state) OCxN=CCxNP, OCxN_EN=1
		1	1	1	OCREF + Polarity + dead-time OCx_EN=1	Complementary to OCREF (not OCREF) + Polarity + dead-time OCxN_EN=1
0	0	X	0	0	Output Disabled (not driven by the timer) Asynchronously: OCx=CCxP, OCx_EN=0, OCxN=CCxNP, OCxN_EN=0 Then if the clock is present: OCx=OISx and OCxN=OISxN after a dead-time, assuming that OISx and OISxN do not correspond to OCx and OCxN both in active state.	
	0		0	1		
	0		1	0		
	0		1	1		
	1		0	0	Off-State (output enabled with inactive state) Asynchronously: OCx=CCxP, OCx_EN=1, OCxN=CCxNP, OCxN_EN=1 Then if the clock is present: OCx=OISx and OCxN=OISxN after a dead-time, assuming that OISx and OISxN do not correspond to OCx and OCxN both in active state.	
	1		0	1		
	1		1	0		
	1		1	1		

1. When both outputs of a channel are not used (CCxE = CCxNE = 0), the OISx, OISxN, CCxP and CCxNP bits must be kept cleared.

16.1.9.6 刹车功能

刹车源可以是刹车输入引脚，也可以是时钟安全系统 (CSS) 从复位时钟控制器产生的时钟故障事件。

从复位退出时，刹车电路被禁用，MOE 位为低。用户可以通过将 TIMx_BDTR 寄存器中的 BKE 位置 1 来使能刹车功能。刹车输入极性可以通过配置 BKP 位来选择相同的寄存器，可以同时修改

BKE 和 **BKP**。当 **BKE** 和 **BKP** 位被写入时，在写入有效之前施加 1 个 **APB** 时钟周期的延迟。因此，必须等待 1 个 **APB** 时钟周期才能在写操作后正确读回该位。

由于 **MOE** 下降沿可以是异步的，因此已在实际信号（作用于输出）和同步控制位（在 **TIMx_BDTR** 寄存器中访问）之间插入了重新同步电路。这会导致异步信号和同步信号之间的一些延迟。特别是，如果 **MOE** 被写为 1 而其为低，则在正确读取 **MOE** 之前必须插入一个延迟（虚拟指令）。这是因为用户写入异步信号，但读取同步信号。

当发生刹车时（在刹车输入上选择的电平）：

MOE 位异步清除，将输出置于无效状态，空闲状态或复位状态（由 **OSSI** 位选择）。即使 **MCU** 振荡器关闭，此功能也可以使用。

MOE = 0 时，将以 **TIMx_CR2** 寄存器的 **OISx** 位中编程的电平驱动每个输出通道。如果 **OSSI = 0**，则定时器释放使能输出，否则使能输出保持高电平。

使用互补输出时：

- 首先将输出置于复位状态无效状态（取决于极性）。此操作是异步完成的，因此即使未向计时器提供时钟也可以工作。
- 如果定时器时钟仍然存在，则死区时间发生器将重新激活，以便在死区时间后以 **OISx** 和 **OISxN** 位中编程的电平驱动输出。即使在这种情况下，也无法将 **OCx** 和 **OCxN** 一起驱动到活动电平。请注意，由于 **MOE** 上的重新同步，空载时间持续时间比平时更长（大约 2 **ck_tim** 时钟周期）。
- 如果 **OSSI = 0**，则定时器释放使能输出，否则，一旦 **CCxE** 或 **CCxNE** 位之一为高，使能输出将保持或变为高电平。

刹车状态标志（**TIMx_SR** 寄存器中的 **BIF** 位）被置 1。如果 **TIMx_DIER** 寄存器中的 **BIE** 位置 1，则可以产生刹车。如果 **TIMx_DIER** 寄存器中的 **BDE** 位置 1，则可以发送 **DMA** 请求。如果 **TIMx_BDTR** 寄存器中的 **AOE** 位置 1，则在下一个更新事件 **UEV** 时，**MOE** 位置将自动重新设置。例如，这可用于执行调节。否则，**MOE** 保持低电平直到再次将其写入“1”。在这种情况下，它可以用于安全性，并且刹车输入可以连接到来自电源驱动器，热传感器或任何安全组件的警报。

注意：刹车输入在水平上起作用，因此，在刹车输入处于活动状态时（既不能自动也不可以通过软件）不能设置 **MOE**。同时，状态标志 **BIF** 无法清除。

可以通过具有可编程极性和 **TIMx_BDTR** 寄存器中的使能位 **BKE** 的 **BRK** 输入产生中断。

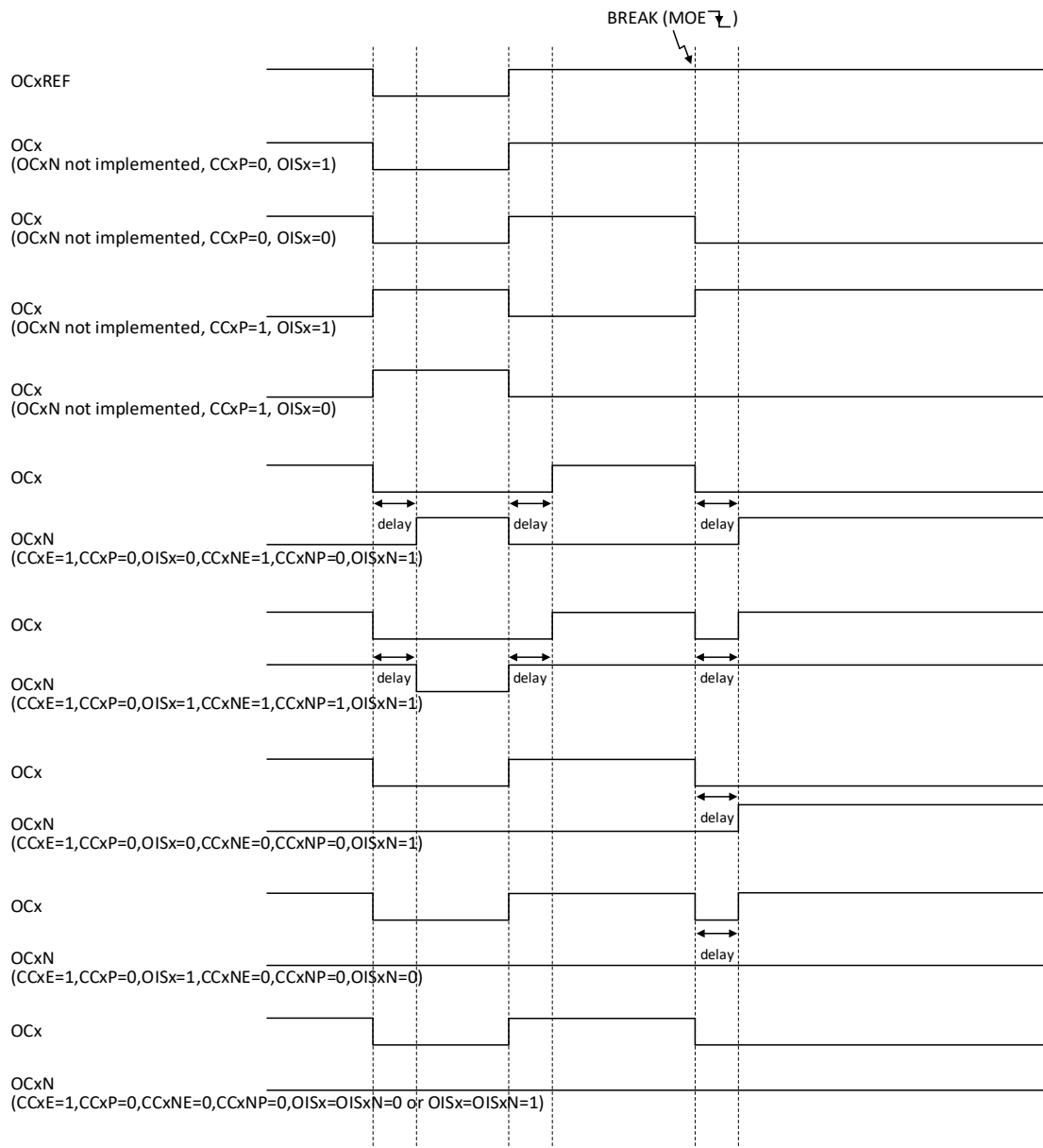
有两种产生刹车的解决方案：

通过使用具有可编程极性和使能位 **BKE** 的 **BRK** 输入 **TIMx_BDTR** 寄存器。

通过软件通过 **TIMx_EGR** 寄存器的 **BG** 位。

除了刹车输入和输出管理外，还提供了写保护。在断路内部实现保护应用程序。它允许冻结几个参数的配置（死区持续时间，**OCx / OCxN** 极性和状态禁用时，**OCxM** 配置，刹车使能和极性）。用户可以选择由 **TIMx_BDTR** 寄存器中的 **LOCK** 位选择的三级保护。请参阅（**TIMx_BDTR**）。**MCU** 复位后，只能写入一次 **LOCK** 位。

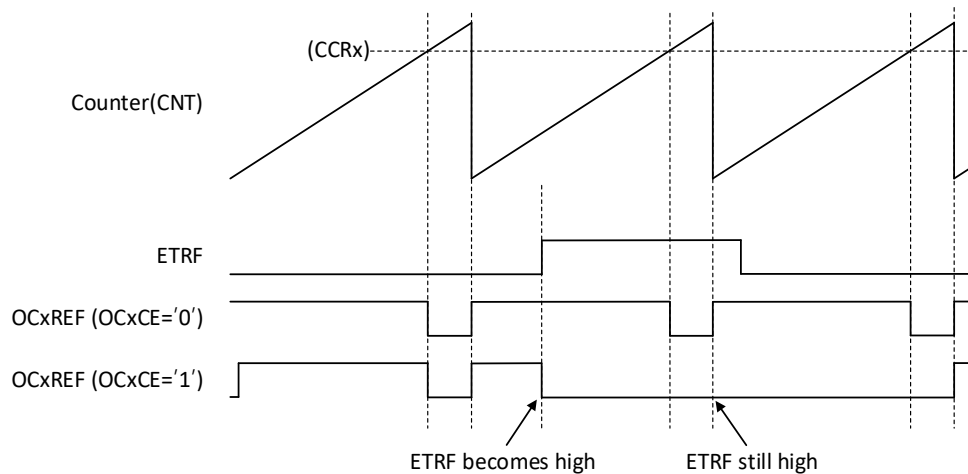
图16.22 响应刹车的输出行为



16.1.9.7 发生外部事件时清除 OCxREF 信号

通过将高电平施加到 ETRF 输入，可以将给定通道的 OCxREF 信号驱动为低电平（相应的 TIMx_CCMRx 寄存器的 OCxCE 使能位设置为 1）。OCxREF 信号保持低电平，直到发生下一个更新事件 UEV。此功能只能在输出比较和 PWM 模式下使用，而不能在强制模式下使用。

图16.23 清除定时器 x OCxREF

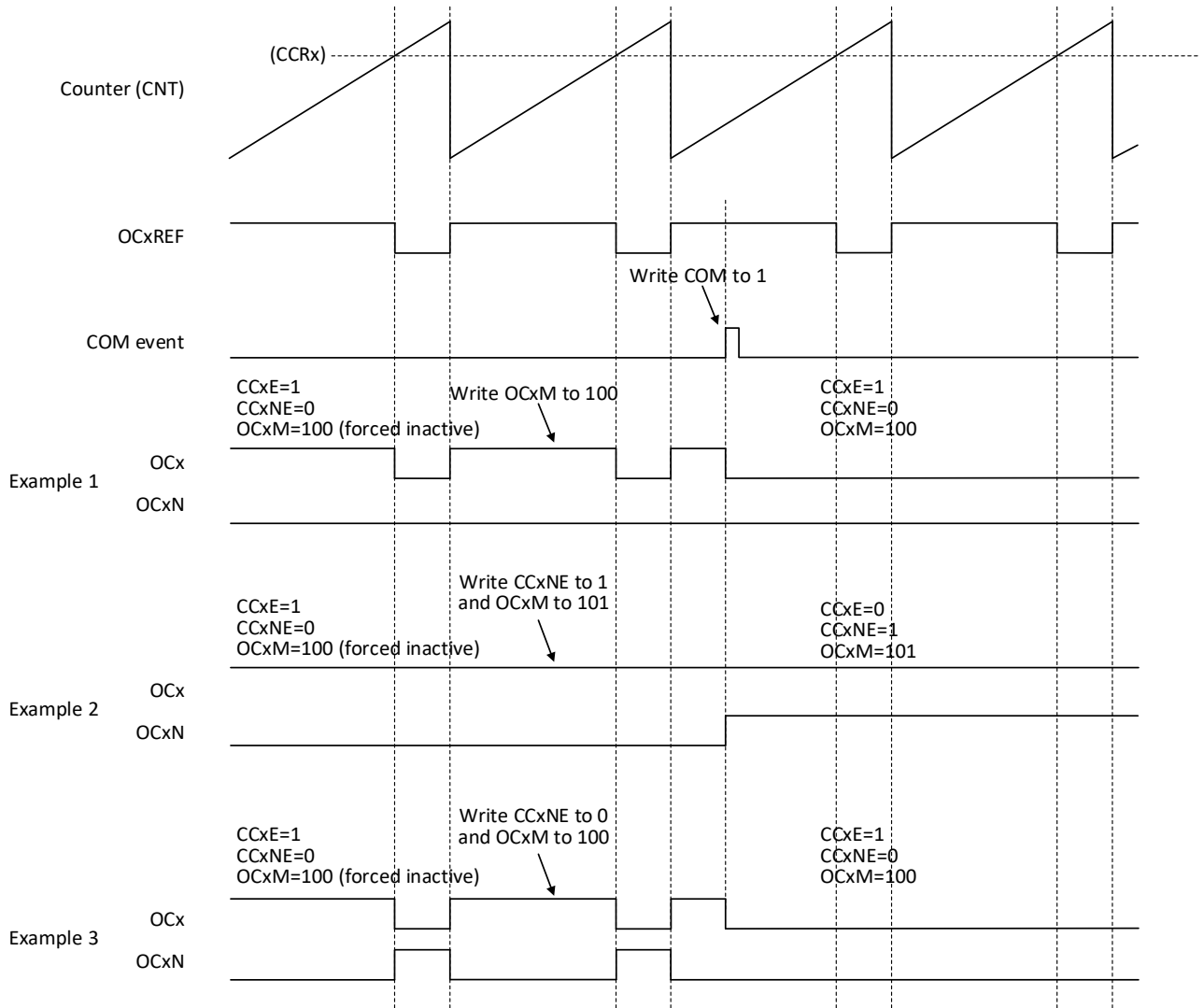


16.1.9.8 6 步 PWM 生成

在通道上使用互补输出时，OCxM，CCxE 和 CCxNE 位上有预加载位。在 COM 换向事件时，预载位被传输到影子位。因此，用户可以预先为下一步编程配置，并同时更改所有通道的配置。可以通过将 TIMx_EGR 寄存器中的 COM 位置 1 或由硬件（在 TRGI 上升沿）来生成 COM。

当发生 COM 事件时（TIMx_SR 寄存器中的 COMIF 位），将设置一个标志，该标志会产生中断（如果 TIMx_DIER 寄存器中的 COMIE 位置 1）或 DMA 请求（如果 TIMx_DIER 寄存器中的 COMDE 位置 1）。

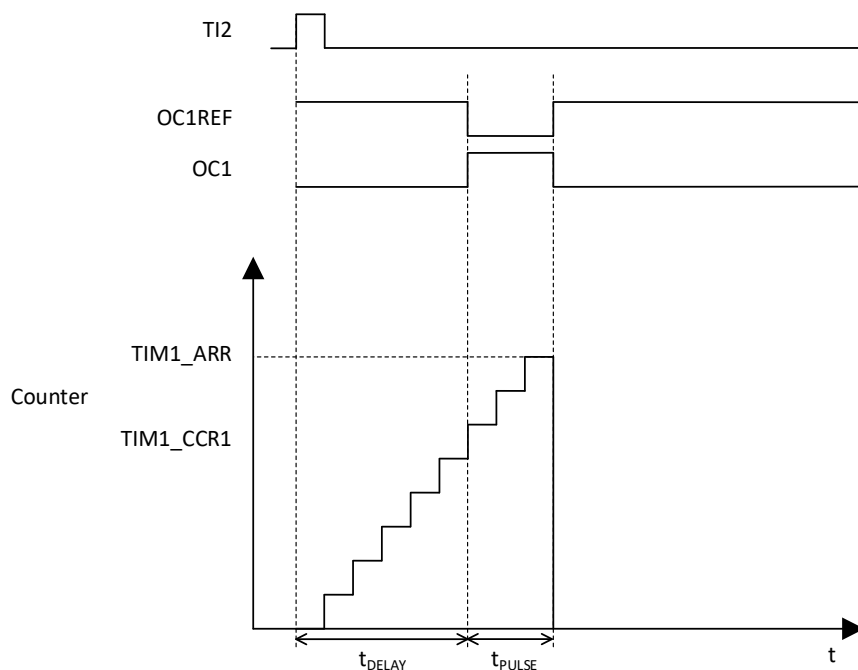
图16.24 步 PWM 生成



16.1.9.9 单脉冲模式 (OPM)

单脉冲模式 (OPM) 是之前模式的特殊情况。它允许计数器响应激励而启动，并在可编程延迟后生成具有可编程长度的脉冲。计数器的启动可以通过从模式控制器来控制，可以在输出比较模式或 PWM 模式下生成波形。通过将 TIMx_CR1 寄存器中的 OPM 位置 1 选择单脉冲模式。这使计数器在下一个更新事件 UEV 处自动停止。

图16.25 单脉冲模式



仅当比较值与计数器不同时，才能正确生成脉冲初始值。在启动之前（计时器等待触发时），配置必须是：

递增模式： $CNT < CCRx \leq ARR$ ($0 < CCRx$)

递减模式： $CNT > CCRx$

单脉冲模式波形是通过写入比较寄存器来定义的（考虑了时钟频率和计数器预分频器）。 t_{DELAY} 由写入 $TIMx_CCR1$ 寄存器的值定义。 t_{PULSE} 由自动重载值和比较值（ $TIMx_ARR - TIMx_CCR1$ ）之差定义。

假设用户想要建立一个从“0”到“1”过渡的波形比较匹配发生，并且当计数器达到自动加载值时，从“1”到“0”的转换。为此，通过在 $TIMx_CCMR1$ 寄存器中写入 $OC1M = 111$ 来启用 PWM 模式 2。用户可以选择通过在 $TIMx_CCMR1$ 寄存器中写入 $OC1PE = 1$ 和在 $TIMx_CR1$ 寄存器中写入 $ARPE$ 来启用预加载寄存器。在这种情况下，必须将比较值写入 $TIMx_CCR1$ 寄存器中，将自动重载值写入 $TIMx_ARR$ 寄存器中，通过将 UG 位置 1 来生成更新并等待 $TI2$ 上的外部触发事件。在此示例中， $CC1P$ 被写入“0”。

在我们的示例中， $TIMx_CR1$ 寄存器中的 DIR 和 CMS 位应为低。用户只需要一个脉冲（单模式），因此必须在 OPM 位的 OPM 位中写入“1”。

$TIMx_CR1$ 寄存器用于在下一个更新事件（计数器翻转时停止计数器）从自动加载值返回到 0）。当 $TIMx_CR1$ 寄存器中的 OPM 位设置为 0 时，因此选择了重复模式。特殊情况： OCx 快速启用：

在单脉冲模式下， TIx 输入上的边沿检测将 CEN 位置 1。启用计数器。然后计数器与比较值之间的比较使输出切换。但是这些操作需要几个时钟周期，这限制了我们可以获得的最小延迟 $t_{DELAY min}$ 。

如果用户希望以最小的延迟输出波形，则必须设置 $TIMx_CCMRx$ 寄存器。然后强制 $OCxRef$ （和 OCx ）响应激励，而不考虑比较。它的新级别个就像比较已经匹配。仅当通道在 PWM1 或 PWM2 中配置时 $OCxFE$ 才起作用模式。

16.1.9.10 编码器接口模式

要选择编码器接口模式，如果计数器仅在 TI2 沿计数，则在 TIMx_SMCR 寄存器中写入 SMS = '001'，如果仅在 TI1 沿计数，则写入 SMS = '010'，如果在 TI2 沿仅计数，则写入 SMS = '011' TI1 和 TI2 边缘。

通过对 TIMx_CCER 寄存器中的 CC1P 和 CC2P 位进行编程，选择 TI1 和 TI2 极性。当需要时，用户也可以对输入过滤器进行编程。两个输入 TI1 和 TI2 用于连接增量编码器。计数器由 TI1FP1 或 TI2FP2 上的每个有效跳变提供时钟（输入滤波器和极性选择后为 TI1 和 TI2，如果未滤波且未反相，则 TI1FP1 = TI1，TI2FP2 = TI2），则假定它被使能（TIMx_CR1 寄存器中的 CEN 位写入 '1'）。评估两个输入的转换顺序，并生成计数脉冲以及方向信号。根据计数器递增或递减的顺序，硬件会相应地修改 TIMx_CR1 寄存器中的 DIR 位。DIR 位是在任何输入（TI1 或 TI2）的每次转换时计算的，无论计数器仅对 TI1，仅对 TI2 还是对 TI1 和 TI2 都进行计数。

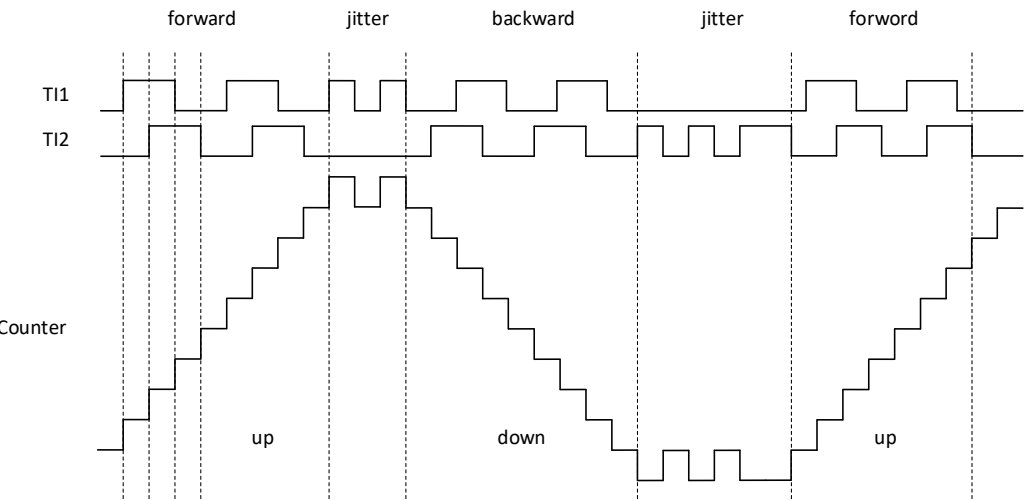
编码器接口模式仅用作带方向的外部时钟选择，这意味着计数器仅在 0 和 TIMx_ARR 寄存器中的自动重载值之间连续计数（0 到 ARR 或 ARR 降低到 0，具体取决于方向）。因此，用户必须在启动之前配置 TIMx_ARR。以相同的方式，捕获，比较，预分频器，重复计数器，触发输出功能继续正常运行。编码器模式和外部时钟模式 2 不兼容，因此不能一起选择。在此模式下，计数器会根据速度和方向自动修改因此，增量编码器及其内容始终代表编码器的位置。计数方向对应于所连接传感器的旋转方向。

表16.2 计数方向和编码信号

Active edge	Level on opposite signal (T1iP1 for T1 2, T12FP2 for T11)	T1iP1 signal		T12FP2 signal	
		Rising	Falling	Rising	Falling
Counting on T11 only	High	Down	Up	No Count	No Count
Counting on T11 only	Low	Up	Down	No Count	No Count
Counting on T12 only	High	No Count	No Count	Up	Down
Counting on T12 only	Low	No Count	No Count	Down	Up
Counting on T11 and T12	High	Down	Up	Up	Down
Counting on T11 and T12	Low	Up	Down	Down	Up

外部增量编码器可以直接连接到 MCU，而无需外部接口逻辑。但是，通常使用比较器将编码器的差分输出转换为数字信号，这大大提高了抗噪能力。指示机械零位的第三个编码器输出可以连接到外部中断输入并触发计数器复位。

图16.26 编码器接口模式



16.1.9.11 霍尔传感器接口 (仅 TIM1)

定时器输入 XOR 功能

TIMx_CR2 寄存器中的 TI1S 位允许将通道 1 的输入滤波器连接到 XOR 门的输出, 并结合三个输入引脚 TIMx_CH1, TIMx_CH2 和 TIMx_CH3。XOR 输出可与所有定时器输入功能一起使用, 例如触发或输入捕捉。

霍尔传感器

使用高级控制定时器 (TIM1) 生成 PWM 信号来驱动电机, 以及另一个定时器 TIMx (TIM3) 称为“接口定时器”。“接口定时器”捕获通过 XOR 连接到 TI1 输入通道 (通过将 TIMx_CR2 寄存器中的 TI1S 位置 1 选择) 的 3 个定时器输入引脚 (TIMx_CH1, TIMx_CH2 和 TIMx_CH3)。

从模式控制器配置为复位模式, 从机输入为 TI1F_ED。因此, 每次 3 个输入之一切换时, 计数器都会从 0 重新开始计数。这将创建一个由霍尔输入的任何变化触发的时基。

在“接口定时器”上, 捕获/比较通道 1 配置为捕获模式, 捕获信号为 TRC。所捕获的值 (对应于输入两次变化之间所经过的时间) 给出了有关电动机速度的信息。

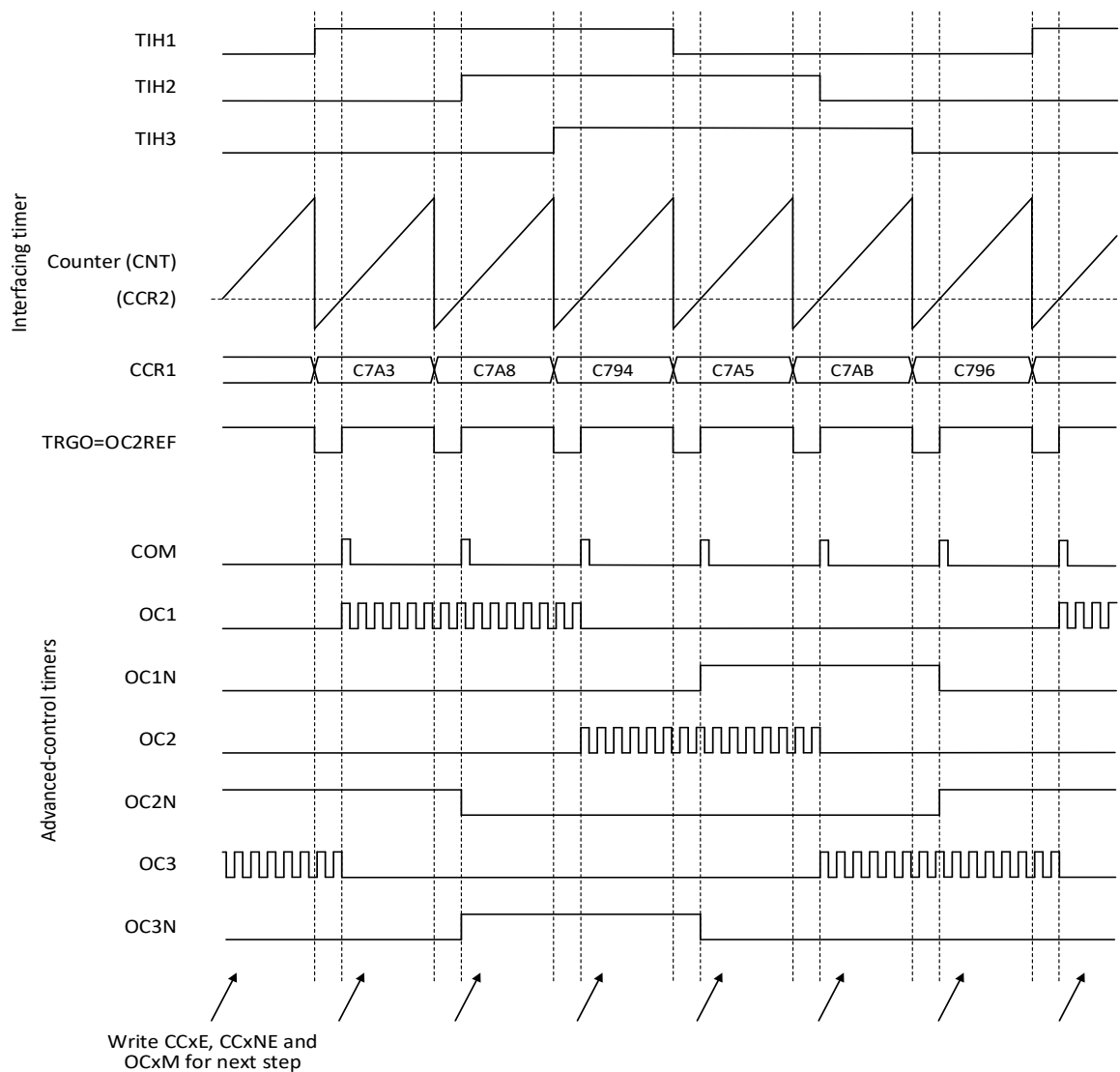
可以在输出模式下使用“接口定时器”来生成一个脉冲, 该脉冲通过触发 COM 事件来更改高级控制定时器 (TIM1) 的通道配置。TIM1 定时器用于生成 PWM 信号以驱动电动机。为此, 必须对接口定时器通道进行编程, 以便在编程的延迟后 (在输出比较或 PWM 模式下) 产生正脉冲。该脉冲被发送到通过 TRGO 输出的高级控制定时器 (TIM1)。

例如: 每当连接到 TIMx 定时器之一的霍尔输入发生变化时, 用户想要更改经过编程延迟的高级控制定时器 TIM1 的 PWM 配置,

- 通过将 TIMx_CR2 寄存器中的 TI1S 位写为 1, 将 3 个定时器输入与 TI1 输入通道进行或运算
- 编程时基: 将 TIMx_ARR 写入最大值 (必须通过 TI1 更改清除计数器。设置预分频器以使最大计数器周期长于传感器两次更改之间的时间
- 在捕获模式下编程通道 1 (选择了 TRC): 将 TIMx_CCMR1 寄存器中的 CC1S 位写入 11。用户还可以根据需要对数字滤波器进行编程
- 以所需的延迟在 PWM 2 模式下对通道 2 进行编程: 将 TIMx_CCMR1 寄存器中的 OC2M 位写入“111”, 将 CC2S 位写入“00”
- 选择 OC2REF 作为 TRGO 上的触发输出: 将 TIMx_CR2 寄存器中的 MMS 位写入“101”

在高级控制定时器 TIM1 中, 必须选择正确的 ITR 输入作为触发输入, 对定时器进行编程以生成 PWM 信号, 捕获/比较控制信号被预加载 (TIMx_CR2 寄存器中的 CCPC = 1), 并且 COM 事件为由触发输入 (TIMx_CR2 寄存器中的 CCUS = 1) 控制。在发生 COM 事件后, 可将 PWM 控制位 (CCxE, OCxM) 写入下一步 (可在 OC2REF 的上升沿生成的中断子程序中完成)。

图16.27 霍尔传感器接口模式



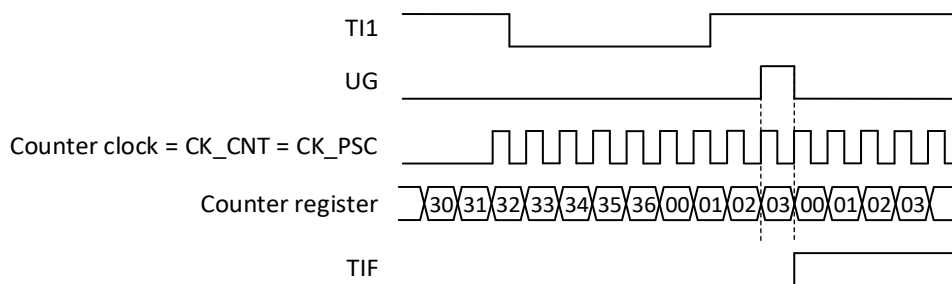
16.1.10 外部触发同步

TIMx 定时器可以在多种模式下与外部触发器同步：复位模式，门控模式和触发模式。

16.1.10.1 从模式：复位模式

计数器及其预分频器可以响应触发输入上的事件而重新初始化。此外，如果 TIMx_CR1 寄存器中的 URS 位为低，则会生成更新事件 UEV。然后，所有预加载的寄存器 (TIMx_ARR, TIMx_CCRx) 都将更新。计数器从内部时钟开始计数，然后正常工作直到 TI1 上升沿。当 TI1 上升时，计数器清零并从 0 重新开始。同时，触发标志置位 (TIMx_SR 寄存器中的 TIF 位) 并产生中断请求，或者如果允许则可以发送 DMA 请求 (取决于 TIMx_DIER 寄存器中的 TIE 和 TDE 位)。

图16.28 从模式下复位模式



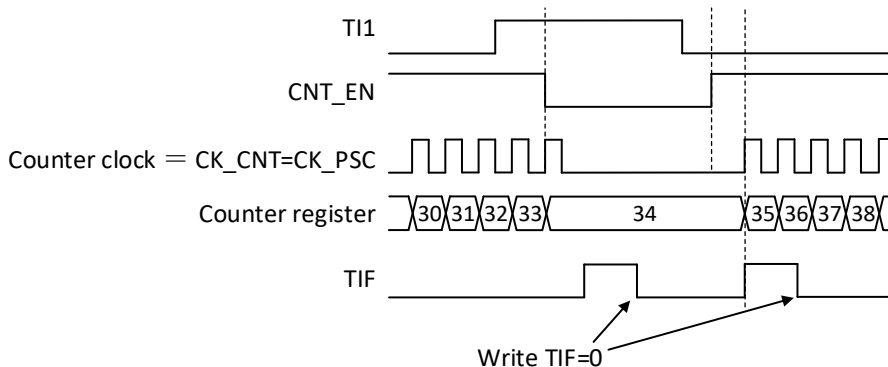
16.1.10.2 从模式下门控模式

可以根据所选输入的电平启用计数器。在以下示例中，仅当 **TI1** 输入为低电平时，递增计数器才会计数：将通道 1 配置为检测 **TI1** 上的低电平。配置输入过滤器的持续时间（在此示例中，我们不需要任何过滤器，因此我们将 **IC1F = 0000** 保留）。捕获预分频器不用于触发，因此用户无需对其进行配置。**CC1S** 位仅选择输入捕捉源，**TIMx_CCMR1** 寄存器中的 **CC1S = 01**。将 **TIMx_CCER** 寄存器中的 **CC1P = 1** 写入以验证极性（仅检测低电平）。

通过在 **TIMx_SMCR** 寄存器中写入 **SMS = 101** 将定时器配置为门控模式。通过在 **TIMx_SMCR** 寄存器中写入 **TS = 101**，选择 **TI1** 作为输入源。

通过在 **TIMx_CR1** 寄存器中写入 **CEN = 1** 来启用计数器（在门控模式下，如果 **CEN = 0**，则无论触发输入电平如何，计数器都不会启动）。只要 **TI1** 为低电平，计数器就开始对内部时钟进行计数，而当 **TI1** 变为高电平时，计数器就停止计数。计数器启动或停止时，**TIMx_SR** 寄存器中的 **TIF** 标志都将置 1。**TI1** 的上升沿与计数器的实际停止之间的延迟是由于 **TI1** 输入上的重新同步电路引起的。

图16.29 从模式下门控模式

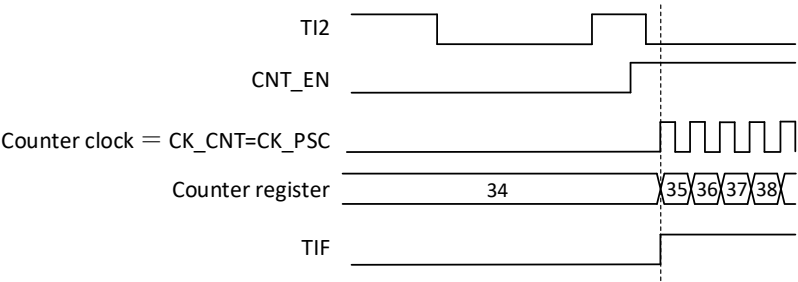


16.1.10.3 从模式下触发模式

计数器可以响应所选输入上的事件而启动。在以下示例中，递增计数器根据 **TI2** 输入上的上升沿而启动：配置通道 2 以检测 **TI2** 上的上升沿。配置输入过滤器的持续时间（在此示例中，我们不需要任何过滤器，因此我们将 **IC2F = 0000** 保持不变）。捕获预分频器不用于触发，因此无需对其进行配置。**CC2S** 位配置为仅选择输入捕捉源，**TIMx_CCMR1** 寄存器中的 **CC2S = 01**。在 **TIMx_CCER** 寄存器中写入 **CC2P = 1** 以验证极性（仅检测低电平）。

通过在 **TIMx_SMCR** 寄存器中写入 **SMS = 110** 将定时器配置为触发模式。通过在 **TIMx_SMCR** 寄存器中写入 **TS = 110**，选择 **TI2** 作为输入源。当 **TI2** 出现上升沿时，计数器开始以内部时钟计数，并且 **TIF** 标志已设置。**TI2** 的上升沿与计数器的实际启动之间的延迟是由于 **TI2** 输入上的重新同步电路。

图16.30 -从模式下触发模式



16.1.10.4 从模式下外部时钟模式 2+触发模式

除了其他从模式外，还可以使用外部时钟模式 2（外部时钟模式 1 和编码器模式除外）。在这种情况下，ETR 信号用作外部时钟输入，并且可以选择另一个输入作为触发输入（在复位模式，门控模式或触发模式下）。建议不要通过 TIMx_SMCR 寄存器的 TS 位选择 ETR 作为 TRGI。

在以下示例中，一旦出现 TI1 的上升沿，则在 ETR 信号的每个上升沿将递增计数：

- 通过如下编程 TIMx_SMCR 寄存器来配置外部触发输入电路：
 - ETF = 0000：无过滤器
 - ETPS = 00：禁用预分频器
 - ETP = 0：检测 ETR 的上升沿，并且 ECE = 1，以启用外部时钟模式 2。
- 如下配置通道 1，以检测 TI 上的上升沿：
 - IC1F = 0000：无过滤器。
 - 捕获预分频器不用于触发，不需要进行配置。
 - TIMx_CCMR1 寄存器中的 CC1S = 01 仅选择输入捕捉源
 - TIMx_CCER 寄存器中的 CC1P = 0 以验证极性（仅检测上升沿）。
- 通过在 TIMx_SMCR 寄存器中写入 SMS = 110 将定时器配置为触发模式。选择通过在 TIMx_SMCR 寄存器中写入 TS = 101，将 TI1 作为输入源。TI1 的上升沿使能计数器并设置 TIF 标志。然后，计数器在 ETR 上升沿计数。ETR 信号的上升沿与计数器的实际复位之间的延迟是由于 ETRP 输入上的重新同步电路引起的。

16.1.11 定时器同步

TIM 定时器在内部链接在一起，用于计时器同步或链接。

注：不同定时器之间存在同步延迟，在功能实现上有延迟。例：使用一个定时器启动另外一个，TRGO 到真正使能 CEN 有 10 个 TIMER_CLK 延迟。

表16.3 ITR 连接

从定时器	ITR0(TS=000)	ITR1(TS=001)	ITR2(TS=010)	ITR3(TS=011)
TIM1	TIM15_TRGO	RESERVED	TIM3_TRGO	TIM17_CC1_EVENT
TIM3	TIM1_TRGO	RESERVED	TIM15_TRGO	TIM14_CC1_EVENT
TIM15	RESERVED	TIM3_TRGO	TIM16_CC1_EVENT	TIM17_CC1_EVENT

图16.31 一个定时器作为另一个的预分频器

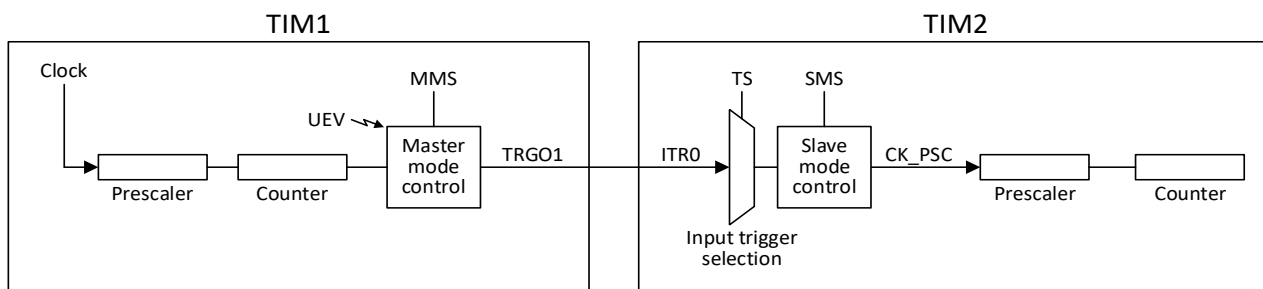


图16.32 一个定时器使能另外一个

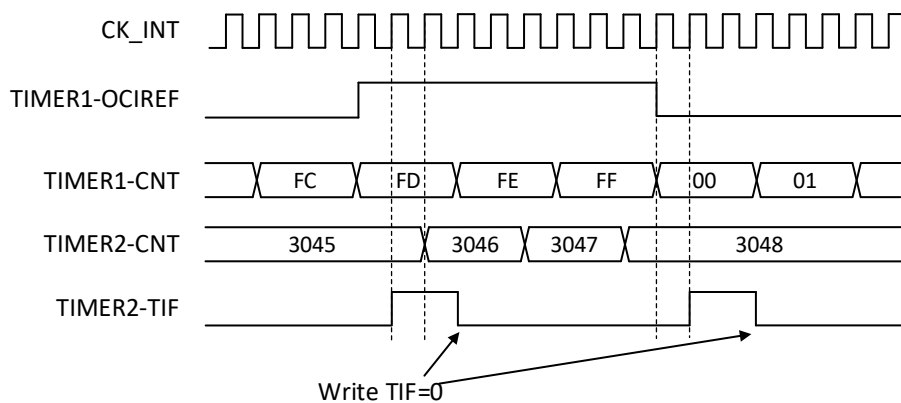


图16.33 一个定时器启动另外一个

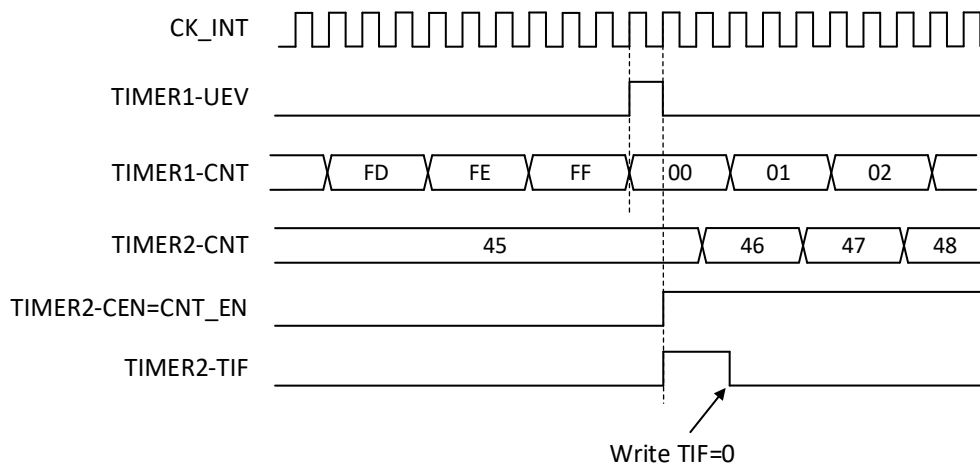
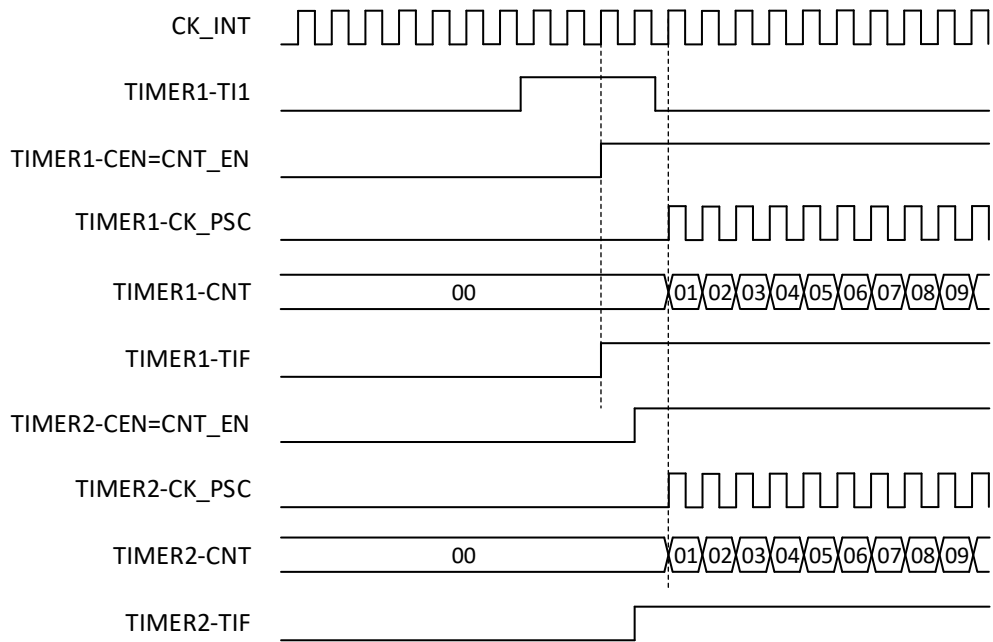


图16.34 针对外部触发同时启动 2 个定时器



16.1.12 调试模式

当微控制器进入调试模式（Cortex®-M0+内核停止）时，TIMx 计数器要么继续正常工作，要么停止，具体取决于 DBG 模块中的 DBG_TIMx_STOP 配置位。为了安全起见，当计数器停止运行（DBGMCU_APBx_FZ 寄存器中的 DBG_TIMx_STOP = 1）时，将禁用输出（就像 MOE 位被复位一样）。可以将输出强制为无效状态（OSSI 位= 1），或将其控制权由 GPIO 控制器（OSSI 位= 0）强制将其强制为高阻状态。

16.2 寄存器

16.2.1 寄存器 (TIMx_CR1)

偏移地址：0x00，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
r						rw		rw	rw		rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:10	Reserved			r	
9:8	CKD	时钟分频此位字段指示定时器时钟 (CK_INT) 频率与死区时间发生器和数字滤波器 (ETR、Tlx) 使用的死区时间和采样时钟 (tDTS) 之间的分频比 00: tDTS=tCK_INT 01: tDTS=2*tCK_INT 10: tDTS=4*tCK_INT 11: 保留，不编程该值	0x0	rw	
7	ARPE	自动重新加载预加载启用 0: TIMx_ARR 寄存器未缓冲 1: TIMx_ARR 寄存器被缓冲	0x0	rw	



Bit	Field	Description	Reset	RW	Remark
6:5	CMS	居中模式选择 00: 边缘对齐模式。计数器根据方向位 (DIR) 向上或向下计数。 01: 居中模式 1。计数器交替计数。输出中配置的通道的输出比较中断标志 (TIMx_CCMRx 寄存器中的 CCxS=00) 仅在计数器向下计数时设置。 10: 居中模式 2。计数器上下交替计数。输出中配置的通道的输出比较中断标志 (TIMx_CCMRx 寄存器中的 CCxS=00) 仅在计数器向上计数时设置。 11: 居中模式 3。计数器上下交替计数。输出中配置的通道的输出比较中断标志 (TIMx_CCMRx 寄存器中的 CCxS=00) 在计数器递增或递减计数时均设置。	0x0	rw	
4	DIR	方向 0: 计数器用作向上计数器 1: 计数器用作向下计数器 注意: 仅当定时器配置为中心对齐模式或编码器模式时, 该位才被读取。	0x0	rw	
3	OPM	一脉冲模式 0: 计数器在更新事件时不停止 1: 计数器在下次更新事件时停止计数 (清除位 CEN)	0x0	rw	
2	URS	更新请求源 该位由软件置位和清零以选择 UEV 事件源。 0: 以下任何事件都会产生更新中断或 DMA 请求。这些事件可以是: 计数器上溢/下溢 设置 UG 位 通过从模式控制器生成更新 1: 只有计数器上溢/下溢才会产生更新中断或 DMA 请求。	0x0	rw	
1	UDIS	更新禁用 该位由软件置位和清零以启用/禁用 UEV 事件生成。 0: UEV 使能。更新 (UEV) 事件由以下事件之一生成: 计数器上溢/下溢 设置 UG 位 通过从模式控制器生成更新 然后缓冲寄存器加载其预加载值。 1: UEV 禁用。不生成更新事件, 影子寄存器保持其值 (ARR、PSC、CCRx)。但是, 如果设置了 UG 位或从从模式控制器接收到硬件复位, 则计数器和预分频器会重新初始化。	0x0	rw	
0	CEN	计数器使能 0: 计数器禁用 1: 计数器使能 注意: 外部时钟、门控模式和编码器模式只有在 CEN 位之前由软件设置后才能工作。但是触发模式可以通过硬件自动设置 CEN 位。	0x0	rw	

16.2.2 寄存器 (TIMx_CR2)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	OIS4	OIS3N	OIS3	OIS2N	OIS2	OIS1N	OIS1	TI1S	MIMS			CCDS	CCUS	Reserved	CCPC
r	rw	rw	rw	Rw	rw	rw	rw	rw	rw			rw	rw	r	rw

Bit	Field	Description	Reset value	RW attribute	Remark
31:15	Reserved			r	
14	OIS4	输出空闲状态 4 (OC4 输出) 参考 OIS1 位	0x0	rw	
13	OIS3N	输出空闲状态 3 (OC3N 输出) 参考 OIS1N 位	0x0	rw	
12	OIS3	输出空闲状态 3 (OC3 输出) 参考 OIS1 位	0x0	rw	
11	OIS2N	输出空闲状态 2 (OC2N 输出)	0x0	rw	
10	OIS2	输出空闲状态 2 (OC2 输出) 参考 OIS1 位	0x0	rw	
9	OIS1N	输出空闲状态 1 (OC1N 输出) 0: 当 MOE=0 时, 死区时间后 OC1N=0 1: 当 MOE=0 时, 死区时间后 OC1N=1 注意: 只要 LOCK 级别 1、2 或 3 已被编程 (TIMx_BDTR 寄存器中的 LOCK 位), 就不能修改该位。	0x0	rw	
8	OIS1	输出空闲状态 1 (OC1 输出) 0: 当 MOE=0 时, OC1=0 (如果实施 OC1N, 则在死区时间之后) 1: 当 MOE=0 时, OC1=1 (如果实施 OC1N, 则在死区时间之后) 注意: 只要 LOCK 级别 1、2 或 3 已被编程 (TIMx_BDTR 寄存器中的 LOCK 位), 就不能修改该位。	0x0	rw	
7	TI1S	TI1 选择 0: TIMx_CH1 引脚连接到 TI1 输入 1: TIMx_CH1、CH2 和 CH3 引脚连接到 TI1 输入 (XOR 组合)	0x0	rw	
6:4	MIMS	主模式选择 这些位允许选择要在主模式下发送到从定时器进行同步 (TRGO) 的信息。组合如下: 000: 复位 - TIMx_EGR 寄存器中的 UG 位用作触发输出 (TRGO)。如果复位是由触发输入 (从模式控制器配置为复位模式) 产生的, 那么 TRGO 上的信号与实际复位相比会延迟。 001: 启用 - 计数器启用信号 CNT_EN 用作触发输出 (TRGO)。同时启动多个定时器或控制启用从属定时器的窗口很有用。当配置为门控模式时, 计数器使能信号由 CEN 控制位和触发输入之间的逻辑或产生。当计数器使能信号由触发输入控制时, TRGO 上存在延迟, 除非选择了主/从模式 (参见 TIMx_SMCR 寄存器中的 MSM 位描述)。 010: 更新 - 更新事件被选为触发输出 (TRGO)。例如, 主定时器然后可以用作从属定时器的预分频器。 011: 比较脉冲 - 一旦发生捕获或比较匹配 (TRGO), 当 CC1IF 标志被设置时 (即使它已经是高电平), 触发输出发送一个正脉冲。 100: 比较 - OC1REF 信号用作触发输出 (TRGO) 101: 比较 - OC2REF 信号用作触发输出 (TRGO) 110: 比较 - OC3REF 信号用作触发输出 (TRGO) 111: 比较 - OC4REF 信号用作触发输出 (TRGO) 注意: 从定时器和 ADC 的时钟必须在从主定时器接收事件之前启用, 并且在从主定时器接收触发时不得即时更改。	0x0	rw	
3	CCDS	捕获/比较 DMA 选择 0: CCx 事件发生时发送 CCx DMA 请求 1: 发生更新事件时发送 CCx DMA 请求	0x0	rw	

Bit	Field	Description	Reset value	RW attribute	Remark
2	CCUS	捕获/比较控制更新选择 0: 当捕捉/比较控制位被预加载 (CCPC=1) 时, 它们仅通过设置 COMG 位来更新 1: 当捕捉/比较控制位被预加载时 (CCPC=1), 它们通过设置 COMG 位或当 TRGI 上出现上升沿来更新	0x0	rw	
1	Reserved		0x0	rw	
0	CCPC	捕获/比较预加载的控件 0: CCxE、CCxNE 和 OCxM 位未预加载 1: 预加载 CCxE、CCxNE 和 OCxM 位, 写入后, 仅在发生换向事件 (COM) 时更新 (COMG 位设置或在 TRGI 上检测到上升沿, 取决于 CCUS 位) 注意: 该位仅作用于具有互补输出的通道。	0x0	rw	

16.2.3 寄存器 (TIMx_SMCR)

偏移地址: 0x08, 复位值: 0x0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS		ETF				MSM	TS			Reserved	SMS		
rw	rw	rw		rw				rw	rw			rw	rw		

Bit	Field	Description	Reset	RW	Remark
31:16	Reserved		0x0	r	
15	ETP	外部触发极性 该位选择是使用 ETR 还是 !ETR 进行触发操作 0: ETR 同相, 高电平或上升沿有效。 1: ETR 反相, 低电平或下降沿有效。	0x0	rw	
14	ECE	外部时钟使能 该位使能外部时钟模式 2。 0: 外部时钟模式 2 禁用 1: 外部时钟模式 2 使能。 计数器由 ETRF 信号上的任何有效边沿计时。	0x0	rw	
13:12	ETPS	外部触发预分频器 外部触发信号 ETRP 频率最多为 TIMxCLK 频率的 1/4。 可以启用预分频器以降低 ETRP 频率。在输入快速外部时钟时很有用。 00: 预分频器关闭 01: ETRP 频率除以 2 10: ETRP 频率除以 4 11: ETRP 频率除以 8	0x0	rw	



Bit	Field	Description	Reset	RW	Remark
11:8	ETF	外部触发滤波器 该位字段然后定义用于采样 ETRP 信号的频率和应用用于 ETRP 的数字滤波器的长度。数字滤波器由事件计数器组成，其中需要 N 个连续事件来验证输出上的转换： 0000: 无过滤器，采样在 fDTS 完成 0001: fSAMPLING=fCK_INT, N=2 0010: fSAMPLING=fCK_INT, N=4 0011: fSAMPLING=fCK_INT, N=8 0100: fSAMPLING=fDTS/2, N=6 0101: fSAMPLING=fDTS/2, N=8 0110: fSAMPLING=fDTS/4, N=6 0111: fSAMPLING=fDTS/4, N=8 1000: fSAMPLING=fDTS/8, N=6 1001: fSAMPLING=fDTS/8, N=8 1010: fSAMPLING=fDTS/16, N=5 1011: fSAMPLING=fDTS/16, N=6 1100: fSAMPLING=fDTS/16, N=8 1101: fSAMPLING=fDTS/32, N=5 1110: fSAMPLING=fDTS/32, N=6 1111: fSAMPLING=fDTS/32, N=8	0x0	rw	
7	MSM	主/从模式 0: 无动作 1: 事件对触发输入 (TRGI) 的影响被延迟，以允许当前定时器与其从属定时器之间完美同步 (通过 TRGO)。如果我们想在单个外部事件上同步多个计时器，这很有用。	0x0	rw	
6:4	TS	触发选择 该位域选择用于同步计数器的触发输入。 000: 内部触发 0 (ITR0) 001: 内部触发 1 (ITR1) 010: 内部触发 2 (ITR2) 011: 内部触发器 3 (ITR3) 100: TI1 边缘检测器 (TI1F_ED) 101: 滤波定时器输入 1 (TI1FP1) 110: 滤波定时器输入 2 (TI2FP2) 111: 外部触发输入 (ETRF) 有关每个定时器的 ITRx 含义的更多详细信息，请参见 表格 。 注意：只有在不使用这些位时 (例如当 SMS=000 时) 才必须更改这些位，以避免在转换时出现错误的边沿检测。	0x0	rw	
3	Reserved		0x0	rw	



Bit	Field	Description	Reset	RW	Remark
2:0	SMS	从机模式选择 选择外部信号时，触发信号 (TRGI) 的有效边与外部输入上选择的极性相连接 (参见输入控制寄存器和控制寄存器说明)。 000: 从模式禁用 - 如果 CEN = '1', 则预分频器直接由内部时钟提供时钟。 001: 编码器模式 1 - 计数器根据 TI1FP2 电平在 TI2FP1 边沿向上/向下计数。 010: 编码器模式 2 - 计数器在 TI1FP2 边沿向上/向下计数，具体取决于 TI2FP1 电平。 011: 编码器模式 3 - 计数器在 TI1FP1 和 TI2FP2 边沿上递增/递减计数，具体取决于另一个输入的电平。 100: 复位模式 - 上升沿 选定的触发输入 (TRGI) 重新初始化计数器并生成寄存器更新。 101: 门控模式 - 当触发输入 (TRGI) 为高电平时启用计数器时钟。一旦触发器变低，计数器就会停止 (但不会重置)。计数器的启动和停止都受到控制。 110: 触发模式 - 计数器在触发 TRGI 的上升沿开始 (但不会复位)。仅控制计数器的启动。 111: 外部时钟模式 1 - 所选触发器 (TRGI) 的上升沿为计数器提供时钟。 注意: 如果选择 TI1F_ED 作为触发输入 (TS='100'), 则不得使用门控模式。实际上, TI1F_ED 为 TI1F 上的每次转换输出 1 个脉冲, 而门控模式检查触发信号的电平。从定时器的时钟必须在从主定时器接收事件之前启用, 并且在从主定时器接收触发时不得即时更改。	0x0	rw	

16.2.4 寄存器 (TIMx_DIER)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	TDE	COMDE	CC4DE	CC3DE	CC2DE	CC1DE	UDE	BIE	TIE	COMIE	CC4IE	CC3IE	CC2IE	CC1IE	UIE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:15	Reserved	必须保持在复位值。		r	
14	TDE	触发 DMA 请求使能 0: 触发 DMA 请求禁用 1: 触发 DMA 请求使能	0x0	rw	
13	COMDE	COM DMA 请求使能 0: COM DMA 请求禁用 1: COM DMA 请求使能	0x0	rw	
12	CC4DE	捕获/比较 4 DMA 请求使能 0: CC4 DMA 请求禁用 1: CC4 DMA 请求使能	0x0	rw	
11	CC3DE	捕获/比较 3 DMA 请求使能 0: CC3 DMA 请求禁用 1: CC3 DMA 请求使能	0x0	rw	
10	CC2DE	捕获/比较 2 DMA 请求使能 0: CC2 DMA 请求禁用 1: CC2 DMA 请求使能	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
9	CC1DE	捕获/比较 1 DMA 请求使能 0: CC2 DMA 请求禁用 1: CC2 DMA 请求使能	0x0	rw	
8	UDE	更新 DMA 请求使能 0: 更新 DMA 请求禁用 1: 更新 DMA 请求使能	0x0	rw	
7	BIE	中断中断使能 0: 中断中断禁用 1: 中断中断使能	0x0	rw	
6	TIE	触发中断使能 0: 触发中断禁用 1: 触发中断使能	0x0	rw	
5	COMIE	COM 中断使能 0: COM 中断禁止 1: COM 中断使能	0x0	rw	
4	CC4IE	捕捉/比较 4 中断使能 0: CC4 中断禁用 1: CC4 中断使能	0x0	rw	
3	CC3IE	捕捉/比较 3 中断使能 0: CC3 中断禁用 1: CC3 中断使能	0x0	rw	
2	CC2IE	捕捉/比较 2 中断使能 0: CC2 中断禁用 1: CC2 中断使能	0x0	rw	
1	CC1IE	捕捉/比较 1 中断使能 0: CC1 中断禁用 1: CC1 中断使能	0x0	rw	
0	UIE	更新中断使能 0: 更新中断禁止 1: 更新中断使能	0x0	rw	

16.2.5 寄存器 (TIMx_SR)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CC4OF	CC3OF	CC3OF	CC1OF	Reserved	BIF	TIF	COMIF	CC4IF	CC3IF	CC2IF	CC1IF	UIF
r			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	wr	wr

Bit	Field	Description	Reset	RW	Remark
31:13	Reserved			r	
12	CC4OF	捕捉/比较 4 捕获标志 参考 CC1OF 描述	0x0	rw	
11	CC3OF	捕捉/比较 3 捕获标志 参考 CC1OF 描述	0x0	rw	
10	CC2OF	捕捉/比较 2 捕获标志 参考 CC1OF 描述	0x0	rw	
9	CC1OF	捕捉/比较 1 捕获标志 仅当相应通道配置为输入捕获模式时, 该标志才由硬件设置。它由软件通过将其写入“0”来清除。 0: 未检测到过捕获。 1: 计数器值已被捕获到 TIMx_CCR1 寄存器中, 而 CC1IF 标志已设置	0x0	rw	
8	Reserved		0x0	rw	



Bit	Field	Description	Reset	RW	Remark
7	BIF	中断中断标志 一旦中断输入变为活动状态，该标志就由硬件设置。 如果中断输入未激活，它可以由软件清除。 0：未发生中断事件。 1：在断点输入上检测到活动电平。	0x0	rw	
6	TIF	触发中断标志 该标志由硬件在触发事件时设置（当从模式控制器在除门控模式之外的所有模式下启用时，在 TRGI 输入上检测到活动边沿，在选择门控模式时两个边沿）。它被软件清除。 0：未发生触发事件。 1：触发中断挂起。	0x0	rw	
5	COMIF	COM 中断标志 该标志由硬件在 COM 事件上设置（当捕获/比较控制位 - CCxE、CCxNE、OCxM - 已更新）。由软件清零。 0：未发生 COM 事件。 1：COM 中断挂起。	0x0	rw	
4	CC4IF	捕捉/比较 4 中断标志 参考 CC1IF 说明	0x0	rw	
3	CC3IF	捕捉/比较 3 中断标志 参考 CC1IF 说明	0x0	rw	
2	CC2IF	捕捉/比较 2 中断标志 参考 CC1IF 说明	0x0	rw	
1	CC1IF	捕捉/比较 4 中断标志 如果通道 CC1 配置为输出：当计数器与比较值匹配时，该标志由硬件置位，但在中心对齐模式下除外（请参阅 TIMx_CR1 寄存器描述中的 CMS 位）。由软件清零。 0：不匹配。 1：计数器 TIMx_CNT 的内容与 TIMx_CCR1 寄存器的内容匹配。当 TIMx_CCR1 的内容大于 TIMx_ARR 的内容时，CC1IF 位在计数器溢出（在向上计数和向上/向下计数模式下）或下溢（在向下计数模式下）时变为高电平 如果通道 CC1 配置为输入：该位由硬件在捕获时设置。它由软件或通过读取 TIMx_CCR1 寄存器清零。 0：未发生输入捕捉 1：计数器值已在 TIMx_CCR1 寄存器中捕获（在 IC1 上检测到与所选极性匹配的边沿）	0x0	rw	
0	UIF	更新中断标志 该位由硬件在更新事件时设置。由软件清零。 0：未发生更新。 1：更新中断挂起。 当寄存器更新时，该位由硬件设置： 关于重复计数器值的上溢或下溢（如果重复计数器值更新 = 0）并且如果 TIMx_CR1 寄存器中的 UDIS=0。 当 CNT 由软件使用 TIMx_EGR 寄存器中的 UG 位重新初始化时，如果 TIMx_CR1 寄存器中的 URS=0 和 UDIS=0。 当 CNT 被触发事件重新初始化时，如果 TIMx_CR1 寄存器中的 URS=0 和 UDIS=0。	0x0	rw	

16.2.6 寄存器 (TIMx_EGR)

偏移地址：0x14，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								BG	TG	COMG	CC4G	CC3G	CC2G	CC1G	UG
r								rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:8	Reserved			r	
7	BG	断代 该位由软件置位以产生事件，由硬件自动清零。 0: 无动作 1: 产生中断事件。 MOE 位清零并设置 BIF 标志。 如果使能，可能会发生相关的中断或 DMA 传输。	0x0	rw	
6	TG	触发生成 该位由软件置位以产生事件，由硬件自动清零。 0: 无动作 1: 在 TIMx_SR 寄存器中设置 TIF 标志。 如果使能，可能会发生相关的中断或 DMA 传输。	0x0	rw	
5	COMG	捕获/比较控件更新生成 该位可由软件设置，由硬件自动清零 0: 无动作 1: 当 CCPC 位置 1 时，它允许更新 CCxE、CCxNE 和 OCxM 位 注：该位仅作用于具有互补输出的通道。	0x0	rw	
4	CC4G	捕获/比较 4 代 参考 CC1G 说明	0x0	rw	
3	CC3G	捕获/比较 3 代 参考 CC1G 说明	0x0	rw	
2	CC2G	捕获/比较 2 代 参考 CC1G 说明	0x0	rw	
1	CC1G	捕获/比较 1 代 该位由软件置位以产生事件，由硬件自动清零。 0: 无动作 1: 在通道 1 上产生捕获/比较事件：如果通道 CC1 配置为输出：设置 CC1IF 标志，如果使能，则发送相应的中断或 DMA 请求。 如果通道 CC1 配置为输入：计数器的当前值在 TIMx_CCR1 寄存器中捕获。 CC1IF 标志置位，如果使能，则发送相应的中断或 DMA 请求。 如果 CC1IF 标志已经为高，则设置 CC1OF 标志。	0x0	rw	
0	UG	更新生成 该位可由软件设置，由硬件自动清零。 0: 无动作 1: 重新初始化计数器并生成寄存器更新。 请注意，预分频计数器也被清除（无论如何预分频比不受影响）。 如果选择了中心对齐模式或 DIR=0（向上计数），则计数器被清零，否则，如果 DIR=1（向下计数），它将采用自动重载值 (TIMx_ARR)。	0x0	rw	

16.2.7 寄存器 (TIMx_CCMR1)

偏移地址：0x18，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC2CE	OC2M			OC2PE	OC2FE	CC2S		OC1CE	OC1M			OC1PE	OC1FE	CC1S	
rw	rw			rw	rw	rw		rw	rw			rw	rw	rw	

Bit	Field	Description	Reset	RW	Remark
31:16	Reserved		0x0	r	
15	OC2CE	输出比较 2 清除使能	0x0	rw	
14:12	OC2M	输出比较 2 模式	0x0	rw	
11	OC2PE	输出比较 2 预加载使能	0x0	rw	
10	OC2FE	输出比较 2 快速启用	0x0	rw	
9:8	CC2S	该位域定义了通道（输入/输出）的方向以及使用的输入。 00: CC2 通道配置为输出 01: CC2 通道配置为输入, IC2 映射到 TI2 10: CC2 通道配置为输入, IC2 映射到 TI1 11: CC2 通道配置为输入, IC2 映射到 TRC。 仅当通过 TS 位 (TIMx_SMCR 寄存器) 选择内部触发输入时, 此模式才有效 注: CC2S 位仅在通道关闭 (TIMx_CCER 中的 CC2E = '0') 时可写。	0x0	rw	
7	OC1CE	OC1CE: 输出比较 1 清除使能 0: OC1Ref 不受 ETRF 输入的影响 1: 一旦在 ETRF 输入上检测到高电平, OC1Ref 就被清除	0x0	rw	



Bit	Field	Description	Reset	RW	Remark
6:4	OC1M	这些位定义了输出参考信号 OC1REF 的行为, OC1 和 OC1N 来自该信号。 OC1REF 为高电平有效, 而 OC1 和 OC1N 有效电平取决于 CC1P 和 CC1NP 位。 000: 冻结 - 输出比较寄存器 TIMx_CCR1 和计数器 TIMx_CNT 之间的比较对输出没有影响。(此模式用于生成时序基准)。 001: 匹配时将通道 1 设置为活动电平。当计数器 TIMx_CNT 匹配捕获/比较寄存器 1 (TIMx_CCR1) 时, OC1REF 信号被强制为高电平。 010: 在匹配时将通道 1 设置为非活动电平。当计数器 TIMx_CNT 与捕获/比较寄存器 1 (TIMx_CCR1) 匹配时, OC1REF 信号被强制为低电平。 011: 切换 - 当 TIMx_CNT=TIMx_CCR1 时, OC1REF 切换。 100: 强制无效电平 - OC1REF 被强制为低电平。 101: 强制激活电平 - OC1REF 被强制为高电平。 110: PWM 模式 1 - 在递增计数中, 只要 TIMx_CNT<TIMx_CCR1 否则通道 1 处于活动状态。在递减计数中, 只要 TIMx_CNT>TIMx_CCR1 处于活动状态 (OC1REF='1'), 通道 1 就处于非活动状态 (OC1REF='0')。 111: PWM 模式 2 - 在递增计数中, 只要 TIMx_CNT<TIMx_CCR1else 有效, 通道 1 就无效。在递减计数中, 只要 TIMx_CNT>TIMx_CCR1 否则通道 1 处于活动状态。 注: 1: 只要 LOCK 级别 3 已编程 (TIMx_BDTR 寄存器中的 LOCK 位) 且 CC1S='00' (通道配置为输出), 就不能修改这些位。 2: 在 PWM 模式 1 或 2 下, OCREF 电平仅在比较结果发生变化或输出比较模式从“冻结”模式切换到“PWM”模式时发生变化。	0x0	rw	
3	OC1PE	输出比较 1 预加载使能 0: 禁用 TIMx_CCR1 上的预加载寄存器。TIMx_CCR1 可以随时写入, 新值会立即被考虑在内。 1: 使能 TIMx_CCR1 上的预加载寄存器。读/写操作访问预加载寄存器。TIMx_CCR1 预加载值在每次更新事件时加载到活动寄存器中。 注: 1: 只要 LOCK 级别 3 已编程 (TIMx_BDTR 寄存器中的 LOCK 位) 且 CC1S='00' (通道配置为输出), 就不能修改这些位。 2: PWM 模式可以在不验证预加载寄存器的情况下仅在一个脉冲模式下使用 (在 TIMx_CR1 寄存器中设置 OPM 位)。否则无法保证行为	0x0	rw	
2	OC1FE	该位用于加速事件对 CC 输出输入触发的影响。 0: CC1 的正常行为取决于计数器和 CCR1 值, 即使在触发器为 ON 时也是如此。当触发输入上出现边沿时激活 CC1 输出的最小延迟为 5 个时钟周期。 1: 触发输入上的有效边沿相当于 CC1 输出上的比较匹配。然后, OC 被设置为独立于比较结果的比较电平。采样触发输入和激活 CC1 输出的延迟减少到 3 个时钟周期。仅当通道配置为 PWM1 或 PWM2 模式时, OCFE 才起作用。	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
1:0	CC1S	捕获/比较 1 个选择 该位域定义了通道 (输入/输出) 的方向以及使用的输入。 00: CC1 通道配置为输出 01: CC1 通道配置为输入, IC1 映射到 TI1 10: CC1 通道配置为输入, IC1 映射到 TI2 11: CC1 通道配置为输入, IC1 映射到 TRC。 仅当通过 TS 位 (TIMx_SMCR 寄存器) 选择内部触发输入时, 此模式才有效 注: CC1S 位仅在通道关闭 (TIMx_CCER 中的 CC1E = '0') 时可写。	0x0	rw	

16.2.8 寄存器 (TIMx_CCMR2)

偏移地址: 0x1C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IC4F				IC4PSC		CC4S		IC3F				IC3PSC		CC3S	
rw				rw		rw		rw				rw		rw	

Bit	Field	Description	Reset value	RW attribute	Remark
31:12	IC4F	输入捕获 4 过滤器	0x0	rw	
11:10	IC4PSC	输入捕捉 4 预分频器	0x0	rw	
9:8	CC4S	CC4S: 捕获/比较 4 选择 此位字段定义通道 (输入/输出) 的方向以及使用的输入。 00: CC4 通道配置为输出 01: CC4 通道配置为输入, IC4 映射到 TI4 10: CC4 通道配置为输入, IC4 映射到 TI3 11: CC4 通道配置为输入, IC4 映射到 TRC。 仅当通过 TS 位 (TIMx_SMCR 寄存器) 选择内部触发输入时, 此模式才有效 注: CC4S 位仅在通道关闭 (TIMx_CCER 中的 CC4E = '0') 时可写。	0x0	rw	
7:4	IC3F	输入捕获 3 过滤器	0x0	rw	
3:2	IC3PSC	输入捕捉 3 预分频器	0x0	rw	
1:0	CC3S	捕获/比较 3 个选择 该位域定义了通道 (输入/输出) 的方向以及使用的输入。 00: CC3 通道配置为输出 01: CC3 通道配置为输入, IC3 映射到 TI3 10: CC3 通道配置为输入, IC3 映射到 TI4 11: CC3 通道配置为输入, IC3 映射到 TRC。 仅当通过 TS 位 (TIMx_SMCR 寄存器) 选择内部触发输入时, 此模式才有效 注: CC3S 位仅在通道关闭 (TIMx_CCER 中的 CC3E = '0') 时可写。	0x0	rw	

16.2.9 寄存器 (TIMx_CCER)

偏移地址: 0x20, 复位值: 0x0000 0000。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E	
r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW	Remark
31:14	Reserved	必须保持在复位值。		r	
13	CC4P	捕获/比较 4 输出极性 参考 CC1P 说明	0x0	rw	
12	CC4E	捕获/比较 4 输出使能 参考 CC1E 说明	0x0	rw	
11	CC3NP	捕获/比较 3 互补输出极性 参考 CC1NP 说明	0x0	rw	
10	CC3NE	捕获/比较 3 互补输出使能 参考 CC1NE 说明	0x0	rw	
9	CC3P	捕获/比较 3 输出极性 参考 CC1P 说明	0x0	rw	
8	CC3E	捕获/比较 3 输出使能 参考 CC1E 说明	0x0	rw	
7	CC2NP	捕获/比较 2 互补输出极性 参考 CC1NP 说明	0x0	rw	
6	CC2NE	捕获/比较 2 互补输出使能 参考 CC1NE 说明	0x0	rw	
5	CC2P	捕获/比较 2 输出极性 参考 CC1P 说明	0x0	rw	
4	CC2E	捕获/比较 2 输出使能 参考 CC1E 说明	0x0	rw	
3	CC1NP	捕获/比较 1 互补输出极性 0: OC1N 高电平有效。 1: OC1N 低电平有效。 注意: 一旦 LOCK 级别 2 或 3 已被编程 (TIMx_BDTR 寄存器中的 LOCK 位) 且 CC1S="00" (通道配置为输出), 则该位不可写。	0x0	rw	
2	CC1NE	捕获/比较 1 个互补输出使能 0: 关闭 - OC1N 未激活。OC1N 电平是 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位的函数。 1: On - OC1N 信号根据 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位在相应的输出引脚上输出。	0x0	rw	
1	CC1P	捕获/比较 1 输出极性 CC1 通道配置为输出: 0: OC1 高电平有效 1: OC1 低电平有效 CC1 通道配置为输入: 根据 IC1 上升沿或者 IC1 下降沿采集。 0: 非反相: 捕捉在 IC1 的上升沿完成。当用作外部触发器时, IC1 是非反相的。 1: 反转: 捕捉在 IC1 的下降沿完成。当用作外部触发器时, IC1 反相。 注意: 一旦 LOCK 级别 2 或 3 被编程 (TIMx_BDTR 中的 LOCK 位)	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
0	CC1E	捕获/比较 1 输出使能 CC1 通道配置为输出： 0: 关闭 - OC1 未激活。OC1 电平是 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1NE 位的函数。 1: 开 - OC1 信号根据 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1NE 位在相应的输出引脚上输出。	0x0	rw	

16.2.10 寄存器 (TIMx_CNT)

偏移地址: 0x24, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNT															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT															
rw															

Bit	Field	Description	Reset value	RW attribute	Remark
31:0	CNT	计数器值	0x0	rw	

16.2.11 寄存器 (TIMx_PSC)

偏移地址: 0x28, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC															
rw															

Bit	Field	Description	Reset	RW	Remark
31:16	Reserved		0x0	r	
15:0	PSC	预分频值	0x0	rw	

16.2.12 寄存器 (TIMx_ARR)

偏移地址: 0x2C, 复位值: 0xFFFF 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ARR															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	ARR	自动重载值 ARR 是要加载到实际自动重载寄存器中的值。	0x0	rw	

16.2.13 寄存器 (TIMx_RCR)

偏移地址: 0x30, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								REP							
r								rw							

Bit	Field	Description	Reset	RW	Remark
31:8	Reserved		0x0	rw	
7:0	REP	重复计数器值。 这些位允许用户在预加载寄存器启用时设置比较寄存器的更新速率（即从预加载到活动寄存器的定期传输），以及更新中断生成速率（如果该中断启用）。 每次与 REP_CNT 相关的递减计数器达到零时，都会生成一个更新事件，并从 REP 值重新开始计数。 由于 REP_CNT 仅在重复更新事件 U_RC 时重新加载 REP 值，因此在下一次重复更新事件之前不会考虑对 TIMx_RCR 寄存器的任何写入。 这意味着在 PWM 模式下 (REP+1) 对应于： -边沿对齐模式下的 PWM 周期数 -中心对齐模式下半个 PWM 周期数	0x0	rw	

16.2.14 寄存器 (TIMx_CCR1)

偏移地址：0x34，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR1															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	CCR1	捕获/比较 1 值 如果通道 CC1 配置为输出： CCR1 是要加载到实际捕获/比较 1 寄存器中的值（预加载值）。 如果未在 TIMx_CCMR1 寄存器（位 OC1PE）中选择预加载功能，则会永久加载它。 否则，当更新事件发生时，预加载值被复制到活动捕获/比较 1 寄存器中。 主动捕获/比较寄存器包含要与计数器 TIMx_CNT 进行比较并在 OC1 输出上发出信号的值。 如果通道 CC1 配置为输入： CCR1 是由最后一个输入捕获 1 事件 (IC1) 传输的计数器值。 TIMx_CCR1 寄存器是只读的，不能编程。	0x0	rw	

16.2.15 寄存器 (TIMx_CCR2)

偏移地址：0x38，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR2															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR2															
rw															



Bit	Field	Description	Reset	RW	Remark
31:0	CCR2	捕获/比较 2 值 如果通道 CC2 配置为输出： CCR2 是要加载到实际捕获/比较 2 寄存器中的值（预加载值）。如果未在 TIMx_CCMR2 寄存器（位 OC2PE）中选择预加载功能，则它会被永久加载。否则，当更新事件发生时，预加载值被复制到主动捕获/比较 2 寄存器中。主动捕获/比较寄存器包含要与计数器 TIMx_CNT 进行比较并在 OC2 输出上发出信号的值。 如果通道 CC2 配置为输入： CCR2 是由最后一个输入捕获 2 事件 (IC2) 传输的计数器值。TIMx_CCR2 寄存器是只读的，不能编程	0x0	rw	

16.2.16 寄存器 (TIMx_CCR3)

偏移地址：0x3C，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR3															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	3	1	0
CCR3															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	CCR3	捕获/比较 3 值 如果通道 CC3 配置为输出： CCR3 是要加载到实际捕获/比较 3 寄存器中的值（预加载值）。如果未在 TIMx_CCMR3 寄存器（位 OC3PE）中选择预加载功能，则它会被永久加载。否则，当更新事件发生时，预加载值被复制到主动捕获/比较 3 寄存器中。主动捕获/比较寄存器包含要与计数器 TIMx_CNT 进行比较并在 OC3 输出上发出信号的值。 如果通道 CC3 配置为输入： CCR3 是由最后一个输入捕获 3 事件 (IC3) 传输的计数器值。TIMx_CCR3 寄存器是只读的，不能编程	0x0	rw	

16.2.17 寄存器 (TIMx_CCR4)

偏移地址：0x40，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR4															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR4															
rw															

Bit	Field	Description	Reset	RW	Remark
31:0	CCR4	捕获/比较 4 值 如果通道 CC4 配置为输出： CCR4 是要加载到实际捕获/比较 4 寄存器中的值（预加载值）。如果未在 TIMx_CCMR4 寄存器（位 OC4PE）中选择预加载功能，则它会被永久加载。否则，当更新事件发生时，预加载值被复制到主动捕获/比较 4 寄存器中。主动捕获/比较寄存器包含要与计数器 TIMx_CNT 进行比较并在 OC4 输出上发出信号的值。 如果通道 CC4 配置为输入： CCR4 是由最后一个输入捕获 4 事件 (IC4) 传输的计数器值。TIMx_CCR4 寄存器是只读的，不能编程	0x0	rw	

16.2.18 寄存器 (TIMx_BDTR4)

偏移地址: 0x44, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK		DTG							
rw	rw	rw	rw	rw	rw	rw		rw							

Bit	Field	Description	Reset	RW	Remark
31:15	Reserved		0x0	r	
15	MOE	主输出使能。 一旦中断输入有效, 该位就由硬件异步清零。它由软件设置或根据 AOE 位自动设置。它仅作用于在输出中配置的通道。 0: OC 和 OCN 输出禁用或强制为空闲状态。 1: 如果 OC 和 OCN 输出各自的使能位被设置 (TIMx_CCER 寄存器中的 CCxE、CCxNE), 则使能 OC 和 OCN 输出。	0x0	rw	
14	AOE	自动输出使能 0: MOE 只能通过软件设置 1: MOE 可由软件设置或在下一次更新事件时自动设置 (如果中断输入未激活) 注意: 只要 LOCK 级别 1 已被编程 (TIMx_BDTR 寄存器中的 LOCK 位), 就不能修改该位。	0x0	rw	
13	BKP	断开极性 0: 中断输入 BRK 为低电平有效 1: 中断输入 BRK 为高电平有效 注意: 只要 LOCK 级别 1 已被编程 (TIMx_BDTR 寄存器中的 LOCK 位), 就不能修改该位。 注意: 对该位的任何写操作都需要 1 个 APB 时钟周期的延迟才能生效。	0x0	rw	
12	BKE	中断使能 0: 中断输入 (BRK 和 CSS 时钟故障事件) 禁用 1: 中断输入 (BRK 和 CSS 时钟故障事件) 已启用 注意: 当 LOCK 级别 1 已编程 (TIMx_BDTR 寄存器中的 LOCK 位) 时, 无法修改该位。 注意: 对该位的任何写操作都需要 1 个 APB 时钟周期的延迟才能生效。	0x0	rw	
11	OSSR	运行模式的关闭状态选择 当 MOE=1 在具有互补输出且配置为输出的通道上使用该位。如果定时器中没有实现互补输出, 则不会实现 OSSR。有关详细信息, 请参阅 OC/OCN 使能描述 (TIM1 捕获/比较使能寄存器 (TIMx_CCER))。 0: 无效时, OC/OCN 输出禁用 (OC/OCN 使能输出信号=0)。 1: 当处于非活动状态时, 一旦 CCxE=1 或 CCxNE=1, OC/OCN 输出就会以其非活动电平启用。然后, OC/OCN 使能输出信号=1 注意: 一旦 LOCK 级别 2 被编程 (TIMx_BDTR 寄存器中的 LOCK 位), 就不能修改该位。	0x0	rw	

Bit	Field	Description	Reset	RW	Remark
10	OSSI	空闲模式的关闭状态选择 该位在配置为输出的通道上 MOE=0 时使用。 有关更多详细信息，请参阅 OC/OCN 使能说明 (TIM1 捕获/比较使能寄存器 (TIMx_CCER))。 0: 无效时，OC/OCN 输出禁用 (OC/OCN 使能输出信号=0)。 1: 当未激活时，一旦 CCxE=1 或 CCxNE=1, OC/OCN 输出首先被强制为其空闲电平。OC/OCN 使能输出信号=1) 注意：一旦 LOCK 级别 2 被编程 (TIMx_BDTR 寄存器中的 LOCK 位)，就不能修改该位。	0x0	rw	
9:8	LOCK	锁定配置。 这些位提供针对软件错误的写保护。 00: 锁定关闭 - 没有位被写保护。 01: LOCK Level 1 = TIMx_BDTR 寄存器中的 DTG 位、TIMx_CR2 寄存器中的 OISx 和 OISxN 位以及 TIMx_BDTR 寄存器中的 BKE/BKP/AOE 位不能再写入。 10: LOCK Level 2 = LOCK Level 1 + CC 极性位 (TIMx_CCER 寄存器中的 CCxP/CCxNP 位，只要相关通道通过 CCxS 位配置为输出) 以及 OSSR 和 OSSI 位不能再写入。 11: 锁定级别 3 = 锁定级别 2 + CC 控制位 (TIMx_CCMRx 寄存器中的 OCxM 和 OCxPE 位，只要相关通道通过 CCxS 位配置为输出) 不能再写入。 注意：LOCK 位只能在复位后写入一次。一旦 TIMx_BDTR 寄存器被写入，它们的内容就会被冻结，直到下一次复位。	0x0	rw	
7:0	DTG	DTG[7:0]: 死区发生器设置 该位域定义了互补输出之间插入的死区持续时间。DT 对应于这个持续时间。 DTG[7:5]=0xx => DT=DTG[7:0]x tdtg, tdtg=tDTS。 DTG[7:5]=10x => DT=(64+DTG[5:0])xtdtg, Tdtg=2xtDTS。 DTG[7:5]=110 => DT=(32+DTG[4:0])xtdtg, 其中 Tdtg=8xtDTS。 DTG[7:5]=111 => DT=(32+DTG[4:0])xtdtg, 其中 Tdtg=16xtDTS。 例如，如果 TDTS=125ns (8MHz)，死区时间可能值为： 0 到 15875 ns, 125 ns 步长, 16 us 到 31750 ns, 250 ns 步长, 32 us 到 63us, 1 us 步长, 64 us 到 126 us, 2 us 步长 注意：只要 LOCK 级别 1、2 或 3 已被编程 (TIMx_BDTR 寄存器中的 LOCK 位)，就不能修改该位字段。	0x0	rw	

16.2.19 寄存器 (TIMx_DCR)

偏移地址: 0x48, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DBL				Reserved				DBA			
rw				rw				rw				rw			

Bit	Field	Description	Reset	RW	Remark
31:13	Reserved		0x0	r	

Bit	Field	Description	Reset	RW	Remark
12:8	DBL	这个 5 位向量定义了 DMA 传输的次数（当对 TIMx_DMAR 寄存器地址执行读或写访问时，定时器检测到突发传输）。 注：DMA burst 只针对更新事件 TIMx_DMAR 地址 00000: 1 次传输 00001: 1 次传输 00010: 2 次传输 ... 10001: 17 次传输	0x0	rw	
7:5	Reserved		0x0	rw	
4:0	DBA	DMA 基址 这个 5 位向量定义了 DMA 传输的基地址（当读/写访问通过 TIMx_DMAR 地址完成时）。DBA 定义为从 TIMx_CR1 寄存器地址开始的偏移量。 例子： 00000: TIMx_CR1, 00001: TIMx_CR2, 00010: TIMx_SMCR, ... 示例：让我们考虑以下传输：DBL = 7 传输和 DBA = TIMx_CR1。在这种情况下，从 TIMx_CR1 地址开始的 7 个寄存器之间进行传输。	0x0	rw	

16.2.20 寄存器 (TIMx_DMAR)

偏移地址：0x4C，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMAB[31:16]															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DMAB[15:0]															
rw															

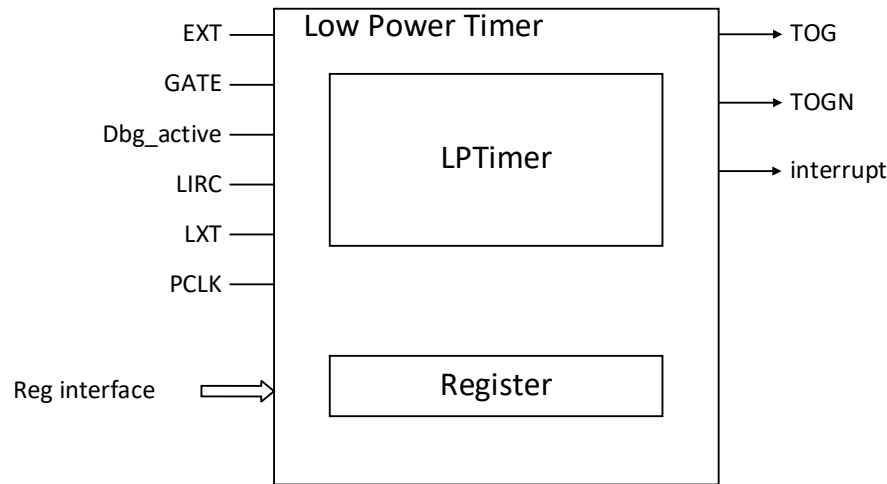
Bit	Field	Description	Reset	RW	Remark
31:0	DMAB	用于突发访问的 DMA 寄存器 对 DMAR 寄存器的读或写操作访问位于地址 (TIMx_CR1 地址) + (DBA + DMA 索引) x 4 处的寄存器，其中 TIMx_CR1 地址是控制寄存器 1 的地址，DBA 是 TIMx_DCR 中配置的 DMA 基地址 寄存器，DMA 索引由 DMA 传输自动控制，范围从 0 到 DBL (在 TIMx_DCR 中配置的 DBL)。	0x0	rw	

17 低功耗定时器 (LPTIM)

17.1 低功耗定时器(LPTIM)

LPTIM 是异步 16 位定时/计数器，在系统时钟关闭后仍然可以通过内部低速 RC 时钟或者外部低速晶体振荡时钟计时/计数，通过中断在低功耗模式下唤醒系统。

图17.1 LPTIMER 结构框图



17.1.1 LPTIM 描述

LPTIM 具有独立的控制启动信号、外部输入时钟和门控信号。

LPTIM 使用 EXT、GATE 来进行计数功能，EXT 用于计数器的外部输入时钟信号，GATE 为有效电平计数使能信号。

LPTIM 的定时器支持两种工作模式，通过设置定时器控制寄存器(LPTIM_CR)中 MODE 选择工作模式。

- 模式 1 为 16 位自由计数模式，计数器从 LPTIM_LOAD 设定的值开始计数，溢出后计数值从 0x0000 开始重新计数。
- 模式 2 为 16 位重载模式，LPTIM 启动时会自动装载重载寄存器 LPTIM_LOAD 的值到计数器中，当溢出后会自动装载 Period LOAD 的值到计数器中。用户写 LPTIM_LOAD 或者 LPTIM_BGLOAD 都会更新 Period LOAD 的值，Period LOAD 的值为用户最近一次更新 LPTIM_LOAD 或 LPTIM_BGLOAD 的值。

LPTIM 可选三种时钟作为定时器时钟，通过控制寄存器 LPTIM_CR.TCK_SEL 来选择。默认选择 PCLK。时钟选择如表：

TCK_SEL	00	01	10	11
定时器时钟	PCLK	PCLK	LXT	LIRC
读定时器计数值	读经过同步	无同步	读经过同步	读经过同步

当选择相应的时钟源，然后设定 TCK_EN 为 1 后可以打开计数器的计数时钟源。当时钟源选择并打开后，设置对应定时器的 TIM_RUN 为 1 后定时器开始运行。

对于模式 1 和模式 2，如果设置 LOAD 值，任何时刻，计数值会被立即更新为 LOAD 值，计数器从 LOAD 值开始重新计数。设置 LOAD 值的优先级高于其他计数器被更新的优先级。

在模式 2，设定 BGLOAD 的值，只有在计数器发生溢出后，设定值才会被更新到计数器。

模式 1：如果没有设置 LOAD 值，计数器从 0 开始计数，计数到最大 0xFFFF 后产生溢出中断。计数器计数到最大 0xFFFF 后，计数器再次从 0 开始计数。

模式 2：如果没有设置 LOAD 值，计数器从 0 开始计数，计数到最大 0xFFFF 后产生溢出中断。计数器计数到最大 0xFFFF 后，计数值会被更新为 Period LOAD 的值，然后向上计数。

图17.2 LPTIMER 模式 1

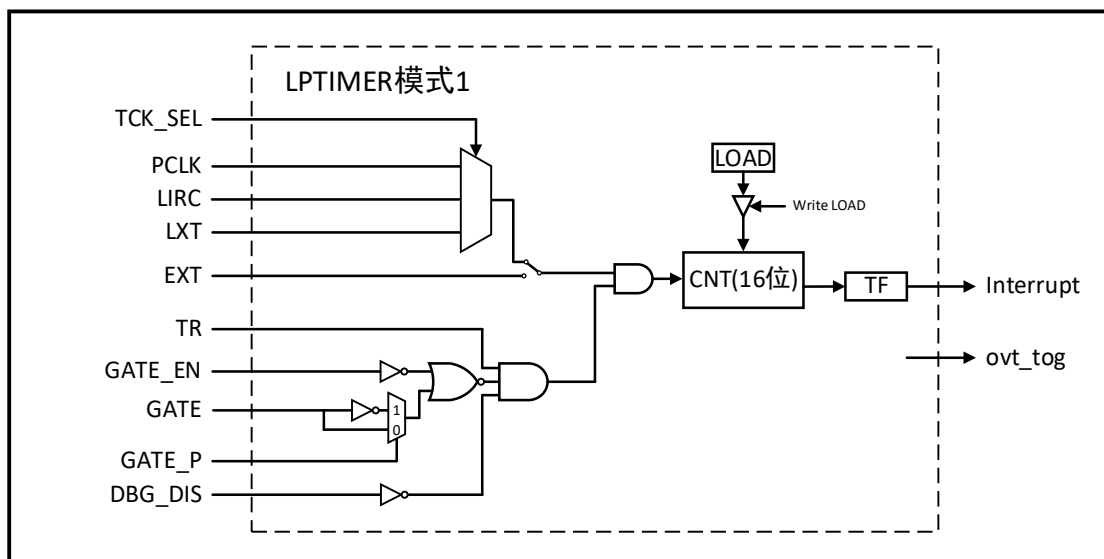
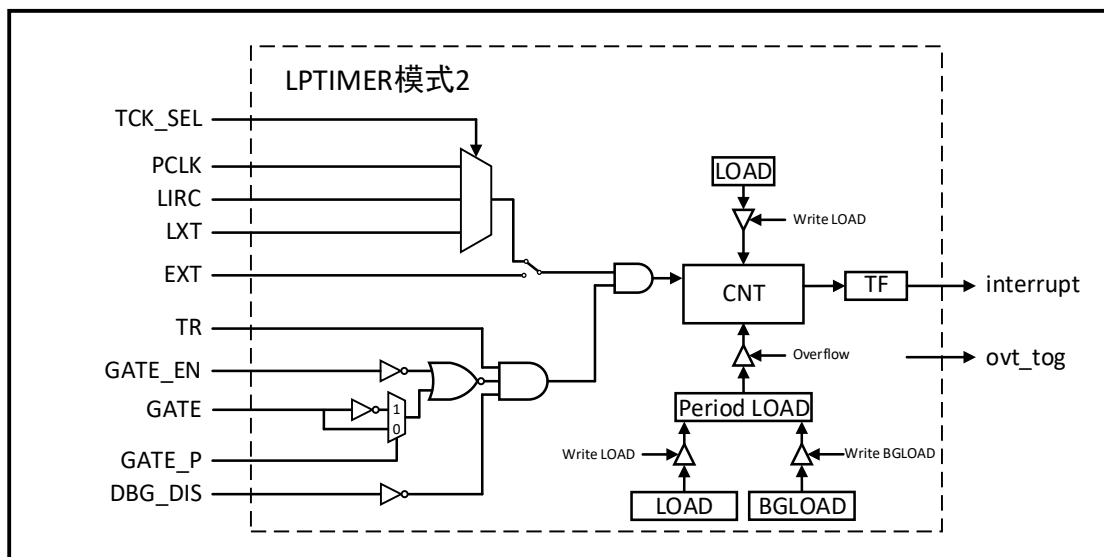


图17.3 LPTIMER 模式 2



17.1.1.1 计数功能

计数功能用于测定某个事件发生的次数。在计数功能中，计数器在每个相应的输入时钟的上升沿累加一次。输入信号被内部的计数时钟采样，因此外部输入时钟频率不能超过系统的计数时钟。计数到最大值会溢出并且产生中断。

17.1.1.2 定时功能

定时功能用于产生间隔定时。在定时功能中，定时器一个时钟累加一次，计数到最大值会溢出并且产生中断。

17.1.2 LPTIM 互连

17.1.2.1 GATE 互连

GATE 输入可以从端口直接输入，也可以输入 UART 的 RX 信号；

通过内部互连配置，可以实现 UART 波特率的自动识别，可以测量 VC 比较输出的脉冲宽度，可以实现外部控制计数。

配置选择 RX 输入在 SYSCON_PORTCR 寄存器控制，VC 控制在 VC 控制寄存器控制。

17.1.2.2 EXT 互连

EXT 输入可以从端口直接输入，也配置为 VC 的输入作为 EXT 信号。

通过内部互连配置，可以测量 VC 脉冲计数。VC 输出控制寄存器在 VC 控制模块。

17.1.2.3 TOGGLE 输出互连

LPTIM 的翻转输出到端口上，可以驱动 BUZZER 实现蜂鸣器的控制。

17.2 LPTIM 寄存器列表

基地址：0x4001 3C00

偏移地址	名称	描述	复位值
0x00	LPTIM_CNTVAL	LPTIM 计数值只读寄存器	0x0000 0000
0x04	LPTIM_CR	LPTIM 控制寄存器	0x0000 0000
0x08	LPTIM_LOAD	LPTIM 立即重载寄存器	0x0000 0000
0x0C	LPTIM_INTSR	LPTIM 中断寄存器	0x0000 0000
0x10	LPTIM_INTCLR	LPTIM 中断清除寄存器	0x0000 0000
0x14	LPTIM_BGLOAD	LPTIM 周期重载寄存器	0x0000 0000

17.3 LPTIM 寄存器说明

17.3.1 LPTIM 计数值只读寄存器 (LPTIM_CNTVAL)

偏移地址：0x00，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPT_CNT															
RO															

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	-
15:0	LPT_CNT	计数值只读寄存器	0x0	RO

17.3.2 LPTIM 控制寄存器 (LPTIM_CR)

偏移地址：0x04，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															WT_F LA G
-															RO
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留						TCK_EN	INT_E N	GATE_P	GATE_E N	TCK_SEL[1:0]		TOG_EN	CT_S EL	MODE	TIM_RUN
-						R/W	R/W	R/W	R/W	R/W		R/W	R/W	R/W	R/W

Bit	Field	Description	Reset	RW
31:17	Reserved	保留	0x0	-
16	WT_FLAG	WT 写同步标志 0: 同步完成, 此时可更改 LOAD/BGLOAD 1: 正在同步, 此时写 LOAD/BGLOAD 无效	0x0	RO
15:10	Reserved	保留	0x0	-
9	TCK_EN	LPTIM 计数时钟使能 0: LPTIM 计数时钟关闭 1: LPTIM 计数时钟使能 只有计数时钟使能后才能进行 LOAD/BGLOAD 的配置	0	R/W
8	INT_EN	中断使能控制, 写 1 后使能中断	0	R/W
7	GATE_P	输入 GATE 的有效极性 默认高电平 GATE 有效, 设置为 1 后低电平有效	0	R/W
6	GATE_EN	定时器门控 0: 无门控 1: 有门控	0	R/W
5:4	TCK_SEL	LPTIM 时钟选择 00: PCLK; 10: LSI; 11: LSE	0x0	R/W
3	TOG_EN	TOG 输出使能 0: TOG, TOGN 同时输出 0 1: TOG, TOGN 输出相位相反的信号。可供 BUZZER 使用。	0	R/W
2	CT_SEL	计数器/定时器功能选择 0: 定时器功能, 定时器使用 TCK_SEL 选择的时钟进行计数。 1: 计数器功能, 计数器使用外部输入的下降沿进行计数。采样时钟使用 TCK_SEL 选择的时钟, 外部输入时钟要低于 1/2 采样时钟。	0	R/W
1	MODE	定时器工作模式 0: 模式 1 无重载模式 16 位计数器/定时器 1: 模式 2 自动重载 16 位计数器/定时器	0	R/W
0	TIM_RUN	定时器运行控制位 0: 定时器停止 1: 定时器运行	0	R/W

17.3.3 LPTIM 立即重载寄存器 (LPTIM_LOAD)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LOAD															
R/W															

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	-

Bit	Field	Description	Reset	RW
15:0	LOAD	即刻重载寄存器 与 Timer 是否运行无关, 与 MODE 无关。 写 LOAD 前需要读取 LPTIM_CR.WT_FLAG, 当且仅当 WT_FLAG 为 0 时, 才能写入数据。写 LOAD 寄存器完成后 WT_FLAG 会变低。 写该寄存器时会立即更新计数器的值。 读该寄存器时返回最新更新到 LOAD 或者 BGLOAD 的值	0x0	R/W

17.3.4 LPTIM 中断寄存器 (LPTIM_INTSR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
R															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留														INTF	
R														RO	

Bit	Field	Description	Reset	RW
31:1	Reserved	保留	0x0	—
0	INTF	中断标志: 0: 未发生中断 1: 发生溢出中断	0x0	RO

17.3.5 LPTIM 中断寄存器 (LPTIM_INTCLR)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
—															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留														ICLR	
—														WO	

Bit	Field	Description	Reset	RW
31:1	Reserved	保留	0x0	—
0	ICLR	写 1 清除中断标志, 写 0 无效	0x0	WO

17.3.6 LPTIM 周期重载寄存器 (LPTIM_BGLOAD)

偏移地址: 0x14, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
—															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BGLOAD[15:0] R/W															

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	—
15:0	BGLOAD	BackGround 周期重载寄存器 写 BGLOAD 前需要读取 LPTIM_CR.WT_FLAG, 当且仅当 WT_FLAG 为 0 时, 才能写入数据。写 BGLOAD 寄存器完成后 WT_FLAG 会变低。 写该寄存器时, 当计数器溢出后, 设定值会更新到计数器。 读该寄存器时返回最新更新到 LOAD 或者 BGLOAD 的值。	0x0	R/W

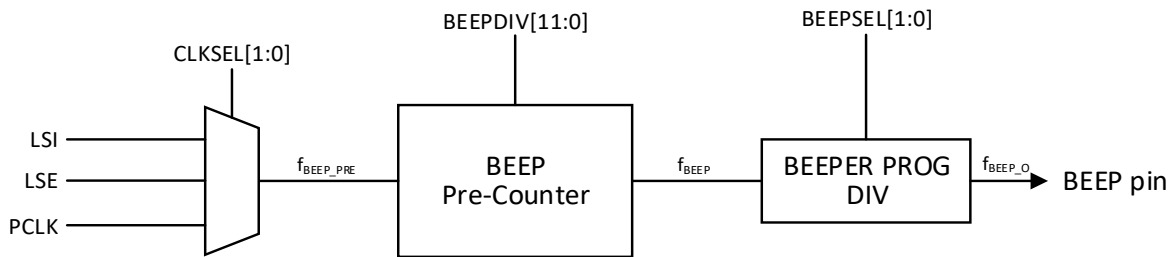
18 蜂鸣器(BEEP)

18.1 蜂鸣器(BEEP)

18.1.1 简介

选择 LSI 时钟、LSE 时钟或者 PCLK，可通过分频设定来产生各种频率的蜂鸣信号。
BEEP_CSR.CLKSEL[1:0]位来选择得到 fBEEP_PRE 时钟，通过设置 BEEPDIV[11:0]将蜂鸣器时钟 fBEEP_PRE 分频得到 fBEEP。通过设置 BEEPSEL[1:0]，得到 fBEEP_O 蜂鸣信号。

图18.1 蜂鸣器功能图



18.1.2 功能描述

18.1.2.1 蜂鸣器操作

为了使用蜂鸣功能，按顺序执行如下的步骤：

- 根据需要的蜂鸣器频率输出值确定 BEEPDIV[11:0]的值；
- 通过写 BEEP_CSR 的 BEEPSEL[1:0]位来选择 fBEEP_O 的输出频率；
- 置位 BEEP_CSR 的 BEEPEN 位来使能时钟源；

注意：

- 预分频计算器仅仅在当 BEEPDIV[11:0]的值不同于复位值 0x0FFF 时才开始运行。
- 在蜂鸣器运行过程中应该保持 BEEPDIV[11:0]的值不变。

18.1.2.2 蜂鸣器校准

该步骤可以用来校准 LIRC 时钟以便达到更标准的 fBEEP_O(1KHz、2KHz 或 4KHz)频率输出。采用如下的步骤：

- TIM1、TIM3 或者 CLKTRIM 模块来测量内部低速时钟(LIRC)的时钟频率
- 采用如下方法计算 BEEPDIV 的值，这里 A 和 x 是 fBEEP_PRE/fBEEP 的整数和小数部分值：当 x 小于或者等于 A/(1+2*A)时，BEEPDIV = A-1；否则 BEEPDIV = A
- 将 BEEPDIV 值写入到 BEEP_CSR 的 BEEPDIV[11:0]位

18.2 寄存器列表

BEEP 基地址：0x4001 5400

偏移地址	名称	描述	复位值
0x000	BEEP_CSR	蜂鸣器控制寄存器	0x0000 0FFF

18.3 寄存器说明

18.3.1 蜂鸣器控制/状态寄存器 (BEEP_CSR)

偏移地址: 0x00, 复位值: 0x0000 0FFF。

Bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										CLKSEL	Reserved	BEEPEN	BEEPSEL		
-										R/W	-	R/W	R/W		
Bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				BEEPDIV											
-				R/W											

Bit	Field	Description	Reset	RW
31:22	Reserved	保留	0x0	-
21:20	CLKSEL	时钟选择。 00: 停止 01: PCLK 10: LSI 11: LSE	0x0	R/W
19	Reserved	保留		
18	BEEPEN	使能蜂鸣	0	R/W
17:16	BEEPSEL	蜂鸣器输出 BEEP_O 频率选择位 00: fBEEP_PRE/8 01: fBEEP_PRE/4 1x: fBEEP_PRE/2	0x0	R/W
15:12	Reserved	保留	0x0	-
11:0	BEEPDIV	蜂鸣器预分频器将 fBEEP_PRE 分频得到 8KHz 周期的信号分频因子为 BEEPDIV + 1 $fBEEP = fBEEP_PRE / (BEEPDIV + 1)$	0xfff	R/W

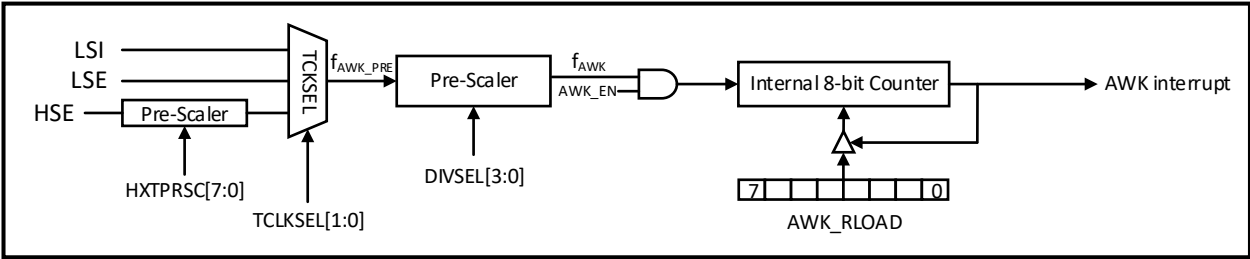
19 自唤醒定时器(AWK)

19.1 自唤醒定时器(AWK)

CLM32L003X 有一个专用的自唤醒定时器(AWK)，为芯片在低功耗模式下提供一个唤醒事件基准。AWK 在 Sleep 或 Deep Sleep 模式保持计数。当 AWK 用作唤醒定时器时，AWK 要在进入省电模式之前开启。AWK 可以配置内部低速时钟源 LSI、外部低速时钟源 LSE、外部高速时钟 HSE 分频后的时钟。注意系统时钟频率必须大于 AWK 时钟两倍以上。如果 AWK 开始计数，在设备进入 Sleep 或 Deep Sleep 模式时，选择的时钟源会也要保持工作。注意选择的 AWK 时钟源不会连同 AWK 的配置自动使能，用户应该手动使能选择的时钟源并等待它稳定来确保操作的成功。

AWK 配备了一个简单的 8 位自动重载向上计数定时器。它的预分频可选择从 1/2 到 1/65536，通过 AWK_CR.DIVSEL[3:0]来设置。用户填写重装载值到 AWK_RLOAD 寄存器来决定它的溢出速率。AWK_CR.AWKEN 置位后，当 CPU 进入 Sleep/Deep Sleep 模式时装载 AWK_RLOAD 寄存器的值到内部 8 位计数器并开始计数。当计数器溢出，AWK_SR.AWUF 置为 1，唤醒 CPU。

图19.1 自唤醒定时器结构图



19.2 寄存器列表

基地址：0x4001 2800

偏移地址	名称	描述	复位值
0x000	AWK_CR	自唤醒定时器控制寄存器	0x0000 BC00
0x004	AWK_RLOAD	自唤醒定时器重装载数据寄存器	0x0000 0000
0x008	AWK_SR	自唤醒定时器状态寄存器	0x0000 0000
0x00C	AWK_INTCLR	自唤醒中断清除寄存器	0x0000 0000

19.2.1 寄存器说明

19.2.2 自唤醒定时器控制寄存器（AWK_CR）

偏移地址：0x00，复位值：0x0000 BC00。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
XTLPRSC[7:0]								Reserved	TCLKSEL[1:0]		AWKEN	DIVSEL[3:0]			
R/W									R/W		R/W				

Bit	Field	Description	Reset	RW
31:16	Reserved	保留	0x0	—
15:8	HXTPRSC	HXT 时钟分频系数: $f_{HXT}/(N+1)$	0xBC	R/W

Bit	Field	Description	Reset	RW
7	Reserved	保留	0x0	—
6:5	TCLKSEL	AWK 计数时钟源选择 00: PCLK 01: HSE 分频后的时钟 10: LSI 时钟 11: LSE 时钟	0x0	R/W
4	AWKEN	AWK 使能 1: 使能 0: 不使能	0	R/W
3:0	DIVSEL	计数器用时钟源选择位 0000: FAWK_PRE/2 ¹ 0001: FAWK_PRE/2 ² 0010: FAWK+PRE/2 ³ ... 1111: FAWK_PRE/2 ¹⁶	0x0	R/W

19.2.3 自唤醒定时器重载数据寄存器 (AWK_RLOAD)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								RLDVAL[7:0]							
								R/W							

Bit	Field	Description	Reset	RW
31:8	Reserved	保留	0x0	—
7:0	RLDVAL	计数器	0x0	R/W

19.2.4 自唤醒定时器状态寄存器 (AWK_SR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															AWU F
															RO

Bit	Field	Description	Reset	RW
31:1	Reserved	保留	0x0	—
0	AWUF	自动唤醒发生, 硬件置 1, 软件清零 0: 未发生自动唤醒 1: 计数器溢出, 自动唤醒发生	0	RO

19.2.5 自唤醒中断清除寄存器 (AWK_INTCLR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															INTC LR
															WO

Bit	Field	Description	Reset	RW
31:1	Reserved	保留	0x0	—

Bit	Field	Description	Reset	RW
0	INTCLR	自动唤醒中断清除 0: 无作用 1: 清除自动唤醒中断	0x0	WO

20 实时时钟 (RTC)

20.1 实时时钟 (RTC)

20.1.1 简介

实时时钟(RTC)是一个独立的 BCD 定时器/计数器, 提供秒、分、时(12/24 小时制)、周、日、月和年的信息。

RTC 模块拥有自动唤醒功能, 用于管理所有的低功耗模式。

两个 32 位寄存器以 BCD 格式存储秒、分、时(12/24 小时制)、周、日、月和年。

RTC 具有自动月份天数补偿功能, 每月的天数和闰年的天数可自动调整。

使用两个 32 位寄存器存储可编程报警信息, 包括秒、分、时、周、日、月和年。

对由晶体本身的频偏、温度漂移及其他原因引起的任何误差, 可以利用 RTC 本身的数字校准功能进行修正。

上电复位后, 所有 RTC 寄存器将被禁止访问, 以防止意外的写操作。

当设备处于运行模式、低功耗模式或复位状态(上电复位(POR 复位)除外), 只要电压在工作范围内, RTC 将保持正常运行。

20.1.2 主要特性

RTC 模块的主要特性如下:

- 日历功能, 可显示秒、分、时(12/24 小时制)、周、日、月和年。
- 可进行自动闰年调整。
- 具有闹钟中断和周期中断功能。
- 数字校准电路(计数器定期修正): 来自一个几秒钟的校准窗口。
- 时钟源可选: 外部低速时钟(LXT)、内部低速时钟(LIRC)、外部高速时钟(HXT)
- 1Hz 方波输出
- 支持 TAMPER 输入

20.1.3 RTC 功能描述

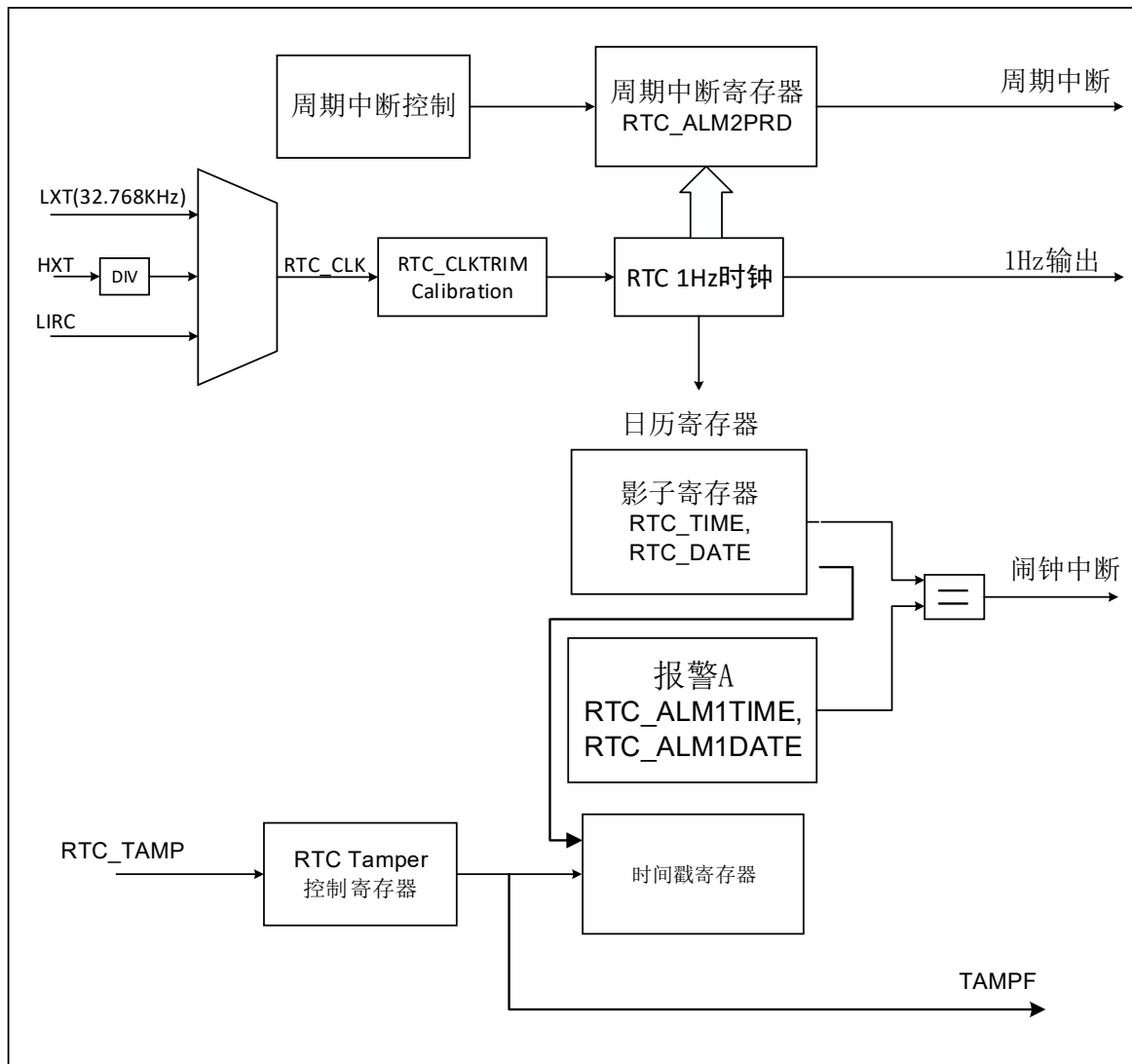
20.1.3.1 RTC 结构框图

LXT 时钟作为 RTC 时钟, 即 LSE 振荡器时钟;

LIRC 时钟作为 RTC 时钟, 即 LSI 振荡器时钟;

HXT 时钟的 128 分频作为 RTC 时钟, 即 HSE 振荡器时钟 128 分频作为 RTC 时钟;

图20.1 RTC 框图



RTC 模块包括:

- 一个闹钟报警中断
- 一个周期中断
- 校准后的 1Hz 时钟输出
- 万年历寄存器
- 5 个备份寄存器
- 复用输入功能: RTC_TAMP
- 复用输出功能: RTC_1Hz 和 RTC_ALM1

20.1.3.2 RTC 时钟

由时钟控制器从以下 3 种时钟中选择 RTC 时钟源(RTC_CLK):

- LXT 时钟作为 RTC 时钟;
- LIRC 时钟作为 RTC 时钟;
- HXT 时钟的 128 分频作为 RTC 时钟。

20.1.3.3 复位过程

任何可用的系统复位源都将导致日历影子寄存器和 RTC 初始化和状态寄存器(RTC_ISR)复位至默认值。

然而, 下列寄存器的复位与系统复位无任何关联, 只与上电复位(POR 复位)有关: RTC 控制寄存器(RTC_CR)、RTC 时钟控制寄存器(RTC_CLKCR)、RTC 校准寄存器(RTC_CLKTRIM)、ALM 寄存器(RTC_ALM1DATE/RTC_ALM1TIME/RTC_ALM2PRD)、RTC 当前日历寄存器(RTC_TIME/RTC_DATA)、RTC 时间戳寄存器(RTC_TSTR,RTC_TSDR)、RTC 侵入和复用功能配置寄存器(RTC_TAFCR)、RTC 备份寄存器(RTC_BKPxR)。

除上电复位外, 发生任何系统复位时 RTC 将维持运行状态。发生上电复位后, RTC 停止运行, 所有 RTC 寄存器复位至默认值。

20.1.3.4 寄存器的写保护

上电复位后, 所有 RTC 寄存器将处于写保护状态。通过向写保护寄存器 RTC_WPR 写入指定关键字来启动 RTC 寄存器的写权限。

通过以下操作解除所有 RTC 寄存器的写保护。

- 向 RTC_WPR 寄存器写入'0xCA';
- 向 RTC_WPR 寄存器写入'0x53'。

注意: 保护解除后, 任何对该寄存器的再一次写将重新激活写保护。

20.1.3.5 日历初始化及配置

按照以下顺序完成时间和日期值的初始化, 包括时间格式和预分频器的配置:

- 通过 RTC_CLKCR.CKSEL 选择 RTC 计时时钟源, 如果选择 HXT 时钟还要先设定预分频器。
- 设定 RTC_CLKCR.RTCCKEN 使能 RTC 计时时钟。
- RTC_ISR 寄存器的 WAIT 位置“1”, 进入初始化模式。
- 等待 RTC_ISR 寄存器的 WAITF 位置“1”, 确保已经正式进入初始化模式。由于时钟同步的延迟, 该过程大约需要 2 个 RTC_CLK 时钟周期。在该模式下日历计数器暂停运行, 此时可更新时间和日期计数器的值。
- 通过 RTC_CR 寄存器的 FMT 位设置时间格式(12 小时制/24 小时制)。
- 将初始时间和日期值加载到时间寄存器(RTC_TIME 与 RTC_DATE)。
- 需要进行时钟误差补偿时, 设定时钟补偿寄存器 RTC_CLKTRIM。
- 清除 RTC_ISR.WAIT 位的值退出初始化模式。日历计数器的实际值将会自动加载, 并在 4 个 RTCCLK 时钟周期后重新启动。

在完成上述一系列初始化操作后, 日历将开始计时。

20.1.3.6 读出计数寄存器

当 RTC_CR 寄存器的 BYPSHAD 控制位被清除时:

为确保在安全同步机制下正常读 RTC 日历寄存器(RTC_TIME 和 RTC_DATE), APB 时钟频率(fPCLK)应至少为 RTC 时钟频率(fRTCCLK)的 7 倍以上。当 APB 时钟频率低于 7 倍 RTC 时钟频率时, 软件必须两次读取日历时间和日期寄存器。如果第二次读取的值与第一次读取的值相同, 说明返回值是正确的, 否则需再次读取。任何情况下, APB 时钟频率都必须大于 RTC 时钟频率。

日历寄存器的内容被复制到 RTC_TIME 和 RTC_DATE 影子寄存器中时, RTC_ISR 寄存器的 RSF 位被置位。复制操作每两个 RTC_CLK 周期执行一次。为确保两者的值保持一致, 读 RTC_TIME 时硬件会锁定 RTC_DATE 影子寄存器的值, 直到 RTC_DATE 的值被读取。



为避免软件在时间间隔少于 2 个 RTC_CLK 周期的情况下多次访问日历，每次读日历 RSF 位应由软件清零，软件必须等待 RSF 位被置位后才能读 RTC_TIME 和 RTC_DATE 寄存器。从低功耗模式唤醒后，RSF 位应由软件清零，软件必须等待 RSF 位被再次置位后才能读 RTC_TIME 和 RTC_DATE 寄存器。RSF 位应在唤醒后被清除，而不是进入低功耗模式前。系统复位后，软件必须等待 RSF 位被置位后才能读 RTC_TIME 和 RTC_DATE 寄存器。事实上系统复位将导致影子寄存器复位至其默认值。

当 RTC_CR 寄存器的 BYPSHAD 控制位被置位时(无需考虑影子寄存器):

读日历寄存器，直接从日历计数器获取值，无需等待 RSF 位被置位。此功能在刚退出低功耗模式时非常有用，因为影子寄存器在低功耗模式下不会自动更新。

在 BYPSHAD 为“1”时，如果两次读寄存器之间出现 RTC_CLK 边沿，不同寄存器中的结果可能会互不相关。此外，如果在读操作过程中遇到 RTC_CLK 边沿，则某个寄存器的值可能不正确。软件必须读取所有的寄存器两次，并比较两次读取的结果；或者通过比较两组最低有效日历寄存器的结果，以检验数据是否正确且有一定关联。

20.1.3.7 写入计数寄存器

- 设定 RTC_ISR.WAIT=1，停止日历寄存器计数，进入写模式；
- 查询直到 RTC_ISR.WAITF=1；
- 写入秒、分、时、周、日、月、年计数寄存器值；
- 设定 RTC_ISR.WAIT=0，计数器重新开始。注意，须在 1 秒内完成所有写操作；
- 查询直到 RTC_ISR.WAITF=0。

20.1.3.8 闹钟设定

设置 RTC_CR 寄存器的 ALM1EN 位，启动闹钟功能。当日历中秒、分、时、周、日、月、年与报警寄存器 RTC_ALM1TIME 和 RTC_ALM1DATE 中设定的值匹配，RTC_ISR.ALM1_F 由硬件置 1。所有日历字段都可以通过 RTC_ALM1DATE 寄存器中的 ALMxEN 位选择为报警源。设置 RTC_CR 寄存器的 ALM1_INTEN 位，会产生报警中断。

20.1.3.9 RTC 侵入检测和时间戳

RTC_TAMPx 输入事件可设置为边沿检测或带过滤功能的电平检测。

篡改输入引脚用于监视系统的特定位置，篡改输入引脚变化表明可能发生了篡改事件。检测事件可以配置为电平转换（高到低转换或者是低到高转换），也可以配置为脉冲（正或负）。脉冲检测时，去抖动电路可配置为滤除小于 6ms 的脉冲。

侵入检测后，直接保存当前的日期和时间到时间戳寄存器，系统有一组时间戳寄存器，如果检测到一个新的侵入事件时，侵入标志（TAMPF）已经被置位，则侵入溢出标志位（TAMPOVF）标志为“溢出”，时间戳寄存器（RTC_TSTR 和 RTC_TSDR）维持上一事件的内容不变。

20.1.3.10 校准 1Hz 输出

RTC 可选择输出校准后的 1Hz 时钟。通过 RTC_CR.RTC1HZOE 设定输出使能，通过 RTC_CLKTRIM 来设定校准值。

20.1.3.11 RTC 时钟校准

RTC 模块通过每隔一个固定的时间周期屏蔽指定的 RTC 时钟周期数来补偿 RTC 时钟的频率，通过 RTC_CLKTRIM.MODE[1:0]来选择调整的时间间隔：

0b00: 每 60 秒(SEC=00)校准一次

0b01: 每 30 秒(SEC=00, 30)校准一次

0b10: 每 15 秒(SEC=00, 15, 30, 45)校准一次

0b11: 每 6 秒(SEC=00, 06, 12, 18, 24, 30, 36, 42, 48, 54)校准一次

通过 RTC_CLKTRIM.TRIM[7:0]来指定屏蔽的 RTC 时钟周期数。

注意 RTC_CLKTRIM.TRIM[7:0]的值时有符号整数, 范围为-128~+127。

RTC 中断

RTC 支持两种中断类型。闹钟中断、定周期中断。闹钟中断与定周期中断共用一个中断信号, 通过标志寄存器位来区分中断源。

20.1.3.12 RTC 闹钟中断

- 设定 RTC_CR.ALM1EN=0, 禁止闹钟功能;
- 设定时间闹钟寄存器 RTC_ALM1TIME、日期闹钟寄存器 RTC_ALM1DATE;
- 设定 RTC_CR.ALM1EN=1, 闹钟许可;
- 清除中断标志位 RTC_ISR.ALM1_F;
- 设定 RTC_CR.ALM1_INTEN=1, 闹钟中断许可, 若当前日历时间与闹钟寄存器相等时, 触发闹钟中断;
- 等待发生中断;

20.1.3.13 RTC 周期中断

控制寄存器 RTC_CR 的 RTC_CR.ALM2_INTEN=1 时, 选择的周期发生后, 触发定周期唤醒中断, 由于闹钟和定周期共用中断, 通过标志寄存器位来区分。

- 设定 RTC_CR.ALM2_INTEN=0, 禁止周期中断功能;
- 设定周期闹钟寄存器 RTC_ALM2PRD;
- 清除中断标志位 RTC_ISR.ALM2_F;
- 设定 RTC_CR.ALM2_INTEN=1, 周期中断许可, 选择的周期发生后, 触发定周期唤醒中断;
- 等待发生中断;

20.1.3.14 RTC 入侵中断

入侵中断配置步骤:

使能入侵中断, RTC_TAFCR.TAMPIE=1;

配置入侵事件类型, 配置 RTC_TAFCR.TAMP_MODE;

入侵管脚 PC13 配置, 配置 RTC_CTRL.RTC_1HZ_OE=0, RTC_CTRL.OSEL=00, RTC_TAFCR.TAMPE=1;

等待中断发生;

20.2 RTC 寄存器列表

基地址: 0x4000 2800

偏移地址	名称	描述	默认值
0x00	RTC_CR	RTC 控制寄存器	0x00000000
0x04	RTC_TIME	RTC 时间寄存器	0x00000000
0x08	RTC_DATE	RTC 日期寄存器	0x00000000

偏移地址	名称	描述	默认值
0x0C	RTC_ALM1TIME	RTC 闹钟时间寄存器	0x00000000
0x10	RTC_ALM1DATE	RTC 闹钟日期寄存器	0x00000000
0x14	RTC_ALM2PRD	RTC 周期闹钟寄存器	0x00000000
0x18	RTC_CLKCALR	RTC 时钟调校寄存器	0x00000000
0x1C	RTC_ISR	初始化和状态寄存器	0x00000000
0x20	RTC_INTCLR	RTC 状态清除寄存器	0x00000000
0x24	RTC_WPR	RTC 写保护寄存器	0x00000000
0x28	RTC_TSTR	RTC 时间戳时间寄存器	0x00000000
0x2C	RTC_TSDR	RTC 时间戳日期寄存器	0x00000000
0x30	RTC_TAFCR	RTC 侵入和复用功能配置寄存器	0x00000000
0x50	RTC_BKP0R	RTC 备份寄存器 0	0x00000000
0x54	RTC_BKP1R	RTC 备份寄存器 1	0x00000000
0x58	RTC_BKP2R	RTC 备份寄存器 2	0x00000000
0x5C	RTC_BKP3R	RTC 备份寄存器 3	0x00000000
0x60	RTC_BKP4R	RTC 备份寄存器 4	0x00000000

20.3 RTC 寄存器说明

20.3.1 RTC 控制寄存器 (RTC_CR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								OSEL		POL	Reserved				
-								R/W		R/W	R/W				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							START	ALM2EN	ALM1 EN	ALM2 INTEN	ALM1 INTEN	Reserved	FMT	RTC1 HZOE	BYPS HAD
							R/W	R/W	R/W	R/W	R/W		R/W	R/W	R/W

Bit	Field	Description	Reset	RW
31:23	Reserved	保留	0x0	-
22:21	OSEL	RTC_Alarm 输出选择: 00: 输出禁止 01: Alarm1 输出使能 10: 保留 11: Alarm2 输出使能	0x0	R/W
20	POL	RTC_Alarm 极性选择: 0: 发生 RTC_Alarm 时, PIN 输出高 1: 发生 RTC_Alarm 时, PIN 输出低	0x0	R/W
19:9	Reserved	保留	0x0	-
8	START	0: 停止 RTC 计数器 1: 使能 RTC 计数器	0	R/W
7	ALM2EN	ALM2 周期中断功能使能。 0: 禁止 ALM2 周期中断功能 1: 使能 ALM2 周期中断功能 注意: 在日历计数(RTC_CLKCR.RTCKEN=1)过程中并且 ALM2 周期中断许可使能(ALM2_INTEN=1)的情况下使能 ALM2EN 时, 为防止误动作, 请将系统中断关闭。使能后请将 ALM2_F 标志位清除。	0	R/W

Bit	Field	Description	Reset	RW
6	ALM1EN	ALM1 闹钟功能使能。 0: 禁止 ALM1 闹钟功能 1: 使能 ALM1 闹钟功能 注意: 在日历计数(RTC_CLKCR.RTCCKEN=1)过程中并且 ALM1 闹钟中断许可使能(ALM1_INTEN=1)的情况下使能 ALM1EN 时, 为防止误动作, 请将系统中断关闭。使能后请将 ALM1_F 标志位清除。	0	R/W
5	ALM2_INTEN	ALM2 周期中断使能。 0: 禁止 ALM2 周期中断 1: 使能 ALM2 周期中断	0	R/W
4	ALM1_INTEN	ALM1 闹钟中断使能。 0: 禁止 ALM1 闹钟中断 1: 使能 ALM1 闹钟中断	0	R/W
3	Reserved	保留	0	-
2	FMT	时间格式。 0: 24 小时制 1: 12 小时制(AM/PM 时间格式)	0	R/W
1	RTC1HZOE	RTC 1Hz 输出时能 0: 禁止 1: 使能	0	R/W
0	BYP SHAD	绕过影子寄存器 0: 从影子寄存器读取日历值, 影子寄存器每两个 RTC_CLK 周期跟新一次 1: 直接从日历计数器读取日历值 注: 如果 APB 时钟频率低于 RTCCLK 频率的 7 倍, BYP SHAD 必须置“1”。	0	R/W

20.3.2 RTC 时间寄存器 (RTC_TIME)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					WEEK[2:0]			Reserved	PM	HOURT[1:0]		HOURU[3:0]			
					R/W				R/W	R/W		R/W			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	MINT[2:0]				MINU[3:0]			Reserved	SECT[2:0]			SECU[3:0]			
	R/W				R/W				R/W			R/W			

Bit	Field	Description	Reset	RW
31:27	Reserved	保留	0x0	-
26:24	WEEK	星期计数器。星期计数器值为二进制计数, 计数区间从 0 到 6 (7 不被使用, 除非不使用星期计数)。星期和星期计数器值得对应关系由用户定义。(比如星期日=0, 星期一=1.....星期六=6)	0x0	R/W
23	Reserved	保留	0x0	-
22	PM	小时标识。在配置时请根据 FMT 的值, 选择正确的标识格式。注 1 0: AM/24 小时格式 1: PM	0x0	-
21:20	HOURT	小时计数十位值。注 1	0	R/W
19:16	HOURU	小时计数个位值。注 1	0x0	R/W
15	Reserved	保留	0x0	-
14:12	MINT	分钟计数十位值。注 2	0x0	R/W
11:8	MINU	分钟计数个位值。注 2	0x0	R/W
7	Reserved	保留	0	-
6:4	SECT	秒钟计数十位值。注 2	0x0	R/W

Bit	Field	Description	Reset	RW
3:0	SECU	秒钟计数个位值。注 2	0x0	R/W

注 1: 24 小时制时, 表示 0-23 小时。12 小时制时, PM=0 表示 AM, 则 01:12 表示上午; PM=1 表示 PM, 则 21:32 表示下午。

请根据控制位 FMT 的值, 设定正确十进制的 0~23 或者 01~12 的 BCD 码。写入超出范围的值将被忽略。

注 2: 采用十进制计数。请写入十进制 0-59 的 BCD 码, 写入错误值时, 写入值将被忽略。

20.3.3 RTC 日期寄存器 (RTC_DATE)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								YEART[3:0]				YEARU[3:0]			
								R/W				R/W			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CEN	Reserved		MONT	MONU [3:0]				Reserved		DAYT[1:0]		DAYU[3:0]			
R/W			R/W	R/W						R/W		R/W			

Bit	Field	Description	Reset	RW
31:24	Reserved	保留	0x0	-
23:20	YEART	年计数十位值。注 1。	0x0	R/W
19:16	YEARU	年计数个位值。注 1。	0x0	R/W
15	CEN	世纪计数器。0 代表 20 世纪, 1 代表 21 世纪	0x1	R/W
14:13	Reserved	保留	0x0	-
12	MONT	月计数十位值。注 2	0x0	R/W
11:8	MONU	月计数个位值。注 2		
7:6	Reserved	保留	0x0	-
5:4	DAYT	日计数十位值。注 3	0x0	R/W
3:0	DAYU	日计数个位值。注 3	0x0	R/W

注 1: 十进制 0~99 表示 0~99 年。根据月进位计数。自动计算闰年如: 00、04、08、...、92、96 等。请写入正确的十进制年计数值, 写入错误值将被忽略。

注 2: 月计数器为 BCD 编码, 计数区间从 01 到 12。当年计数器从 12 数到 01 时, 年计数器增长 1。

注 3: 日计数器为 BCD 编码, 计数区间如下:

01 到 31: 一月, 三月, 五月, 七月, 八月, 十月, 十二月;

01 到 30: 四月, 六月, 九月, 十一月;

01 到 29: 闰年的二月

01 到 28: 非闰年的二月

当年计数器从 99 数到 00 时, 世纪计数器增长 1。

20.3.4 RTC 时间闹钟寄存器 (RTC_ALM1TIME)

偏移地址: 0xC0, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					ALMWEEK[2:0]			Reserved	ALMPM	ALMHOURT[1:0]		ALMHOURU[3:0]			
					R/W				R/W	R/W		R/W			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	ALMMINT[2:0]			ALMMINU[3:0]				Reserved	ALMSECT[2:0]			ALMSECU[3:0]			
	R/W			R/W					R/W			R/W			

Bit	Field	Description	Reset	RW
31:27	Reserved	保留	0x0	-
26:24	ALMWEEK	闹钟星期设定	0x0	R/W
23	Reserved	保留	0x0	-
22	ALMPM	闹钟小时标识。在配置时请根据 FMT 的值，选择正确的标识格式。 0: AM/24 小时格式 1: PM	0x0	-
21:20	ALMHOURT	小时计数十位值。	0	R/W
19:16	ALMHOURU	小时计数个位值。	0x0	R/W
15	Reserved	保留	0x0	-
14:12	ALMMINT	分钟计数十位值。	0x0	R/W
11:8	ALMMINU	分钟计数个位值。	0x0	R/W
7	Reserved	保留	0	-
6:4	ALMSECT	秒钟计数十位值。	0x0	R/W
3:0	ALMSECU	秒钟计数个位值。	0x0	R/W

注：请设定正确的十进制 BCD 码。写入其他值，不会发生闹钟匹配。

20.3.5 RTC 日期闹钟寄存器 (RTC_ALM1DATE)

偏移地址：0x10，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	ALMYE AREN	ALMMO NEN	ALMDAY EN	ALMWEEK EN	ALMHO REN	ALMMIN EN	ALMSECE N	ALMYEART[3:0]				ALMYEARU[3:0]			
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				R/W			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ALMCEN	Reserved		ALMMO NT	ALMMONU [3:0]				Reserved	ALMDAYT[1:0]		ALMDAYU[3:0]				
R/W			R/W	R/W					R/W		R/W				

Bit	Field	Description	Reset	RW
31	Reserved	保留	0	-
30	ALMYEAREN	闹钟年设定使能	0	R/W
29	ALMMONEN	闹钟月设定使能	0	R/W
28	ALMDAYEN	闹钟日设定使能	0	R/W
27	ALMWEEKEN	闹钟星期设定使能	0	R/W
26	ALMHOUREN	闹钟小时设定使能	0	R/W
25	ALMMINEN	闹钟分钟设定使能	0	R/W
24	ALMSECE N	闹钟秒设定使能	0	R/W
23:20	ALMYEART	闹钟年设定十位值。	0x0	R/W
19:16	ALMYEARU	闹钟年设定个位值。	0x0	R/W
15	ALMCEN	闹钟世纪设定值。	0	R/W
14:13	Reserved	保留	0x0	-
12	ALMMONT	闹钟月设定十位值。	0x0	R/W
11:8	ALMMONU	闹钟月设定个位值。		
7:6	Reserved	保留	0x0	-

Bit	Field	Description	Reset	RW
5:4	DAYT	闹钟日设定十位值。	0x0	R/W
3:0	DAYU	闹钟日设定个位值。	0x0	R/W

注：请设定正确的十进制 BCD 码。写入其他值，不会发生闹钟匹配。

20.3.6 RTC 周期闹钟寄存器 (RTC_ALM2PRD)

偏移地址：0x14，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												ALM2PR_CNT[3:0]			
												R/W			

Bit	Field	Description	Reset	RW
31:4	Reserved	保留	0x0	-
3:0	ALM2PR_CNT	周期闹钟 2 计数周期设定。 0x0: 关闭周期闹钟 2 0x1: 1 秒 0x2: 1/2 秒 0x3: 1/4 秒 0x4: 1/8 秒 0x5: 1/16 秒 0x6: 1/32 秒 0x7: 1/64 秒 0x8: 1/128 秒 0x9: 10 秒 0xA: 30 秒 0xB: 1 分钟 0xC: 30 分钟 0xD: 60 分钟 0xE: 12 小时 0xF: 24 小时	0x0	R/W

20.3.7 RTC 时钟调校寄存器 (RTC_CLKCALR)

偏移地址：0x18，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															TRIM_EN
Reserved															R/W
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							MODE[1:0]		TRIM[7:0]						
Reserved							R/W		R/W						

Bit	Field	Description	Reset	RW
31:17	Reserved	保留	0x0	-
16	TRIM_EN	时钟补偿功能使能控制 0: 时钟补偿不使能 1: 时钟补偿使能	0	
15:10	Reserved	保留	0x0	-
9:8	MODE	时钟调节寄存器。决定了时钟调节的频率。 0x0: 每 60 秒(SEC=00) 0x1: 每 30 秒(SEC=00, 30) 0x2: 每 15 秒(SEC=00, 15, 30, 45) 0x3: 每 6 秒(SEC=00, 06, 12, 18, 24, 30, 36, 42, 48, 54)	0x0	R/W
7:0	TRIM	时钟补偿时间寄存器。此寄存器为有符号整数。(-128~+127)	0x0	R/W

20.3.8 RTC 初始化和状态寄存器 (RTC_ISR)

偏移地址: 0x1C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		TAMPOVF	TAMPF	Reserved						ALM2_F	ALM1_F	Reserved	RSF	WAITF	WAIT
		RO	RO							RO	RO		R/W	R/W	RW

Bit	Field	Description	Reset	RW
31:6	Reserved	保留	0x0	-
13	TAMPOVF	RTC_TAMP 检测溢出标志 0:无 1:有		
12	TAMPF	RTC_TAMP 检测标志 0:无 1: 有		
11:6	Reserved			
5	ALM2_F	周期中断 2 标志。 当此寄存器被读出时, 状态值被返回: 0: 无周期中断 2。 1: 有周期中断 2。	0	RO
4	ALM1_F	闹钟中断 1 标志。 当此寄存器被读出时, 状态值被返回: 0: 无闹钟中断 1 标志。 1: 有闹钟中断 1 标志。	0	RO
3	Reserved		0	-
2	RSF	寄存器同步标志 每当日历寄存器中的内容复制到影子寄存器(RTC_TIME、RTC_DATE)中时, 该位由硬件置位。当处于忽略影子寄存器模式(BYPHAD=1)下时, 该位由硬件在初始化模式下清除。该位也可由软件清除。 在初始化模式下, 该位可由硬件/软件清除。 0: 日历影子寄存器尚未同步; 1: 日历影子寄存器已经同步。 注意不能软件写 1	0	R/W
1	WAITF	0: 非写入/配置状态 1: 写入/配置状态 注意: WAITF 是 WAIT 位设定是否有效标志。在写入/读出前请确认该位是否为“1”。计数过程中, 在 WAIT 位清“0”后等待写入完成后该位才清“0”。	0	R/W
0	WAIT	0: 正常计数模式 1: 写入/配置模式 注意: 在写入/配置时请将该位置“1”, 由于计数器在连续计数, 请在 1 秒的时间内完成写入/读出操作并将该位清“0”。	0	R/W

20.3.9 TC 状态清除寄存器 (RTC_INTCLR)

偏移地址: 0x20, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		TAMPOV_F_CLR	TAMPF_CLR	Reserved						ALM2_CLR	ALM1_CLR	Reserved			
										WO	WO				

Bit	Field	Description	Reset	RW
31:6	Reserved	保留	0x0	-
13	TAMPOVF_CLR	TAMPOVF 入侵溢出标志清除 0: 没有影响 1: 对应的标志位清除		WO
12	TAMPF_CLR	TAMPF 入侵标志清除 0: 没有影响 1: 对应的标志位清除		WO
11:6	Reserved			
5	ALM2_CLR	周期闹钟 2 中断原始状态清除寄存器。 当此寄存器被写入时, 中断原始状态被要求清除: 0: 没有操作。 1: 周期闹钟 2 中断原始状态被清除。	0	WO
4	ALM1_CLR	闹钟中断原始状态清除寄存器。 当此寄存器被写入时, 中断原始状态被要求清除: 0: 没有操作。 1: 闹钟中断原始状态被清除。	0	WO
3:0	Reserved	保留	0x0	-

20.3.10 RTC 写保护寄存器 (RTC_WPR)

偏移地址: 0x24, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								WPR[7:0]							
								WO							

Bit	Field	Description	Reset	RW
31:8	Reserved	保留	0x0	-
7:0	WPR	写入指定关键字来启动 RTC 寄存器的写权限。 向 RTC_WPR 寄存器写入'0xCA'; 向 RTC_WPR 寄存器写入'0x53'。 注意: 保护解除后, 任何对该寄存器的再一次写将重新激活写保护。	0x0	WO

20.3.11 RTC 时间戳时间寄存器 (RTC_TSTR)

偏移地址: 0x28, 复位值: 0x0000 0000。

仅当 RTC_ISR 中 TAMPF 置“1”时, 该寄存器的内容才有效。当 TAMPF 位复位时, 该寄存器的内容将被清除, 等待 2 个 RTC 时钟周期后硬件置位功能才能再次有效。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					WEEK[2:0]			Reserved	PM	HOURT[1:0]		HOURU[3:0]			
					RO				RO	RO		RO			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	MINT[2:0]				MINU[3:0]			Reserved	SECT[2:0]			SECU[3:0]			
	RO				RO				RO			RO			

Bit	Field	Description	Reset	RW
31:27	Reserved	保留	0x0	-
26:24	WEEK	星期计数器。星期计数器值为二进制计数，计数区间从 0 到 6 (7 不被使用，除非不使用星期计数)。星期和星期计数器值得对应关系由用户定义。(比如星期日=0，星期一=1.....星期六=6)	0x0	RO
23	Reserved	保留	0x0	-
22	PM	小时标识。在配置时请根据 FMT 的值，选择正确的标识格式。注 1 0: AM/24 小时格式 1: PM	0x0	RO
21:20	HOURT	小时计数十位值。注 1	0	RO
19:16	HOURU	小时计数个位值。注 1	0x0	RO
15	Reserved	保留	0x0	-
14:12	MINT	分钟计数十位值。注 2	0x0	RO
11:8	MINU	分钟计数个位值。注 2	0x0	RO
7	Reserved	保留	0	-
6:4	SECT_1	秒钟计数十位值。注 2	0x0	RO
3:0	SECT_0	秒钟计数个位值。注 2	0x0	RO

20.3.12 RTC 时间戳日期寄存器 (RTC_TSDR)

偏移地址：0x2C，复位值：0x0000 0000。

仅当 RTC_ISR 中 TAMPF 置“1”时，该寄存器的内容才有效。当 TAMPF 位复位时，该寄存器的内容将被清除，等待 2 个 RTC 时钟周期后硬件置位功能才能再次有效。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								YEART[3:0]				YEARU[3:0]			
								RO				RO			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CEN	Reserved		MONT	MONU [3:0]				Reserved		DAYT[1:0]		DAYU[3:0]			
RO			RO	RO						RO		RO			

Bit	Field	Description	Reset	RW
31:24	Reserved	保留	0x0	-
23:20	YEART	年计数十位值。	0x0	RO
19:16	YEARU	年计数个位值。	0x0	RO
15	CEN	世纪计数器。0 代表 20 世纪，1 代表 21 世纪	0	RO
14:13	Reserved	保留	0x0	-
12	MONT	月计数十位值。	0x0	RO
11:8	MONU	月计数个位值。	0x0	RO
7:6	Reserved	保留	0x0	-
5:4	DAYT	日计数十位值。	0x0	RO
3:0	DAYU	日计数个位值。	0x0	RO

20.3.13 RTC 侵入和复用功能配置寄存器 (RTC_TAFCR)

偏移地址：0x30，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
								PC15MODE	PC15VALUE	PC14MODE	PC14VALUE	PC13MODE	PC13VALUE		
								R/W	R/W	R/W	R/W	R/W	R/W		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		TAMP_DB[1:0]			TAMP_MODE[2:0]								TAMPIE		TMAPE
		R/W			R/W								R/W		R/W

Bit	Field	Description	Reset	RW
31:24	Reserved	保留	0x0	-
23	PC15MODE	PC15 模式 0: PC15 受 GPIO 配置寄存器约束, 因此 PC15 在待机模式下浮空。 1: LSE 禁用时, PC15 强制为推挽输出模式。	0	R/W
22	PC15VALUE	PC15 值 如果 LSE 禁用且 PC15MODE = 1, PC15VALUE 设置 PC15 的输出值。	0	R/W
21	PC14MODE	PC14 模式 0: PC14 受 GPIO 配置寄存器约束, 因此 PC14 在待机模式下浮空。 1: LSE 禁用时, PC14 强制为推挽输出模式。	0	R/W
20	PC14VALUE	如果 LSE 禁用且 PC14MODE = 1, PC14VALUE 设置 PC14 的输出值。	0	R/W
19	PC13MODE	PC13 模式 0: PC13 受 GPIO 配置寄存器约束, 因此 PC13 在待机模式下浮空。 1: RTC 复用功能禁用时, PC13 强制为推挽输出模式。	0	R/W
18	PC13VALUE	RTC_ALARM 输出形式 / PC13 值 如果 PC13 用于输出 RTC_ALARM, PC13VALUE 设置输出: 0: RTC_ALARM 为开漏输出; 1: RTC_ALARM 为推挽输出 如果所有 RTC 复用功能禁用且 PC13MODE = 1, PC13VALUE 设置 PC13 输出值。	0	R/W
17:14	Reserved	-	0	-
13:12	TAMP_DB	入侵检测的去抖动设定 01: 2ms 10: 4ms 11: 6ms 00: 没有去抖动功能	0	R/W
11	Reserved	-	0	-
10:8	TAMP_MODE	入侵检测事件配置 0x0: 上升沿 0x1: 下降沿 0x2: 上升沿和下降沿 0x3: 带有去抖动功能的正脉冲 0x4: 带有去抖动功能的负脉冲	0	R/W
7:3	Reserved	保留	0	-
2	TAMPIE	入侵检测中断使能控制 0: IRQ 关闭 1: IRQ 使能	0x0	R/W
1	Reserved	保留	0	-
0	TMAPE	入侵 PIN 上的检测功能控制 0x0: 关闭 0x1: 使能	0x0	R/W

20.3.14 RTC 备份寄存器 0 (RTC_BKPxR) (x=0~4)

偏移地址: 0x50 to 0x60, 复位值: 0x0000 0000。

这些位不会被系统复位所复位。当设备在低功耗模式下运行时, 这些位的内容仍然有效。

侵入检测事件发生（即 TAMPxF=1）或闪存读出保护被禁用时，该寄存器复位

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BKP[31:16]															
R/W															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BKP[15:0]															
R/W															

Bit	Field	Description	Reset	RW
31:0	BKP	备份值	0x0	R/W

第四部分 模拟接口

21 模拟数字转换 (ADC)

21.1 模拟数字转换 (ADC)

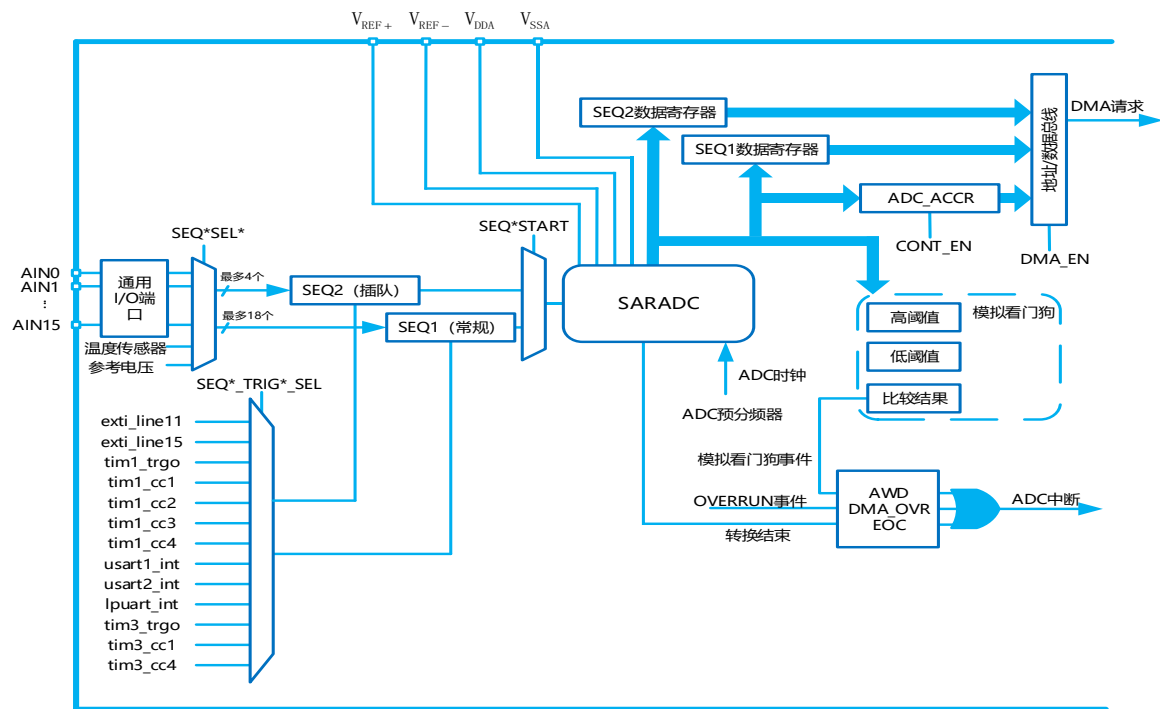
21.1.1 介绍

12 位 ADC 是逐次逼近式模数转换器。它具有多达 18 个多路复用通道，可以测量来自 16 个外部和两个内部信号源的信号。可以以多种方式定义 2 个序列进行各通道的模数转换。模拟看门狗功能允许应用程序检测输入电压是否超出用户定义的高/低阈值。ADC 输入时钟由 PCLK2 时钟除以预分频器产生，并且不得超过 16MHz。

21.1.2 主要特征

- 12 位分辨率
- 2 个序列定义，序列 1 顺序采集最多 18 个通道（任意定义）的信号，序列 2 最多定义 4 个通道；
- 2 个序列都可分别被外部事件或者软件控制开始；
- 序列 2 优先级高于序列 1；
- 序列 1 可以连续执行；
- 2 个序列都能进行看门狗比较；
- 序列 1 有累计模式；
- 12 种中断源可配；
- 支持 DMA 通道；
- ADC 转换时间：64MHz 主频下，若预分频倍数为 4 且采样周期为 4，则 ADC 时钟为 16MHz，转换时间则为 4 个采样周期加上 12 个转换周期，即 1us
- ADC 的电源要求：2.4 V 至 3.6 V
- ADC 输入范围：VREF- ≤ VIN ≤ VREF+

图21.1 ADC 模块功能框图



21.1.3 引脚描述

表21.1 ADC 的引脚

名称	信号类型	描述
V _{REF+}	输入, 模拟参考正电压	ADC 的高/正参考电压, $2.0V^{(3)} \leq V_{REF+} \leq V_{DDA}$
V _{REF-}	输入, 模拟参考负电压	ADC 的低/负参考电压, $V_{REF-} = V_{SSA}$
V _{DDA} ⁽¹⁾	输入, 模拟电源	模拟电源, $2.0V^{(3)} \leq V_{DDA} = V_{DD} \leq 5.5V$
V _{SSA} ⁽¹⁾	输入, 模拟地	模拟地, $V_{SSA} = V_{SS}$
ADCx_IN[15:0]	输入, 模拟信号	最多 16 个模拟通道 ⁽²⁾

[1] V_{DDA} 和 V_{SSA} 需要分别接到 VDD 和 VSS

[2] 详细了解 ADC 的 I/O 引脚请参考相应器件数据手册中“IO 功能-模拟模式”章节

[3] 2.0V 到 2.4V 性能稍差

21.1.4 通道选择

ADC 有 16 个外部通道, 2 个内部通道:

SEQ*_CH*_SEL 配置值	选择通道:
0	外部输入 AIN0
1	外部输入 AIN1
...	外部输入...
14	外部输入 AIN14
15	外部输入 AIN15
16	温度传感器
17	内部模拟 1.8v 电源

21.1.5 外部启动条件选择

ADC 有十多种外部启动条件可以选择, 每个序列可以同时选择最多 2 个进行启动:

SEQ*_TRIG*_SEL 配置值	序列启动条件	SEQ*_TRIG*_SEL 配置值	序列启动条件
0	无	8	USART1_INT
1	EXTI_LINE11	9	USART2_INT
2	EXTI_LINE15	10	LPUART_INT
3	TIM1_TRGO	11	无
4	TIM1_CC1	12	TIM3_TRGO
5	TIM1_CC2	13	TIM3_CC1
6	TIM1_CC3	14	TIM3_CC4
7	TIM1_CC4	其他	无

21.1.6 单序列中断产生举例

只打开序列 1 的情况下：

Case1: START 写 1,

- 对 seq1_sel1 选择的通道进行转换,
- 转换结束后同时产生 seq1 的 3 个中断: 1ch/1seq/1all。

Case2: START 写 1,

- 对 seq1_sel1 选择的通道进行转换,
- 转换结束后产生 seq1 的 1 个中断: 1ch,
- 对 seq1_sel2 选择的通道进行转换,
- 转换结束后产生 seq1 的 1 个中断: 1ch,
- ...
- 对 seq1_sel5 选择的通道进行转换,
- 转换结束后同时产生 seq1 的 3 个中断: 1ch/1seq/1all。

Case3: START 写 1,

- 对 seq1_sel1 选择的通道进行转换,
- 转换结束后产生 seq1 的 2 个中断: 1ch/1seq,
- 对 seq1_sel1 选择的通道进行转换,
- 转换结束后同时产生 seq1 的 3 个中断: 1ch/1seq/1all。

	case1	case2	case3	case4	case5
SEQ1_NUM	0	4	0	2	0
SEQ2_NUM	0	0	0	0	0
CONT_EN	0	0	1	1	1
CONT_NUM	0	0	1	2	0
SEQ1_START	1	1	1	1	1
SEQ2_START	0	0	0	0	0
	seq1_sel1	seq1_sel1	seq1_sel1	seq1_sel1	seq1_sel1
	1ch/1seq/all	1ch	1ch/1seq	1ch	1ch/1seq
		seq1_sel2	seq1_sel1	seq1_sel2	seq1_sel1
		1ch	1ch/1seq/all	1ch	1ch/1seq
		seq1_sel3		seq1_sel3	seq1_sel1
		1ch		1ch/1seq	1ch/1seq
		seq1_sel4		seq1_sel1	seq1_sel1
		1ch		1ch	1ch/1seq
		seq1_sel5		seq1_sel2	seq1_sel1
		1ch/1seq/all		1ch	1ch/1seq
				seq1_sel3	seq1_sel1
				1ch/1seq	1ch/1seq
				seq1_sel1	seq1_sel1
				1ch	1ch/1seq
				seq1_sel2	seq1_sel1

	case1	case2	case3	case4	case5
				1ch	1ch/1seq
				seq1_sel3	seq1_sel1
				1ch/1seq/all	1ch/1seq

21.1.7 双序列中断产生举例

在序列 1 工作过程中，打开序列 2，会插队优先完成序列 2 的采集，结束后再恢复序列 1：

SEQ1_NUM	2	
SEQ2_NUM	3	
CONT_EN	1	
CONT_NUM	2	
SEQ1_START	1	给 SEQ1_START 写 1 启动 SEQ1
SEQ2_START	0	
	seq1_sel1	采集 SEQ1 第一个通道
	1ch	产生 SEQ1_1CH 中断
	seq1_sel2	采集 SEQ1 第二个通道
	1ch	产生 SEQ1_1CH 中断
seq2_START	1	启动 SEQ2
	seq2_sel1	采集 SEQ2 第一个通道
	1ch	产生 SEQ2_1CH 中断
	seq2_sel2	采集 SEQ2 第二个通道
	1ch	产生 SEQ2_1CH 中断
	seq2_sel3	
	1ch	
	seq2_sel4	采集 SEQ2 第思个通道
	1ch/1seq	产生 SEQ2_1CH/1SEQ 中断
	seq1_sel3	采集 SEQ1 第三个通道
	1ch/1seq	产生 SEQ1_1CH/1SEQ 中断
	seq1_sel1	
	1ch	
	seq1_sel2	
	1ch	
	seq1_sel3	
	1ch/1seq	
	seq1_sel1	
	1ch	
	seq1_sel2	
	1ch	
	seq1_sel3	

SEQ1_NUM	2	
	1ch/1seq/all	

21.2 ADC 寄存器描述

21.2.1 ADC 中断状态寄存器 (ADC_INTR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				Dma_overnun	Seq1_1ch	Seq1_1seq	Seq1_all	Seq1_awd_reg	Seq1_awd_ht	Seq1_awd_lt	Seq2_1ch	Seq2_1seq	Seq2_awd_reg	Seq2_awd_ht	Seq2_awd_lt
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				Dma_overnun_sts	Seq1_1ch_sts	Seq1_1seq_sts	Seq1_all_sts	Seq1_awd_reg_sts	Seq1_awd_ht_sts	Seq1_awd_lt_sts	Seq2_1ch_sts	Seq2_1seq_sts	Seq2_awd_reg_sts	Seq2_awd_ht_sts	Seq2_awd_lt_sts
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:28	Reserved		0x0	rw
27	Dma_overnun	中断使能; 该位=1: 上一次的采集结果未被 DMA 搬走时来了下一次则产生中断;	0x0	rw
26	Seq1_1ch	中断使能; 该位=1: 序列 1 每完成 1 个通道采集就产生中断;	0x0	rw
25	Seq1_1seq	中断使能; 该位=1: 序列 1 每完成一整个序列的采集后产生中断;	0x0	rw
24	Seq1_all	中断使能; 该位=1: 序列 1 连续模式下最后一个序列采集后产生中断;	0x0	rw
23	Seq1_awd_reg	中断使能; 该位=1: 序列 1 的采集结果处于[LT,HT]区间内产生中断;	0x0	rw
22	Seq1_awd_ht	中断使能; 该位=1: 序列 1 的采集结果大于 HT 则产生中断;	0x0	rw
21	Seq1_awd_lt	中断使能; 该位=1: 序列 1 的采集结果小于 LT 则产生中断;	0x0	rw
20	Seq2_1ch	中断使能; 该位=1: 序列 2 每完成 1 个通道采集就产生中断;	0x0	rw
19	Seq2_1seq	中断使能; 该位=1: 序列 2 每完成一整个序列的采集后产生中断;	0x0	rw
18	Seq2_awd_reg	中断使能; 该位=1: 序列 2 的采集结果处于[LT,HT]区间内产生中断;	0x0	rw
17	Seq2_awd_ht	中断使能; 该位=1: 序列 2 的采集结果大于 HT 则产生中断;	0x0	rw
16	Seq2_awd_lt	中断使能; 该位=1: 序列 2 的采集结果小于 LT 则产生中断;	0x0	rw
15:12	Reserved		0x0	rw
11	Dma_overnun_sts	中断状态; bit27=1 后生效; 该位写 1 可清零;	0x0	rw
10	Seq1_1ch_sts	中断状态; bit26=1 后生效; 该位写 1 可清零;	0x0	rw
9	Seq1_1seq_sts	中断状态; bit25=1 后生效; 该位写 1 可清零;	0x0	rw
8	Seq1_all_sts	中断状态; bit24=1 后生效; 该位写 1 可清零;	0x0	rw
7	Seq1_awd_reg_sts	中断状态; bit23=1 后生效; 该位写 1 可清零;	0x0	rw
6	Seq1_awd_ht_sts	中断状态; bit22=1 后生效; 该位写 1 可清零;	0x0	rw

Bit	Field	Description	Reset	RW
5	Seq1_awd_lt_sts	中断状态; bit21=1 后生效; 该位写 1 可清零;	0x0	rw
4	Seq2_1ch_sts	中断状态; bit20=1 后生效; 该位写 1 可清零;	0x0	rw
3	Seq2_1seq_sts	中断状态; bit19=1 后生效; 该位写 1 可清零;	0x0	rw
2	Seq2_awd_reg_sts	中断状态; bit18=1 后生效; 该位写 1 可清零;	0x0	rw
1	Seq2_awd_ht_sts	中断状态; bit17=1 后生效; 该位写 1 可清零;	0x0	rw
0	Seq2_awd_lt_sts	中断状态; bit16=1 后生效; 该位写 1 可清零;	0x0	rw

21.2.2 ADC 控制寄存器 (ADC_CR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								SEQ1_CONT_NUM							
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
State_rst	Reserved	Ts_en	Dma_en	Reserved	SEQ1_ACC_EN	SEQ1_CONT_EN	SAMP	CLK_SEL		VREFSEL		Seq2_start	Seq1_start	adcen	
rw	r	r	rw	rw	r	rw	rw	rw	rw	rw	rw	w	w	w	rw

Bit	Field	Description	Reset	RW
31:24	Reserved		0x0	rw
23:16	SEQ1_CONT_NUM	序列 1 连续模式运行次数; 0: 永不结束; 1: 连续 2 次后结束; 2: 连续 3 次后结束; ... 255: 连续 256 次后结束;	0x0	rw
15	State_rst	1: 状态机复位	0x0	rw
14:13	Reserved		0x0	rw
12	TS_EN	1: 打开温度检测器模块;	0x0	rw
11	DMA_EN	1: 打开 DMA 接口;	0x0	rw
10	Reserved		0x0	rw
9	SEQ1_ACC_EN	1: 序列 1 的累计功能打开;	0x0	rw
8	SEQ1_CONT_EN	1: 序列 1 连续模式打开;	0x0	rw
7	SAMP	采样时间, 0: 4 个时钟, 1: 8 个时钟;	0x0	rw
6:4	CLK_SEL	ADC 时钟分频选择, 从 APB2 时钟分频; 0:1 分频; 1:2 分频; 2:4 分频; 3:8 分频; 4:16 分频; 5:32 分频; 6:64 分频; 7:128 分频;	0x0	rw
3	VREFSEL	参考电压选择信号: 0: 选择从 PB0 输入的参考电压 1: 选择电源作为参考电压	0x0	rw
2	Seq2_start	写 1 开始序列 2 的采集	0x0	rw
1	Seq1_start	写 1 开始序列 1 的采集	0x0	rw
0	ADCEN	1: 打开 ADC 使能	0x0	rw

21.2.3 ADC 数据寄存器 (ADC_DR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				SEQ2_RESULT											
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				SEQ1_RESULT											
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:28	Reserved		0x0	rw
27:16	SEQ2_RESULT	最新一次 SEQ2 采集结果	0x0	rw
15:12	Reserved		0x0	rw
11:0	SEQ1_RESULT	最新一次 SEQ1 采集结果	0x0	rw

21.2.4 ADC 状态寄存器 (ADC_SR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved							Seq1_work	Seq2_work	Cur_state			Seq1_cont[7:4]			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Seq1_cont[3:0]				Seq2_ch_cnt		Seq1_ch_cnt					Adc_ch				
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:25	Reserved		0x0	rw
24	Seq1_work	=1: 序列 1 正在排队采集中	0x0	rw
23	Seq2_work	=1: 序列 2 正在排队采集中	0x0	rw
22:20	Cur_state	当前工作状态	0x0	rw
19:12	Seq1_cont	循环模式下, 整个序列 1 的循环次数	0x0	rw
11:10	Seq2_ch_cnt	序列 2 内已完成的采集次数	0x0	rw
9:5	Seq1_ch_cnt	序列 1 内已完成的采集次数	0x0	rw
4:0	Adc_ch	当前采集通道编号	0x0	rw

21.2.5 ADC 序列 1 控制寄存器 1 (ADC_SEQ1CR1)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SEQ1_NUM				
r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		SEQ1_TRIG2_SEL						Reserved		SEQ1_TRIG1_SEL					
r	r	rw	rw	rw	rw	rw	rw	r	r	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:21	Reserved		0x0	rw
20:16	SEQ1_NUM	序列 1 采样次数; 0: 采样 1 次; 1: 采样 2 次; ... 17: 采样 18 次;	0x0	rw
15:14	Reserved		0x0	rw
13:8	SEQ1_TRIG2_SEL	序列 1 的 TRIG2 选择控制;	0x0	rw
7:6	Reserved		0x0	rw
5:0	SEQ1_TRIG1_SEL	序列 1 的 TRIG1 选择控制;	0x0	rw

21.2.6 ADC 序列 1 控制寄存器 2 (ADC_SEQ1CR2)

偏移地址: 0x14, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SEQ1_SEL6						SEQ1_SEL5				SEQ1_SEL4[4:1]			
r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEQ1_SEL4[0]		SEQ1_SEL3					SEQ1_SEL2				SEQ1_SEL1				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:30	Reserved		0x0	rw
29:25	SEQ1_SEL6	序列 1 第 6 次采样的通道编号	0x0	rw

Bit	Field	Description	Reset	RW
24:20	SEQ1_SEL5	序列 1 第 5 次采样的通道编号	0x0	rw
19:15	SEQ1_SEL4	序列 1 第 4 次采样的通道编号	0x0	rw
14:10	SEQ1_SEL3	序列 1 第 3 次采样的通道编号	0x0	rw
9:5	SEQ1_SEL2	序列 1 第 2 次采样的通道编号	0x0	rw
4:0	SEQ1_SEL1	序列 1 第 1 次采样的通道编号	0x0	rw

21.2.7 ADC 序列 1 控制寄存器 3 (ADC_SEQ1CR3)

偏移地址: 0x18, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SEQ1_SEL12					SEQ1_SEL11					SEQ1_SEL10[4:1]			
r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEQ1_SEL10[0]		SEQ1_SEL9					SEQ1_SEL8					SEQ1_SEL7			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:30	Reserved		0x0	rw
29:25	SEQ1_SEL12	序列 1 第 12 次采样的通道编号	0x0	rw
24:20	SEQ1_SEL11	序列 1 第 11 次采样的通道编号	0x0	rw
19:15	SEQ1_SEL10	序列 1 第 10 次采样的通道编号	0x0	rw
14:10	SEQ1_SEL9	序列 1 第 9 次采样的通道编号	0x0	rw
9:5	SEQ1_SEL8	序列 1 第 8 次采样的通道编号	0x0	rw
4:0	SEQ1_SEL7	序列 1 第 7 次采样的通道编号	0x0	rw

21.2.8 ADC 序列 1 控制寄存器 4 (ADC_SEQ1CR4)

偏移地址: 0x1C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SEQ1_SEL18					SEQ1_SEL17					SEQ1_SEL16[4:1]			
r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEQ1_SEL16[0]		SEQ1_SEL15					SEQ1_SEL14					SEQ1_SEL13			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:30	Reserved		0x0	rw
29:25	SEQ1_SEL18	序列 1 第 18 次采样的通道编号	0x0	rw
24:20	SEQ1_SEL17	序列 1 第 17 次采样的通道编号	0x0	rw
19:15	SEQ1_SEL16	序列 1 第 16 次采样的通道编号	0x0	rw
14:10	SEQ1_SEL15	序列 1 第 15 次采样的通道编号	0x0	rw
9:5	SEQ1_SEL14	序列 1 第 14 次采样的通道编号	0x0	rw
4:0	SEQ1_SEL13	序列 1 第 13 次采样的通道编号	0x0	rw

21.2.9 ADC 序列 1 状态寄存器 1/..18 (ADC_SEQ1SRx) (x=1/..18)

偏移地址: 0x20-0x64, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				SEQ1_RESULT1/2/.../18											
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:12	Reserved		0x0	rw
11:0	SEQ1_RESULT1/2/.../18	SEQ1 中各次采样转换的结果	0x0	rw

21.2.10 ADC 序列 2 控制寄存器 1 (ADC_SEQ2CR1)

偏移地址: 0x68, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														SEQ2_NUM	
r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		SEQ2_TRIG2_SEL						Reserved		SEQ2_TRIG1_SEL					
r	r	rw	rw	rw	rw	rw	rw	r	r	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:18	Reserved		0x0	rw
17:16	SEQ2_NUM	序列 2 采样次数; 0: 采样 1 次; 1: 采样 2 次; 2: 采样 3 次; 3: 采样 4 次;	0x0	rw
15:14	Reserved		0x0	rw
13:8	SEQ2_TRIG2_SEL	序列 2 的 TRIG2 选择控制;	0x0	rw
7:6	Reserved		0x0	rw
5:0	SEQ2_TRIG1_SEL	序列 2 的 TRIG1 选择控制;	0x0	rw

21.2.11 ADC 序列 2 控制寄存器 2 (ADC_SEQ2CR2)

偏移地址: 0x6C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												SEQ2_SEL4[4:1]			
r	r	r	r	r	r	r	r	r	r	r	r	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEQ2_SEL4[0]		SEQ2_SEL3					SEQ2_SEL2					SEQ2_SEL1			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:20	Reserved		0x0	rw
19:15	SEQ2_SEL4	序列 2 第 4 次采样的通道编号	0x0	rw
14:10	SEQ2_SEL3	序列 2 第 3 次采样的通道编号	0x0	rw
9:5	SEQ2_SEL2	序列 2 第 2 次采样的通道编号	0x0	rw
4:0	SEQ2_SEL1	序列 2 第 1 次采样的通道编号	0x0	rw

21.2.12 ADC 序列 2 状态寄存器 1/2/3/4 (ADC_SEQ2SRx) (x=1/2/3/4)

偏移地址: 0x70-0x7c, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				SEQ2_RESULT1/2/3/4											
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:12	Reserved		0x0	rw
11:0	SEQ2_RESULT	SEQ2 中各次采样转换的结果	0x0	rw

21.2.13 ADC 累计寄存器 (ADC_ACCR)

偏移地址: 0x80, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												ACC[19:16]			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ACC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:20	Reserved		0x0	rw
19:0	ACC	转换累计值	0x0	rw

21.2.14 ADC 看门狗控制寄存器 (ADC_AWDCR)

偏移地址: 0x84, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								AWD_HT[11:4]							
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AWD_HT[3:0]				AWD_LT											
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:24	Reserved		0x0	rw
23:12	AWD_HT	看门狗高阈值, 当转换结果超过该阈值时触发 HT 相关事项	0x0	rw
11:0	AWD_LT	看门狗低阈值, 当转换结果低于该阈值时触发 LT 相关事项	0x0	rw

22 电压比较器(VC)

22.1 电压比较器(VC)

22.1.1 VC 简介

模拟电压比较器 VC 用于比较两个输入模拟电压的大小，并根据比较结果输出高/低电平。当“+”输入端电压高于“-”输入端电压时，电压比较器输出高电平；当“+”输入端电压低于“-”输入端电压时，电压比较器输出低电平。

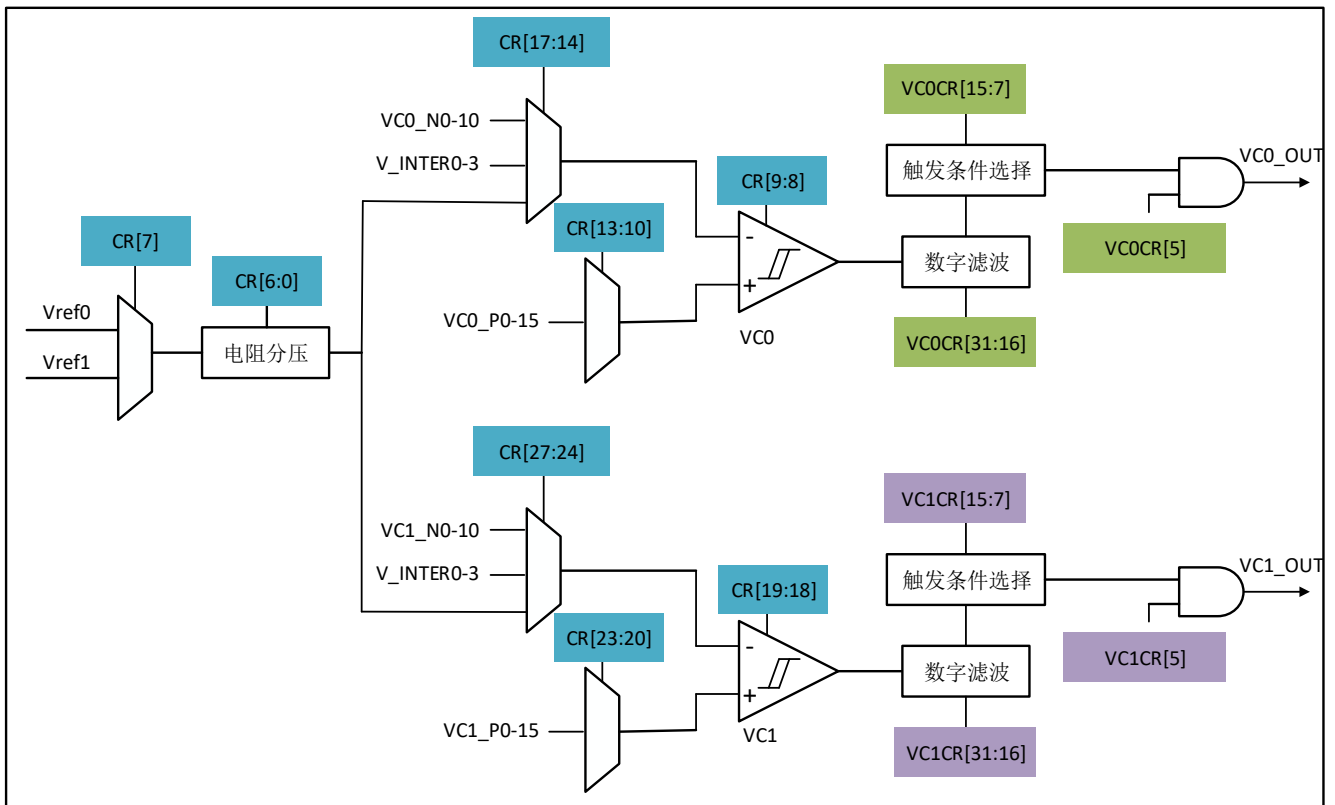
“+”输入端与“-”输入端均支持 4 路电源输入选择。

两种触发条件：高电平、上升沿、下降沿组合。

2 种触发结果：中断、复位信号。中断与复位信号不能同时产生。

22.1.2 VC 框图

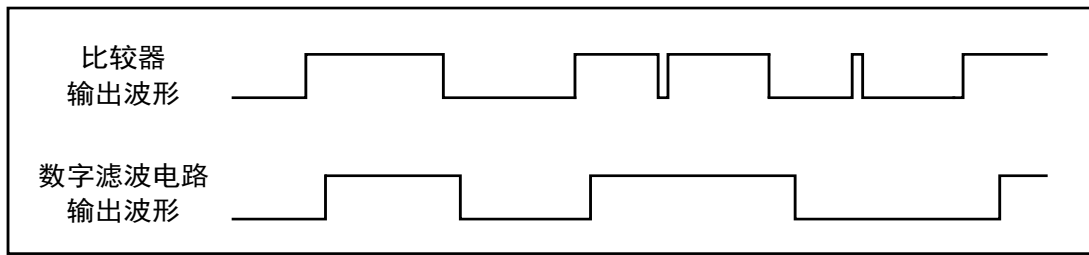
图22.1 VC 框图



22.1.3 数字滤波

如果芯片的工作环境恶劣，迟滞比较器的输出会出现噪声信号。使能数字滤波模块，则迟滞比较器的输出波形中脉宽小于FLT_NUM[15:0]设定时间的噪声信号都可以被滤除。禁止数字滤波模块，则数字滤波模块的输入输出信号相同。使能数字滤波模块，滤波示意如下所示：

图22.2 VC 滤波输出



22.2 VC 寄存器列表

基地址: 0x4000 5C00

偏移地址	名称	描述	默认值
0x00	VCCR	VC 控制寄存器	0x00000000
0x04	VCSR	VC 状态寄存器	0x00000000
0x08	VC0CR	VC0 控制寄存器	0x00000000
0x10	VC1CR	VC1 控制寄存器	0x00000000

22.3 VC 寄存器说明

22.3.1 VC 控制寄存器 (VCCR)

偏移地址: 0x00, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				VC1_PSEL				VC1_NSEL				VC1_HYS_SEL		VC0_PSEL[3:2]	
r	r	r	r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VC0_PSEL[1:0]		VC0_NSEL				VC0_HYS_SEL		VREFSEL	REF_RSEL						
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset	RW
31:2 8	Reserved		0x0	r
27:2 4	VC1_N_SEL	0:VC_VREF*N/128 1:Vref_ADC 2:Vts 3:VCAP 4:VC1_N0; 5:VC1_N1; ... 14:VC1_N10; 15:GND	0x0	rw
23:2 0	VC1_P_SEL	0:VC1_P0; 1:VC1_P1; ... 15:VC1_P15;	0x0	rw
19:1 8	VC1_HYS_SEL	VC1 迟滞选择; 0:无迟滞; 1:10mV; 2:20mV; 3:30mV;	0x0	rw

Bit	Field	Description	Reset	RW
17:1 4	VC0_N_SEL	0:VC_VREF*N/128 1:Vref_ADC 2:Vts 3:VCAP 4:VC0_N0; 5:VC0_N1; ... 14:VC0_N10; 15:GND	0x0	rw
13:1 0	VC0_P_SEL	0:VC0_P0; 1:VC0_P1; ... 15:VC0_P15;	0x0	rw
9:8	VC0_HYS_SEL	VC0 迟滞选择; 0:无迟滞; 1:10mV; 2:20mV; 3:30mV;	0x0	rw
7	VREFSEL	0:AVDD50; 1:Vref_ADC;	0x0	rw
6:0	VREF_RSEL	参考电压电阻分压选择; 0x0-0x3F:0 0x40:1*Vref/64; 0x41:2*Vref/64; ... 0x7F:64*Vref/64;	0x0	rw

22.3.2 VC 状态寄存器 (VCSR)

偏移地址: 0x04, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												Vc1out	Vc0out	Vc1int	Vc0int
r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw	rw

Bit	Field	Description	Reset	RW
31:4	Reserved		0x0	r
3	Vc1out	Vc1 输出状态 1: VC1 输出 0: VC1 无输出	0x0	rw
2	Vc0out	Vc0 输出状态 1: VC0 输出 0: VC0 无输出	0x0	rw
1	Vc1int	读 1: 发生 vc1 中断; 写 0: 清除;	0x0	rw
0	Vc0int	读 1: 发生 vc0 中断; 写 0: 清除;	0x0	rw

22.3.3 VC0 控制寄存器 (VC0CR)

偏移地址: 0x08, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
filter number															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
inten	Det_hien	Det_riseen	Det_fallen	Inv_en	Mix_en	Clk_sel		Filter_en	Reserved	vcen	Reserved				
rw	rw	rw	rw	rw	rw	rw	rw	rw	r	rw	r	r	r	r	r

Bit	Field	Description	Reset	RW
31:16	FILTER_NUM	VC0 采样滤波计数值 采样时钟为 APB 时钟或者 LSI 等。 滤波计数值可配置。采样次数达到滤波计数值时，结果一致输出。 采样计数周期=FLT_NUM[15:0]	0x0	rw
15	INT_EN	VC0 中断使能 0: 禁止 1: 使能	0x0	rw
14	DET_HI_EN	VC0 输出信号高电平触发使能 0: 禁止 1: 使能	0x0	rw
13	DET_RISE_EN	VC0 输出信号上升沿触发使能 0: 禁止 1: 使能	0x0	rw
12	DET_FALL_EN	VC0 输出信号下降沿触发使能 0: 禁止 1: 使能	0x0	rw
11	INV_EN	VC0 模拟比较输出信号极性 0: 高有效; 1: 低有效;	0x0	rw
10	MIX_EN	VC0 模拟比较输出信号混合 0: 只看 VC0; 1: 将 VC0 和 VC1 混合;	0x0	rw
9:8	CLK_SEL	VC0 滤波时钟选择 00: 滤波时钟无效 01: 滤波时钟选择为 PCLK 10: 滤波时钟选择为 LSI 11: 滤波时钟选择为 LSE	0x0	rw
7	FILTER_EN	VC0 数字滤波功能配置 0: 禁止数字滤波 1: 使能数字滤波	0x0	rw
6	Reserved		0x0	r
5	Vc0_EN	VC0 Enable 0: 电压比较功能禁止 1: 电压比较功能使能	0x0	rw
4:0	Reserved		0x0	r

22.3.4 VC1 控制寄存器 (VC1CR)

偏移地址: 0x10, 复位值: 0x0000 0000。

bit31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
filter number															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
bit15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
inten	Det_hien	Det_riseen	Det_fallen	Inv_en	Mix_en	Clk_sel		Filter_en	Reserved	vcen	Reserved				
rw	rw	rw	rw	rw	rw	rw	rw	rw	r	rw	r	r	r	r	r



Bit	Field	Description	Reset	RW
31:16	FILTER_NUM	VC1 采样滤波计数值 采样时钟为 APB 时钟或者 LSI 等。 滤波计数值可配置。采样次数达到滤波计数值时，结果一致输出。 采样计数周期=FLT_NUM[15:0]	0x0	rw
15	INT_EN	VC1 中断使能 0: 禁止 1: 使能	0x0	rw
14	DET_HI_EN	VC1 输出信号高电平触发使能 0: 禁止 1: 使能	0x0	rw
13	DET_RISE_EN	VC1 输出信号上升沿触发使能 0: 禁止 1: 使能	0x0	rw
12	DET_FALL_EN	VC1 输出信号下降沿触发使能 0: 禁止 1: 使能	0x0	rw
11	INV_EN	VC1 模拟比较输出信号极性 0: 高有效; 1: 低有效;	0x0	rw
10	MIX_EN	VC1 模拟比较输出信号混合 0: 只看 VC1; 1: 将 VC0 和 VC1 混合;	0x0	rw
9:8	CLK_SEL	VC1 滤波时钟选择 00: 滤波时钟无效 01: 滤波时钟选择为 PCLK 10: 滤波时钟选择为 LSI 11: 滤波时钟选择为 LSE	0x0	rw
7	FILTER_EN	VC0 数字滤波功能配置 0: 禁止数字滤波 1: 使能数字滤波	0x0	rw
6	Reserved		0x0	r
5	Vc1_EN	VC1 Enable 0: 电压比较功能禁止 1: 电压比较功能使能	0x0	rw
4:0	Reserved		0x0	r

23 低电压检测器(LVD)

23.1 低电压检测器(LVD)

23.1.1 LVD 简介

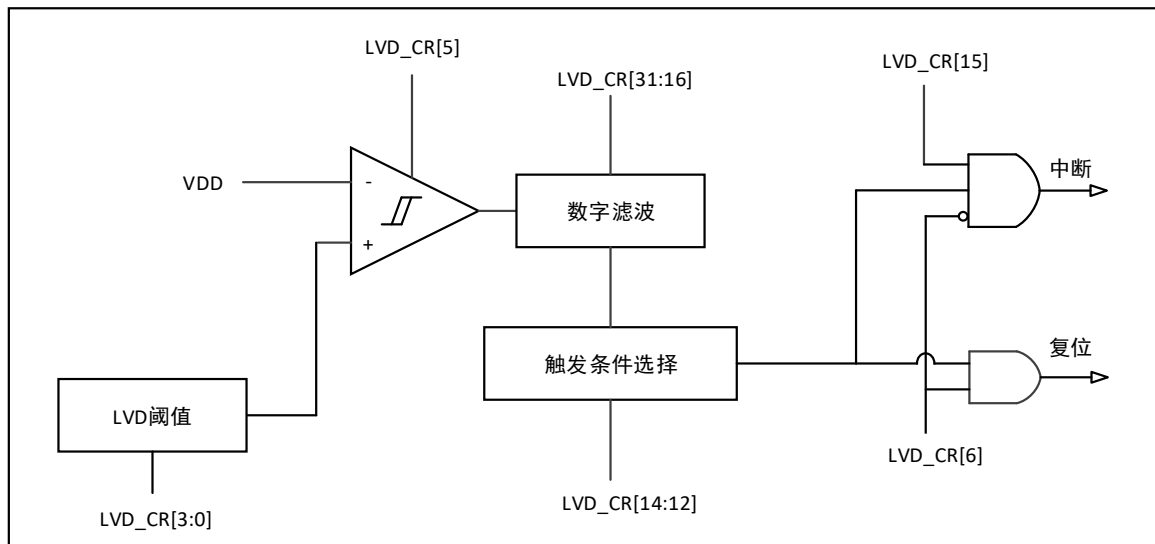
LVD 可用于监测工作电压，当被监测电压与 LVD 阈值的比较结果满足触发条件时，LVD 会产生中断或复位信号。中断或复位信号只能被中断或复位清零信号清除。只有当中断或复位信号被清零后，才会在触发条件下，再次产生中断或复位信号。

采样滤波时钟可配置，可配置为 APB 时钟或者 LIRC。滤波计数值可配置。采样达到滤波计数值次数时结果一致输出。

- 种触发条件：高电平、上升沿、下降沿组合。
- 种触发结果：中断、复位信号(禁止在滤波时钟选择 PCLK 时选择产生复位信号)。中断与复位信号不能同时产生。

23.1.2 LVD 框图

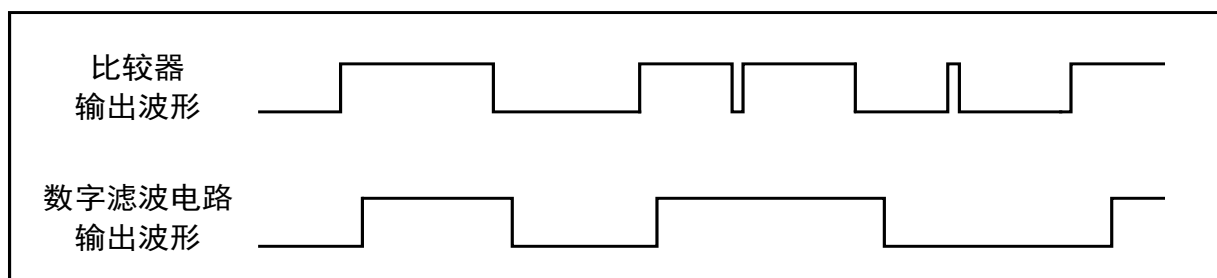
图23.1 LVD 结构框图



23.1.3 数字滤波

如果芯片的工作环境恶劣，迟滞比较器的输出会出现噪声信号。使能数字滤波模块，则迟滞比较器的输出波形中脉宽小于 LVD_CR.FLT_NUM[15:0] 设定时间的噪声信号都可以被滤除。禁止数字滤波模块，则数字滤波模块的输入输出信号相同。使能数字滤波模块，滤波示意如下所示：

图23.2 LVD 滤波输出



23.1.4 配置示例

23.1.4.1 LVD 配置为低电压复位

在本模式下，监测电压低于阈值电压时复位 MCU。配置方法如下所示：

Step1: 配置 LVD_CR.DIV_SEL，选择待监测的电压分压。

Step2: 配置 LVD_CR.FLT_NUM，选择 LVD 滤波时间。

Step3: 配置 LVD_CR.FLTCLK_SEL，选择滤波时钟。

Step4: 配置 LVD_CR.FLTEN，使能 LVD 滤波。

Step5: 设置 LVD_CR.HIGHINTEN 为 1，选择高电平触发 LVD 动作。

Step6: 设置 LVD_CR.ACT 为 1，选择 LVD 动作为复位。

Step7: 设置 LVD_CR.LVDEN 为 1，使能 LVD。

23.1.4.2 LVD 配置为电压变化中断

在本模式下，监测电压高于或低于阈值电压时产生中断。配置方法如下所示：

Step1: 配置 LVD_CR.DIV_SEL 选择待监测的电压来源。

Step2: 配置 LVD_CR.FLT_NUM，选择 LVD 滤波时间。

Step3: 配置 LVD_CR.FLTCLK_SEL，选择滤波时钟。

Step4: 配置 LVD_CR.FLTEN，使能 LVD 滤波。

Step5: 设置 LVD_CR.RISEINTEN 为 1、或 LVD_CR.FALLINTEN 为 1、或两者都为 1，选择电平变化触发 LVD 动作。

Step6: 设置 LVD_CR.ACT 为 0，选择 LVD 动作为中断。

Step7: 设置 LVD_CR.INT_EN 为 1，使能 LVD 中断。

Step8: 设置 LVD_CR.LVDEN 为 1，使能 LVD。

Step9: 在中断服务程序中向 LVD_SR.INTF 写入 0 以清除中断标志。

23.2 寄存器列表

基地址：0x4001 5000

偏移地址	名称	描述	默认值
0x00	LVD_CR	LVD 控制寄存器	0x000 0000
0x04	LVD_SR	LVD 状态寄存器	0x000 0000

23.3 寄存器说明

23.3.1 LVD 控制寄存器 (LVD_CR)

偏移地址：0x00，复位值：0x0000 0007。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLT_NUM[15:0]															
R/W															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INT_EN	HIGHINTEN	RISEINTEN	FALLINTEN	Reserved		FLTCLK_SEL[1:0]		FLTEN	ACT	LVDEN	Reserved		DIV_SEL[2:0]		
R/W	R/W	R/W	R/W	-		R/W		R/W	R/W	R/W	-		R/W		

Bit	Field	Description	Reset	RW
31:16	FLT_NUM	LVD 采样滤波计数值 采样时钟为 ABP 时钟或者 LIRC。滤波计数值可配置。采样次数达到滤波计数值时，结果一致输出。 采样计数周期=FLT_NUM[15:0]	0x0	R/W
15	INT_EN	LVD 中断使能 0: 禁止 1: 使能	0	R/W
14	HIGHINTEN	高电平触发使能(VDD 低于阈值电压) 0: 禁止 1: 使能	0	R/W
13	RISEINTEN	上升沿触发使能(VDD 从高于阈值电压变为低于阈值电压) 0: 禁止 1: 使能	0	R/W
12	FALLINTEN	下降沿触发使能(VDD 从低于阈值电压变为高于阈值电压) 0: 禁止 1: 使能	0	R/W
11:10	Reserved	保留	0x0	-
9:8	FLTCLK_SEL	滤波时钟选择 00: 滤波时钟无效 01: 滤波时钟选择为 PCLK(只能配置成中断模式) 10: 滤波时钟选择为 LSI 11: 滤波时钟选择为 LSE	0x0	R/W
7	FLTEN	数字滤波功能配置 0: 禁止数字滤波 1: 使能数字滤波	0	R/W
6	ACT	LVD 中断复位选择位 0: 产生中断 1: 产生复位	0	R/W
5	LVDEN	LVD 使能 0: 禁止 LVD 1: 使能 LVD	0	R/W
4	Reserved	保留	0x0	-
3:0	DIV_SEL	LVD 分压配置，带迟滞效果； 0: 低压 1.75V，高压 1.85V； 1: 低压 1.85V，高压 1.95V； 2: 低压 1.95V，高压 2.05V； 3: 低压 2.05V，高压 2.15V； 4: 低压 2.15V，高压 2.25V； 5: 低压 2.25V，高压 2.35V； 6: 低压 2.35V，高压 2.45V； 7: 低压 2.45V，高压 2.55V； 8: 低压 2.55V，高压 2.65V； 9: 低压 2.65V，高压 2.75V； 10: 低压 2.75V，高压 2.85V； 11: 低压 2.85V，高压 2.95V； 12: 低压 2.95V，高压 3.05V； 13: 低压 3.05V，高压 3.15V； 14: 低压 3.15V，高压 3.25V； 15: 低压 3.25V，高压 3.35V；	0x0	R/W

23.3.2 LVD 状态寄存器 (LVD_SR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24		23	22	21	20	19	18	17	16
Reserved																
-																
15	14	13	12	11	10	9	8		7	6	5	4	3	2	1	0
Reserved															LVDOUT	INTF
-															R	W0C

Bit	Field	Description	Reset	RW
31:2	Reserved	保留	0x0	-
1	LVDOUT	LVD 输出值。	0	R
0	INTF	LVD 中断标志: 0: 未发生 LVD 中断 1: 发生 LVD 中断 写 0 清除中断标志, 写 1 无效。	0	W0C

第五部分 通信接口

24 通用同步异步收发器 (USART)

24.1 功能描述

通用同步异步接收器发送器 (USART) 提供了灵活的选择与需要行业标准 NRZ 异步串行数据格式的外部设备进行全双工数据交换的方式。USART 使用分数波特率发生器提供了非常广泛的波特率。它支持同步单向通信和半双工单线通信, 它还支持 LIN (本地互连网络), 智能卡协议和 IrDA (红外数据协会) SIR ENDEC 规范以及调制解调器操作 (CTS / RTS)。它允许多处理器通信。

通过使用 DMA 进行多缓冲区配置, 可以进行高速数据通信。

该接口通过三个引脚从外部连接到另一个设备。任何 USART 双向通信都至少需要两个引脚: 接收数据输入 (RX) 和发送数据输出 (TX):

RX: 接收数据输入是串行数据输入。通过区分有效的输入数据和噪声, 通过采样技术可用于数据恢复。

TX: 发送数据输出。禁用发送器后, 输出引脚将返回其 IO 端口配置。使能发送器且不发送任何内容时, TX 引脚为高电平。在单线和智能卡模式下, 此 IO 用于发送数据和接收数据 (在 USART 级别, 然后在 SW_RX 上接收数据)。

- 总线在发送或接收前应处于空闲状态
- 一个起始位
- 一个数据字(8 或 9 位), 最低有效位在前
- 0.5, 1.5, 2 个的停止位, 由此表明数据帧的结束
- 使用分数波特率发生器 —— 12 位整数和 4 位小数的表示方法。
- 一个状态寄存器(USART_SR)
- 数据寄存器(USART_DR)
- 一个波特率寄存器(USART_BRR), 12 位的整数和 4 位小数
- 一个智能卡模式下的保护时间寄存器(USART_GTPR)

关于以上寄存器中每个位的具体定义, 请参考寄存器描述: USART 寄存器描述。

在同步模式中需要下列引脚:

- **CK:** 发送器时钟输出。此引脚输出用于同步传输的 时钟, (在 Start 位和 Stop 位上没有时钟
- 脉冲, 软件可选地, 可以在最后一个数据位送出一个时钟脉冲)。数据可以在 RX 上同步被接
- 收。这可以用来控制带有移位寄存器的外部设备(例如 LCD 驱动器)。时钟相位和极性都是软
- 件可编程的。在智能卡模式里, CK 可以为智能卡提供时钟。
- 在 IrDA 模式里需要下列引脚:
- **IrDA_RDI:** IrDA 模式下的数据输入。
- **IrDA_TDO:** IrDA 模式下的数据输出。

下列引脚在硬件流控模式中需要:

- **nCTS:** 清除发送, 若是高电平, 在当前数据传输结束时阻断下一次的数据发送。
- **nRTS:** 发送请求, 若是低电平, 表明 USART 准备好接收数据

主要特点

- 全双工，异步通信
- NRZ 标准格式
- 小数波特率发生器系统
 - 通用的可编程发送和接收波特率，最高可达 4.5 MBits/s
- Data: 可编程数据字长 (8 或 9 位)
- Stop: 可配置的停止位-支持 1 或 2 个停止位
- LIN 主站同步断开发送功能和 LIN 从站断开检测功能
 - 当 USART 配置硬件为 LIN 时，产生 13 位断开符和 10/11 位断开符检测
- 发送器提供时钟输出，用于同步发送
- IrDA SIR 编码器解码器
 - 在正常模式下支持 3/16 位的持续时间
- 智能卡仿真功能
 - 智能卡接口支持 ISO 7816-3 标准中定义的异步协议
 - 智能卡 0.5、1.5 停止位，用于智能卡操作
- 单线半双工通讯
- 可配置的使用 DMA 的多缓冲器通信
 - 在 SRAM 里利用集中式 DMA 缓冲接收/发送字节
- 发送器和接收器的单独使能位
- 传输检测标志:
 - 接收缓冲区已满
 - 发送缓冲区为空
 - 传输结束标志
- 奇偶校验控制:
 - 发送奇偶校验位
 - 检查接收到的数据字节的奇偶校验
- 四个错误检测标志:
 - 溢出错误
 - 噪声错误
 - 帧误差
 - 奇偶校验错误
- 带有标志的十个中断源:
 - CTS 更改
 - LIN 断开符检测
 - 发送数据寄存器为空
 - 发送完成
 - 接收数据寄存器为满
 - 检测到总线为空闲线
 - 溢出错误
 - 帧错误
 - 噪声错误
 - 奇偶校验错误
- 多处理器通信-如果未发生地址匹配，则进入静音模式
- 从静音模式中唤醒 (通过空闲总线检测或地址标记检测)
- 两种接收器唤醒模式: 地址位 (MSB, 第 9 位), 空闲总线

24.2 USART 特性描述

字长可以通过编程 USART_CR1 寄存器中的 M 位, 选择成 8 或 9 位(见图 249)。在起始位期间, TX

脚处于低电平, 在停止位期间处于高电平。

空闲符号被视为完全由'1'组成的一个完整的数据帧, 后面跟着包含了数据的下一帧的开始位('1'的位数也包括了停止位的位数)。

断开符号 被视为在一个帧周期内全部收到'0'(包括停止位期间, 也是'0')。在断开帧结束时, 发送器再插入 1 或 2 个停止位('1')来应答起始位。

发送和接收由一共用的波特率发生器驱动, 当发送器和接收器的使能位分别置位时, 分别为其产生时钟。

24.2.1 发送器

发送器根据 M 位的状态发送 8 位或 9 位的数据字。当发送使能位(TE)被设置时, 发送移位寄存器中的数据在 TX 脚上输出, 相应的时钟脉冲在 CK 脚上输出。

24.2.2 字符发送

在 USART 发送期间, 在 TX 引脚上首先移出数据的最低有效位。在此模式里, USART_DR 寄存器包含了一个内部总线和发送移位寄存器之间的缓冲器。

每个字符之前都有一个低电平的起始位; 之后跟着的停止位, 其数目可配置。

USART 支持多种停止位的配置: 0.5、1、1.5 和 2 个停止位。

注: 1 在数据传输期间不能复位 TE 位, 否则将破坏 TX 脚上的数据, 因为波特率计数器停止计数。正在传输的当前数据将丢失。2, TE 位被激活后将发送一个空闲帧。

配置步骤:

- 通过在 USART_CR1 寄存器上置位 UE 位来激活 USART
- 编程 USART_CR1 的 M 位来定义字长。
- 在 USART_CR2 中编程停止位的位数。
- 如果采用多缓冲器通信, 配置 USART_CR3 中的 DMA 使能位(DMAT)。按多缓冲器通信中的描述配置 DMA 寄存器。
- 利用 USART_BRR 寄存器选择要求的波特率。
- 设置 USART_CR1 中的 TE 位, 发送一个空闲帧作为第一次数据发送。
- 把要发送的数据写进 USART_DR 寄存器(此动作清除 TXE 位)。在只有一个缓冲器的情况下, 对每个待发送的数据重复步骤 7。
- 在 USART_DR 寄存器中写入最后一个数据字后, 要等待 TC=1, 它表示最后一个数据帧的传输结束。当需要关闭 USART 或需要进入停机模式之前, 需要确认传输结束, 避免破坏最后一次传输。

24.2.3 可配置的停止位

随每个字符发送的停止位的位数可以通过控制寄存器 CR2 的位 13、12 进行编程。

- 1 个停止位: 停止位位数的默认值。
- 2 个停止位: 可用于常规 USART 模式、单线模式以及调制解调器模式。
- 0.5 个停止位: 在智能卡模式下接收数据时使用。

- 1.5 个停止位：在智能卡模式下发送和接收数据时使用。

空闲帧包括了停止位。

断开帧是 10 位低电平，后跟停止位(当 $m=0$ 时)；或者 11 位低电平，后跟停止位($m=1$ 时)。不可能传输更长的断开帧(长度大于 10 或者 11 位)。

24.2.4 单字节通讯

清零 TXE 位总是通过对数据寄存器的写操作来完成的。TXE 位由硬件来设置，它表明：

- 数据已经从 TDR 移送到移位寄存器，数据发送已经开始
- TDR 寄存器被清空
- 下一个数据可以被写进 USART_DR 寄存器而不会覆盖先前的数据

如果 TXEIE 位被设置，此标志将产生一个中断。

如果此时 USART 正在发送数据，对 USART_DR 寄存器的写操作把数据存进 TDR 寄存器，并在当前传输结束时把该数据复制进移位寄存器。

如果此时 USART 没有在发送数据，处于空闲状态，对 USART_DR 寄存器的写操作直接把数据放进移位寄存器，数据传输开始，TXE 位立即被置起。

当一帧发送完成时(停止位发送后)并且设置了 TXE 位，TC 位被置起，如果 USART_CR1 寄存器中的 TCIE 位被置起时，则会产生中断。

在 USART_DR 寄存器中写入了最后一个数据字后，在关闭 USART 模块之前或设置微控制器进入低功耗模式(详见下图)之前，必须先等待 TC=1。

使用下列软件过程清除 TC 位：

- 读一次 USART_SR 寄存器；
- 写一次 USART_DR 寄存器。

注：TC 位也可以通过软件对它写 '0' 来清除。此清零方式只推荐在多缓冲器通信模式下使用。

24.2.5 断开符号

设置 SBK 可发送一个断开符号。断开帧长度取决 M 位(见图 249)。如果设置 SBK=1，在完成当前数据发送后，将在 TX 线上发送一个断开符号。断开字符发送完成时(在断开符号的停止位时)SBK 被硬件复位。USART 在最后一个断开帧的结束处插入一逻辑'1'，以保证能识别下一帧的起始位。

注意：如果在开始发送断开帧之前，软件又复位了 SBK 位，断开符号将不被发送。如果要发送两个连续的断开帧，SBK 位应该在前一个断开符号的停止位之后置起。

24.2.6 空闲符号

置位 TE 将使得 USART 在第一个数据帧前发送一空闲帧。

24.2.7 接收器

USART 可以根据 USART_CR1 的 M 位接收 8 位或 9 位的数据字。

24.2.8 起始位侦测

在 USART 中，如果辨认出一个特殊的采样序列，那么就认为侦测到一个起始位。

该序列为：1 1 1 0 X 0 X 0 X 0 0 0 0

24.2.9 字符接收

在 USART 接收期间，数据的最低有效位首先从 RX 脚移进。在此模式里，USART_DR 寄存器包

含的缓冲器位于内部总线和接收移位寄存器之间。

配置步骤:

- 将 USART_CR1 寄存器的 UE 置 1 来激活 USART。
- 编程 USART_CR1 的 M 位定义字长
- 在 USART_CR2 中编写停止位的个数
- 如果需多缓冲器通信, 选择 USART_CR3 中的 DMA 使能位(DMAR)。按多缓冲器通信所
- 要求的配置 DMA 寄存器。
- 利用波特率寄存器 USART_BRR 选择希望的波特率。
- 设置 USART_CR1 的 RE 位。激活接收器, 使它开始寻找起始位。

当一字符被接收到时,

- RXNE 位被置位。它表明移位寄存器的内容被转移到 RDR。换句话说, 数据已经被接收并且可以被读出(包括与之有关的错误标志)。
- 如果 RXNEIE 位被设置, 产生中断。
- 在接收期间如果检测到帧错误, 噪音或溢出错误, 错误标志将被置起,
- 在多缓冲器通信时, RXNE 在每个字节接收后被置起, 并由 DMA 对数据寄存器的读操作而清零。
- 在单缓冲器模式里, 由软件读 USART_DR 寄存器完成对 RXNE 位清除。RXNE 标志也可以通过对它写 0 来清除。RXNE 位必须在下一字符接收结束前被清零, 以避免溢出错误。

注意: 在接收数据时, RE 位不应该被复位。如果 RE 位在接收时被清零, 当前字节的接收被丢失。

24.2.10 断开符号

当接收到一个断开帧时, USART 像处理帧错误一样处理它

24.2.11 空闲符号

当一空闲帧被检测到时, 其处理步骤和接收到普通数据帧一样, 但如果 IDLEIE 位被设置将产生一个中断。

24.2.12 溢出错误

如果 RXNE 还没有被复位, 又接收到一个字符, 则发生溢出错误。数据只有当 RXNE 位被清零后才能从移位寄存器转移到 RDR 寄存器。RXNE 标记是接收到每个字节后被置位的。如果下一个数据已被收到或先前 DMA 请求还没被服务时, RXNE 标志仍是置起的, 溢出错误产生。

当溢出错误产生时:

- ORE 位被置位。
- RDR 内容将不会丢失。读 USART_DR 寄存器仍能得到先前的数据。
- 移位寄存器中以前的内容将被覆盖。随后接收到的数据都将丢失。
- 如果 RXNEIE 位被设置或 EIE 和 DMAR 位都被设置, 中断产生。
- 顺序执行对 USART_SR 和 USART_DR 寄存器的读操作, 可复位 ORE 位

注意: 当 ORE 位置位时, 表明至少有 1 个数据已经丢失。有两种可能性:

- 如果 RXNE=1, 上一个有效数据还在接收寄存器 RDR 上, 可以被读出。
- 如果 RXNE=0, 这意味着上一个有效数据已经被读走, RDR 已经没有东西可读。当上一个有效数据在 RDR 中被读取的同时又接收到新的(也就是丢失的)数据时, 此种情况可能

发生。在读序列期间(在 USART_SR 寄存器读访问和 USART_DR 读访问之间)接收到新的数据, 此种情况也可能发生。

24.2.13 噪音错误

使用过采样技术(同步模式除外), 通过区别有效输入数据和噪音来进行数据恢复。

当在接收帧中检测到噪音时:

- 在 RXNE 位的上升沿设置 NE 标志。
- 无效数据从移位寄存器传送到 USART_DR 寄存器。
- 在单个字节通信情况下, 没有中断产生。然而, 因为 NE 标志位和 RXNE 标志位是同时被设置, RXNE 将产生中断。在多缓冲器通信情况下, 如果已经设置了 USART_CR3 寄存器中 EIE 位, 将产生一个中断。

先读出 USART_SR, 再读出 USART_DR 寄存器, 将清除 NE 标志位

24.2.14 帧错误

当以下情况发生时检测到帧错误:

由于没有同步上或大量噪音的原因, 停止位没有在预期的时间上接和收识别出来。

当帧错误被检测到时:

- FE 位被硬件置起
- 无效数据从移位寄存器传送到 USART_DR 寄存器。
- 在单字节通信时, 没有中断产生。然而, 这个位和 RXNE 位同时置起, 后者将产生中断。在多缓冲器通信情况下, 如果 USART_CR3 寄存器中 EIE 位被置位的话, 将产生中断。

顺序执行对 USART_SR 和 USART_DR 寄存器的读操作, 可复位 FE 位。

24.2.15 接收期间的可配置的停止位

被接收的停止位的个数可以通过控制寄存器 2 的控制位来配置, 在正常模式时, 可以是 1 或 2 个,

在智能卡模式里可能是 0.5 或 1.5 个。

- 0.5 个停止位(智能卡模式中的接收): 不对 0.5 个停止位进行采样。因此, 如果选择 0.5 个停止位则不能检测帧错误和断开帧。
- 1 个停止位: 对 1 个停止位的采样在第 8, 第 9 和第 10 采样点上进行。
- 1.5 个停止位(智能卡模式): 当以智能卡模式发送时, 器件必须检查数据是否被正确的发送出去。所以接收器功能块必须被激活(USART_CR1 寄存器中的 RE =1), 并且在停止位的发送期间采样数据线上的信号。如果出现校验错误, 智能卡会在发送方采样 NACK 信号时, 即总线上停止位对应的时间内时, 拉低数据线, 以此表示出现了帧错误。FE 在 1.5 个停止位结束时和 RXNE 一起被置起。对 1.5 个停止位的采样是在第 16, 第 17 和第 18 采样点进行的。1.5 个的停止位可以被分成 2 部分: 一个是 0.5 个时钟周期, 期间不做任何事情。随后是 1 个时钟周期的停止位, 在这段时间的中点处采样。详见第 25.3.11 节: 智能卡。
- 2 个停止位: 对 2 个停止位的采样是在第一停止位的第 8, 第 9 和第 10 个采样点完成的。如果第一个停止位期间检测到一个帧错误, 帧错误标志将被设置。第二个停止位不再检查帧错误。在第一个停止位结束时 RXNE 标志将被设置。

24.2.16 分数波特率的产生

接收器和发送器的波特率在 USARTDIV 的整数和小数寄存器中的值应设置成相同。

$$Tx / Rx \text{ 波特率} = f_{CK} / (16 * USARTDIV)$$

这里的 f_{CK} 是给外设的时钟 (PCLK1 用于 USART2 、 3 、 4 、 5 , PCLK2 用于 USART1)

USARTDIV 是一个无符号的定点数。这 12 位的值设置在 USART_BRR 寄存器。

注：在写入 USART_BRR 之后，波特率计数器会被波特率寄存器的新值替换。因此，不要在通信进行中改变波特率寄存器的数值。

24.2.17 如何从 USART_BRR 寄存器值得到 USARTDIV

例 1 :

如果 $DIV_Mantissa = 27$, $DIV_Fraction = 12$ (USART_BRR=0x1BC),

于是

$$Mantissa (USARTDIV) = 27$$

$$Fraction (USARTDIV) = 12/16 = 0.75$$

$$\text{所以 } USARTDIV = 27.75$$

例 2 :

要求 $USARTDIV = 25.62$,

就有:

$$DIV_Fraction = 16 * 0.62 = 9.92$$

最接近的整数是: $10 = 0x0A$

$$DIV_Mantissa = mantissa (25.620) = 25 = 0x19$$

于是, $USART_BRR = 0x19A$

例 3 :

要求 $USARTDIV = 50.99$

就有:

$$DIV_Fraction = 16 * 0.99 = 15.84$$

最接近的整数是: $16 = 0x10 \Rightarrow DIV_frac[3:0]$ 溢出 $\Rightarrow$ 进位必须加到小数部分

$$DIV_Mantissa = mantissa (50.990 + \text{进位}) = 51 = 0x33$$

于是: $USART_BRR = 0x330$, $USARTDIV=51$

24.2.18 USART 接收器容忍时钟的变化

只有当整体的时钟系统地变化小于 USART 异步接收器能够容忍的范围, USART 异步接收器才能正常地工作。影响这些变化的因素有:

- DTRA: 由于发送器误差而产生的变化(包括发送器端振荡器的变化)

- DQUANT: 接收器端波特率取整所产生的误差
- DREC: 接收器端振荡器的变化
- DTCL: 由于传输线路产生的变化(通常是由于收发器在由低变高的转换时序, 与由高变低转换时序之间的不一致性所造成)。

需要满足: $DTRA + DQUANT + DREC + DTCL < USART$ 接收器的容忍度

对于正常接收数据, USART 接收器的容忍度等于最大能容忍的变化, 它依赖于下述选择:

- 由 USART_CR1 寄存器的 M 位定义的 10 或 11 位字符长度
- 是否使用分数波特率产生

24.2.19 多处理器通信

通过 USART 可以实现多处理器通信(将几个 USART 连在一个网络里)。例如某个 USART 设备可以是主, 它的 TX 输出和其他 USART 从设备的 RX 输入相连接; USART 从设备各自的 TX 输出逻辑地连在一起, 并且和主设备的 RX 输入相连接。

在多处理器配置中, 我们通常希望只有被寻址的接收者才被激活, 来接收随后的数据, 这样就可以减少由未被寻址的接收器的参与带来的多余的 USART 服务开销。

未被寻址的设备可启用其静默功能置于静默模式。在静默模式里:

- 任何接收状态位都不会被设置。
- 所有接收中断被禁止。
- USART_CR1 寄存器中的 RWU 位被置 1。RWU 可以被硬件自动控制或在某个条件下由软件写入。

根据 USART_CR1 寄存器中的 WAKE 位状态, USART 可以用二种方法进入或退出静默模式。

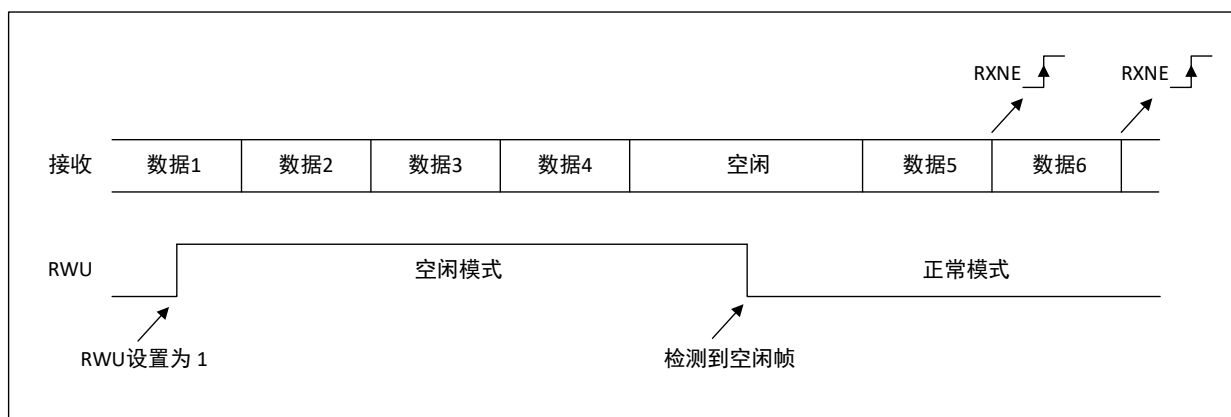
- 如果 WAKE 位被复位: 进行空闲总线检测。
- 如果 WAKE 位被设置: 进行地址标记检测。

24.2.20 线检测 空闲总线检测(WAKE=0)

当 RWU 位被写 1 时, USART 进入静默模式。当检测到一空闲帧时, 它被唤醒。然后 RWU 被硬件清零, 但是 USART_SR 寄存器中的 IDLE 位并不置起。RWU 还可以被软件写 0。

下图给出利用空闲总线检测来唤醒和进入静默模式的一个例子

图24.1 空闲总线检测



24.2.21 地址标记(address mark) 检测(WAKE=1)

在这个模式里，如果 MSB 是 1，该字节被认为是地址，否则被认为是数据。在一个地址字节中，目标接收器的地址被放在 4 个 LSB 中。这个 4 位地址被接收器同它自己地址做比较，接收器的地址被编程在 USART_CR2 寄存器的 ADD。

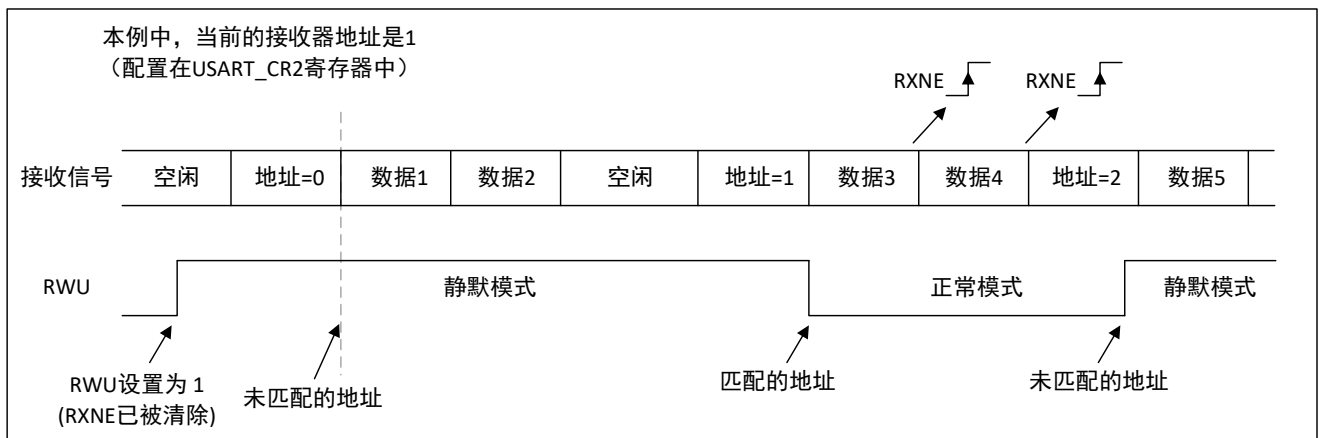
如果接收到的字节与它的编程地址不匹配时，USART 进入静默模式。此时，硬件设置 RWU 位。

接收该字节既不会设置 RXNE 标志也不会产生中断或发出 DMA 请求，因为 USART 已经在静默模式。

当接收到的字节与接收器内编程地址匹配时，USART 退出静默模式。然后 RWU 位被清零，随后的字节被正常接收。收到这个匹配的地址字节时将设置 RXNE 位，因为 RWU 位已被清零。

当接收缓冲器不包含数据时(USART_SR 的 RXNE=0)，RWU 位可以被写 0 或 1。否则，该次写操作被忽略。下图给出利用地址标记检测来唤醒和进入静默模式的例子。

图24.2 地址标志检测唤醒



24.2.22 校验控制

设置 USART_CR1 寄存器上的 PCE 位，可以使能奇偶控制(发送时生成一个奇偶位，接收时进行奇偶校验)。

24.2.23 LIN(局域互联网)

LIN 模式是通过设置 USART_CR2 寄存器的 LINEN 位选择。在 LIN 模式下，下列位必须保持为 0：

- USART_CR2 寄存器的 CLKEN 位
- USART_CR3 寄存器的 STOP[1:0], SCEN, HDSEL 和 IREN

24.2.24 LIN 发送

前面所描述的同样步骤适用于 LIN 主发送，但和正常 USART 发送有以下区别：

- 清零 M 位以配置 8 位字长
- 置位 LINEN 位以进入 LIN 模式。这时，置位 SBK 将发送 13 位'0'作为断开符号。然后发一位'1'，以允许对下一个开始位的检测。

24.2.25 LIN 接收

当 LIN 模式被使能时，断开符号检测电路被激活。该检测完全独立于 USART 接收器。断开符号只要一出现就能检测到，不管是在总线空闲时还是在发送某数据帧其间，数据帧还未完成，又插入了断开符号的发送。

当接收器被激活时(USART_CR1 的 RE=1), 电路监测 RX 上的起始信号。监测起始位的方法同检测断开符号或数据是一样的。当起始位被检测到后, 电路对每个接下来的位, 在每个位的第 8, 9, 10 个过采样时钟点上进行采样。如果 10 个(当 USART_CR2 的 LBDL = 0)或 11 个(当 USART_CR2 的 LBDL = 1)连续位都是'0', 并且又跟着一个定界符, USART_SR 的 LBD 标志被设置。如果 LBDIE 位=1, 中断产生。在确认定界符前, 要检查定界符, 因为它意味 RX 线已经回到高电平。

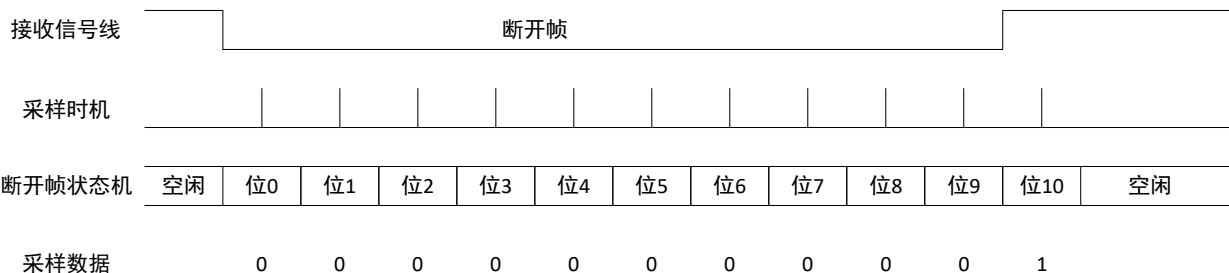
如果在第 10 或 11 个采样点之前采样到了'1', 检测电路取消当前检测并重新寻找起始位。如果 LIN 模式被禁止, 接收器继续如正常 USART 那样工作, 不需要考虑检测断开符号。

如果 LIN 模式没有被激活(LINEN=0), 接收器仍然正常工作于 USART 模式, 不会进行断开检测。

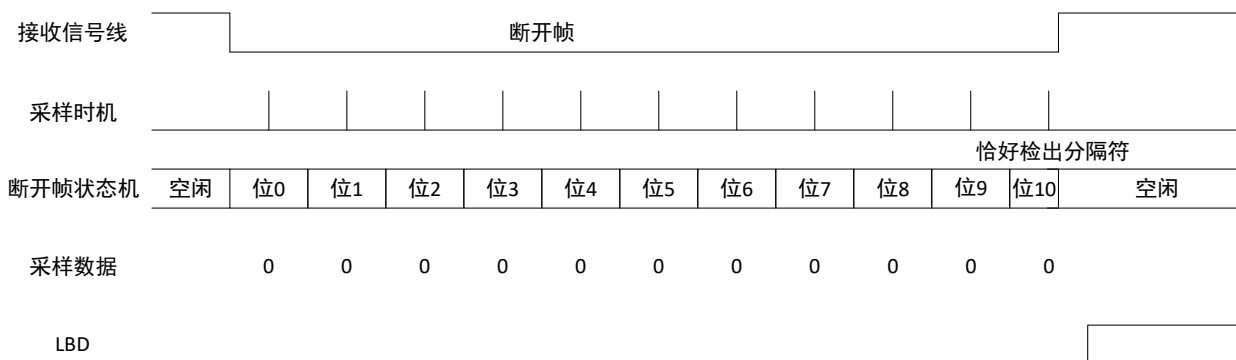
如果 LIN 模式被激活(LINEN=1), 只要一发生帧错误(也就是停止位检测到'0', 这种情况出现在断开帧), 接收器就停止, 直到断开符号检测电路接收到一个'1'(这种情况发生于断开符号没有完整的发出来), 或一个定界符(这种情况发生于已经检测到一个完整的断开符号)。

图24.3 LIN 模式下的断开检测(11 位断开长度 – 设置了 LBDL 位)

情形1: 断开信号不够长 => 丢弃这个帧, 不设置LBD



情形2: 断开信号刚好够长 => 检测到断开, 设置LBD



情形3: 断开信号足够长 => 检测到断开, 设置LBD

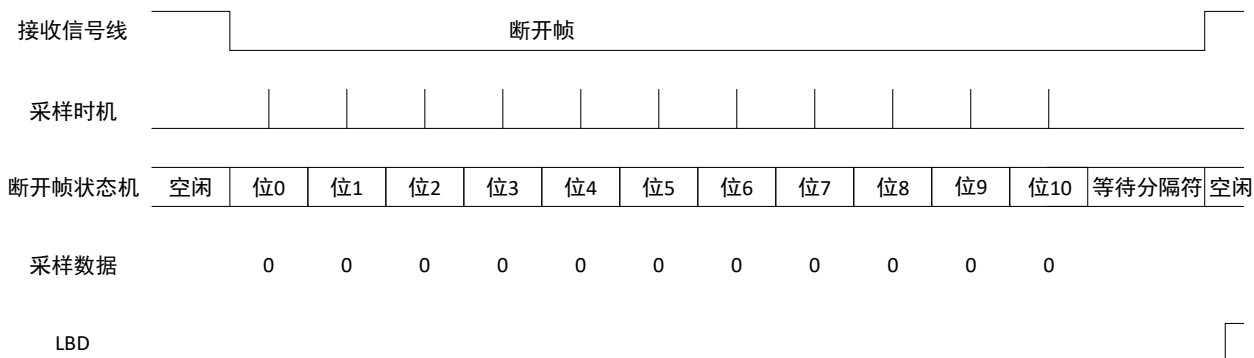
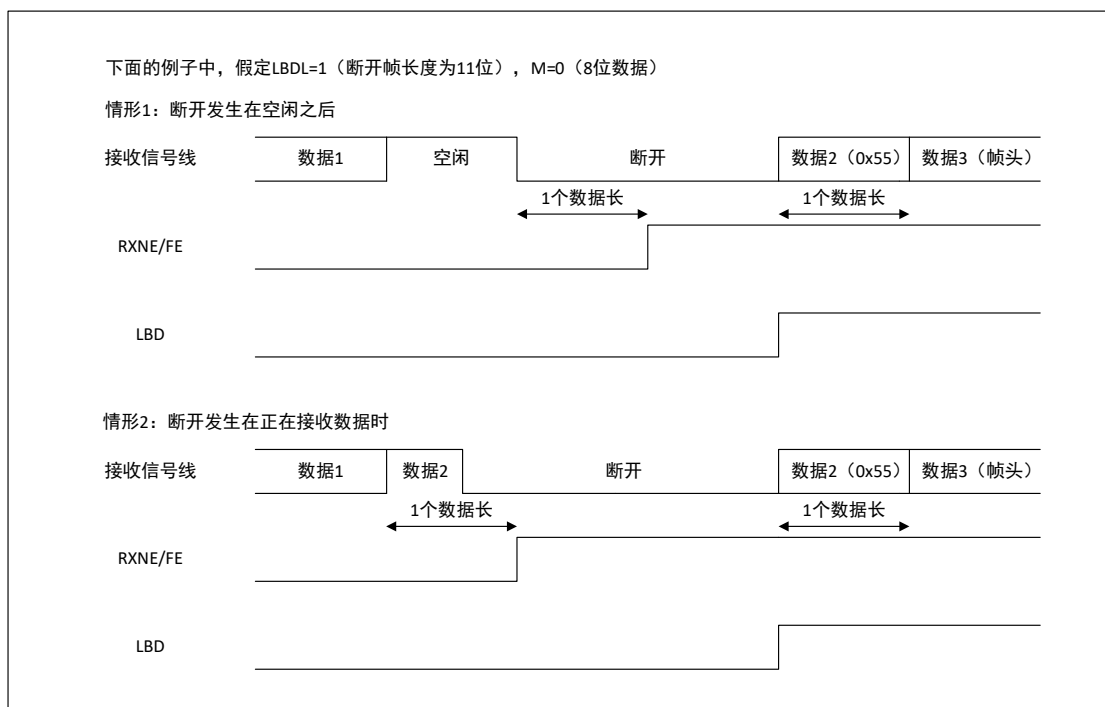


图24.4 LIN 模式下的断开检测与帧错误的检测



24.2.26 USART 同步模式

通过在 USART_CR2 寄存器上写 CLKEN 位选择同步模式

在同步模式里，下列位必须保持清零状态：

- USART_CR2 寄存器中的 LINEN 位
- USART_CR3 寄存器中的 SCEN, HDSEL 和 IREN 位

USART 允许用户以主模式方式控制双向同步串行通信。CK 脚是 USART 发送器时钟的输出。在起始位和停止位期间，CK 脚上没有时钟脉冲。根据 USART_CR2 寄存器中 LBCL 位的状态，决定在最后一个有效数据位期间产生或不产生时钟脉冲。USART_CR2 寄存器的 CPOL 位允许用户选择时钟极性，USART_CR2 寄存器上的 CPHA 位允许用户选择外部时钟的相位

在总线空闲期间，实际数据到来之前以及发送断开符号的时候，外部 CK 时钟不被激活。

同步模式时，USART 发送器和异步模式里工作一模一样。但是因为 CK 是与 TX 同步的(根据 CPOL 和 CPHA)，所以 TX 上的数据是随 CK 同步发出的。

同步模式的 USART 接收器工作方式与异步模式不同。如果 RE=1，数据在 CK 上采样(根据 CPOL 和 CPHA 决定在上升沿还是下降沿)，不需要任何的过采样。但必须考虑建立时间和持续时间(取决于波特率，1/16 位时间)。

注意：

- CK 脚同 TX 脚一起联合工作。因而，只有在使能了发送器 (TE = 1)，并且发送数据时 (写入数据至 USART_DR 寄存器) 才提供时钟。这意味着在没有发送数据时是不可能接收一个同步数据的。
- LBCL, CPOL 和 CPHA 位的正确配置，应该在发送器和接收器都被禁止时；当使能了发送器或接收器时，这些位不能被改变
- 建议在同一条指令中设置 TE 和 RE，以减少接收器的建立时间和保持时间。
- USART 只支持主模式：它不能用来自其他设备的输入时钟接收或发送数据 (CK 永远是输出)。

图24.5 USART 同步传输的例子

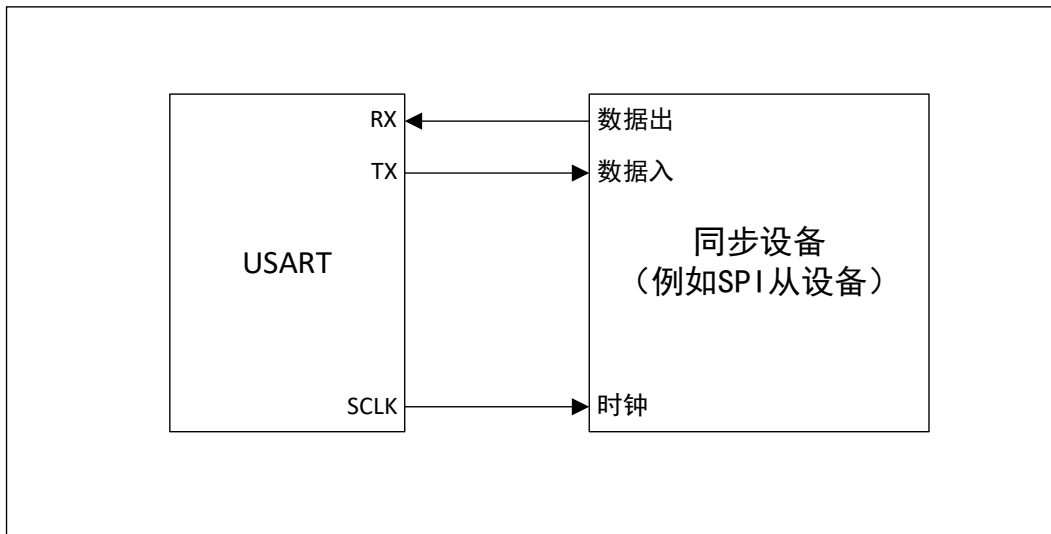
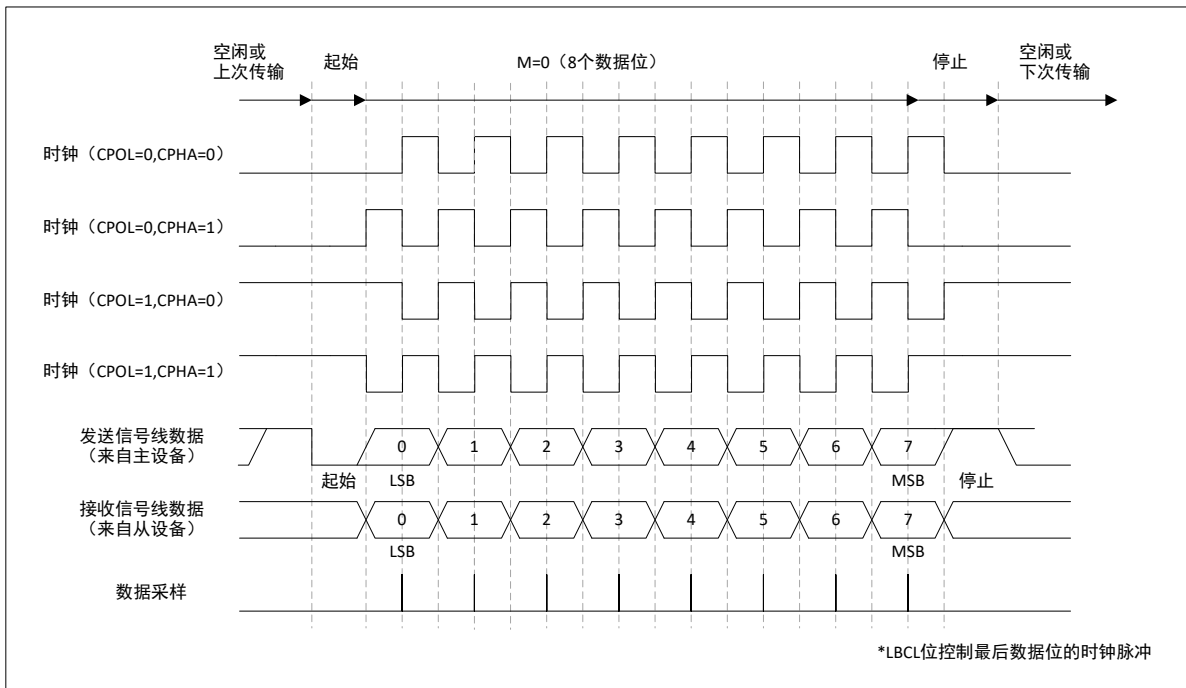


图24.6 USART 数据时钟时序示例(M=0)



The diagram illustrates the timing for an SPI interface with $M=1$ (9 data bits). It shows the relationship between the clock signal, the data being sent, and the data being received.

Timing Diagram Details:

- Clock Signals:** Four clock signals are shown, corresponding to different combinations of CPOL and CPHA:
 - 时钟 (CPOL=0, CPHA=0): Rising edge at the start of the data transfer.
 - 时钟 (CPOL=0, CPHA=1): Falling edge at the start of the data transfer.
 - 时钟 (CPOL=1, CPHA=0): Rising edge at the start of the data transfer.
 - 时钟 (CPOL=1, CPHA=1): Falling edge at the start of the data transfer.
- Data Transfer:** The data is transferred in 9 bits (M=1). The data is shown as a sequence of bits (0, 1, 2, 3, 4, 5, 6, 7, 8) being sent and received. The data is labeled as "发送信号线数据 (来自主设备)" and "接收信号线数据 (来自从设备)".
- Start and Stop:** The data transfer starts at the "起始" (Start) point and ends at the "停止" (Stop) point. The "空闲或上次传输" (Idle or last transfer) and "空闲或下次传输" (Idle or next transfer) periods are also indicated.
- Data Sampling:** The "数据采样" (Data Sampling) signal is shown as a series of pulses, indicating when the data is sampled.

***LBCL位控制最后数据位的时钟脉冲**

SCLK (SCLK上的采样点
本例为上升沿)

接收信号线数据
(来自从设备)

有效数据位

$t_{\text{SETUP}} = t_{\text{HOLD}} = 1/16 \text{ 位时间}$

t_{SETUP} t_{HOLD}

[提交文档反馈](#)
[最新版本文档链接](#)

除此以外, 通信与正常 USART 模式类似。由软件来管理线上的冲突(例如通过使用一个中央仲裁器)。特别的是, 发送从不会被硬件所阻碍。当 TE 位被设置时, 只要数据一写到数据寄存器上, 发送就继续。

24.2.28 智能卡

设置 USART_CR3 寄存器的 SCEN 位选择智能卡模式。在智能卡模式下, 下列位必须保持清零:

- USART_CR2 寄存器的 LINEN 位
- USART_CR3 寄存器的 HDSEL 位和 IREN 位

此外, CLKEN 位可以被设置, 以提供时钟给智能卡。

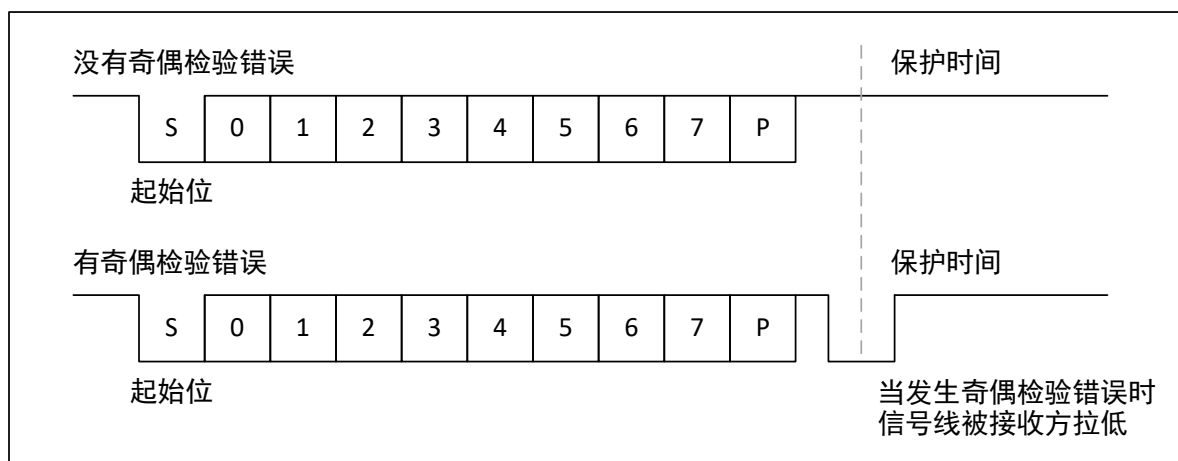
该接口符合 ISO7816-3 标准, 支持智能卡异步协议。USART 应该被设置为:

- 8 位数据位加校验位: 此时 USART_CR1 寄存器中 M=1、PCE=1
- 发送和接收时为 1.5 个停止位: 即 USART_CR2 寄存器的 STOP=11

注: 也可以在接收时选择 0.5 个停止位, 但为了避免在 2 种配置间转换, 建议在发送和接收时使用 1.5 个停止位。

下图给出的例子说明了数据线上, 在有校验错误和没校验错误两种情况下的信号。

图24.9 ISO7816-3 异步协议



当与智能卡相连接时, USART 的 TX 驱动一根智能卡也驱动的双向线。为了做到这点, SW_RX 必须和 TX 连接到相同的 I/O 口。在发送开始位和数据字节期间, 发送器的输出使能位 TX_EN 被置起, 在发送停止位期间被释放(弱上拉), 因此在发现校验错误的情况下接收器可以将数据线拉低。如果 TX_EN 不被使用, 在停止位期间 TX 被拉到高电平: 这样的话, 只要 TX 配置成开漏, 接收器也可以驱动这根线。

智能卡是一个单线半双工通信协议

- 从发送移位寄存器把数据发送出去, 要被延时最小 1/2 波特时钟。在正常操作时, 一个满的发送移位寄存器将在下一个波特时钟沿开始向外移出数据。在智能卡模式里, 此发送被延迟 1/2 波特时钟。
- 如果在接收一个设置为 0.5 或 1.5 个停止位的数据帧期间, 检测到一奇偶校验错误, 在完成接收该帧后(即停止位结束时), 发送线被拉低一个波特时钟周期。这是告诉智能卡发送到
- USART 的数据没有被正确地接收到。此 NACK 信号(拉低发送线一个波特时钟周期)在发送端将产生一个帧错误(发送端被配置成 1.5 个停止位)。应用程序可以根据协议处理重新发送数据。如果设置了 NACK 控制位, 发生校验错误时接收器会给出一个 NACK 信号; 否则就不会发送 NACK。

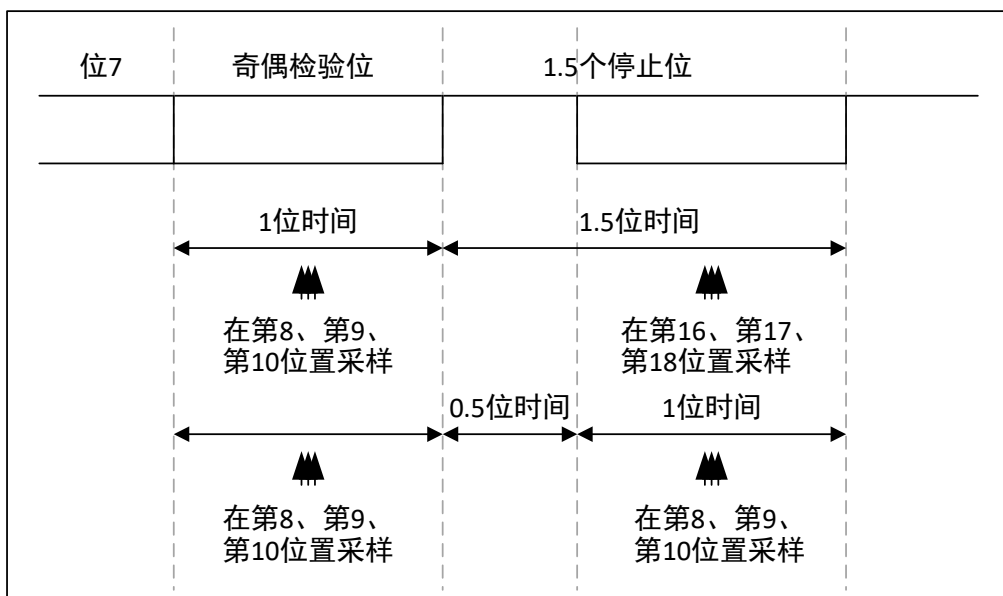
- TC 标志的置起可以通过编程保护时间寄存器得以延时。在正常操作时，当发送移位寄存器变空并且没有新的发送请求出现时，TC 被置起。在智能卡模式里，空的发送移位寄存器将触发保护时间计数器开始向上计数，直到保护时间寄存器中的值。TC 在这段时间被强制拉低。当保护时间计数器达到保护时间寄存器中的值时，TC 被置高。
- TC 标志的撤销不受智能卡模式的影响。
- 如果发送器检测到一个帧错误(收到接收器的 NACK 信号)，发送器的接收功能模块不会把 NACK 当作起始位检测。根据 ISO 协议，接收到的 NACK 的持续时间可以是 1 或 2 波特时钟周期。
- 在接收器这边，如果一个校验错误被检测到，并且 NACK 被发送，接收器不会把 NACK 检测成起始位。

注意：

- 断开符号在智能卡模式里没有意义。一个带帧错误的 00h 数据将被当成数据而不是断开符号。
- 当来回切换 TE 位时，没有 IDLE 帧被发送。ISO 协议没有定义 IDLE 帧。

下图详述了 USART 是如何采样 NACK 信号的。在这个例子里，USART 正在发送数据，并且被配置成 1.5 个停止位。为了检查数据的完整性和 NACK 信号，USART 的接收功能块被激活。

图24.10 使用 1.5 停止位检测奇偶检验错



USART 可以通过 CK 输出为智能卡提供时钟。在智能卡模式里，CK 不和通信直接关联，而是先通过一个 5 位预分频器简单地用内部的外设输入时钟来驱动智能卡的时钟。分频率在预分频寄存器 USART_GTPR 中配置。CK 频率可以从 $f_{CK}/2$ 到 $f_{CK}/62$ ，这里的 f_{CK} 是外设输入时钟。

24.2.29 IrDA SIR ENDEC 功能模块

通过设置 USART_CR3 寄存器的 IREN 位选择 IrDA 模式。在 IRDA 模式里，下列位必须保持清零：

- USART_CR2 寄存器的 LINEN, STOP 和 CLKEN 位
- USART_CR3 寄存器的 SCEN 和 HDSEL 位。

IrDA SIR 物理层规定使用反相归零调制方案(RZI)，该方案用一个红外光脉冲代表逻辑'0'(见下面图

)。SIR 发送编码器对从 USART 输出的 NRZ(非归零)比特流进行调制。输出脉冲流被传送到一个外部输出驱动器和红外 LED。USART 为 SIR ENDEC 最高只支持到 115.2Kbps 速率。在正常模式里, 脉冲宽度规定为一个位周期的 3/16。

SIR 接收解码器对来自红外接收器的归零位比特流进行解调, 并将接收到的 NRZ 串行比特流输出到 USART。在空闲状态里, 解码器输入通常是高(标记状态 marking state)。发送编码器输出的极性和解码器的输入相反。当解码器输入低时, 检测到一个起始位。

- IrDA 是一个半双工通信协议。如果发送器忙(也就是 USART 正在送数据给 IrDA 编码器), IrDA 接收线上的任何数据将被 IrDA 解码器忽视。如果接收器忙(也就是 USART 正在接收从 IrDA 解码器来的解码数据), 从 USART 到 IrDA 的 TX 上的数据将不会被 IrDA 编码。当接收数据时, 应该避免发送, 因为将被发送的数据可能被破坏。
- SIR 发送逻辑把'0'作为高脉冲发送, 把'1'作为低电平发送。脉冲的宽度规定为正常模式时位周期的 3/16。
- SIR 接收逻辑把高电平状态解释为'1', 把低脉冲解释为'0'。
- 发送编码器输出与解码器输入有着相反的极性。当空闲时, SIR 输出处于低状态。
- SIR 解码器把 IrDA 兼容的接收信号转变成给 USART 的比特流。
- IrDA 规范要求脉冲要宽于 1.41us。脉冲宽度是可编程的。接收器端的尖峰脉冲检测逻辑滤除宽度小于 2 个 PSC 周期的脉冲(PSC 是在 IrDA 低功耗波特率寄存器 USART_GTPR 中编程的预分频值)。宽度小于 1 个 PSC 周期的脉冲一定被滤除掉, 但是那些宽度大于 1 个而小于 2 个 PSC 周期的脉冲可能被接收或滤除, 那些宽度大于 2 个周期的将被视为一个有效的脉冲。当 PSC=0 时, IrDA 编码器/解码器不工作。
- 接收器可以与一低功耗发送器通信。
- 在 IrDA 模式里, USART_CR2 寄存器上的 STOP 位必须配置成 1 个停止位。

24.2.30 IrDA 低功耗模式

24.2.30.1 发送器

在低功耗模式, 脉冲宽度不再持续 3/16 个位周期。取而代之, 脉冲的宽度是低功耗波特率的 3 倍, 它最小可以是 1.42MHz。通常这个值是 1.8432MHz($1.42 \text{ MHz} < \text{PSC} < 2.12 \text{ MHz}$)。一个低功耗模式可编程分频器把系统时钟进行分频以达到这个值。

24.2.30.2 接收器

低功耗模式的接收类似于正常模式的接收。为了滤除尖峰干扰脉冲, USART 应该滤除宽度短于 1 个 PSC 的脉冲。只有持续时间大于 2 个周期的 IrDA 低功耗波特率时钟(USART_GTPR 中的 PSC)的低电平信号才被接受为有效的信号。

注意:

- 宽度小于 2 个大于 1 个 PSC 周期的脉冲可能会也可能不会被滤除。
- 接收器的建立时间应该由软件管理。IrDA 物理层技术规范规定了在发送和接收之间最小要有 10ms 的延时 (IrDA 是一个半双工协议)。

图24.11 IrDA SIR ENDEC – 框图

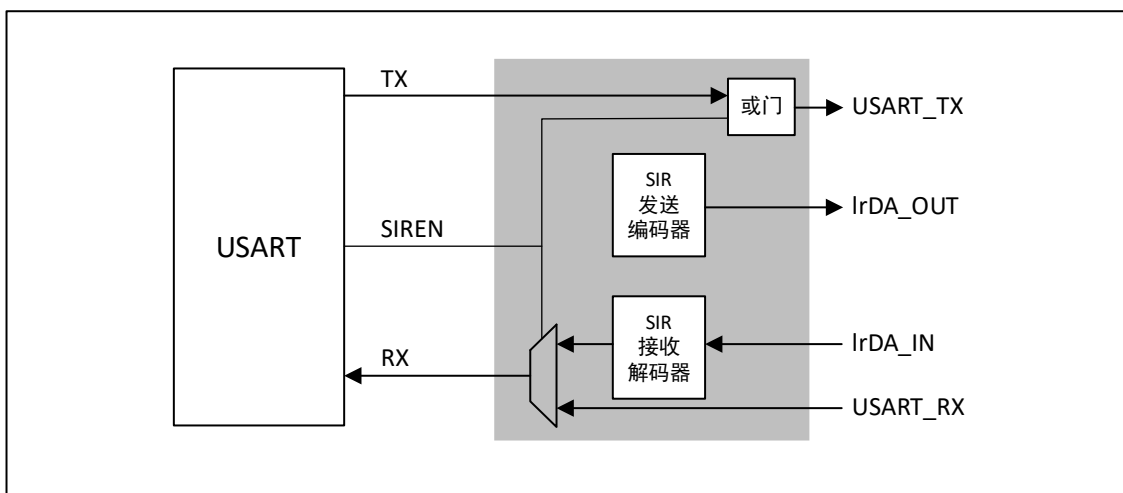
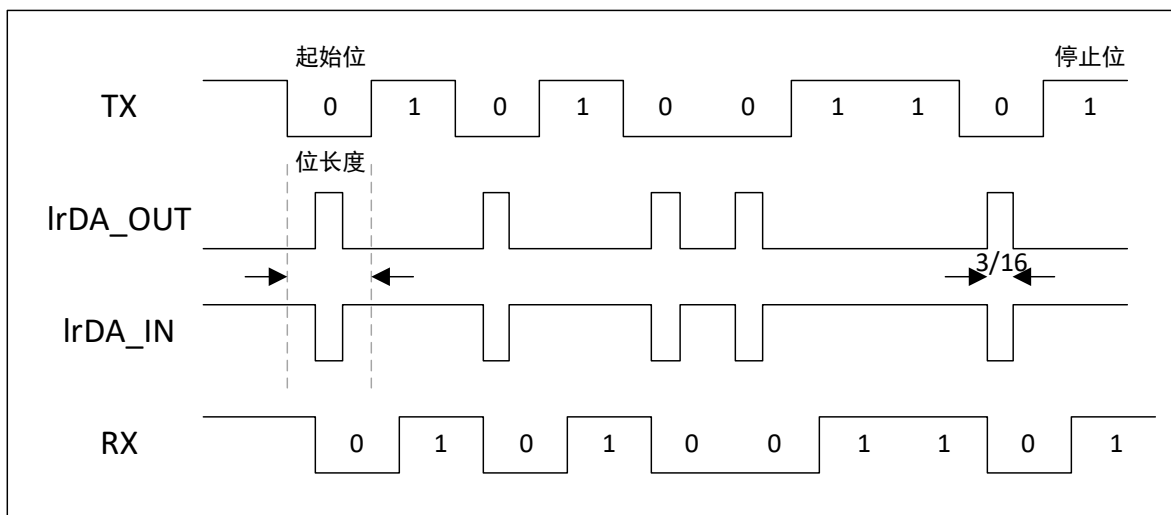


图24.12 IrDA 数据调制(3/16) – 普通模式



24.2.31 利用 DMA 连续通信

USART 可以利用 DMA 连续通信。Rx 缓冲器和 Tx 缓冲器的 DMA 请求是分别产生的。

注意：参考产品技术说明以确定有可用 DMA 控制器。在 USART2_SR 寄存器里，可以清零 TXE/RXNE 标志来实现连续通信。

24.2.32 利用 DMA 发送

使用 DMA 进行发送，可以通过设置 USART_CR3 寄存器上的 DMAT 位激活。当 TXE 位被置为‘1’时，DMA 就从指定的 SRAM 区传送数据到 USART_DR 寄存器。为 USART 的发送分配一个 DMA 通道的步骤如下(x 表示通道号)：

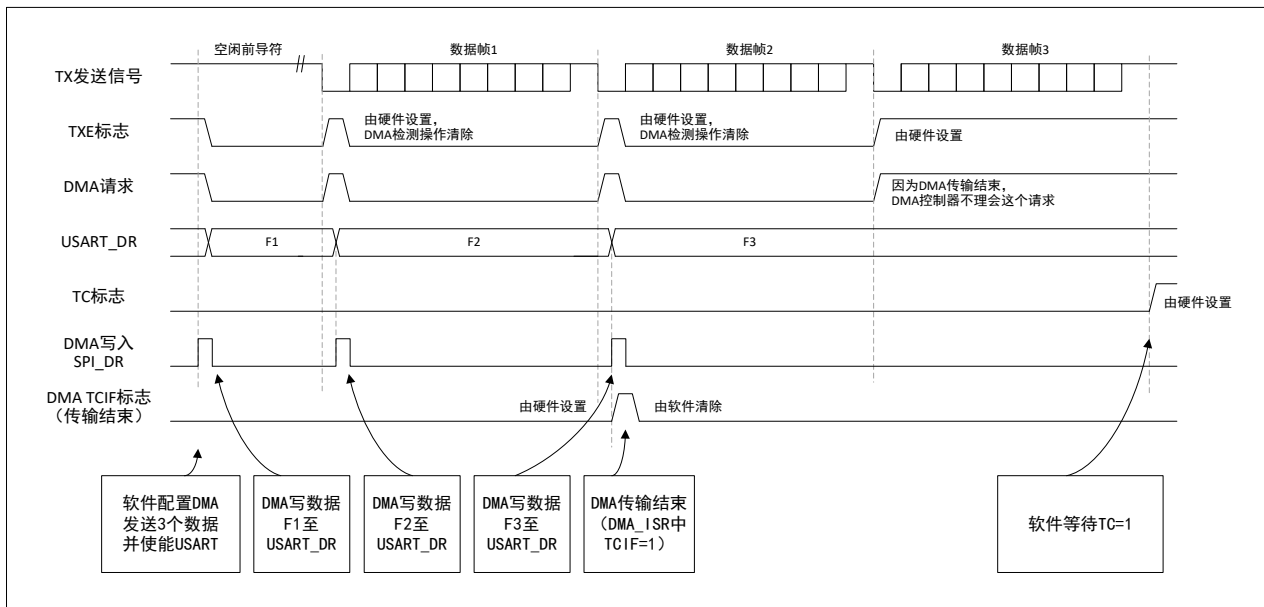
- 在 DMA 控制寄存器上将 USART_DR 寄存器地址配置成 DMA 传输的目的地址。在每个 TXE 事件后，数据将被传送到这个地址。
- 在 DMA 控制寄存器上将存储器地址配置成 DMA 传输的源地址。在每个 TXE 事件后，将从此存储器区读出数据并传送到 USART_DR 寄存器。
- 在 DMA 控制寄存器中配置要传输的总的字节数。
- 在 DMA 寄存器上配置通道优先级。
- 根据应用程序的要求，配置在传输完成一半还是全部完成时产生 DMA 中断。

- 在 DMA 寄存器上激活该通道。

当传输完成 DMA 控制器指定的数据量时，DMA 控制器在该 DMA 通道的中断向量上产生一中断。

在发送模式下，当 DMA 传输完所有要发送的数据时，DMA 控制器设置 DMA_ISR 寄存器的 TCIF 标志；监视 USART_SR 寄存器的 TC 标志可以确认 USART 通信是否结束，这样可以在关闭 USART 或进入停机模式之前避免破坏最后一次传输的数据；软件需要先等待 TXE=1，再等待 TC=1。

图24.13 利用 DMA 发送



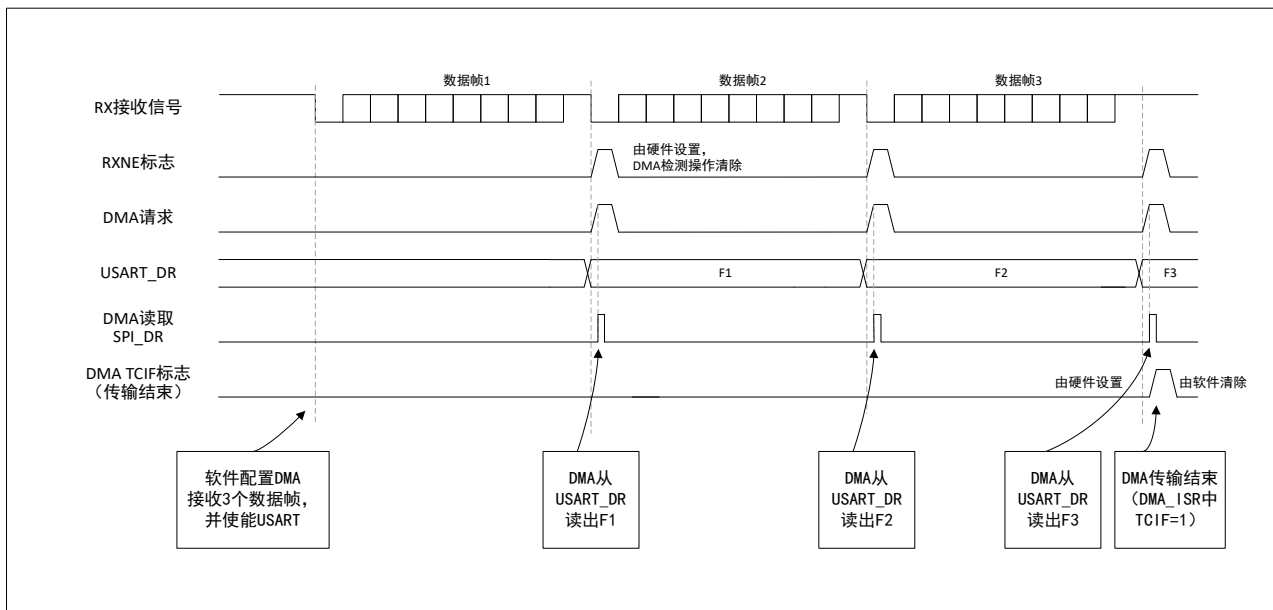
24.2.33 利用 DMA 接收

可以通过设置 USART_CR3 寄存器的 DMAR 位激活使用 DMA 进行接收，每次接收到一个字节，DMA 控制器就把数据从 USART_DR 寄存器传送到指定的 SRAM 区(参考 DMA 相关说明)。为 USART 的接收分配一个 DMA 通道的步骤如下(x 表示通道号)：

- 通过 DMA 控制寄存器把 USART_DR 寄存器地址配置成传输的源地址。在每个 RXNE 事件后，将从此地址读出数据并传输到存储器。
- 通过 DMA 控制寄存器把存储器地址配置成传输的目的地址。在每个 RXNE 事件后，数据将从 USART_DR 传输到此存储器区。
- 在 DMA 控制寄存器中配置要传输的总的字节数。
- 在 DMA 寄存器上配置通道优先级。。
- 根据应用程序的要求配置在传输完成一半还是全部完成时产生 DMA 中断。
- 在 DMA 控制寄存器上激活该通道。

当接收完成 DMA 控制器指定的传输量时，DMA 控制器在该 DMA 通道的中断矢量上产生一中断。

图24.14 利用 DMA 接收



24.2.34 多缓冲器通信中的错误标志和中断产生

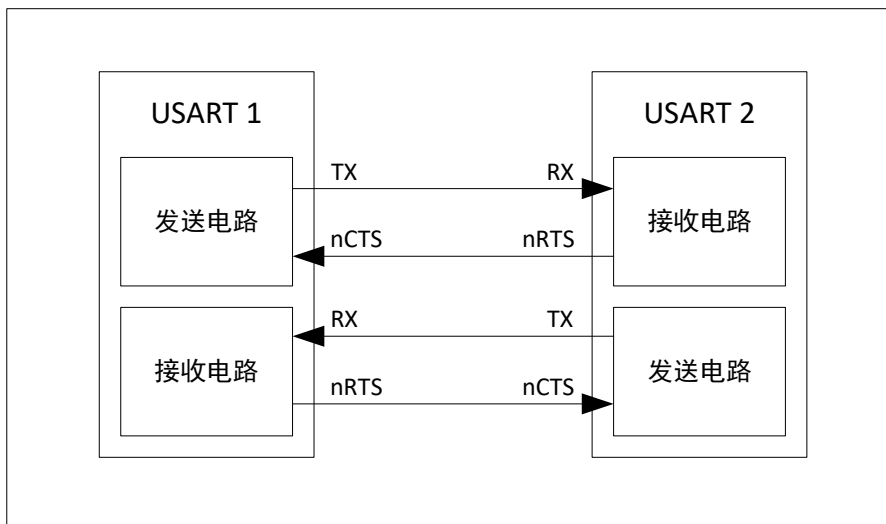
在多缓冲器通信的情况下，通信期间如果发生任何错误，在当前字节传输后将置起错误标志。

如果中断使能位被设置，将产生中断。在单个字节接收的情况下，和 **RXNE** 一起被置起的帧错误、溢出错误和噪音标志，有单独的错误标志中断使能位；如果设置了，会在当前字节传输结束后，产生中断。

24.2.35 硬件流控制

利用 **nCTS** 输入和 **nRTS** 输出可以控制 2 个设备间的串行数据流。下图表明在这个模式里如何连接 2 个设备。

图24.15 两个 USART 间的硬件流控制

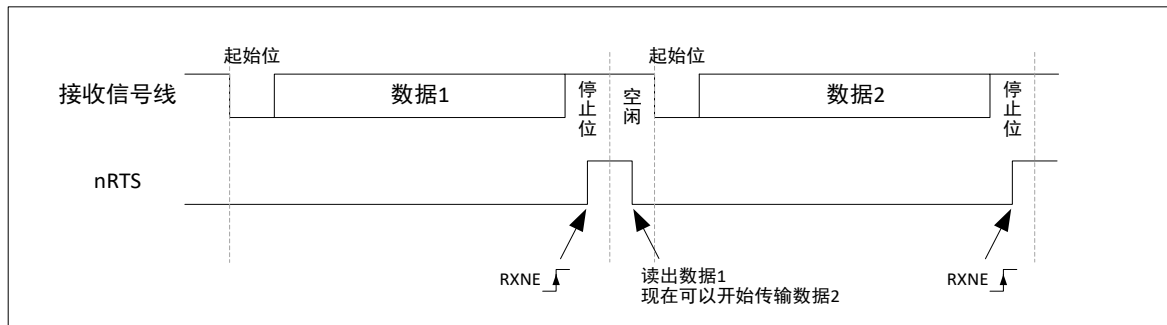


通过将 **USART_CR3** 中的 **RTSE** 和 **CTSE** 置位，可以分别独立地使能 **RTS** 和 **CTS** 流控制。

24.2.36 RTS 流控制

如果 RTS 流控制被使能($RTSE=1$)，只要 USART 接收器准备好接收新的数据， $nRTS$ 就变成有效(接低电平)。当接收寄存器内有数据到达时， $nRTS$ 被释放，由此表明希望在当前帧结束时停止数据传输。下图是一个启用 RTS 流控制的通信的例子。

图24.16 RTS 流控制

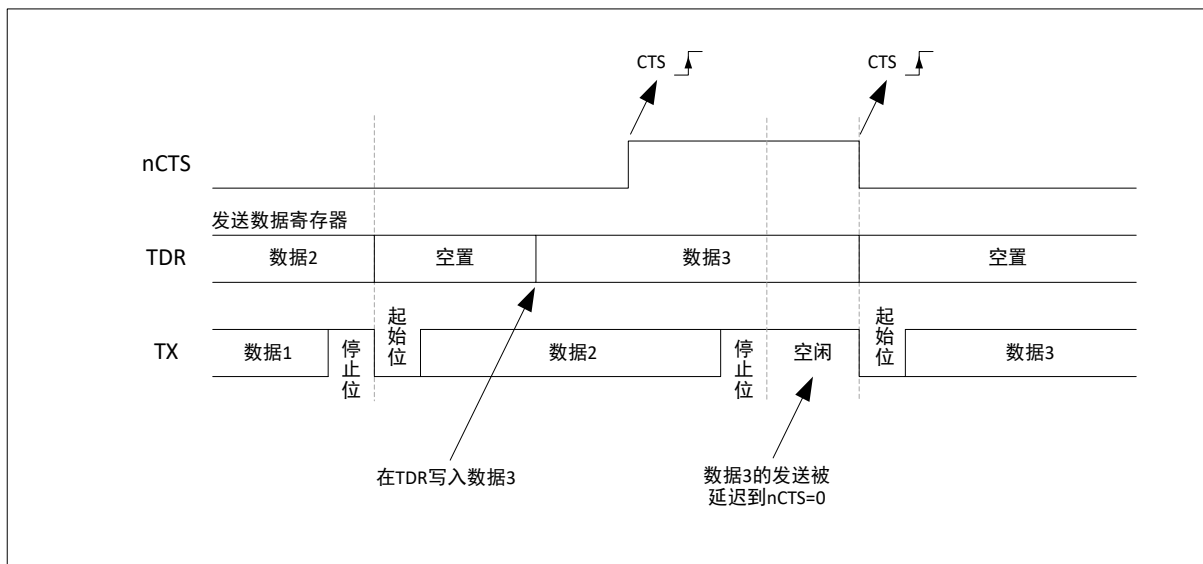


24.2.37 CTS 流控制

如果 CTS 流控制被使能($CTSE=1$)，发送器在发送下一帧前检查 $nCTS$ 输入。如果 $nCTS$ 有效(被拉成低电平)，则下一个数据被发送(假设那个数据是准备发送的，也就是 $TXE=0$)，否则下一帧数据不被发出去。若 $nCTS$ 在传输期间被变成无效，当前的传输完成后停止发送。

当 $CTSE=1$ 时，只要 $nCTS$ 输入一变换状态，硬件就自动设置 $CTSIF$ 状态位。它表明接收器是否准备好进行通信。如果设置了 USART_CT3 寄存器的 $CTSIE$ 位，则产生中断。下图是一个启用 CTS 流控制通信的例子。

图24.17 CTS 流控制



24.3 USART 中断请求

表24.1 USART 中断请求

中断事件	事件标志	使能位
发送数据寄存器空	TXE	TXEIE
CTS 标志	CTS	CTSIE
发送完成	TC	TCIE
接收数据就绪可读	TXNE	TXNEIE
检测到数据溢出	ORE	TXNEIE
检测到空闲线路	IDLE	IDLEIE
奇偶检验错	PE	PEIE
断开标志	LBD	LBDIE
噪声标志，多缓冲通信中的溢出错误和帧错误	NE 或 ORT 或 FE	EIE (1)

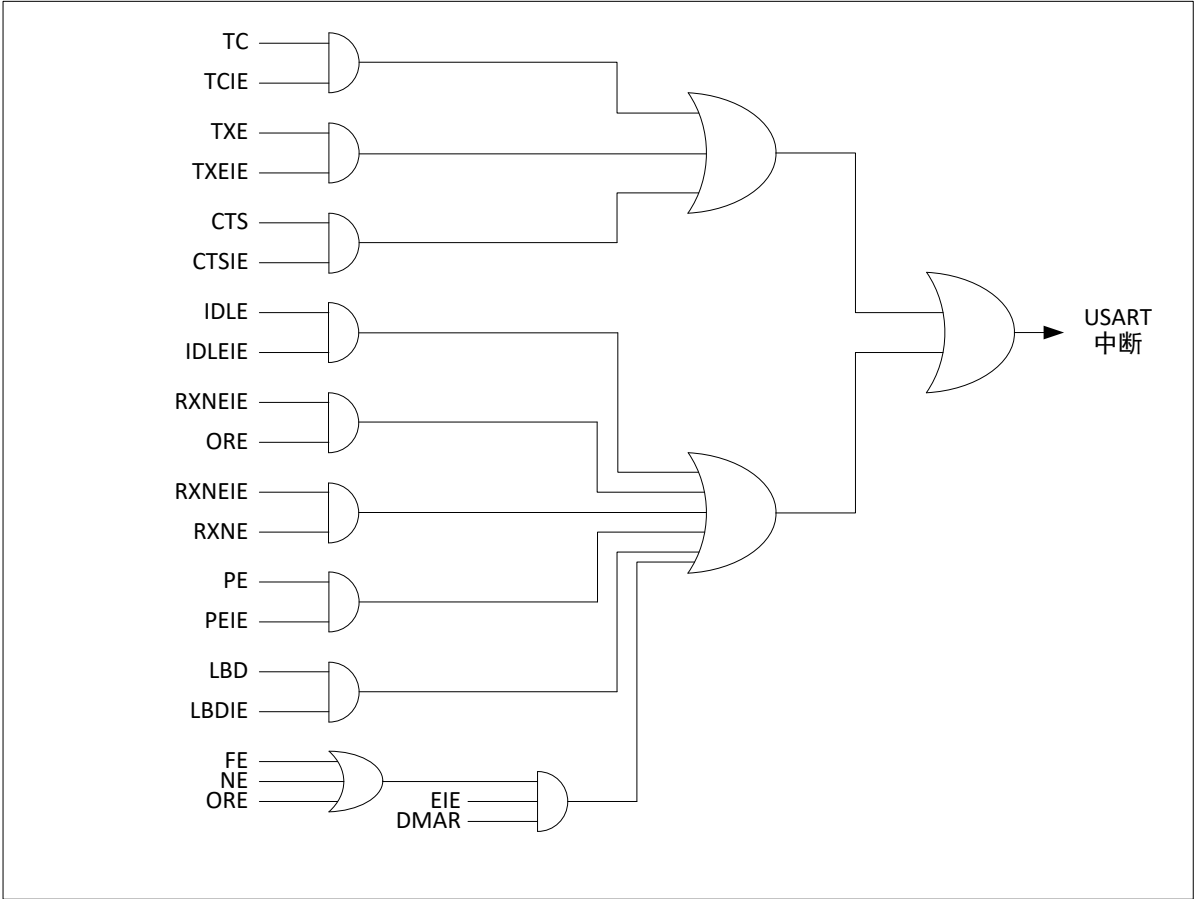
注：(1) 仅当使用 DMA 接收数据时，才使用这个标志位。

USART 的各种中断事件被连接到同一个中断向量(见下图)，有以下各种中断事件：

- 发送期间：发送完成、清除发送、发送数据寄存器空。
- 接收期间：空闲总线检测、溢出错误、接收数据寄存器非空、校验错误、LIN 断开符号检测、噪声标志(仅在多缓冲器通信)和帧错误(仅在多缓冲器通信)。

如果设置了对应的使能控制位，这些事件就可以产生各自的中断。

图24.18 USART 中断映像图



24.4 USART 模式配置

表24.2 USART 模式配置

USART 模式	USART1	USART2	USART3	UART4	UART5
异步模式	X	X	X	X	X
硬件流控制	X	X	X	NA	NA
多缓存通讯(DMA)	X	X	X	X	X
多处理器通讯	X	X	X	X	X
同步	X	X	X	NA	NA
智能卡	X	X	X	NA	NA
半双工(单线模式)	X	X	X	X	X
IrDA	X	X	X	X	X
LIN	X	X	X	X	X

注：X = 支持，NA = 不支持该应用

24.5 串口接收发送配置示例

以下所有配置示例在 DEMO 板的配套程序中有源代码

24.5.1 串口 1 配置示例

具体可以参看 DEMO 板的示例程序

//初始化 IO 串口 1 。PA9 PA10 分别为 Tx Rx

//pclk2:PCLK2 时钟频率(Mhz)

```
//bound:波特率

//此函数在 DEMO 板的配套程序有源代码

void uart_init(u32 pclk2,u32 bound)

    float temp;

    u16 mantissa;

    u16 fraction;

//第一步：根据波特率公式计算出要写入 BRR 寄存器的值

    temp=(float)(pclk2*1000000)/(bound*16);//得到 USARTDIV

    mantissa=temp;得到整数部分

    fraction=(temp-mantissa)*16; //得到小数部分

    mantissa<=<4;

    mantissa+=fraction;

//第二步：使能 TX,RX 脚和 USART 的时钟

    RCMU->APB2ENR|=1<<2;    //使能 PORTA 口时钟

    RCMU->APB2ENR|=1<<14;    //使能串口 1 时钟

//第三步：TX RX 脚模式配置

//TX->PA9      RX->PA10

    GPIOA->CRH&=0XFFFFFF0F;//IO 状态设置

    GPIOA->CRH|=0X000008B0;//IO 状态设置

    RCMU->APB2RSTR|=1<<14;    //复位串口 1

    RCMU->APB2RSTR&=~(1<<14);//停止复位

//第四步：波特率写入到 BRR 寄存器

    //波特率设置

    USART1->BRR=mantissa; // 波特率设置

//第五步：配置 CR1 寄存器,具体参看 CR1 寄存器

    USART1->CR1|=0X200C; //1 位停止,无校验位.

//第六步：开启接收中断

//使能接收中断

    USART1->CR1|=1<<5;    //接收缓冲区非空中断使能

    MY_NVIC_Init(3,3,USART1_IRQn,2);//串口中断的优先级分组。组 2，最低优先级

以下其他串口配置步骤和 USART 相同
```

24.5.2 串口 2 配置示例

```
//初始化 IO 串口 2

//pclk2:PCLK2 时钟频率(Mhz)
```

```
//bound:波特率

void uart_init(u32 pclk2,u32 bound)

    float temp;

    u16 mantissa;

    u16 fraction;

//根据公式算出 BRR 寄存器值

    temp=(float)(pclk2*1000000)/(bound*16);//得到 USARTDIV

    mantissa=temp; //得到整数部分

    fraction=(temp-mantissa)*16; //得到小数部分

    mantissa<=<4;

    mantissa+=fraction;

    RCMU->APB2ENR |= (1<<2); //使能 PORTA 口时钟

    RCMU->APB1ENR |= (1<<17); //使能串口时钟

//TX->PA2    RX->PA3

    GPIOA->CRL&=0xFFFF00FF;//IO 状态设置

    GPIOA->CRL|=0X00008B00;//IO 状态设置

    RCMU->APB1RSTR|=(1<<17); //复位串口 1

    RCMU->APB1RSTR&=~(1<<17);//停止复位

//波特率设置

    USART2->BRR=mantissa; // 波特率设置

    USART2->CR1|=0X200C; //1 位停止,无校验位.

//使能接收中断

    USART2->CR1|=1<<5; //接收缓冲区非空中断使能

    MY_NVIC_Init(3,3,USART2_IRQn,2);//组 2, 最低优先级
```

24.5.3 串口 3 配置示例

```
//初始化 IO 串口 3

//pclk2:PCLK2 时钟频率(Mhz)

//bound:波特率

void uart_init(u32 pclk2,u32 bound)

{

    float temp;

    u16 mantissa;

    u16 fraction;

    temp=(float)(pclk2*1000000)/(bound*16);//得到 USARTDIV
```

```
mantissa=temp; //得到整数部分
fraction=(temp-mantissa)*16; //得到小数部分
mantissa<<=4;
mantissa+=fraction;
RCMU->APB2ENR |= (1<<3); //使能 PORTB 口时钟
RCMU->APB1ENR |= (1<<18); //使能串口 3 时钟
//TX->PB10 RX->PB11
GPIOB->CRH&=0xFFFF00FF;//IO 状态设置
GPIOB->CRH|=0X00008B00;//IO 状态设置
RCMU->APB1RSTR|=(1<<18); //复位串口 3
RCMU->APB1RSTR&=~(1<<18);//停止复位
//波特率设置
USART3->BRR=mantissa; // 波特率设置
USART3->CR1|=0X200C; //1 位停止,无校验位.
//使能接收中断
USART3->CR1|=1<<5; //接收缓冲区非空中断使能
MY_NVIC_Init(3,3,USART3_IRQn,2);//组 2, 最低优先级
```

24.5.4 串口 4 配置示例

```
//初始化 IO 串口 4
//pclk2:PCLK2 时钟频率(Mhz)
//bound:波特率
void uart_init(u32 pclk2,u32 bound)
{
    float temp;
    u16 mantissa;
    u16 fraction;
    temp=(float)(pclk2*1000000)/(bound*16);//得到 USARTDIV
    mantissa=temp; //得到整数部分
    fraction=(temp-mantissa)*16; //得到小数部分
    mantissa<=<4;
    mantissa+=fraction;
    RCMU->APB2ENR |= (1<<4); //使能 PORTC 口时钟
    RCMU->APB1ENR |= (1<<19); //使能串口 4 时钟
    //TX->PC10 RX->PC11
    GPIOC->CRH&=0xFFFF00FF;//IO 状态设置
    GPIOC->CRH|=0X00008B00;//IO 状态设置
    RCMU->APB1RSTR|=(1<<19); //复位串口 3
    RCMU->APB1RSTR&=~(1<<19);//停止复位
    //波特率设置
    UART4->BRR=mantissa; // 波特率设置
    UART4->CR1|=0X200C; //1 位停止,无校验位.
    //使能接收中断
    UART4->CR1|=1<<5; //接收缓冲区非空中断使能
    MY_NVIC_Init(3,3,UART4_IRQn,2);//组 2, 最低优先级
}
```

24.5.5 串口 5 配置示例

```
//初始化 IO 串口 5
//初始化 IO 串口 5
//pclk2:PCLK2 时钟频率(Mhz)
//bound:波特率
void uart_init(u32 pclk2,u32 bound)
{
    float temp;
    u16 mantissa;
```

```

u16 fraction;

temp=(float)(pclk2*1000000)/(bound*16); //得到 USARTDIV

mantissa=temp; //得到整数部分

fraction=(temp-mantissa)*16; //得到小数部分

mantissa<=<=4;

mantissa+=fraction;

RCMU->APB2ENR |= (1<<4); //使能 PORTC 口时钟

RCMU->APB2ENR |= (1<<5); //使能 PORTD 口时钟

RCMU->APB1ENR |= (1<<20); //使能串口 4 时钟

//TX->PC12    RX->PD2

GPIOC->CRH&=0xFFFF0FFFF; //IO 状态设置

GPIOC->CRH|=0X000B0000; //IO 状态设置

GPIOD->CRL&=0xFFFFF0FF; //IO 状态设置

GPIOD->CRL|=0X00000800; //IO 状态设置

RCMU->APB1RSTR|=(1<<20); //复位串口 3

RCMU->APB1RSTR&=~(1<<20); //停止复位

//波特率设置

UART5->BRR=mantissa; // 波特率设置

UART5->CR1|=0X200C; //1 位停止,无校验位.

//使能接收中断

UART5->CR1|=1<<5; //接收缓冲区非空中断使能

MY_NVIC_Init(3,3,UART5_IRQn,2); //组 2, 最低优先级
    
```

24.6 USART 寄存器

24.6.1 寄存器 (USART_SR)

偏移地址: 0x00, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r						W0c	W0c	ro	W0c	W0c	ro	ro	ro	ro	ro

Bit	Field	Description	Reset value	RW attribute	Remark
31:10	Reserved				

Bit	Field	Description	Reset value	RW attribute	Remark
9	CTS	CTS 标志 如果 CTSE 位被设置, 当 n CTS 输入切换时, 该位由硬件置位。它由软件清零 (通过将其写入 0)。如果 USART_CR3 寄存器中的 CTSIE = 1, 则产生中断。 0: n CTS 状态行没有发生变化 1: n CTS 状态行发生变化	0x0	W0c	
8	LBD	LIN 断开检测标志 当检测到 LIN 断开时, 该位由硬件置位。它由软件清零 (通过将其写入 0)。如果 USART_CR2 寄存器中的 LBDIE = 1, 则产生中断。 0: 未检测到 LIN 断开 1: 检测到 LIN 断开 注意: 若 LBDIE=1, 当 LBD 为'1'时要产生中断。	0x0	W0c	
7	TXE	发送数据寄存器为空 当 TDR 寄存器的内容被传送到移位寄存器时, 该位由硬件置位。如果 USART_CR1 寄存器中的 TXEIE 位 = 1, 则产生中断。它通过写入 USART_DR 寄存器清除。 0: 数据还没有传送到移位寄存器 1: 数据已经传送到移位寄存器	0x1	ro	
6	TC	发送完成 如果包含数据的帧传输完成并且 TXE 被设置, 则该位由硬件置位。如果 USART_CR1 寄存器中的 TCIE = 1, 则产生中断。它由软件序列清除 (先读 USART_SR, 然后写入 USART_DR)。TC 位也可以通过向其写入“0”来清除。此清除序列仅推荐用于多缓冲区通信。 0: 发送未完成 1: 发送完成	0x1	W0c	
5	RXNE	读数据寄存器非空 (Read data register not empty) 当 RDR 移位寄存器的内容传输到 USART_DR 寄存器时, 该位由硬件置位, 如果 USART_CR1 寄存器中的 RXNEIE = 1, 则产生中断。 它通过读取 USART_DR 寄存器来清除。RXNE 标志也可以通过向其写入零来清除。这种清除序列仅推荐用于多缓冲区通信。 0: 未收到数据 1: 接收到的数据, 可以读取。	0x0	W0c	
4	IDLE	IDLE: 检测到总线空闲 (IDLE line detected) 当检测到空闲线时, 该位由硬件置位。如果 USART_CR1 寄存器中的 IDLEIE = 1, 则产生中断。 它由软件序列清除 (先读取 USART_SR 寄存器, 然后读取 USART_DR 寄存器) .. 0: 未检测到空闲总线 1: 检测到空闲总线	0x0	r	
3	ORE	ORE: 过载错误 (Overrun error) 当 RXNE = 1 时, 当移位寄存器中当前正在接收的字准备好传输到 RDR 寄存器时, 该位由硬件置位。如果 USART_CR1 寄存器中的 RXNEIE = 1, 则会产生中断。它由软件序列清除 (先读取 USART_SR 寄存器, 然后读取 USART_DR 寄存器)。 0: 无过载错误 1: 检测到过载错误	0x0	r	

Bit	Field	Description	Reset value	RW attribute	Remark
2	NE	NE: 噪声错误标志 (Noise error flag) 当在接收帧上检测到噪声时, 该位由硬件置位, 由软件序列清零 (先读 USART_SR 寄存器, 然后读 USART_DR 寄存器)。 0: 未检测到噪音 1: 检测到噪音	0x0	r	
1	FE	FE: 帧错误 (Framing error) 当检测到同步错位、过多的噪声或检测到断开字符时, 该位由硬件置位, 由软件序列清零 (先读 USART_SR 寄存器, 然后读 USART_DR 寄存器)。 0: 未检测到成帧错误 1: 检测到帧错误或中断字符	0x0	r	
0	PE	PE: 校验错误 (Parity error) 该位在接收器模式下发生奇偶校验错误时由硬件设置。它由软件序列清除 (先读取状态寄存器 USART_SR, 然后读取 USART_DR 数据寄存器)。软件必须等待 RXNE 标志位在清除 PE 位之前设置。如果 USART_CR1 寄存器中的 PEIE = 1, 则会产生中断。 0: 无奇偶校验错误 1: 奇偶校验错误	0x0	ro	

24.6.2 寄存器 (USART_DR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DR							
r								rw							

Bit	Field	Description	Reset value	RW attribute	Remark
31:9	Reserved				
8:0	DR	数据值 包含接收或发送的数据字符, 取决于它是读取还是写入。数据寄存器执行双重功能 (读取和写入), 因为它由两个寄存器组成, 一个用于发送 (TDR), 一个用于接收 (RDR)。TDR 寄存器提供内部总线和输出移位寄存器之间的并行接口 (见图 1)。RDR 寄存器提供输入移位寄存器和内部总线之间的并行接口。(PCE 位在 USART_CR1 寄存器中设置为 1), 写入 MSB 的值 (第 7 位或第 8 位取决于数据长度) 没有影响, 因为它被奇偶校验代替。当在奇偶校验使能的情况下接收时, 读取的值 MSB 位中是接收到的奇偶校验位。	0x0	RW	

24.6.3 寄存器 (USART_BRR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_MANTISSA												DIV_FRACITON			
rw												rw			

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved				
15:4	DIV_MANTISSA	USARTDIV 的整数部分 这 12 位定义了 USART 分频器除法因子 (USARTDIV) 的整数部分。	0x0	RW	
3:0	DIV_FRACITON	USARTDIV 的小数部分 这 4 位定义了 USART 分频器除法因子 (USARTDIV) 的小数部分。	0x0	RW	

注: 整数和小数部分具体写入数值:

- 首先根据波特率计算公式算出 USARTDIV 的值:
- 例如: 波特率 115200, USART 时钟为 96MHZ
- 则有 $115200 = 96000000 / (16 * \text{USARTDIV})$ 推算出 $\text{USARTDIV} = 52.08333$

- 则 USART_BRR 的整数部分写 0x34(52), 小数部分写 $0.08333 \times 16 = 1.33328$
- 即 小数部分约取 1 即可
- 最后得出 USART_BR 寄存器写入的数值为 0x341

24.6.4 寄存器 (USART_CR1)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
r		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
31:14	Reserved				
13	UE	USART 使能 当该位清零时, USART 预分频器和输出停止并结束当前字节传输以降低功耗。该位由软件置位和清零。 0: USART 预分频器和输出禁用 1: USART 使能	0x0	rw	
12	WD_LEN	字长 该位决定字长, 由软件置位或清零。 0: 1 个起始位, 8 个数据位, n 个停止位 1: 1 个起始位, 9 个数据位, n 个停止位 注意: 在数据传输过程中(发送或者接收时), 不能修改这个位。	0x0	rw	
11	WAKE	唤醒方式 该位决定 USART 唤醒方式, 由软件置位或清零。 0: 被空闲总线唤醒; 1: 被地址标记唤醒。	0x0	rw	
10	PCE	奇偶校验使能 该位选择硬件奇偶校验控制 (对于发送来说就是校验位的产生; 对于接收来说就是校验位的检测)。当奇偶校验控制使能时, 计算出的奇偶校验被插入到 MSB 位置 (如果 M = 1, 则为第 9 位; 如果 M = 0, 则为第 8 位) 并检查奇偶校验。接收到的数据。该位由软件置位和清零。一旦设置了该位, 当前字节传输完成后, 校验控制才生效。 0: 奇偶校验控制禁用 1: 奇偶校验控制使能	0x0	rw	
9	PS	奇偶选择 该位在奇偶校验生成/检测使能 (PCE 位设置) 时选择奇偶校验, 由软件置位和清零, 奇偶校验将在当前字节之后选择。 0: 偶校验 1: 奇校验	0x0	rw	
8	PEIE	PE 中断使能 该位由软件置位和清零。 0: 禁止中断 1: 只要 USART_SR 寄存器中的 PE = 1 时, 就会产生 USART 中断	0x0	rw	

Bit	Field	Description	Reset value	RW attribute	Remark
7	TXEIE	TXE 发送缓冲区为空中断使能 该位由软件置位和清零。 0: 禁止中断 1: 只要 USART_SR 寄存器中的 TXE = 1 时, 就会产生 USART 中断	0x1	rw	
6	TCIE	发送完成中断使能 该位由软件置位和清零。 0: 禁止中断 1: 只要 USART_SR 寄存器中的 TC = 1 时, 就会产生 USART 中断	0x1	rw	
5	RXNEIE	RXNE 接收缓冲区非空中断使能 该位由软件置位和清零。 0: 禁止中断 1: 只要 USART_SR 寄存器中的 ORE = 1 或 RXNE = 1, 就会产生 USART 中断	0x0	rw	
4	IDLEIE	空闲中断使能 该位由软件置位和清零。 0: 禁止中断 1: 只要 USART_SR 寄存器中的 IDLE = 1 时, 就会产生 USART 中断	0x0	rw	
3	TE	发射器使能 该位使能发送器, 由软件置位和清零。 0: 发送器禁用 1: 发送使能	0x0	rw	
2	RE	接收器使能 该位使能接收器, 由软件置位和清零。 0: 接收器被禁用 1: 接收器使能并开始搜索起始位	0x0	rw	
1	RWU	接收器唤醒 该位决定 USART 是否处于静音模式, 由软件置位和清零, 当识别到唤醒序列时可由硬件清零。 0: 接收器处于正常工作模式 1: 接收器处于静音模式	0x0	rw	
0	SBK	发送断开帧 该位设置用于发送断开字符, 可由软件设置和清除, 应由软件设置, 在中断停止位期间由硬件复位。 0: 不传输断开字符 1: 发送断开字符	0x0	rw	

24.6.5 寄存器 (USART_CR2)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	LINEN	STOP	CLKEN	CPOL	CPHA	LBCL	Reserved	LBDIE	LBDL	Reserved	ADD				
r	RW	rw	rw	rw	rw	rw	rw	r	rw	rw	r	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
31:15	Reserved	Reserved	0x0	r	

Bit	Field	Description	Reset value	RW attribute	Remark
14	LINEN	LIN 模式使能 该位由软件置位和清零。 0: 禁用 LIN 模式 1: LIN 模式使能 LIN 模式能够使用 USART_CR1 寄存器中的 SBK 位发送 LIN 同步中断 (13 个低位), 并检测 LIN 同步断开符	0x0	rw	
13:12	STOP	停止位 这些位用于编程停止位。 00: 1 个停止位 01: 0.5 个停止位 10: 2 个停止位 11: 1.5 个停止位 0.5 个停止位和 1.5 个停止位不适用于 UART4 和 UART5。	0x0	rw	
11	CLKEN	时钟使能 该位允许用户使能 CK 引脚。 0: CK 引脚禁用 1: CK 引脚使能 该位不适用于 UART4 和 UART5。	0x0	rw	
10	CPOL	时钟极性 该位允许用户在同步模式下选择 CK 引脚上时钟输出的极性, 它与 CPHA 位配合产生所需的时钟/数据关系 0: 总线空闲时 CK 引脚上保持低电平; 1: 总线空闲时 CK 引脚上保持高电平。该位不适用于 UART4 和 UART5。	0x1	rw	
9	CPHA	时钟相位 该位允许用户在同步模式下选择 CK 引脚上时钟输出的相位。它与 CPOL 位一起工作以产生所需的时钟/数据关系 (见图 289 至 290) 0: 第一个时钟跳变是第一个数据的捕捉沿 (在时钟的第一个边沿进行数据捕获)。 1: 第二个时钟跳变是第一个数据的捕捉沿 (在时钟的第二个边沿进行数据捕获)。 该位不适用于 UART4 和 UART5。	0x1	rw	
8	LBCL	最后一位时钟脉冲 在同步模式下, 使用该位来控制是否在 CK 引脚上输出最后发送的那个数据字节(MSB)对应的时钟脉冲 0: 最后一个数据位的时钟脉冲不输出到 CK 引脚 1: 最后一个数据位的时钟脉冲输出到 CK 引脚 最后一位是传输的第 8 位或第 9 位数据位, 具体取决于 USART_CR1 寄存器中的 M 位选择的 8 位或 9 位格式。 该位不适用于 UART4 和 UART5。	0x0	rw	
7	Reserved	Reserved	0x0	r	
6	LBDIE	LIN 断开符检测中断使能 断开符中断屏蔽 (使用断点分隔符的断点检测)。 0: 禁止中断 1: 只要 USART_SR 寄存器中的 LBD = 1, 就会产生中断	0x0	rw	

Bit	Field	Description	Reset value	RW attribute	Remark
5	LBDL	LIN 开符检测长度 该位用于在 11 位或 10 位中断检测之间进行选择。 0:10 位的断开符检测 1:11 位的断开符检测	0x0	rw	
4	Reserved	Reserved	0x0	r	
3:0	ADD	USART 节点地址 该位域给出了 USART 节点的地址。 这用于静音模式期间的多处理器通信, 用于通过地址标记检测唤醒某个设备。	0x0	rw	

24.6.6 寄存器 (USART_CR3)

偏移地址: 0x14, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HDSEL	IRLP	IREN	EIE
r					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
10	CTSIE	CTS 中断使能 0: 禁止中断 1: 只要 USART_SR 寄存器中的 CTS = 1, 就会产生中断。 该位不适用于 UART4 和 UART5。	0x0	rw	
9	CTSE	启用 CTS 0: 禁用 CTS 硬件流控制 1: CTS 模式使能, 只有 nCTS 输入信号有效(拉成低电平)时才能发送数据。如果在数据传输的过程中, nCTS 信号变成无效, 那么发完这个数据后, 传输就停止下来。如果当 nCTS 为无效时, 往数据寄存器里写数据, 则要等到 nCTS 有效时才会发送这个数据。 该位不适用于 UART4 和 UART5。	0x0	rw	
8	RTSE	RTS 启用 0: 禁用 RTS 硬件流控制 1: RTS 中断使能 只有接收缓冲区内有空余的空间时才请求下一个数据。当前数据发送完成后, 发送操作就需要暂停下来。如果可以接收数据了, 将 nRTS 输出置为有效(拉至低电平)。 注: 该位对于 UART4 和 UART5 不可用。	0x0	rw	
7	DMAT	DMA 使能发送器 该位由软件设置/复位 0: 禁止发送时的 DMA 模式。 1: 使能发送时的 DMA 模式; 该位不可用于 UART 5。	0x0	rw	
6	DMAR	DMA 使能接收器 该位由软件设置/复位 1: DMA 模式接收使能 0: DMA 模式禁止接收 该位不可用于 UART 5。	0x0	rw	

Bit	Field	Description	Reset value	RW attribute	Remark
5	SCEN	智能卡模式启用 该位用于启用智能卡模式。 0: 禁用智能卡模式 1: 启用智能卡模式 该位不适用于 UART4 和 UART5。	0x0	rw	
4	NACK	智能卡 NACK 使能 0: 在奇偶校验错误的情况下 NACK 传输被禁用 1: 奇偶校验错误时的 NACK 传输使能 该位不适用于 UART4 和 UART5。	0x0	rw	
3	HDSEL	半双工选择 单线半双工模式选择 0: 未选择半双工模式 1: 选择半双工模式	0x0	rw	
2	IRLP	IrDA 低功耗 该位用于在正常和低功耗 IrDA 模式之间进行选择 0: 正常模式 1: 低功耗模式	0x0	rw	
1	IREN	IrDA (红外) 模式启用 该位由软件置位和清零。 0: IrDA 禁用 1: IrDA 使能	0x0	rw	
0	EIE	错误中断使能 错误中断使能位需要在帧错误、溢出错误或噪声错误 (USART_SR 寄存器中的 FE = 1 或 ORE = 1 或 NE = 1) 的情况下启用中断生成。(在) USART_CR3 寄存器)。 0: 禁止中断 1: 只要 USART_CR3 寄存器中的 DMAR = 1 且 USART_SR 寄存器中的 FE = 1 或 ORE = 1 或 NE = 1, 就会产生中断。	0x0	rw	

24.6.7 寄存器 (USART_GTPR)

偏移地址: 0x18, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
GT								PSC							
rw								rw							

Bit	Field	Description	Reset value	RW attribute	Remark
15:8	GT	保护时间值 该位域规定了以波特时钟为单位的保护时间。在智能卡模式下, 需要这个功能。当保护时间过去后, 才会设置发送完成标志。 该位不适用于 UART4 和 UART5。	0x0	RW	

Bit	Field	Description	Reset value	RW attribute	Remark
7:0	PSC	f 预分频器值 – 在 IrDA 低功耗模式下: PSC [7: 0] = IrDA 低功耗波特率 用于对预分频器进行编程以对系统时钟进行分频以实现低功耗频率 源时钟除以寄存器中给定的值 (8 个有效位): 00000000: 保留 -- 不对该值进行编程 00000001: 将源时钟除以 1 00000010: 将源时钟除以 2 - 在红外(IrDA) 的正常模式下: PSC 只能设置为 00000001 - 在智能卡模式下: PSC[4:0]: 预分频值 对系统时钟进行分频, 给智能卡提供时钟。 寄存器中给出的值(低 5 位有效)乘以 2 后, 作为对源时钟的分频因子 00000: 保留 – 不要写入该值; 00001: 对源时钟进行 2 分频; 00010: 对源时钟进行 4 分频; 00011: 对源时钟进行 6 分频; 注意: 1. 位[7:5]在智能卡模式下没有意义。	0x0	RW	

25 内部集成电路总线接口 (I2C)

25.1 I2C 接口

25.1.1 功能描述

I2C 总线是两线串行接口，由串行数据线 (SDA) 和串行时钟 (SCL) 组成。这些电线在连接到总线的设备之间传送信息。每个设备都有一个唯一的地址，并且可以根据设备的功能充当“发送方”或“接收方”。在执行数据传输时，设备也可以视为主机或从机。主机是一种在总线上启动数据传输并生成时钟信号以允许该传输的设备，任何寻址的设备都被视为从设备。

25.1.2 主要特点

- 两线制 I2C 串行接口—由串行数据线 (SDA) 和串行时钟 (SCL) 组成
- 二种速度：
 - 标准模式 (0 to 100 Kb/s)
 - 快速模式 (≤ 400 Kb/s) 和快速模式+ (≤ 1000 Kb/s)
- 时钟同步
- 主或从的 I2C 操作
- 7 或 10bit 寻址
- 7 或 10bit 组合格式传输
- Bulk 传输模式
- 忽略 CBUS 地址
- 发送和接受缓冲
- 中断或者轮询模式操作
- 处理所有总线下的位或字节等待
- 用于可配置软件驱动程序支持的组件参数
- 状态标志：
 - 发送器/接收器模式标志
 - 字节发送结束标志
 - 字节接收状态标志
- 四字节缓冲的 DMA
- 2 个中断向量
 - 1 个中断用于地址/数据通讯状态
 - 1 个中断用于出错
- 支持 SMBUS/PMBUS
- SMBus 从站检测并响应 ARP 命令

25.1.3 概述

I2C 接口接收和发送数据，并将数据从串行转换成并行，或并行转换成串行。可以开启或禁止中断。接口通过数据引脚(SDA)和时钟引脚(SCL)连接到 I2C 总线。 允许连接到标准(高至 100 kHz)或快速(高至 1 MHz)I2C 总线。 模式选择接口可以下述 4 种模式中的一种运行：

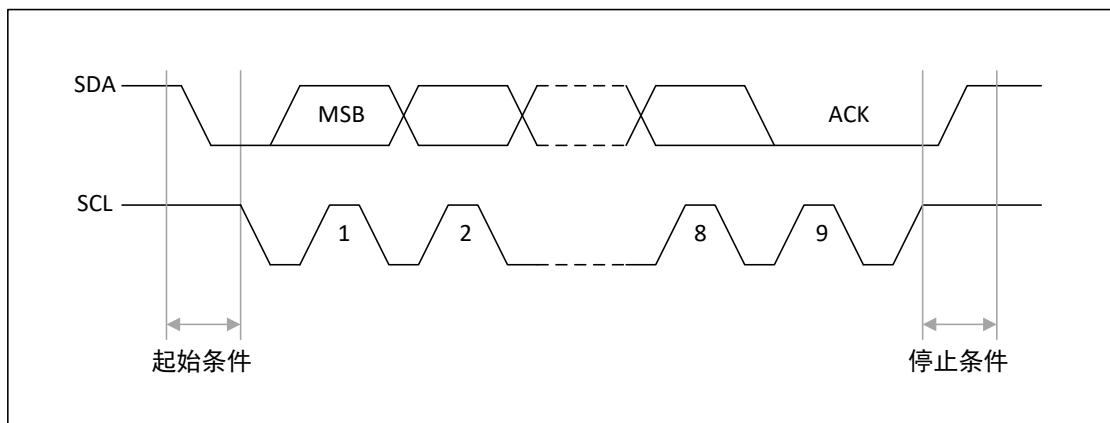
- 从发送器模式
- 从接收器模式
- 主发送器模式
- 主接收器模式

默认模式为从模式。接口在生成起始条件后自动从从模式切换到主模式；当仲裁丢失或产生停止信号，则从主模式切换到从模式。允许多主机功能。

通信流

主模式时，I2C 接口启动数据传输并产生时钟信号。串行数据传输总是以起始条件开始和停止条件结束。主模式时，由软件控制产生起始条件和停止条件。从模式时，I2C 接口能识别它自己的地址(7 位或 10 位)和广播呼叫地址。软件控制开启或禁止广播呼叫地址的识别。数据和地址按 8 位/字节进行传输，高位在前。跟在起始条件后的第一、二个字节是地址(7 位模式为 1 个字节，10 位模式为 2 个字节)。地址只在主模式发送。在一个字节传输的 8 个时钟后的第 9 个时钟期间，接收器必须回送一个应答位 (ACK)给发送器。参考下图

图25.1 IIC 传输协议



软件可以开启或禁止应答(ACK)，I2C 接口的地址(7 位、10 位地址或广播呼叫地址)可通过软件设置。

25.1.4 功能描述

默认情况下，I2C 接口总是工作在从模式。从默认的从模式切换到主模式，需要产生一个起始条件。

25.1.4.1 I2C 从设备模式

25.1.4.2 初始配置：

- 关闭 I2C，将 IC_ENABLE[0]=1'b0；
- 设置从设备地址 I2C_SAR[9:0]，假如双地址模式 I2C_CON[12], I2C_SAR_DUAL_EN=1，则同时设置第二个地址 I2C_SAR[22:16]；
- 设置 I2C_CON[3] 确定是 7bit 还是 10bit 地址模式。设置 I2C_CON[6]=0，打开从模式，并且将 I2C_CON[0]=0，关闭主模式。
- 打开 I2C 使能，I2C_ENABLE[0]=1'b1；

25.1.4.3 从设备单字节发送模式：

- 其他的 I2C 主设备发起和当前从设备地址 I2C_SAR 匹配的传输；
- 当前设备响应了该传输并且识别了当前设备作为从设备发送模式工作；
- 当前设备发起 RD_REQ 中断，I2C_RAW_INTR_STAT[5]被置 1，并且将 SCL 拉低，等待软件响应；
- 假设 I2C_INTR_MASK[5] =0，则 RD_REQ 中断被屏蔽，那么建议采用轮询的方式读 I2C_RAW_INTR_STAT[5]；

- 当读出 I2C_RAW_INTR_STAT[5]=1, 则认为 RD_REQ 中断已发起;
- 软件对该事件进行响应;
- 时间间隔至少为 SCL 周期的 10 倍, 比如 400kb/s, 那么间隔应该为 25us;
- 假设在收到读请求时, TX FIFO 仍有数据残留, 则设置 TX_ABRT 中断, I2C_RAW_INTR_STAT[6]=1, 并将 TX FIFO 清空。
- 假设 I2C_INTR_MASK[6] =0, 则 TX_ABRT 中断被屏蔽, 同样要采用轮询的方式读取 I2C_RAW_INTR_STAT[6]:
 - 当读出 I2C_RAW_INTR_STAT[6]=1, 则认为 TX_ABRT 中断已发起;
 - 软件无需对该事件进行响应;
 - 时间间隔至少为 SCL 周期的 10 倍, 比如 400kb/s, 那么间隔应该为 25us;
- 软件将发送数据写入 I2C_DATA_CMD 寄存器, bit8 写 0;
- 软件必须清除 RD_REQ 和 TX_ABRT 中断, I2C_RAW_INTR_STAT[5]/[6], 假设中断被屏蔽, 则 R_RD_REQ 或者 R_TX_ABRT 被读出为 1 时, 则可以达到相同的效果;
- 当前设备释放 SCL, 发送数据;
- 主设备通过 RESTART 维持对 I2C 的掌控或者发起 STOP 指令。

25.1.4.4 从设备单字节接收模式:

当总线上其他 I2C 主设备发送数据, 当前设备以从设备接收模式工作, 步骤如下:

- 其他的 I2C 主设备发起和当前从设备地址 I2C_SAR 匹配的传输;
- 当前设备响应了该传输并且识别了当前设备作为从设备接收模式工作;
- 当前设备接收数据, 并将数据存放在接收 buffer 内。
- 当前设备设置 RX_FULL 中断, I2C_RAW_INTR_STAT[2]设定为 1, 假如 I2C_INTR_MASK[2]=0 或者 I2C_TX_TL 大于 0, 则 RX_FULL 中断被屏蔽。则通过轮询的方式读 I2C_STATUS[3], 结果为 1 时, 则认为 RX_FULL 中断发生。
- 软件通过读取 I2C_DATA_CMD 得到接收数据;
- 主设备通过 RESTART 维持对 I2C 的掌控或者发起 STOP 指令。

25.1.4.5 I2C 主设备模式

25.1.4.6 初始配置:

- 关闭 I2C, 将 I2C_ENABLE[0]=1'b0;
- 设置 I2C_CON[2:1]: 最大速度模式; I2C_CON[4]: 7/10bit 地址模式; I2C_CON[6]=1: 关闭从机模式; I2C_CON[0]=1: 打开主机模式;
- 设置设备地址 I2C_TAR[9:0];
- 打开 I2C 使能, I2C_ENABLE[0]=1'b1;
- 写入 I2C_DATA_CMD, 包括传输方向比特 I2C_DATA_CMD [8]和数据 I2C_DATA_CMD[7:0]。本步骤产生 START 条件以及地址。当数据存入 TX FIFO, 传输开始;
- I2C_DATA_CMD [8]=0 时, 表示主设备发送模式, 此时从设备采用从设备接收模式;
- I2C_DATA_CMD [8]=1 时, 表示主设备接收模式, 此时从设备采用从设备发送模式。

25.1.4.7 DMA 传输

25.1.4.8 主设备发送 DMA

数据从 sram 以 DMA 的方式搬运至 IIC Master, 步骤如下:

- 关闭 IIC, ENABLE=0;
- 设定 CON 控制寄存器;
- 设定 TAR, TX_TL, INTR_MASK 等;
- 打开 DMA_CR[1]=1, TX DMA 使能;
- 设置 DMA_TDLR, 发送 DMA FIFO 的数据传输等级;
- 设定 DMA 相关参数, 包括:
 - DMA->CONFIGURATION
 - DMA_Channelx->SRCADDR,
 - DMA_Channelx->DESTADDR,
 - DMA_Channelx->CONTROL
- 打开 IIC, ENABLE=1;
- 设定 DMA_Channelx->CONFIGURATION;
- 设定 DMA->PERIPG2SEL
- 传输结束需要在 DAM 中断函数中清空相关中断, 并关闭 IIC TX DMA, DMA_CR[1]=0;

25.1.4.9 从设备接收 DMA

数据从 IIC Slaver 以 DMA 的方式搬运至 sram, 步骤如下:

- 关闭 IIC, ENABLE=0;
- 设定 CON 控制寄存器;
- 设定 TAR, TX_TL, INTR_MASK 等;
- 打开 DMA_CR[0]=1, RX DMA 使能;
- 设置 DMA_RDLR, 接收 DMA FIFO 的数据传输等级;
- 设定 DMA 相关参数, 包括:
 - DMA->CONFIGURATION
 - DMA_Channelx->SRCADDR,
 - DMA_Channelx->DESTADDR,
 - DMA_Channelx->CONTROL
- 设定 DMA_Channelx->CONFIGURATION;
- 设定 DMA->PERIPG2SEL
- 打开 IIC, ENABLE=1
- 传输结束需要在 DAM 中断函数中清空相关中断, 并关闭 IIC RX DMA, DMA_CR[0]=0;

25.1.4.10 中断请求

图25.2 错误中断

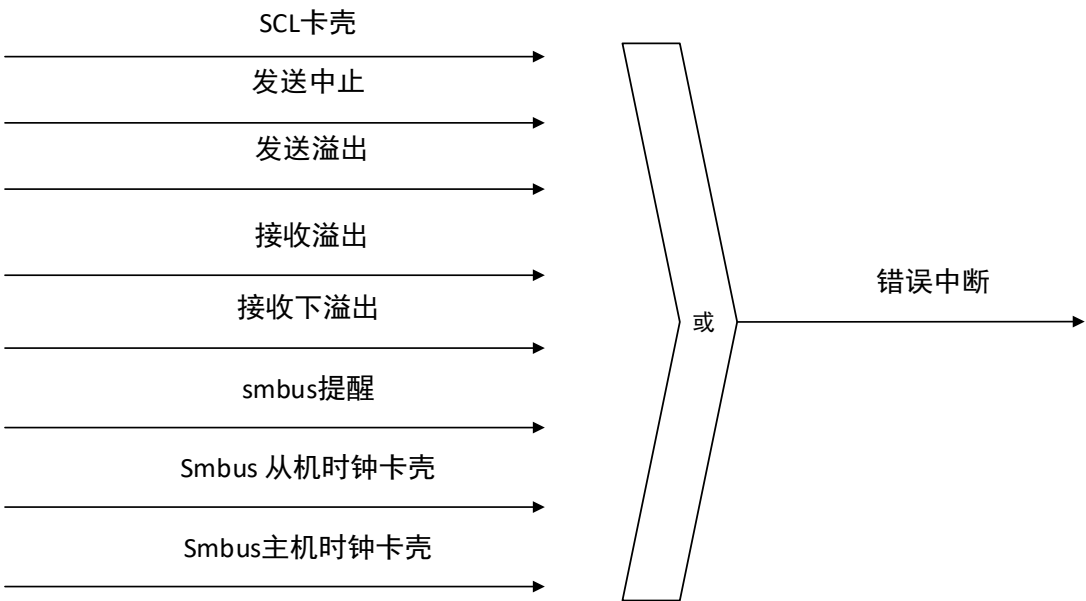
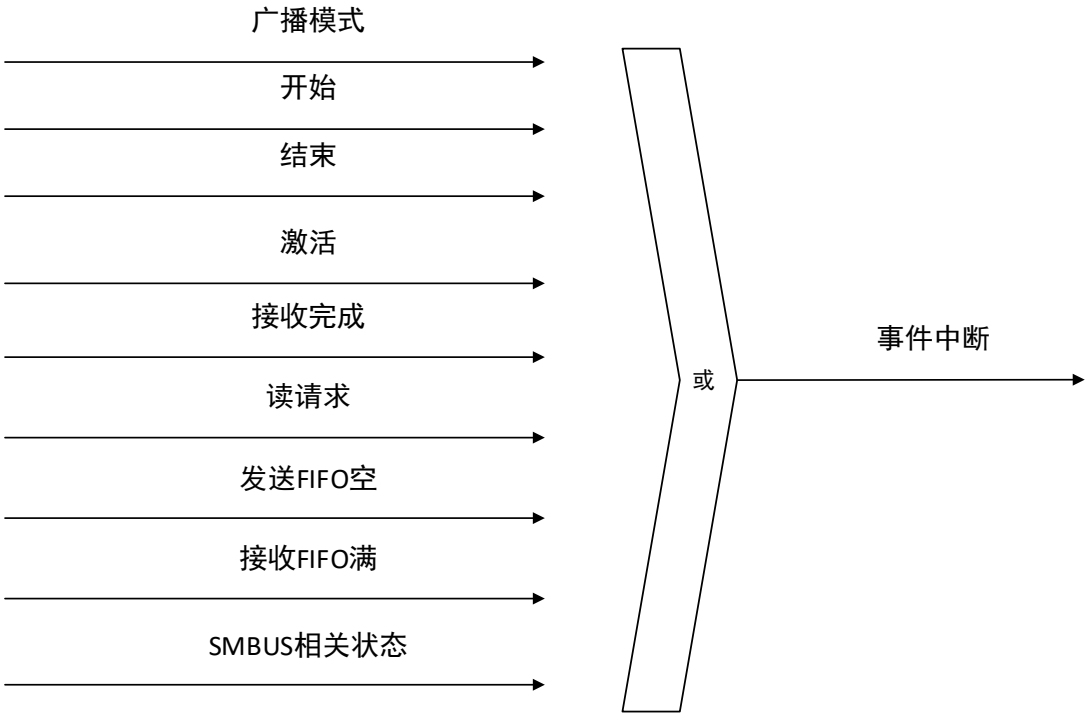


图25.3 事件中断



25.2 寄存器

25.2.1 寄存器（I2C_CON）

偏移地址：0x00，复位值：0x0000 0030。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													SMBUS_	Reserved	
-													RW	-	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			SAR_DUAL_EN	BUS_CLEAR_FEATURE_CTRL	Reserved	RX_FIFO_FULL_HLD_CTRL	TX_EMPTY_CTRL	STOP_DET_IFADDRESSED	SLAVE_DISABLEABLE	RESTART_EN	10BITADDR_MASTER	10BITADDR_SLAVE	SPEED		MASTER_MODE
-			RW	RW	-	RW	RW	RW	RW	RW	RW	RW	RW		RW

Bit	Field	Description	Reset value	RW attribute	Remark
31:19	Reserved				
18	SMBUS_ARP_EN	该位控制设备是否应在 SMBus 模式下启用地址解析逻辑。从模式将解码地址解析协议命令并响应它。从设备还包括地址解析协议命令的 PEC 字节的生成/有效性。该位仅适用于从模式。	0x0	RW	
17:13	Reserved				
12	SAR_DUAL_EN	1: 在 比特 7 从机模式下启用双地址	1	RW	
11	BUS_CLEAR_FEATURE_CTRL	在主模式下: 1: 总线清除功能已启用 0: 总线清除功能被禁用 在从模式下, 该寄存器位不适用。	0	RW	
10	Reserved				
9	RX_FIFO_FULL_HLD_CTRL	该位控制设备是否应在 Rx FIFO 物理满至其 RX_BUFFER_DEPTH 时保持总线, 如 I2C_RX_FULL_HLD_BUS_EN 参数中所述。	0	RW	
8	TX_EMPTY_CTRL	该位控制 TX_EMPTY 中断的产生, 如 I2C_RAW_INTR_STAT 寄存器中所述。	0	RW	
7	STOP_DET_IFADDRESSED	从机模式下: 1'b1 – 仅在被寻址到时发出 STOP_DET 中断。 1'b0 – 发出 STOP_DET, 不管是否被寻址到。	0	RW	
6	SLAVE_DISABLE	该位控制 I2C 是否禁用其从设备, 这意味着一旦复位, 则该位采用配置参数 I2C_SLAVE_DISABLE 的值。您可以选择应用复位后启用或禁用从站, 这意味着软件不必配置从站。默认情况下, 从站始终处于启用状态 (也处于复位状态)。如果您需要在复位后禁用它, 请将此位设置为 1。 如果设置了该位 (从站被禁用), 设备仅用作主站, 不会执行任何需要从站的操作。 0: 从机已启用 1: 从机被禁用	0	RW	

Bit	Field	Description	Reset value	RW attribute	Remark
5	RESTART_EN	确定作为主机时是否可以发送 RESTART 条件。一些较旧的从站不支持处理 RESTART 条件；但是在多个操作中使用了 RESTART 条件。 0: 禁用 1: 启用 当 RESTART 被禁用时，设备主站无法执行以下功能： 发送 START BYTE ； 执行任何高速模式操作； 在组合格式模式下执行方向更改 ； 使用 10 位地址执行读操作 ； 通过替换 RESTART 条件为一个 STOP 和随后的 START ，则操作被分解为多次传输。如果执行上述操作，将导致设置 I2C_RAW_INTR_STAT 寄存器的第 6 位 (TX_ABRT) 。	0	RW	
4	10BITADDR_MASTER	控制设备作为主机时是否以 7 位或 10 位寻址模式开始其传输。 0: 7 位寻址 1: 10 位寻址	0	RW	
3	10BITADDR_SLAVE	当作为从机时，该位控制设备是响应 7 位地址还是 10 位地址。 0: 7 位寻址。设备忽略涉及 10 位寻址的事务；对于 7 位寻址，仅比较 I2C_SAR 寄存器的低 7 位。 1: 10 位寻址。设备仅响应与 I2C_SAR 寄存器的全部 10 位匹配的 10 位寻址传输。	0	RW	
2:1	SPEED	这些位控制运行速度。硬件可以防止软件编程非法值。这些位必须针对从模式进行适当编程，因为它用于根据速度模式设定尖峰滤波器的正确值。 该寄存器只能编程为 0x1 或者 0x2；否则硬件会使用 0x2 更新该寄存器： 1: 标准模式 (0 到 100 Kb/s) 2: 快速模式 (≤ 400 Kb/s) 或快速模式加 (≤ 1000 Kb/s)	0	RW	
0	MASTER_MODE	该位控制是否启用设备主站。 0: 主机禁用 1: 主机使能	0	RW	

25.2.2 寄存器 (I2C_TAR)

偏移地址: 0x04, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															SMBUS_QUICK_CMD
-															RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				SPECIAL	GC_OR_START	I2C_TAR									
-	RW	RW	RW	RW	RW	RW									

Bit	Field	Description	Reset value	RW attribute	Remark
31:17	Reserved				

Bit	Field	Description	Reset value	RW attribute	Remark
16	SMBUS_QUICK_CMD	如果第 11 位 (SPECIAL) 设置为 1, 则该位指示设备是否要执行快速命令。	0	RW	
15:12	Reserved				
11	SPECIAL	该位指示软件执行的是否为特殊命令 (广播呼叫或起始字节命令) 0: 忽略位 10 GC_OR_START, 正常使用 TAR 1: 实行特殊 I2C 命令, 细节请参考 GC_OR_START 位的描述	0	RW	
10	GC_OR_START	如果第 11 位 (SPECIAL) 设置为 1, 则该位指示设备是要执行广播呼叫还是 START byte 字节命令。 0: 广播地址——发出广播后, 只能执行写操作。尝试发出读命令会导致设置 I2C_RAW_INTR_STAT 寄存器的第 6 位 (TX_ABORT)。设备保持在广播呼叫模式, 直到 SPECIAL 位值 (位 11) 被清除。 1: 开始字节	0	RW	
9:0	TAR	这是任何主交易的目标地址。发送广播呼叫时, 这些位被忽略。要生成 START BYTE, CPU 只需向这些位写入一次。	0	RW	

25.2.3 寄存器 (I2C_SAR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									I2C_SAR_DUAL						
-									RW						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						I2C_SAR									
-						RW									

Bit	Field	Description	Reset value	RW attribute	Remark
31:23	Reserved				
22:16	I2C_SAR_DUAL	双地址, 仅用于 7 位从机模式	0	RW	
15:10	Reserved				
9:0	I2C_SAR	当 I2C 作为从设备运行时, I2C_SAR 保存从设备地址。对于 7 位寻址, 只使用 I2C_SAR[6:0]。该寄存器只有在 I2C 接口禁用时才能写入, 对应 I2C_ENABLE[0] 设置为 0, 其他时间写入无效。	0	RW	

25.2.4 寄存器 (I2C_DATA_CMD)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				FIRST_DATA_BYTE	RESATRT	STOP	CMD	DAT							
				R	W	W	W	RW							

Bit	Field	Description	Reset value	RW attribute	Remark
31:12	Reserved				

Bit	Field	Description	Reset value	RW attribute	Remark
11	FIRST_DATA_BYTE	指示在主接收器或从接收器模式下接收传输的地址阶段后接收到的第一个数据字节。			
10	RESTART	该位控制在发送或接收字节之前是否发出 RESTART。 1: 如果 I2C_RESTART_EN 为 1, 则在发送/接收数据之前发出 RESTART (根据 CMD 的值), 无论传输方向是否从上一个命令改变; 如果 I2C_RESTART_EN 为 0, 则改为发出 STOP 后跟 START。 0: 如果 I2C_RESTART_EN 为 1, 则仅当传输方向从前一个命令改变时才发出 RESTART; 如果 I2C_RESTART_EN 为 0, 则改为发出 STOP 后跟 START。			
9	STOP	该位控制在发送或接收字节后是否发出 STOP。 1: 在该字节后发出 STOP, 无论 Tx FIFO 是否为空。如果 Tx FIFO 不为空, 则主机立即尝试通过发出 START 并为总线进行仲裁来开始新的传输。 0: 在该字节后不发出 STOP, 无论 Tx FIFO 是否为空。如果 Tx FIFO 不为空, 则主机通过根据 CMD 位的值发送/接收数据字节来继续当前传输。如果 Tx FIFO 为空, 则主机将 SCL 线保持为低电平并暂停总线, 直到 Tx FIFO 中有新命令可用。			
8	CMD	该位控制是执行读操作还是写操作。当设备作为从机时, 该位不控制方向。当它作为主机时, 它只控制方向。 1 = 读取 0 = 写 当在 TX FIFO 中输入命令时, 该位区分写命令和读命令。在从接收器模式下, 该位是“无关位”, 因为不需要写入该寄存器。在 slave-transmitter 模式, “0”表示要发送 I2C_DATA_CMD 中的数据。编程该位时, 您应该记住以下几点: 在发送通用呼叫命令后尝试执行读操作会导致 TX_ABRT 中断 (I2C_RAW_INTR_STAT 寄存器的第 6 位), 除非 I2C_TAR 寄存器中的第 11 位 (特殊) 已被清除。如果在接收到 RD_REQ 中断后向该位写入 “1”, 则发生 TX_ABRT 中断。			
7:0	DAT	该寄存器包含要在 I2C 总线上发送或接收的数据。 如果您正在写入该寄存器并希望执行读取, 则设备将忽略位 7:0 (DAT)。但是, 当您读取该寄存器时, 这些位会返回在设备接口上接收到的数据值。			

25.2.5 寄存器 (I2C_SS_SCL_HCNT)

偏移地址: 0x14, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_SS_SCL_HCNT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved	-	-	-	
15:0	I2C_SS_SCL_HCNT	该寄存器必须在任何 I2C 总线事务发生之前设置, 以确保正确的 I/O 时序。该寄存器为标准速度设置 SCL 时钟高周期计数。该寄存器仅在 I2C 接口禁用时才能写入, 对应于 I2C_ENABLE[0] 设置为 0。其他时间写入无效。最小有效值为 6; 硬件会阻止写入小于此值的值, 如果尝试会导致设置 6。对于 APB_DATA_WIDTH = 8 的设计, 编程顺序对于确保设备的正确操作很重要。必须首先编程低字节。然后高字节被编程。	0	RW	

25.2.6 寄存器 (I2C_SS_SCL_LCNT)

偏移地址: 0x18, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_SS_SCL_LCNT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved				
15:0	I2C_SS_SCL_LCNT	该寄存器必须在任何 I2C 总线事务发生之前设置, 以确保正确的 I/O 时序。该寄存器为标准速度设置 SCL 时钟低周期计数。该寄存器仅在 I2C 接口禁用时才能写入, 对应于 I2C_ENABLE[0] 设置为 0。其他时间写入无效。最小有效值为 8; 硬件会阻止写入小于此值的值, 如果尝试, 则会导致设置为 8。对于 APB_DATA_WIDTH = 8 的设计, 编程顺序对于确保设备的正确操作很重要。必须先编程低字节, 然后编程高字节。	0	RW	

25.2.7 寄存器 (I2C_FS_SCL_HCNT)

偏移地址: 0x1C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_FS_SCL_HCNT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved				

Bit	Field	Description	Reset value	RW attribute	Remark
15:0	I2C_FS_SCL_HCNT	该寄存器必须在任何 I2C 总线事务发生之前设置, 以确保正确的 I/O 时序。该寄存器设置快速模式或快速模式增强的 SCL 时钟高周期计数。用于快速模式下发送 Master Code 和 START BYTE 或 General CALL。该寄存器只有在 I2C 接口禁用时才能写入, 对应 I2C_ENABLE[0] 设置为 0。其他时间写入无效。最小有效值为 6; 硬件会阻止写入小于此值的值, 如果尝试会导致设置 6。对于 APB_DATA_WIDTH == 8 的设计, 编程顺序对于确保设备的正确操作很重要。必须首先编程低字节。然后高字节被编程。	0	RW	

25.2.8 寄存器 (I2C_FS_SCL_LCNT)

偏移地址: 0x20, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_FS_SCL_LCNT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved				
15:0	I2C_FS_SCL_LCNT	该寄存器必须在任何 I2C 总线事务发生之前设置, 以确保正确的 I/O 时序。该寄存器设置快速模式或快速模式增强的 SCL 时钟低周期计数。用于快速模式下发送 Master Code 和 START BYTE 或 General CALL。该寄存器只有在 I2C 接口禁用时才能写入, 对应 I2C_ENABLE[0] 设置为 0。其他时间写入没有效果。最小有效值为 8; 硬件会阻止写入小于此值的值, 如果尝试导致设置为 8。对于 APB_DATA_WIDTH = 8 的设计, 编程顺序对于确保设备的正确操作很重要。必须首先编程低字节。然后高字节被编程。如果该值小于 8, 则计数值将更改为 8。	0	RW	

25.2.9 寄存器 (I2C_INTR_STAT)

偏移地址: 0x2C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	R_SCL_STUCK_AT_LOW	Reserved		R_GEN_CAL	R_START_DET	R_STOP_DET	R_ACTIVITY	R_RX_DONE	R_TX_ABORT	R_RD_REQ	R_TX_EMPTY	R_TX_OVER	R_RX_FULL	R_RX_OVER	R_RX_UNDER
-	R	-		R	R	R	R	R	R	R	R	R	R	R	R

Bit	Field	Description	Reset	RW
31:15	Reserved		0x0	r
14	R_SCL_STUCK_AT_LOW	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	r
13:12	Reserved		0x0	rw
11	R_GEN_CALL	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
10	R_START_DET	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw

Bit	Field	Description	Reset	RW
9	R_STOP_DET	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
8	R_ACTIVITY	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
7	R_RX_DONE	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
6	R_TX_ABRT	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
5	R_RD_REQ	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
4	R_TX_EMPTY	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
3	R_TX_OVER	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
2	R_RX_FULL	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
1	R_RX_OVER	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw
0	R_RX_UNDER	有关这些位的详细说明, 请参阅 I2C_RAW_INTR_STAT。	0x0	rw

25.2.10 寄存器 (I2C_INTR_MASK)

偏移地址: 0x30, 复位值: 0x0000 48FF。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M_SCL_STUCK_AT_LOW	Reserved			M_GEN_CALL	M_START_DET	M_STOP_DET	M_ACTIVITY	M_RX_DONE	M_TX_ABRT	M_RD_REQ	M_TX_EMPTY	M_TX_OVER	M_RX_FULL	M_RX_OVER	M_RX_UNDER
-	RW			RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

Bit	Field	Description	Reset	RW
31:15	Reserved		0x1	rw
14	M_SCL_STUCK_AT_LOW	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位。	0x1	rw
13:12	Reserved		0x1	rw
11	M_GEN_CALL	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
10	M_START_DET	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
9	M_STOP_DET	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
8	M_ACTIVITY	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
7	M_RX_DONE	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
6	M_TX_ABRT	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
5	M_RD_REQ	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
4	M_TX_EMPTY	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
3	M_TX_OVER	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
2	M_RX_FULL	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
1	M_RX_OVER	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw
0	M_RX_UNDER	这些位在 I2C_INTR_STAT 寄存器中屏蔽了它们对应的中断状态位	0x1	rw

25.2.11 寄存器 (I2C_RAW_INTR_STAT)

偏移地址: 0x34, 复位值: 0x0000 48FF。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	SCL_STUCK_AT_LOW	Reserved		GEN_CALL	START_DET	STOP_DET	ACTIVITY	RX_DONE	TX_ABORT	RD_REQ	TX_EMPTY	TX_OVERR	RX_FULL	RX_OVERR	RX_UNDEERR
-	R	-		R	R	R	R	R	R	R	R	R	R	R	R

Bit	Field	Description	Reset value	RW attribute	Remark
31:15	Reserved				
14	SCL_STUCK_AT_LOW	指示 SCL 线是否在 I2C_SCL_STUCK_LOW_TIMEOUT 数量的 i2c_clk 周期内卡在低电平。	0	R	
13:12	Reserved				
11	GEN_CALL	仅在接收到广播呼叫地址并得到确认时设置。它保持设置状态,直到通过禁用设备或 CPU 读取 I2C_CLR_GEN_CALL 寄存器的位 0 将其清除。设备将接收到的数据存储在 Rx 缓冲区中。	0	R	
10	START_DET	指示 I2C 接口上是否发生了 START 或 RESTART 条件,而不管设备是在从模式还是主模式下运行。	0	R	
9	STOP_DET	指示 I2C 接口上是否发生了 STOP 条件,而不管设备是在从模式还是主模式下运行。 在从模式下: 如果 I2C_CON[7]=1'b1 (STOP_DET_IFADDRESSED),则 STOP_DET 只有当从机被寻址时才会产生中断。注意:在广播地址期间,如果 STOP_DET_IFADDRESSED=1'b1,即使从机通过生成 ACK 响应广播地址,该从机也不会发出 STOP_DET 中断。STOP_DET 中断仅在传输地址与从机地址 (SAR) 匹配时产生。 如果 I2C_CON[7]=1'b0 (STOP_DET_IFADDRESSED),则 STOP_DET 无论是否被寻址,都会发出中断。 在主机模式下:则无论主机是否处于活动状态,都会发出 STOP_DET 中断。	0	R	
8	ACTIVITY	该位捕获设备活动并保持设置直到它被清除。有四种方法可以清除它: 禁用设备 读取 I2C_CLR_ACTIVITY 寄存器 读取 I2C_CLR_INTR 寄存器 系统复位一旦该位被置位,它就会保持置位,除非使用四种方法之一来清除它。即使设备模块空闲,该位也一直保持置位直到被清除,表明曾经有效过。	0	R	
7	RX_DONE	当设备用作从发送器时,如果主机不确认发送的字节,则该位设置为 1。这发生在传输的最后一个字节上,表明传输已完成。	0	R	
6	TX_ABORT	该位指示作为 I2C 发送器的设备是否无法完成对发送 FIFO 内容的预期操作。这种情况既可以作为 I2C 主设备也可以作为 I2C 从设备发生,称为“传输中止”。当该位设置为 1 时,I2C_TX_ABORT_SOURCE 寄存器指示发送中止发生的原因。	0	R	

Bit	Field	Description	Reset value	RW attribute	Remark
5	RD_REQ	当设备作为从设备并且另一个 I2C 主设备尝试从设备读取数据时, 该位设置为 1。设备将 I2C 总线保持在等待状态 (SCL=0) 直到该中断被服务, 这意味着从设备已被要求传输数据的远程主设备寻址。 处理器必须响应此中断, 然后将请求的数据写入 I2C_DATA_CMD 寄存器。 该位在处理器读取 I2C_CLR_RD_REQ 寄存器后立即设置为 0。	0	R	
4	TX_EMPTY	TX_EMPTY 中断状态的行为因 I2C_CON 寄存器中的 TX_EMPTY_CTRL 选择而异。 当 TX_EMPTY_CTRL = 0 时: 当发送缓冲区等于或低于 I2C_TX_TL 寄存器中设置的阈值时, 该位设置为 1。当 TX_EMPTY_CTRL = 1 时: 当发送缓冲区等于或低于阈值时, 该位设置为 1 在 I2C_TX_TL 寄存器中设置, 并完成从内部移位寄存器传输最近弹出命令的地址/数据。 当缓冲水平超过阈值时, 它由硬件自动清除。 当 I2C_ENABLE[0] 设置为 0 时, TX FIFO 被刷新并保持在复位状态。 TX FIFO 看起来好像里面没有数据, 所以如果主状态机或从状态机中有活动, 则该位设置为 1。 当不再有任何活动时, 如果 i2c_en=0, 则该位设置为 0。	0	R	
3	TX_OVER	如果发送缓冲区已填充到 I2C_TX_BUFFER_DEPTH 并且处理器尝试通过写入 I2C_DATA_CMD 寄存器来发出另一个 I2C 命令, 则在发送期间设置。当模块被禁止时, 该位保持其电平直到主或从状态机进入空闲状态, 当 i2c_en 变为 0 时, 该中断被清除。	0	R	
2	RX_FULL	当接收缓冲器达到或超过 I2C_RX_TL 寄存器中的 RX_TL 阈值时设置。 当缓冲区水平低于阈值时, 它会被硬件自动清除。 如果模块被禁用 (I2C_ENABLE[0]=0), 则 RX FIFO 被刷新并保持在复位状态; 因此 RX FIFO 未充满。 因此, 一旦 I2C_ENABLE[0] 设置为 0, 该位将被清除, 无论活动是否继续。	0	R	
1	RX_OVER	如果接收缓冲区完全填充到 I2C_RX_BUFFER_DEPTH 并且从外部 I2C 设备接收到额外字节, 则设置。设备对此进行确认, 但在 FIFO 已满后接收到的任何数据字节都将丢失。 如果模块被禁用 (I2C_ENABLE[0]=0), 该位保持其电平直到主或从状态机进入空闲状态, 当 i2c_en 变为 0 时, 该中断被清除。	0	R	
0	RX_UNDER	设置处理器是否在接收缓冲区为空时通过读取 I2C_DATA_CMD 寄存器尝试读取该缓冲区。 如果模块被禁用 (I2C_ENABLE[0]=0), 该位保持其电平直到主或从状态机进入空闲状态, 当 i2c_en 变为 0 时, 该中断被清除。	0	R	

25.2.12 寄存器 (I2C_RX_TL)

偏移地址: 0x38, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								RX_TL							
-								RW							

Bit	Field	Description	Reset value	RW attribute	Remark
31:8	Reserved				
7:0	RX_TL	接受 FIFO 阈值级别 控制触发 RX_FULL 中断 (I2C_RAW_INTR_STAT 寄存器中的第 2 位) 的条目 (或更高) 的级别。有效范围是 0-255, 附加限制是硬件不允许将此值设置为超过最大深度, 实际设置的值将是缓冲区的最大深度。值 0 设置 1 个条目的阈值, 值 255 设置 256 个条目的阈值。	0	RW	

25.2.13 寄存器 (I2C_TX_TL)

偏移地址: 0x3C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TX_TL							
-								RW							

Bit	Field	Description	Reset value	RW attribute	Remark
31:8	Reserved				
7:0	TX_TL	发送 FIFO 阈值级别 控制触发 TX_EMPTY 中断 (I2C_RAW_INTR_STAT 寄存器中的第 4 位) 的条目级别 (或更低级别)。有效范围是 0-255, 附加限制是它不能设置为大于缓冲区深度的值。如果尝试这样做, 实际设置的值将是缓冲区的最大深度。值 0 设置 0 个条目的阈值, 值 255 设置 255 个条目的阈值。	0	RW	

25.2.14 寄存器 (I2C_CLR_INTR)

偏移地址: 0x40, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_INTR	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_INTR	读取该寄存器以清除组合中断、所有单个中断和 I2C_TX_ABORT_SOURCE 寄存器。该位不会清除硬件可清除中断, 而是软件可清除中断。有关清除 I2C_TX_ABORT_SOURCE 的例外情况, 请参阅 I2C_TX_ABORT_SOURCE 寄存器的第 9 位。	0	R	

25.2.15 寄存器 (I2C_CLR_RX_UNDER)

偏移地址: 0x44, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_RX_UNDER	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_RX_UNDER	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 RX_UNDER 中断 (位 0)。	0	R	

25.2.16 寄存器 (I2C_CLR_RX_OVER)

偏移地址: 0x48, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															CLR_RX_OVER
-															R

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_RX_OVER	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 RX_OVER 中断 (位 1)。	0	R	

25.2.17 寄存器 (I2C_CLR_TX_OVER)

偏移地址: 0x4C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															CLR_TX_OVER
-															R

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_TX_OVER	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 TX_OVER 中断 (位 3)。	0	R	

25.2.18 寄存器 (I2C_CLR_RD_REQ)

偏移地址: 0x50, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															CLR_RD_REQ
-															R

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_RD_REQ	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 RD_REQ 中断 (位 5)。	0	R	

25.2.19 寄存器 (I2C_CLR_TX_ABORT)

偏移地址: 0x54, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_TX_ABRT	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_TX_ABRT	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器和 I2C_TX_ABRT_SOURCE 寄存器的 TX_ABRT 中断 (位 6)。这也将 Tx FIFO 从刷新/复位状态释放, 允许对 Tx FIFO 进行更多写入。有关清除 I2C_TX_ABRT_SOURCE 的例外情况, 请参阅 I2C_TX_ABRT_SOURCE 寄存器的第 9 位。	0	R	

25.2.20 寄存器 (I2C_CLR_RX_DONE)

偏移地址: 0x58, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_RX_DONE	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_RX_DONE	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 RX_DONE 中断 (位 7)。	0	R	

25.2.21 寄存器 (I2C_CLR_ACTIVITY)

偏移地址: 0x5C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_ACTIVITY	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_ACTIVITY	如果 I2C 不再处于活动状态, 则读取该寄存器会清除 ACTIVITY 中断。如果 I2C 模块在总线上仍处于活动状态, 则 ACTIVITY 中断位将继续置位。如果模块被禁用并且存在总线上没有进一步的活动。读取该寄存器可以获取 I2C_RAW_INTR_STAT 寄存器的 ACTIVITY 中断 (位 8) 的状态。	0	R	

25.2.22 寄存器 (I2C_CLR_STOP_DET)

偏移地址: 0x60, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_STOP_DET	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_STOP_DET	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 STOP_DET 中断 (位 9)。	0	R	

25.2.23 寄存器 (I2C_CLR_START_DET)

偏移地址: 0x64, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_START_DET	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_START_DET	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 START_DET 中断 (位 10)。	0	R	

25.2.24 寄存器 (I2C_CLR_GEN_CALL)

偏移地址: 0x68, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_GEN_CALL	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_START_DET	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 GEN_CALL 中断 (位 11)。	0	R	

25.2.25 寄存器 (I2C_ENABLE)

偏移地址: 0x6C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													SMBUS_ALERT_EN	SMBUS_SUSPEND_EN	SMBUS_CLK_RESET
-													RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										SDA_STUCK_RECOVERY_ENABLE		TX_CMD_BLOCK	ABORT	ENABLE	
-										RW		RW	RW	RW	

Bit	Field	Description	Reset value	RW attribute	Remark
31:19	Reserved				
18	SMBUS_ALERT_EN	SMBUS_ALERT_CTRL 寄存器位用于控制 SMBALERT 信号的使能。 1: 置位 SMBALERT 信号 该寄存器位在检测到来自主机的对警报响应地址的确认后自动清零。			
17	SMBUS_SUSPEND_EN	SMBUS_SUSPEND_EN 寄存器位用于控制 SMBSUS 信号中止使能。 0: 取消 SMBSUS 信号 1: 置位 SMBSUS 信号			
16	SMBUS_CLK_RESET	该位用于 SMBus Host 模式以启动 SMBus 主时钟复位。 仅当 Master 处于空闲状态时才应启用该位。 每当该位启用时, SMBCLK 将在 I2C_SCL_STUCK_TIMEOUT i2c_clk 周期内保持低电平以复位 SMBus 从设备。			
15:4	Reserved				
3	SDA_STUCK_RECOVERY_ENABLE	如果 SDA 卡在通过 TX_ABORT 中断 (I2C_TX_ABORT_SOURCE[17]) 指示的低电平, 则该位用作控制旋钮以启动 SDA 恢复机制 (即, 最多发送 9 个 SCL 时钟和 STOP 以释放 SDA 线) 然后这个位会自动清除。			
2	TX_CMD_BLOCK	在主模式 1: 即使 Tx FIFO 有数据要传输, 也阻止 I2C 总线上的数据传输。 0: 一旦 Tx FIFO 中的第一个数据可用, I2C 总线上的数据传输将自动开始。	0		一直为 0, 不可修改
1	ABORT	设置后, 控制器启动传输中止。 0: ABORT 未启动或 ABORT 已完成 1: ABORT 操作正在进行中			

Bit	Field	Description	Reset value	RW attribute	Remark
0	ENABLE	控制是否启用设备。 0: 禁用设备 (TX 和 RX FIFO 保持在擦除状态) 1: 启用设备软件可以在设备处于活动状态时禁用它。但是, 务必注意确保正确禁用设备。第 62 页的“禁用设备”中描述了推荐的过程。 当设备被禁用时, 会发生以下情况: TX FIFO 和 RX FIFO 被刷新。 I2C_INTR_STAT 寄存器中的状态位仍然有效, 直到设备进入空闲状态。如果模块正在发送, 它会在当前传输完成后停止并删除发送缓冲区的内容。如果模块正在接收, 则设备在当前字节结束时停止当前传输并且不确认传输。在具有异步 pclk 和 i2c_clk 的系统中, 当 I2C_CLK_TYPE 参数设置为异步 (1) 时, 启用或禁用设备时会有两个 i2c_clk 延迟。有关如何进行的详细说明			

25.2.26 寄存器 (I2C_STATUS)

偏移地址: 0x70, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SMBUS_ALERT_STATUS	SMBUS_SUSPEND_STATUS	SMBUS_SLAVE_ADDR_RESOLVED	SMBUS_SLAVE_ADDR_VALID	SMBUS_QUICK_CMD_BIT
-											R	R	R	R	R
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				SDA_STUCK_NOT_RECOVERED	SLV_HOLD_RX_FIFO_FULL	SLV_HOLD_TX_FIFO_EMPTY	MST_HOLD_ERX_FIFO_FULL	MST_HOLD_TX_FIFO_EMPTY	SLV_ACTIVITY	MST_ACTIVITY	RFF	RFNE	TFE	TFNF	ACTIVITY
-				R	R	R	R	R	R	R	R	R	R	R	R

Bit	Field	Description	Reset value	RW attribute	Remark
31:21	Reserved				
20	SMBUS_ALERT_STATUS	该位指示输入信号的状态是否为 i2c_smbus_alert_in_n。当 SMBus 设备声明 SMBus 警报信号时, 该信号被声明。	0	R	
19	SMBUS_SUSPEND_STATUS	该位指示输入信号的状态是否为 i2c_smbus_sus_in_n。当 SMBus 主机置位 SMBus 挂起信号时, 该信号置位。	0	R	
18	SMBUS_SLAVE_ADDR_RESOLVED	该位指示 SMBus Slave 地址 (i2c_sar[6:0]) 是否由 ARP Master 解析。	0	R	
17	SMBUS_SLAVE_ADDR_VALID	该位指示 SMBus 从地址 (i2c_sar[6:0]) 是否有效。	0	R	
16	SMBUS_QUICK_CMD_BIT	该位表示接收到的快速命令的 R/W 位。用户读取该位后, 该位将被清除。	0	R	
15:13	Reserved				
12	SAR_DUAL_FLAG	双地址标志	0	R	

Bit	Field	Description	Reset value	RW attribute	Remark
11	SDA_STUCK_NOT_RECOVERED	该位表示在恢复机制后未恢复为低电平的 SDA。	0	R	
10	SLV_HODE_RX_FIFO_FULL	由于 Rx FIFO 已满且接收到额外字节 (如果 I2C_RX_FULL_HLD_BUS_EN 设置为 1, 则这种总线保持适用), 该位指示从模式下的总线保持。	0	R	固定为 0
9	SLV_HOLD_TX_FIFO_EMPTY	当 Tx FIFO 为空时, 该位指示读取请求在从模式下的总线保持。总线处于保持状态, 直到 Tx FIFO 有数据要发送以用于读取请求。	0	R	固定为 0
8	MST_HODE_RX_FIFO_FULL	该位表示由于 Rx FIFO 已满且已接收额外字节而导致主模式下的总线保持	0	R	固定为 0
7	MST_HOLD_TX_FIFO_EMPTY	如果 I2C_EMPTYFIFO_HOLD_MASTER_EN 参数设置为 1, 则设备主机在 Tx FIFO 为空时暂停写传输, 并且最后一个字节没有设置停止位。该位表示当主机因为 Tx FIFO 为空而保持总线时的 BUS 保持, 并且之前传输的命令没有设置停止位。	0	R	固定为 0
6	SLV_ACTIVITY	从 FSM 活动状态。当从设备有限状态机 (FSM) 不处于空闲状态时, 该位被设置。 0: Slave FSM 处于 IDLE 状态, 因此设备的 Slave 部分不是 Active 1: Slave FSM 不处于 IDLE 状态, 所以设备的 Slave 部分是 Active	0	R	
5	MST_ACTIVITY	掌握 FSM 活动状态。当主有限状态机 (FSM) 不处于空闲状态时, 该位被设置。 0: Master FSM 处于 IDLE 状态, 所以设备的 Master 部分不是 Active 1: Master FSM 不在 IDLE 状态, 所以设备的 Master 部分是 Active	0	R	
4	RFF	接收 FIFO 完全满。当接收 FIFO 完全满时, 该位被设置。当接收 FIFO 包含一个或多个空单元时, 该位清零。 0: 接收 FIFO 未滿 1: 接收 FIFO 已滿	0	R	
3	RFNE	接收 FIFO 非空。当接收 FIFO 包含一个或多个条目时设置该位; 当接收 FIFO 为空时清零。 0: 接收 FIFO 为空 1: 接收 FIFO 不为空	0	R	
2	TFE	发送 FIFO 完全为空。当发送 FIFO 完全为空时, 该位被设置。当它包含一个或多个有效条目时, 该位被清除。该位域不请求中断。 0: 发送 FIFO 不为空 1: 发送 FIFO 为空	1	R	
1	TFNF	发送 FIFO 未滿。当发送 FIFO 包含一个或多个空单元时置位, 当 FIFO 已滿时清零。 0: 发送 FIFO 已滿 1: 发送 FIFO 未滿	1	R	
0	ACTIVITY	I2C 活动状态。	0	R	

25.2.27 寄存器 (I2C_TXFLR)

偏移地址: 0x74, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													TXFLR		
													R		

Bit	Field	Description	Reset value	RW attribute	Remark
31:3	Reserved				
2:0	TXFLR	发送 FIFO 级别。 包含发送 FIFO 中有效数据条目的数量。	0	R	

25.2.28 寄存器 (I2C_RXFLR)

偏移地址: 0x78, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												RXFLR			
-												R			

Bit	Field	Description	Reset value	RW attribute	Remark
31:3	Reserved				
2:0	RXFLR	接收 FIFO 级别。包含接收 FIFO 中有效数据条目的数量。	0	R	

25.2.29 寄存器 (I2C_SDA_HOLD)

偏移地址: 0x7C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								I2C_SDA_RX_HOLD							
-								RW							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_TX_HOLD															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:24	Reserved				
23:16	I2C_SDA_RX_HOLD	当设备作为接收器时, 以 i2c_clk 周期为单位设置所需的 SDA 保持时间。	0	RW	
15:0	I2C_SDA_TX_HOLD	当设备作为发送器时, 以 i2c_clk 周期为单位设置所需的 SDA 保持时间。	0	RW	

25.2.30 寄存器 (I2C_TX_ABRT_SOURCE)

偏移地址: 0x80, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TX_FLUSH_CNT									Reserved		ABRT_D EVICE WRITE	ABRT_DE VIC E SLV ADDR_NOA CK	ABRT_D E VICEN OACK	ABRT_SD A_STU CK_A T_LOW	ABRT_USER _ABRT
R									-		R	R	R	R	R
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRT_S LVRD_I NTX	ABRT_S LV_ARB LOST	ABRT_SL VFLUSH TXFIFO	ARB_L OST	ABRT_MASTE R_DIS	T_10B_R D_NORST RT	ABRT_SB YTE_NO RSTRT	Reserve d	ABRT_S BYTE_A CKDET	Reserve d	ABRT_G CALL_R EAD	ABRT_G CALL_N OACK	ABRT_TXD ATA_NOA CK	ABRT_10 ADDR2 NOACK	ABRT_10A DDR1_NO ACK	ABRT_7B_AD DR_NOACK
R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R

Bit	Field	Description	Reset value	RW attribute	Remark
25:23	TX_FLUSH_CNT	该字段指示由于 TX_ABRT 中断而刷新的 Tx FIFO 数据命令的数量。每当 I2C 被禁用时它就会被清除	0	R	
22:18	Reserved				
17	ABRT_SDA_STUCK_AT_LOW	这是仅主模式位。对于 i2c_clks 的 I2C_SDA_STUCK_AT_LOW_TIMEOUT 值, 主机检测到 SDA 处于低电平。	0	R	

Bit	Field	Description	Reset value	RW attribute	Remark
16	ABRT_USER_ABRT	这是仅主模式位。主机检测到传输中止 (I2C_ENABLE[1])。	0	R	
15	ABRT_SLVRD_INTX	1: 当处理器侧响应从模式请求将数据传输到远程主机并且用户在 I2C_DATA_CMD 寄存器的 CMD (位 8) 中写入 1 时。	0	R	
14	ABRT_SLV_ARBLOST	1: 在向远程主机传输数据时, 从机丢失了总线。 I2C_TX_ABRT_SOURCE[12] 同时置位。	0	R	
13	ABRT_SLVFLUSH_TXFIFO	1: Slave 收到了一个读命令并且一些数据存在于 TX FIFO 中, 所以 Slave 发出一个 TX_ABRT 中断来刷新 TX FIFO 中的旧数据。	0	R	
12	ARB_LOST	1: Master 已失去仲裁, 或者如果 C_TX_ABRT_SOURCE[14] 也置位, 则从发送器已失去仲裁。	0	R	
11	ABRT_MASTER_DIS	1: 用户尝试在禁用主模式的情况下启动主操作。	0	R	
10	ABRT_10B_RD_NORSTRT	1: 禁止重启 (I2C_RESTART_EN 位 (I2C_CON[5]) = 0), 主机在 10 位寻址模式下发送读命令。	0	R	
9	ABRT_SBYTE_NORSTRT	要清除第 9 位, 必须首先修复 ABRT_SBYTE_NORSTRT 的来源; 必须使能重启 (I2C_CON[5]=1), 必须清除 SPECIAL 位 (I2C_TAR[11]), 或者必须清除 GC_OR_START 位 (I2C_TAR[10])。一旦 ABRT_SBYTE_NORSTRT 的来源被固定, 那么该位可以与该寄存器中的其他位相同的方式被清除。 如果在尝试清除该位之前 ABRT_SBYTE_NORSTRT 的源未固定, 则位 9 清除一个周期, 然后重新置位。 1: 重启被禁用 (I2C_RESTART_EN 位 (I2C_CON[5]) = 0) 并且用户正在尝试发送一个 START 字节。	0	R	
8	Reserved		0	R	
7	ABRT_SBYTE_ACKDET	1: 主机发送了一个 START Byte 并且 START Byte 被确认 (错误行为)。	0	R	
6	Reserved		0	R	
5	ABRT_GCALL_READ	1: 主模式下的设备发送了一个广播, 但用户将广播之后的字节编程为从总线读取 (I2C_DATA_CMD[9] 设置为 1)。	0	R	
4	ABRT_GCALL_NOACK	1: 主模式下的设备发送了一个广播, 总线上没有从机确认广播。	0	R	
3	ABRT_TXDATA_NOACK	1: 这是仅主模式位。 主机已收到地址确认, 但当它发送地址后的数据字节时, 它没有收到来自远程从机的确认。	0	R	
2	ABRT_10ADDR2_NOACK	1: 主机处于 10 位地址模式, 10 位地址的第二个地址字节未被任何从机确认。	0	R	
1	ABRT_10ADDR1_NOACK	1: 主机处于 10 位地址模式且第一个 10 位地址字节未被任何从机确认。	0	R	
0	ABRT_7B_ADDR_NOACK	1: 主机处于 7 位寻址模式, 发送的地址未被任何从机确认。	0	R	

25.2.31 寄存器 (I2C_DMA_CR)

偏移地址: 0x88, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													TDMAE		RDMAE
-													RW		RW

Bit	Field	Description	Reset value	RW attribute	Remark
31:2	Reserved				
1	TDMAE	发送 DMA 使能。该位启用/禁用发送 FIFO DMA 通道。 0 = 发送 DMA 禁用 1 = 发送 DMA 使能	0	RW	
0	RDMAE	接收 DMA 使能。该位启用/禁用接收 FIFO DMA 通道。 0 = 接收 DMA 禁用 1 = 接收 DMA 使能	0	RW	

25.2.32 寄存器 (I2C_DMA_TDLR)

偏移地址: 0x8C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													DMATDL		
-													RW		

Bit	Field	Description	Reset value	RW attribute	Remark
31:3	Reserved				
2:0	DMATDL	传输数据级别。该位字段控制发送逻辑发出 DMA 请求的级别。它等于所示级别; 即当发送 FIFO 中的有效数据条目数等于或低于该字段值且 TDMAE = 1 时生成 dma_tx_req 信号。	0	RW	

25.2.33 寄存器 (I2C_DMA_RDLR)

偏移地址: 0x90, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													DMARDL		
-													RW		

Bit	Field	Description	Reset value	RW attribute	Remark
31:3	Reserved				
2:0	DMARDL	接收数据电平。该位字段控制接收逻辑发出 DMA 请求的级别。水印等级=DMARDL+1; 即当接收 FIFO 中的有效数据条目数等于或大于该字段值+1 且 RDMAE = 1 时生成 dma_rx_req。例如, 当 DMARDL 为 0 时, 则 dma_rx_req 在 1 或更多时被置位。数据条目存在于接收 FIFO 中。目前只支持小于等于 3 的值, 硬件限制。	0	RW	

25.2.34 寄存器 (I2C_SDA_SETUP)

偏移地址: 0x94, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								SDA_SETUP							
-								RW							

Bit	Field	Description	Reset value	RW attribute	Remark
31:8	Reserved				
7:0	SDA_SETUP	SDA 设置。建议如果所需延迟为 1000ns, 那么对于 10 MHz 的 I2C_clk 频率, I2C_SDA_SETUP 应编程为值 11。I2C_SDA_SETUP 必须编程为最小值 2。	0x64	RW	

25.2.35 寄存器 (I2C_ACK_GENERAL_CALL)

偏移地址: 0x98, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														ACK_GEN_CALL	
-														RW	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	ACK_GEN_CALL	ACK 广播呼叫。当设置为 1 时, 设备在收到广播呼叫时以 ACK 响应 (通过声明 I2C_data_oe)。当设置为 0 时, 设备不会产生广播中断。	1	RW	

25.2.36 寄存器 (I2C_ENABLE_STATUS)

偏移地址: 0x9C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												SLV_RX_DATA_LOST		SLV_DISABLED_WHILE_BUSY	
-												R		R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:3	Reserved				
2	SLV_RX_DATA_LOST	从设备接收数据丢失。该位指示是否由于将 I2C_ENABLE[0] 从 1 设置为 0, 从 I2C 传输接收到至少一个数据字节时从接收器操作被中止。当读为 1 时, 设备被视为已积极参与已中止 I2C 传输 (具有匹配地址) 且已进入 I2C 传输的数据阶段, 即使已使用 NACK 响应数据字节。			

Bit	Field	Description	Reset value	RW attribute	Remark
1	SLV_DISABLED_WHILE_BUSY	从机忙时禁用 (发送、接收)。该位指示是否由于将 I2C_ENABLE 寄存器的位 0 从 1 设置为 0 而中止了潜在的或活动的 Slave 操作。当 CPU 将 0 写入 I2C_ENABLE 的位 0 时, 该位被设置, 同时: (a)设备正在接收来自远程主站的从站发送器操作的地址字节; OR, (b) 来自远程主机的从机-接收器操作的地址和数据字节。当读为 1 时, 无论 I2C 地址是否与设备 (I2C_SAR 寄存器) 中设置的从地址匹配, 或者是否在 I2C_ENABLE 的位 0 为 设置为 0, 但没有生效。			
0	I2C_EN	I2c_en 状态。该位始终反映在输出端口 i2c_en 上驱动的值。 当读为 1 时, 设备被视为处于启用状态。 当读为 0 时, 设备被视为完全不活动。			

25.2.37 寄存器 (I2C_FS_SPKLEN)

偏移地址: 0xA0, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								I2C_FS_SPKLEN							
								RW							

Bit	Field	Description	Reset value	RW attribute	Remark
31:8	Reserved	-	-	-	
7:0	I2C_FS_SPKLEN	该寄存器必须在任何 I2C 总线事务发生之前设置, 以确保稳定运行。该寄存器设置 SCL 或 SDA 线路中被尖峰抑制逻辑过滤掉的最长尖峰的持续时间, 以 i2c_clk 周期为单位; 有关更多信息, 请参阅第 64 页上的“尖峰抑制”。仅当 I2C 接口禁用时才能写入该寄存器, 这对应于 I2C_ENABLE[0] 设置为 0。其他时间写入无效。最小有效值为 1; 硬件会阻止写入小于此值的值, 如果尝试, 则会导致设置为 1。	1	RW	

25.2.38 寄存器 (I2C_SCL_STUCK_AT_LOW_TIMEOUT)

偏移地址: 0xAC, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
I2C_SCL_STUCK_AT_LOW_TIMEOUT															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_SCL_STUCK_AT_LOW_TIMEOUT															
RW															

Bit	Field	Description	Reset value	RW attribute
31:0	I2C_SCL_STUCK_AT_LOW_TIMEOUT	如果设备在 I2C_SCL_STUCK_AT_LOW_TIMEOUT 以 i2c_clk 周期为单位检测到 SCL 卡在低电平, 则设备会生成中断以指示 SCL 卡在低电平。	0xFFFFFFFF	RW

25.2.39 寄存器 (I2C_SDA_STUCK_AT_LOW_TIMEOUT)

偏移地址: 0xB0, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
I2C_SDA_STUCK_AT_LOW_TIMEOUT															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_SDA_STUCK_AT_LOW_TIMEOUT															
RW															

Bit	Field	Description	Reset value	RW attribute
31:0	I2C_SDA_STUCK_AT_LOW_TIMEOUT	设备通过使能 SDA_STUCK_RECOVERY_EN (I2C_ENABLE[3]) 寄存器位来启动 SDA 线的恢复, 如果它检测到 SDA 以 i2c_clk 周期为单位在 I2C_SDA_STUCK_AT_LOW_TIMEOUT 内卡在低电平。	0xFFFFFFFF	RW

25.2.40 寄存器 (I2C_CLR_SCL_STUCK_DET)

偏移地址: 0xb4, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														CLR_SCL_STUCK	
-														R	

Bit	Field	Description	Reset value	RW attribute	Remark
31:1	Reserved				
0	CLR_SCL_STUCK	读取该寄存器以清除 I2C_RAW_INTR_STAT 寄存器的 SCL_STUCK_DET 中断 (位 14)。	0	RW	

25.2.41 寄存器 (I2C_SMBUS_CLK_LOW_SEXT_TIMEOUT)

偏移地址: 0xBC, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SMBUS_CLK_LOW_SEXT_TIMEOUT															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMBUS_CLK_LOW_SEXT_TIMEOUT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:0	SMBUS_CLK_LOW_SEXT_TIMEOUT	该字段用于检测从设备在从初始 START 到 STOP 的一条消息中扩展的主模式下的从时钟扩展超时 (tLOW:SEXT)。该寄存器中的值以 I2C_clk 周期为单位。	0xFFFFFFFF	RW	

25.2.42 寄存器 (I2C_SMBUS_CLK_LOW_MEXT_TIMEOUT)

偏移地址: 0xc0, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SMBUS_CLK_LOW_MEXT_TIMEOUT															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMBUS_CLK_LOW_MEXT_TIMEOUT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:0	SMBUS_CLK_LOW_MEXT_TIMEOUT	该字段用于检测主机模式下从 START-to-ACK、ACK-to-ACK 或 ACK-to-STOP 定义的主机扩展 SMBus 时钟 (SCL) 超时。该寄存器中的值以 i2c_clk 周期为单位。	0xFFFFFFFF	RW	

25.2.43 寄存器 (I2C_SMBUS_THIGH_MAX_IDLE_COUNT)

偏移地址: 0xc4, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMBUS_THIGH_MAX_IDLE_COUNT															
RW															

Bit	Field	Description	Reset value	RW attribute	Remark
31:16	Reserved				
15:0	SMBUS_THIGH_MAX_IDLE_COUNT	此字段用于设置当主设备已动态添加到总线并且可能未检测到 SMBCLK 或 SMBDAT 上的状态转换时使用的所需总线空闲时间段线。在这种情况下, 主设备必须等待以确保当前没有进行传输。该寄存器中的值以 i2c_clk 周期为单位。	0xFFFF	RW	

25.2.44 寄存器 (I2C_SMBUS_INTR_STAT)

偏移地址: 0xC8, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					R_SMBUS_ALERT_DET	R_SMBUS_SUSPEND_DET	R_SLV_RX_PEC_NACK	R_ARP_ASSGN_ADDR_CMD_DET	R_ARP_GET_UDID_CMD_DET	R_ARP_RST_CMD_DET	R_ARP_PREPARE_CMD_DET	R_HOST_NOTIFY_MST_DET	R_QUICK_CMD_DET	R_MST_CLOCK_EXTND_TIMEOUT	R_SLV_CLOCK_EXTND_TIMEOUT
-					R	R	R	R	R	R	R	R	R	R	R

Bit	Field	Description	Reset value	RW attribute
31:11	Reserved			
10	R_SMBUS_ALERT_DET	指示 SMBALERT (i2c_smbalert_in_n) 信号是否由从设备驱动为低电平。	0	R
9	R_SMBUS_SUSPEND_DET	指示主机是否将 SMBSUS (i2c_smbsus_in_n) 信号驱动为低电平。	0	R
8	R_SLV_RX_PEC_NACK	指示从站是否为从站发送的 ARP 命令的 PEC 字节生成 NACK。	0	R
7	R_ARP_ASSGN_ADDR_CMD_DET	指示是否已收到分配地址 ARP 命令。	0	R
6	R_ARP_GET_UDID_CMD_DET	指示是否已收到通用或定向的 Get UDID ARP 命令。	0	R
5	R_ARP_RST_CMD_DET	指示是否已收到通用或定向复位 ARP 命令。	0	R
4	R_ARP_PREPARE_CMD_DET	指示是否已收到准备 ARP 命令。	0	R
3	R_HOST_NOTIFY_MST_DET	指示是否已收到主机通知命令。	0	R
2	R_QUICK_CMD_DET	指示无论设备是在从机模式还是主机模式下运行, SMBus 接口是否已接收到快速命令。该位仅在 I2C_SMBUS=1 设置为 1 时启用。	0	R

Bit	Field	Description	Reset value	RW attribute
1	R_MST_CLOCK_EXTND_TIMEOUT	指示从 START 到 STOP 的 Master 设备事务 (START-to-ACK、ACK-to-ACK 或 ACK-to-STOP) 是否超过 I2C_SMBUS_CLOCK_LOW_MEXT 时间在消息的每个字节中。该位仅在以下情况下启用: I2C_SMBUS=1 I2C_CON[0]=1 I2C_EMPTYFIFO_HOLD_MASTER_EN=1 或 I2C_RX_FULL_HLD_BUS_EN=1	0	R
0	R_SLV_CLOCK_EXTND_TIMEOUT	指示从 Slave (即从 START 到 STOP) 的事务是否超过 I2C_SMBUS_CLOCK_LOW_SEXT 时间。该位仅在以下情况下启用 I2C_SMBUS=1 I2C_CON[0]=1	0	R

25.2.45 寄存器 (I2C_SMBUS_INTR_MASK)

偏移地址: 0xCC, 复位值: 0x0000 3FFF。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					M_SMBUS_ALERT_DET	M_SMBUS_SUSPEND_DET	M_SLV_RX_PEC_NACK	M_ARP_ASSGN_ADDR_CMD_DET	M_ARP_GET_UDID_CMD_DET	M_ARP_RST_CMD_DET	M_ARP_EPARE_CMD_DET	M_HOST_NOTIFY_MST_DET	M_QUICK_CMD_DET	M_MST_CLOCK_EXTND_TIMEOUT	M_SLV_CLOCK_EXTND_TIMEOUT
-					RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

Bit	Field	Description	Reset value	RW attribute	Remark
31:11	Reserved				
10	M_SMBUS_ALERT_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_SMBUS_ALERT_DET 中断位。该位仅在以下情况下启用 I2C_SMBUS_SUSPEND_ALERT=1。	1	RW	
9	M_SMBUS_SUSPEND_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_SMBUS_SUSPEND_DET 中断位。该位仅在 I2C_SMBUS_SUSPEND_ALERT=1 时启用。	1	RW	
8	M_SLV_RX_PEC_NACK	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_SLV_RX_PEC_NACK 中断位。该位仅在 I2C_SMBUS_ARP=1 时启用。	1	RW	
7	M_ARP_ASSGN_ADDR_CMD_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_ARP_ASSGN_ADDR_CMD_DET 中断位。该位仅在 I2C_SMBUS_ARP=1 时启用。	1	RW	
6	M_ARP_GET_UDID_CMD_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_ARP_GET_UDID_CMD_DET 中断位。该位仅在 I2C_SMBUS_ARP=1 时启用。	1	RW	

Bit	Field	Description	Reset value	RW attribute	Remark
5	M_ARP_RST_CMD_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_ARP_RST_CMD_DET 中断位。该位仅在 I2C_SMBUS_ARP=1 时启用。	1	RW	
4	M_ARP_PREPARE_CMD_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_ARP_PREPARE_CMD_DET 中断位。该位仅在 I2C_SMBUS_ARP=1 时启用。	1	RW	
3	M_HOST_NOTIFY_MST_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_HOST_NOTIFY_DET 中断位。该位仅在 I2C_SMBUS=1 时启用。	1	RW	
2	M_QUICK_CMD_DET	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_QUICK_CMD_DET 中断位。该位仅在 I2C_SMBUS=1 时启用。	1	RW	
1	M_MST_CLOCK_EXTND_TIMEOUT	该位屏蔽了 I2C_SMBUS_INTR_STAT 寄存器中的 R_MST_CLOCK_EXTND_TIMEOUT 中断位。该位仅在 I2C_SMBUS=1 时启用。	1	RW	
0	M_SLV_CLOCK_EXTND_TIMEOUT	该位屏蔽 I2C_SMBUS_INTR_STAT 寄存器中的 R_SLV_CLOCK_EXTND_TIMEOUT 中断位。该位仅在 I2C_SMBUS=1 时启用。	1	RW	

25.2.46 寄存器 (I2C_SMBUS_INTR_RAW_STATUS)

偏移地址: 0Xd0, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					SMBUS_ALERT_DET	SMBUS_SUSPEND_DET	SLV_RX_PEC_NACK	ARP_ASSGN_ADDR_CMD_DET	ARP_GET_UDID_CMD_DET	ARP_RST_CMD_DET	ARP_PREPARE_CMD_DET	HOST_NOTIFY_MST_DET	QUICK_CMD_DET	MST_CLOCK_EXTND_TIMEOUT	SLV_CLOCK_EXTND_TIMEOUT
-					R	R	R	R	R	R	R	R	R	R	R

Bit	Field	Description	Reset value	RW attribute	Remark
31:11	Reserved	-	-	-	
10	SMBUS_ALERT_DET	指示 SMBALERT (i2c_smbalert_in_n) 信号是否由从设备驱动为低电平。	0	R	
9	SMBUS_SUSPEND_DET	指示主机是否将 SMBSUS (i2c_smbsus_in_n) 信号驱动为低电平。	0	R	
8	SLV_RX_PEC_NACK	指示从站是否为从站发送的 ARP 命令的 PEC 字节生成 NACK。	0	R	
7	ARP_ASSGN_ADDR_CMD_DET	指示是否已收到分配地址 ARP 命令。	0	R	
6	ARP_GET_UDID_CMD_DET	指示是否已接收到通用或定向的 Get UDID ARP 命令。	0	R	
5	ARP_RST_CMD_DET	指示是否已收到通用或定向复位 ARP 命令。	0	R	
4	ARP_PREPARE_CMD_DET	指示是否已收到准备 ARP 命令。	0	R	
3	HOST_NOTIFY_MST_DET	指示是否已收到主机通知命令。	0	R	

Bit	Field	Description	Reset value	RW attribute	Remark
2	QUICK_CMD_DET	指示无论设备是在从机模式还是主机模式下运行, SMBus 接口是否已接收到快速命令。该位仅在 I2C_SMBUS=1 设置为 1 时启用。	0	R	
1	MST_CLOCK_EXTND_TIMEOUT	指示从 START 到 STOP 的 Master 设备事务 (START-to-ACK、ACK-to-ACK 或 ACK-to-STOP) 是否超过消息的每个字节中的 I2C_SMBUS_CLOCK_LOW_MEXT 时间。该位仅在以下情况下启用: I2C_SMBUS=1 I2C_CON[0]=1 I2C_EMPTYFIFO_HOLD_MASTER_EN=1 或 I2C_RX_FULL_HLD_BUS_EN=1	0	R	
0	SLV_CLOCK_EXTND_TIMEOUT	指示从 Slave (即从 START 到 STOP) 的事务是否超过 I2C_SMBUS_CLOCK_LOW_SEXT 时间。该位仅在以下情况下启用 I2C_SMBUS=1 I2C_CON[0]=1	0	R	

25.2.47 寄存器 (I2C_CLR_SMBUS_INTR)

偏移地址: 0Xd4, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					CLR_SMBUS_ALERT_DET	CLR_SMBUS_SUSPEND_DET	CLR_SLV_RX_PEC_NACK	CLR_ARP_ASSGN_ADDR_CMD_DET	CLR_ARP_GET_UDID_CMD_DET	CLR_ARP_RST_CMD_DET	CLR_ARP_PREPARE_CMD_DET	CLR_HOST_NOTIFY_MST_DET	CLR_QUICK_CMD_DET	CLR_MST_CLOCK_EXTND_TIMEOUT	CLR_SLV_CLOCK_EXTND_TIMEOUT
-					W	W	W	W	W	W	W	W	W	W	W

Bit	Field	Description	Reset value	RW attribute	Remark
31:11	Reserved				
10	CLR_SMBUS_ALERT_DET	写入该寄存器以清除 I2C_SMBUS_RAW_INTR_STAT 寄存器的 SMBUS_ALERT_DET 中断 (位 10)。	0	W	
9	CLR_SMBUS_SUSPEND_DET	写入该寄存器以清除 R_SMBUS_SUSPEND_DET I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 9)。	0	W	
8	CLR_SLV_RX_PEC_NACK	写入该寄存器以清除 I2C_SMBUS_RAW_INTR_STAT 寄存器的 SLV_RX_PEC_NACK 中断 (位 8)。	0	W	
7	CLR_ARP_ASSGN_ADDR_CMD_DET	写这个寄存器清除 ARP_ASSGN_ADDR_CMD_DET I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 7)。	0	W	
6	CLR_ARP_GET_UDID_CMD_DET	写这个寄存器清除 ARP_GET_UDID_CMD_DET I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 6)。	0	W	

Bit	Field	Description	Reset value	RW attribute	Remark
5	CLR_ARP_RST_CMD_DET	写入该寄存器以清除 I2C_SMBUS_RAW_INTR_STAT 寄存器的 ARP_RST_CMD_DET 中断 (位 5)。	0	W	
4	CLR_ARP_PREPARE_CMD_DET	写这个寄存器清除 ARP_PREPARE_CMD_DET I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 4)。	0	W	
3	CLR_HOST_NOTIFY_MST_DET	写入该寄存器以清除 HOST_NOTIFY_MST_DET I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 3)。	0	W	
2	CLR_QUICK_CMD_DET	写入该寄存器以清除 I2C_SMBUS_RAW_INTR_STAT 寄存器的 QUICK_CMD_DET 中断 (位 2)。	0	W	
1	CLR_MST_CLOCK_EXTND_TIMEOUT	写入该寄存器以清除 MST_CLOCK_EXTND_TIMEOUT I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 1)。	0	W	
0	CLR_SLV_CLOCK_EXTND_TIMEOUT	写入该寄存器以清除 SLV_CLOCK_EXTND_TIMEOUT I2C_SMBUS_RAW_INTR_STAT 寄存器的中断 (位 0)。	0	W	

25.2.48 寄存器 (I2C_SAR_OPT)

偏移地址: 0xd8, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								Sar_opt							
-															

Bit	Field	Description	Reset value	RW attribute	Remark
31:8	Reserved				
7:0	Sar_opt	可选的从设备地址	0	RW	

25.2.49 寄存器 (I2C_SMBUS_UDID)

偏移地址: 0xdc, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SMBUS_UDID															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMBUS_UDID															
-															

Bit	Field	Description	Reset value	RW attribute	Remark
31:0	SMBUS_UDID	SMBUS ARP UDID LSB	32'hffffff	RW	

26 串行外设接口 (SPI)

26.1 串行外设接口 (SPI)

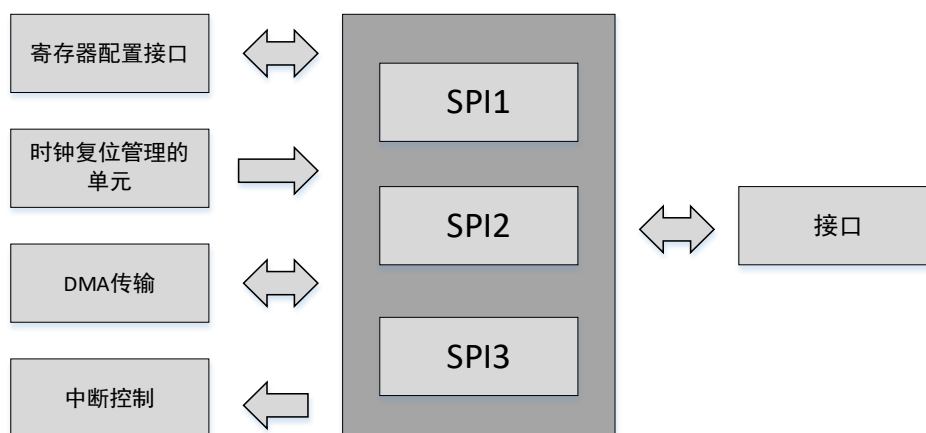
26.1.1 功能描述

SPI 接口支持 SPI 协议。串行外围设备接口 (SPI) 允许与外部设备进行半/全双工，同步，串行通信。该接口可以配置为主设备，在这种情况下，它向外部从设备提供通信时钟 (SCK)。该接口还可以在多主机配置中运行，可以用于多种目的，包括在两根线上进行单工同步传输以及可能的双向传输，或者使用 CRC 检查进行可靠的通信。

SPI 功能：

- 三线全双工同步传输
- 在具有或不具有双向数据线的两条线上进行单工同步传输
- 8 位或 16 位传输帧格式选择
- 主从操作
- 多主机模式功能
- 8 个主模式波特率预分频器 (最大 $f_{PCLK}/2$)
- 从模式频率 (最大 $f_{PCLK}/2$)
- 主从机通信更快
- 通过主机和从机的硬件或软件对 NSS 进行管理：主/从操作的动态变化
- 可编程的时钟极性和相位
- MSB 优先或 LSB 优先移位的可编程数据顺序
- 具有中断功能的专用发送接收标志
- SPI 总线忙状态标志
- 硬件 CRC 功能，可实现可靠的通信：
 - CRC 值可以在发送模式下作为最后一个字节发送
 - 对最后收到的字节进行自动 CRC 错误检查
- 具有中断功能的主机模式故障，溢出和 CRC 错误标志
- 具有 DMA 功能的 1 字节发送和接收缓冲区：Tx 和 Rx 请求

图26.1 SPI/ I2S 框图



通常，SPI 通过四个引脚连接到外部设备：

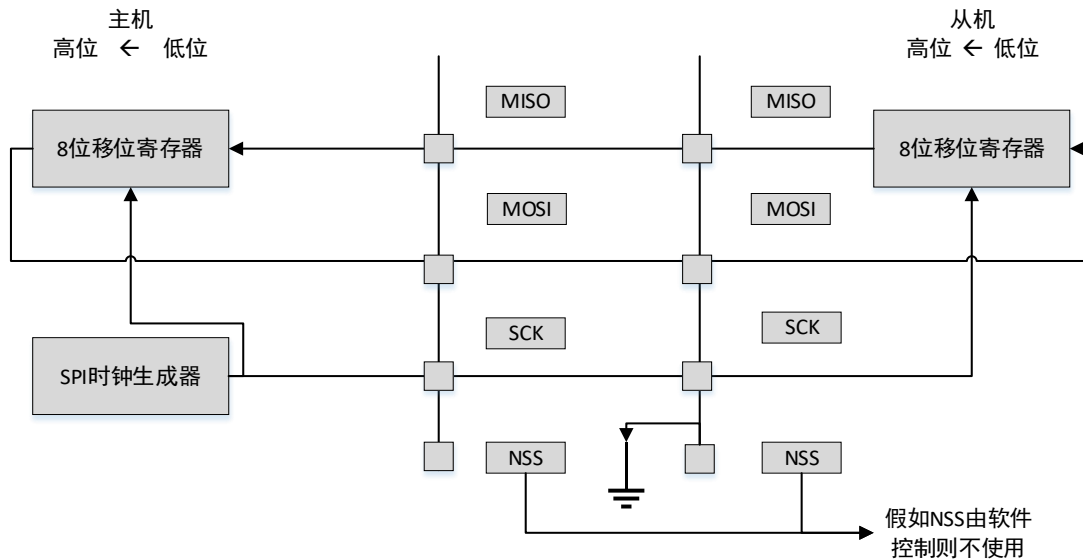
MISO：主输入/从输出数据。该引脚可用于在从机模式下发送数据并在主机模式下接收数据。

MOSI: 主输出/从输入数据。该引脚可用于在主机模式下发送数据和在从机模式下接收数据。

CK SCK: SPI 主设备的串行时钟输出和 SPI 从设备的输入。

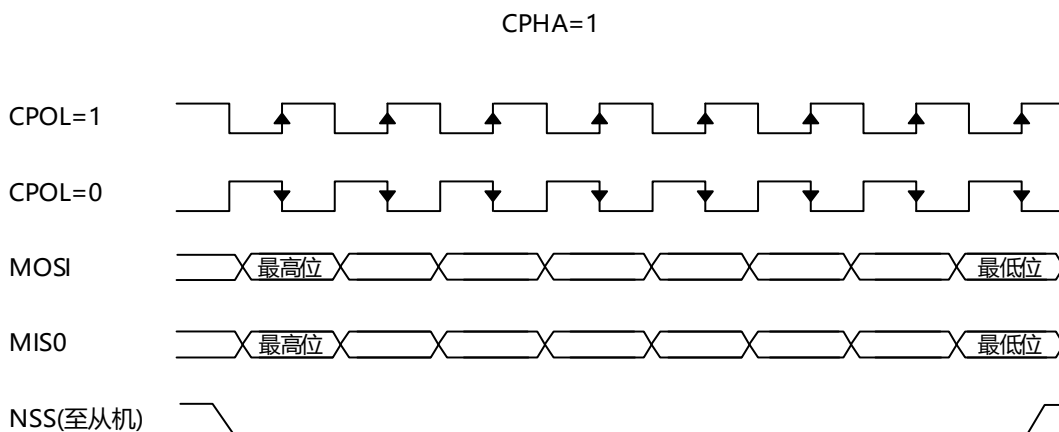
NSS: 从机选择，这是用于选择从设备的可选引脚。该引脚充当“片选”功能，以使 SPI 主机分别与从机通信，并避免争用数据线。从 NSS 输入可以由主设备上的标准 IO 端口驱动。如果使能 (SSOE 位)，则 NSS 引脚也可用作输出；如果 SPI 处于主配置，则将其驱动为低电平。通过这种方式，当将连接到主 NSS 引脚的设备上的所有 NSS 引脚配置为 NSS 硬件模式时，它们会变为低电平并成为从设备。当在主模式下配置 NSS 作为输入 (MSTR = 1 和 SSOE = 0)，并且如果 NSS 被拉低，则 SPI 进入主模式故障状态：MSTR 位自动清零，产品配置为从模式

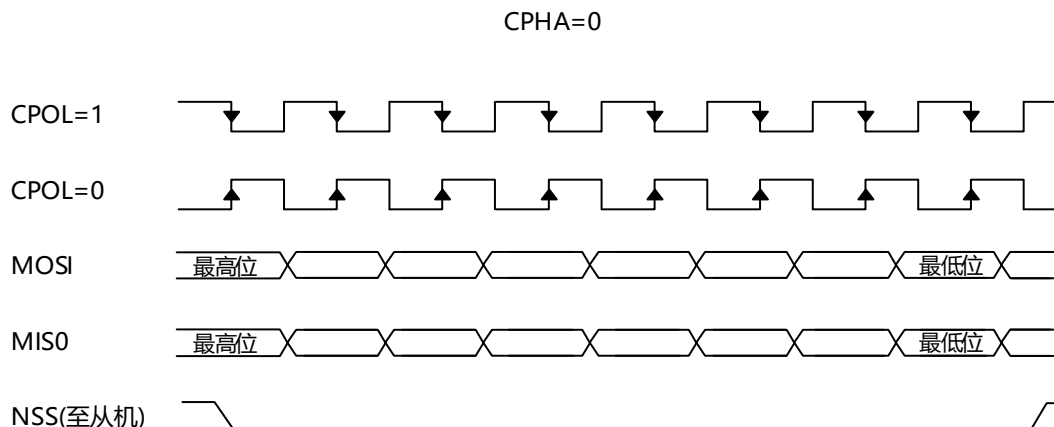
图26.2 单主/单从模式应用实例



这里 NSS 引脚被配置为输入模式。

图26.3 SPI 时序图





26.1.2 配置 SPI 从模式

在从模式下，串行时钟从主设备的 SCK 引脚上接收。SPI_CR1 寄存器的 BR [2: 0] 位中设置的值不会影响数据传输速率。建议在主机发送时钟之前使能 SPI 从机。否则，可能会发生不希望的数据传输。从站的数据寄存器需要在通信时钟的第一个边沿之前或正在进行通信结束之前准备好。必须将通信时钟的极性设置为启用从机和主机之前的稳态值。请按照以下过程在从机模式下配置 SPI:

- 设置 DFF 位以定义 8 或 16 位数据帧格式
- 选择 CPOL 和 CPHA 位以定义数据传输和串行时钟之间的四个关系之一。为了正确传输数据，必须在从设备和主设备中以相同的方式配置 CPOL 和 CPHA 位。
- 帧格式 (MSB 优先或 LSB 优先，取决于 SPI_CR1 寄存器中 LSBFIRST 位的值) 必须与主机设备相同。
- 在硬件模式下 (请参见从选择 (NSS) 引脚管理)，必须在完整的字节发送序列中将 NSS 引脚连接到低电平信号。在 NSS 软件模式下，将 SSM 位置 1 并将 SPI_CR1 寄存器中的 SSI 位清零。
- 将 MSTR 位清零并将 SPE 位 (都在 SPI_CR1 寄存器中) 置 1，以将引脚分配给备用功能。

在这种配置中，MOSI 引脚是数据输入，MISO 引脚是数据输出。

发送顺序：在写周期内，数据字节被并行加载到 Tx 缓冲区中。当从器件在其 MOSI 引脚上接收到时钟信号和数据的最高有效位时，发送序列开始，其余位 (8 位数据帧格式的 7 位和 16 位数据帧格式的 15 位) 被装入移位寄存器。在将数据从 Tx 缓冲区传输到移位寄存器时，将 SPI_SR 寄存器中的 TXE 标志置 1，并且如果将 SPI_CR2 寄存器中的 TXEIE 位置 1，则会产生中断。

接收顺序：对于接收器，完成数据传输后：

- 移位寄存器中的数据被传送到 Rx 缓冲区，并且 RXNE 标志 (SPI_SR 寄存器) 被置 1。
- 如果 SPI_CR2 寄存器中的 RXNEIE 位置 1，将产生一个中断。

在最后一个采样时钟沿后，将 RXNE 位置 1，然后将移位寄存器中接收到的数据字节的副本移至 Rx 缓冲区。当读取 SPI_DR 寄存器时，SPI 外设返回该缓冲值，通过读取 SPI_DR 寄存器来清除 RXNE 位。

26.1.3 配置 SPI 主模式

- 在主配置中，串行时钟在 SCK 引脚上生成。流程：
- 选择 BR [2: 0] 位以定义串行时钟波特率 (请参见 SPI_CR1 寄存器)。
- 选择 CPOL 和 CPHA 位以定义数据传输和串行时钟之间的四个关系之一。
- 将 DFF 位置 1 以定义 8 或 16 位数据帧格式

- 配置 SPI_CR1 寄存器中的 LSBFIRST 位以定义帧格式。
- 如果在输入模式下需要 NSS 引脚，则在硬件模式下，在完整的字节发送序列中将 NSS 引脚连接到高电平信号。在 NSS 软件模式下，将 SPI_CR1 寄存器中的 SSM 和 SSI 位置 1。如果在输出模式下需要 NSS 引脚，则仅应将 SSOE 位置 1。
- 必须将 MSTR 和 SPE 位置 1（仅当 NSS 引脚连接到高电平信号时，它们才保持设置状态）。

在这种配置中，MOSI 引脚为数据输出，而 MISO 引脚为数据输入。发送顺序：当在发送缓冲器中写入一个字节时，发送序列开始。在第一个数据字节被并行加载到移位寄存器中（从内部总线）

位传输，然后串行移出到 MOSI 引脚，首先是 MSB 或 LSB，这取决于 SPI_CR1 寄存器中的 LSBFIRST 位。TXE 标志在传输时置位从 Tx 缓冲区到移位寄存器的数据，如果 TXEIE 位为 1，则产生中断 SPI_CR2 寄存器被置 1。接收顺序对于接收器，完成数据传输后：

- 移位寄存器中的数据被传送到 RX 缓冲区并设置 RXNE 标志
- 如果 SPI_CR2 寄存器中的 RXNEIE 位置 1，将产生一个中断

在最后一个采样时钟沿，RXNE 位置 1，移位寄存器中接收到的数据字节的副本移至 Rx 缓冲区。当读取 SPI_DR 寄存器时，SPI 外设返回该缓冲值。通过读取 SPI_DR 寄存器来清除 RXNE 位。一旦开始传输，如果将要传输的下一个数据放入 Tx 缓冲区，则可以保持连续的传输流。在尝试写入 Tx 缓冲区之前，TXE 标志应为 1。当主机与 SPI 从机通信时，需要在传输之间取消选择时，必须将 NSS 引脚配置为 GPIO，或者必须使用另一个 GPIO 并通过软件进行切换。

26.1.4 配置 SPI 半双工

SPI 能够以 2 种配置在半双工模式下工作。

- 1 个时钟和 1 条双向数据线
- 1 条时钟和 1 条数据线（仅接收或仅发送）

1 条时钟和 1 条双向数据线（BIDIMODE = 1）：

通过将 SPI_CR1 寄存器中的 BIDIMODE 位置 1 使能该模式。在此模式下，SCK 用于时钟，主模式下的 MOSI 或从模式下的 MISO 用于数据通信。传输方向（输入/输出）由 SPI_CR1 寄存器中的 BIDIOE 位选择，当该位为 1 时，数据线为输出，否则为输入。

1 条时钟和 1 条单向数据线（BIDIMODE = 0）：

在这种模式下，应用程序可以在仅发送模式或仅接收模式下使用 SPI。

- 仅发送模式类似于全双工模式（BIDIMODE = 0，RXONLY = 0）：数据可以通过发送引脚（主机模式下的 MOSI 或从机模式下的 MISO）进行发送，而接收引脚（主机模式下的 MISO 或从机模式下的 MOSI）可以用作通用 IO。在这种情况下，应用程序只需要忽略 Rx 缓冲区（如果读取了数据寄存器，则它不包含接收到的值）。
- 在仅接收模式下，应用程序可以通过将 SPI_CR1 寄存器中的 RXONLY 位置 1 来禁用 SPI 输出功能。在这种情况下，它将释放发送 IO 引脚（主机模式下为 MOSI 或从机模式下为 MISO），因此可以将其用于其他目的。

要以仅接收模式开始通信，请配置并启用 SPI：

- 在主机模式下，当 SPE 位为 1 时，通信立即开始且当 SPE 位清除时停止，并且当前接收停止。在这种模式下，无需读取 BSY 标志。SPI 通信正在进行时，它总是置位。
- 在从机模式下，只要拉低 NSS（或在 NSS 软件模式下清零 SSI 位）并且 SCK 正在运行，SPI 就会继续接收。

26.1.5 数据发送和接受流程

接受和发送缓冲区

在接收时，先接收数据，然后将其存储到内部 Rx 缓冲区中；而在发送时，先将数据存储在内部 Tx 缓冲区中，然后再进行发送。对 SPI_DR 寄存器的读访问返回 Rx 缓冲的值，而对 SPI_DR 的写访问将写入的数据存储到 Tx 缓冲区中。

在主模式下启动流程：

在全双工模式 (BIDIMODE = 0 和 RXONLY = 0)

- 当数据写入 SPI_DR 寄存器 (Tx 缓冲区) 时，序列开始。
- 然后，在第一位传输过程中，将数据从 Tx 缓冲区并行加载到 8 位移位寄存器中，然后串行移出到 MOSI 引脚。
- 同时，在 MISO 引脚上接收的数据被串行移入到 8 位移位寄存器然后并行装入 SPI_DR 寄存器 (Rx 缓冲区)。

在单向仅接收模式下 (BIDIMODE = 0 和 RXONLY = 1)

- 序列从 SPE = 1 开始
- 仅激活接收器，并且将 MISO 引脚上的接收数据串行移入 8 位移位寄存器，然后并行装入 SPI_DR 寄存器 (Rx 缓冲区)。
- 在双向模式下，传输时 (BIDIMODE = 1 和 BIDIOE = 1)
- 当数据写入 SPI_DR 寄存器 (Tx 缓冲)。
- 然后，在第一位传输过程中，将数据从 Tx 缓冲区并行加载到 8 位移位寄存器中，然后串行移出到 MOSI 引脚。
- 没有接收到数据。

在双向模式下，接收时 (BIDIMODE = 1 和 BIDIOE = 0)

- 序列从 SPE = 1 和 BIDIOE = 0 开始。
- MOSI 引脚上接收的数据被串行移入 8 位移位寄存器，然后并行加载到 SPI_DR 寄存器 (Rx 缓冲器) 中。
- 发送器未激活，没有数据串行移出到 MOSI 引脚。

在从模式下启动流程：

在全双工模式下 (BIDIMODE = 0 和 RXONLY = 0)

- 当从设备在其 MOSI 引脚上接收到时钟信号和数据的第一位时，该序列开始。剩余的 7 位被加载到移位寄存器中。
- 同时，在第一位传输过程中，数据从 Tx 缓冲区并行加载到 8 位移位寄存器中，然后串行移出到 MISO 引脚。在 SPI 主设备启动传输之前，软件必须已编写要发送的数据。

在单向仅接收模式下 (BIDIMODE = 0 和 RXONLY = 1)

- 当从设备在其 MOSI 引脚上接收到时钟信号和数据的第一位时，该序列开始。剩余的 7 位被加载到移位寄存器中。
- 发送器未激活，并且没有数据串行移出到 MISO 引脚。

在双向模式下，传输时 (BIDIMODE = 1 和 BIDIOE = 1)

- 当从器件接收到时钟信号并且 Tx 缓冲器中的第一位在 MISO 引脚上发送时，序列开始。
- 然后，在第一位传输过程中，将数据从 Tx 缓冲区并行加载到 8 位移位寄存器中，然后串行移出到 MISO 引脚。发起移位。
- 没有接收到数据。

在双向模式下，接收时 (BIDIMODE = 1 和 BIDIOE = 0)

- 当从设备在其 MISO 引脚上接收到时钟信号和数据的第一位时，该序列开始。
- 在 MISO 引脚上接收的数据被串行移入 8 位移位寄存器，然后并行加载到 SPI_DR 寄存器 (Rx 缓冲区) 中。发送器未激活，并且没有数据串行移出到 MISO 引脚。

处理数据发送和接受

当数据从 Tx 缓冲区传输到移位寄存器时，TXE 标志 (Tx 缓冲区为空) 置位。它指示内部 Tx 缓冲区已准备好与下一个数据一起加载。

如果将 SPI_CR2 寄存器中的 TXEIE 位置 1，则可以产生中断。通过写 SPI_DR 寄存器来清除 TXE 位。在尝试写入 Tx 缓冲区之前，软件必须确保 TXE 标志设置为 1。否则，它将覆盖先前写入 Tx 缓冲区的数据。当数据从移位寄存器传输到 Rx 缓冲区时，在最后一个采样时钟沿设置 RXNE 标志 (Rx 缓冲区不为空)。它指示已准备好从 SPI_DR 寄存器读取数据。如果将 SPI_CR2 寄存器中的 RXNEIE 位置 1，则会产生一个中断。通过读取 SPI_DR 寄存器来清除 RXNE 位。

对于某些配置，可以在最后一次数据传输期间使用 BSY 标志，以等待直到传输完成。主或从模式下的全双工发送和接收过程 (BIDIMODE = 0 和 RXONLY = 0)。

软件必须遵循以下流程来发送和接收数据：

- 通过将 SPE 位置 1 来使能 SPI。
- 将要发送的第一个数据项写入 SPI_DR 寄存器 (这将清除 TXE 标志)。
- 等待直到 TXE = 1 并写入要发送的第二个数据项。然后等待直到 RXNE = 1 并读取 SPI_DR 以获取第一个接收到的数据项 (这将清除 RXNE 位)。对每个要发送/接收的数据项重复此操作，直到 n-1 个接收到的数据为止。
- 等待直到 RXNE = 1 并读取最后收到的数据。
- 等待直到 TXE = 1，然后等待直到 BSY = 0，然后再禁用 SPI。

也可以使用在 RXNE 或 TXE 标志的每个上升沿启动的专用中断子例程来实现此过程。

图26.4 连续传输主/全双工模式 (BIDIMODE/RXONLY = 0) 下的 TXE/RXNE/BSY 行为

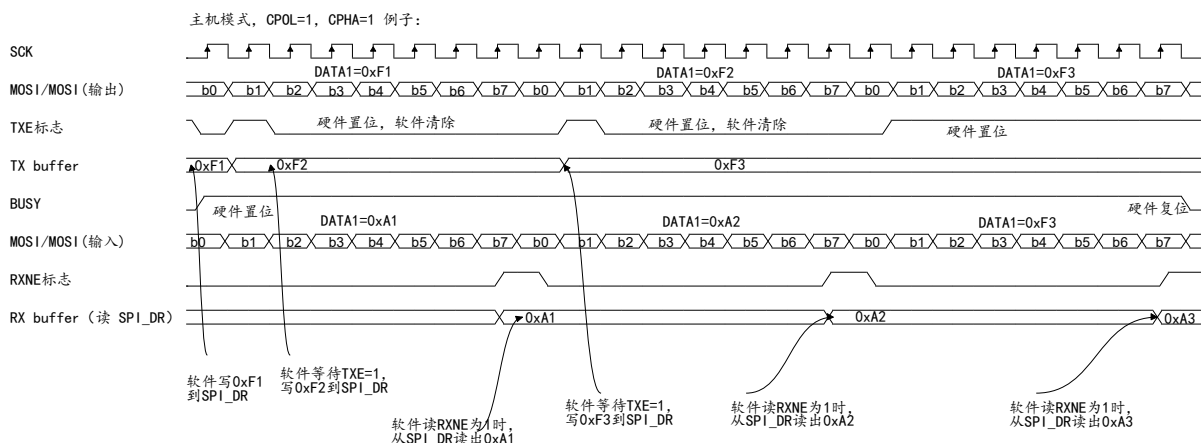
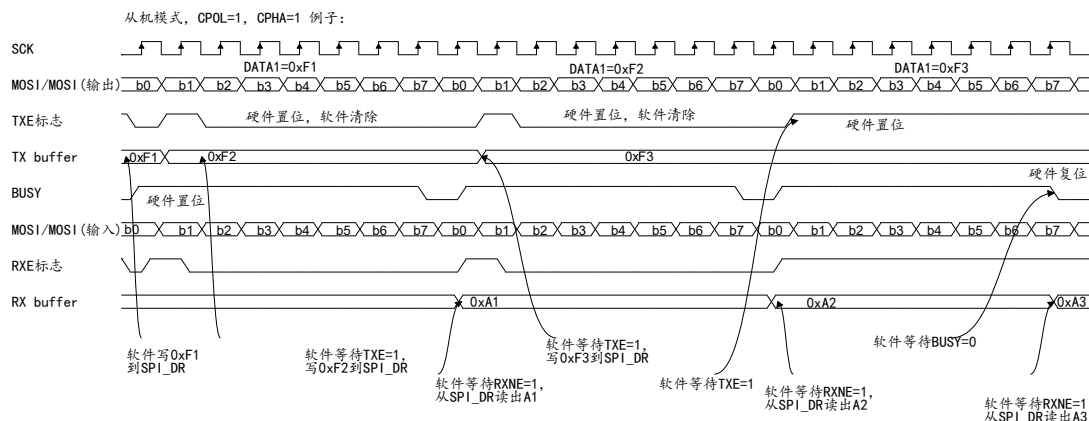


图26.5 连续传输/全双工模式 (BIDIMODE/RXONLY = 0) 下的 TXE/RXNE/BSY 行为



仅发送过程 (BIDIMODE = 0 RXONLY = 1)

在这种模式下, 可以如下所述减少过程, 并且可以使用 BSY 位来等待直到传输完成。

- 通过将 SPE 位置 1 来使能 SPI。
- 将第一个数据项写入入发送到 SPI_DR 寄存器 (这将清除 TXE 位)。
- 等待直到 TXE = 1 并写入下一个要发送的数据项。对要传输的每个数据项重复此步骤。
- 将最后一个数据项写入 SPI_DR 寄存器之后, 等待直到 TXE = 1, 然后等待直到 BSY = 0, 这表明最后一个数据的传输已完成。

也可以使用在 TXE 标志的每个上升沿启动的专用中断子例程来实现此过程。

在不连续通信期间, 对 SPI_DR 的写操作与 BSY 位设置之间存在 2 APB 时钟周期延迟。因此, 在仅发送模式下, 必须先等待, 直到设置 TXE, 然后再写入最后一个数据之后, 清除 BSY。在仅发送模式下发送两个数据项后, 由于从不读取接收到的数据, 因此在 SPI_SR 寄存器中设置了 OVR 标志

图26.6 连续传输主发送模式 (BIDIMODE/RXONLY=0) TXE/BSY 行为

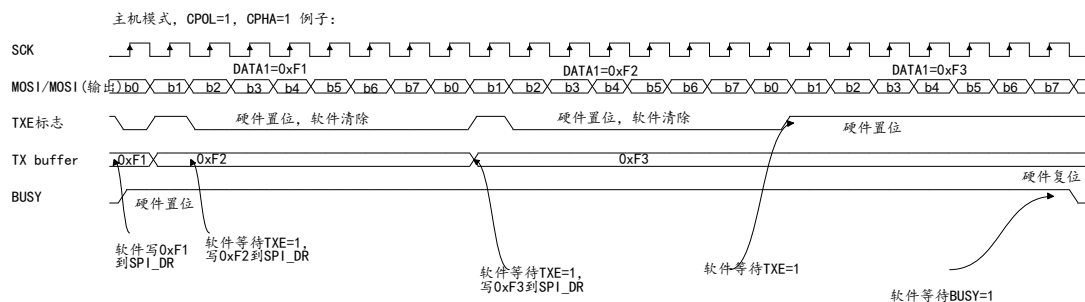
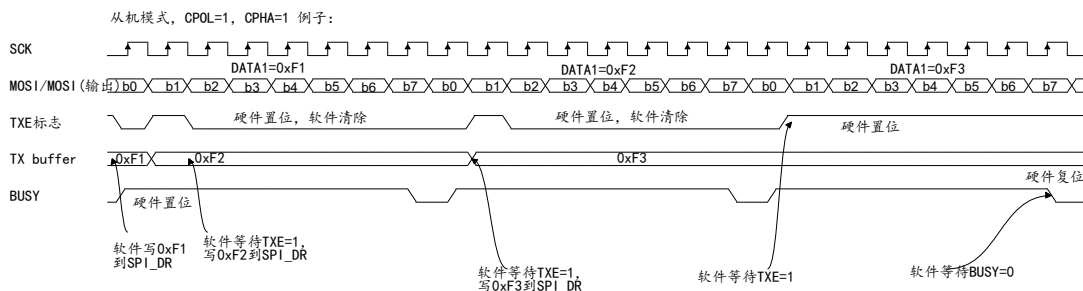


图26.7 连续传输从发送模式 (BIDIMODE/RXONLY=0) TXE/BSY 行为



双向传输过程 (BIDIMODE = 1 和 BIDIOE = 1):

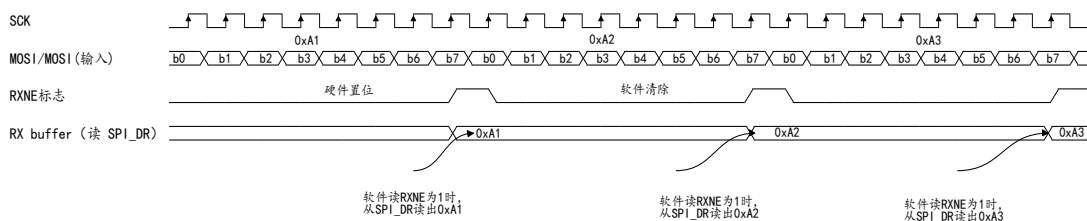
在此模式下, 该过程与仅发送模式下的过程类似, 不同之处在于, 在使能 SPI 之前, 必须将 SPI_CR2 寄存器中的 BIDIMODE 和 BIDIOE 位都置 1。

单向仅接收过程 (BIDIMODE = 0 和 RXONLY = 1)

在这种模式下, 可以如下所述减少过程:

- 将 SPI_CR1 寄存器中的 RXONLY 位置 1。
- 通过将 SPE 位置 1 来使能 SPI:
 - 在主机模式下, 这立即激活了 SCK 时钟的生成, 并且串行接收数据, 直到 SPI 被禁止 (SPE = 0)。
 - 在从机模式下, 当 SPI 主设备将 NSS 驱动为低电平并产生 SCK 时钟时, 将接收数据。
- 等待直到 RXNE = 1 并读取 SPI_DR 寄存器以获取接收到的数据 (这将清除 RXNE 位)。对要接收的每个数据项重复此操作。
- 也可以使用在 RXNE 标志的每个上升沿启动的专用中断子例程来实现此过程。

图26.8 连续传输仅接收模式 (BIDIRMODE=0 和 RXONLY=1) 下的 RXNE 行为



双向接收过程 (BIDIMODE = 1 和 BIDIOE = 0)

在该模式下, 该过程与仅接收模式下的过程类似, 除了在使能 SPI 之前必须将 SPI_CR2 寄存器中的 BIDIMODE 位置 1 并将 BIDIOE 位清零。

连续和不连续转移

在主机模式发送数据时, 如果软件足够快以检测 TXE (或 TXE 中断) 的每个上升沿并在正在进行的数据传输完成之前立即写入 SPI_DR 寄存器, 则表示通信是连续的。在这种情况下, 每个数据项之间的 SPI 时钟的生成不会中断, 并且在每次数据传输之间都不会清除 BSY 位。相反, 如果软件不够快, 则可能导致通信中的某些中断。在这种情况下, 每次数据传输之间的 BSY 位被清除。

在仅主机接收模式 (RXONLY = 1) 下, 通信始终是连续的, 并且 BSY 标志始终读取为 1。

在从模式下，通信的连续性由 SPI 主设备决定。无论如何，即使通信是连续的，每次传输之间的 BSY 标志也会变为低电平，并持续一个 SPI 时钟周期的最小持续时间。

26.1.6 CRC 校验

独立的 CRC 校验用于发送数据和接收数据，保证了通信的可靠性。

CRC 是使用可编程多项式在每个位上串行计算的，它是根据 SPI_CR1 寄存器中 CPHA 和 CPOL 位定义的采样时钟沿计算的。此 SPI 提供两种 CRC 计算标准，它们直接取决于为传输和/或接收选择的数据帧格式：8 位数据 (CR8) 和 16 位数据 (CRC16)。

通过将 SPI_CR1 寄存器中的 CRCEN 位置 1 使能 CRC 校验。该操作将重置 CRC 寄存器 (SPI_RXCRCR 和 SPI_TXCRCR)。在全双工或仅发送模式下，当通过软件管理传输 (CPU 模式) 时，必须在将要传输的最后一个数据写入 SPI_DR 之后立即写入 CRCNEXT 位。在最后一次数据传输结束时，将发送 SPI_TXCRCR 值。

在仅接收模式下，并且当传输由软件管理 (CPU 模式) 时，必须在接收到倒数第二个数据之后将 CRCNEXT 位写入。在最后一次接收数据之后立即接收 CRC，然后执行 CRC 检查。在数据和 CRC 传输结束时，如果在传输期间发生损坏，则 SPI_SR 寄存器中的 CRCERR 标志将置 1。

如果 TX 缓冲区中存在数据，则仅在发送数据字节之后才发送 CRC 值。在 CRC 传输期间，CRC 计算器被关闭并且寄存器值保持不变。

通过以下过程，可以使用 CRC 进行 SPI 通信：

- 编程 CPOL, CPHA, LSBFirst, BR, SSM, SSI 和 MSTR 值。
- 在 SPI_CR1 寄存器中编程多项式。
- 通过将 SPI_CR1 寄存器中的 CRCEN 位置 1 来使能 CRC 计算。这也将清除 SPI_RXCRCR 和 SPI_TXCRCR 寄存器。
- 通过将 SPI_CR1 寄存器中的 SPE 位置 1 来使能 SPI。
- 开始通信并维持通信，直到发送或接收了除一个字节或半字以外的所有字节。
 - 在全双工或仅发送器模式下，当传输由软件管理时，将最后一个字节或半字写入 Tx 缓冲区时，将 SPI_CR1 寄存器中的 CRCNEXT 位置 1，以指示 CRC 将在发送后发送最后一个字节的
 - 在仅接收器模式下，在接收到倒数第二个数据之后立即将 CRCNEXT 位置 1，以使 SPI 在接收到最后一个数据结束时进入 CRC 阶段。CRC 传输在 CRC 传输期间被冻结。
- 传输完最后一个字节或半个字后，SPI 进入 CRC 传输和检查阶段。在全双工模式或仅接收器模式下，会将接收到的 CRC 与 SPI_RXCRCR 值进行比较。如果该值不匹配，则将 SPI_SR 中的 CRCERR 标志置 1，并且将 SPI_CR2 寄存器中的 ERRIE 位置 1 时可以产生中断。

当 SPI 处于从模式时，仅当时钟稳定时，即时钟处于稳定状态时，请小心启用 CRC 校验。如果不是，则可能会执行错误的 CRC 校验。实际上，只要设置了 CRCEN，CRC 就对 SCK 从设备输入时钟敏感，而这与 SPE 位的值无关。

对于高比特率频率，在传输 CRC 时要小心。由于在 CRC 传输阶段使用的 CPU 周期数必须尽可能少，因此禁止在 CRC 传输序列中调用软件功能，以避免最后数据和 CRC 接收中的错误。实际上，必须在最后一个数据的发送/接收结束之前写入 CRCNEXT 位。

对于高比特率频率，建议使用 DMA 模式，以避免由于 CPU 访问影响 SPI 带宽而导致 SPI 速度性能下降。当器件被配置为从器件并且使用 NSS 硬件模式时，NSS 引脚需要在数据阶段和 CRC 阶段之间保持低电平。

当将 SPI 配置为从模式并启用 CRC 功能时，即使在 NSS 引脚上施加了高电平，也会进行 CRC 计算。例如，在多主机环境中，通信主机交替寻址从站时，可能会发生这种情况。

在从机取消选择 (NSS 为高电平) 和新的从机选择 (NSS 为低电平) 之间, 应在主侧和从侧均清除 CRC 值, 以便重新同步主从机进行各自的 CRC 计算。

要清除 CRC, 请按照以下步骤操作:

- 禁用 SPI (SPE = 0)
- 清除 CRCEN 位
- 将 CRCEN 位置 1
- 启用 SPI (SPE = 1)

26.1.7 状态标识

四个状态标志提供给了应用程序, 以完全监视 SPI 总线的状态。

发送缓冲区空标志 (TXE)

设置时, 此标志表示 Tx 缓冲区为空, 可以将下一个要发送的数据装入该缓冲区。写入 SPI_DR 寄存器时, TXE 标志被清除。

接收缓冲区不为空 (RXNE)

置位时, 此标志指示 Rx 缓冲区中有有效的接收数据。读取 SPI_DR 时清除。

忙标志 (BSY)

该 BSY 标志由硬件设置和清除 (写入该标志无效)。BSY 标志指示 SPI 通信层的状态。

设置 BSY 后, 表明 SPI 正在通信。在主模式/双向接收模式 (MSTR = 1 和 BDM = 1 并且 BDOE = 0) 中, 有一个例外, 即 BSY 标志在接收期间保持为低电平。如果软件希望使用 BSY 标志来检测传输结束, 禁用 SPI 并进入暂停模式 (或禁用外设时钟)。这样可以避免破坏最后的传输。为此, 必须严格遵守下述步骤。BSY 标志对于避免在多主系统中发生写冲突也很有用。除主机模式/双向接收模式 (MSTR = 1 和 BDM = 1 和 BDOE = 0) 外, 传输开始时设置 BSY 标志。

清除标志:

传输完成后 (如果是连续通讯, 则在主模式下除外)

禁用 SPI 时

当发生主模式故障 (MODF = 1)

当通信不连续时, 每次通信之间的 BSY 标志为低。当通讯持续进行时:

在主模式下, 所有传输期间 BSY 标志均保持为高

在从机模式下, 每次传输之间的一个 SPI 时钟周期, BSY 标志变为低电平

注意: 不要使用 BSY 标志来处理每个数据发送或接收。最好改用 TXE 和 RXNE 标志。

26.1.8 禁用 SPI

传输终止后, 应用程序可以通过禁用 SPI 外设来停止通信。这是通过将 SPE 位清零来完成的。

对于某些配置, 在正在进行传输的同时禁用 SPI 并进入暂停模式可能会导致当前传输损坏和/或 BSY 标志可能变得不可靠。为避免这些影响, 建议在禁用 SPI 时遵循以下步骤:

在主机或从机全双工模式下 (BIDIMODE = 0, RXONLY = 0)

- 等待直到 RXNE = 1 接收最后一个数据
- 等到 TXE = 1
- 然后等待, 直到 BSY = 0

- 禁用 SPI ($SPE = 0$), 并最终进入暂停模式 (或禁用外设时钟)

在主机或从机单向仅发送模式 ($BIDIMODE = 0$, $RXONLY = 0$) 或双向发送模式 ($BIDIMODE = 1$, $BIDIOE = 1$) 下, 将最后一个数据写入 SPI_DR 寄存器后:

- 等待直到 $TXE = 1$
- 然后等待, 直到 $BSY = 0$
- 禁用 SPI ($SPE = 0$), 并最终进入暂停模式 (或禁用外设时钟)

在主单向仅接收模式 ($MSTR = 1$, $BIDIMODE = 0$, $RXONLY = 1$) 或双向接收模式 ($MSTR = 1$, $BIDIMODE = 1$, $BIDIOE = 0$)

必须以特定方式管理这种情况, 以确保 SPI 不会启动新的传输:

- 等待倒数第二次出现 $RXNE = 1$ ($n - 1$)
- 然后等待一个 SPI 时钟周期 (使用软件循环), 然后再禁用 SPI ($SPE = 0$)
- 然后在进入暂停模式 (或禁用外设时钟) 之前等待最后一个 $RXNE = 1$ 。

注意: 在主机双向接收模式 ($MSTR = 1$ 和 $BDM = 1$ 并且 $BDOE = 0$) 中, 传输期间 BSY 标志保持低电平。

在从机仅接收模式 ($MSTR = 0$, $BIDIMODE = 0$, $RXONLY = 1$) 或双向接收模式 ($MSTR = 0$, $BIDIMODE = 1$, $BIDOE = 0$)

- 可以随时禁用 SPI (写 $SPE = 1$): 当前传输将完成在有效禁用 SPI 之前
- 然后, 如果要进入暂停模式, 则必须首先等到 $BSY = 0$ 后才能进入暂停模式 (或禁用外设时钟)。

26.1.9 使用 DMA 的 SPI 通信

为了以最大速度运行, 需要向 SPI 馈送数据以进行传输, 并且应读取 Rx 缓冲区上接收到的数据以避免溢出。为了促进传输, SPI 具有实现简单请求/确认协议的 DMA 功能。

当使能 SPI_CR2 寄存器中的使能位时, 将请求 DMA 访问。必须向 Tx 和 Rx 缓冲区发出单独的请求:

- 在发送过程中, 每次 TXE 设置为 1 时, 都会发出 DMA 请求。然后 DMA 写入 SPI_DR 寄存器 (这将清除 TXE 标志)。
- 在接收过程中, 每次将 $RXNE$ 设置为 1 时, 都会发出 DMA 请求。然后 DMA 读取 SPI_DR 寄存器 (这将清除 $RXNE$ 标志)。

当 SPI 仅用于传输数据时, 可以仅启用 SPI Tx DMA 通道。在这种情况下, 设置 OVR 标志是因为接收到的数据未读。

当 SPI 仅用于接收数据时, 可以仅使能 SPI Rx DMA 通道。

在发送模式下, 当 DMA 已写入所有要发送的数据时 (在 DMA_ISR 寄存器中设置了标志 $TCIF$), 可以监视 BSY 标志以确保 SPI 通信完成。这样做是为了避免在禁用 SPI 或进入停止模式之前破坏最后的传输。软件必须首先等待, 直到 $TXE = 1$, 然后直到 $BSY = 0$ 。

注意: 在不连续通信期间, 对 SPI_DR 的写操作与 BSY 位设置之间存在 2 APB 时钟周期延迟。因此, 在写入最后一个数据之后, 必须先等待直到 $TXE = 1$, 然后再等待 $BSY = 0$ 。

具有 CRC 的 DMA 功能

当通过 CRC 通信和 DMA 模式启用 SPI 通信时, 在通信结束时将自动进行 CRC 的发送和接收, 而无需使用 $CRCNEXT$ 位。CRC 接收后, 必须在 SPI_DR 寄存器中读取 CRC 以清除 $RXNE$ 标志。

在数据和 CRC 传输结束时, 如果在传输期间发生损坏, 则 SPI_SR 中的 $CRCERR$ 标志将置 1。

图26.9 DMA 传输 1

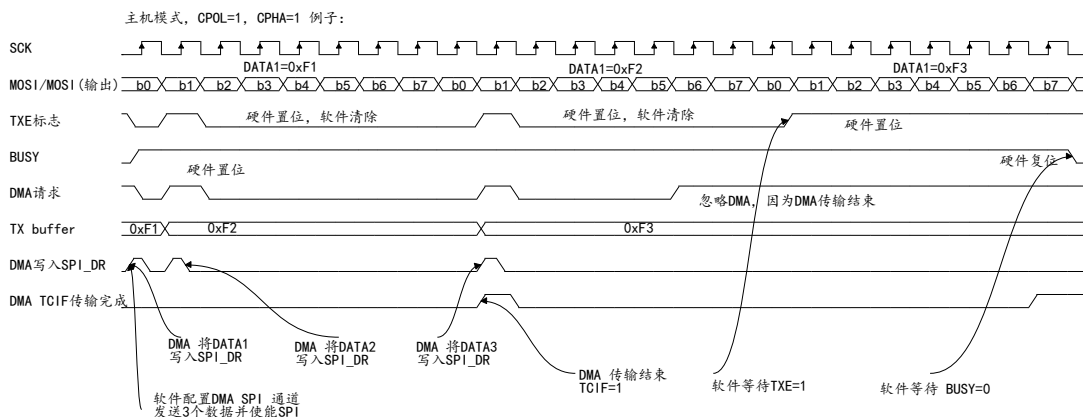
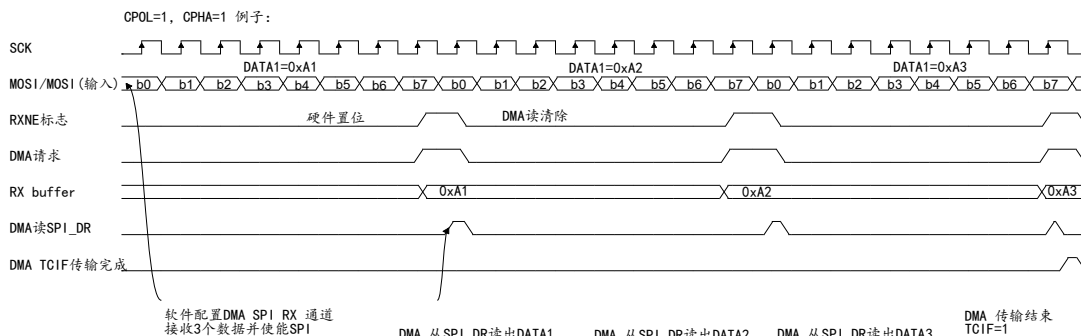


图26.10 DMA 传输 2



26.1.10 错误标识

主模式故障 (MODF)

当主机设备将其 NSS 引脚拉低 (在 NSS 硬件模式下) 或将 SSI 位拉低 (在 NSS 软件模式下) 时, 发生主机模式故障, 这将自动将 MODF 位置 1。主模式故障通过以下方式影响 SPI 外设:

- 如果 ERRIE 位置 1, 则 MODF 位置 1, 并产生 SPI 中断。
- SPE 位被清除, 这会阻止该设备的所有输出并禁用 SPI 接口。
- MSTR 位被清除, 从而迫使产品进入从模式。

使用以下软件序列清除 MODF 位:

- 在 MODF 位置 1 时, 对 SPI_SR 寄存器进行读或写访问。
- 然后写入 SPI_CR1 寄存器。

为了避免在包含多个 MCU 的系统中发生任何多个从设备冲突, 必须在 MODF 位清除序列期间将 NSS 引脚拉高。在执行该清除序列后, 可以将 SPE 和 MSTR 位恢复到其原始状态。

为安全起见, 当 MODF 位置 1 时, 硬件不允许将 SPE 和 MSTR 位置 1。

在从设备中, 无法将 MODF 位置 1。但是, 在多主机配置中, 通过将 MODF 位置 1, 设备可以处于从机模式。在这种情况下, MODF 位指示系统控制可能存在多主机冲突。通过执行复位或返回默认状态, 可以使用中断例程从此状态完全恢复。

溢出条件

当主设备发送了数据字节，而从设备尚未清零由先前发送的数据字节引起的 RXNE 位时，就会发生溢出条件。当发生溢出情况时：

- OVR 位置 1，如果 ERRIE 位置 1，则产生中断。

在这种情况下，接收器缓冲区的内容将不会使用从主设备接收到的新数据进行更新。从 SPI_DR 寄存器读取将返回该字节。随后传输的所有其他字节均丢失。清除 OVR 位是通过对 SPI_DR 寄存器的读取，然后是对 SPI_SR 寄存器的读取访问来完成的。

CRC 错误

当 SPI_CR1 寄存器中的 CRCEN 位置 1 时，该标志用于验证所接收值的有效性。如果移位寄存器中接收到的值与接收器 SPI_RXCRCR 值不匹配，则 SPI_SR 寄存器中的 CRCERR 标志将置 1。

26.1.11 SPI 中断

图26.11 SPI 中断请求

中断事件	事件标志	控制使能
发送缓存空标志	TXE	TXEIE
接收缓存非空标志	RXNE	RXNEIE
主模式故障	MODF	ERRIE
超时错误	OVR	
CRC 错误标志	CRCERR	

26.2 SPI 寄存器

26.2.1 SPI 控制寄存器 1 (SPI_CR1) (I2S 模式下不可用)

偏移地址：0x00，复位值：0x0000 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDIMODE	BIDIOE	CRCEN	CRCNEXT	DFF	RXONLY	SSM	SSI	LSBFIRST	SPE	BR[2:0]		MSTR	CPOL	CPHA	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
15	BIDIMODE	双向数据模式使能 0：选择 2 线单向数据模式 1：选择 1 线双向数据模式	0x0	rw	该位在 I2S 模式下不使用
14	BIDIOE	双向模式下的输出使能 该位与 BIDImode 位结合选择双向模式下的传输方向 0：输出禁用（仅接收模式） 1：输出使能（仅传输模式）	0x0	rw	该位在 I2S 模式下不使用。 在主模式下，使用 MOSI 引脚，而在从模式下使用 MISO 引脚。

Bit	Field	Description	Reset value	RW attribute	Remark
13	CRCEN	硬件 CRC 计算使能 0: CRC 计算禁用 1: CRC 计算使能 注: 只有在禁止 SPI 时(SPE=0), 才能写该位, 否则出错。 该位只能在全双工模式下使用。 注: I2S 模式下不使用。	0x0	rw	注: 只有在 SPI 被禁用 (SPE = '0') 时才应写入该位, 以便正确操作。它不用于 I2S 模式。
12	CRCNEXT	CRC 传输下一步 0: 数据阶段 (无 CRC 阶段) 1: 下一个传输是 CRC (CRC 阶段) CRCNEXT: 下一个发送 CRC (Transmit CRC next) 0: 下一个发送的值来自发送缓冲区。 1: 下一个发送的值来自发送 CRC 寄存器。 注: 在 SPI_DR 寄存器写入最后一个数据后应马上设置该位。 注: I2S 模式下不使用。	0x0	rw	当 SPI 配置为全双工或仅发送器模式时, 必须在最后一个数据写入 SPI_DR 寄存器后立即写入 CRCNEXT。 当 SPI 配置为仅接收模式时, CRCNEXT 必须在倒数第二个数据接收后设置。当传输由 DMA 管理时, 该位应保持清零。它不用于 I2S 模式。
11	DFF	数据帧格式 0: 发送/接收选择 8 位数据帧格式 1: 发送/接收选择 16 位数据帧格式 注: 只有当 SPI 禁止(SPE=0)时, 才能写该位, 否则出错。 注: I2S 模式下不使用。	0x0	rw	仅当 SPI 被禁用 (SPE = '0') 时才应写入该位, 以便正确操作。它不用于 I2S 模式。
10	RXONLY	只收 该位与 BIDI MODE 位结合选择 2 线单向模式中的传输方向。该位在多从系统中也很有用, 在这种系统中, 这个特定的从站不被访问, 被访问从站的输出没有被破坏。 0: 全双工 (发送和接收) 1: 输出禁用 (仅接收模式) 注: I2S 模式下不使用。	0x0	rw	该位在 I2S 模式下不使用
9	SSM	软件从机管理 当 SSM 位被设置时, NSS 引脚输入被 SSI 位的值替换。 0: 软件从机管理禁用 1: 软件从机管理使能 注: I2S 模式下不使用。	0x0	rw	该位在 I2S 模式下不使用
8	SSI	内部从机选择 该位仅在 SSM 位被设置时有效。该位的值被强制加到 NSS 引脚上, NSS 引脚的 IO 值被忽略。 该位只在 SSM 位为 '1' 时有意义。它决定了 NSS 上的电平, 在 NSS 引脚上的 I/O 操作无效。 注: I2S 模式下不使用。	0x0	rw	该位在 I2S 模式下不使用

Bit	Field	Description	Reset value	RW attribute	Remark
7	LSBFIRST	帧格式 0: MSB 先发送 1: LSB 先发送 注: 当通信在进行时不能改变该位的值。 注: I2S 模式下不使用。	0x0	rw	当通信正在进行时, 不应更改该位。它不用于 I2S 模式
6	SPE	SPI 使能 0: SPI 外设禁用 1: SPI 外设使能 注: I2S 模式下不使用。	0x0	rw	该位在 I2S 模式下不使用。禁用 SPI 时, 请按照第 1.4.8 节: 禁用 SPI 中描述的步骤进行操作。
5:3	BR	波特率控制 000: fPCLK/2 001: fPCLK/4 010: fPCLK/8 011: fPCLK/16 100: fPCLK/32 101: fPCLK/64 110: fPCLK/128 111: fPCLK/256 当通信正在进行的时候, 不能修改这些位。 注意: I2S 模式下不使用。	0x0	rw	当通信正在进行时, 不应更改这些位。它们不用于 I2S 模式。
2	MST	Master 选择 0: 配置为从设备; 1: 配置为主设备。 注: 当通信正在进行的时候, 不能修改该位。 注: I2S 模式下不使用。	0x0	rw	当通信正在进行时, 不应更改该位。它不用于 I2S 模式。
1	CPOL	时钟极性 0: 空闲时 CK 为 0 1: 空闲时 CK 为 1 0: 空闲状态时, SCK 保持低电平; 1: 空闲状态时, SCK 保持高电平。 注: 当通信正在进行的时候, 不能修改该位。 注: I2S 模式下不使用。	0x0	rw	当通信正在进行时, 不应更改该位。它不用于 I2S 模式
0	CPHA	时钟相位 0: 数据采样从第一个时钟边沿开始; 1: 数据采样从第二个时钟边沿开始。 注: 当通信正在进行的时候, 不能修改该位。 注: I2S 模式下不使用。	0x0	rw	当通信正在进行时, 不应更改该位。它不用于 I2S 模式

26.2.2 SPI 控制寄存器 2 (SPI_CR2)

偏移地址: 0x04, 复位值: 0x0000 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TXEIE	RXNEIE	ERRIE	Reserved.	Reserved.	SSOE	TXDMAEN	RXDMAEN
								rw	rw	rw			rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
15:8	Reserved	必须保持在复位值。	0x0	rw	
7	TXEIE	Tx 缓冲区空中断使能 0: TXE 中断被屏蔽 1: TXE 中断未被屏蔽。 用于在设置 TXE 标志时产生中断请求。	0x0	rw	
6	RXNEIE	RX 缓冲区非空中断使能 0: RXNE 中断屏蔽 1: RXNE 中断未被屏蔽。 用于在设置 RXNE 标志时产生中断请求。	0x0	rw	
5	ERRIE	错误中断使能 该位控制发生错误条件时产生中断 (SPI 模式下的 CRCERR、OVR、MODF 和 I2S 模式下的 UDR、OVR)。 0: 错误中断被屏蔽 1: 错误中断使能	0x0	rw	
4:3	Reserved	必须保持在复位值。	0x0	rw	
2	SSOE	SS 输出使能 0: SS 输出在 master 模式下被禁用, cell 可以在 multimaster 配置下工作 1: SS 输出在主模式和单元启用时启用。 该单元无法在多主环境中工作。 0: 禁止在主模式下 SS 输出, 该设备可以工作在多主设备模式; 1: 设备开启时, 开启主模式下 SS 输出, 该设备不能工作在多主设备模式。 注: I2S 模式下不使用。	0x0	rw	该位在 I2S 模式下不使用
1	TXDMAEN	Tx 缓冲区 DMA 使能 当该位被设置时, 只要 TXE 标志被设置, 就会发出 DMA 请求。 0: Tx 缓冲区 DMA 禁用 1: Tx 缓冲区 DMA 使能	0x0	rw	
0	RXDMAEN	接收缓冲区 DMA 使能 当该位被设置时, 只要 RXNE 标志被设置, 就会发出 DMA 请求。 0: Rx 缓冲区 DMA 禁用 1: Rx 缓冲区 DMA 使能	0x0	rw	

26.2.3 SPI 状态寄存器 (SPI_SR)

偏移地址: 0x08, 复位值: 0x0002 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								BSY	OVR	MODF	CRCERR	UDR	CHSIDE	TXE	RXNE
								r	r	r	rc_w0	r	r	r	r

Bit	Field	Description	Reset value	RW attribute	Remark
15:8	Reserved	必须保持在复位值。	0x0	rw	
7	BSY	忙标志 0: SPI (或 I2S) 不忙 1: SPI (或 I2S) 正忙于通信或 Tx 缓冲区不为空 该标志由硬件设置和清除。	0x0	rw	必须谨慎使用 BSY 标志: 请参阅第 1.4.7 节: 状态标志和第 1.4.8 节: 禁用 SPI。
6	OVR	溢出标志 0: 未发生溢出 1: 发生溢出 该标志由硬件设置并由软件序列复位。 请参阅第 25.4.7 节: 软件序列的错误标志。	0x0	rw	
5	MODF	模式故障 0: 无模式故障发生 1: 发生模式故障 该标志由硬件设置并由软件序列复位。 请参阅第 25.3.10 节: 软件序列的错误标志。 注: I2S 模式下不使用。	0x0	rw	该位在 I2S 模式下不使用
4	CRCERR	CRC 错误标志 0: 接收到的 CRC 值与 SPI_RXCRCR 值匹配 1: 接收到的 CRC 值与 SPI_RXCRCR 值不匹配 该标志由硬件置位, 由软件写 0 清零。	0x0	rw	该位在 I2S 模式下不使用
3	UDR	下溢标志 0: 未发生下溢 1: 发生下溢 该标志由硬件设置并由软件序列复位。 请参阅第 25.4.7 节: 软件序列的错误标志。	0x0	rw	该位在 I2S 模式下不使用
2	CHSIDE	通道侧 0: Channel Left 必须发送或已接收 1: Channel Right 已发送或已接收 CHSIDE: 声道 (Channel side) 0: 需要传输或者接收左声道; 1: 需要传输或者接收右声道。 注: 在 SPI 模式下不使用。在 PCM 模式下无意义。	0x0	rw	该位不用于 SPI 模式, 在 PCM 模式下无意义。

Bit	Field	Description	Reset value	RW attribute	Remark
1	TXE	发送缓冲区为空 0: Tx 缓冲区不为空 1: Tx 缓冲区为空	0x1	rw	
0	RXNE	接收缓冲区不为空 0: Rx 缓冲区为空 1: Rx 缓冲区不为空	0x0	rw	

26.2.4 SPI 状态寄存器 (SPI_DR)

偏移地址: 0x0C, 复位值: 0x0000 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
15:0	DR	数据寄存器。 已接收或将要发送的数据。 数据寄存器分为 2 个缓冲区 - 一个用于写入 (发送缓冲区), 另一个用于读取 (接收缓冲区)。写入数据寄存器将写入 Tx 缓冲区, 读取数据寄存器将返回保存在 Rx 缓冲区中的值。 待发送或者已经收到的数据 数据寄存器对应两个缓冲区: 一个用于写(发送缓冲); 另外一个用于读(接收缓冲)。写操作将数据写到发送缓冲区; 读操作将返回接收缓冲区里的数据。 对 SPI 模式的注释: 根据 SPI_CR1 的 DFF 位对数据帧格式的选择, 数据的发送和接收可以是 8 位或者 16 位的。为保证正确的操作, 需要在启用 SPI 之前就确定好数据帧格式。 对于 8 位的数据, 缓冲器是 8 位的, 发送和接收时只会用到 SPI_DR[7:0]。在接收时, SPI_DR[15:8]被强制为 0。 对于 16 位的数据, 缓冲器是 16 位的, 发送和接收时会用到整个数据寄存器, 即 SPI_DR[15:0]。	0x0	rw	这些注释适用于 SPI 模式: 根据数据帧格式选择位 (SPI_CR1 寄存器中的 DFF), 发送或接收的数据为 8 位或 16 位。此选择必须在启用 SPI 之前进行, 以确保正确操作。对于 8 位数据帧, 缓冲区为 8 位, 仅寄存器的 LSB (SPI_DR[7:0]) 用于发送/接收。在接收模式下, 寄存器 (SPI_DR[15:8]) 的 MSB 被强制为 0。对于 16 位数据帧, 缓冲区为 16 位, 整个寄存器, SPI_DR[15:0]为用于发送/接收。

26.2.5 SPI CRC 多项式寄存器 (SPI_CRCPR) (I2S 模式下不可用)

偏移地址: 0x10, 复位值: 0x0007 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRCPOLY[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
15:0	CRCPOLY	CRC 多项式寄存器。 该寄存器包含用于 CRC 计算的多项式。CRC 多项式 (0007h) 是该寄存器的复位值。 可以根据需要配置另一个多项式。 该寄存器包含了 CRC 计算时用到的多项式。 其复位值为 0x0007，根据应用可以设置其他数值。 注：在 I2S 模式下不使用。	0x7	rw	这些位不用于 I2S 模式。

26.2.6 SPI RX CRC 寄存器 (SPI_RXCRCR) (I2S 模式下不可用)

偏移地址：0x14，复位值：0x0000 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RXCRC [15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
15:0	RXCRC	Rx CRC: 接收 CRC 寄存器。 启用 CRC 计算时, RxCRC[15:0]位包含随后接收字节的计算 CRC 值。当 SPI_UCR1 寄存器中的 CRCEN 位写入 1 时, 该寄存器将被重置。CRC 是使用 SPIu_CRCPR 寄存器中编程的多项式串行计算的。当数据帧格式设置为 8 位数据 (清除 SPI_CR1 的 DFF 位) 时, 仅考虑 8 个 LSB 位。CRC 计算是根据任何 CRC8 标准进行的。当选择 16 位数据帧格式时 (设置 SPI_CR1 寄存器的 DFF 位), 考虑该寄存器的整个 16 位。CRC 计算是根据任何 CRC16 标准进行的。 在启用 CRC 计算时, RXCRC[15:0]中包含了依据收到的字节计算的 CRC 数值。当在 SPI_CR1 的 CRCEN 位写入'1'时, 该寄存器被复位。CRC 计算使用 SPI_CRCPR 中的多项式。 当数据帧格式被设置为 8 位时, 仅低 8 位参与计算, 并且按照 CRC8 的方法进行; 当数据帧格式为 16 位时, 寄存器中的所有 16 位都参与计算, 并且按照 CRC16 的标准。 注: 当 BSY 标志为'1'时读该寄存器, 将可能读到不正确的数值。 注: 在 I2S 模式下不使用。	0x0	rw	设置 BSY 标志时读取该寄存器可能返回不正确的值。这些位不用于 I2S 模式。

26.2.7 SPI TX CRC 寄存器 (SPI_TXCRCR) (I2S 模式下不可用)

偏移地址: 0x18, 复位值: 0x0000 0000。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TXCRC [15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	Field	Description	Reset value	RW attribute	Remark
15:0	TXCRC	发送 CRC 寄存器 当启用 CRC 计算时, TxCRC[7:0] 位包含计算出的后续传输字节的 CRC 值。当 SPI_CR1 的 CRCEN 位写入 1 时, 该寄存器复位。使用 SPI_CRCPR 寄存器中编程的多项式连续计算 CRC。当数据帧格式设置为 8 位数据 (SPI_CR1 的 DFF 位清零) 时, 只考虑 8 个 LSB 位。CRC 计算是基于任何 CRC8 标准完成的。当选择 16 位数据帧格式时 (设置 SPI_CR1 寄存器的 DFF 位), 考虑该寄存器的整个 16 位。CRC 计算基于任何 CRC16 标准。 在启用 CRC 计算时, TXCRC[15:0]中包含了依据将要发送的字节计算的 CRC 数值。当在 SPI_CR1 中的 CRCEN 位写入'1'时, 该寄存器被复位。CRC 计算使用 SPI_CRCPR 中的多项式。 当数据帧格式被设置为 8 位时, 仅低 8 位参与计算, 并且按照 CRC8 的方法进行; 当数据帧格式为 16 位时, 寄存器中的所有 16 个位都参与计算, 并且按照 CRC16 的标准。 注: 当 BSY 标志为'1'时读该寄存器, 将可能读到不正确的数值。 注: 在 I2S 模式下不使用。	0x0	rw	设置 BSY 标志时对该寄存器的读取可能会返回不正确的值。这些位不用于 I2S 模式。

27 低功耗通用异步收发器(LPUART)

27.1 低功耗通用异步收发器(LPUART)

27.1.1 概述

本产品带有 1 个 LPUART 模块，支持半双工和全双工传输；支持 8BIT、9BIT 数据格式；支持 Mode 0/1/2/3 四种不同传输模式；LPUART 的波特率由 LPTIM 产生，也可以由内部自动波特率发生器产生；支持多机通讯模式；支持自动地址识别；支持给定地址和广播地址，支持低功耗模式。

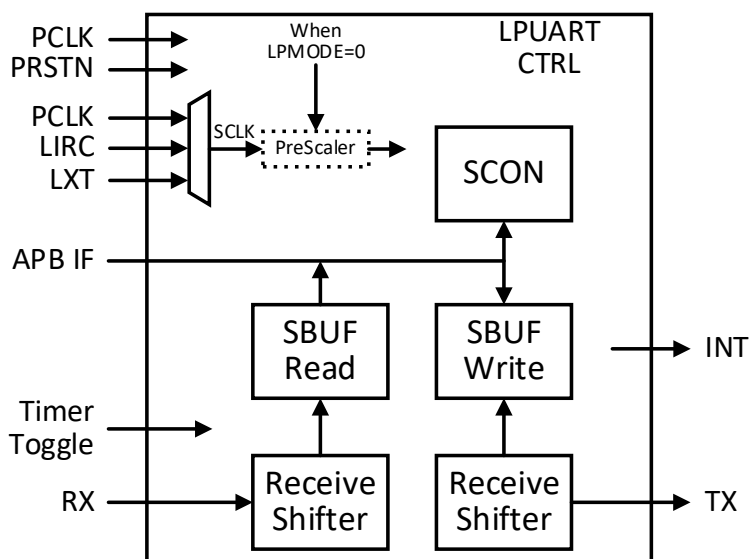
LPUART 为支持低功耗应用，除了原本的 PCLK 时钟外，增加了一路 SCLK 时钟，并可以控制 LPUART 工作状态。LPUART 模块内部寄存器配置逻辑工作于 PCLK 时钟域，数据收发逻辑工作于 SCLK 时钟域。当系统进入低功耗模式且后且处在 LPUART 工作状态，关闭高频 PCLK 时钟，打开低频 SCLK 时钟，LPUART 仍旧可以进行正常的数据收发。关闭工作状态则停止波特率的生成。

SCLK 时钟来源可以选择：PCLK、外部低速时钟(LSE)，内部低速时钟(LSI)。在 LPMODE=1 时，SCLK 时钟还支持 1/2/4/8/16/32/64/128 倍的预分频。

注意，在 LPMODE=0 时，LPUART 接收的是 LPTIM 时钟的 TOGGLE 输出信号而非 OVERFLOW 信号，因此必须使能 LPTIM 的 TOGGLE 输出。

27.1.2 结构框图

图27.1 LPUART 结构图



27.1.3 工作模式

与通用 UART(UART0/1)相比，LPUART 增加了一个 LPMODE 控制位。当该位置“1”时，只支持 Mode 1/3 工作模式，并且波特率生成方式也会发生改变。具体描述请参考以下章节。Mode 0(同步模式，半双工)

当工作在 Mode 0 时，UART 工作在同步模式，其波特率为固定的 SCLK 时钟的 1/12。UART

接收数据由 RXD 输入、UART 发送数据有 RXD 输出，RXD 此时为输入输出端口。UART 同步移位时钟由 TXD 输出，TXD 此时为输出端口。注意，本模式只能作为主机发送同步移位时

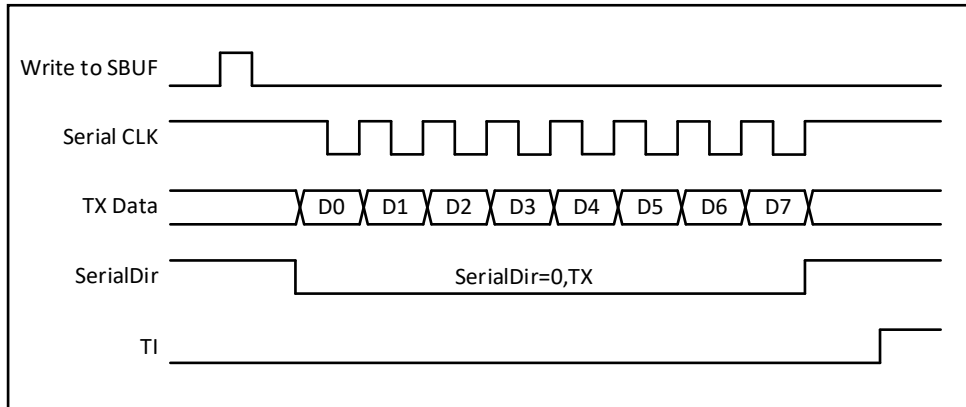
钟， 不可以作为从机从外部接收该时钟。该模式下， 传输的数据位宽只能是 8 位的， 没有起始位和结束位。

将 LPUART_SCON.SM0 和 LPUART_SCON.SM1 清零， 可进入 Mode 0 工作模式。当 LPMODE=1 时， 不支持 Mode 0 工作模式。

发送数据

发送数据时， 清除 LPUART_SCON.REN 位， 并将数据写入 LPUART_SBUF 寄存器。此时， 发送数据将从 RXD 输出(低位在先， 高位在后)， 同步移位时钟从 TXD 输出。

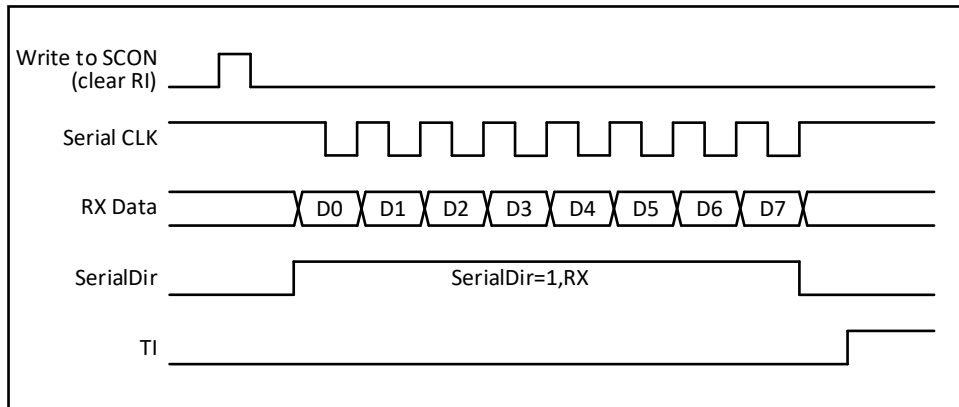
图27.2 Mode 0 发送数据



接收数据

接收数据时， 将 LPUART_SCON.REN 位置 1， 并将 LPUART_INTSR.RI 位清零。当接收结束， 数据可从 LPUART_SBUF 寄存器读出。此时， 接收数据从 RXD 输入(低位在先， 高位在后)， 同步移位时钟从 TXD 输出。

图27.3 Mode 0 接收数据



27.1.3.1 Mode 1(异步模式， 全双工)

当工作在 Mode 1 时， 发送数据通过 TXD 发送， 接收数据通过 RXD 接收。该数据由 10 位组成： 起始位“0”开始， 紧接着 8 位数据位(低位在先， 高位在后)， 最后是结束位“1”。

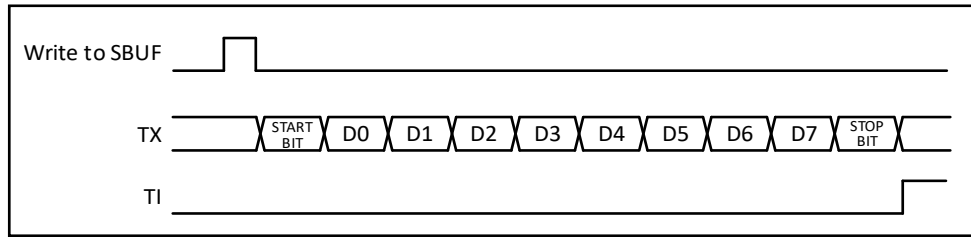
将 LPUART_SCON.SM0 清 0， LPUART_SCON.SM1 置 1， 可进入 Mode 1 工作模式。 该模式下， 当 LPMODE=0 时， LPUART 的波特率可以选择由自动波特率发生器或者定时器 LPTIM 模块产生， 并且是可编程的。

当 LPMODE=1 时， 波特率计算方式发生改变， 具体参考波特率编程章节。

发送数据

发送数据时，与 LPUART_SCON.REN 的值无关，将所发送数据写入 LPUART_SBUF 寄存器中，数据就会从 TXD 移出(低位在先，高位在后)。

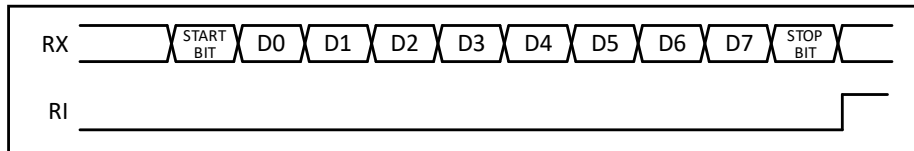
图27.4 Mode 1 发送数据



接收数据

接收数据时，需将 LPUART_SCON.REN 位置 1，并将 LPUART_INTSR.RI 位清 0。开始接收 RXD 上数据(低位在先，高位在后)，当接收完毕，可以从 LPUART_SBUF 寄存器读出。

图27.5 Mode 1 接收数据



27.1.3.2 Mode 2(异步模式，全双工)

当工作在 Mode 2 时，发送数据通过 TXD 发送，接收数据通过 RXD 接收。该数据由 11 位组成：起始位“0”开始，接着是 8 个数据位，1 个 TB8 位和结束位。额外的 TB8 位是用来在多机通讯环境下使用，当 TB8=1，表明所接收的是地址帧；当 TB8=0，表明所接收的是数据帧。当不需要多机通讯时，此位也可以作为奇偶校验位来使用。

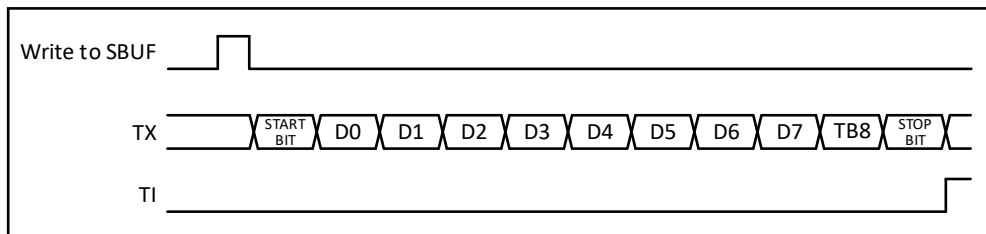
将 LPUART_SCON.SM0 置 1，LPUART_SCON.SM1 清 0，可进入 Mode 2 工作模式。该模式下，波特率可以独立产生，不需要外部 Timer 产生。

当 LPMODE=1 时，不支持 Mode 2 工作模式。

发送数据

发送数据时，与 LPUART_SCON.REN 的值无关，并将所发送数据写入 LPUART_SBUF 寄存器中，数据就会从 TXD 移出(低位在先，高位在后)。

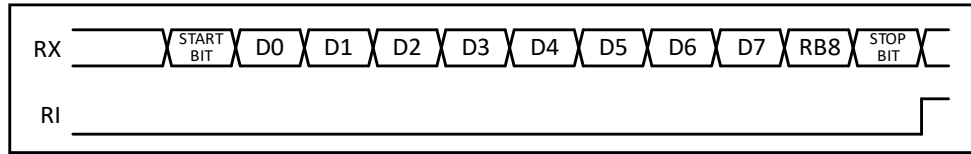
图27.6 Mode 2 发送数据



接收数据

接收数据时，需将 LPUART_SCON.REN 位置 1，并将 LPUART_INTSR.RI 位清 0。开始接收 RXD 上数据(低位在先，高位在后)，当接收完毕，可以从 LPUART_SBUF 寄存器读出。

图27.7 Mode 2 接收数据



27.1.3.3 Mode 3(异步模式，全双工)

Mode 3 的数据格式，传输时序以及操作方式都与 Mode 2 相同，唯一的区别是 Mode 3 的波特率由 LPTIM 产生或者内部自动波特率发生器产生，而不是像 Mode 2 由设备自己独立产生。Mode 3 的波特率是可编程的，波特率生成方式与 Mode 1 相同。

将 LPUART_SCON.SM0 置 1，LPUART_SCON.SM1 置 1，可进入 Mode 3 工作模式。

当 LPMODE=1 时，支持 Mode 3 工作模式。

27.1.4 波特率编程

27.1.4.1 Mode 0

LPMODE=0

当工作在 Mode 0 时，波特率被固定在 PCLK 的 1/12，不需要 LPTIM 的支持。

LPMODE=1

当 LPMODE=1 时，不支持该模式。

27.1.4.2 Mode 1/3

LPMODE=0

当工作在 Mode 1 或者 Mode 3 时，波特率可以由 LPTIM 的溢出时间决定。具体公式如下图所示

$$\text{BaudRate} = \frac{(\text{LPUART_SCON.DBAUD} + 1) * F_{\text{SCLK}}}{32 * (2^{16} - \text{LPTIM_BGLOAD}[15:0])}$$

其中，LPUART_SCON.DBAUD 表示双倍波特率，FSCLK 为 SCLK 时钟频率 LPTIM_BGLOAD 为 LPTIM 的周期装载计数值。

注意，LPTIM 必须配置为 16 位自动重载模式，立即重载寄存器(LPTIM_LOAD)和周期重载寄存器(LPTIM_BGLOAD)要写入相同的初始值。也可以使用自身波特率生成模式：

$$\text{BAUDRATE} = \frac{(\text{LPUART_SCON.DBAUD} + 1) * F_{\text{SCLK}}}{32 * (\text{LPUART_BAUDCR.BRG} + 1)}$$

其中，UARTX_SCON.DBAUD 表示双倍波特率，FSCLK 为 SCLK 时钟频率。

27.1.4.3 LPMODE=1

当 LPMODE 设为“1”时，波特计算公式与上述公式不同，简化为：

$$\text{BAUDRATE} = \frac{F_{\text{SCLK}}}{4 * \text{LPUART_SCON.PRSC}}$$

其中，FSCLK 为 SCLK 时钟频率，LPUART_SCON.PRSC 为预分频系数。

27.1.4.4 Mode 2

LPMODE=0

当工作在 Mode 2 时，传输时钟只能选择 PCLK，波特率被固定在如下公式所得值：

$$\text{BAUDRATE} = \frac{(\text{LPUART_SCON.DBAUD} + 1) * F_{\text{PCLK}}}{64}$$

其中，LPUART_SCON.DBAUD 表示双倍波特率，FPCLK 为 PCLK 时钟频率。

当 LPMODE=1 时，不支持该模式。

27.1.5 帧错误检测

Mode 1/2/3 具有帧错误检测功能，硬件会自动检测接收到的帧数据是否带有效的 STOP 位。如果没有收到有效 STOP 位，则 LPUART_INTSR.FE 置 1。LPUART_INTSR.FE 位由硬件置 1，软件清 0，如果软件未及时清 0，则后续收到数据即使带有效 STOP 位，也不会把 LPUART_INTSR.FE 标志清 0。

27.1.6 多机通讯

Mode 2/3 具有多机通讯功能，为此在其帧格式中增加了 1 位 TB8/RB8。将 LPUART_SCON.SM2 置“1”，可开启多机通讯位。当开启多机通讯位后，发送数据时，主机可以通过 LPUART_SCON.TB8 来区分当前帧是地址帧(LPUART_SCON.TB8=1)还是数据帧(LPUART_SCON.TB8=0)。接收数据时，从机会忽略 RB8 位(第 9 位)为“0”的当前接收帧。当收到帧的 RB8 位(第 9 位)为“1”表明其是地址帧，从机会继续判断接收到的地址与其自身地址是否相等。如果匹配，则从机会对 LPUART_SCON.RB8 置“1”，并对 LPUART_INTSR.RI 置“1”，以表明该帧为地址帧并且地址已经匹配。从机软件看到 LPUART_SCON.RB8=1 并且 LPUART_INTSR.RI=1 后，先把 LPUART_SCON.SM2 位清“0”，然后准备接受给它的数据帧。如果地址不等，表明主机并不是寻址该从机，从机硬件保持 LPUART_SCON.RB8 和 LPUART_INTSR.RI 为“0”，软件保持 LPUART_SCON.SM2 位为“1”，从机继续处于监听状态。

27.1.7 自动地址识别

当开启多机通讯位后(LPUART_SCON.SM2 置“1”)，自动地址识别功能也将开启。该功能由硬件实现，使得从机可以检测接收到每个地址帧，如果该地址与从机地址匹配，接收端会给出 LPUART_INTSR.RI 接收标志。如果地址不匹配，则接收端不会给出任何接收标志。

如果有需要，也可以在 Mode 1 下开启多机通讯位，此时 TB8 位由 STOP 位代替。当从机接收到匹配的地址帧和有效的 STOP 位时，LPUART_INTSR.RI 会被置“1”。为了支持自动地址识别，定义了广播地址和给定地址的概念。

27.1.8 给定地址

LPUART 设备的 LPUART_SADDR 寄存器用来表示自己的设备给定地址，LPUART_SADEN 寄存器是地址掩码，可以用来定义地址中的无关位。当 LPUART_SADEN 的某一位为“0”，表示该位地址为无关位，也就是说在地址匹配过程中，该位地址不参与地址匹配。这些无关位增加了寻址的灵活性，使得主机可以同时寻址一个或者多个从机设备。注意，如果需要给出唯一匹配地址，LPUART_SADEN 寄存器必须设为 0xFF。

GIVENADDR = SADDR & SADEN

27.1.9 广播地址

广播地址是用来同时寻址所有从机设备的，一般广播地址为 0xFF。

BoardCastAddr = SADDR | SADEN

27.1.9.1 给定地址和广播地址举例

假设某从机的 LPUART_SADDR 和 LPUART_SADEN 配置如下：

SADDR: 0b01101001 SADEN: 0b11111011

那么其给定地址和广播地址如下： Given Address: 0b01101x01 BroadCast Address: 0b11111x11

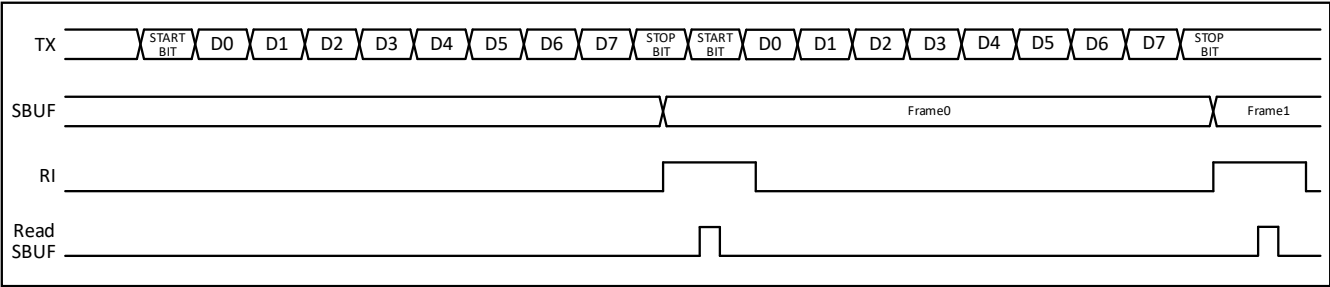
可见，主机可以用四个地址寻址到本从机，分别是：0b01101001 和 0b01101101(given address) 0b11111011 和 0b11111111(broadcast address)。

27.1.10 收发端缓存

27.1.10.1 接收缓存

LPUART 接收端有一个帧长度(8/9BITS)的接收缓存，也就是说当一帧数据接收完毕后，接收缓存中的数据会被一直保持，直到下一帧数据的 STOP 位接收完毕后，接收缓存才会更新为新一帧数据。

图27.8 接收缓存



27.1.10.2 发送缓存

LPUART 发送端不支持发送缓存。如果在发送数据过程中，填写 LPUART_SBUF 寄存器，将会屏蔽该写操作。软件应该避免这种操作。

27.2 寄存器列表

LPUART 基地址：0x4000 5000

偏移地址	名称	描述	复位值
0x00	LPUART_SBUF	数据寄存器	0x0000 0000
0x04	LPUART_SCON	控制寄存器	0x0000 E000
0x08	LPUART_SADDR	地址寄存器	0x0000 0000
0x0C	LPUART_SADEN	地址掩码寄存器	0x0000 0000
0x10	LPUART_INTSR	中断标志位寄存器	0x0000 0000
0x14	LPUART_INTCLR	中断标志位清除寄存器	0x0000 0000
0x18	LPUART_BAUDCR	波特率控制寄存器	0x0000 0000

27.3 寄存器说明

27.3.1 LPUART 数据寄存器（LPUART_SBUF）

偏移地址：0x00，复位值：0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								SBUF[7:0]							
-								R/W							

Bit	Field	Description	Reset	RW
31:8	Reserved	保留	0x0	rw

Bit	Field	Description	Reset	RW
7:0	SBUF	发送数据时，当发送数据写入该寄存器；接收数据时，数据接收完毕后，从该寄存器中读出。	0x0	R/W

27.3.2 LPUART 控制寄存器 (LPUART_SCON)

偏移地址：0x04，复位值：0x0000 E000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														FEEN	EN
-														R/W	R/W
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRSC[2:0]			SCLKSEL		LPMODE	DBAUD	TEEN	SM0:SM1		SM2	REN	TB8	RB8	TIEN	RIEN
R/W			R/W		R/W	R/W	R/W	R/W		R/W	R/W	R/W	R/W	R/W	R/W

Bit	Field	Description	Reset	RW
31:18	Reserved	保留	0x0	-
17	FEEN	错误帧中断使能 0: DISABLE 1: ENABLE	0x0	R/W
16	EN	Low-Power UART 工作使能 0: Low-Power UART 关闭，不接收/发送数据 1: Low-Power UART 使能，进行数据传输之前必须将该位置 1	0	R/W
15:13	PRSC	传输时钟 SCLK 预分频选择 000: DIV128; 001: DIV64; 010: DIV32; 011: DIV16; 100: DIV8; 101: DIV4; 110: DIV2; 111: DIV1。 PRSC 只有当 LPMODE=1 时有效；当 LPMODE=0 时，PRS 不会对 SCLK 预分频。	0x7	R/W
12:11	SCLKSEL	传输时钟 SCLK 选择 00 / 01: PCLK 10: LXT 11: LIRC	0x0	R/W
10	LPMODE	低功耗模式 0: 正常工作模式 1: 低功耗工作模式	0	R/W
9	DBAUD	双倍波特率 0: 单倍波特率 1: 双倍波特率	0	R/W
8	TEEN	发送缓存空中断使能 0: DISABLE 1: ENABLE	0	R/W

Bit	Field	Description	Reset	RW																									
7:6	SM0_SM1	工作模式； 00: Mode 0; 01: Mode 1; 10: Mode 2; 11: Mode 3 <table border="1"> <thead> <tr> <th>SM0</th><th>SM1</th><th>MODE</th><th>描述</th><th>波特率</th></tr> </thead> <tbody> <tr> <td>0</td><td>0</td><td>0</td><td>位移寄存器</td><td>PCLK/12</td></tr> <tr> <td>0</td><td>1</td><td>1</td><td>8 位串口传输</td><td>可变波特率</td></tr> <tr> <td>1</td><td>0</td><td>2</td><td>9 位串口传输</td><td>PCLK/32, PCLK/64</td></tr> <tr> <td>1</td><td>1</td><td>3</td><td>9 位串口传输</td><td>可变波特率</td></tr> </tbody> </table>	SM0	SM1	MODE	描述	波特率	0	0	0	位移寄存器	PCLK/12	0	1	1	8 位串口传输	可变波特率	1	0	2	9 位串口传输	PCLK/32, PCLK/64	1	1	3	9 位串口传输	可变波特率	0x0	R/W
SM0	SM1	MODE	描述	波特率																									
0	0	0	位移寄存器	PCLK/12																									
0	1	1	8 位串口传输	可变波特率																									
1	0	2	9 位串口传输	PCLK/32, PCLK/64																									
1	1	3	9 位串口传输	可变波特率																									
5	SM2	多主机通讯； 0: DISABLE, 1: ENABLE SM2: 软件配置多机通讯以及自动地址匹配模式 1: 启动多从机通讯以及地址自动匹配 0: 关闭多从机通讯以及地址自动匹配 在模式 2 和模式 3 中: 如果 SM2=1, 并且 REN=1, 则接收机处于地址帧监测模式, 可以使用接收到的第 9 位 RB8 来进行地址筛选。RB8=1 为地址帧, 通讯数据可以进入 SBUF, 置位 RI, 进入中断服务程序中进行地址比较; RB8=0 为数据帧, 接收机忽略这些数据帧并保持 RI=0 如果 SM2=0, 并且 REN=1, 则接收机不使用地址监测模式, 无论收到的 RB8 为 0 或 1, 都直接接收并且进入 SBUF, 置位 RI, RB8 在这种模式下为校验位。	0	R/W																									
4	REN	接收使能 Mode 0: 0: 发送, 1: 接收其他:	0	R/W																									
3	TB8	发送 TB8 位	0	R/W																									
2	RB8	接收 RB8 位	0	R/W																									
1	T1EN	接收完成中断使能 0: DISABLE 1: ENABLE	0	R/W																									
0	RIEN	接收完成中断使能 0: DISABLE 1: ENABLE	0	R/W																									

27.3.3 LPUART 地址寄存器 (LPUART_SADDR)

偏移地址: 0x08, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留								SADDR[7:0]							
-								R/W							

Bit	Field	Description	Reset	RW
31:8	Reserved	保留	0x0	-
7:0	SADDR	从机设备地址寄存器	0x0	R/W

27.3.4 LPUART 地址掩码寄存器 (LPUART_SADEN)

偏移地址: 0x0C, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留								SADEN[7:0]							
-								R/W							

Bit	Field	Description	Reset	RW
31:8	Reserved	保留	0x0	-
7:0	SADEN	从机设备地址掩码寄存器	0x0	R/W

27.3.5 LPUART 标志位寄存器 (LPUART_INTSR)

偏移地址: 0x10, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留												FE		TI	RI
-												RO		RO	RO

Bit	Field	Description	Reset	RW
31:4	Reserved	保留	0x0	-
3	TE	发送 FIFO 空标志位, 硬件置位, 软件清零 1:TE 中断有效 0:TE 中断无效	0	RO
2	FE	接收帧错误标志位, 硬件置位, 软件清零 1:FE 中断有效 0:FE 中断无效	0	RO
1	TI	发送完成中断标志位, 硬件置位, 软件清零 1:TI 中断有效 0:TI 中断无效	0	RO
0	RI	接收完成中断标志位, 硬件置位, 软件清零	0	RO

27.3.6 LPUART 标志位清除寄存器 (LPUART_INTCLR)

偏移地址: 0x14, 复位值: 0x0000 0000。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留												TECLR	FECLR	TICLR	RICLR
-												WO		WO	WO

Bit	Field	Description	Reset	RW
31:4	Reserved	保留	0x0	-
3	TECLR	清除发送空标志位; 写 1 清零, 写 0 无效	0	WO
2	FECLR	清除接收帧错误标志位; 写 1 清零, 写 0 无效	0	WO
1	TICLR	清除发送完成中断标志位; 写 1 清零, 写 0 无效	0	WO
0	RICLR	清除接收完成中断标志位; 写 1 清零, 写 0 无效	0	WO

27.3.7 LPUART 波特率控制寄存器 (LPUART_BAUDCR)

偏移地址: 0x18, 复位值: 0x0000 0000。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															SELF _BR G
-															R/W
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BRG[15:0]															
R/W															

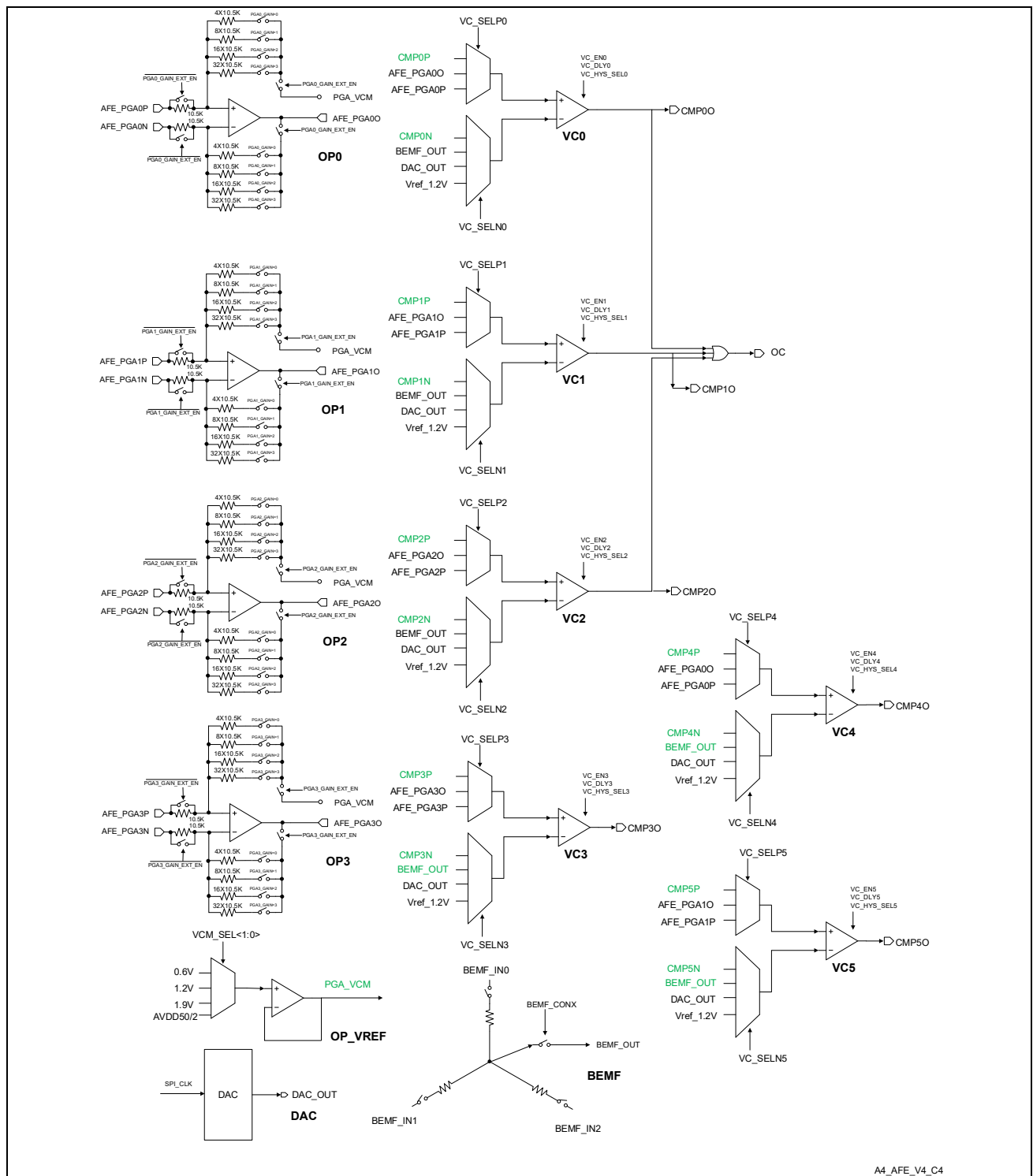
Bit	Field	Description	Reset	RW
31:9	Reserved	保留	0x0	-
16	SELF_BRG	LPUART 波特率选择位 0: LPUART 的波特率由 LPTIMER 产生 1: LPUART 的波特率由(DBAUD+1)*FSCLK/(32*(BRG[15:0]+1))生成	0	R/W
15:0	BRG	LPUART 自动波特率生成配置位: 波特率=(BAUD+1)*FSCLK/(32*(BRG[15:0]+1))	0x0	R/W

第六部分 模拟前端模块

28 模拟前端模块 (AFE)

M030 内包含一个 AFE 模块, 由 4 个 OP, 6 个 VC, 1 个 BEMF 和 DAC 组成。

图28.1 模拟前端模块 (AFE) 框图



A4_AFE_V4_C4

AFE 内部寄存器需要通过专用同步串行输入输出接口的接口进行配置，由芯片对应配置的 Firmware 接口实现。内部对应连接为：SCLK 对应 PB3，SCE 对应 PA15，SIO_I 对应 PB5，SIO_O 对应 PB4。

28.1 可编程运算放大器 (PGA)

可编程增益放大器，具有正向输入和反向输入及一个模拟信号输出。

AFE 中共有四组 PGA：PGA0、PGA1、PGA2 和 PGA3。

每个 PGA 均可单独使能，如果被禁止，则输出呈高阻态。

每个 PGA 单元能够对模拟输入进行四档 4x、8x、16x、32x 的放大倍数或者开环使用。

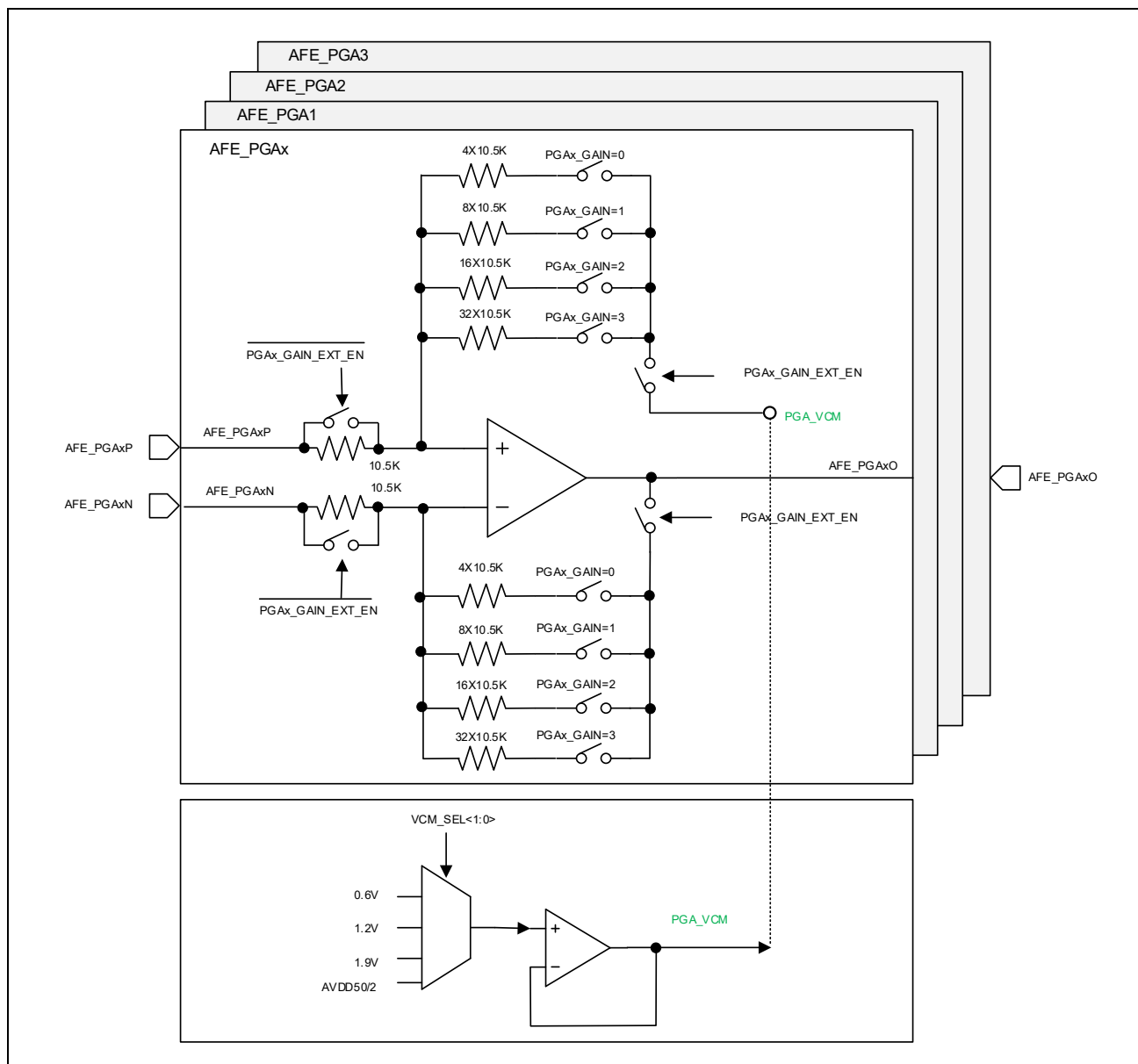
四组 PGA 共享 VCM 输入，可选择四档共模电平。

28.1.1 主要特性

- 低输入失调电压
- 10MHz 增益带宽
- 支持高的压摆率
- 共模电平可配置 (VCM_SEL)

28.1.2 PGA 结构框图

图28.2 PGA 结构框图



28.1.3 功能描述

PGAx_EN 位使能或禁止 PGA 单元工作。在使能 PGAx_EN 前, 应该先配置 PGA 相关寄存器, 如增益、外部模式等。

如果不再需要 PGA 工作, 则可禁止 PGA 以降低功耗。禁止 PGAx_EN 位时, 会保留所有配置不变。

28.1.4 初始配置

每个 PGA 单元均可单独使能, 使能后其处于默认开环功能模式, 在该模式下, 正负输入端口与输出端口连接到外部引脚。每个 PGA 单元的两个输入引脚和一个输出引脚可按照信号走线中的定义进行连接。

一旦寄存器中的 PGAx_EN 位置 1, 运算放大器即可工作。

注意:

运放输入输出引脚与 GPIO 在同一个 I/O Pad, 必须在相应 GPIO 中将输入和输出引脚配置为模拟模式 (默认状态)。

28.1.5 信号走线

运算放大器引脚走线由 PGA_REF_SELx 和 PGA_IN_SELx 寄存器位确定。

下表介绍了运算放大器 (PGAx) 的引脚连接。

表28.1 运算放大器 (PGAx) 的引脚连接表

信号	外部打线引脚	内部	注释
PGAx_VINN		PGA_REF_SELx = 0x0: OP 正端连接引脚 OPxN PGA_REF_SELx = 0x1: OP 正端内部悬空(Default)	PGAx_GAIN 和 PGA_REF_SELx 位控制该引脚。
PGAx_VINP		PGA_REF_INx = 0x0: OP 正端连接引脚 OPxP PGA_REF_INx = 0x1: OP 正端内部悬空(Default)	PGAx_GAIN 和 PGA_IN_SELx 位控制该引脚。
PGAx_VOUT		AFE_PGAXO to VCx_INP1	在 PGA 使能时连接该引脚。是否使用由 VCx 的输入信号配置位旋转

28.1.6 工作模式

放大器可用于多种配置环境：

- 开环模式（外部匹配阻抗而定）
- PGA 模式

注意：

放大器输出引脚直接连接到输出引脚的焊盘，以最大限度地降低输出阻抗。如果使能了放大器，那么即使放大器被配置为 PGA 并仅连接到内部通道，它的输出引脚也不能用作通用 I/O。

开环模式

下面介绍将 PGA 用作开环的步骤。

- [4] 将 PGAx_GAIN_EXT_EN 位配置为“1”；
- [5] 将 PGA_IN_SEL 位配置为“GPIO 连接到 PGAx_VINP”（为 0x0）。
- [6] 或将 PGA_REF_SEL 位配置为“GPIO 连接到 PGAx_VINN”（为 0x0）。

可编程增益放大器模式

下面介绍将 PGA 用作可编程增益放大器的步骤。

- [1] 将 PGAx_GAIN_EXT_EN 位配置为“0”，选择内部 PGA 放大模式。
- [2] 将 PGAx_GAIN 位配置为“内部增益 4x、8x、16x 或 32x（0x0 到 0x3）”四档选一。
- [3] 将 PGA_IN_SEL 位配置为“GPIO 连接到 PGAx_VINP”（为 0x0）；并将 PGA_REF_SEL 位配置为“GPIO 连接到 PGAx_VINN”（为 0x0）。
- [4] 通过配置共模电平选择 VCM_SEL 得到可编程运算放大器的输入参考电压 VCM。

一旦 PGAX_EN 位置 1，所选增益就会放大同相输入电压，并可在引脚 PGAx_VOUT 上测得放大后的电压。

注意： 为避免饱和，输入电压应保持低于：VDDA/G_{GAIN} 即 VDDA 除以所选增益。

$$VOUT = VCM + GAIN(VIN - VREF)$$

28.2 模拟前端中的电压比较器模块(AFE_VC)

28.2.1 简介

模拟电压比较器 VC 用于比较两个输入模拟电压的大小，并根据比较结果输出高/低电平。当同相输入端电压高于反相输入端电压时，电压比较器输出高电平；当同相输入端电压低于反相输入端电压时，电压比较器输出低电平。

模拟前端中的电压比较器模块支持 6 个 VC 单元，支持独立使用和组合支持 BEMF 使用。

每个 VC 单元的同相输入端均支持 3 路信号输入选择，反相输入端均支持 4 路信号输入选择。

每个 VC 单元可选支持 Hysteresis (0x0: 20mV, 0x1: 40mV, 0x2: 80mV, 0x3: 100mv)

每个 VC 单元可选支持模拟输入低通滤波器延迟控制 (低速模式或高速模式)

通过组合 VC0, VC1, VC2, 结合可编程运算放大器与内置 BEMF 可实现对于无刷直流电机的转子位置检测。

28.2.2 VC 框图

图28.3 VC 框图

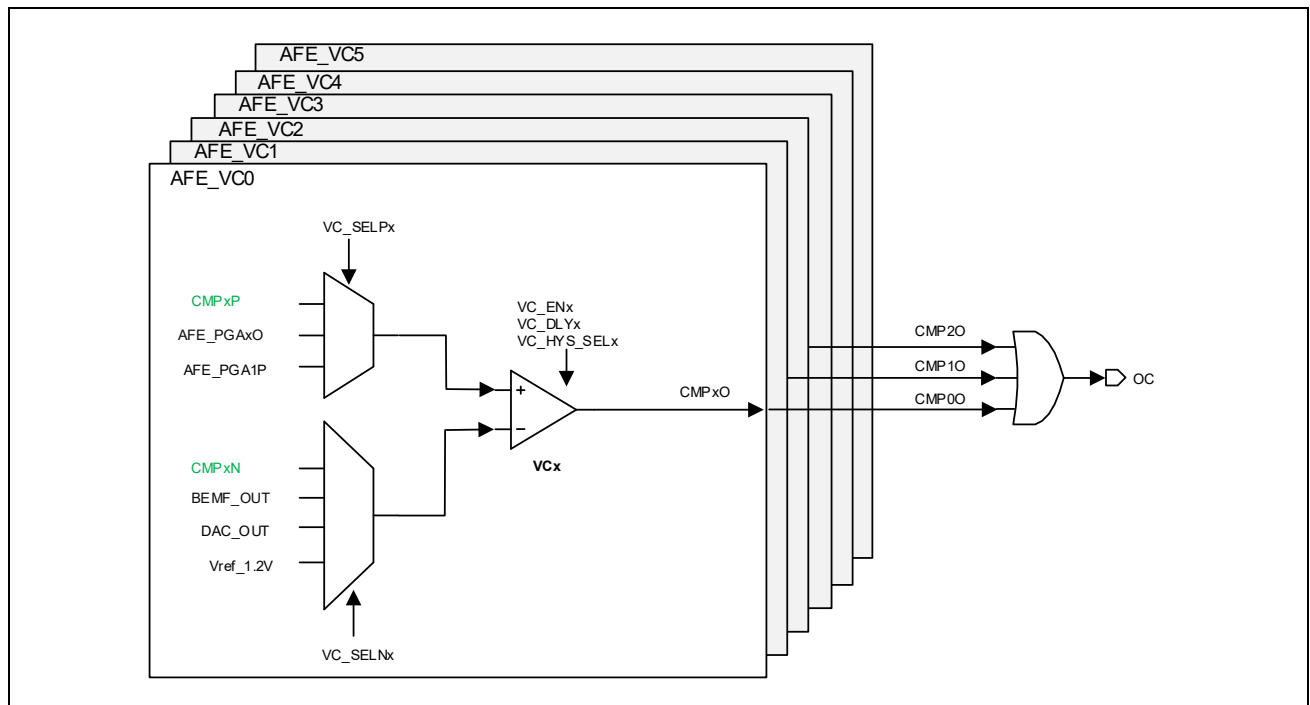
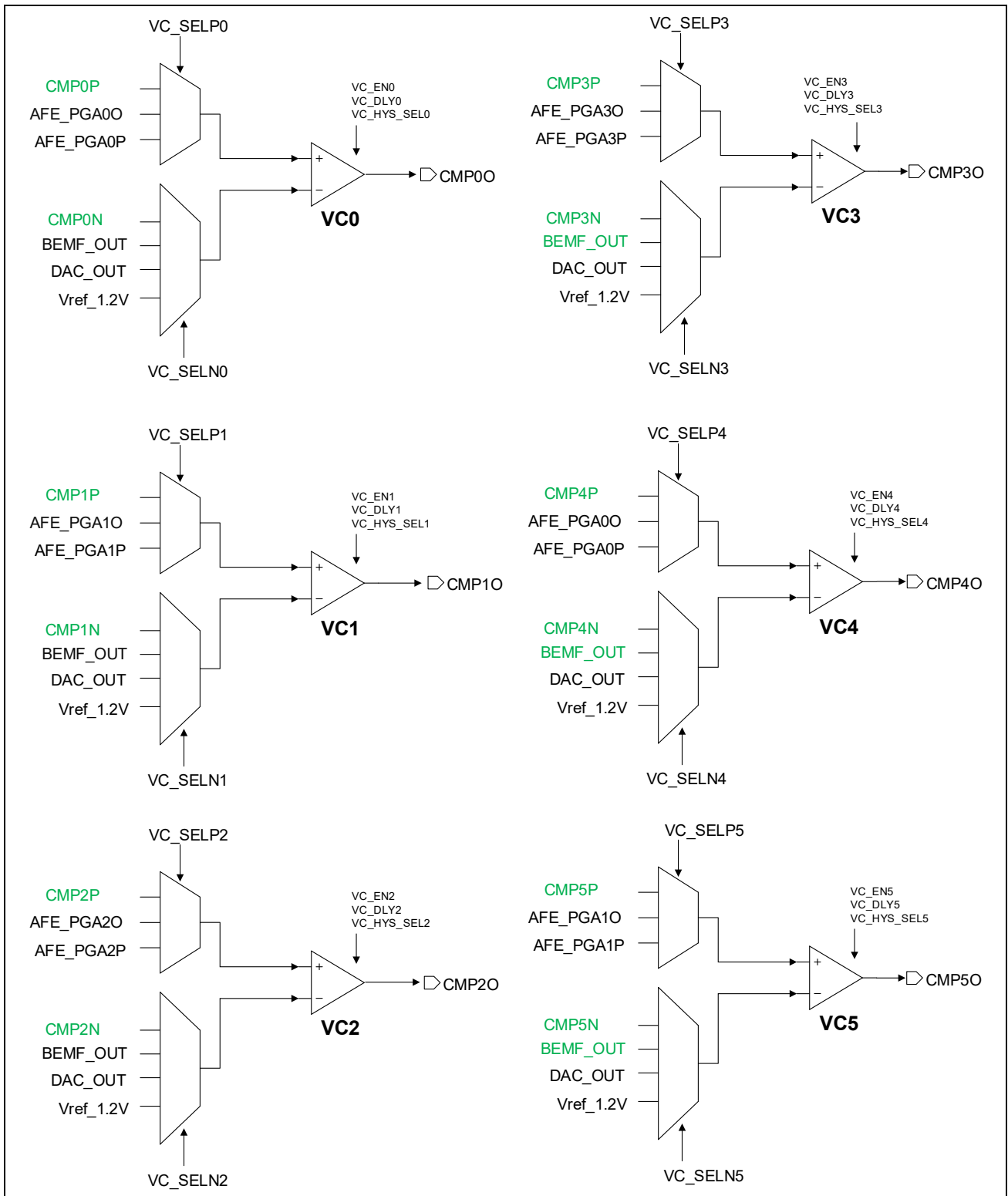


图28.4 6 个 VC 单元示意图

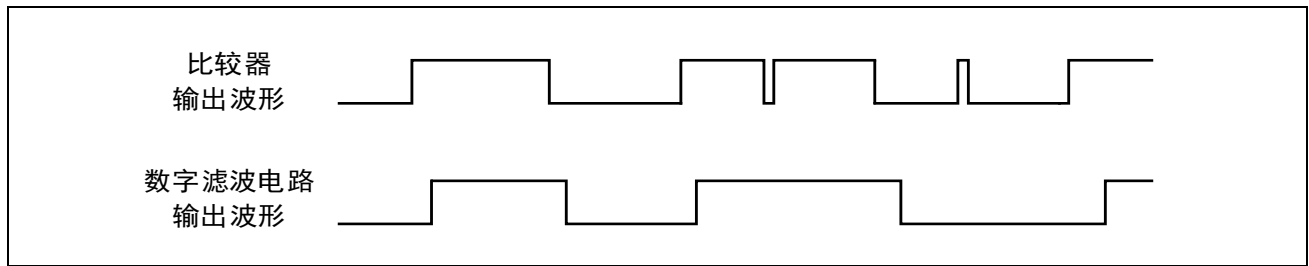


28.2.3 Hysteresis 设置与低通滤波设置

比较器具有可编程迟滞，可在有噪声信号时避免发生意外输出转换。迟滞可在不需要时禁止，这样就可以使用外部组件强制迟滞值。如果芯片的工作环境恶劣，迟滞比较器的输出会出现噪声信号。使能低通滤波模块，则迟滞比较器的输出波形中脉宽小于设定时间的噪声信号都可以被滤除。

禁止低通滤波模块，则低通滤波模块的输入输出信号相同。使能低通滤波模块，滤波示意如下所示：

图28.5 VC 滤波输出



28.2.4 VC 脚和内部信号

用作比较器输入的如是与通用 I/O 复用在一起时，因为是通过 Double Banding 在一起的，所以必须在 GPIO 寄存器中配置为模拟模式；用作比较器输入也是与 AFE 模块中的内置的 PGA (OP) 或 DAC 输出、AFE_VREF12V、BEMF_OUT，通过 VC_SEL Px 寄存器位和 VC_SEL N x 寄存器位选择。比较器输出可以在内部同时，可通过 Double Banding 功能连接到 I/O 引脚。

比较器输出更多详细信息，请参见产品数据手册。

28.2.5 VC 启动稳定时间

在 VC 使能（上电）后的一段时间内（定义为启动稳定时间），比较器参数及其输出均标注在数据手册。更多详细信息，请参见产品数据手册。

28.3 数模转换器 (DAC)

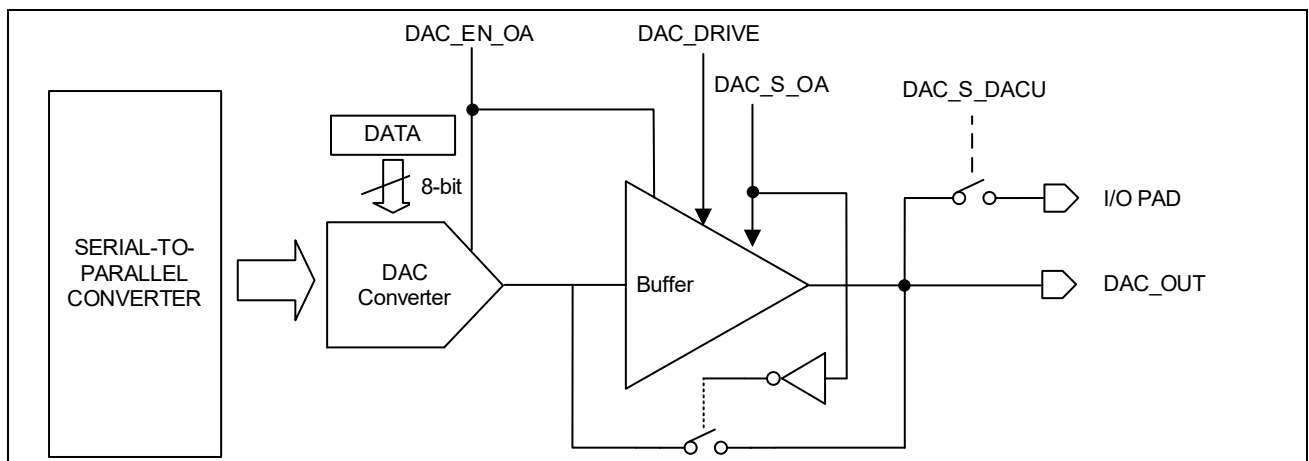
DAC 模块是 8 位电压输出数模转换器。DAC 可以按 8 位写入待转换数据。DAC 通过 1 个独立的数模转换器支持 1 个输出通道。

28.3.1 主要特性

- 一个 8 位 DAC 接口，该接口支持 1 个通道输出
- 模式下数据采用左对齐或右对齐
- DAC 输出通道缓冲/非缓冲模式
- 每路 DAC 输出均可与 DACx_OUTy 输出引脚断开连接
- DAC 输出可与片上外设连接

28.3.2 DAC 框图

图28.6 DAC 框图



28.3.3 功能描述

DACx_OUTy 引脚是与通用输入输出 (GPIO)做 Double Banding 的。

如果要使能 DAC 输出，此时如果是连接到外部 I/O Pad，必须配置对应的 I/O 中的 GPIO 为模拟输入。

DACx_OUT 配置输出模拟信号步骤：

- [1] 使能 DAC_EN_OA=1
- [2] 可配置 DAC_S_OA=1，使 DAC 输出至驱动模块。
- [3] 可选择使能 DAC 输出缓冲器 (DAC_DRIVE = 1)，从而获得 DAC 输出强驱动能力。
- [4] 最后写入输出数据到 DAC_DATA，输出模拟信号。

如需配置 DACx_OUT 对应 Double Banding 的 I/O Pad 为 GPIO，必须使 DAC_EN_OA=0 关闭 DAC 模块。

28.4 AFE 寄存器

28.4.1 AFE 寄存器列表

表28.2 AFE 寄存器表

Offset	Name	Description	Reset
0x0000	REG_00	reg_00 寄存器	0x00
0x0001	REG_01	reg_01 寄存器	0x00
0x0002	REG_02	reg_02 寄存器	0x00
0x0003	REG_03	reg_03 寄存器	0xFF
0x0004	REG_04	reg_04 寄存器	0xFF
0x0005	REG_05	reg_05 寄存器	0xFF
0x0006	REG_06	reg_06 寄存器	0x7F
0x0007	REG_07	reg_07 寄存器	0x00
0x0008	REG_08	reg_08 寄存器	0xA6
0x0009	REG_09	reg_09 寄存器	0xA6
0x000A	REG_10	reg_10 寄存器	0xA6
0x000B	REG_11	reg_11 寄存器	0x00
0x000E	REG_14	reg_14 寄存器	0x00

28.4.2 AFE 寄存器说明

28.4.2.1 reg_00 寄存器 (REG_00)

寄存器地址：0x0000

复位值：0x00

7	6	5	4	3	2	1	0
DAC_DATA							
RW							

Bit	Field	Description	Reset	RW
[7:0]	DAC_DATA	input data, DAC 输入数据	0x0	RW

28.4.2.2 reg_01 寄存器 (REG_01)

寄存器地址: 0x0001

复位值: 0x00

7	6	5	4	3	2	1	0
PGA_GAIN3		PGA_GAIN2		PGA_GAIN1		PGA_GAIN0	
RW		RW		RW		RW	

Bit	Field	Description	Reset	RW
[7:6]	PGA_GAIN3	PGA3 放大倍数 0x0: 4 倍 0x1: 8 倍 0x2: 16 倍 0x3: 32 倍	0x0	RW
[5:4]	PGA_GAIN2	PGA2 放大倍数 0x0: 4 倍 0x1: 8 倍 0x2: 16 倍 0x3: 32 倍	0x0	RW
[3:2]	PGA_GAIN1	PGA1 放大倍数 0x0: 4 倍 0x1: 8 倍 0x2: 16 倍 0x3: 32 倍	0x0	RW
[1:0]	PGA_GAIN0	PGA0 放大倍数 0x0: 4 倍 0x1: 8 倍 0x2: 16 倍 0x3: 32 倍	0x0	RW

28.4.2.3 reg_02 寄存器 (REG_02)

寄存器地址: 0x0002

复位值: 0x00

7	6	5	4	3	2	1	0
RESERVED			f030_init_done	f030_init_done			
RO			RO	RW			

Bit	Field	Description	Reset	RW
[4]	f030_init_done	f030_init_done 0x0: 030 输出到模拟的信号隔离为 0; 0x1: 030 信号直接输出到模拟。		RO
[3:0]	f030_init_done	f030_init_done: 写 0x9 代表 030 初始化完成, 030 信号输出给模拟; 写 0x3 退出, 输出到模拟信号被隔离为 0。		RW

28.4.2.4 reg_03 寄存器 (REG_03)

寄存器地址: 0x0003

复位值: 0xFF

7	6	5	4	3	2	1	0
PGA_IN_SEL3	PGA_IN_SEL2	PGA_IN_SEL1	PGA_IN_SEL0	PGA_REF_SEL3	PGA_REF_SEL2	PGA_REF_SEL1	PGA_REF_SEL0
RW	RW	RW	RW	RW	RW	RW	RW

Bit	Field	Description	Reset	RW
[7]	PGA_IN_SEL3	PGA3 输入电压选择 0x0:OP 正端连接引脚 OPxP 0x1:OP 正端内部悬空	0x1	RW
[6]	PGA_IN_SEL2	PGA2 输入电压选择 0x0:OP 正端连接引脚 OPxP 0x1:OP 正端内部悬空	0x1	RW
[5]	PGA_IN_SEL1	PGA1 输入电压选择 0x0:OP 正端连接引脚 OPxP 0x1:OP 正端内部悬空	0x1	RW
[4]	PGA_IN_SEL0	PGA0 输入电压选择 0x0:OP 正端连接引脚 OPxP 0x1:OP 正端内部悬空	0x1	RW
[3]	PGA_REF_SEL3	PGA3 参考电压选择 0x0:OP 负端连接引脚 OPxP 0x1:OP 负端内部悬空	0x1	RW
[2]	PGA_REF_SEL2	PGA2 参考电压选择 0x0:OP 负端连接引脚 OPxP 0x1:OP 负端内部悬空	0x1	RW
[1]	PGA_REF_SEL1	PGA1 参考电压选择 0x0:OP 负端连接引脚 OPxP 0x1:OP 负端内部悬空	0x1	RW
[0]	PGA_REF_SEL0	PGA0 参考电压选择 0x0:OP 负端连接引脚 OPxP 0x1:OP 负端内部悬空	0x1	RW

28.4.2.5 reg_04 寄存器 (REG_04)

寄存器地址: 0x0004

复位值: 0xFF

7	6	5	4	3	2	1	0
PGA_EN3	PGA_EN2	PGA_EN1	PGA_EN0	PGA_GAIN_EXT_EN3	PGA_GAIN_EXT_EN2	PGA_GAIN_EXT_EN1	PGA_GAIN_EXT_EN0
RW	RW	RW	RW	RW	RW	RW	RW

Bit	Field	Description	Reset	RW
[7]	PGA_EN3	PGA3 使能 0: 关闭 1: 开启	0x1	RW
[6]	PGA_EN2	PGA2 使能 0: 关闭 1: 开启	0x1	RW
[5]	PGA_EN1	PGA1 使能 0: 关闭 1: 开启	0x1	RW
[4]	PGA_EN0	PGA0 使能 0: 关闭 1: 开启	0x1	RW
[3]	PGA_GAIN_EXT_EN3	PGA3 外部配置放大使能 0x0: PGA 放大倍数使用内部提供的四档 (见 PGA_GAIN3) 0x1: 屏蔽内部放大档位, 用户可以自己在外部输入输出 PAD 上配置放大倍数或者开环使用	0x1	RW

Bit	Field	Description	Reset	RW
[2]	PGA_GAIN_EXT_EN2	PGA2 外部配置放大使能 0x0: PGA 放大倍数使用内部提供的四档 (见 PGA_GAIN2) 0x1: 屏蔽内部放大档位, 用户可以自己在外部输入输出 PAD 上配置放大倍数或者开环使用	0x1	RW
[1]	PGA_GAIN_EXT_EN1	PGA1 外部配置放大使能 0x0: PGA 放大倍数使用内部提供的四档 (见 PGA_GAIN1) 0x1: 屏蔽内部放大档位, 用户可以自己在外部输入输出 PAD 上配置放大倍数或者开环使用	0x1	RW
[0]	PGA_GAIN_EXT_EN0	PGA0 外部配置放大使能 0x0: PGA 放大倍数使用内部提供的四档 (见 PGA_GAIN0) 0x1: 屏蔽内部放大档位, 用户可以自己在外部输入输出 PAD 上配置放大倍数或者开环使用	0x1	RW

28.4.2.6 reg_05 寄存器 (REG_05)

寄存器地址: 0x0005

复位值: 0xFF

7	6	5	4	3	2	1	0
VC_HYS_SEL5		VC_HYS_SEL4		VC_HYS_SEL3		VC_HYS_SEL2	
RW		RW		RW		RW	

Bit	Field	Description	Reset	RW
[7:6]	VC_HYS_SEL5	Hysteresis option select of VC5: 0x0: 20mv (default) 0x1: 40mv 0x2: 80mv 0x3: 100mv	0x3	RW
[5:4]	VC_HYS_SEL4	Hysteresis option select of VC4: 0x0: 20mv (default) 0x1: 40mv 0x2: 80mv 0x3: 100mv	0x3	RW
[3:2]	VC_HYS_SEL3	Hysteresis option select of VC3: 0x0: 20mv (default) 0x1: 40mv 0x2: 80mv 0x3: 100mv	0x3	RW
[1:0]	VC_HYS_SEL2	Hysteresis option select of VC2: 0x0: 20mv (default) 0x1: 40mv 0x2: 80mv 0x3: 100mv	0x3	RW

28.4.2.7 reg_06 寄存器 (REG_06)

寄存器地址: 0x0006

复位值: 0x7F

7	6	5	4	3	2	1	0
VC_HYS_SEL1		VC_HYS_SEL0		VCM_SEL<1:0>		BEMF_CONX	DAC_SEL
RW		RW		RW		RW	RW

Bit	Field	Description	Reset	RW
[7:6]	VC_HYS_SEL1	Hysteresis option select of VC1: 0x0: 20mv (default) 0x1: 40mv 0x2: 80mv 0x3: 100mv	0x1	RW
[5:4]	VC_HYS_SEL0	Hysteresis option select of VC0: 0x0: 20mv (default) 0x1: 40mv 0x2: 80mv 0x3: 100mv	0x3	RW
[3:2]	VCM_SEL<1:0>	共模电平选择 0x0: 1.9V 0x1: AVDD50/2 0x2: 0.6V 0x3: 1.2V	0x3	RW
[1]	BEMF_CONX	MID 过零检测开关 0x0: 关闭 (外部电阻分压) 0x1: 开启 (内部电阻分压)	0x1	RW
[0]	DAC_SEL	DAC 输出电压范围选择 0x0: 5V 0x1: 1.2V	0x1	RW

28.4.2.8 reg_07 寄存器 (REG_07)

寄存器地址: 0x0007

复位值: 0x00

7	6	5	4	3	2	1	0
VC_SEL5		VC_SEL4		VC_SEL3		VC_SEL2	
RW		RW		RW		RW	

Bit	Field	Description	Reset	RW
[7:6]	VC_SEL5	Analog positive input selection of VC5 0x0:OPxP(Default) 0x1:OPxO 0x2:CMPPxP 0x3: not use	0x0	RW
[5:4]	VC_SEL4	Analog positive input selection of VC4 0x0:OPxP(Default) 0x1:OPxO 0x2:CMPPxP 0x3: not use	0x0	RW
[3:2]	VC_SEL3	Analog positive input selection of VC3 0x0:OPxP(Default) 0x1:OPxO 0x2:CMPPxP 0x3: not use	0x0	RW
[1:0]	VC_SEL2	Analog positive input selection of VC2 0x0:OPxP(Default) 0x1:OPxO 0x2:CMPPxP 0x3: not use	0x0	RW

28.4.2.9 reg_08 寄存器 (REG_08)

寄存器地址: 0x0008

复位值: 0xA6

7	6	5	4	3	2	1	0
VC_SEL1		VC_SEL0		DAC_DRIVE	DAC_EN_OA	DAC_S_DACU	DAC_S_OA
RW		RW		RW	RW	RW	RW

Bit	Field	Description	Reset	RW
[7:6]	VC_SEL1	Analog positive input selection of VC1 0x0:OPxP(default) 0x1:OPxO 0x2:CMPPxP 0x3: not use	0x2	RW
[5:4]	VC_SEL0	Analog positive input selection of VC0 0x0:OPxP(default) 0x1:OPxO 0x2:CMPPxP 0x3: not use	0x2	RW
[3]	DAC_DRIVE	high enable OA high drive level, DAC 输出驱动能力开关 0x0: DAC 输出弱驱动能力 0x1: DAC 输出强驱动能力	0x0	RW
[2]	DAC_EN_OA	high enable OA DAC 驱动输出使能 0x0: 开启 0x1: 关闭	0x1	RW
[1]	DAC_S_DACU	high close DAC output to PAD	0x1	RW
[0]	DAC_S_OA	high open DAC output to OA input 0x0: DAC 输出隔离 0x1: DAC 输出至驱动模块	0x0	RW

28.4.2.10 reg_09 寄存器 (REG_09)

寄存器地址: 0x0009

复位值: 0xA6

7	6	5	4	3	2	1	0
VC_SELN5		VC_SELN4		VC_SELN3		VC_SELN2	
RW		RW		RW		RW	

Bit	Field	Description	Reset	RW
[7:6]	VC_SELN5	Analog negative input of reference selection of VC5 0x0:1.2V(Default) 0x1:DAC_OUT 0x2:BRME_MID 0x3:CMPPxN	0x2	RW
[5:4]	VC_SELN4	Analog negative input of reference selection of VC4 0x0:1.2V(Default) 0x1:DAC_OUT 0x2:BRME_MID 0x3:CMPPxN	0x2	RW
[3:2]	VC_SELN3	Analog negative input of reference selection of VC3 0x0:1.2V(Default) 0x1:DAC_OUT 0x2:BRME_MID 0x3:CMPPxN	0x1	RW
[1:0]	VC_SELN2	Analog negative input of reference selection of VC2 0x0:1.2V(Default) 0x1:DAC_OUT 0x2:BRME_MID 0x3:CMPPxN	0x2	RW

28.4.2.11 reg_10 寄存器 (REG_10)

寄存器地址: 0x000A

复位值: 0xA6

7	6	5	4	3	2	1	0
VC_SELN1		VC_SELN0		VC_DLY5	VC_DLY4	VC_DLY3	VC_DLY2
RW		RW		RW	RW	RW	RW

Bit	Field	Description	Reset	RW
[7:6]	VC_SELN1	Analog negative input of reference selection of VC1 0x0:1.2V(Default) 0x1:DAC_OUT 0x2:BRME_MID 0x3:CMpN	0x2	RW
[5:4]	VC_SELN0	Analog negative input of reference selection of VC0 0x0:1.2V(Default) 0x1:DAC_OUT 0x2:BRME_MID 0x3:CMpN	0x2	RW
[3]	VC_DLY5	analog input low pass filter delay control of VC5 0x0: slow speed mode(Default) 0x1: high speed mode	0x0	RW
[2]	VC_DLY4	analog input low pass filter delay control of VC4 0x0: slow speed mode(Default) 0x1: high speed mode	0x1	RW
[1]	VC_DLY3	analog input low pass filter delay control of VC3 0x0: slow speed mode(Default) 0x1: high speed mode	0x1	RW
[0]	VC_DLY2	analog input low pass filter delay control of VC2 0x0: slow speed mode(Default) 0x1: high speed mode	0x0	RW

28.4.2.12 reg_11 寄存器 (REG_11)

寄存器地址: 0x000B

复位值: 0x00

7	6	5	4	3	2	1	0
VC_DLY1	VC_DLY0	VC_EN5	VC_EN4	VC_EN3	VC_EN2	VC_EN1	VC_EN0
RW	RW	RW	RW	RW	RW	RW	RW

Bit	Field	Description	Reset	RW
[7]	VC_DLY1	analog input low pass filter delay control of VC1 0x0: slow speed mode(Default) 0x1: high speed mode	0x0	RW
[6]	VC_DLY0	analog input low pass filter delay control of VC0 0x0: slow speed mode(Default) 0x1: high speed mode	0x0	RW
[5]	VC_EN5	VC5 使能 0x0: 关闭(Default) 0x1: 开启	0x0	RW

Bit	Field	Description	Reset	RW
[4]	VC_EN4	VC4 使能 0x0: 关闭(Default) 0x1: 开启	0x0	RW
[3]	VC_EN3	VC3 使能 0x0: 关闭(Default) 0x1: 开启	0x0	RW
[2]	VC_EN2	VC2 使能 0x0: 关闭(Default) 0x1: 开启	0x0	RW
[1]	VC_EN1	VC1 使能 0x0: 关闭(Default) 0x1: 开启	0x0	RW
[0]	VC_EN0	VC0 使能 0x0: 关闭(Default) 0x1: 开启	0x0	RW

28.4.2.13 reg_14 寄存器 (REG_14)

寄存器地址: 0x000E

复位值: 0x00

7	6	5	4	3	2	1	0
RESERVED						MID_RES	
RO						RW	

Bit	Field	Description	Reset	RW
[1:0]	MID_RES	反电动势电阻大小配置 0x0:30K 0x1:8.2K 0x2:4.7K 0x3:1K	0x0	RW

第七部分 版本与目录索引

29 版本记录 (REVISION)

表29.1 版本记录

版本	修订日期	修订内容摘要
0.0	2024/11/13	初版
0.1	2024/12/16	Update reg_01
0.2	2024/12/24	Update 寄存器描述
0.3	2025/1/7	Update ADC_INTR 描述
0.4	2025/4/8	Update 定时器位数描述
0.5	2025/4/24	添加定时器位 ITR 表
0.6	2025/7/23	格式修改
0.7	2025/9/24	修改 AFE 寄存器位
0.8	2025/10/29	修改 HSI 支持时钟为仅 8MHz，及 RCMU、TIMx 寄存器位描述
0.9	2025/11/7	修改 AFE 模块描述和功能框图

30 文档目录

第一部分	内核与架构	2
1	存储器和总线架构	2
1.1	系统架构	2
1.2	存储架构	2
1.3	地址分配	2
1.3.1	内嵌 SRAM	3
1.3.2	嵌入式闪存	4
1.3.3	启动配置	4
2	SRAM_ECC 控制器 (SRAM_ECC)	5
2.1	SRAM 控制	5
2.1.1	简介	5
2.2	SRAM 寄存器	5
2.2.1	SRAM 控制寄存器 (SRAM_CR)	5
2.2.2	SRAM 状态寄存器 (SRAM_SR)	5
3	嵌入式闪存 (FLASH)	6
3.1	嵌入式闪存	6
3.1.1	闪存介绍	6
3.1.2	闪存结构	6
3.1.3	读操作	8
3.1.4	编程擦除控制器	8
3.1.5	存储保护	13
3.1.6	选项字节区描述	14
3.2	闪存寄存器描述	16
3.2.1	闪存访问控制寄存器 (FLASH_ACR)	16
3.2.2	闪存控制器密钥寄存器 (FLASH_KEYR)	16
3.2.3	闪存选项字节密钥寄存器 (FLASH_OPTKEYR)	17
3.2.4	闪存状态寄存器 (FLASH_SR)	17
3.2.5	闪存控制寄存器 (FLASH_CR)	17
3.2.6	闪存地址寄存器 (FLASH_AR)	18
3.2.7	字节选项字节寄存器 (FLASH_OBR)	18
3.2.8	闪存写保护寄存器 (FLASH_WRP)	19
3.2.9	闪存操作模式寄存器 (FLASH_OP_MODE)	20
4	复位和时钟控制单元 (RCMU)	21
4.1	复位	21
4.1.1	系统复位	21
4.1.2	WWDG 复位	21
4.1.3	IWDG 复位	21
4.1.4	NRST 复位	21
4.1.5	软件复位	22
4.1.6	低功耗模式复位	22
4.1.7	CPU 锁死复位	22
4.1.8	电压比较复位	22
4.1.9	电源复位	22
4.2	时钟	22
4.2.1	HSE 时钟	23
4.2.2	HSI 时钟	24
4.2.3	锁相环 (PLL)	24
4.2.4	LSE 时钟	24
4.2.5	LSI 时钟	24
4.2.6	系统时钟选择 (SYSCLK)	24
4.2.7	时钟安全系统 (CSS)	25
4.2.8	RTC 时钟	25
4.2.9	看门狗时钟	25
4.2.10	时钟输出 (MCO)	25
4.3	寄存器	25
4.3.1	时钟控制寄存器 (RCMU_CR)	25
4.3.2	时钟配置寄存器 (RCMU_CFGR)	26
4.3.3	时钟中断寄存器 (RCMU_CIR)	28
4.3.4	APB2 外设复位寄存器 (RCMU_APB2RSTR)	29

4.3.5	APB1 外设复位寄存器 (RCMU_APB1RSTR)	30
4.3.6	AHB 外设时钟使能寄存器 (RCMU_AHBENR)	32
4.3.7	APB2 外设时钟使能寄存器 (RCMU_APB2ENR)	33
4.3.8	APB1 外设时钟使能寄存器 (RCMU_APB1ENR)	34
4.3.9	备份控制寄存器 (RCMU_BDCR)	35
4.3.10	控制/状态寄存器 (RCMU_CSR)	36
4.3.11	AHB 外设复位寄存器 (RCMU_AHBRSTR)	37
4.3.12	时钟配置寄存器 2 (RCMU_CFGR2)	38
4.3.13	PLL 配置寄存器 (RCMU_PLLCR)	38
4.3.14	PLL 配置寄存器 2 (RCMU_PLLCR2)	39
4.3.15	ST 寄存器 (RCMU_STCR)	40
4.3.16	中断延迟配置寄存器 (RCMU_IRQLATENCY)	40
5	电源管理单元 (PMU)	41
5.1	电源模块 (PMU)	41
5.1.1	功能描述	41
5.1.2	电源	41
5.1.3	电源管理器	41
5.1.4	低功耗模式	42
5.1.5	电源控制寄存器	47
6	中断和事件 (IEMU)	50
6.1	中断和事件 (IEMU)	50
6.1.1	嵌套向量中断控制器	50
6.1.2	系统节拍 (systick) 校准值寄存器	50
6.1.3	中断和异常向量列表	50
6.1.4	外部中断/事件控制器 (EXTI)	51
6.1.5	中断事件管理	51
6.1.6	中断产生	51
6.1.7	事件产生	53
6.1.8	软件中断/事件产生	53
6.1.9	外部中断/事件线映射	53
6.2	寄存器	54
6.2.1	中断屏蔽寄存器 (EXTI_IMR)	54
6.2.2	事件屏蔽寄存器 (EXTI_EMR)	54
6.2.3	上升沿触发选择寄存器 (EXTI_RTSTR)	54
6.2.4	下降沿触发选择寄存器 (EXTI_FTSTR)	55
6.2.5	软件中断事件寄存器 (EXTI_SWIER)	55
6.2.6	挂起寄存器 (EXTI_PR)	55
7	直接存储访问 (DMA)	57
7.1	直接存储访问 (DMA)	57
7.1.1	介绍	57
7.1.2	主要特征	57
7.1.3	外设列表	57
7.1.4	使用示例	57
7.2	DMA 寄存器列表	58
7.2.1	DMA 可屏蔽通道的总中断寄存器 (DMA_INTSTA)	58
7.2.2	DMA 传输中断完成寄存器 (DMA_INTTCSTA)	58
7.2.3	DMA 中断清除寄存器 (DMA_INCTCCLR)	58
7.2.4	DMA 可屏蔽错误中断寄存器 (DMA_INTERRSTA)	59
7.2.5	DMA 错误中断清除寄存器 (DMA_INTERRCLR)	59
7.2.6	DMA 通道不可屏蔽中断寄存器 (DMA_RAWINTTCSTA)	59
7.2.7	DMA 不可屏蔽错误中断寄存器 (DMA_RAWINTERRSTA)	60
7.2.8	DMA 通道工作状态寄存器 (DMA_ENBLDCHNSTA)	60
7.2.9	DMA 单次数据搬移寄存器 (DMA_SOFTSREQ)	60
7.2.10	DMA 过载寄存器 (DMA_OVSTA)	61
7.2.11	DMA 溢出清除寄存器 (DMA_OVCLR)	61
7.2.12	DMA 配置寄存器 (DMA_CONFIG)	61
7.2.13	DMA 外设选择寄存器 (DMA_PERIPHx_SEL) (x=0/1/..5)	62
7.2.14	通道源地址寄存器 (DMA_CHx_SRCADDR) (x=0/1/..5)	62
7.2.15	通道目标地址寄存器 (DMA_CHx_DESTADDR) (x=0/1/..5)	62
7.2.16	通道控制寄存器 (DMA_CHx_CONTROL) (x=0/1/..5)	63
7.2.17	通道配置寄存器 (DMA_CHx_CONFIG) (x=0/1/..5)	63
8	调试支持 (DBGMCU)	65
8.1	调试支持 (DBGMCU)	65
8.1.1	简介	65
8.1.2	SW 调试端口	65

	8.1.3	ID 代码和锁定机制	65
8.2	寄存器描述		65
	8.2.1	调试 ID 寄存器 (DBGMCU_ID)	65
	8.2.2	调试控制寄存器 (DBGMCU_CTL)	66
	8.2.3	调试预留寄存器 (DBGMCU_RESERVED)	67
	8.2.4	调试开关寄存器 (DBGMCU_DBGEN)	67
第二部分	IO 配置		68
9	通用输入输出 (GPIO)		68
9.1	IO 功能 (IO)		68
	9.1.1	功能描述	68
9.2	寄存器描述		72
	9.2.1	主模式配置寄存器 (GPRx_MODER) (x=A/B/C/D)	72
	9.2.2	推挽/开漏模式配置寄存器 (GPRx_OTYPER) (x=A/B/C/D)	73
	9.2.3	输出速度配置寄存器 (GPRx_OSPEEDER) (x=A/B/C/D)	74
	9.2.4	上拉下拉配置寄存器 (GPRx_PUPDER) (x=A/B/C/D)	76
	9.2.5	输入观测寄存器 (GPRx_IDR) (x=A/B/C/D)	78
	9.2.6	输出配置寄存器 (GPRx_ODR) (x=A/B/C/D)	78
	9.2.7	输出逻辑位配置 0/1 寄存器 (GPRx_BSRR) (x=A/B/C/D)	79
	9.2.8	锁定配置寄存器 (GPRx_LCKR) (x=A/B/C/D)	81
	9.2.9	复用功能配置寄存器低 (GPRx_AFR1) (x=A/B/C/D)	82
	9.2.10	复用功能配置寄存器高 (GPRx_AFR2) (x=A/B/C/D)	83
	9.2.11	输出逻辑位配置 0 寄存器 (GPRx_BRR) (x=A/B/C/D)	84
	9.2.12	输出驱动配置寄存器 (GPRx_SR) (x=A/B/C/D)	85
第三部分	控制系统		87
10	系统配置 (SYSCFG)		87
10.1	系统配置 (SYSCFG)		87
	10.1.1	功能描述	87
10.2	复用寄存器		87
	10.2.1	系统配置寄存器 1 (SYSCFG_CFGR1)	87
	10.2.2	系统配置寄存器 2 (SYSCFG_CFGR2)	87
	10.2.3	外部中断配置寄存器 1 (SYSCFG_EXTICR1)	88
	10.2.4	外部中断配置寄存器 2 (SYSCFG_EXTICR2)	89
	10.2.5	外部中断配置寄存器 3 (SYSCFG_EXTICR3)	89
	10.2.6	外部中断配置寄存器 4 (SYSCFG_EXTICR4)	90
	10.2.7	红外输出调制配置寄存器 (SYSCFG_IRCR)	90
11	独立看门狗 (IWDG)		92
11.1	独立看门狗 (IWDG)		92
	11.1.1	简介	92
	11.1.2	主要性能	92
	11.1.3	功能描述	92
11.2	寄存器		93
	11.2.1	密钥寄存器 (IWDG_KR)	93
	11.2.2	预分寄存器 (IWDG_PR)	94
	11.2.3	重装载寄存器 (IWDG_RLR)	94
	11.2.4	状态寄存器 (IWDG_SR)	95
12	窗口看门狗 (WWDG)		96
12.1	窗口看门狗 (WWDG)		96
	12.1.1	WWDG 简介	96
	12.1.2	WWDG 主要特性	96
	12.1.3	WWDG 功能描述	96
	12.1.4	编写超时程序	96
	12.1.5	工作模式	98
12.2	寄存器		98
	12.2.1	寄存器 (WWDG_CR)	98
	12.2.2	寄存器 (WWDG_CFR)	99
	12.2.3	寄存器 (WWDG_SR)	99
13	循环冗余校验计算单元 (CRC)		101
13.1	循环冗余校验计算单元 (CRC)		101
	13.1.1	CRC 功能描述	101
	13.1.2	CRC 主要特性	101
13.2	CRC 寄存器		101
	13.2.1	CRC 数据寄存器 (CRC_DR)	101



	13.2.2	CRC 独立数据寄存器 (CRC_IDR)	102
	13.2.3	CRC 控制寄存器 (CRC_CR)	102
	13.2.4	CRC 初始寄存器 (CRC_INIT)	102
14		随机数发生器 (TRNG)	103
	14.1	随机数发生器 (TRNG)	103
	14.1.1	功能描述	103
	14.2	寄存器	103
	14.2.1	TRNG 控制寄存器 (TRNG_CR)	103
	14.2.2	TRNG 状态寄存器 (TRNG_SR)	103
	14.2.3	TRNG 数据寄存器 (TRNG_DR)	104
15		AES 硬件加速器 (AES)	106
	15.1	AES 硬件加速器	106
	15.1.1	AES 算法	106
	15.1.2	主要特点	106
	15.1.3	功能描述	106
	15.1.4	密钥派生	107
	15.1.5	AES 链接模式	107
	15.1.6	数据格式转换	109
	15.1.7	使用流程	109
	15.1.8	DMA 接口	111
	15.2	AES 寄存器	111
	15.2.1	AES 控制寄存器 (AES_CR)	111
	15.2.2	AES 状态寄存器 (AES_SR)	113
	15.2.3	AES 数据输入寄存器 (AES_DINR)	114
	15.2.4	AES 数据输出寄存器 (AES_OUTR)	114
	15.2.5	AES 关键寄存器 0 (AES_KEYR0) (key[31:0])	114
	15.2.6	AES 关键寄存器 1 (AES_KEYR1) (LSB: key[63:32])	115
	15.2.7	AES 关键寄存器 2 (AES_KEYR2) (key[95:64])	115
	15.2.8	AES 关键寄存器 3 (AES_KEYR3) (key[127:96])	115
	15.2.9	AES 初始化向量寄存器 0 (AES_IVR0) (IVR[31:0])	115
	15.2.10	AES 初始化向量寄存器 1 (AES_IVR1) (IVR[63:32])	116
	15.2.11	AES 初始化向量寄存器 2 (AES_IVR2) (IVR[95:64])	116
	15.2.12	AES 初始化向量寄存器 3 (AES_IVR3) (IVR[127:96])	116
16		定时器 (TIMX)	117
	16.1	定时器 (TIMX)	117
	16.1.1	功能描述	117
	16.1.2	时基单元	123
	16.1.3	递增模式	123
	16.1.4	递减模式	124
	16.1.5	中央对齐模式	124
	16.1.6	重复计数器	126
	16.1.7	时钟选择	126
	16.1.8	输入捕获模式	129
	16.1.9	输出捕获模式	131
	16.1.10	外部触发同步	143
	16.1.11	定时器同步	145
	16.1.12	调试模式	147
	16.2	寄存器	147
	16.2.1	寄存器 (TIMx_CR1)	147
	16.2.2	寄存器 (TIMx_CR2)	148
	16.2.3	寄存器 (TIMx_SMCR)	150
	16.2.4	寄存器 (TIMx_DIER)	152
	16.2.5	寄存器 (TIMx_SR)	153
	16.2.6	寄存器 (TIMx_EGR)	154
	16.2.7	寄存器 (TIMx_CCMR1)	155
	16.2.8	寄存器 (TIMx_CCMR2)	158
	16.2.9	寄存器 (TIMx_CCER)	158
	16.2.10	寄存器 (TIMx_CNT)	160
	16.2.11	寄存器 (TIMx_PSC)	160
	16.2.12	寄存器 (TIMx_ARR)	160
	16.2.13	寄存器 (TIMx_RCR)	160
	16.2.14	寄存器 (TIMx_CCR1)	161
	16.2.15	寄存器 (TIMx_CCR2)	161
	16.2.16	寄存器 (TIMx_CCR3)	162
	16.2.17	寄存器 (TIMx_CCR4)	162



	16.2.18	寄存器 (TIMx_BDTR4)	163
	16.2.19	寄存器 (TIMx_DCR)	164
	16.2.20	寄存器 (TIMx_DMAR)	165
17	低功耗定时器 (LPTIM)		166
17.1	低功耗定时器(LPTIM).....		166
	17.1.1	LPTIM 描述	166
	17.1.2	LPTIM 互连	168
17.2	LPTIM 寄存器列表		168
17.3	LPTIM 寄存器说明		168
	17.3.1	LPTIM 计数值只读寄存器 (LPTIM_CNTVAL)	168
	17.3.2	LPTIM 控制寄存器 (LPTIM_CR)	168
	17.3.3	LPTIM 立即重载寄存器 (LPTIM_LOAD)	169
	17.3.4	LPTIM 中断寄存器 (LPTIM_INTSR)	170
	17.3.5	LPTIM 中断寄存器 (LPTIM_INTCLR)	170
	17.3.6	LPTIM 周期重载寄存器 (LPTIM_BGLOAD)	170
18	蜂鸣器(BEEP)		172
18.1	蜂鸣器(BEEP)		172
	18.1.1	简介	172
	18.1.2	功能描述	172
18.2	寄存器列表		172
18.3	寄存器说明		173
	18.3.1	蜂鸣器控制/状态寄存器 (BEEP_CSR)	173
19	自唤醒定时器(AWK)		174
19.1	自唤醒定时器(AWK)		174
19.2	寄存器列表		174
	19.2.1	寄存器说明	174
	19.2.2	自唤醒定时器控制寄存器 (AWK_CR)	174
	19.2.3	自唤醒定时器重载数据寄存器 (AWK_RLOAD)	175
	19.2.4	自唤醒定时器状态寄存器 (AWK_SR)	175
	19.2.5	自唤醒中断清除寄存器 (AWK_INTCLR)	175
20	实时时钟 (RTC)		177
20.1	实时时钟 (RTC)		177
	20.1.1	简介	177
	20.1.2	主要特性	177
	20.1.3	RTC 功能描述	177
20.2	RTC 寄存器列表		181
20.3	RTC 寄存器说明		182
	20.3.1	RTC 控制寄存器 (RTC_CR)	182
	20.3.2	RTC 时间寄存器 (RTC_TIME)	183
	20.3.3	RTC 日期寄存器 (RTC_DATE)	184
	20.3.4	RTC 时间闹钟寄存器 (RTC_ALM1TIME)	184
	20.3.5	RTC 日期闹钟寄存器 (RTC_ALM1DATE)	185
	20.3.6	RTC 周期闹钟寄存器 (RTC_ALM2PRD)	186
	20.3.7	RTC 时钟调校寄存器 (RTC_CLKCALR)	186
	20.3.8	RTC 初始化和状态寄存器 (RTC_ISR)	187
	20.3.9	TC 状态清除寄存器 (RTC_INTCLR)	188
	20.3.10	RTC 写保护寄存器 (RTC_WPR)	188
	20.3.11	RTC 时间戳时间寄存器 (RTC_TSTR)	188
	20.3.12	RTC 时间戳日期寄存器 (RTC_TSDR)	189
	20.3.13	RTC 侵入和复用功能配置寄存器 (RTC_TAFCR)	189
	20.3.14	RTC 备份寄存器 0 (RTC_BKPxR) (x=0~4)	190
第四部分	模拟接口		192
21	模拟数字转换 (ADC)		192
21.1	模拟数字转换 (ADC)		192
	21.1.1	介绍	192
	21.1.2	主要特征	192
	21.1.3	引脚描述	194
	21.1.4	通道选择	194
	21.1.5	外部启动条件选择	194
	21.1.6	单序列中断产生举例	195
	21.1.7	双序列中断产生举例	196
21.2	ADC 寄存器描述		197
	21.2.1	ADC 中断状态寄存器 (ADC_INTR)	197



	21.2.2	ADC 控制寄存器 (ADC_CR)	198
	21.2.3	ADC 数据寄存器 (ADC_DR)	198
	21.2.4	ADC 状态寄存器 (ADC_SR)	200
	21.2.5	ADC 序列 1 控制寄存器 1 (ADC_SEQ1CR1)	200
	21.2.6	ADC 序列 1 控制寄存器 2 (ADC_SEQ1CR2)	200
	21.2.7	ADC 序列 1 控制寄存器 3 (ADC_SEQ1CR3)	201
	21.2.8	ADC 序列 1 控制寄存器 4 (ADC_SEQ1CR4)	201
	21.2.9	ADC 序列 1 状态寄存器 1/./18 (ADC_SEQ1SRx) (x=1/./18).....	201
	21.2.10	ADC 序列 2 控制寄存器 1 (ADC_SEQ2CR1)	202
	21.2.11	ADC 序列 2 控制寄存器 2 (ADC_SEQ2CR2)	202
	21.2.12	ADC 序列 2 状态寄存器 1/2/3/4 (ADC_SEQ2SRx) (x=1/2/3/4).....	202
	21.2.13	ADC 累计寄存器 (ADC_ACCR)	203
	21.2.14	ADC 看门狗控制寄存器 (ADC_AWDCR)	203
22		电压比较器(VC).....	204
	22.1	电压比较器(VC).....	204
	22.1.1	VC 简介.....	204
	22.1.2	VC 框图.....	204
	22.1.3	数字滤波	204
	22.2	VC 寄存器列表	205
	22.3	VC 寄存器说明	205
	22.3.1	VC 控制寄存器 (VCCR)	205
	22.3.2	VC 状态寄存器 (VCSR)	206
	22.3.3	VC0 控制寄存器 (VC0CR)	206
	22.3.4	VC1 控制寄存器 (VC1CR)	207
23		低电压检测器(LVD).....	209
	23.1	低电压检测器(LVD).....	209
	23.1.1	LVD 简介.....	209
	23.1.2	LVD 框图.....	209
	23.1.3	数字滤波	209
	23.1.4	配置示例	210
	23.2	寄存器列表	210
	23.3	寄存器说明	210
	23.3.1	LVD 控制寄存器 (LVD_CR)	210
	23.3.2	LVD 状态寄存器 (LVD_SR)	212
第五部分		通信接口	213
24		通用同步异步收发器 (USART)	213
	24.1	功能描述.....	213
	24.2	USART 特性描述.....	215
	24.2.1	发送器.....	215
	24.2.2	字符发送	215
	24.2.3	可配置的停止位.....	215
	24.2.4	单字节通讯.....	216
	24.2.5	断开符号	216
	24.2.6	空闲符号	216
	24.2.7	接收器.....	216
	24.2.8	起始位检测.....	216
	24.2.9	字符接收	216
	24.2.10	断开符号	217
	24.2.11	空闲符号	217
	24.2.12	溢出错误	217
	24.2.13	噪音错误	218
	24.2.14	帧错误.....	218
	24.2.15	接收期间的可配置的停止位	218
	24.2.16	分数波特率的产生	219
	24.2.17	如何从 USART_BRR 寄存器值得到 USARTDIV	219
	24.2.18	USART 接收器容忍时钟的变化.....	219
	24.2.19	多处理器通信	220
	24.2.20	线检测 空闲总线检测(WAKE=0)	220
	24.2.21	地址标记(address mark) 检测(WAKE=1)	221
	24.2.22	校验控制	221
	24.2.23	LIN(局域网).....	221
	24.2.24	LIN 发送.....	221
	24.2.25	LIN 接收.....	221
	24.2.26	USART 同步模式.....	224

	24.2.27	单线半双工通信.....	226
	24.2.28	智能卡.....	227
	24.2.29	IrDA SIR ENDEC 功能模块.....	228
	24.2.30	IrDA 低功耗模式.....	229
	24.2.31	利用 DMA 连续通信.....	230
	24.2.32	利用 DMA 发送.....	230
	24.2.33	利用 DMA 接收.....	231
	24.2.34	多缓冲器通信中的错误标志和中断产生.....	232
	24.2.35	硬件流控制.....	232
	24.2.36	RTS 流控制.....	233
	24.2.37	CTS 流控制.....	233
24.3		USART 中断请求.....	234
24.4		USART 模式配置.....	235
24.5		串口接收发送配置示例.....	235
	24.5.1	串口 1 配置示例.....	235
	24.5.2	串口 2 配置示例.....	236
	24.5.3	串口 3 配置示例.....	237
	24.5.4	串口 4 配置示例.....	239
	24.5.5	串口 5 配置示例.....	239
24.6		USART 寄存器.....	240
	24.6.1	寄存器 (USART_SR).....	240
	24.6.2	寄存器 (USART_DR).....	243
	24.6.3	寄存器 (USART_BRR).....	243
	24.6.4	寄存器 (USART_CR1).....	244
	24.6.5	寄存器 (USART_CR2).....	245
	24.6.6	寄存器 (USART_CR3).....	247
	24.6.7	寄存器 (USART_GTPR).....	248
25		内部集成电路总线接口 (I2C).....	250
25.1		I2C 接口.....	250
	25.1.1	功能描述.....	250
	25.1.2	主要特点.....	250
	25.1.3	概述.....	250
	25.1.4	功能描述.....	251
25.2		寄存器.....	254
	25.2.1	寄存器 (I2C_CON).....	254
	25.2.2	寄存器 (I2C_TAR).....	256
	25.2.3	寄存器 (I2C_SAR).....	257
	25.2.4	寄存器 (I2C_DATA_CMD).....	257
	25.2.5	寄存器 (I2C_SS_SCL_HCNT).....	259
	25.2.6	寄存器 (I2C_SS_SCL_LCNT).....	259
	25.2.7	寄存器 (I2C_FS_SCL_HCNT).....	259
	25.2.8	寄存器 (I2C_FS_SCL_LCNT).....	260
	25.2.9	寄存器 (I2C_INTR_STAT).....	260
	25.2.10	寄存器 (I2C_INTR_MASK).....	261
	25.2.11	寄存器 (I2C_RAW_INTR_STAT).....	262
	25.2.12	寄存器 (I2C_RX_TL).....	263
	25.2.13	寄存器 (I2C_TX_TL).....	264
	25.2.14	寄存器 (I2C_CLR_INTR).....	264
	25.2.15	寄存器 (I2C_CLR_RX_UNDER).....	264
	25.2.16	寄存器 (I2C_CLR_RX_OVER).....	265
	25.2.17	寄存器 (I2C_CLR_TX_OVER).....	265
	25.2.18	寄存器 (I2C_CLR_RD_REQ).....	265
	25.2.19	寄存器 (I2C_CLR_TX_ABRT).....	265
	25.2.20	寄存器 (I2C_CLR_RX_DONE).....	266
	25.2.21	寄存器 (I2C_CLR_ACTIVITY).....	266
	25.2.22	寄存器 (I2C_CLR_STOP_DET).....	266
	25.2.23	寄存器 (I2C_CLR_START_DET).....	267
	25.2.24	寄存器 (I2C_CLR_GEN_CALL).....	267
	25.2.25	寄存器 (I2C_ENABLE).....	267
	25.2.26	寄存器 (I2C_STATUS).....	269
	25.2.27	寄存器 (I2C_TXFLR).....	271
	25.2.28	寄存器 (I2C_RXFLR).....	272
	25.2.29	寄存器 (I2C_SDA_HOLD).....	272
	25.2.30	寄存器 (I2C_TX_ABRT_SOURCE).....	272
	25.2.31	寄存器 (I2C_DMA_CR).....	274
	25.2.32	寄存器 (I2C_DMA_TDLR).....	274

	25.2.33	寄存器 (I2C_DMA_RDLR)	274
	25.2.34	寄存器 (I2C_SDA_SETUP)	275
	25.2.35	寄存器 (I2C_ACK_GENERAL_CALL)	275
	25.2.36	寄存器 (I2C_ENABLE_STATUS)	275
	25.2.37	寄存器 (I2C_FS_SPKLEN)	276
	25.2.38	寄存器 (I2C_SCL_STUCK_AT_LOW_TIMEOUT)	276
	25.2.39	寄存器 (I2C_SDA_STUCK_AT_LOW_TIMEOUT)	277
	25.2.40	寄存器 (I2C_CLR_SCL_STUCK_DET)	278
	25.2.41	寄存器 (I2C_SMBUS_CLK_LOW_SEXT_TIMEOUT)	278
	25.2.42	寄存器 (I2C_SMBUS_CLK_LOW_MEXT_TIMEOUT)	278
	25.2.43	寄存器 (I2C_SMBUS_THIGH_MAX_IDLE_COUNT)	279
	25.2.44	寄存器 (I2C_SMBUS_INTR_STAT)	279
	25.2.45	寄存器 (I2C_SMBUS_INTR_MASK)	280
	25.2.46	寄存器 (I2C_SMBUS_INTR_RAW_STATUS)	281
	25.2.47	寄存器 (I2C_CLR_SMBUS_INTR)	282
	25.2.48	寄存器 (I2C_SAR_OPT)	283
	25.2.49	寄存器 (I2C_SMBUS_UDID)	283
26		串行外设接口 (SPI)	284
	26.1	串行外设接口 (SPI)	284
	26.1.1	功能描述	284
	26.1.2	配置 SPI 从模式	286
	26.1.3	配置 SPI 主模式	286
	26.1.4	配置 SPI 半双工	287
	26.1.5	数据发送和接受流程	288
	26.1.6	CRC 校验	292
	26.1.7	状态标识	293
	26.1.8	禁用 SPI	293
	26.1.9	使用 DMA 的 SPI 通信	294
	26.1.10	错误标识	295
	26.1.11	SPI 中断	296
	26.2	SPI 寄存器	296
	26.2.1	SPI 控制寄存器 1 (SPI_CR1) (I2S 模式下不可用)	296
	26.2.2	SPI 控制寄存器 2 (SPI_CR2)	298
	26.2.3	SPI 状态寄存器 (SPI_SR)	299
	26.2.4	SPI 状态寄存器 (SPI_DR)	301
	26.2.5	SPI CRC 多项式寄存器 (SPI_CRCPR) (I2S 模式下不可用)	301
	26.2.6	SPI RX CRC 寄存器 (SPI_RXCRCR) (I2S 模式下不可用)	302
	26.2.7	SPI TX CRC 寄存器 (SPI_TXCRCR) (I2S 模式下不可用)	303
27		低功耗通用异步收发器(LPUART)	304
	27.1	低功耗通用异步收发器(LPUART)	304
	27.1.1	概述	304
	27.1.2	结构框图	304
	27.1.3	工作模式	304
	27.1.4	波特率编程	307
	27.1.5	帧错误检测	308
	27.1.6	多机通讯	308
	27.1.7	自动地址识别	308
	27.1.8	给定地址	308
	27.1.9	广播地址	308
	27.1.10	收发端缓存	309
	27.2	寄存器列表	309
	27.3	寄存器说明	309
	27.3.1	LPUART 数据寄存器 (LPUART_SBUF)	309
	27.3.2	LPUART 控制寄存器 (LPUART_SCON)	310
	27.3.3	LPUART 地址寄存器 (LPUART_SADDR)	311
	27.3.4	LPUART 地址掩码寄存器 (LPUART_SADEN)	311
	27.3.5	LPUART 标志位寄存器 (LPUART_INTSR)	312
	27.3.6	LPUART 标志位清除寄存器 (LPUART_INTCLR)	312
	27.3.7	LPUART 波特率控制寄存器 (LPUART_BAUDCR)	312
第六部分		模拟前端模块	314
28		模拟前端模块 (AFE)	314
	28.1	可编程运算放大器 (PGA)	315
	28.1.1	主要特性	315
	28.1.2	PGA 结构框图	315

	28.1.3	功能描述	316
	28.1.4	初始配置	316
	28.1.5	信号走线	317
	28.1.6	工作模式	317
28.2		模拟前端中的电压比较器模块(AFE_VC)	318
	28.2.1	简介	318
	28.2.2	VC 框图	318
	28.2.3	Hysteresis 设置与低通滤波设置	319
	28.2.4	VC 脚和内部信号	320
	28.2.5	VC 启动稳定时间	320
28.3		数模转换器 (DAC)	320
	28.3.1	主要特性	320
	28.3.2	DAC 框图	320
	28.3.3	功能描述	321
28.4		AFE 寄存器	321
	28.4.1	AFE 寄存器列表	321
	28.4.2	AFE 寄存器说明	321
第七部分		版本与目录索引	329
29		版本记录 (REVISION)	329
30		文档目录	330
31		表索引	339
32		图索引	340

31 表索引

表 1.1	地址分配	2
表 1.2	闪存地址	4
表 1.3	启动方式	4
表 3.1	Flash 组织结构	6
表 3.2	闪存保护状态	13
表 3.3	选项字节格式	14
表 3.4	选项字节结构	15
表 3.5	选项字节描述	15
表 5.1	低功耗模式	43
表 5.2	Sleep-now 模式	44
表 5.3	Sleep-on-exit 模式	44
表 5.4	停止模式	45
表 5.5	待机模式	46
表 6.1	外部中断/事件连接	53
表 8.1	SW-DP 引脚分配	65
表 11.1	独立看门狗定时时间 (LSI 40kHz)	93
表 12.1	36MHz 时钟下窗口看门狗的最小最大超时表	97
表 16.1	具有中断功能的互补 OCx 和 OCxN 通道的输出控制	135
表 16.2	计数方向和编码信号	141
表 16.3	ITR 连接	145
表 21.1	ADC 的引脚	194
表 24.1	USART 中断请求	234
表 24.2	USART 模式配置	235
表 28.1	运算放大器 (PGAx) 的引脚连接表	317
表 28.2	AFE 寄存器表	321
表 29.1	版本记录	329

32 图索引

图 1.1	系统顶层架构	2
图 3.1	主闪存区编程流程	9
图 3.2	主闪存页擦除	10
图 3.3	主存储区整片擦除	11
图 4.1	复位电路框图	22
图 4.2	时钟树	23
图 5.1	上电复位和掉电复位波形	42
图 6.1	中断与事件结构框图	53
图 9.1	IO 端口位的基本结构	68
图 11.1	独立看门狗框图	92
图 12.1	窗口看门狗时序图	96
图 15.1	AES 算法	106
图 15.2	加密流程:	108
图 15.3	解密流程:	108
图 16.1	计数器 1	118
图 16.2	计数器 3	118
图 16.3	计数器 6/7	120
图 16.4	预分频比从 1 变为 4 的计数器时序图	123
图 16.5	递增模式内部时钟除以 N	124
图 16.6	递减模式内部时钟除以 N	124
图 16.7	内部时钟除以 4, TIMx_ARR=0x36	125
图 16.8	使用 APE=1 更新时间 (计数器溢出)	125
图 16.9	更新速率示例取决于模式和 TIMx_RCR 寄存器设置	126
图 16.10	正常模式内部时钟源除以 1	127
图 16.11	外部时钟连	127
图 16.12	ETR 输入模块	128
图 16.13	捕获通道 (通道 1 输入级)	128
图 16.14	捕获通道 (通道 1~3 输出级)	129
图 16.15	捕获通道 (通道 4 输出级)	129
图 16.16	-PWM 输入模式	131
图 16.17	输出比较模式	132
图 16.18	PWM 边缘对齐模式	133
图 16.19	PWM 中央对齐模式	133
图 16.20	带死区插入的互补输出	134
图 16.21	延迟大于负脉冲的死区时间波形	134
图 16.22	响应刹车的输出行为	137
图 16.23	清除定时器 x OCxREF	138
图 16.24	步 PWM 生成	139
图 16.25	单脉冲模式	140
图 16.26	编码器接口模式	141
图 16.27	霍尔传感器接口模式	143
图 16.28	从模式下复位模式	144
图 16.29	从模式下门控模式	144
图 16.30	-从模式下触发模式	145
图 16.31	一个定时器作为另一个的预分频器	146
图 16.32	一个定时器使能另外一个	146
图 16.33	一个定时器启动另外一个	146
图 16.34	针对外部触发同时启动 2 个定时器	147
图 17.1	LPTIMER 结构框图	166
图 17.2	LPTIMER 模式 1	167



图 17.3	LPTIMER 模式 2.....	167
图 18.1	蜂鸣器功能图.....	172
图 19.1	自唤醒定时器结构图.....	174
图 20.1	RTC 框图.....	178
图 21.1	ADC 模块功能框图.....	192
图 22.1	VC 框图.....	204
图 22.2	VC 滤波输出.....	205
图 23.1	LVD 结构框图.....	209
图 23.2	LVD 滤波输出.....	209
图 24.1	空闲总线检测.....	220
图 24.2	地址标志检测唤醒.....	221
图 24.3	LIN 模式下的断开检测(11 位断开长度 - 设置了 LBDL 位).....	223
图 24.4	LIN 模式下的断开检测与帧错误的检测.....	224
图 24.5	USART 同步传输的例子.....	225
图 24.6	USART 数据时钟时序示例(M=0).....	225
图 24.7	USART 数据时钟时序示例(M=1).....	226
图 24.8	RX 数据采样/保持时间.....	226
图 24.9	ISO7816-3 异步协议.....	227
图 24.10	使用 1.5 停止位检测奇偶检验错.....	228
图 24.11	IrDA SIR ENDEC - 框图.....	230
图 24.12	IrDA 数据调制(3/16) - 普通模式.....	230
图 24.13	利用 DMA 发送.....	231
图 24.14	利用 DMA 接收.....	232
图 24.15	两个 USART 间的硬件流控制.....	232
图 24.16	RTS 流控制.....	233
图 24.17	CTS 流控制.....	233
图 24.18	USART 中断映像图.....	234
图 25.1	IIC 传输协议.....	251
图 25.2	错误中断.....	254
图 25.3	事件中断.....	254
图 26.1	SPI/ I2S 框图.....	284
图 26.2	单主/单从模式应用实例.....	285
图 26.3	SPI 时序图.....	285
图 26.4	连续传输主/全双工模式 (BIDIMODE/RXONLY = 0) 下的 TXE/RXNE/BSY 行为.....	289
图 26.5	连续传输从/全双工模式 (BIDIMODE/RXONLY = 0) 下的 TXE/RXNE/BSY 行为.....	290
图 26.6	连续传输主发送模式 (BIDIMODE/RXONLY=0) TXE/BSY 行为.....	290
图 26.7	连续传输从发送模式 (BIDIMODE/RXONLY=0) TXE/BSY 行为.....	291
图 26.8	连续传输仅接收模式 (BIDIROMODE=0 和 RXONLY=1) 下的 RXNE 行为.....	291
图 26.9	DMA 传输 1.....	295
图 26.10	DMA 传输 2.....	295
图 26.11	SPI 中断请求.....	296
图 27.1	LPUART 结构图.....	304
图 27.2	Mode 0 发送数据.....	305
图 27.3	Mode 0 接收数据.....	305
图 27.4	Mode 1 发送数据.....	306
图 27.5	Mode 1 接收数据.....	306
图 27.6	Mode 2 发送数据.....	306
图 27.7	Mode 2 接收数据.....	306
图 27.8	接收缓存.....	309
图 28.1	模拟前端模块 (AFE) 框图.....	314
图 28.2	PGA 结构框图.....	315
图 28.3	VC 框图.....	318
图 28.4	6 个 VC 单元示意图.....	319

图 28.5	VC 滤波输出	320
图 28.6	DAC 框图	320

重要声明 - 仔细阅读

上海朔集半导体科技有限公司(“朔集半导体”)保留随时对 朔集半导体 产品和/或本文件进行更改、更正、增强、修改和改进的权利，恕不另行通知。购买者在下单前应获取有关 朔集半导体 产品的最新相关信息。朔集半导体产品的销售依据订单确认时 朔集半导体 的销售条款和条件。

购买者全权负责 朔集半导体 产品的选择、挑选和使用，朔集半导体 对购买者产品的应用和产品设计不承担任何责任。

朔集半导体 在此未授予任何知识产权（无论是明示还是暗示）的许可。

以与本文件所述信息不同的条款转售 朔集半导体 产品，将使 朔集半导体 对该产品的任何担保失效。

朔集半导体 和 朔集半导体 徽标是 朔集半导体 的商标。有关 朔集半导体 的更多信息，请参阅 www.SoulSemi.com。所有其他产品或服务名称均为其各自所有者的财产。

本文件中的信息取代并替换之前任何版本文件中提供的信息。

© 2025 上海朔集半导体科技有限公司 - 保留所有权利